

DBLog Virtual Cuts: Certified Snapshot-Equivalent Replay of Live Databases

Andreas Andreakis

September 7, 2026

Abstract

DBLog reconstructs a logical, replay-equivalent snapshot of a live database scope without taking a physical database snapshot, by interleaving chunked reads with change-data-capture (CDC) events. This entry machine-checks the theory behind that protocol, following the companion paper [2]: it formalizes *certified virtual cuts* — finite, asynchronously gathered mixes of CDC events and chunk reads whose certified replay reaches the source state at a chosen frontier on a chosen key scope — and proves the layered theorem ladder relating run well-formedness, verifier acceptance, and replay equality, together with a source-side continuation and restriction algebra and a whole-table specialization.

The development is axiom-free: it contains no *axiomatization*, *type-decl*, or *consts* declaration, and its only *typedef* is a conservative copy of the naturals for source coordinates. All witnesses and fixtures are constructed models with locale interpretations, and a closing non-vacuity theory exercises every main theorem at concrete witnesses, the certificate-layer results at genuinely accepted certificates. Every theorem is conditional on the premises stated in its statement; the machine check does not discharge the deployment obligations or the external-observation assumption that a real deployment must establish.

Contents

Entry overview	9
1 Carrier-independent Layer 0/1 core: chunk-read records and the canonical clean prefix	11
1.1 Chunk-read record (Layer 0 self-contained)	12
1.2 Canonical clean prefix (Layer 0 building block)	12
1.3 Normalized clean prefix: Apply-equivalence to the raw form	15
1.4 Source key/coordinate slice helper	19

2	Carrier-independent Layer 2 supporting helpers	19
3	Certificate-free virtual-cut core (carrier-independent)	34
3.1	Virtual cut state (shared Layer-0 predicate)	35
3.2	Anchor and table-scope definitions	35
3.3	Replay-side no-unclaimed-key condition	35
3.4	Outside-scope bridge lemmas	36
4	Layer 1: the DBLog run carrier, accessors, and wellformed-	38
	ness	
4.1	Run carrier (concrete datatype)	40
4.2	Run accessors (7 accessors)	40
4.3	Chunks enumeration accessor	41
4.4	Chunk-plan accessors	42
4.5	Chunk watermarks and read coordinates	43
4.6	Layer 1 clean-prefix definitional equality (<i>clean-prefix-of</i>) .	43
4.7	Coverage predicates	44
4.8	The wellformed-DBLog-run predicate	46
5	WF7 clean-prefix CDC coherence holds for every run	50
6	Further redundant conjuncts of the wellformed-run body	53
7	Layer 1 run substrate as a locale (abstract carrier)	56
7.1	Canonical concrete interpretation (at the public <i>run</i> datatype)	61
8	Layer 2 source-side soundness in the run substrate locale	62
8.1	Refresh-coordinate match and clean-prefix coordinate facts	62
8.2	Provenance of clean-prefix events: responsible-chunk refreshes and observed CDC	64
8.3	Clean-prefix CDC order respects source position	68
8.4	Latest per-key replay-event correspondence	73
8.5	Per-key source/CDC coordinate-slice equalities	76
8.6	Clean-prefix structure at a fixed coordinate	79
8.7	Consequences of a latest Cdc event for a key	83
8.8	Per-key replay equality: the Cdc case	89
8.9	Per-key replay equality: main lemma and the Refresh case	92
8.10	Layer 2 run-side soundness (locale form of the paper's bridge lemma)	97
9	Virtual cuts, certificates, and the Layer 2/3 theorems	98
9.1	Certificate carrier and accessors	98
9.2	Virtual cut state (shared Layer-0 predicate)	99
9.3	Virtual cut (cert-side instance of virtual cut state)	99

9.4	Layer 2 source-side soundness (over runs)	100
9.5	Layer 3 checker substrate: cert/evidence bridge and soundness	101
9.6	Mechanical / external factoring of run wellformedness . . .	103
9.7	The public checker primitives	105
9.8	The four checker obligations hold globally for the public checker	107
9.9	The locale holds for every public certificate / evidence pair	109
10	Minimum-viable wellformed-run witness data (carrier- independent)	110
10.1	Witness base state and source history	110
10.2	Helper computation lemmas for the WF discharge	110
10.3	Wellformedness of the witness source history	111
11	Topic 1 and Topic 2 witness data (carrier-independent)	112
12	Topic 1 witness data	112
12.1	Topic 1 base state and source history	112
12.2	Source-state computations on the Topic 1 instance	113
12.3	Wellformedness of the Topic 1 source history	113
13	Topic 2 witness data	113
13.1	Topic 2 base state and source history	113
13.2	Source-state computations on the Topic 2 instance	114
13.3	Wellformedness of the Topic 2 source history	114
14	Running-example data (carrier-independent)	114
14.1	Witness source-coordinate ordering facts	115
14.2	Witness base state and source history	116
14.3	Source-state computations on the example	116
14.4	Wellformedness of the example source history	119
15	Layer 3/4 witness anchor data (carrier-independent)	121
15.1	Anchor boundary	122
15.2	Anchor and post-anchor source facts (independent of certificates)	122
16	Carrier-independent Layer 0/1 fixture data	124
16.1	WF1 (b): overlapping chunk domains (<i>fix-overlap</i>)	125
16.2	WF1 (c): chunk owns out-of-scope key (<i>fix-miss-cov</i>)	125
16.3	WF1 (f): infinite chunk domain (<i>fix-inf-dom</i>)	125
16.4	WF5: chunk read after frontier (<i>fix-after-f</i>)	125
16.5	WF6 main: wrong refresh value (<i>fix-wrong-r</i>)	125

16.6	WF6 specialized: same-coord read/event ambiguity (<i>fix-same-c</i>)	126
16.7	WF2 faithfulness: extra observed CDC (<i>fix-extra-cdc</i>) . . .	126
16.8	WF2 coverage: missing dominator (<i>fix-miss-dom</i>)	126
16.9	WF2 multiplicity: duplicate multiset gap (<i>fix-dup-mset</i>) . .	127
16.10	WF2 order: duplicate order gap (<i>fix-dup-order</i>)	127
16.11	Boundary: empty scope (<i>boundary-empty-scope</i>)	127
16.12	WF1 (d): responsible-chunk vs chunk-domain mis- match (<i>fix-resp-dom</i>)	127
16.13	WF6 row-absence: absence-Refresh without empty source (<i>fix-row-abs</i>)	127
17	Layer 0 source-history fixture	128
18	Carrier-independent Layer 3 fixture data	129
19	Parametric certificate-coherence facts	129
20	Carrier-independent Layer 4 fixture data	130
21	Further anchor-domain facts	140
22	Layer 2 canonical-clean-prefix and frontier fixtures	140
22.1	Canonical-clean-prefix structural fixtures	140
22.2	Frontier off-by-one subcases	141
23	Constructed model for the running-example run configura- tion	142
23.1	Concrete carriers and accessor functions	143
23.2	The model realizes the example's accessor values	144
24	Constructed interpretation for the running-example virtual cut	145
24.1	Constructed interpretation at the running-example model .	145
24.2	Model accessor-value lemmas	146
24.3	Determinate clean prefix and Apply over the model	147
24.4	Constructed wellformed-run fact	148
24.5	Constructed virtual-cut replay equality	158
25	Layer 4: the whole-table specialization	159
25.1	Whole-table claim scope (cert-side)	160
25.2	Outside-scope bridge to whole-table equality (cert-side) . .	160
25.3	Layer 4 theorem surface over accepted certificates	161

26	Source-side continuation and restriction of virtual cuts	161
26.1	The continuation segment	162
26.2	Layer 0 — replay-append and source-state interval lemmas	166
26.3	Continuation across frontiers	168
26.4	Restriction to a sub-scope	171
26.5	Whole-table continuation	172
26.6	Sub-scope restriction over a certificate	172
26.7	Continuation over an accepted certificate	173
27	Carrier-independent continuation fixture data	174
28	Positive-witness corollaries and the out-of-scope padding boundary	184
29	Constructed model for the minimum-viable wellformed-run witness	185
30	Constructed interpretation for the minimum-viable wellformed-run witness	187
30.1	Constructed interpretation at the witness model	188
30.2	Model accessor-value lemmas	188
30.3	Determinate clean prefix and Apply over the model	189
30.4	Constructed wellformed-run fact	190
30.5	Constructed positive witness	198
31	Constructed models for the Topic 1 and Topic 2 witnesses	200
31.1	Topic 1: single chunk, one CDC event	200
31.2	Topic 2: two chunks partitioning the scope, empty history .	201
32	Constructed interpretations for the Topic 1 and Topic 2 witnesses	203
33	Topic 1: constructed interpretation	204
33.1	Topic 1 model accessor-value lemmas	204
33.2	Topic 1 determinate clean prefix and Apply over the model	205
33.3	Topic 1 constructed wellformed-run fact	205
33.4	Topic 1 constructed replay-equality conclusion (non-trivial)	210
33.5	Topic 1 constructed positive witness	211
34	Topic 2: constructed interpretation	211
34.1	Topic 2 model accessor-value lemmas	212
34.2	Topic 2 determinate clean prefix over the model	213
34.3	Topic 2 constructed wellformed-run fact	213
34.4	Topic 2 constructed positive witness	219

35	Concrete run models for the Layer 0/1 fixtures	220
35.1	A reusable concrete run model	220
35.2	WF1 (b): overlapping chunk domains (<i>fix-overlap</i>)	221
35.3	WF1 (c): chunk owns out-of-scope key (<i>fix-miss-cov</i>)	221
35.4	WF1 (f): infinite chunk domain (<i>fix-inf-dom</i>)	222
35.5	WF5: chunk read after frontier (<i>fix-after-f</i>)	222
35.6	WF6 main: wrong refresh value (<i>fix-wrong-r</i>)	222
35.7	WF6 specialized: same-coord read/event ambiguity (<i>fix-same-c</i>)	223
35.8	WF2 faithfulness: extra observed CDC (<i>fix-extra-cdc</i>)	223
35.9	WF2 coverage: missing dominator (<i>fix-miss-dom</i>)	223
35.10	WF2 multiplicity: duplicate multiset gap (<i>fix-dup-mset</i>)	224
35.11	WF2 order: duplicate order gap (<i>fix-dup-order</i>)	224
35.12	WF1 (d): responsible-chunk vs chunk-domain mis- match (<i>fix-resp-dom</i>)	224
35.13	WF6 row-absence: absence-Refresh without empty source (<i>fix-row-abs</i>)	225
35.14	Boundary: empty scope, vacuously wellformed (<i>bound- ary-empty-scope</i>)	225
36	Layer 3/4 certificate substrate as a locale hierarchy (abstract carriers)	226
36.1	Certificate substrate locale	227
36.2	Layer 3 checker substrate locale	230
37	Certificate substrate: continuation corollaries and a canon- ical concrete interpretation	232
37.1	Continuation corollaries over the abstract cert carriers	232
37.2	Canonical certificate-substrate interpretation	233
38	Constructed interpretation for the Layer 3 accepted- certificate and Layer 4 whole-table witnesses	234
38.1	One-point certificate / evidence / observation accessors	235
38.2	Constructed accepting interpretation of the checker substrate locale	236
38.3	Constructed Layer 3 certificate soundness	236
38.4	Constructed Layer 4 whole-table witness	237
38.5	Constructed positive witness (non-vacuity)	238
39	Constructed interpretations for the Layer 0/1 fixtures	239
39.1	Constructed interpretation at the fixture model carrier	239
39.2	WF1 (b): overlapping chunk domains (<i>fix-overlap</i>)	240
39.3	WF1 (c): chunk owns out-of-scope key (<i>fix-miss-cov</i>)	240
39.4	WF1 (f): infinite chunk domain (<i>fix-inf-dom</i>)	241

39.5	WF5: chunk read after frontier (<i>fix-after-f</i>)	241
39.6	WF6 main: wrong refresh value (<i>fix-wrong-r</i>)	241
39.7	WF6 specialized: same-coord read/event ambiguity (<i>fix-same-c</i>)	242
39.8	WF2 faithfulness: extra observed CDC (<i>fix-extra-cdc</i>) . . .	242
39.9	WF2 coverage: missing dominator (<i>fix-miss-dom</i>)	243
39.10	WF2 multiplicity: duplicate multiset gap (<i>fix-dup-mset</i>) . .	243
39.11	WF2 order: duplicate order gap (<i>fix-dup-order</i>)	245
39.12	WF1 (d): responsible-chunk vs chunk-domain mis- match (<i>fix-resp-dom</i>)	246
39.13	WF6 row-absence: absence-Refresh without empty source (<i>fix-row-abs</i>)	246
39.14	Boundary: empty scope, vacuously wellformed (<i>bound- ary-empty-scope</i>)	247
39.15	Constructed existential bundles	250
40	Constructed Layer 2 fixtures	251
40.1	Positive Layer 2 witness lifts over the constructed runs . .	251
40.2	WF-clause rejection rows over the constructed Layer 1 fixtures	252
40.3	Fixture bundle	253
41	Constructed Layer 3 fixtures	254
41.1	Constructed certificate / evidence / observation carriers . .	254
41.2	Constructed certificate-side accessor and predicate functions	255
41.3	Constructed certificate substrate interpretation (reject- ing fixtures)	256
41.4	Certificate/evidence binding failures	257
41.5	Materialization and non-circular substrate	257
41.6	Coherence and determinacy	258
41.7	Faithfulness negatives	258
41.8	Empty-scope boundary	259
41.9	Unknown or ignored evidence fields	259
41.10	Fixture bundle	259
42	D15 regression: the accessor fixture reaches comparison	260
43	Constructed Layer 4 fixtures	261
43.1	Constructed certificate carrier	261
43.2	Reusable helpers	262
43.3	Constructed certificate substrate interpretation (cert- accessor fixtures)	263
43.4	Post-anchor insert not covered (constructed)	263

43.5	Post-anchor delete as conservative touch coverage (constructed)	265
43.6	Empty anchor-domain boundary (constructed)	267
43.7	Claim scope broader than materialized replay coverage (constructed)	268
43.8	Anchor boundary after certified frontier (constructed) . . .	270
43.9	Post-anchor update materialized by a later refresh (constructed)	272
43.10	No-unclaimed replay-key failure (constructed)	274
43.11	b0 base-state contribution to the anchor domain (constructed)	276
43.12	Same-coordinate anchor and frontier boundaries (constructed)	278
43.13	Materialized-run ambiguity is not Layer 4 evidence (constructed)	282
43.14	Constructed Layer 4 fixture bundle	285
44	Constructed continuation fixtures	286
44.1	Constructed extended certificate	286
44.2	Accessor equality of an extended certificate is not verifier acceptance (constructed)	287
44.3	Constructed continuation negative / boundary fixture bundle	288
45	Public accepting witness for the Layer 3 checker (non-vacuity gate)	289
45.1	The running-example run over the concrete run datatype .	290
45.2	Accessor values	290
45.3	Determinate clean prefix and Apply	291
45.4	The running-example run is wellformed (global carrier) . .	292
45.5	The accepting public certificate / evidence pair	296
45.6	Exercising the headline theorems at the public witness . . .	297
45.7	Exercising the remaining state-level headlines: Layer 2 and sub-scope restriction	298
45.8	A mid-frontier accepted certificate and non-empty continuation segments (the active-interval continuation case) .	300
45.9	The non-vacuity gate theorem	309
45.10	Independence exhibits (formal non-circularity)	310

Entry overview

What is proved

The load-bearing spine is the Layer 2 result: the canonical clean prefix of a wellformed DBLog run, replayed from the empty state, reproduces the source state at the run’s frontier restricted to the run’s scope (*wellformed-run-implies-virtual-cut*); the base state enters through the chunk reads, not the replay seed. On top of it sit an interface layer and an algebra:

- *Layer 3 (certificates)*. Inside the *layer3-checker-substrate* locale — which abstracts the verifier as a set of soundness obligations — an accepted certificate under faithful source observation replays to the source state at the certified frontier on the certified scope. The obligations are proved for the concrete verifier shipped with the entry, so the locale hypotheses are satisfiable.
- *Layer 4 (whole table)*. The specialization of the Layer 3 result to whole-table claim scopes: the replay reproduces the entire source state.
- *Continuation algebra*. Source-side continuation across frontiers (appending the faithful CDC segment for a half-open frontier interval), sub-scope restriction, the whole-table continuation instance, and an accepted-certificate continuation result.

By name, the nine main theorems are *wellformed-run-implies-virtual-cut* (Layer 2); *accepted-certificate-implies-wellformed-run* and *accepted-virtual-cut-sound* (Layer 3, in the checker locale); *accepted-whole-table-anchor-domain-specialization* (Layer 4, in the checker locale); the four state-level continuation/restriction results *virtual-cut-state-continuation*, *virtual-cut-state-restrict-scope*, *whole-table-state-continuation*, and *virtual-cut-restrict-to-subscope*; and *accepted-certificate-continuation-sound* (both in the theory *Continuation*). The closing theory fires each of the nine at concrete witnesses — the certificate-layer results at genuinely accepted certificates.

Composition is proved; single-coordinate chunk-read attribution and CDC faithfulness are explicit premises, and the operational watermark is not a formal object the theorems consult. The deployment obligations and the external-observation assumption of the paper [2] appear here as explicit theorem premises and locale obligations; the entry makes them visible rather than discharging them. The protocol itself is from the original DBLog system paper [3]. This document corresponds to version 2.1 of the development, archived on Zenodo [1]; its frozen predecessors remain separately archived (v2.0 at DOI 10.5281/zenodo.20652511, v1.0 at DOI 10.5281/zenodo.20389697). The development is distributed under the BSD 3-Clause license.

Trust base

The session and its shared *Dual-Write-Layer0* parent contain no *axiomatization*, *typeddecl*, or *consts* declaration. The development’s single *typedef* — in the parent theory *Source-History* — introduces the source-coordinate type as a copy of *nat* (a conservative extension over a provably nonempty set), with its *linorder* and *order-bot* instances proved. Witness and fixture facts are established constructively: concrete models are exhibited as closed-form data and the substrate locales are interpreted at them. The final theory, *Public-Checker-Witness*, exhibits concrete certificate/evidence pairs the verifier genuinely accepts and fires all nine main theorems at them, so none of the headline results is vacuously true.

Modelling notes

The key type $'k$ carries a *linorder* hypothesis in the run- and certificate-layer theorems: it enters through the canonical clean-prefix construction, which enumerates chunk-read domains with *sorted-list-of-set* and interleaves emitted events by source coordinate with a stable *sort-key*, making the construction deterministic. The four state-level continuation/restriction results are constraint-free. Database primary keys are totally ordered in practice, so the hypothesis does not narrow the intended interpretation.

Source coordinates abstract positions in the source’s totally ordered change history (in deployments: a commit timestamp, log sequence number, or binlog position). Chunk identifiers are plain naturals; the run and certificate carriers are single-constructor datatypes with named selectors.

Layer-to-theory map

Layer	Theories
0 (source/replay substrate)	<i>Source-History</i> , <i>Replay</i> , <i>Scope</i> , <i>Source-Coordinates</i> , <i>Virtual-Cut-State</i> — the shared <i>Dual-Write-Layer0</i> parent session
0/1 (carrier-independent run core)	<i>DBLog-Run-Core</i>
2–4 (carrier-independent cut core)	<i>Virtual-Cut-Core</i>
1 (runs and wellformedness)	<i>DBLog-Run</i> , <i>DBLog-Run-Substrate</i>
2 (run-side soundness)	<i>DBLog-Run-Substrate-Layer2</i>
2–3 (virtual cuts, certificates)	<i>Virtual-Cut</i>
3–4 (certificate substrate locales)	<i>DBLog-Cert-Substrate(-Inst)</i>
4 (whole table)	<i>Layer4-Whole-Table</i>
post-core extension	<i>Continuation</i>
witnesses/fixtures	the <i>-Core/-Model/-Inst</i> families and the <i>Layer3-Defect-Regressions</i> regression theory
non-vacuity gate	<i>Public-Checker-Witness</i>

The witness/fixture families follow a uniform division of labor: *-Core* theories hold carrier-independent data (named coordinates, base states, source histories, expected values), *-Model* theories exhibit small concrete carrier realizations, and *-Inst* theories interpret the substrate locales at those models and prove the witness, boundary, and rejection facts.

Naming conventions

Convention	Meaning
<i>-of</i> -suffixed accessors	run accessors (<i>scope-of</i> , <i>frontier-of</i> , <i>clean-prefix-of</i> , ...)
bare-named accessors	certificate accessors (<i>scope</i> , <i>frontier</i> , <i>clean-prefix</i>)
<i>rr-</i> / <i>crr-</i> prefixes	run-datatype / chunk-read-record field selectors
<i>wit-</i> , <i>ex-</i> , <i>t1-/t2-</i> prefixes	witness-family constants (also as suffixes: <i>b0-w</i> , <i>H-ex</i>)
<i>fix-</i> prefix	negative/boundary fixture facts
<i>layer3-</i> / <i>l4-</i> prefixes	Layer-3 / Layer-4 fixture data and bundle facts
<i>cf-</i> prefix	certificate-substrate fixture-interpretation constants and facts
WF0–WF7	the wellformedness clauses (WF3 is a deliberate gap)

```

theory DBLog-Run-Core
imports
  Dual-Write-Layer0.Source-History
  Dual-Write-Layer0.Replay
  Dual-Write-Layer0.Scope
  HOL-Library.Multiset
  HOL-Library.Sublist
begin

```

1 Carrier-independent Layer 0/1 core: chunk-read records and the canonical clean prefix

Layer 0 / Layer 1 carrier-independent core.

This theory collects the “pure” building blocks of the Layer 1 run substrate that reference no run/chunk carrier: the Layer 0 self-contained *chunk-read-record*, the canonical-clean-prefix construction (*canonical-clean-prefix* and its normalized companion *canonical-clean-prefix-normalized*) together with the normalization lemmas culminating in *apply-canonical-clean-prefix-normalized-eq*, and the *source-key-coord-slice* per-key / per-coordinate filtering helper.

These building blocks are factored out here, below the run carrier, so that *both* the global theory *DBLog-Run* (which defines the concrete run carrier, its accessors, and the *chunks-list* lemmas) *and* the *dblog-run-substrate* locale rest on the *same* carrier-independent definitions.

1.1 Chunk-read record (Layer 0 self-contained)

$(\text{'}k, \text{'}v)$ *chunk-read-record* is the Layer 0 self-contained record carrying the per-chunk evidence the canonical-clean-prefix construction's chunk-read step consumes:

- *crr-domain* :: $\text{'}k$ set — the chunk's claimed key set (paper *domain* field; Isabelle selector prefixed to avoid shadowing standard library names);
- *crr-read-coord* :: *src-coord* — the source coordinate at which the chunk read executed (paper *read-coord* field);
- *crr-observation* :: $\text{'}k \Rightarrow \text{'}v$ option option — the per-key observation, where the outer option distinguishes "key outside *domain*" from "observation recorded for an in-domain key" and the inner option distinguishes "value found" (*Some v*) from "no row found" (*None*) (paper *observation* field).

Paper "Auxiliary Layer 0 definitions" / "Canonical clean prefix" / "ChunkReadRecord".

```
record ( $\text{'}k, \text{'}v$ ) chunk-read-record =
  crr-domain      ::  $\text{'}k$  set
  crr-read-coord  :: src-coord
  crr-observation ::  $\text{'}k \Rightarrow \text{'}v$  option option
```

1.2 Canonical clean prefix (Layer 0 building block)

Paper "Auxiliary Layer 0 definitions" / "Canonical clean prefix". The Layer 0 construction the Layer 1 *clean-prefix-of* accessor is defined in terms of. Its inputs are typed — chunk-read records, observed-CDC pairs, scope, frontier — and the paper states finiteness premises on them.

The construction is a real Isabelle **definition** mirroring the paper's 4-step construction:

1. Step 1 (chunk-read Refresh emission). For each *chunk-read* in *chunk-reads* and each k in *crr-domain chunk-read* with *crr-observation chunk-read k = Some m*, emit *Refresh k m (crr-read-coord chunk-read)*. The *crr-domain* set enumeration uses *sorted-list-of-set* (requires *linorder* on the $\text{'}k$ type variable; surfaces the implicit deterministic-enumeration assumption the paper makes when it speaks of "for each k in domain").
2. Step 2 (Cdc emission, scope+frontier filter). For each (c, e) in *cdc-events* with *key-of e ∈ scope* and $c \leq \text{frontier}$, emit *Cdc c e*.
3. Step 3 (interleave by source coordinate). Stable *sort-key* by *cdc-coord*: at any coord c , the Cdc's (input order: *cdc-events* list) come before

the Refreshes (input order: *chunk-reads* list). The Cdc-before-Refresh same-coord convention is realized by placing the Cdc-events list *first* in the concatenated input to *sort-key*; *sort-key* is stable, so same-coord same-type relative order is preserved.

4. Step 4 (retain duplicate stutters). The construction does not deduplicate; same-coord same-key duplicates are preserved. Apply's per-key fold semantics (latest-event-wins, *apply-latest-event-wins*) handles intra-batch duplicates at the state level.

The suppression-dominance step is split into a raw form (this *canonical-clean-prefix*) and a normalized form (*canonical-clean-prefix-normalized*); the two produce Apply-equivalent per-key states — proved below as *apply-canonical-clean-prefix-normalized-eq* (paper "Suppression dominance" / "Apply state-equivalence remark"). The Layer 1 definitional equality picks the raw form; deployments may pre-normalize and remain wellformed under the same predicate.

Five properties of *clean-prefix-of*(*R*) — refresh-generated, cdc-generated, no-future-cdcs, no-extra-events, coord-ordered — are derived as separate supporting lemmas in *DBLog-Run-Substrate-Layer2* (four are consumed by the per-key replay-equality proof; the no-future-CDCs bound is recorded as a standalone sanity fact).

Helper accessors used by the construction's interleave + the normalized companion's suppression filter. *event-key* and *is-refresh* live in theory *Replay* as Layer 0 projectors on (*'k*, *'v*) *replay-event*; this theory imports them transitively.

Step 1 builder: per *chunk-read* record, emit one Refresh per in-domain key with an observation. The *sorted-list-of-set* enumeration requires *linorder* on the *'k* type variable and a finite *crr-domain* (the latter is a paper-level construction premise; if violated, *sorted-list-of-set* returns [] and the construction emits no Refreshes for that chunk read).

definition *chunk-read-refreshes*

$:: ('k :: \text{linorder}, 'v) \text{ chunk-read-record} \Rightarrow ('k, 'v) \text{ replay-event list}$

where

$\text{chunk-read-refreshes } r =$
 List.map-filter
 $(\lambda k. \text{case crr-observation } r \text{ } k \text{ of}$
 $\quad \text{Some } m \Rightarrow \text{Some } (\text{Refresh } k \text{ } m \text{ } (\text{crr-read-coord } r))$
 $\quad | \text{None} \Rightarrow \text{None})$
 $(\text{sorted-list-of-set } (\text{crr-domain } r))$

Step 2 builder: filter *cdc-events* by scope + frontier and lift to *Cdc c e* replay events. Preserves input list order.

definition *cdc-event-replays*

$:: 'k \text{ set} \Rightarrow \text{src-coord}$
 $\Rightarrow (\text{src-coord} \times ('k, 'v) \text{ source-event}) \text{ list}$
 $\Rightarrow ('k, 'v) \text{ replay-event list}$

where

$\text{cdc-event-replays scope frontier es} =$
 List.map-filter
 $(\lambda p. \text{ if key-of (snd p) } \in \text{scope} \wedge \text{fst p} \leq \text{frontier}$
 $\text{ then Some (Cdc (fst p) (snd p))}$
 $\text{ else None})$
 es

The construction proper. Steps 1+2 generate the Refresh and Cdc event lists; step 3 interleaves them by source coordinate via stable *sort-key*; step 4 is implicit (no deduplication). The Cdc-event list is placed *first* in the *sort-key* input so that, at any same source coordinate, Cdc's precede Refreshes (the same-coord read convention from paper "Source state").

definition canonical-clean-prefix

$:: ('k :: \text{linorder}, 'v) \text{ chunk-read-record list}$
 $\Rightarrow (\text{src-coord} \times ('k, 'v) \text{ source-event}) \text{ list}$
 $\Rightarrow 'k \text{ set}$
 $\Rightarrow \text{src-coord}$
 $\Rightarrow ('k, 'v) \text{ replay-event list}$

where

$\text{canonical-clean-prefix chunk-reads cdc-events scope frontier} =$
 $\text{sort-key cdc-coord}$
 $(\text{cdc-event-replays scope frontier cdc-events}$
 $\text{@ concat (map chunk-read-refreshes chunk-reads))}$

Suppression dominance: a Refresh at position i is dominated when some later position carries a non-Refresh (i.e., a Cdc) event for the same key at a strictly later source coordinate. Cdc events are never themselves dominated.

definition refresh-dominated

$:: ('k, 'v) \text{ replay-event list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

$\text{refresh-dominated } \sigma \ i \longleftrightarrow$
 $i < \text{length } \sigma \wedge \text{is-refresh } (\sigma ! i) \wedge$
 $(\exists j. i < j \wedge j < \text{length } \sigma$
 $\wedge \neg \text{is-refresh } (\sigma ! j)$
 $\wedge \text{src-lt (cdc-coord } (\sigma ! i)) \text{ (cdc-coord } (\sigma ! j))$
 $\wedge \text{event-key } (\sigma ! j) = \text{event-key } (\sigma ! i))$

definition canonical-clean-prefix-normalized

$:: ('k :: \text{linorder}, 'v) \text{ chunk-read-record list}$
 $\Rightarrow (\text{src-coord} \times ('k, 'v) \text{ source-event}) \text{ list}$
 $\Rightarrow 'k \text{ set}$
 $\Rightarrow \text{src-coord}$
 $\Rightarrow ('k, 'v) \text{ replay-event list}$

where

$\text{canonical-clean-prefix-normalized chunk-reads cdc-events scope frontier} =$

```

    (let  $\sigma$  = canonical-clean-prefix chunk-reads cdc-events scope frontier
    in map ( $\lambda i. \sigma ! i$ )
      (filter ( $\lambda i. \neg \text{refresh-dominated } \sigma i$ ) [0.. $\text{length } \sigma$ ]))

```

1.3 Normalized clean prefix: Apply-equivalence to the raw form

Paper "Suppression dominance" / "Apply state-equivalence remark": *canonical-clean-prefix-normalized* (dominated refreshes suppressed) and the raw *canonical-clean-prefix* produce the same per-key *Apply* state. This is the machine-checked "proof-facing bridge" the paper relies on when it says a deployment may pre-normalize and remain wellformed under the same predicate.

The argument: *Apply* is latest-event-wins per key (*apply-eq-apply-filter-key*), so *Apply* σ k is fixed by the last k -event of σ . A position that *refresh-dominated* removes is a refresh that has a strictly later same-key event, so it is never the last k -event; conversely the last k -event is never *refresh-dominated* and therefore survives normalization, in the same final position. Hence every key's last event — and so *Apply* — is unchanged.

Ownership note: this trio is not consumed by the main theorems — the headline ladder uses only the raw *canonical-clean-prefix*, and no definition or proof elsewhere in the development references *refresh-dominated*, *canonical-clean-prefix-normalized*, or *apply-canonical-clean-prefix-normalized-eq*. It is retained as the paper's state-equivalence bridge.

lemma *cpn-filter-swap*:

```

  filter P (filter Q xs) = filter Q (filter P xs)
  by (induction xs) auto

```

lemma *cpn-sorted-filter*:

```

  fixes xs :: nat list
  shows sorted xs  $\implies$  sorted (filter P xs)
  by (induction xs) auto

```

lemma *cpn-sorted-le-last*:

```

  fixes zs :: nat list
  assumes sorted zs and  $zs \neq []$  and  $z \in \text{set } zs$ 
  shows  $z \leq \text{last } zs$ 

```

proof —

```

  from  $\langle z \in \text{set } zs \rangle$  obtain m where m:  $m < \text{length } zs$   $zs ! m = z$ 
  by (meson in-set-conv-nth)
  have  $m \leq \text{length } zs - 1$  using m(1) by simp
  moreover have  $\text{length } zs - 1 < \text{length } zs$  using  $\langle zs \neq [] \rangle$  by simp
  ultimately have  $zs ! m \leq zs ! (\text{length } zs - 1)$ 
  using  $\langle \text{sorted } zs \rangle$  sorted-nth-mono by blast
  thus ?thesis using m(2)  $\langle zs \neq [] \rangle$  by (simp add: last-conv-nth)

```

qed

If two replay-event lists agree on whether they carry any k -event, and (when they do) on the last k -event, then *Apply* agrees on k : *Apply* at a key is fixed by that key's last event.

lemma *cpn-apply-eq-of-filter-key-last*:

fixes $xs\ ys :: ('k, 'v)\ \text{replay-event list}$ **and** $k :: 'k$

assumes *empty-iff*:

$(\text{filter } (\lambda e. \text{event-key } e = k)\ xs = [])$
 $= (\text{filter } (\lambda e. \text{event-key } e = k)\ ys = [])$

assumes *last-eq*:

$\text{filter } (\lambda e. \text{event-key } e = k)\ xs \neq []$
 $\implies \text{last } (\text{filter } (\lambda e. \text{event-key } e = k)\ xs)$
 $= \text{last } (\text{filter } (\lambda e. \text{event-key } e = k)\ ys)$

shows $\text{Apply } xs\ k = \text{Apply } ys\ k$

proof –

have $ax: \text{Apply } xs\ k = \text{Apply } (\text{filter } (\lambda e. \text{event-key } e = k)\ xs)\ k$
by (*rule apply-eq-filter-key*)

have $ay: \text{Apply } ys\ k = \text{Apply } (\text{filter } (\lambda e. \text{event-key } e = k)\ ys)\ k$
by (*rule apply-eq-filter-key*)

show *?thesis*

proof (*cases filter* $(\lambda e. \text{event-key } e = k)\ xs = []$)

case *True*

with *empty-iff* **have** $\text{filter } (\lambda e. \text{event-key } e = k)\ ys = []$ **by** *simp*

with *True ax ay* **show** *?thesis* **by** (*simp add: Apply-def*)

next

case *False*

with *empty-iff* **have** $yne: \text{filter } (\lambda e. \text{event-key } e = k)\ ys \neq []$ **by** *simp*

have $xu: \forall e \in \text{set } (\text{filter } (\lambda e. \text{event-key } e = k)\ xs). \text{event-key } e = k$
by *auto*

have $yu: \forall e \in \text{set } (\text{filter } (\lambda e. \text{event-key } e = k)\ ys). \text{event-key } e = k$
by *auto*

have $e1: \text{Apply } (\text{filter } (\lambda e. \text{event-key } e = k)\ xs)\ k$
 $= (\text{case last } (\text{filter } (\lambda e. \text{event-key } e = k)\ xs) \text{ of}$
 $\quad \text{Cdc - (Insert - } v) \Rightarrow \text{Some } v$
 $\quad | \text{Cdc - (Update - } v) \Rightarrow \text{Some } v$
 $\quad | \text{Cdc - (Delete -)} \Rightarrow \text{None}$
 $\quad | \text{Refresh - m-obs -} \Rightarrow \text{m-obs})$

by (*rule apply-filter-uniform-last[OF xu False]*)

have $e2: \text{Apply } (\text{filter } (\lambda e. \text{event-key } e = k)\ ys)\ k$
 $= (\text{case last } (\text{filter } (\lambda e. \text{event-key } e = k)\ ys) \text{ of}$
 $\quad \text{Cdc - (Insert - } v) \Rightarrow \text{Some } v$
 $\quad | \text{Cdc - (Update - } v) \Rightarrow \text{Some } v$
 $\quad | \text{Cdc - (Delete -)} \Rightarrow \text{None}$
 $\quad | \text{Refresh - m-obs -} \Rightarrow \text{m-obs})$

by (*rule apply-filter-uniform-last[OF yu yne]*)

show *?thesis* **using** $ax\ ay\ e1\ e2\ \text{last-eq}[OF\ False]$ **by** *simp*

qed

qed

theorem *apply-canonical-clean-prefix-normalized-eq*:


```

fixes chunk-reads :: ('k :: linorder, 'v) chunk-read-record list
and cdc-events :: (src-coord × ('k, 'v) source-event) list
and scope :: 'k set
and frontier :: src-coord
shows Apply (canonical-clean-prefix-normalized chunk-reads cdc-events scope
frontier)
  = Apply (canonical-clean-prefix chunk-reads cdc-events scope frontier)
proof (intro ext)
fix k :: 'k
define  $\sigma$  where  $\sigma$  = canonical-clean-prefix chunk-reads cdc-events scope frontier
have norm-unfold:
  canonical-clean-prefix-normalized chunk-reads cdc-events scope frontier
    = map ( $\lambda i. \sigma ! i$ ) (filter ( $\lambda i. \neg$  refresh-dominated  $\sigma i$ ) [0..length  $\sigma$ ])
  by (simp add: canonical-clean-prefix-normalized-def Let-def  $\sigma$ -def)
define J where J = filter ( $\lambda i. \text{event-key } (\sigma ! i) = k$ ) [0..length  $\sigma$ ]
define Jn where Jn = filter ( $\lambda i. \neg$  refresh-dominated  $\sigma i$ ) J
have sigma-k: filter ( $\lambda e. \text{event-key } e = k$ )  $\sigma$  = map ( $\lambda i. \sigma ! i$ ) J
proof -
  have filter ( $\lambda e. \text{event-key } e = k$ )  $\sigma$ 
    = filter ( $\lambda e. \text{event-key } e = k$ ) (map ( $\lambda i. \sigma ! i$ ) [0..length  $\sigma$ ])
  by (simp only: map-nth)
  also have ... = map ( $\lambda i. \sigma ! i$ )
    (filter ( $\lambda i. \text{event-key } (\sigma ! i) = k$ ) [0..length  $\sigma$ ])
  by (simp add: filter-map o-def)
  finally show ?thesis by (simp add: J-def)
qed
have norm-k: filter ( $\lambda e. \text{event-key } e = k$ )
  (map ( $\lambda i. \sigma ! i$ )
    (filter ( $\lambda i. \neg$  refresh-dominated  $\sigma i$ ) [0..length  $\sigma$ ]))
  = map ( $\lambda i. \sigma ! i$ ) Jn
proof -
  have filter ( $\lambda e. \text{event-key } e = k$ )
    (map ( $\lambda i. \sigma ! i$ )
      (filter ( $\lambda i. \neg$  refresh-dominated  $\sigma i$ ) [0..length  $\sigma$ ]))
    = map ( $\lambda i. \sigma ! i$ )
      (filter ( $\lambda i. \text{event-key } (\sigma ! i) = k$ )
        (filter ( $\lambda i. \neg$  refresh-dominated  $\sigma i$ ) [0..length  $\sigma$ ]))
  by (simp add: filter-map o-def)
  also have ... = map ( $\lambda i. \sigma ! i$ ) Jn
  by (subst cpn-filter-swap) (simp add: J-def Jn-def)
  finally show ?thesis .
qed
have sortedJ: sorted J by (simp add: J-def cpn-sorted-filter)
have keyJ:  $\bigwedge i. i \in \text{set } J \implies i < \text{length } \sigma \wedge \text{event-key } (\sigma ! i) = k$ 
  by (simp add: J-def)
have crux:  $J \neq [] \implies \neg$  refresh-dominated  $\sigma$  (last J)
proof (rule notI)
  assume Jne:  $J \neq []$  and dom: refresh-dominated  $\sigma$  (last J)
  have lin: last J  $\in \text{set } J$  using Jne by simp

```

```

from dom obtain j where j: last J < j j < length σ
  event-key (σ ! j) = event-key (σ ! (last J))
  unfolding refresh-dominated-def by blast
from keyJ[OF lin] j(3) have event-key (σ ! j) = k by simp
with j(2) have j ∈ set J by (simp add: J-def)
with sortedJ Jne have j ≤ last J by (simp add: cpn-sorted-le-last)
with j(1) show False by simp
qed
have emptyiff:
  (filter (λe. event-key e = k)
    (map (λi. σ ! i)
      (filter (λi. ¬ refresh-dominated σ i) [0..proof -
  have (Jn = []) = (J = [])
  proof
    assume J = [] thus Jn = [] by (simp add: Jn-def)
  next
    assume Jnnil: Jn = []
    show J = []
    proof (rule ccontr)
      assume Jne: J ≠ []
      have last J ∈ set Jn
      using Jne crux[OF Jne] by (simp add: Jn-def)
      with Jnnil show False by simp
    qed
  qed
thus ?thesis by (simp add: sigma-k norm-k)
qed
have lasteq:
  filter (λe. event-key e = k)
    (map (λi. σ ! i)
      (filter (λi. ¬ refresh-dominated σ i) [0..proof -
  assume filter (λe. event-key e = k)
    (map (λi. σ ! i)
      (filter (λi. ¬ refresh-dominated σ i) [0..hence Jnne: Jn ≠ [] by (simp add: norm-k)
  hence Jne: J ≠ [] by (auto simp: Jn-def)
  have lastJn: last Jn = last J
  proof -
    have Jn = filter (λi. ¬ refresh-dominated σ i) (butlast J @ [last J])
      by (simp add: Jn-def append-butlast-last-id[OF Jne])
    also have ... = filter (λi. ¬ refresh-dominated σ i) (butlast J) @ [last J]
      using crux[OF Jne] by simp
  qed

```

```

    finally show ?thesis by simp
qed
have last (map ( $\lambda i. \sigma ! i$ )  $Jn$ ) =  $\sigma ! (last\ Jn)$ 
  using  $Jnne$  by (simp add: last-map)
also have ... =  $\sigma ! (last\ J)$  by (simp add: lastJn)
also have ... = last (map ( $\lambda i. \sigma ! i$ )  $J$ )
  using  $Jne$  by (simp add: last-map)
finally show ?thesis by (simp add: sigma-k norm-k)
qed
show Apply (canonical-clean-prefix-normalized chunk-reads cdc-events scope frontier)  $k$ 
  = Apply (canonical-clean-prefix chunk-reads cdc-events scope frontier)  $k$ 
  unfolding norm-unfold  $\sigma$ -def[symmetric]
  by (rule cpn-apply-eq-of-filter-key-last[OF emptyiff lasteq])
qed

```

1.4 Source key/coordinate slice helper

source-key-coord-slice filters a source/observed event list down to the records at one source coordinate carrying one key. It is consumed by the WF2 post-read multiplicity clause and the Layer 2 per-key replay-equality lemmas; it references no run/chunk accessor.

definition *source-key-coord-slice*

```

:: 'k  $\Rightarrow$  src-coord
 $\Rightarrow$  (src-coord  $\times$  ('k, 'v) source-event) list
 $\Rightarrow$  (src-coord  $\times$  ('k, 'v) source-event) list

```

where

```

source-key-coord-slice  $k\ c\ xs =$ 
  filter ( $\lambda p. hist\ coord\ p = c \wedge key\ of\ (hist\ event\ p) = k$ )  $xs$ 

```

2 Carrier-independent Layer 2 supporting helpers

The pure, carrier-independent list / source-history / replay helper lemmas below reference no run/chunk accessor. They are collected here so that both the Layer 2 substrate locale *dblog-run-substrate* (whose Layer 2 lemmas live in *DBLog-Run-Substrate-Layer2*) and the certificate core *Virtual-Cut-Core* can reuse them directly.

Auxiliary list lemmas used by *clean-prefix-of-cdc-order-respects-source-position* and *clean-prefix-same-coord-refresh-before-cdc-false*: *filter* preserves the strict order of P -satisfying positions in the underlying list. These are generic facts about *filter*, *take*, and (!) on Isabelle lists; they package the standard “position-of-the- n -th-survivor” argument in a form that lemma can consume without explicit *nths* bookkeeping.

Three pieces:

- *filter-take-nth*: for any position i satisfying P , the *length* (*filter* P (*take*

i xs))-th element of $\text{filter } P \text{ } xs$ is exactly $xs ! i$; furthermore that index lies strictly below $\text{length } (\text{filter } P \text{ } xs)$. This gives the *forward* direction of the position correspondence (positions in $xs \rightarrow$ positions in $\text{filter } P \text{ } xs$).

- *length-filter-take-mono / length-filter-take-strict-mono*: $\text{length} \circ \text{filter } P \circ \text{take } \cdot$ is monotone (resp. strictly monotone at any P -satisfying position) in the take-prefix length. The strict version drives the $n-i < n-j$ step in *clean-prefix-of-cdc-order-respects-source-position* (in theory *DBLog-Run-Substrate-Layer2*).
- *filter-index-strict-mono*: for any $n < \text{length } (\text{filter } P \text{ } xs)$, there exists a position $p < \text{length } xs$ with $xs ! p = \text{filter } P \text{ } xs ! n$ and $\text{length } (\text{filter } P \text{ } (\text{take } p \text{ } xs)) = n$. This gives the *backward* direction (positions in $\text{filter } P \text{ } xs \rightarrow$ positions in xs). The $\text{length } (\text{filter } P \text{ } (\text{take } p \text{ } xs)) = n$ witness is what lets us recover the original ordering from a pair of n 's via the *length-filter-take-mono* contrapositive.

lemma *filter-take-nth*:

fixes $xs :: 'a \text{ list}$ **and** $P :: 'a \Rightarrow \text{bool}$ **and** $i :: \text{nat}$

assumes $i\text{-lt}$: $i < \text{length } xs$

and Pi : $P (xs ! i)$

shows $\text{filter } P \text{ } xs ! \text{length } (\text{filter } P \text{ } (\text{take } i \text{ } xs)) = xs ! i$

$\wedge \text{length } (\text{filter } P \text{ } (\text{take } i \text{ } xs)) < \text{length } (\text{filter } P \text{ } xs)$

proof –

have $xs\text{-split}$: $xs = \text{take } i \text{ } xs @ xs ! i \# \text{drop } (\text{Suc } i) \text{ } xs$

using $i\text{-lt}$ **by** ($\text{simp add: id-take-nth-drop}$)

have filter-split :

$\text{filter } P \text{ } xs = \text{filter } P \text{ } (\text{take } i \text{ } xs) @ xs ! i \# \text{filter } P \text{ } (\text{drop } (\text{Suc } i) \text{ } xs)$

proof –

have $\text{filter } P \text{ } xs = \text{filter } P \text{ } (\text{take } i \text{ } xs @ xs ! i \# \text{drop } (\text{Suc } i) \text{ } xs)$

using $xs\text{-split}$ **by** simp

also have $\dots = \text{filter } P \text{ } (\text{take } i \text{ } xs) @ \text{filter } P \text{ } (xs ! i \# \text{drop } (\text{Suc } i) \text{ } xs)$

by simp

also have $\text{filter } P \text{ } (xs ! i \# \text{drop } (\text{Suc } i) \text{ } xs) = xs ! i \# \text{filter } P \text{ } (\text{drop } (\text{Suc } i) \text{ } xs)$

xs)

using Pi **by** simp

finally show $?thesis$ **by** simp

qed

have nth-eq :

$\text{filter } P \text{ } xs ! \text{length } (\text{filter } P \text{ } (\text{take } i \text{ } xs)) = xs ! i$

using filter-split **by** ($\text{simp add: nth-append}$)

have len-lt :

$\text{length } (\text{filter } P \text{ } (\text{take } i \text{ } xs)) < \text{length } (\text{filter } P \text{ } xs)$

using filter-split **by** simp

show $?thesis$ **using** nth-eq len-lt **by** simp

qed

lemma *length-filter-take-mono*:

fixes $xs :: 'a \text{ list}$ **and** $P :: 'a \Rightarrow \text{bool}$ **and** $i\ j :: \text{nat}$
assumes $i \leq j$
shows $\text{length} (\text{filter } P (\text{take } i\ xs)) \leq \text{length} (\text{filter } P (\text{take } j\ xs))$
proof –
have *take-i-eq*: $\text{take } i (\text{take } j\ xs) = \text{take } i\ xs$
using *assms* **by** (*simp add: min-def take-take*)
have $\text{take } i\ xs @ \text{drop } i (\text{take } j\ xs) = \text{take } j\ xs$
using *take-i-eq append-take-drop-id* **by** *metis*
hence $\text{filter } P (\text{take } j\ xs)$
 $= \text{filter } P (\text{take } i\ xs) @ \text{filter } P (\text{drop } i (\text{take } j\ xs))$
by (*metis filter-append*)
thus *?thesis* **by** *simp*
qed

lemma *length-filter-take-strict-mono*:

fixes $xs :: 'a \text{ list}$ **and** $P :: 'a \Rightarrow \text{bool}$ **and** $i\ j :: \text{nat}$
assumes *ij*: $i < j$
and *j-bound*: $j \leq \text{length } xs$
and *Pi*: $P (xs ! i)$
shows $\text{length} (\text{filter } P (\text{take } i\ xs)) < \text{length} (\text{filter } P (\text{take } j\ xs))$
proof –
have *i-bound*: $i < \text{length } xs$ **using** *ij j-bound* **by** *linarith*
have *suc-i-le-j*: $\text{Suc } i \leq j$ **using** *ij* **by** *simp*
have *take-suc-i-eq*: $\text{take } (\text{Suc } i) (\text{take } j\ xs) = \text{take } (\text{Suc } i)\ xs$
using *suc-i-le-j* **by** (*simp add: min-def take-take*)
have *suc-split*: $\text{take } (\text{Suc } i)\ xs @ \text{drop } (\text{Suc } i) (\text{take } j\ xs) = \text{take } j\ xs$
using *take-suc-i-eq append-take-drop-id* **by** *metis*
have *take-suc-i*: $\text{take } (\text{Suc } i)\ xs = \text{take } i\ xs @ [xs ! i]$
using *i-bound* **by** (*simp add: take-Suc-conv-app-nth*)
have *decompose*:
 $\text{take } j\ xs = \text{take } i\ xs @ [xs ! i] @ \text{drop } (\text{Suc } i) (\text{take } j\ xs)$
using *suc-split take-suc-i* **by** *simp*
have $\text{filter } P (\text{take } j\ xs)$
 $= \text{filter } P (\text{take } i\ xs @ [xs ! i] @ \text{drop } (\text{Suc } i) (\text{take } j\ xs))$
using *decompose* **by** *simp*
also have $\dots = \text{filter } P (\text{take } i\ xs) @ \text{filter } P [xs ! i] @ \text{filter } P (\text{drop } (\text{Suc } i) (\text{take } j\ xs))$
by *simp*
also have $\text{filter } P [xs ! i] = [xs ! i]$ **using** *Pi* **by** *simp*
finally have $\text{filter } P (\text{take } j\ xs)$
 $= \text{filter } P (\text{take } i\ xs) @ [xs ! i] @ \text{filter } P (\text{drop } (\text{Suc } i) (\text{take } j\ xs))$
by *simp*
thus *?thesis* **by** *simp*
qed

lemma *filter-index-strict-mono*:

fixes $xs :: 'a \text{ list}$ **and** $P :: 'a \Rightarrow \text{bool}$ **and** $n :: \text{nat}$
assumes $n < \text{length} (\text{filter } P\ xs)$

```

shows  $\exists p. p < \text{length } xs$ 
       $\wedge P (xs ! p)$ 
       $\wedge xs ! p = \text{filter } P \ xs ! n$ 
       $\wedge \text{length } (\text{filter } P \ (\text{take } p \ xs)) = n$ 
using assms
proof (induction xs arbitrary: n)
  case Nil
  thus ?case by simp
next
  case (Cons a xs')
  show ?case
  proof (cases P a)
    case Pa: True
    show ?thesis
    proof (cases n)
      case 0
      have at0:  $\text{filter } P \ (a \# xs') ! n = a$  using Pa 0 by simp
      have (0 :: nat) < length (a # xs') by simp
      moreover have  $P \ ((a \# xs') ! 0)$  using Pa by simp
      moreover have  $(a \# xs') ! 0 = \text{filter } P \ (a \# xs') ! n$  using at0 by simp
      moreover have  $\text{length } (\text{filter } P \ (\text{take } 0 \ (a \# xs'))) = n$  using 0 by simp
      ultimately show ?thesis by (intro exI[where x = 0]) auto
    next
      case (Suc n')
      have filter-nth-eq:
         $\text{filter } P \ (a \# xs') ! n = \text{filter } P \ xs' ! n'$ 
        using Pa Suc by simp
      have n'-lt:  $n' < \text{length } (\text{filter } P \ xs')$ 
        using Cons.premis Pa Suc by simp
      obtain p' where p'-props:
         $p' < \text{length } xs' \ P \ (xs' ! p')$ 
         $xs' ! p' = \text{filter } P \ xs' ! n'$ 
         $\text{length } (\text{filter } P \ (\text{take } p' \ xs')) = n'$ 
        using Cons.IH[OF n'-lt] by blast
      have Suc p' < length (a # xs') using p'-props by simp
      moreover have  $P \ ((a \# xs') ! \text{Suc } p')$  using p'-props by simp
      moreover have  $(a \# xs') ! \text{Suc } p' = \text{filter } P \ (a \# xs') ! n$ 
        using filter-nth-eq p'-props by simp
      moreover have  $\text{length } (\text{filter } P \ (\text{take } (\text{Suc } p') \ (a \# xs'))) = n$ 
        using Pa Suc p'-props by simp
      ultimately show ?thesis by (intro exI[where x = Suc p']) auto
    qed
  next
    case nPa: False
    have filter-a-eq:  $\text{filter } P \ (a \# xs') = \text{filter } P \ xs'$  using nPa by simp
    have n < length (filter P xs')
      using Cons.premis filter-a-eq by simp
    obtain p' where p'-props:
       $p' < \text{length } xs' \ P \ (xs' ! p')$ 

```

```

    xs' ! p' = filter P xs' ! n
    length (filter P (take p' xs')) = n
    using Cons.IH ⟨n < length (filter P xs')⟩ by blast
    have Suc p' < length (a # xs') using p'-props by simp
    moreover have P ((a # xs') ! Suc p') using p'-props by simp
    moreover have (a # xs') ! Suc p' = filter P (a # xs') ! n
    using filter-a-eq p'-props by simp
    moreover have length (filter P (take (Suc p') (a # xs'))) = n
    using nPa p'-props by simp
    ultimately show ?thesis by (intro exI[where x = Suc p']) auto
qed
qed

```

```

lemma subseq-mset-eq-imp-eq:
  fixes xs ys :: 'a list
  assumes sub: subseq xs ys
  and ms: mset xs = mset ys
  shows xs = ys
  by (rule subseq-same-length[OF sub mset-eq-length[OF ms]])

```

```

lemma subseq-from-strict-mono-nth:
  fixes xs ys :: 'a list
  and f :: nat ⇒ nat
  assumes mono: strict-mono f
  and hit: ∧i. i < length xs ⇒ f i < length ys ∧ xs ! i = ys ! f i
  shows subseq xs ys
  using assms
proof (induction xs arbitrary: ys f)
  case Nil
  show ?case by simp
next
  case (Cons x xs)
  from Cons.prem1[2][of 0] have f0-lt: f 0 < length ys
  and x-eq: x = ys ! f 0
  by simp-all
  let ?tail = drop (Suc (f 0)) ys
  let ?g = λi. f (Suc i) - Suc (f 0)

  have g-mono: strict-mono ?g
  proof (rule strict-monoI)
    fix i j :: nat
    assume ij: i < j
    have f0-lt-i: f 0 < f (Suc i)
    using strict-monoD[OF Cons.prem1, of 0 Suc i] by simp
    have f (Suc i) < f (Suc j)
    using strict-monoD[OF Cons.prem1, of Suc i Suc j] ij by simp
    with f0-lt-i show ?g i < ?g j by simp
  qed

```

```

have g-hit:
   $\bigwedge i. i < \text{length } xs \implies$ 
     $?g\ i < \text{length } ?tail \wedge xs\ !\ i = ?tail\ !\ ?g\ i$ 
proof -
  fix i
  assume i-lt:  $i < \text{length } xs$ 
  from Cons.prem(2)[of Suc i] i-lt
  have fSi-lt:  $f\ (Suc\ i) < \text{length } ys$ 
    and xs-i:  $xs\ !\ i = ys\ !\ f\ (Suc\ i)$ 
    by simp-all
  have f0-lt-fSi:  $f\ 0 < f\ (Suc\ i)$ 
    using strict-monoD[OF Cons.prem(1), of 0 Suc i] by simp
  hence fSi-eq:  $Suc\ (f\ 0) + ?g\ i = f\ (Suc\ i)$ 
    by simp
  have g-lt:  $?g\ i < \text{length } ?tail$ 
    using fSi-lt fSi-eq by simp
  have tail-nth:  $?tail\ !\ ?g\ i = ys\ !\ f\ (Suc\ i)$ 
    using g-lt fSi-eq by simp
  show  $?g\ i < \text{length } ?tail \wedge xs\ !\ i = ?tail\ !\ ?g\ i$ 
    using g-lt xs-i tail-nth by simp
qed

have tail-sub:  $\text{subseq } xs\ ?tail$ 
  by (rule Cons.IH[OF g-mono g-hit])
have cons-sub:  $\text{subseq } (x \# xs)\ (ys\ !\ f\ 0 \# ?tail)$ 
  using tail-sub x-eq by simp
have ys-split:  $ys = \text{take } (f\ 0)\ ys @ ys\ !\ f\ 0 \# ?tail$ 
  using f0-lt by (simp add: id-take-nth-drop)
have prefixed:  $\text{subseq } (x \# xs)\ (\text{take } (f\ 0)\ ys @ (ys\ !\ f\ 0 \# ?tail))$ 
  by (rule subseq-drop-many[OF cons-sub])
show ?case
  by (subst ys-split, rule prefixed)
qed

lemma last-filter-index:
  fixes xs :: 'a list
  assumes nonempty:  $\text{filter } P\ xs \neq []$ 
  shows  $\exists i. i < \text{length } xs$ 
     $\wedge P\ (xs\ !\ i)$ 
     $\wedge xs\ !\ i = \text{last } (\text{filter } P\ xs)$ 
     $\wedge (\forall j. i < j \wedge j < \text{length } xs \longrightarrow \neg P\ (xs\ !\ j))$ 
  using nonempty
proof (induction xs rule: rev-induct)
  case Nil
  thus ?case by simp
next
  case (snoc x xs)
  show ?case
  proof (cases P x)

```



```

case True
have idx-lt: length xs < length (xs @ [x]) by simp
have P-idx: P ((xs @ [x]) ! length xs) using True by simp
have last-idx: (xs @ [x]) ! length xs = last (filter P (xs @ [x]))
  using True by simp
have no-later:
   $\forall j. \text{length } xs < j \wedge j < \text{length } (xs @ [x])$ 
     $\longrightarrow \neg P ((xs @ [x]) ! j)$ 
  by auto
show ?thesis
  using idx-lt P-idx last-idx no-later
  by (intro exI[where x = length xs]) simp
next
case False
note nPx = False
have filter-xs-nonempty: filter P xs  $\neq []$ 
  using snoc.premis nPx by simp
obtain i where i-lt-xs: i < length xs
  and Pi-xs: P (xs ! i)
  and last-xs: xs ! i = last (filter P xs)
  and no-later-xs:  $\forall j. i < j \wedge j < \text{length } xs \longrightarrow \neg P (xs ! j)$ 
  using snoc.IH[OF filter-xs-nonempty] by blast
have i-lt: i < length (xs @ [x]) using i-lt-xs by simp
have Pi: P ((xs @ [x]) ! i) using Pi-xs i-lt-xs by (simp add: nth-append)
have filter-append-eq: filter P (xs @ [x]) = filter P xs
  using nPx by simp
have last-eq: (xs @ [x]) ! i = last (filter P (xs @ [x]))
proof -
  have (xs @ [x]) ! i = xs ! i
    using i-lt-xs by (simp add: nth-append)
  also have ... = last (filter P xs)
    using last-xs .
  also have ... = last (filter P (xs @ [x]))
    using filter-append-eq by simp
  finally show ?thesis .
qed
have no-later:
   $\forall j. i < j \wedge j < \text{length } (xs @ [x]) \longrightarrow \neg P ((xs @ [x]) ! j)$ 
proof (intro allI impI)
  fix j
  assume j-assm: i < j  $\wedge j < \text{length } (xs @ [x])$ 
  show  $\neg P ((xs @ [x]) ! j)$ 
  proof (cases j < length xs)
    case True
    thus ?thesis using no-later-xs j-assm by (simp add: nth-append)
  next
    case False
    hence j = length xs using j-assm by simp
    thus ?thesis using nPx by simp
  end
end

```

```

    qed
  qed
  show ?thesis
    using i-lt Pi last-eq no-later by blast
  qed
qed

lemma last-filter-upt-eq:
  fixes P :: nat ⇒ bool
  assumes i-lt: i < n
    and Pi: P i
    and no-after: ∀ j. i < j ∧ j < n ⟶ ¬ P j
  shows filter P [0..

```

```

    and xs-A:  $\forall x \in \text{set } xs. A \ x$ 
    and ys-B:  $\forall y \in \text{set } ys. B \ y$ 
    and disj:  $\bigwedge z. A \ z \implies B \ z \implies \text{False}$ 
    and ij:  $i < j$ 
    and j-lt:  $j < \text{length } zs$ 
    and zi-B:  $B \ (zs \ ! \ i)$ 
    and zj-A:  $A \ (zs \ ! \ j)$ 
  shows False
proof (cases  $i < \text{length } xs$ )
  case True
    hence A  $(zs \ ! \ i)$ 
      using zs-eq xs-A by (simp add: nth-append)
    thus False using zi-B disj by blast
  next
    case False
      hence j-ge:  $\text{length } xs \leq j$  using ij by simp
      have j - length xs < length ys
        using zs-eq j-lt j-ge by simp
      hence B  $(zs \ ! \ j)$ 
        using zs-eq ys-B j-ge by (simp add: nth-append)
      thus False using zj-A disj by blast
qed

lemma wellformed-src-history-coord-mono:
  fixes H :: ('k, 'v) src-history
  assumes wf-h: wellformed-src-history H
    and ij:  $i \leq j$ 
    and j-lt:  $j < \text{length } H$ 
  shows src-le  $(\text{hist-coord } (H \ ! \ i)) \ (\text{hist-coord } (H \ ! \ j))$ 
proof -
  from wf-h have adj:
     $\forall i. \text{Suc } i < \text{length } H \implies \text{src-le } (\text{hist-coord } (H \ ! \ i)) \ (\text{hist-coord } (H \ ! \ \text{Suc } i))$ 
  unfolding wellformed-src-history-def by blast
  have step:
     $\bigwedge n. i + n < \text{length } H \implies \text{src-le } (\text{hist-coord } (H \ ! \ i)) \ (\text{hist-coord } (H \ ! \ (i + n)))$ 
  proof -
    fix n
    assume bound:  $i + n < \text{length } H$ 
    show src-le  $(\text{hist-coord } (H \ ! \ i)) \ (\text{hist-coord } (H \ ! \ (i + n)))$ 
      using bound
    proof (induction n)
      case 0
        show ?case by (simp add: src-le-refl)
      next
        case (Suc n)
          have ih-bound:  $i + n < \text{length } H$  using Suc.premis by simp
          have ih:

```

```

    src-le (hist-coord (H ! i)) (hist-coord (H ! (i + n)))
  using Suc.IH[OF ih-bound] .
have adj-step:
  src-le (hist-coord (H ! (i + n))) (hist-coord (H ! Suc (i + n)))
  using adj Suc.prem by simp
have src-le (hist-coord (H ! i)) (hist-coord (H ! Suc (i + n)))
  using src-le-trans[OF ih adj-step] .
thus ?case by (simp add: add-Suc-right)
qed
qed
have j-eq: j = i + (j - i) using ij by simp
show ?thesis
  using step[of j - i] j-lt j-eq by simp
qed

```

Auxiliary lemma used by *clean-prefix-of-per-key-replay-equals-source*: the *Apply* value at a key k reduces to the per-event constructor effect of the k -event at the *latest* position, when one such position is named.

Generic fact about *Apply* on lists of *replay-event*: given a position $i\text{-max}$ in a sequence σ such that $\text{event-key } (\sigma ! i\text{-max}) = k$ and no later position in σ carries $\text{event-key} = k$, the per-key value $\text{Apply } \sigma \ k$ equals the per-constructor effect of $\sigma ! i\text{-max}$: *Some* v for the *Cdc Insert / Update* shapes, *None* for the *Cdc Delete* shape, and the observed value $m\text{-obs}$ for the *Refresh* shape.

Composes $\text{Apply } ?\sigma \ ?k = (\text{case filter } (\lambda e. \text{event-key } e = ?k) \ ?\sigma \text{ of } [] \Rightarrow \text{None} \mid x \# xa \Rightarrow \text{case last } (\text{filter } (\lambda e. \text{event-key } e = ?k) \ ?\sigma) \text{ of } \text{Cdc } x \ (\text{Insert } xa \ v) \Rightarrow \text{Some } v \mid \text{Cdc } x \ (\text{Update } xa \ v) \Rightarrow \text{Some } v \mid \text{Cdc } x \ (\text{Delete } xa) \Rightarrow \text{None} \mid \text{Refresh } x \ m\text{-obs } xa \Rightarrow m\text{-obs})$ with a list decomposition argument: split σ at $i\text{-max}$, observe that the suffix $\text{drop } (\text{Suc } i\text{-max}) \ \sigma$ contributes no k -events to the per-key filter (by the $i\text{-max}$ latest-position assumption), and conclude that the last element of the per-key filter is $\sigma ! i\text{-max}$; then apply $\text{Apply } ?\sigma \ ?k = (\text{case filter } (\lambda e. \text{event-key } e = ?k) \ ?\sigma \text{ of } [] \Rightarrow \text{None} \mid x \# xa \Rightarrow \text{case last } (\text{filter } (\lambda e. \text{event-key } e = ?k) \ ?\sigma) \text{ of } \text{Cdc } x \ (\text{Insert } xa \ v) \Rightarrow \text{Some } v \mid \text{Cdc } x \ (\text{Update } xa \ v) \Rightarrow \text{Some } v \mid \text{Cdc } x \ (\text{Delete } xa) \Rightarrow \text{None} \mid \text{Refresh } x \ m\text{-obs } xa \Rightarrow m\text{-obs})$ to read off the per-constructor case.

Used twice in the Layer 2 per-key replay-equality proof in *DBLog-Run-Substrate-Layer2*: once in the *Cdc* case (where $\sigma ! i\text{-max} = \text{Cdc } c \ e$, via the *Cdc*-case helper lemma) and once in the *Refresh* case (where $\sigma ! i\text{-max} = \text{Refresh } k \ m \ c\text{-read}$). Lives here, with the other carrier-independent helpers, rather than in *Dual-Write-Layer0.Replay* because that proof is its only consumer; relocation to that shared theory is reasonable if a second consumer ever materializes.

lemma *apply-at-latest-k-event*:

```

  fixes  $\sigma :: ('k, 'v) \text{ replay-event list}$ 
  and  $k :: 'k$ 
  and  $i :: \text{nat}$ 

```

assumes $i\text{-lt}$: $i < \text{length } \sigma$
and $i\text{-key}$: $\text{event-key } (\sigma ! i) = k$
and $i\text{-last}$: $\forall j. i < j \wedge j < \text{length } \sigma \longrightarrow \text{event-key } (\sigma ! j) \neq k$
shows $\text{Apply } \sigma k = (\text{case } \sigma ! i \text{ of}$
 $\quad \text{Cdc - (Insert - } v) \Rightarrow \text{Some } v$
 $\quad | \text{Cdc - (Update - } v) \Rightarrow \text{Some } v$
 $\quad | \text{Cdc - (Delete -)} \Rightarrow \text{None}$
 $\quad | \text{Refresh - m-obs -} \Rightarrow \text{m-obs})$

proof –

let $?P = \lambda e :: ('k, 'v) \text{ replay-event. event-key } e = k$
let $? \sigma\text{-}k = \text{filter } ?P \sigma$

— Decompose σ at position i .

have decomp : $\sigma = \text{take } i \sigma @ [\sigma ! i] @ \text{drop } (\text{Suc } i) \sigma$
using $i\text{-lt}$
by (*metis Cons-nth-drop-Suc append-Cons append-Nil append-take-drop-id*)

— The drop part has no k -events.

have $\text{drop-no-}k$: $\text{filter } ?P (\text{drop } (\text{Suc } i) \sigma) = []$

proof –

have $\forall e \in \text{set } (\text{drop } (\text{Suc } i) \sigma). \text{event-key } e \neq k$

proof

fix e **assume** $e\text{-in}$: $e \in \text{set } (\text{drop } (\text{Suc } i) \sigma)$
then obtain j **where** $j\text{-lt-drop}$: $j < \text{length } (\text{drop } (\text{Suc } i) \sigma)$
and $e\text{-at}$: $\text{drop } (\text{Suc } i) \sigma ! j = e$
by (*auto simp: in-set-conv-nth*)

let $?j' = \text{Suc } (i + j)$

have $j'\text{-lt}$: $?j' < \text{length } \sigma$

using $j\text{-lt-drop}$ **by** *simp*

have $e\text{-eq}$: $e = \sigma ! ?j'$

using $e\text{-at } j\text{-lt-drop}$

by (*simp add: add.commute*)

have $i\text{-lt-}j'$: $i < ?j'$ **by** *simp*

from $i\text{-last } i\text{-lt-}j' j'\text{-lt}$

have $\text{event-key } (\sigma ! ?j') \neq k$ **by** *blast*

thus $\text{event-key } e \neq k$ **using** $e\text{-eq}$ **by** *simp*

qed

thus $?thesis$ **by** (*simp add: filter-empty-conv*)

qed

— Filter splits + drop has no k -events: filter equals filter on the prefix, with $[\sigma ! i]$ appended.

have filter-eq :

$? \sigma\text{-}k = \text{filter } ?P (\text{take } i \sigma @ [\sigma ! i])$

proof –

have $? \sigma\text{-}k = \text{filter } ?P (\text{take } i \sigma @ [\sigma ! i] @ \text{drop } (\text{Suc } i) \sigma)$

using decomp **by** *simp*

also have $\dots = \text{filter } ?P (\text{take } i \sigma @ \text{filter } ?P [\sigma ! i])$

$@ \text{filter } ?P (\text{drop } (\text{Suc } i) \sigma)$

```

    by simp
  also have filter ?P  $[\sigma ! i] = [\sigma ! i]$ 
    using i-key by simp
  also have filter ?P  $(\text{drop } (\text{Suc } i) \sigma) = []$ 
    by (rule drop-no-k)
  finally show ?thesis by simp
qed

```

```

have nonempty:  $?\sigma\text{-}k \neq []$ 
  using filter-eq by simp
have last-eq:  $\text{last } ?\sigma\text{-}k = \sigma ! i$ 
  using filter-eq by simp

```

— Apply via *apply-latest-event-wins*, then read off via *last-eq*.

```

have apply-eq:
  Apply  $\sigma k = (\text{case } ?\sigma\text{-}k \text{ of}$ 
     $[] \Rightarrow \text{None}$ 
     $| - \# - \Rightarrow (\text{case last } ?\sigma\text{-}k \text{ of}$ 
       $\text{Cdc - (Insert - } v) \Rightarrow \text{Some } v$ 
       $| \text{Cdc - (Update - } v) \Rightarrow \text{Some } v$ 
       $| \text{Cdc - (Delete -)} \Rightarrow \text{None}$ 
       $| \text{Refresh - } m\text{-obs -} \Rightarrow m\text{-obs}))$ 
    by (rule apply-latest-event-wins)
  also have ... = (case last  $?\sigma\text{-}k$  of
     $\text{Cdc - (Insert - } v) \Rightarrow \text{Some } v$ 
     $| \text{Cdc - (Update - } v) \Rightarrow \text{Some } v$ 
     $| \text{Cdc - (Delete -)} \Rightarrow \text{None}$ 
     $| \text{Refresh - } m\text{-obs -} \Rightarrow m\text{-obs})$ 
    using nonempty by (cases  $?\sigma\text{-}k$ ) auto
  also have ... = (case  $\sigma ! i$  of
     $\text{Cdc - (Insert - } v) \Rightarrow \text{Some } v$ 
     $| \text{Cdc - (Update - } v) \Rightarrow \text{Some } v$ 
     $| \text{Cdc - (Delete -)} \Rightarrow \text{None}$ 
     $| \text{Refresh - } m\text{-obs -} \Rightarrow m\text{-obs})$ 
    using last-eq by simp
  finally show ?thesis .
qed

```

Auxiliary lemma used by *clean-prefix-of-per-key-replay-equals-source*: if no k -event in *src-history-of* $R = H$ lies strictly past a coordinate c' and at-or-before the frontier, then $\text{Src } b0 H f k$ restricted to k agrees with $\text{Src } b0 H c' k$. Generic property of $\text{Src} + \text{latest-src-event}$ on histories: when *latest-src-event* cand-set agrees on the two thresholds (no k -events in the half-open interval $(c', f]$), the per-key state is identical.

Used in the *Refresh* case of *clean-prefix-of-per-key-replay-equals-source*: lifts *refresh-coordinate-match* (Src at chunk-read coordinate) to Src at frontier when no later source event for k appears in the half-open interval $(c\text{-read}, \text{frontier-of}(R)]$.

lemma *src-eq-when-no-later-src-event-for-k*:

fixes $b0 :: 'k \rightarrow 'v$

and $H :: ('k, 'v) \text{ src-history}$

and $c' f :: \text{frontier}$

and $k :: 'k$

assumes $c'\text{-le-}f$: $\text{src-le } c' f$

and *no-later*:

$\forall i < \text{length } H. \text{src-lt } c' (\text{hist-coord } (H ! i))$
 $\quad \wedge \text{src-le } (\text{hist-coord } (H ! i)) f$
 $\quad \longrightarrow \text{key-of } (\text{hist-event } (H ! i)) \neq k$

shows $\text{Src } b0 H f k = \text{Src } b0 H c' k$

proof –

let $?cand\text{-}f =$

$\text{filter } (\lambda i. \text{src-le } (\text{hist-coord } (H ! i)) f$
 $\quad \wedge \text{key-of } (\text{hist-event } (H ! i)) = k)$
 $[0..<\text{length } H]$

let $?cand\text{-}c' =$

$\text{filter } (\lambda i. \text{src-le } (\text{hist-coord } (H ! i)) c'$
 $\quad \wedge \text{key-of } (\text{hist-event } (H ! i)) = k)$
 $[0..<\text{length } H]$

let $?Pf = \lambda i. \text{src-le } (\text{hist-coord } (H ! i)) f$
 $\quad \wedge \text{key-of } (\text{hist-event } (H ! i)) = k$

let $?Pc' = \lambda i. \text{src-le } (\text{hist-coord } (H ! i)) c'$
 $\quad \wedge \text{key-of } (\text{hist-event } (H ! i)) = k$

— The two candidate filters coincide.

have *cand-eq*: $?cand\text{-}f = ?cand\text{-}c'$

proof –

have $\bigwedge i. i \in \text{set } [0..<\text{length } H] \implies ?Pf i = ?Pc' i$

proof –

fix i **assume** $i\text{-in}$: $i \in \text{set } [0..<\text{length } H]$

hence $i\text{-lt}$: $i < \text{length } H$ **by** *simp*

show $?Pf i = ?Pc' i$

proof

assume Pf : $?Pf i$

hence $le\text{-}f$: $\text{src-le } (\text{hist-coord } (H ! i)) f$

and $key\text{-}eq$: $\text{key-of } (\text{hist-event } (H ! i)) = k$

by *auto*

have $\text{src-le } (\text{hist-coord } (H ! i)) c'$

proof (*rule ccontr*)

assume $\neg \text{src-le } (\text{hist-coord } (H ! i)) c'$

hence $\text{src-le } c' (\text{hist-coord } (H ! i))$
 $\quad \text{hist-coord } (H ! i) \neq c'$

using *src-le-total*

by (*auto simp: less-eq-src-coord-def*)

hence $\text{src-lt } c' (\text{hist-coord } (H ! i))$

unfolding *src-lt-def* **by** *simp*

with *no-later i-lt le-f* **have** $\text{key-of } (\text{hist-event } (H ! i)) \neq k$

by *blast*

```

    with key-eq show False by simp
  qed
  thus ?Pc' i using key-eq by simp
next
  assume Pc': ?Pc' i
  hence le-c': src-le (hist-coord (H ! i)) c'
    and key-eq: key-of (hist-event (H ! i)) = k
    by auto
  have src-le (hist-coord (H ! i)) f
    using le-c' c'-le-f src-le-trans by blast
  thus ?Pf i using key-eq by simp
qed
qed
thus ?thesis by (rule filter-cong[OF refl])
qed

have latest-src-event H f k = latest-src-event H c' k
  unfolding latest-src-event-def using cand-eq by simp
thus ?thesis
  unfolding Src-def by simp
qed

```

lemma *src-frontier-eq-effect-of-last-key-coord-slice*:

```

fixes b0 :: 'k  $\rightarrow$  'v
and H :: ('k, 'v) src-history
and f c :: src-coord
and k :: 'k
and e :: ('k, 'v) source-event
assumes wf-h: wellformed-src-history H
and c-le-f: src-le c f
and no-later:
   $\forall j < \text{length } H.$ 
    src-lt c (hist-coord (H ! j))
     $\wedge$  src-le (hist-coord (H ! j)) f
     $\longrightarrow$  key-of (hist-event (H ! j))  $\neq$  k
and slice-nonempty: source-key-coord-slice k c H  $\neq$  []
and last-slice: last (source-key-coord-slice k c H) = (c, e)
shows Src b0 H f k = (case e of
  Insert - v  $\Rightarrow$  Some v
  | Update - v  $\Rightarrow$  Some v
  | Delete -  $\Rightarrow$  None)

```

proof –

```

let ?Q =  $\lambda p::(\text{src-coord} \times ('k, 'v) \text{ source-event}).$ 
  hist-coord p = c  $\wedge$  key-of (hist-event p) = k
let ?P =  $\lambda j.$  src-le (hist-coord (H ! j)) f
   $\wedge$  key-of (hist-event (H ! j)) = k

```

obtain i where


```

i-lt:  $i < \text{length } H$  and
Q-i:  $?Q (H ! i)$  and
H-i-last:  $H ! i = \text{last } (\text{filter } ?Q H)$  and
no-after-same:  $\forall j. i < j \wedge j < \text{length } H \longrightarrow \neg ?Q (H ! j)$ 
using last-filter-index[of ?Q H] slice-nonempty
unfolding source-key-coord-slice-def by blast
have H-i:  $H ! i = (c, e)$ 
using H-i-last last-slice
unfolding source-key-coord-slice-def by simp
have coord-i:  $\text{hist-coord } (H ! i) = c$ 
using H-i by simp
have event-i:  $\text{hist-event } (H ! i) = e$ 
using H-i by simp
have P-i:  $?P i$ 
using c-le-f Q-i by simp

have no-after-P:  $\forall j. i < j \wedge j < \text{length } H \longrightarrow \neg ?P j$ 
proof (intro allI impI)
  fix j
  assume j-assm:  $i < j \wedge j < \text{length } H$ 
  hence i-le-j:  $i \leq j$  and j-lt:  $j < \text{length } H$  by auto
  show  $\neg ?P j$ 
  proof
    assume P-j:  $?P j$ 
    hence j-le-f:  $\text{src-le } (\text{hist-coord } (H ! j)) f$ 
    and j-key:  $\text{key-of } (\text{hist-event } (H ! j)) = k$ 
    by auto
    have c-le-j:  $\text{src-le } c (\text{hist-coord } (H ! j))$ 
    using wellformed-src-history-coord-mono[OF wf-h i-le-j j-lt]
    coord-i
    by simp
    show False
    proof (cases hist-coord (H ! j) = c)
      case True
      hence  $?Q (H ! j)$  using j-key by simp
      with no-after-same j-assm show False by blast
    next
    case False
    hence c-lt-j:  $\text{src-lt } c (\text{hist-coord } (H ! j))$ 
    using c-le-j unfolding src-lt-def by simp
    from no-later j-lt c-lt-j j-le-f
    have  $\text{key-of } (\text{hist-event } (H ! j)) \neq k$  by blast
    with j-key show False by simp
  qed
qed
qed

have cand-last:
   $\text{filter } ?P [0..<\text{length } H] \neq []$ 

```

```

    ∧ last (filter ?P [0..<length H]) = i
  by (rule last-filter-upt-eq[OF i-lt P-i no-after-P])
  hence latest-eq: latest-src-event H f k = Some i
  unfolding latest-src-event-def by (simp add: Let-def)

  have Src b0 H f k =
    (case hist-event (H ! i) of
      Insert - v ⇒ Some v
    | Update - v ⇒ Some v
    | Delete -   ⇒ None)
  unfolding Src-def using latest-eq by simp
  thus ?thesis using event-i by simp
qed

end

```

```

theory Virtual-Cut-Core
  imports DBLog-Run-Core Dual-Write-Layer0.Virtual-Cut-State
begin

```

3 Certificate-free virtual-cut core (carrier-independent)

This theory collects the carrier-independent “pure” core that the Layer 2/3/4 substrate locales rest on. It sits below the certificate carrier: the substrate theories *DBLog-Run-Substrate-Layer2* and *DBLog-Cert-Substrate* import this core, not the certificate-side theories *Virtual-Cut / Layer4-Whole-Table*.

Two groups of material live here:

- *virtual-cut-state* — the source-side extensional-equality predicate imported from the shared Layer-0 entry. This theory no longer owns a duplicate core predicate; Layer 2, Layer 3, DBLog, and Dual-Write refer to the same HOL constant.
- the pure Layer 4 anchor / table-scope definitions and the outside-scope bridge lemmas. They reference no certificate accessor. The scoped-equality bridge *whole-table-equality-from-scoped-equality* is stated over the *raw* restricted equality (predicate-agnostic), so *both* the cert-side *Layer4-Whole-Table.whole-table-equality-from-claim-scope* and the *DBLog-Cert-Substrate* locale reuse it without coupling to either state predicate.

3.1 Virtual cut state (shared Layer-0 predicate)

virtual-cut-state is imported from the shared Layer-0 theory *Dual-Write-Layer0.Virtual-Cut-State*. It is the one source-side extensional-equality core that both Layer 2 (over runs) and the Layer 3 cert-side *virtual-cut* reduce to:

$$\text{virtual-cut-state } b0 \ \sigma \ K \ f \ H \longleftrightarrow \text{restrict } (\text{Apply } \sigma) \ K = \text{restrict } (\text{Src } b0 \ H \ f) \ K.$$

3.2 Anchor and table-scope definitions

definition *anchor-domain*

$$:: ('k \rightarrow 'v) \Rightarrow ('k, 'v) \text{ src-history} \Rightarrow \text{frontier} \Rightarrow 'k \text{ set}$$

where

$$\text{anchor-domain } b0 \ H \ b = \{k. \exists v. \text{Src } b0 \ H \ b \ k = \text{Some } v\}$$

definition *touched-between*

$$:: ('k, 'v) \text{ src-history} \Rightarrow \text{frontier} \Rightarrow \text{frontier} \Rightarrow 'k \text{ set}$$

where

$$\begin{aligned} \text{touched-between } H \ b \ f = \\ \{k. \exists c \ e. (c, e) \in \text{set } H \wedge b < c \wedge c \leq f \wedge \text{key-of } e = k\} \end{aligned}$$

definition *table-scope-between*

$$\begin{aligned} &:: ('k \rightarrow 'v) \Rightarrow ('k, 'v) \text{ src-history} \Rightarrow \text{frontier} \Rightarrow \text{frontier} \\ &\Rightarrow 'k \text{ set} \end{aligned}$$

where

$$\begin{aligned} \text{table-scope-between } b0 \ H \ b \ f = \\ \text{anchor-domain } b0 \ H \ b \cup \text{touched-between } H \ b \ f \end{aligned}$$

definition *anchor-complete-at-boundary*

$$\begin{aligned} &:: ('k \rightarrow 'v) \Rightarrow ('k, 'v) \text{ src-history} \Rightarrow \text{frontier} \Rightarrow 'k \text{ set} \\ &\Rightarrow \text{bool} \end{aligned}$$

where

$$\begin{aligned} \text{anchor-complete-at-boundary } b0 \ H \ b \ K \longleftrightarrow \\ \text{anchor-domain } b0 \ H \ b \subseteq K \end{aligned}$$

definition *all-post-anchor-changes-covered*

$$:: ('k, 'v) \text{ src-history} \Rightarrow \text{frontier} \Rightarrow \text{frontier} \Rightarrow 'k \text{ set} \Rightarrow \text{bool}$$

where

$$\begin{aligned} \text{all-post-anchor-changes-covered } H \ b \ f \ K \longleftrightarrow \\ \text{touched-between } H \ b \ f \subseteq K \end{aligned}$$

3.3 Replay-side no-unclaimed-key condition

definition *replay-keys* $:: ('k, 'v) \text{ replay-event list} \Rightarrow 'k \text{ set}$

where

$$\text{replay-keys } \sigma = \text{event-key } ' \text{ set } \sigma$$

definition *no-unclaimed-replay-keys*

$$:: ('k, 'v) \text{ replay-event list} \Rightarrow 'k \text{ set} \Rightarrow \text{bool}$$

where

$no-unclaimed-replay-keys\ \sigma\ K \longleftrightarrow replay-keys\ \sigma \subseteq K$

3.4 Outside-scope bridge lemmas

lemma *source-outside-table-scope-absent*:

assumes *b-le-f*: $b \leq f$

and *outside*: $k \notin table-scope-between\ b0\ H\ b\ f$

shows $Src\ b0\ H\ f\ k = None$

proof –

have *anchor-none*: $Src\ b0\ H\ b\ k = None$

using *outside*

unfolding *table-scope-between-def* *anchor-domain-def*

by (*cases Src b0 H b k*) *auto*

have *no-later*:

$\forall i < length\ H.$

$src-lt\ b\ (hist-coord\ (H\ !\ i))$

$\wedge\ src-le\ (hist-coord\ (H\ !\ i))\ f$

$\longrightarrow\ key-of\ (hist-event\ (H\ !\ i)) \neq k$

proof (*intro allI impI*)

fix *i*

assume *i-lt*: $i < length\ H$

assume *between*:

$src-lt\ b\ (hist-coord\ (H\ !\ i))$

$\wedge\ src-le\ (hist-coord\ (H\ !\ i))\ f$

show $key-of\ (hist-event\ (H\ !\ i)) \neq k$

proof

assume *key-eq*: $key-of\ (hist-event\ (H\ !\ i)) = k$

have *pair-in*:

$(hist-coord\ (H\ !\ i), hist-event\ (H\ !\ i)) \in set\ H$

using *nth-mem[OF i-lt]* **by** (*metis surjective-pairing*)

have $k \in touched-between\ H\ b\ f$

proof –

have $(hist-coord\ (H\ !\ i), hist-event\ (H\ !\ i)) \in set\ H$

$\wedge\ b < hist-coord\ (H\ !\ i)$

$\wedge\ hist-coord\ (H\ !\ i) \leq f$

$\wedge\ key-of\ (hist-event\ (H\ !\ i)) = k$

using *pair-in* *between* *key-eq*

by (*auto simp: src-lt-eq-less src-le-eq-less-eq*)

thus *?thesis*

unfolding *touched-between-def*

by *blast*

qed

hence $k \in table-scope-between\ b0\ H\ b\ f$

unfolding *table-scope-between-def* **by** *simp*

with *outside* **show** *False* **by** *contradiction*

qed

qed

```

have Src b0 H f k = Src b0 H b k
  by (rule src-eq-when-no-later-src-event-for-k)
    (use b-le-f no-later in ⟨auto simp: src-le-eq-less-eq⟩)
thus ?thesis
  using anchor-none by simp
qed

```

```

lemma apply-outside-replay-keys-none:
  assumes contained: replay-keys  $\sigma \subseteq K$ 
    and outside:  $k \notin K$ 
  shows Apply  $\sigma k = \text{None}$ 
proof -
  have no-key-events: filter ( $\lambda e. \text{event-key } e = k$ )  $\sigma = []$ 
  proof -
    have  $\forall e \in \text{set } \sigma. \text{event-key } e \neq k$ 
      using contained outside
      unfolding replay-keys-def
      by blast
    thus ?thesis by simp
  qed
show ?thesis
  using apply-latest-event-wins[of  $\sigma k$ ] no-key-events
  by simp
qed

```

```

lemma unrestricted-equality-from-scoped-equality:
  fixes lhs rhs :: 'k  $\rightarrow$  'v
  assumes scoped: restrict lhs K = restrict rhs K
    and lhs-outside:  $\bigwedge k. k \notin K \implies \text{lhs } k = \text{None}$ 
    and rhs-outside:  $\bigwedge k. k \notin K \implies \text{rhs } k = \text{None}$ 
  shows lhs = rhs
proof (rule ext)
  fix k
  show lhs k = rhs k
  proof (cases  $k \in K$ )
    case True
    with scoped show ?thesis
      by (metis restrict-def)
  next
    case False
    thus ?thesis
      by (simp add: lhs-outside rhs-outside)
  qed
qed

```

The scoped-equality bridge, stated over the *raw* restricted equality (predicate-agnostic). The cert-side / run-side callers each unfold their own state predicate (*Virtual-Cut.virtual-cut-state* resp. *virtual-cut-state*) to the

raw form before applying it.

```

lemma whole-table-equality-from-scoped-equality:
  assumes scoped:
     $\text{restrict } (\text{Apply } \sigma) K = \text{restrict } (\text{Src } b0 \ H \ f) K$ 
  and table-contained:
     $\text{table-scope-between } b0 \ H \ b \ f \subseteq K$ 
  and no-unclaimed:
    no-unclaimed-replay-keys  $\sigma \ K$ 
  and b-le-f:
     $b \leq f$ 
  shows  $\text{Apply } \sigma = \text{Src } b0 \ H \ f$ 
proof (rule unrestricted-equality-from-scoped-equality)
  show  $\text{restrict } (\text{Apply } \sigma) K = \text{restrict } (\text{Src } b0 \ H \ f) K$ 
    using scoped .
next
  fix  $k$ 
  assume outside:  $k \notin K$ 
  show  $\text{Apply } \sigma \ k = \text{None}$ 
    by (rule apply-outside-replay-keys-none[OF - outside])
      (use no-unclaimed in  $\langle \text{simp add: no-unclaimed-replay-keys-def} \rangle$ )
next
  fix  $k$ 
  assume outside:  $k \notin K$ 
  hence  $k \notin \text{table-scope-between } b0 \ H \ b \ f$ 
    using table-contained by blast
  thus  $\text{Src } b0 \ H \ f \ k = \text{None}$ 
    by (rule source-outside-table-scope-absent[OF b-le-f])
qed

end

```

```

theory DBLog-Run
  imports DBLog-Run-Core
begin

```

4 Layer 1: the DBLog run carrier, accessors, and wellformedness

Layer 1 abstract DBLog run substrate.

Layer 1 vocabulary:

- chunks: the paper's $(\text{'}k, \text{'}v)$ *chunk* carrier, modeled here concretely as the chunk index *nat*;
- $(\text{'}k, \text{'}v)$ *chunk-read-record*: Layer 0 self-contained chunk-read record carrying (*domain*, *read-coord*, *observation*); the paper places this at

Layer 0 inside the canonical-clean-prefix construction's input shape, and the Isabelle artifact lives in the imported theory *DBLog-Run-Core* (the carrier-independent Layer 0/1 core), consumed via the Layer 1 *clean-prefix-of* accessor's definitional equality;

- (*'k*, *'v*) *run*: Layer 1 run carrier (a concrete single-constructor **datatype**, below);
- Run accessors (paper "Run carrier and accessors", 7 accessors): *scope-of* / *frontier-of* / *clean-prefix-of* / *chunks* / *src-history-of* / *chunk-read-result* / *cdc-events-of*;
- Chunk-plan accessors (paper "Chunk-plan accessors"): *chunk-domain* / *owns* / *responsible-chunk* / *canonical-chunk-ownership-domain*;
- Chunk watermarks (paper "Watermarks and Chunk-Read Coordinates"): *chunk-lower-watermark* / *chunk-upper-watermark* / *chunk-read-coordinate*;
- Coverage predicates (paper "Coverage predicates"): *covers-ordinary-cdc* / *row-absence-meaningful*;
- *canonical-clean-prefix*: Layer 0 building block paper "Auxiliary Layer 0 definitions" / "Canonical clean prefix"; the paper places this at Layer 0 and the Isabelle artifact lives in the imported theory *DBLog-Run-Core*, with the Layer 1 *clean-prefix-of* accessor (below) its consumer;
- *wellformed-dblog-run*: 3-argument predicate *wellformed-dblog-run* *b0* *R* *H*; the body is the seven-clause conjunction *WF0* / *WF1* / *WF2* / *WF4* / *WF5* / *WF6* / *WF7*. There is no *WF3* clause: the numbering carries a deliberate gap recording a clause that the model no longer uses.

The run carrier and its accessors are defined here; *wellformed-dblog-run*, *covers-ordinary-cdc*, and *row-absence-meaningful* are real Isabelle definitions transliterating their paper bodies, and *clean-prefix-of* is determined by the canonical-clean-prefix construction in the imported theory *DBLog-Run-Core*.

owns is an **abbreviation** over *chunk-domain* — *owns* *R* *ch* *k* $\equiv k \in \text{chunk-domain } R \text{ } ch$ — following the paper's reader-facing definitional equivalence; this rules out any gap where two independent accessors could disagree on a non-wellformed run.

Run accessors use the *-of* suffix — *scope-of* *R*, *frontier-of* *R*, *clean-prefix-of* *R*, *src-history-of* *R*, *cdc-events-of* *R*; certificate accessors live in *Virtual-Cut* with bare names.

Naming note on the chunk-read record: the record-component selectors in Isabelle/HOL **record** blocks default to plain names (*domain* / *read-coord* /

observation); to avoid shadowing pre-existing Isabelle names (the standard *Domain* relation-domain projector lives in *Relation*), the selectors here are prefixed *crr*- (*crr-domain* / *crr-read-coord* / *crr-observation*). The paper-side names are unaffected.

4.1 Run carrier (concrete datatype)

The run carrier is a concrete single-constructor **datatype** with named selectors (prefixed *rr*-). It is a **datatype** rather than a **record**: a record introduces an extensible *-ext* type and makes the bare name a type synonym, whereas the datatype keeps *run* a clean two-parameter type constructor. The *dblog-run-substrate* locale's canonical **global-interpretation** interprets the locale at this datatype via the global accessors below, so every locale-proved fact is available for the global headline theorems.

Chunks are modeled as *nat* (the chunk index): the chunk-plan accessors and watermarks are function fields keyed by the chunk index. The raw chunk enumeration *rr-chunks-list* is a *nat list*; the public *chunks-list* accessor is *remdups* of it, so the two *chunks-list* facts below hold definitionally as lemmas.

The Layer 0 self-contained *chunk-read-record* (selectors *crr-domain* / *crr-read-coord* / *crr-observation*) is defined in theory *DBLog-Run-Core*, imported above: a carrier-independent building block shared by this global development and the *dblog-run-substrate* locale.

```
datatype ('k, 'v) run =
  Run (rr-scope: 'k set)
      (rr-frontier: frontier)
      (rr-chunks-list: nat list)
      (rr-src-history: ('k, 'v) src-history)
      (rr-cdc: (src-coord  $\times$  ('k, 'v) source-event) list)
      (rr-rresult: nat  $\Rightarrow$  'k  $\Rightarrow$  'v option option)
      (rr-dom: nat  $\Rightarrow$  'k set)
      (rr-resp: 'k  $\Rightarrow$  nat option)
      (rr-lwm: nat  $\Rightarrow$  src-coord)
      (rr-uwm: nat  $\Rightarrow$  src-coord)
      (rr-rcoord: nat  $\Rightarrow$  src-coord)
```

4.2 Run accessors (7 accessors)

Paper "Run carrier and accessors". Seven accessors project the run-level structure that the rest of Layer 1 + the Layer 2 theorem range over. Chunk-plan accessors and watermarks live in their own subsections below; the *clean-prefix-of* accessor's definitional equality (paper "Clean-prefix construction at Layer 1") is recorded at the *canonical-clean-prefix* subsection below.

```
definition scope-of :: ('k, 'v) run  $\Rightarrow$  'k set
where scope-of R = rr-scope R
```


definition *frontier-of* :: ('k, 'v) run $\Rightarrow$ frontier
where *frontier-of* R = rr-frontier R

definition *chunks* :: ('k, 'v) run $\Rightarrow$ nat set
where *chunks* R = set (rr-chunks-list R)

definition *src-history-of* :: ('k, 'v) run $\Rightarrow$ ('k, 'v) src-history
where *src-history-of* R = rr-src-history R

definition *chunk-read-result* :: ('k, 'v) run $\Rightarrow$ nat $\Rightarrow$ 'k $\Rightarrow$ 'v option option
where *chunk-read-result* R = rr-rresult R

definition *cdc-events-of* :: ('k, 'v) run $\Rightarrow$ (src-coord $\times$ ('k, 'v) source-event) list
where *cdc-events-of* R = rr-cdc R

The *clean-prefix-of* accessor is not declared here. It is defined as the Layer 1 definitional equality (paper "Clean-prefix construction at Layer 1") below, after the *chunks-list* accessor and the *canonical-clean-prefix* construction have been introduced.

4.3 Chunks enumeration accessor

The paper's clean-prefix construction at Layer 1 writes the chunk-reads input to *canonical-clean-prefix* as the list comprehension [*ChunkReadRecord*(*chunk-domain*(R, *ch*), *chunk-read-coordinate*(R, *ch*), *chunk-read-result*(R, *ch*)) | *ch* $\in$ *chunks*(R)]. This requires enumerating the chunk set *chunks*(R) as a list in a deterministic order. The run carrier therefore stores a raw enumeration field *rr-chunks-list*; the public accessor *chunks-list* is its *remdups*, *chunks* is its *set*, and the two properties tying the enumeration to *chunks* are proved just below as lemmas:

- *chunks-list-set*: *set* (*chunks-list* R) = *chunks* R;
- *chunks-list-distinct*: *distinct* (*chunks-list* R).

Two properties of this enumeration must be stated plainly, because they are easy to mistake for a harmless enumeration convenience.

The first is global finiteness. *chunks-list-set* equates *chunks* R with the set of a list, so *finite* (*chunks* R) holds for every run, not only for well-formed ones. This is a genuine modeling decision baked into the carrier — a run's chunk plan is stored as a finite list — and it is intended and sound for the DBLog domain: a run's chunk plan is bounded by construction, because chunking proceeds over a determinable key range (a maximum primary key, or an equivalent keyspace bound, is fixed before chunking begins as the termination criterion), so the chunk plan is finite for every run. The lemma *chunks-finite* just below records this consequence as a named,

machine-checked fact. The wellformed-run predicate's WF1 (f) clause separately requires *finite (scope-of R)* and finiteness of each chunk domain, but does not restate chunk-set finiteness — that fact holds globally and is established here.

The second is that the enumeration order is not canonical. The accessor returns the run's stored enumeration with duplicates removed, so two runs with the same chunk set may enumerate it in different orders. The Layer 2 source-side soundness theorem does not depend on the choice: at one source coordinate, cross-chunk-read Refresh events affect distinct keys (by the strict canonical partition), so the per-key replay semantics is invariant across enumeration choices.

Pragmatic alternative considered + rejected: Hilbert epsilon (*SOME xs. distinct xs $\wedge$ set xs = chunks R*) would avoid the stored enumeration field but would force every downstream proof to reason about non-determinism explicitly; a documented enumeration accessor over a stored field is cleaner.

definition *chunks-list* :: ('k, 'v) run $\Rightarrow$ nat list
where *chunks-list* R = remdups (rr-chunks-list R)

lemma *chunks-list-set*: set (chunks-list R) = chunks R
by (simp add: chunks-list-def chunks-def)

lemma *chunks-list-distinct*: distinct (chunks-list R)
by (simp add: chunks-list-def)

The global finiteness documented above, recorded as a named fact. Since *chunks-list-set* equates *chunks R* with *set (chunks-list R)*, the set of a list, *finite (chunks R)* holds for every run. Stating it as a lemma keeps the global modeling assumption visible and machine-checked rather than left implicit in the carrier definition.

lemma *chunks-finite*: finite (chunks R)
by (metis chunks-list-set List.finite-set)

4.4 Chunk-plan accessors

Paper "Chunk-plan accessors". Naming: *chunk-domain* is the keys a chunk owns; *responsible-chunk* is the inverse projecting from a key to its owning chunk; *canonical-chunk-ownership-domain* is the union of chunk domains over all chunks of R.

owns is an **abbreviation** over *chunk-domain* — *owns R ch k* $\equiv k \in \text{chunk-domain } R \text{ ch}$ — per the paper's reader-facing definitional equivalence (paper "Chunk-plan accessors": *owns(R, ch, k)* :: bool $\Leftrightarrow k \in \text{chunk-domain}(R, \text{ch})$). Making it an abbreviation rather than an independent accessor rules out any gap where *owns* and *chunk-domain* could disagree on a non-wellformed run.

definition *chunk-domain* :: ('k, 'v) run $\Rightarrow$ nat $\Rightarrow$ 'k set

where *chunk-domain* $R = rr-dom\ R$

definition *responsible-chunk* $:: ('k, 'v)\ run \Rightarrow 'k \Rightarrow nat\ option$
where *responsible-chunk* $R = rr-resp\ R$

abbreviation *owns*

$:: ('k, 'v)\ run \Rightarrow nat \Rightarrow 'k \Rightarrow bool$

where

owns $R\ ch\ k \equiv k \in chunk-domain\ R\ ch$

definition *canonical-chunk-ownership-domain*

$:: ('k, 'v)\ run \Rightarrow 'k\ set$

where

canonical-chunk-ownership-domain $R =$
 $(\bigcup ch \in chunks\ R.\ chunk-domain\ R\ ch)$

4.5 Chunk watermarks and read coordinates

Paper "Watermarks and Chunk-Read Coordinates". Three source-coordinate accessors per chunk: lower watermark, upper watermark, and the actual read coordinate. WF4 (clause body in the paper) requires *lower* $\leq_s rc\ read \leq_s rc\ upper$; WF5 additionally requires *read* $\leq_s rc\ frontier-of(R)$.

definition *chunk-lower-watermark* $:: ('k, 'v)\ run \Rightarrow nat \Rightarrow src-coord$
where *chunk-lower-watermark* $R = rr-lwm\ R$

definition *chunk-upper-watermark* $:: ('k, 'v)\ run \Rightarrow nat \Rightarrow src-coord$
where *chunk-upper-watermark* $R = rr-uwm\ R$

definition *chunk-read-coordinate* $:: ('k, 'v)\ run \Rightarrow nat \Rightarrow src-coord$
where *chunk-read-coordinate* $R = rr-rcoord\ R$

The Layer 0 canonical-clean-prefix construction — *canonical-clean-prefix* with its builders *chunk-read-refreshes* / *cdc-event-replays*, the *refresh-dominated* / *canonical-clean-prefix-normalized* normalized companion, and the Apply-equivalence theorem *apply-canonical-clean-prefix-normalized-eq* — is defined in theory *DBLog-Run-Core*, imported above. It references no run/chunk accessor, so it is shared verbatim by this global theory and the *dblog-run-substrate* locale.

4.6 Layer 1 clean-prefix definitional equality (*clean-prefix-of*)

Paper "Clean-prefix construction at Layer 1": Layer 1's *clean-prefix-of*(R) is not a fresh primitive — it is the Layer 0 *canonical-clean-prefix* building block instantiated for the run, with the chunk-read argument lifted out of an opaque "observed rows" assertion into per-chunk *chunk-read-record* values populated from the named accessors *chunk-domain*, *chunk-read-coordinate*,

and *chunk-read-result*, and with the CDC-side argument lifted out of "the source history's events" into the run-derived *cdc-events-of* accessor.

clean-prefix-of is a real **definition** via this Layer 1 definitional equality. The signature carries a *linorder* constraint on *'k* (propagated from *canonical-clean-prefix*); consumers (*wellformed-dblog-run*, the Layer 2 source-side soundness theorem, downstream Layer 3 certificate composition) inherit the constraint.

Consequences:

- the run carrier does not have to commit independently to a clean-prefix list shape; the wellformed-DBLog-run predicate's body constrains the run's *chunk-read-result*, *chunks*, *scope-of*, *src-history-of*, *cdc-events-of*, and *frontier-of* accessors, and *clean-prefix-of* is then determined by the construction;
- the Layer 2 source-side soundness theorem can reason about *clean-prefix-of*(*R*) as a closed-form expression in the run's accessors, without needing a separate "clean prefix is some interleaving of chunk reads and CDC events" wellformedness clause;
- the potential counterexample shape — a *Cdc-before-Refresh* order at distinct source coordinates that satisfies all set-membership-shaped WF clauses but breaks the Layer 2 conclusion — is structurally neutralized: under the construction body, *clean-prefix-of*(*R*) is sorted by source coordinate, so a low-coord *Cdc* cannot follow a high-coord *Refresh* for the same key.

definition *clean-prefix-of*

$:: ('k :: \text{linorder}, 'v) \text{run} \Rightarrow ('k, 'v) \text{replay-event list}$

where

clean-prefix-of *R* =
canonical-clean-prefix
 (map ($\lambda ch. \begin{cases} \text{crr-domain} & = \text{chunk-domain } R \text{ } ch, \\ \text{crr-read-coord} & = \text{chunk-read-coordinate } R \text{ } ch, \\ \text{crr-observation} & = \text{chunk-read-result } R \text{ } ch \end{cases}$)
 (*chunks-list* *R*))
 (*cdc-events-of* *R*)
 (*scope-of* *R*)
 (*frontier-of* *R*)

4.7 Coverage predicates

Paper "Coverage predicates". *covers-ordinary-cdc* unwraps *responsible-chunk*'s *option* via an existential and compares against *cdc-events-of* (the run's observed CDC events) rather than *src-history-of* (the true source history); *row-absence-meaningful* is parameterized against *Src b0 H*

(*chunk-read-coordinate* R ch) $k = \text{None}$ rather than against an unstated " k was never inserted or deleted" reading.

covers-ordinary-cdc is a real **definition** transliterating the paper "Coverage predicates" / "*covers-ordinary-cdc*(R, k, f)" body. The half-open interval (*chunk-read-coordinate* R $ch, f]$ becomes *src-lt* (*chunk-read-coordinate* R ch) (*hist-coord* p) $\wedge$ *src-le* (*hist-coord* p) f (strict on left, inclusive on right); the paper's "is reflected, with the same source coordinate, as a (c, e) pair in *cdc-events-of*(R)" becomes *src-history-of* $R ! i \in \text{set } (\text{cdc-events-of } R)$ (the (c, e) pair preserved by element identity).

row-absence-meaningful is a 5-argument predicate *row-absence-meaningful* $b0\ H\ R\ ch\ k$, a real **definition** capturing the formalizable structural + source-state implication: if the key is in the chunk's domain and the chunk read for it produced an absence-Refresh in the clean prefix, then the source state at the chunk-read coordinate is absent (paper "Coverage predicates" / "*row-absence-meaningful*"). WF6 (below) invokes it with the implication *chunk-read-result* $R\ ch\ k = \text{Some None} \longrightarrow \text{row-absence-meaningful } b0\ H\ R\ ch\ k$ on in-domain keys. That WF6 implication is logically auto-satisfied under WF6 main alone: WF6 main forces *Src* $b0\ H\ (\text{chunk-read-coordinate } R\ ch)\ k = \text{None}$, which discharges the predicate's body directly (machine-checked as *wf6-row-absence-redundant* in the redundancy section below). The predicate's role is structural rather than gating: the paper-side invocation exposes the deployment-level "no error / no partial result" obligation as reader-facing prose, and the formal-side predicate provides the type-level hook a deployment locale can state stronger assumptions against if a stronger operational obligation is to be expressed inside the formal model.

definition *covers-ordinary-cdc*

$:: ('k, 'v)\ \text{run} \Rightarrow 'k \Rightarrow \text{frontier} \Rightarrow \text{bool}$

where

$\text{covers-ordinary-cdc } R\ k\ f \longleftrightarrow$
 $(\exists\ ch.\ \text{responsible-chunk } R\ k = \text{Some } ch$
 $\wedge (\forall\ i.\ i < \text{length } (\text{src-history-of } R)$
 $\longrightarrow \text{src-lt } (\text{chunk-read-coordinate } R\ ch)$
 $\quad (\text{hist-coord } (\text{src-history-of } R ! i))$
 $\longrightarrow \text{src-le } (\text{hist-coord } (\text{src-history-of } R ! i))\ f$
 $\longrightarrow \text{key-of } (\text{hist-event } (\text{src-history-of } R ! i)) = k$
 $\longrightarrow \text{src-history-of } R ! i \in \text{set } (\text{cdc-events-of } R)))$

source-key-coord-slice (the per-key / per-coordinate filtering helper consumed by WF2 and the Layer 2 multiplicity lemmas) is defined in theory *DBLog-Run-Core*, imported above.

definition *row-absence-meaningful*

$:: ('k :: \text{linorder} \rightharpoonup 'v) \Rightarrow ('k, 'v)\ \text{src-history} \Rightarrow ('k, 'v)\ \text{run} \Rightarrow \text{nat} \Rightarrow 'k \Rightarrow \text{bool}$

where

$\text{row-absence-meaningful } b0\ H\ R\ ch\ k \longleftrightarrow$
 $(k \in \text{chunk-domain } R\ ch$
 $\wedge \text{Refresh } k\ \text{None } (\text{chunk-read-coordinate } R\ ch) \in \text{set } (\text{clean-prefix-of } R)$

$\longrightarrow \text{Src } b0 \ H \ (\text{chunk-read-coordinate } R \ ch) \ k = \text{None})$

4.8 The wellformed-DBLog-run predicate

Paper "The *wellformed-DBLog-run* predicate". The three-argument signature *wellformed-dblog-run* $b0 \ R \ H$ threads the source-side baseline $b0$ through the predicate body, so chunk-read observations are checked against $\text{Src } b0 \ H \ (\text{chunk-read-coordinate } R \ ch)$ (clause WF6). The 7-clause body is:

- WF0: source-history binding (*src-history-of* $R = H + \text{wellformed-src-history } H$);
- WF1: strict canonical chunk partition over scope + *responsible-chunk* coherence + finite-domain / finite-scope + non-empty-chunk-domain obligations (sub-clauses (a)...(g));
- WF2 (four labeled sub-conjuncts): ordinary-CDC coverage on every key in scope (paper *covers-ordinary-cdc*) + observed-CDC membership faithfulness against *src-history-of* + observed-CDC order-preserving sublist coherence on the in-scope / at-or-before-frontier slice + post-read same-key / same-coordinate multiplicity. The order-preserving sublist condition strengthens the membership-only faithfulness conjunct with source-position-order preservation; this is what rules out a same-coordinate CDC-order reversal, where two source events at one coordinate appear in *cdc-events-of* in the opposite order to *src-history-of* and break the Layer 2 conclusion. The post-read multiplicity conjunct requires, for an in-scope key, that every source coordinate strictly after its responsible chunk read and at-or-before the frontier carries the same multiset of matching records in *cdc-events-of* as in *src-history-of*, closing the duplicate-record gap in the Cdc case of the per-key replay-equality lemma;
- (no WF3 — the numbering carries a deliberate gap recording a clause the model no longer uses);
- WF4: chunk-read evidence (watermark bracketing + *chunk-read-result* totality on chunk domain + *Refresh*-event agreement with *chunk-read-result*);
- WF5: chunk-read coordinate at-or-before frontier;
- WF6: refresh correctness against $\text{Src } b0 \ H \ (\text{chunk-read-coordinate } R \ ch)$; and, when *chunk-read-result* $R \ ch \ k = \text{Some None}$ is observed for an in-domain key, *row-absence-meaningful* $b0 \ H \ R \ ch \ k$ holds (the implication is auto-satisfied under WF6 main alone — machine-checked as *wf6-row-absence-redundant*; see the Coverage predicates header above);

- WF7: clean-prefix CDC coherence — bidirectional coherence between $\text{cdc-events-of}(R)$ and Cdc events in $\text{clean-prefix-of}(R)$ on the in-scope / at-or-before-frontier filter.

$\text{wellformed-dblog-run}$ is a real **definition** transliterating the paper "wellformed-DBLog-run predicate" body.

Two of the clauses warrant a closer note.

WF7 (clean-prefix CDC coherence) makes explicit a fact that is otherwise only implicit in the paper's $\text{covers-ordinary-cdc}$ parenthetical "so that $\text{Cdc } c \ e$ appears in $\text{clean-prefix-of}(R)$ ". With WF7, the body enforces both directions — forward: $\text{cdc-events-of} \rightarrow \text{clean-prefix-of}$; reverse: $\text{clean-prefix-of} \rightarrow \text{cdc-events-of}$ — on the in-scope / at-or-before-frontier filter.

WF2's order-preservation conjunct is the order-preserving sublist condition: there exists a strictly increasing index function ι from indices of (the in-scope, at-or-before-frontier filter on $\text{cdc-events-of}(R)$) into indices of (the same filter on $\text{src-history-of}(R)$) with elements matching pairwise. This is equivalent to " $\text{cdc-events-of}(R)$ is an order-preserving sublist of $\text{src-history-of}(R)$ on that slice". A bare membership condition would permit the same-coordinate reversal $H = [(c, \text{Update } k \ v1), (c, \text{Update } k \ v2)]$ versus $\text{cdc-events-of}(R) = [(c, \text{Update } k \ v2), (c, \text{Update } k \ v1)]$, where set membership holds both ways but the source-position order is reversed, causing $\text{Apply } (\text{clean-prefix-of } R) \ k$ to disagree with $\text{Src } b0 \ H \ (\text{frontier-of } R) \ k$ on the in-scope key k . The membership condition is kept as a separate conjunct alongside the order-preserving one (it is logically subsumed, but stating it explicitly keeps the extraction proofs simple). The $\text{covers-ordinary-cdc}$ forward direction is independent and still requires coverage in the post-chunk-read interval.

definition $\text{wellformed-dblog-run}$

$:: ('k :: \text{linorder} \rightarrow 'v) \Rightarrow ('k, 'v) \text{ run} \Rightarrow ('k, 'v) \text{ src-history} \Rightarrow \text{bool}$

where

$\text{wellformed-dblog-run } b0 \ R \ H \longleftrightarrow$

— WF0: source-history binding

$\text{src-history-of } R = H \wedge \text{wellformed-src-history } H$

$\wedge$

— WF1 (a): $\exists!$ responsible chunk per in-scope key. Derivable from WF1 (b) +

(c) — $\text{wf1a-unique-owner-redundant}$ below.

$(\forall k \in \text{scope-of } R. \exists! ch. ch \in \text{chunks } R \wedge \text{owns } R \ ch \ k)$

$\wedge$

— WF1 (b): chunk domains are pairwise disjoint

$(\forall ch1 \in \text{chunks } R. \forall ch2 \in \text{chunks } R.$

$ch1 \neq ch2 \longrightarrow \text{chunk-domain } R \ ch1 \cap \text{chunk-domain } R \ ch2 = \{\})$

$\wedge$

- WF1 (c): canonical chunk-ownership domain equals scope
 $\text{canonical-chunk-ownership-domain } R = \text{scope-of } R$
- $\wedge$
- WF1 (d): *responsible-chunk* $\longleftrightarrow$ ownership predicate (in scope)
 $(\forall k \in \text{scope-of } R. \forall ch. \text{responsible-chunk } R k = \text{Some } ch \longleftrightarrow ch \in \text{chunks } R \wedge \text{owns } R ch k)$
- $\wedge$
- WF1 (e): out-of-scope keys have no responsible chunk
 $(\forall k. k \notin \text{scope-of } R \longrightarrow \text{responsible-chunk } R k = \text{None})$
- $\wedge$
- WF1 (f): scope and chunk domains are finite
 $\text{finite } (\text{scope-of } R)$
 $\wedge (\forall ch \in \text{chunks } R. \text{finite } (\text{chunk-domain } R ch))$
- $\wedge$
- WF1 (g): every chunk has a non-empty domain — no inert dead chunks.

With WF1 (a)-(f) this makes the chunks of R a strict partition of $\text{scope-of } R$ (a partition has non-empty parts). An empty-domain chunk would be inert — it owns no key and emits no Refresh events — so this clause is not needed for the headline theorems; it is included so the formal body matches the paper’s strict-partition wording.

- $(\forall ch \in \text{chunks } R. \text{chunk-domain } R ch \neq \{\})$
- $\wedge$
- WF2 (coverage): observed CDC reaches every in-scope key through the frontier (paper *covers-ordinary-cdc*)
 $(\forall k \in \text{scope-of } R. \text{covers-ordinary-cdc } R k (\text{frontier-of } R))$
- $\wedge$
- WF2 (faithfulness): observed CDC events that fall in scope and at-or-before frontier are real source events (membership condition). Logically subsumed by the order-preservation clause below (strict-monotonic indexing implies pairwise membership), but kept explicit as a separate conjunct so that the extraction proofs need not derive it from the embedding.
 $(\forall p \in \text{set } (\text{cdc-events-of } R). \text{key-of } (\text{hist-event } p) \in \text{scope-of } R \wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } R) \longrightarrow p \in \text{set } (\text{src-history-of } R))$
- $\wedge$
- WF2 (order-preserving sublist): on the in-scope / at-or-before-frontier slice, $\text{cdc-events-of}(R)$ is an order-preserving sublist of $\text{src-history-of}(R)$. The strict-monotonic indexing embedding strengthens the membership-only WF2 faithfulness clause above with source-position-order preservation, ruling out the same-coordinate CDC-order reversal. Consumed by the Layer 2 slice-subsequence supporting lemma (in theory *DBLog-Run-Substrate-Layer2*), which extracts this strict-monotonic embedding to show the per-key / per-coordinate CDC slice is an order-preserving subsequence of the source-history slice, and through it by the slice-equality lemma there.
 $(\exists \iota :: \text{nat} \Rightarrow \text{nat}. \text{strict-mono } \iota \wedge (\text{let } \text{in-slice} = (\lambda p. \text{key-of } (\text{hist-event } p) \in \text{scope-of } R \wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } R));$

$$\begin{aligned}
& \text{cdc-in-slice} = \text{filter in-slice } (\text{cdc-events-of } R); \\
& \text{hist-in-slice} = \text{filter in-slice } (\text{src-history-of } R) \\
& \text{in } \forall i. i < \text{length cdc-in-slice} \longrightarrow \\
& \quad \iota i < \text{length hist-in-slice} \\
& \quad \wedge \text{cdc-in-slice ! } i = \text{hist-in-slice ! } \iota i)) \\
& \wedge \\
& \text{— WF2 (post-read same-key/same-coordinate multiplicity): for an in-scope} \\
& \text{key } k, \text{ any source coordinate } c \text{ strictly after } k\text{'s responsible chunk read and at-} \\
& \text{or-before the run frontier has the same multiset of matching source records in} \\
& \text{cdc-events-of}(R) \text{ as in src-history-of}(R). \text{ Together with the order-preserving sublist} \\
& \text{clause, this closes the duplicate-record gap in the Cdc case of the per-key replay-} \\
& \text{equality lemma without requiring global source-history distinctness. Multiset equal-} \\
& \text{ity rather than set membership is essential: an [A, B, A] source slice versus an [A,} \\
& \text{B] CDC slice is rejected by multiplicity, while a same-multiset permutation such as} \\
& \text{[A, B, A] versus [A, A, B] is excluded instead by the order-preserving sublist clause} \\
& \text{above.} \\
& (\forall k \in \text{scope-of } R. \forall ch \ c. \\
& \quad \text{responsible-chunk } R \ k = \text{Some } ch \\
& \quad \wedge \text{src-lt } (\text{chunk-read-coordinate } R \ ch) \ c \\
& \quad \wedge \text{src-le } c \ (\text{frontier-of } R) \\
& \quad \longrightarrow \\
& \quad \text{mset } (\text{source-key-coord-slice } k \ c \ (\text{cdc-events-of } R)) \\
& \quad = \text{mset } (\text{source-key-coord-slice } k \ c \ (\text{src-history-of } R))) \\
& \wedge \\
& \text{— (no WF3 — the numbering carries a deliberate gap recording a clause the} \\
& \text{model no longer uses)} \\
& \text{— WF4: chunk-read evidence (watermark bracketing + chunk-read-result to-} \\
& \text{tality on domain + Refresh-event agreement with chunk-read-result on domain).} \\
& \text{The watermark-bracketing conjunct is not needed for the headline theorems; it is} \\
& \text{included so the formal body matches the paper's WF4 clause. The result/Refresh} \\
& \text{agreement biconditional is derivable from WF1 (b) + (f) and the clean-prefix-of} \\
& \text{construction — wf4-refresh-agreement-redundant below.} \\
& (\forall ch \in \text{chunks } R. \\
& \quad \text{src-le } (\text{chunk-lower-watermark } R \ ch) \ (\text{chunk-read-coordinate } R \ ch) \\
& \quad \wedge \text{src-le } (\text{chunk-read-coordinate } R \ ch) \ (\text{chunk-upper-watermark } R \ ch) \\
& \quad \wedge (\forall k \in \text{chunk-domain } R \ ch. \\
& \quad \quad \exists m. \text{chunk-read-result } R \ ch \ k = \text{Some } m) \\
& \quad \wedge (\forall k \in \text{chunk-domain } R \ ch. \forall m. \\
& \quad \quad \text{chunk-read-result } R \ ch \ k = \text{Some } m \\
& \quad \quad \longleftrightarrow \text{Refresh } k \ m \ (\text{chunk-read-coordinate } R \ ch) \\
& \quad \quad \in \text{set } (\text{clean-prefix-of } R))) \\
& \wedge \\
& \text{— WF5: chunk-read coordinate at-or-before frontier} \\
& (\forall ch \in \text{chunks } R. \\
& \quad \text{src-le } (\text{chunk-read-coordinate } R \ ch) \ (\text{frontier-of } R)) \\
& \wedge \\
& \text{— WF6 (main): refresh correctness against Src b0 H (chunk-read-coordinate } R \\
& \text{ch) k.} \\
& (\forall ch \in \text{chunks } R. \forall k \in \text{chunk-domain } R \ ch. \forall m.
\end{aligned}$$

$chunk-read-result\ R\ ch\ k = Some\ m$
 $\longrightarrow m = Src\ b0\ H\ (chunk-read-coordinate\ R\ ch)\ k)$
 $\wedge$
 — WF6 (row-absence meaningfulness): when $chunk-read-result\ R\ ch\ k = Some\ None$ is observed for an in-domain key, $row-absence-meaningful\ b0\ H\ R\ ch\ k$ holds. Auto-satisfied under WF6 main alone — *wf6-row-absence-redundant* below; carries the paper's WF6 row-absence content into the formal body.
 $(\forall\ ch \in chunks\ R. \forall\ k \in chunk-domain\ R\ ch.$
 $chunk-read-result\ R\ ch\ k = Some\ None$
 $\longrightarrow row-absence-meaningful\ b0\ H\ R\ ch\ k)$
 $\wedge$
 — WF7 (clean-prefix CDC coherence): forward direction — every observed CDC event whose key is in scope and whose coordinate is at-or-before frontier appears as a Cdc replay event in $clean-prefix-of(R)$.
 $(\forall\ p \in set\ (cdc-events-of\ R).$
 $key-of\ (hist-event\ p) \in scope-of\ R$
 $\wedge\ src-le\ (hist-coord\ p)\ (frontier-of\ R)$
 $\longrightarrow Cdc\ (hist-coord\ p)\ (hist-event\ p)$
 $\in set\ (clean-prefix-of\ R))$
 $\wedge$
 — WF7 (clean-prefix CDC coherence): reverse direction — every Cdc replay event in $clean-prefix-of(R)$ whose key is in scope and whose coordinate is at-or-before frontier is justified by the corresponding pair in $cdc-events-of(R)$.
 $(\forall\ e\ c. Cdc\ c\ e \in set\ (clean-prefix-of\ R)$
 $\wedge\ key-of\ e \in scope-of\ R$
 $\wedge\ src-le\ c\ (frontier-of\ R)$
 $\longrightarrow (c, e) \in set\ (cdc-events-of\ R))$

5 WF7 clean-prefix CDC coherence holds for every run

The WF7 clean-prefix CDC-coherence conjunct of *wellformed-dblog-run* is provably redundant — *wf7-clean-prefix-cdc-coherence-redundant* establishes it for *every* run, wellformed or not, because the *canonical-clean-prefix* construction emits its *Cdc* events exactly from the in-scope / at-or-before-frontier filter of *cdc-events-of*. The conjunct is retained in the predicate body for documentation: the formal body matches the paper's wellformedness clause list. The run-parametric meta-theorems live here alongside *wellformed-dblog-run* / *clean-prefix-of*.

lemma *in-set-cdc-event-replaysI*:
fixes $es :: (src-coord \times ('k, 'v)\ source-event)\ list$
assumes $(c, e) \in set\ es$
and $key-of\ e \in scope$
and $c \leq frontier$
shows $Cdc\ c\ e \in set\ (cdc-event-replays\ scope\ frontier\ es)$
using *assms(1)*
proof (*induction es*)

```

    case Nil
    then show ?case by simp
next
  case (Cons p ps)
  show ?case
  proof (cases p = (c, e))
    case True
    then show ?thesis
      using assms(2,3) by (simp add: cdc-event-replays-def)
  next
    case False
    with Cons.prem1 have (c, e) ∈ set ps by simp
    then have Cdc c e ∈ set (cdc-event-replays scope frontier ps)
      by (rule Cons.IH)
    then show ?thesis
      by (simp add: cdc-event-replays-def split: option.split)
  qed
qed

lemma wf7-forward-holds-for-all-runs:
  fixes R :: ('k :: linorder, 'v) run
  and c :: src-coord
  and e :: ('k, 'v) source-event
  assumes cdc-in: (c, e) ∈ set (cdc-events-of R)
  and in-scope: key-of e ∈ scope-of R
  and le-front: src-le c (frontier-of R)
  shows Cdc c e ∈ set (clean-prefix-of R)
proof -
  have c-le: c ≤ frontier-of R
    using le-front by (simp add: less-eq-src-coord-def)
  have Cdc c e ∈ set (cdc-event-replays (scope-of R) (frontier-of R)
    (cdc-events-of R))
    using cdc-in in-scope c-le by (rule in-set-cdc-event-replaysI)
  thus ?thesis
    by (auto simp: clean-prefix-of-def canonical-clean-prefix-def)
qed

lemma wf7-reverse-holds-for-all-runs:
  fixes R :: ('k :: linorder, 'v) run
  and c :: src-coord
  and e :: ('k, 'v) source-event
  assumes e-in: Cdc c e ∈ set (clean-prefix-of R)
  shows (c, e) ∈ set (cdc-events-of R)
    ∧ key-of e ∈ scope-of R
    ∧ src-le c (frontier-of R)
proof -
  let ?mkRec = λch. (| crr-domain = chunk-domain R ch,
    crr-read-coord = chunk-read-coordinate R ch,
    crr-observation = chunk-read-result R ch |)

```

— Cdc events cannot come from the *chunk-read-refreshes* branch, which emits only Refresh events.

have *not-refresh-branch*:
 $\forall ch. Cdc\ c\ e \notin set\ (chunk-read-refreshes\ (?mkRec\ ch))$
unfolding *chunk-read-refreshes-def* *List.map-filter-def*
by (*auto split: option.splits*)
have *split*:
 $Cdc\ c\ e \in set\ (cdc-event-replays\ (scope-of\ R)$
 $\quad (frontier-of\ R)$
 $\quad (cdc-events-of\ R))$
 $\vee (\exists\ ch \in set\ (chunks-list\ R).$
 $\quad Cdc\ c\ e \in set\ (chunk-read-refreshes\ (?mkRec\ ch)))$
using *e-in*
by (*auto simp: clean-prefix-of-def canonical-clean-prefix-def*)
from *split not-refresh-branch* **have** *in-cdc-replays*:
 $Cdc\ c\ e \in set\ (cdc-event-replays\ (scope-of\ R)$
 $\quad (frontier-of\ R)$
 $\quad (cdc-events-of\ R))$
by *blast*
from *in-cdc-replays* **show** *?thesis*
unfolding *cdc-event-replays-def* *List.map-filter-def*
by (*auto split: if-splits simp: less-eq-src-coord-def*)
qed

theorem *wf7-clean-prefix-cdc-coherence-redundant*:

fixes $R :: ('k :: linorder, 'v)\ run$
shows $(\forall\ p \in set\ (cdc-events-of\ R).$
 $\quad key-of\ (hist-event\ p) \in scope-of\ R$
 $\quad \wedge\ src-le\ (hist-coord\ p)\ (frontier-of\ R)$
 $\quad \longrightarrow\ Cdc\ (hist-coord\ p)\ (hist-event\ p)$
 $\quad \in set\ (clean-prefix-of\ R))$
 $\wedge (\forall\ e\ c. Cdc\ c\ e \in set\ (clean-prefix-of\ R)$
 $\quad \wedge\ key-of\ e \in scope-of\ R$
 $\quad \wedge\ src-le\ c\ (frontier-of\ R)$
 $\quad \longrightarrow\ (c, e) \in set\ (cdc-events-of\ R))$

proof

show $\forall\ p \in set\ (cdc-events-of\ R).$
 $\quad key-of\ (hist-event\ p) \in scope-of\ R$
 $\quad \wedge\ src-le\ (hist-coord\ p)\ (frontier-of\ R)$
 $\quad \longrightarrow\ Cdc\ (hist-coord\ p)\ (hist-event\ p)$
 $\quad \in set\ (clean-prefix-of\ R)$

proof (*intro ballI impI*)

fix p

assume *p-in*: $p \in set\ (cdc-events-of\ R)$

and *hyp*: $key-of\ (hist-event\ p) \in scope-of\ R$
 $\quad \wedge\ src-le\ (hist-coord\ p)\ (frontier-of\ R)$

have $(fst\ p, snd\ p) \in set\ (cdc-events-of\ R)$ **using** *p-in* **by** *simp*

moreover **have** $key-of\ (snd\ p) \in scope-of\ R$ **using** *hyp* **by** *simp*

moreover **have** $src-le\ (fst\ p)\ (frontier-of\ R)$ **using** *hyp* **by** *simp*

```

ultimately have Cdc (fst p) (snd p) ∈ set (clean-prefix-of R)
  by (rule wf7-forward-holds-for-all-runs)
thus Cdc (hist-coord p) (hist-event p) ∈ set (clean-prefix-of R)
  by simp
qed
next
show ∀ e c. Cdc c e ∈ set (clean-prefix-of R)
  ∧ key-of e ∈ scope-of R
  ∧ src-le c (frontier-of R)
  → (c, e) ∈ set (cdc-events-of R)
proof (intro allI impI)
  fix e c
  assume Cdc c e ∈ set (clean-prefix-of R)
    ∧ key-of e ∈ scope-of R
    ∧ src-le c (frontier-of R)
  from wf7-reverse-holds-for-all-runs[OF conjunct1[OF this]]
  show (c, e) ∈ set (cdc-events-of R) by simp
qed
qed

```

6 Further redundant conjuncts of the wellformed-run body

Three further conjuncts of *wellformed-dblog-run* are provably redundant, completing the redundancy documentation that *wf7-clean-prefix-cdc-coherence-redundant* begins. Each is retained in the predicate body so the formal clause list matches the paper's; the lemmas here replace comment-level derivability claims with machine-checked facts. *wf1a-unique-owner-redundant*: the WF1 (a) unique-responsible-chunk clause follows from WF1 (b) disjointness and WF1 (c) ownership-domain coverage. *wf4-refresh-agreement-redundant*: the WF4 result/Refresh agreement biconditional follows from WF1 (b) + WF1 (f) and the *clean-prefix-of* construction. *wf6-row-absence-redundant*: the WF6 row-absence conjunct follows from WF6 main alone. The *in-set-chunk-read-refreshes-iff* membership characterization is the Refresh-side sibling of *in-set-cdc-event-replaysI* above.

lemma *in-set-chunk-read-refreshes-aux*:

fixes $r :: ('k :: \text{linorder}, 'v) \text{ chunk-read-record}$

shows $\text{Refresh } k \ m \ c$

$\in \text{set } (\text{List.map-filter}$

$(\lambda k'. \text{case } \text{crr-observation } r \ k' \text{ of}$

$\text{Some } m' \Rightarrow \text{Some } (\text{Refresh } k' \ m' \ (\text{crr-read-coord } r))$

$| \text{None} \Rightarrow \text{None})$

$xs)$

$\longleftrightarrow k \in \text{set } xs$

$\wedge \text{crr-observation } r \ k = \text{Some } m$

$\wedge c = \text{crr-read-coord } r$

```

by (induction xs)
  (auto simp: List.map-filter-simps split: option.splits)

lemma in-set-chunk-read-refreshes-iff:
  fixes r :: ('k :: linorder, 'v) chunk-read-record
  shows Refresh k m c ∈ set (chunk-read-refreshes r)
    ⟷ k ∈ set (sorted-list-of-set (crr-domain r))
      ∧ crr-observation r k = Some m
      ∧ c = crr-read-coord r
  unfolding chunk-read-refreshes-def
  by (rule in-set-chunk-read-refreshes-aux)

lemma wf1a-unique-owner-redundant:
  fixes R :: ('k :: linorder, 'v) run
  assumes disj: ∀ ch1 ∈ chunks R. ∀ ch2 ∈ chunks R.
    ch1 ≠ ch2 ⟶ chunk-domain R ch1 ∩ chunk-domain R ch2 = {}
    and cov: canonical-chunk-ownership-domain R = scope-of R
  shows ∀ k ∈ scope-of R. ∃! ch. ch ∈ chunks R ∧ owns R ch k
proof
  fix k assume k ∈ scope-of R
  with cov have k ∈ (⋃ ch ∈ chunks R. chunk-domain R ch)
    by (simp add: canonical-chunk-ownership-domain-def)
  then obtain ch where ch-in: ch ∈ chunks R and k-dom: k ∈ chunk-domain R
    ch
  by blast
  show ∃! ch. ch ∈ chunks R ∧ owns R ch k
proof
  show ch ∈ chunks R ∧ owns R ch k using ch-in k-dom by simp
next
  fix ch' assume ch' ∈ chunks R ∧ owns R ch' k
  then have ch'-in: ch' ∈ chunks R and k-dom': k ∈ chunk-domain R ch' by
    auto
  show ch' = ch
proof (rule ccontr)
  assume ch' ≠ ch
  then have chunk-domain R ch' ∩ chunk-domain R ch = {}
    using disj ch'-in ch-in by blast
  with k-dom k-dom' show False by blast
qed
qed
qed

lemma wf4-refresh-agreement-redundant:
  fixes R :: ('k :: linorder, 'v) run
  assumes disj: ∀ ch1 ∈ chunks R. ∀ ch2 ∈ chunks R.
    ch1 ≠ ch2 ⟶ chunk-domain R ch1 ∩ chunk-domain R ch2 = {}
    and fin: ∀ ch ∈ chunks R. finite (chunk-domain R ch)
    and ch-in: ch ∈ chunks R
    and k-dom: k ∈ chunk-domain R ch

```

shows $\text{chunk-read-result } R \text{ ch } k = \text{Some } m$
 $\longleftrightarrow \text{Refresh } k \text{ m } (\text{chunk-read-coordinate } R \text{ ch}) \in \text{set } (\text{clean-prefix-of } R)$

proof

let $?mkRec = \lambda ch. (\mid \text{crr-domain} = \text{chunk-domain } R \text{ ch},$
 $\text{crr-read-coord} = \text{chunk-read-coordinate } R \text{ ch},$
 $\text{crr-observation} = \text{chunk-read-result } R \text{ ch } \mid)$

assume $\text{obs}: \text{chunk-read-result } R \text{ ch } k = \text{Some } m$
have $k\text{-enum}: k \in \text{set } (\text{sorted-list-of-set } (\text{chunk-domain } R \text{ ch}))$
using $\text{fin } ch\text{-in } k\text{-dom}$ **by** simp

have $\text{Refresh } k \text{ m } (\text{chunk-read-coordinate } R \text{ ch})$
 $\in \text{set } (\text{chunk-read-refreshes } (?mkRec \text{ ch}))$
using $k\text{-enum}$ obs **by** $(\text{simp add: in-set-chunk-read-refreshes-iff})$

moreover **have** $ch \in \text{set } (\text{chunks-list } R)$
using $ch\text{-in}$ **by** $(\text{simp add: chunks-list-set})$

ultimately show $\text{Refresh } k \text{ m } (\text{chunk-read-coordinate } R \text{ ch})$
 $\in \text{set } (\text{clean-prefix-of } R)$
by $(\text{auto simp: clean-prefix-of-def canonical-clean-prefix-def})$

next

let $?mkRec = \lambda ch. (\mid \text{crr-domain} = \text{chunk-domain } R \text{ ch},$
 $\text{crr-read-coord} = \text{chunk-read-coordinate } R \text{ ch},$
 $\text{crr-observation} = \text{chunk-read-result } R \text{ ch } \mid)$

assume $\text{mem}: \text{Refresh } k \text{ m } (\text{chunk-read-coordinate } R \text{ ch}) \in \text{set } (\text{clean-prefix-of } R)$

have $\text{not-cdc-branch}:$
 $\text{Refresh } k \text{ m } (\text{chunk-read-coordinate } R \text{ ch})$
 $\notin \text{set } (\text{cdc-event-replays } (\text{scope-of } R) (\text{frontier-of } R) (\text{cdc-events-of } R))$
unfolding $\text{cdc-event-replays-def}$ $\text{List.map-filter-def}$
by $(\text{auto split: if-splits})$

from $\text{mem not-cdc-branch}$ **have**
 $\exists ch' \in \text{set } (\text{chunks-list } R).$
 $\text{Refresh } k \text{ m } (\text{chunk-read-coordinate } R \text{ ch})$
 $\in \text{set } (\text{chunk-read-refreshes } (?mkRec \text{ ch}'))$
by $(\text{auto simp: clean-prefix-of-def canonical-clean-prefix-def})$

then obtain ch' **where** $ch'\text{-l}: ch' \in \text{set } (\text{chunks-list } R)$
and $r\text{-mem}: \text{Refresh } k \text{ m } (\text{chunk-read-coordinate } R \text{ ch})$
 $\in \text{set } (\text{chunk-read-refreshes } (?mkRec \text{ ch}'))$

by blast

have $ch'\text{-in}: ch' \in \text{chunks } R$
using $ch'\text{-l}$ **by** $(\text{simp add: chunks-list-set})$

from $r\text{-mem}$ **have** $k\text{-enum}': k \in \text{set } (\text{sorted-list-of-set } (\text{chunk-domain } R \text{ ch}'))$
and $\text{obs}': \text{chunk-read-result } R \text{ ch}' k = \text{Some } m$
by $(\text{auto simp: in-set-chunk-read-refreshes-iff})$

have $k\text{-dom}': k \in \text{chunk-domain } R \text{ ch}'$
using $k\text{-enum}'$ $\text{fin } ch'\text{-in}$ **by** simp

have $ch' = ch$

proof (rule ccontr)
assume $ch' \neq ch$
then have $\text{chunk-domain } R \text{ ch}' \cap \text{chunk-domain } R \text{ ch} = \{\}$
using $\text{disj } ch'\text{-in } ch\text{-in}$ **by** blast

```

    with  $k\text{-dom } k\text{-dom}'$  show False by blast
  qed
  with  $obs'$  show  $chunk\text{-read}\text{-result } R \text{ ch } k = \text{Some } m$  by simp
  qed

lemma wff6-row-absence-redundant:
  fixes  $R :: ('k :: \text{linorder}, 'v)$  run
  and  $b0 :: 'k \multimap 'v$ 
  and  $H :: ('k, 'v)$  src-history
  assumes main:  $\forall \text{ ch} \in \text{chunks } R. \forall k \in \text{chunk-domain } R \text{ ch}. \forall m. \\
    chunk\text{-read}\text{-result } R \text{ ch } k = \text{Some } m \\
    \longrightarrow m = \text{Src } b0 \ H \ (chunk\text{-read}\text{-coordinate } R \text{ ch}) \ k$ 
  and  $ch\text{-in}: ch \in \text{chunks } R$ 
  and  $k\text{-dom}: k \in \text{chunk-domain } R \text{ ch}$ 
  and  $obs: chunk\text{-read}\text{-result } R \text{ ch } k = \text{Some } None$ 
  shows row-absence-meaningful  $b0 \ H \ R \text{ ch } k$ 
proof -
  have  $None = \text{Src } b0 \ H \ (chunk\text{-read}\text{-coordinate } R \text{ ch}) \ k$ 
  using main[THEN bspec, OF ch-in, THEN bspec, OF k-dom,
    THEN spec, of None] obs
  by simp
  then have  $\text{Src } b0 \ H \ (chunk\text{-read}\text{-coordinate } R \text{ ch}) \ k = None$  by simp
  thus ?thesis by (simp add: row-absence-meaningful-def)
  qed

end

```

```

theory DBLog-Run-Substrate
  imports DBLog-Run
begin

```

7 Layer 1 run substrate as a locale (abstract carrier)

The Layer 1 run/chunk interface presented as a *locale* over abstract carrier type variables, together with a canonical concrete interpretation.

Theory *DBLog-Run* defines the run carrier as a concrete single-constructor **datatype** (chunks modeled as *nat*) with definitional accessors, and proves the two *chunks-list* facts as lemmas. This theory restates the same interface abstractly:

- the locale *dblog-run-substrate* fixes the twelve run/chunk accessors over plain type variables *'run* / *'chunk* (with *'k* for the key type), and assumes only the two *chunks-list* obligations — as locale assumptions;
- the carrier-independent “pure” building blocks (*canonical-clean-prefix*, *source-key-coord-slice*, the *chunk-read-record*, and the normalization

lemmas) are defined in the imported theory *DBLog-Run-Core*: they reference no accessor, so both this locale and the global theory *DBLog-Run* reuse them directly;

- the carrier-*dependent* Layer 1 definitions (*clean-prefix-of*, *covers-ordinary-cdc*, *row-absence-meaningful*, *canonical-chunk-ownership-domain*, and the seven-clause *wellformed-dblog-run*) are restated inside the locale with bodies identical to the global ones; the Layer 2 source-side soundness lemmas are proved in this locale context, in theory *DBLog-Run-Substrate-Layer2*. The restatement is deliberate: the public Layer 1 surface of *DBLog-Run* is self-contained (its definitions are stated directly over the concrete carrier, with no locale machinery, and are imported by this theory), so deriving the global constants *from* the locale — a **global-interpretation** with *defines* clauses — is not available here and would invert the layering. The two copies are kept honest by the pointwise *canonical.X = X* equalities proved at the end of this theory: any drift between the bodies breaks those lemmas, and with them the build.

The companion **global-interpretation** *canonical* instantiates the locale at the concrete public carrier (k, v) *run* through the global accessors. The chunk enumeration is realized as *remdups* $\circ$ *rr-chunks-list*, so the two locale assumptions hold definitionally for *every* carrier value — the interpretation discharges them with *simp*, introducing no axiom. It exports every locale-proved fact at the concrete carrier, which is how the global headline theorems are obtained from the locale proofs.

The locale-fixed names deliberately coincide with the global accessor and definition names; inside the locale context they shadow the global constants, and the concrete versions are recovered under the *canonical.* qualifier from the interpretation.

```

locale dblog-run-substrate =
  fixes scope-of          :: 'run  $\Rightarrow$  ('k::linorder) set
  and frontier-of        :: 'run  $\Rightarrow$  frontier
  and chunks              :: 'run  $\Rightarrow$  'chunk set
  and src-history-of      :: 'run  $\Rightarrow$  ('k, 'v) src-history
  and chunk-read-result   :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  'k  $\Rightarrow$  'v option option
  and cdc-events-of       :: 'run  $\Rightarrow$  (src-coord  $\times$  ('k, 'v) source-event) list
  and chunks-list         :: 'run  $\Rightarrow$  'chunk list
  and chunk-domain        :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  'k set
  and responsible-chunk   :: 'run  $\Rightarrow$  'k  $\Rightarrow$  'chunk option
  and chunk-lower-watermark :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  src-coord
  and chunk-upper-watermark :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  src-coord
  and chunk-read-coordinate :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  src-coord
  assumes chunks-list-set: set (chunks-list R) = chunks R
  and chunks-list-distinct: distinct (chunks-list R)
begin

```

owns as an abbreviation over the fixed *chunk-domain*, mirroring the global reader-facing definitional equivalence.

abbreviation *owns* :: 'run $\Rightarrow$ 'chunk $\Rightarrow$ 'k $\Rightarrow$ bool
where *owns* *R* *ch* *k* $\equiv$ *k* $\in$ *chunk-domain* *R* *ch*

Global chunk-set finiteness, as a derived fact of the locale.

lemma *chunks-finite*: *finite* (*chunks* *R*)
by (*metis* *chunks-list-set* *List.finite-set*)

definition *canonical-chunk-ownership-domain* :: 'run $\Rightarrow$ 'k set
where
canonical-chunk-ownership-domain *R* =
 $(\bigcup ch \in \text{chunks } R. \text{chunk-domain } R \text{ } ch)$

Layer 1 clean-prefix definitional equality: the run instance of the global *canonical-clean-prefix* building block.

definition *clean-prefix-of* :: 'run $\Rightarrow$ ('k, 'v) replay-event list
where
clean-prefix-of *R* =
canonical-clean-prefix
 $(\text{map } (\lambda ch. \langle \text{crr-domain} = \text{chunk-domain } R \text{ } ch,$
 $\text{crr-read-coord} = \text{chunk-read-coordinate } R \text{ } ch,$
 $\text{crr-observation} = \text{chunk-read-result } R \text{ } ch \rangle)$
 $(\text{chunks-list } R))$
 $(\text{cdc-events-of } R)$
 $(\text{scope-of } R)$
 $(\text{frontier-of } R)$

definition *covers-ordinary-cdc* :: 'run $\Rightarrow$ 'k $\Rightarrow$ frontier $\Rightarrow$ bool
where
covers-ordinary-cdc *R* *k* *f* $\longleftrightarrow$
 $(\exists \text{ } ch. \text{responsible-chunk } R \text{ } k = \text{Some } ch$
 $\wedge (\forall i. i < \text{length } (\text{src-history-of } R)$
 $\longrightarrow \text{src-lt } (\text{chunk-read-coordinate } R \text{ } ch)$
 $\quad (\text{hist-coord } (\text{src-history-of } R \text{ } i))$
 $\longrightarrow \text{src-le } (\text{hist-coord } (\text{src-history-of } R \text{ } i)) \text{ } f$
 $\longrightarrow \text{key-of } (\text{hist-event } (\text{src-history-of } R \text{ } i)) = k$
 $\longrightarrow \text{src-history-of } R \text{ } i \in \text{set } (\text{cdc-events-of } R)))$

definition *row-absence-meaningful*
:: ('k $\rightarrow$ 'v) $\Rightarrow$ ('k, 'v) *src-history* $\Rightarrow$ 'run $\Rightarrow$ 'chunk $\Rightarrow$ 'k $\Rightarrow$ bool
where
row-absence-meaningful *b0* *H* *R* *ch* *k* $\longleftrightarrow$
 $(k \in \text{chunk-domain } R \text{ } ch$
 $\wedge \text{Refresh } k \text{ None } (\text{chunk-read-coordinate } R \text{ } ch) \in \text{set } (\text{clean-prefix-of } R)$
 $\longrightarrow \text{Src } b0 \text{ } H \text{ } (\text{chunk-read-coordinate } R \text{ } ch) \text{ } k = \text{None})$

The seven-clause wellformed-DBLog-run predicate, body identical to the global *DBLog-Run.wellformed-dblog-run*.

definition *wellformed-dblog-run*

$:: ('k \rightarrow 'v) \Rightarrow 'run \Rightarrow ('k, 'v) \text{ src-history} \Rightarrow \text{bool}$

where

$\text{wellformed-dblog-run } b0 \ R \ H \longleftrightarrow$

— WF0: source-history binding

$\text{src-history-of } R = H \wedge \text{wellformed-src-history } H$

$\wedge$

— WF1 (a): $\exists!$ responsible chunk per in-scope key

$(\forall k \in \text{scope-of } R. \exists! ch. ch \in \text{chunks } R \wedge \text{owns } R \ ch \ k)$

$\wedge$

— WF1 (b): chunk domains are pairwise disjoint

$(\forall ch1 \in \text{chunks } R. \forall ch2 \in \text{chunks } R.$

$ch1 \neq ch2 \longrightarrow \text{chunk-domain } R \ ch1 \cap \text{chunk-domain } R \ ch2 = \{\})$

$\wedge$

— WF1 (c): canonical chunk-ownership domain equals scope

$\text{canonical-chunk-ownership-domain } R = \text{scope-of } R$

$\wedge$

— WF1 (d): *responsible-chunk* $\longleftrightarrow$ ownership predicate (in scope)

$(\forall k \in \text{scope-of } R. \forall ch.$

$\text{responsible-chunk } R \ k = \text{Some } ch$

$\longleftrightarrow ch \in \text{chunks } R \wedge \text{owns } R \ ch \ k)$

$\wedge$

— WF1 (e): out-of-scope keys have no responsible chunk

$(\forall k. k \notin \text{scope-of } R \longrightarrow \text{responsible-chunk } R \ k = \text{None})$

$\wedge$

— WF1 (f): scope and chunk domains are finite

$\text{finite } (\text{scope-of } R)$

$\wedge (\forall ch \in \text{chunks } R. \text{finite } (\text{chunk-domain } R \ ch))$

$\wedge$

— WF1 (g): every chunk has a non-empty domain

$(\forall ch \in \text{chunks } R. \text{chunk-domain } R \ ch \neq \{\})$

$\wedge$

— WF2 (coverage): observed CDC reaches every in-scope key through the frontier

$(\forall k \in \text{scope-of } R. \text{covers-ordinary-cdc } R \ k \ (\text{frontier-of } R))$

$\wedge$

— WF2 (faithfulness): observed CDC events in scope and at-or-before frontier are real source events

$(\forall p \in \text{set } (\text{cdc-events-of } R).$

$\text{key-of } (\text{hist-event } p) \in \text{scope-of } R$

$\wedge \text{src-le } (\text{hist-coord } p) \ (\text{frontier-of } R)$

$\longrightarrow p \in \text{set } (\text{src-history-of } R))$

$\wedge$

— WF2 (order-preserving sublist): on the in-scope / at-or-before-frontier slice, *cdc-events-of*(*R*) is an order-preserving sublist of *src-history-of*(*R*)

$(\exists \iota :: \text{nat} \Rightarrow \text{nat}. \text{strict-mono } \iota$

$\wedge (\text{let in-slice} = (\lambda p. \text{key-of } (\text{hist-event } p) \in \text{scope-of } R$

$\wedge \text{src-le } (\text{hist-coord } p) \ (\text{frontier-of } R));$

$\text{cdc-in-slice} = \text{filter in-slice } (\text{cdc-events-of } R);$

$$\begin{aligned}
& \text{hist-in-slice} = \text{filter in-slice } (\text{src-history-of } R) \\
& \text{in } \forall i. i < \text{length cdc-in-slice} \longrightarrow \\
& \quad \iota i < \text{length hist-in-slice} \\
& \quad \wedge \text{cdc-in-slice } ! i = \text{hist-in-slice } ! \iota i) \\
\wedge \\
& \text{— WF2 (post-read same-key/same-coordinate multiplicity)} \\
& (\forall k \in \text{scope-of } R. \forall ch\ c. \\
& \quad \text{responsible-chunk } R\ k = \text{Some } ch \\
& \quad \wedge \text{src-lt } (\text{chunk-read-coordinate } R\ ch)\ c \\
& \quad \wedge \text{src-le } c\ (\text{frontier-of } R) \\
& \longrightarrow \\
& \quad \text{mset } (\text{source-key-coord-slice } k\ c\ (\text{cdc-events-of } R)) \\
& \quad = \text{mset } (\text{source-key-coord-slice } k\ c\ (\text{src-history-of } R))) \\
\wedge \\
& \text{— (no WF3 — the numbering carries a deliberate gap)} \\
& \text{— WF4: chunk-read evidence (watermark bracketing + chunk-read-result totality on domain + Refresh-event agreement with chunk-read-result on domain)} \\
& (\forall ch \in \text{chunks } R. \\
& \quad \text{src-le } (\text{chunk-lower-watermark } R\ ch)\ (\text{chunk-read-coordinate } R\ ch) \\
& \quad \wedge \text{src-le } (\text{chunk-read-coordinate } R\ ch)\ (\text{chunk-upper-watermark } R\ ch) \\
& \quad \wedge (\forall k \in \text{chunk-domain } R\ ch. \\
& \quad \quad \exists m. \text{chunk-read-result } R\ ch\ k = \text{Some } m) \\
& \quad \wedge (\forall k \in \text{chunk-domain } R\ ch. \forall m. \\
& \quad \quad \text{chunk-read-result } R\ ch\ k = \text{Some } m \\
& \quad \quad \longleftrightarrow \text{Refresh } k\ m\ (\text{chunk-read-coordinate } R\ ch) \\
& \quad \quad \in \text{set } (\text{clean-prefix-of } R))) \\
\wedge \\
& \text{— WF5: chunk-read coordinate at-or-before frontier} \\
& (\forall ch \in \text{chunks } R. \\
& \quad \text{src-le } (\text{chunk-read-coordinate } R\ ch)\ (\text{frontier-of } R)) \\
\wedge \\
& \text{— WF6 (main): refresh correctness against Src b0 H (chunk-read-coordinate R ch) k.} \\
& (\forall ch \in \text{chunks } R. \forall k \in \text{chunk-domain } R\ ch. \forall m. \\
& \quad \text{chunk-read-result } R\ ch\ k = \text{Some } m \\
& \quad \longrightarrow m = \text{Src } b0\ H\ (\text{chunk-read-coordinate } R\ ch)\ k) \\
\wedge \\
& \text{— WF6 (row-absence meaningfulness)} \\
& (\forall ch \in \text{chunks } R. \forall k \in \text{chunk-domain } R\ ch. \\
& \quad \text{chunk-read-result } R\ ch\ k = \text{Some None} \\
& \quad \longrightarrow \text{row-absence-meaningful } b0\ H\ R\ ch\ k) \\
\wedge \\
& \text{— WF7 (clean-prefix CDC coherence): forward direction} \\
& (\forall p \in \text{set } (\text{cdc-events-of } R). \\
& \quad \text{key-of } (\text{hist-event } p) \in \text{scope-of } R \\
& \quad \wedge \text{src-le } (\text{hist-coord } p)\ (\text{frontier-of } R) \\
& \quad \longrightarrow \text{Cdc } (\text{hist-coord } p)\ (\text{hist-event } p) \\
& \quad \quad \in \text{set } (\text{clean-prefix-of } R)) \\
\wedge
\end{aligned}$$

— WF7 (clean-prefix CDC coherence): reverse direction
 $(\forall e \ c. \ Cdc \ c \ e \in set \ (clean\text{-}prefix\text{-}of \ R)$
 $\wedge \ key\text{-}of \ e \in scope\text{-}of \ R$
 $\wedge \ src\text{-}le \ c \ (frontier\text{-}of \ R)$
 $\longrightarrow (c, e) \in set \ (cdc\text{-}events\text{-}of \ R))$

end

7.1 Canonical concrete interpretation (at the public *run* datatype)

The locale is interpreted at the concrete *run* datatype (defined in *DBLog-Run*) through the *global accessors* *scope-of* / *frontier-of* / ... / *chunk-read-coordinate*. The two *chunks-list* obligations are exactly the *DBLog-Run* lemmas *chunks-list-set* / *chunks-list-distinct*, so the interpretation introduces no axiom. Because the locale parameters *are* the global accessors, every locale-defined constant coincides with its global *DBLog-Run* namesake; the *canonical.X = X* unfold lemmas below expose that coincidence to the public headline theorem *wellformed-run-implies-virtual-cut* in *Virtual-Cut*.

global-interpretation *canonical: dblog-run-substrate*
scope-of frontier-of chunks src-history-of chunk-read-result
cdc-events-of chunks-list chunk-domain responsible-chunk
chunk-lower-watermark chunk-upper-watermark chunk-read-coordinate
by *unfold-locales (simp-all add: chunks-list-set chunks-list-distinct)*

lemma *canonical-canonical-chunk-ownership-domain-eq:*
canonical.canonical-chunk-ownership-domain = canonical-chunk-ownership-domain
by *(rule ext)*
(simp add: canonical.canonical-chunk-ownership-domain-def
canonical-chunk-ownership-domain-def)

lemma *canonical-clean-prefix-of-eq:*
canonical.clean-prefix-of = clean-prefix-of
by *(rule ext)*
(simp add: canonical.clean-prefix-of-def clean-prefix-of-def)

lemma *canonical-covers-ordinary-cdc-eq:*
canonical.covers-ordinary-cdc = covers-ordinary-cdc
by *(rule ext)+*
(simp add: canonical.covers-ordinary-cdc-def covers-ordinary-cdc-def)

lemma *canonical-row-absence-meaningful-eq:*
canonical.row-absence-meaningful = row-absence-meaningful
by *(rule ext)+*
(simp add: canonical.row-absence-meaningful-def row-absence-meaningful-def
canonical-clean-prefix-of-eq)

```

lemma canonical-wellformed-dblog-run-eq:
  canonical.wellformed-dblog-run = wellformed-dblog-run
by (rule ext)+
  (simp add: canonical.wellformed-dblog-run-def wellformed-dblog-run-def
    canonical-clean-prefix-of-eq canonical-covers-ordinary-cdc-eq
    canonical-canonical-chunk-ownership-domain-eq
    canonical-row-absence-meaningful-eq)

end

```

```

theory DBLog-Run-Substrate-Layer2
  imports DBLog-Run-Substrate Virtual-Cut-Core
begin

```

8 Layer 2 source-side soundness in the run substrate locale

The carrier-dependent Layer 2 supporting clean-prefix lemmas, the per-key replay-equality lemma, and the run-side soundness theorem *wellformed-run-implies-virtual-cut* are proved inside the *dblog-run-substrate* locale, over the locale’s fixed run/chunk accessors and its two *chunks-list* assumptions.

Every proof step refers to the locale definitions (*clean-prefix-of*, *covers-ordinary-cdc*, *wellformed-dblog-run*, ...) and the locale assumptions, or to the carrier-independent “pure” helpers (the *filter-take-nth* / *subseq** / *last-filter** list lemmas, *apply-at-latest-k-event*, *src-eq-when-no-later-src-event-for-k*, *src-frontier-eq-effect-of-last-key-coord-slice*, *wellformed-src-history-coord-mono*) that live in *DBLog-Run-Core* and are reused directly via this theory’s import. The *Virtual-Cut-Core* import supplies the carrier-independent state predicate *virtual-cut-state* for the concluding theorem.

Under the canonical **global-interpretation** *canonical* (in *DBLog-Run-Substrate*) the locale results specialize to the concrete $(\text{'}k, \text{'}v)$ *run* carrier; the public headline theorem *Virtual-Cut.wellformed-run-implies-virtual-cut* is the canonical specialization of the locale theorem at the end of this theory, rephrased through the public state predicate.

```

context dblog-run-substrate
begin

```

8.1 Refresh-coordinate match and clean-prefix coordinate facts

```

theorem refresh-coordinate-match:
  fixes b0 ::  $\text{'}k$  :: linorder  $\rightarrow$   $\text{'}v$  and R ::  $\text{'}run$ 

```

and $H :: ('k, 'v) \text{ src-history}$ **and** $ch :: 'chunk$
and $k :: 'k$ **and** $m :: 'v \text{ option}$
assumes $wf: \text{wellformed-dblog-run } b0 \ R \ H$
assumes $ch\text{-in}: ch \in \text{chunks } R$
assumes $k\text{-in}: k \in \text{chunk-domain } R \ ch$
assumes $obs: \text{chunk-read-result } R \ ch \ k = \text{Some } m$
shows $m = \text{Src } b0 \ H \ (\text{chunk-read-coordinate } R \ ch) \ k$
using $assms$ **unfolding** $\text{wellformed-dblog-run-def}$ **by** $blast$

lemma $\text{clean-prefix-of-order-respects-coordinate}$:

fixes $b0 :: 'k :: \text{linorder} \rightarrow 'v$
and $H :: ('k, 'v) \text{ src-history}$
and $R :: 'run$
and $i \ j :: \text{nat}$
assumes $\text{wellformed-dblog-run } b0 \ R \ H$
and $i < j$
and $j < \text{length } (\text{clean-prefix-of } R)$
shows $\text{src-le } (\text{cdc-coord } (\text{clean-prefix-of } R \ ! \ i))$
 $(\text{cdc-coord } (\text{clean-prefix-of } R \ ! \ j))$

proof –

have $\text{sorted-cp}: \text{sorted } (\text{map } \text{cdc-coord } (\text{clean-prefix-of } R))$
unfolding $\text{clean-prefix-of-def}$ $\text{canonical-clean-prefix-def}$
by $(\text{rule sorted-sort-key})$
from $assms(2)$ **have** $\text{le-ij}: i \leq j$ **by** simp
from $assms(3)$ **have** $j\text{-bound}$:
 $j < \text{length } (\text{map } \text{cdc-coord } (\text{clean-prefix-of } R))$ **by** simp
from $\text{sorted-nth-mono}[OF \ \text{sorted-cp} \ \text{le-ij} \ j\text{-bound}]$
have $\text{map } \text{cdc-coord } (\text{clean-prefix-of } R) \ ! \ i$
 $\leq \text{map } \text{cdc-coord } (\text{clean-prefix-of } R) \ ! \ j$.
hence $\text{cdc-coord } (\text{clean-prefix-of } R \ ! \ i)$
 $\leq \text{cdc-coord } (\text{clean-prefix-of } R \ ! \ j)$
using $assms(2,3)$ **by** $(\text{simp add: nth-map})$
thus $?thesis$ **by** $(\text{simp add: less-eq-src-coord-def})$

qed

lemma $\text{clean-prefix-of-no-future-in-scope-events}$:

fixes $b0 :: 'k :: \text{linorder} \rightarrow 'v$
and $H :: ('k, 'v) \text{ src-history}$
and $R :: 'run$
and $e :: ('k, 'v) \text{ replay-event}$
assumes $wf: \text{wellformed-dblog-run } b0 \ R \ H$
and $e\text{-in}: e \in \text{set } (\text{clean-prefix-of } R)$
shows $\text{src-le } (\text{cdc-coord } e) \ (\text{frontier-of } R)$

proof –

let $?mkRec = \lambda ch. \ \langle \text{crr-domain} \quad = \text{chunk-domain } R \ ch,$
 $\text{crr-read-coord} = \text{chunk-read-coordinate } R \ ch,$
 $\text{crr-observation} = \text{chunk-read-result } R \ ch \ \rangle$

have *split*: $e \in \text{set } (\text{cdc-event-replays } (\text{scope-of } R) (\text{frontier-of } R) (\text{cdc-events-of } R))$
 $\vee (\exists \text{ ch} \in \text{set } (\text{chunks-list } R). \text{ e} \in \text{set } (\text{chunk-read-refreshes } (?mkRec \text{ ch})))$
using *e-in*
by (*auto simp: clean-prefix-of-def canonical-clean-prefix-def*)
from *split* **show** *?thesis*
proof
assume $e \in \text{set } (\text{cdc-event-replays } (\text{scope-of } R) (\text{frontier-of } R) (\text{cdc-events-of } R))$
— Case A: *e* is a Cdc replay event whose coordinate is $\leq$ frontier by the *cdc-event-replays* filter. The *less-eq-src-coord-def* unfolding bridges $\leq$ on *src-coord* to *src-le*.
then show *?thesis*
unfolding *cdc-event-replays-def List.map-filter-def*
by (*auto split: if-splits simp: less-eq-src-coord-def*)
next
assume $\exists \text{ ch} \in \text{set } (\text{chunks-list } R). \text{ e} \in \text{set } (\text{chunk-read-refreshes } (?mkRec \text{ ch}))$
then obtain *ch* **where**
ch-in: $\text{ch} \in \text{set } (\text{chunks-list } R)$
and *e-in-refs*: $\text{e} \in \text{set } (\text{chunk-read-refreshes } (?mkRec \text{ ch}))$
by *blast*
— Case B: *e* is a Refresh event whose coordinate equals *chunk-read-coordinate* *R ch* (by the *chunk-read-refreshes* construction), which is $\leq$ frontier by WF5.
from *ch-in* **have** *ch-in-chunks*: $\text{ch} \in \text{chunks } R$
using *chunks-list-set* **by** *blast*
from *e-in-refs* **have** *e-coord*:
 $\text{cdc-coord } e = \text{chunk-read-coordinate } R \text{ ch}$
unfolding *chunk-read-refreshes-def List.map-filter-def*
by (*auto split: option.splits*)
from *wf* **have** *wf5*:
 $\forall \text{ ch} \in \text{chunks } R. \text{ src-le } (\text{chunk-read-coordinate } R \text{ ch}) (\text{frontier-of } R)$
unfolding *wellformed-dblog-run-def* **by** *blast*
from *wf5 ch-in-chunks*
have *src-le* $(\text{chunk-read-coordinate } R \text{ ch}) (\text{frontier-of } R)$ **by** *blast*
with *e-coord* **show** *?thesis* **by** *simp*
qed
qed

8.2 Provenance of clean-prefix events: responsible-chunk refreshes and observed CDC

lemma *clean-prefix-of-refresh-generated-by-responsible-chunk*:
fixes *b0* :: '*k* :: linorder $\rightarrow$ '*v*
and *H* :: ('*k*, '*v*) *src-history*
and *R* :: '*run*
and *k* :: '*k*

and $m :: 'v \text{ option}$
and $c :: \text{src-coord}$
assumes $wf: \text{wellformed-dblog-run } b0 \ R \ H$
and $e\text{-in}: \text{Refresh } k \ m \ c \in \text{set } (\text{clean-prefix-of } R)$
shows $k \in \text{scope-of } R$
 $\wedge (\exists \ ch. \text{responsible-chunk } R \ k = \text{Some } ch$
 $\wedge \text{chunk-read-coordinate } R \ ch = c$
 $\wedge \text{chunk-read-result } R \ ch \ k = \text{Some } m)$

proof –

let $?mkRec = \lambda ch. (\mid \text{crr-domain} \quad = \text{chunk-domain } R \ ch,$
 $\text{crr-read-coord} = \text{chunk-read-coordinate } R \ ch,$
 $\text{crr-observation} = \text{chunk-read-result } R \ ch \mid)$
— Refresh events cannot come from the *cdc-event-replays* branch (which produces only Cdc events).
have *not-cdc*:
 $\text{Refresh } k \ m \ c \notin \text{set } (\text{cdc-event-replays } (\text{scope-of } R)$
 $\quad (\text{frontier-of } R)$
 $\quad (\text{cdc-events-of } R))$
unfolding *cdc-event-replays-def* *List.map-filter-def*
by *auto*
have *split*:
 $\text{Refresh } k \ m \ c \in \text{set } (\text{cdc-event-replays } (\text{scope-of } R)$
 $\quad (\text{frontier-of } R)$
 $\quad (\text{cdc-events-of } R))$
 $\vee (\exists \ ch \in \text{set } (\text{chunks-list } R).$
 $\quad \text{Refresh } k \ m \ c \in \text{set } (\text{chunk-read-refreshes } (?mkRec \ ch)))$
using *e-in*
by (*auto simp: clean-prefix-of-def canonical-clean-prefix-def*)
from *split not-cdc* **obtain** *ch* **where**
 $ch\text{-in}: ch \in \text{set } (\text{chunks-list } R)$ **and**
 $\text{refresh-in}: \text{Refresh } k \ m \ c \in \text{set } (\text{chunk-read-refreshes } (?mkRec \ ch))$
by *blast*
from *ch-in* **have** *ch-in-chunks*: $ch \in \text{chunks } R$
using *chunks-list-set* **by** *blast*
— Extract the conjuncts the downstream steps need: WF1 (c), WF1 (d), and WF1 (f)’s chunk-domain finiteness. *blast* on the individual conjuncts can time out because of the WF2 *strict-mono* existential conjunct’s complexity in the unfolded body; a single simp-based unfolding + elimination is faster.
note $wf\text{-body} = wf[\text{unfolded wellformed-dblog-run-def}]$
from $wf\text{-body}$ **have** $wf1c: \text{canonical-chunk-ownership-domain } R = \text{scope-of } R$
by (*elim conjE*)
from $wf\text{-body}$ **have** $wf1d$:
 $\forall k \in \text{scope-of } R. \forall ch.$
 $\text{responsible-chunk } R \ k = \text{Some } ch$
 $\longleftrightarrow ch \in \text{chunks } R \wedge \text{owns } R \ ch \ k$
by (*elim conjE*)
from $wf\text{-body}$ **have** $wf1f\text{-chunks}$:
 $\forall ch \in \text{chunks } R. \text{finite } (\text{chunk-domain } R \ ch)$
by (*elim conjE*)

from *wf1f-chunks ch-in-chunks* **have** *fin-dom: finite (chunk-domain R ch)*
by *blast*
 — Pattern-match the Refresh against the *chunk-read-refreshes* emission to extract $k \in \text{chunk-domain } R \text{ ch}$, $\text{chunk-read-result } R \text{ ch } k = \text{Some } m$, and $c = \text{chunk-read-coordinate } R \text{ ch}$.
from *refresh-in fin-dom*
have *k-props:*
 $k \in \text{chunk-domain } R \text{ ch}$
 $\wedge \text{chunk-read-result } R \text{ ch } k = \text{Some } m$
 $\wedge c = \text{chunk-read-coordinate } R \text{ ch}$
unfolding *chunk-read-refreshes-def List.map-filter-def*
by *(auto split: option.splits)*
hence *k-in-dom: k ∈ chunk-domain R ch*
and *crr-res: chunk-read-result R ch k = Some m*
and *c-eq: c = chunk-read-coordinate R ch*
by *auto*
 — Establish $k \in \text{scope-of } R$ via $\text{WF1 } (c) + \text{chunk-domain} \subseteq \text{canonical-chunk-ownership-domain}$.
from *ch-in-chunks k-in-dom*
have $k \in \text{canonical-chunk-ownership-domain } R$
unfolding *canonical-chunk-ownership-domain-def* **by** *blast*
with *wf1c* **have** *k-in-scope: k ∈ scope-of R* **by** *blast*
 — Establish *responsible-chunk* $R \text{ k} = \text{Some } ch$ via $\text{WF1 } (d)$.
from *wf1d k-in-scope ch-in-chunks k-in-dom*
have *rc: responsible-chunk R k = Some ch* **by** *blast*
from *k-in-scope rc c-eq crr-res* **show** *?thesis* **by** *blast*
qed

lemma *clean-prefix-of-in-scope-responsible-chunk-refresh-exists:*

fixes $b0 :: 'k :: \text{linorder} \rightarrow 'v$
and $H :: ('k, 'v) \text{src-history}$
and $R :: 'run$
and $k :: 'k$

assumes *wf: wellformed-dblog-run b0 R H*

and *k-in-scope: k ∈ scope-of R*

shows $\exists m \ c. \text{Refresh } k \ m \ c \in \text{set } (\text{clean-prefix-of } R)$

proof —

note *wf-body = wf[unfolded wellformed-dblog-run-def]*

from *wf-body* **have** *wf1a:*

$\forall k \in \text{scope-of } R. \exists! ch. ch \in \text{chunks } R \wedge \text{owns } R \text{ ch } k$

by *(elim conjE)*

from *wf-body* **have** *wf4:*

$\forall ch \in \text{chunks } R.$

$\text{src-le } (\text{chunk-lower-watermark } R \text{ ch}) (\text{chunk-read-coordinate } R \text{ ch})$

$\wedge \text{src-le } (\text{chunk-read-coordinate } R \text{ ch}) (\text{chunk-upper-watermark } R \text{ ch})$

$\wedge (\forall k \in \text{chunk-domain } R \text{ ch}.$

$\exists m. \text{chunk-read-result } R \text{ ch } k = \text{Some } m)$

$\wedge (\forall k \in \text{chunk-domain } R \text{ ch}. \forall m.$

$chunk-read-result\ R\ ch\ k = Some\ m$
 $\longleftrightarrow Refresh\ k\ m\ (chunk-read-coordinate\ R\ ch)$
 $\in set\ (clean-prefix-of\ R)$
by (*elim conjE*)
 — WF1 (a) extracts the responsible chunk.
from *wf1a k-in-scope* **obtain** *ch* **where**
 ch-in-chunks: $ch \in chunks\ R$ **and**
 k-in-dom: $k \in chunk-domain\ R\ ch$
by *blast*
 — WF4 totality on *chunk-domain* produces the observation.
from *wf4 ch-in-chunks k-in-dom* **obtain** *m* **where**
 crr-res: $chunk-read-result\ R\ ch\ k = Some\ m$
by *blast*
 — WF4 iff (forward) lifts the observation into *clean-prefix-of*.
from *wf4 ch-in-chunks k-in-dom crr-res*
have $Refresh\ k\ m\ (chunk-read-coordinate\ R\ ch)$
 $\in set\ (clean-prefix-of\ R)$
by *blast*
thus *?thesis* **by** *blast*
qed

lemma *clean-prefix-of-cdc-generated-from-observed-cdc*:

fixes *b0* :: '*k* :: *linorder* $\rightarrow$ '*v*
and *H* :: ('*k*, '*v*) *src-history*
and *R* :: '*run*
and *c* :: *src-coord*
and *e* :: ('*k*, '*v*) *source-event*
assumes *wf*: *wellformed-dblog-run* *b0* *R* *H*
and *e-in*: $Cdc\ c\ e \in set\ (clean-prefix-of\ R)$
shows $(c, e) \in set\ (cdc-events-of\ R)$
 $\wedge key-of\ e \in scope-of\ R$
 $\wedge src-le\ c\ (frontier-of\ R)$

proof —

let *?mkRec* = $\lambda ch. \langle \mid crr-domain = chunk-domain\ R\ ch,$
 $crr-read-coord = chunk-read-coordinate\ R\ ch,$
 $crr-observation = chunk-read-result\ R\ ch \mid \rangle$

— Cdc events cannot come from the *chunk-read-refreshes* branch (which produces only Refresh events).

have *not-refresh-branch*:

$\forall ch. Cdc\ c\ e \notin set\ (chunk-read-refreshes\ (?mkRec\ ch))$
unfolding *chunk-read-refreshes-def* *List.map-filter-def*
by (*auto split: option.splits*)

have *split*:

$Cdc\ c\ e \in set\ (cdc-event-replays\ (scope-of\ R)$
 $(frontier-of\ R)$
 $(cdc-events-of\ R))$

$\vee (\exists ch \in set\ (chunks-list\ R).$
 $Cdc\ c\ e \in set\ (chunk-read-refreshes\ (?mkRec\ ch)))$

using $e\text{-in}$
by (*auto simp: clean-prefix-of-def canonical-clean-prefix-def*)
from *split not-refresh-branch* **have** *in-cdc-replays*:
 $Cdc\ c\ e \in set\ (cdc\text{-event-replays}\ (scope\text{-of}\ R)$
 $(frontier\text{-of}\ R)$
 $(cdc\text{-events-of}\ R))$
by *blast*
— The *cdc-event-replays* filter directly yields the three conjuncts: $(c, e) \in cdc\text{-events-of}\ R + key\text{-of}\ e \in scope\text{-of}\ R + c \leq frontier\text{-of}\ R$ (bridged to *src-le* via *less-eq-src-coord-def*).
from *in-cdc-replays* **show** *?thesis*
unfolding *cdc-event-replays-def List.map-filter-def*
by (*auto split: if-splits simp: less-eq-src-coord-def*)
qed

8.3 Clean-prefix CDC order respects source position

lemma *clean-prefix-of-cdc-order-respects-source-position*:

fixes $b0 :: 'k :: linorder \rightarrow 'v$
and $H :: ('k, 'v)\ src\text{-history}$
and $R :: 'run$
and $c :: src\text{-coord}$
and $e1\ e2 :: ('k, 'v)\ source\text{-event}$
and $i\ j :: nat$
assumes *wf*: $wellformed\text{-dblog-run}\ b0\ R\ H$
and *ij*: $i < j$
and *j-bound*: $j < length\ (clean\text{-prefix-of}\ R)$
and *at-i*: $clean\text{-prefix-of}\ R ! i = Cdc\ c\ e1$
and *at-j*: $clean\text{-prefix-of}\ R ! j = Cdc\ c\ e2$
shows $\exists\ p\ q. p < q$
 $\wedge\ q < length\ (cdc\text{-events-of}\ R)$
 $\wedge\ cdc\text{-events-of}\ R ! p = (c, e1)$
 $\wedge\ cdc\text{-events-of}\ R ! q = (c, e2)$

proof —

let $?mkRec = \lambda ch. (\downarrow\ crr\text{-domain} = chunk\text{-domain}\ R\ ch,$
 $crr\text{-read-coord} = chunk\text{-read-coordinate}\ R\ ch,$
 $crr\text{-observation} = chunk\text{-read-result}\ R\ ch\ \downarrow)$
let $?cdc\text{-part} = cdc\text{-event-replays}\ (scope\text{-of}\ R)\ (frontier\text{-of}\ R)\ (cdc\text{-events-of}\ R)$
let $?refresh\text{-part} = concat\ (map\ chunk\text{-read-refreshes}\ (map\ ?mkRec\ (chunks\text{-list}\ R)))$
let $?inp = ?cdc\text{-part}\ @\ ?refresh\text{-part}$
let $?Pc = \lambda x :: ('k, 'v)\ replay\text{-event}. \exists\ e. x = Cdc\ c\ e$
let $?Qc = \lambda x :: ('k, 'v)\ replay\text{-event}. cdc\text{-coord}\ x = c$
let $?Qpair = \lambda p :: (src\text{-coord} \times ('k, 'v)\ source\text{-event}).$
 $fst\ p = c$
 $\wedge\ key\text{-of}\ (snd\ p) \in scope\text{-of}\ R$
 $\wedge\ fst\ p \leq frontier\text{-of}\ R$
let $?in\text{-filter} = \lambda p :: (src\text{-coord} \times ('k, 'v)\ source\text{-event}).$

$key-of (snd\ p) \in scope-of\ R$
 $\wedge\ fst\ p \leq frontier-of\ R$

define S **where** $S = filter\ ?Qpair\ (cdc-events-of\ R)$

have $i-lt-cp$: $i < length\ (clean-prefix-of\ R)$ **using** $ij\ j-bound$ **by** $simp$
have Pi : $?Pc\ (clean-prefix-of\ R\ !\ i)$ **using** $at-i$ **by** $auto$
have Pj : $?Pc\ (clean-prefix-of\ R\ !\ j)$ **using** $at-j$ **by** $auto$

— Step (a): $local.clean-prefix-of$ as the $sort-key$ over $cdc-part$ @ $refresh-part$.
have $cp-eq$: $clean-prefix-of\ R = sort-key\ cdc-coord\ ?inp$
unfolding $clean-prefix-of-def\ canonical-clean-prefix-def$ **by** $simp$

— Step (b): $cdc-part$ as map over a filter of $cdc-events-of\ R$.
have $cdc-part-eq$:
 $?cdc-part = map\ (\lambda p. Cdc\ (fst\ p)\ (snd\ p))\ (filter\ ?in-filter\ (cdc-events-of\ R))$
unfolding $cdc-event-replays-def$
by $(simp\ add: map-filter-map-filter[symmetric])$

— Step (c): $refresh-part$ contributes no Cdc events.
have $refresh-no-Cdc$:
 $\bigwedge\ ch\ x. x \in set\ (chunk-read-refreshes\ (?mkRec\ ch)) \implies \neg\ ?Pc\ x$
by $(auto\ simp: chunk-read-refreshes-def\ List.map-filter-def\ split: option.splits)$
have $refresh-filter-empty$: $filter\ ?Pc\ ?refresh-part = []$
proof —
have $\forall\ x \in set\ ?refresh-part. \neg\ ?Pc\ x$
using $refresh-no-Cdc$ **by** $auto$
thus $?thesis$ **by** $(simp\ add: filter-empty-conv)$
qed

— Step (d): $filter\ (\lambda y. ?f\ y = ?k)\ (sort-key\ ?f\ ?xs) = filter\ (\lambda y. ?f\ y = ?k)\ ?xs$
at coordinate c .
have $stab-Qc$:
 $filter\ ?Qc\ (clean-prefix-of\ R) = filter\ ?Qc\ ?inp$
using $cp-eq\ sort-key-stable[of\ cdc-coord\ c\ ?inp]$ **by** $simp$

— Chained filtering: $?Pc\ x$ implies $?Qc\ x$, so filtering by $?Pc$ commutes with first filtering by $?Qc$.
have $Pc-chain$:
 $filter\ ?Pc\ xs = filter\ ?Pc\ (filter\ ?Qc\ xs)$
for $xs :: ('k, 'v)\ replay-event\ list$
by $(induction\ xs)\ auto$

— Composition: filter $?Pc$ collapses to the $cdc-part$ contribution.
have $filter-Pc-cp$:
 $filter\ ?Pc\ (clean-prefix-of\ R) = filter\ ?Pc\ ?cdc-part$
proof —
have $filter\ ?Pc\ (clean-prefix-of\ R) = filter\ ?Pc\ (filter\ ?Qc\ (clean-prefix-of\ R))$
using $Pc-chain$ **by** $simp$
also have $\dots = filter\ ?Pc\ (filter\ ?Qc\ ?inp)$

```

    using stab-Qc by simp
  also have ... = filter ?Pc ?inp
    using Pc-chain by simp
  also have filter ?Pc ?inp
    = filter ?Pc ?cdc-part @ filter ?Pc ?refresh-part
    by simp
  also have ... = filter ?Pc ?cdc-part
    using refresh-filter-empty by simp
  finally show ?thesis .
qed

```

— *filter ?Pc cdc-part* as a map over *S*.

have *filter-Pc-cdc-part*:

```

  filter ?Pc ?cdc-part = map (λp. Cdc c (snd p)) S

```

proof —

```

  have filter ?Pc ?cdc-part
    = filter ?Pc (map (λp. Cdc (fst p) (snd p))
      (filter ?in-filter (cdc-events-of R)))
    using cdc-part-eq by simp
  also have ... = map (λp. Cdc (fst p) (snd p))
    (filter (λp. ?Pc (Cdc (fst p) (snd p)))
      (filter ?in-filter (cdc-events-of R)))
    by (simp add: filter-map o-def)
  also have pred-simp:
    (λp::(src-coord × ('k, 'v) source-event). ?Pc (Cdc (fst p) (snd p)))
    = (λp. fst p = c)
    by (auto intro: ext)
  also have filter (λp::(src-coord × ('k, 'v) source-event). fst p = c)
    (filter ?in-filter (cdc-events-of R))
    = filter ?Qpair (cdc-events-of R)
    by (simp add: filter-filter conj-commute)
  also have filter ?Qpair (cdc-events-of R) = S
    unfolding S-def by simp
  finally have step:
    filter ?Pc ?cdc-part = map (λp. Cdc (fst p) (snd p)) S by simp
  have rewrite-map:
    map (λp. Cdc (fst p) (snd p)) S = map (λp. Cdc c (snd p)) S
    by (rule map-cong[OF refl]) (auto simp: S-def)
  show ?thesis using step rewrite-map by simp
qed

```

have *filter-Pc-eq-map*:

```

  filter ?Pc (clean-prefix-of R) = map (λp. Cdc c (snd p)) S
  using filter-Pc-cp filter-Pc-cdc-part by simp

```

— Forward step: positions *i*, *j* in *clean-prefix-of R* lift to positions *n-i* < *n-j* in *filter ?Pc (clean-prefix-of R)* via *filter-take-nth + length-filter-take-strict-mono*.

```

  define n-i where n-i = length (filter ?Pc (take i (clean-prefix-of R)))
  define n-j where n-j = length (filter ?Pc (take j (clean-prefix-of R)))

```

have *nth-i-val-lt*:
filter ?*Pc* (*clean-prefix-of* *R*) ! *n-i* = *clean-prefix-of* *R* ! *i*
 $\wedge$ *n-i* < *length* (*filter* ?*Pc* (*clean-prefix-of* *R*))
using *filter-take-nth*[**where** *xs* = *clean-prefix-of* *R*
and *P* = $\lambda x::('k, 'v)$ *replay-event*. $\exists e. x = \text{Cdc } c \ e$
and *i* = *i*, *OF i-lt-cp* *Pi*]
unfolding *n-i-def* **by** *simp*
have *nth-j-val-lt*:
filter ?*Pc* (*clean-prefix-of* *R*) ! *n-j* = *clean-prefix-of* *R* ! *j*
 $\wedge$ *n-j* < *length* (*filter* ?*Pc* (*clean-prefix-of* *R*))
using *filter-take-nth*[**where** *xs* = *clean-prefix-of* *R*
and *P* = $\lambda x::('k, 'v)$ *replay-event*. $\exists e. x = \text{Cdc } c \ e$
and *i* = *j*, *OF j-bound* *Pj*]
unfolding *n-j-def* **by** *simp*
have *ni-lt-nj*: *n-i* < *n-j*
using *length-filter-take-strict-mono*[**where** *xs* = *clean-prefix-of* *R*
and *P* = $\lambda x::('k, 'v)$ *replay-event*. $\exists e. x = \text{Cdc } c \ e$
and *i* = *i* **and** *j* = *j*,
OF ij - Pi] *j-bound*
unfolding *n-i-def* *n-j-def* **by** *simp*

— S-positions at *n-i*, *n-j* carry (*c*, *e1*), (*c*, *e2*).
have *len-eq*: *length* (*filter* ?*Pc* (*clean-prefix-of* *R*)) = *length* *S*
using *filter-Pc-eq-map* **by** (*simp* *add*: *length-map*)
have *ni-lt-S*: *n-i* < *length* *S* **using** *nth-i-val-lt* *len-eq* **by** *simp*
have *nj-lt-S*: *n-j* < *length* *S* **using** *nth-j-val-lt* *len-eq* **by** *simp*
have *S-at-ni*: *S* ! *n-i* = (*c*, *e1*)
proof —
have (*map* ($\lambda p. \text{Cdc } c \ (\text{snd } p)$) *S*) ! *n-i* = *Cdc* *c* (*snd* (*S* ! *n-i*))
using *ni-lt-S* **by** *simp*
moreover **have** (*map* ($\lambda p. \text{Cdc } c \ (\text{snd } p)$) *S*) ! *n-i* = *Cdc* *c* *e1*
using *filter-Pc-eq-map* *nth-i-val-lt* *at-i* **by** *simp*
ultimately **have** *Cdc* *c* (*snd* (*S* ! *n-i*)) = *Cdc* *c* *e1* **by** *simp*
hence *snd-eq*: *snd* (*S* ! *n-i*) = *e1* **by** *simp*
have *S* ! *n-i* $\in$ *set* *S* **using** *ni-lt-S* *nth-mem* **by** *blast*
hence *fst-eq*: *fst* (*S* ! *n-i*) = *c*
using *S-def* **by** *auto*
show *S* ! *n-i* = (*c*, *e1*) **using** *snd-eq* *fst-eq* **by** (*cases* *S* ! *n-i*) *simp*
qed
have *S-at-nj*: *S* ! *n-j* = (*c*, *e2*)
proof —
have (*map* ($\lambda p. \text{Cdc } c \ (\text{snd } p)$) *S*) ! *n-j* = *Cdc* *c* (*snd* (*S* ! *n-j*))
using *nj-lt-S* **by** *simp*
moreover **have** (*map* ($\lambda p. \text{Cdc } c \ (\text{snd } p)$) *S*) ! *n-j* = *Cdc* *c* *e2*
using *filter-Pc-eq-map* *nth-j-val-lt* *at-j* **by** *simp*
ultimately **have** *Cdc* *c* (*snd* (*S* ! *n-j*)) = *Cdc* *c* *e2* **by** *simp*
hence *snd-eq*: *snd* (*S* ! *n-j*) = *e2* **by** *simp*
have *S* ! *n-j* $\in$ *set* *S* **using** *nj-lt-S* *nth-mem* **by** *blast*

hence *fst-eq*: $\text{fst } (S ! n-j) = c$
using *S-def* **by** *auto*
show $S ! n-j = (c, e2)$ **using** *snd-eq fst-eq* **by** (*cases* $S ! n-j$) *simp*
qed

— Backward step: positions $n-i, n-j$ in S lift to positions p, q in *cdc-events-of* R via *filter-index-strict-mono* + *length-filter-take-mono* contrapositive.

have *ni-lt-S-filter*: $n-i < \text{length } (\text{filter } ?Qpair \text{ (cdc-events-of } R))$
using *ni-lt-S S-def* **by** *simp*
have *nj-lt-S-filter*: $n-j < \text{length } (\text{filter } ?Qpair \text{ (cdc-events-of } R))$
using *nj-lt-S S-def* **by** *simp*

obtain p **where** *p-props*:
 $p < \text{length } (\text{cdc-events-of } R)$
 $?Qpair \text{ (cdc-events-of } R ! p)$
 $\text{cdc-events-of } R ! p = \text{filter } ?Qpair \text{ (cdc-events-of } R) ! n-i$
 $\text{length } (\text{filter } ?Qpair \text{ (take } p \text{ (cdc-events-of } R))) = n-i$
using *filter-index-strict-mono*[*OF ni-lt-S-filter*] **by** *blast*
obtain q **where** *q-props*:
 $q < \text{length } (\text{cdc-events-of } R)$
 $?Qpair \text{ (cdc-events-of } R ! q)$
 $\text{cdc-events-of } R ! q = \text{filter } ?Qpair \text{ (cdc-events-of } R) ! n-j$
 $\text{length } (\text{filter } ?Qpair \text{ (take } q \text{ (cdc-events-of } R))) = n-j$
using *filter-index-strict-mono*[*OF nj-lt-S-filter*] **by** *blast*

have *p-val*: $\text{cdc-events-of } R ! p = (c, e1)$
using *p-props S-at-ni S-def* **by** *simp*
have *q-val*: $\text{cdc-events-of } R ! q = (c, e2)$
using *q-props S-at-nj S-def* **by** *simp*

— $p < q$ follows from monotonicity of $\text{length} \circ \text{filter } ?Qpair \circ \text{take} \cdot$ composed with the contrapositive of $n-i < n-j$.

have *p-lt-q*: $p < q$
proof (*rule ccontr*)
assume $\neg p < q$
hence *q-le-p*: $q \leq p$ **by** *simp*
have $\text{length } (\text{filter } ?Qpair \text{ (take } q \text{ (cdc-events-of } R)))$
 $\leq \text{length } (\text{filter } ?Qpair \text{ (take } p \text{ (cdc-events-of } R)))$
using *length-filter-take-mono*[**where** $xs = \text{cdc-events-of } R$
and $P = ?Qpair$
and $i = q$ **and** $j = p$,
OF q-le-p]
by *simp*
hence $n-j \leq n-i$ **using** *p-props q-props* **by** *simp*
thus *False* **using** *ni-lt-nj* **by** *simp*
qed

show *?thesis*
proof (*intro exI conjI*)
show $p < q$ **by** (*rule p-lt-q*)


```

  show  $q < \text{length}(\text{cdc-events-of } R)$  using  $q\text{-props}$  by simp
  show  $\text{cdc-events-of } R ! p = (c, e1)$  by (rule  $p\text{-val}$ )
  show  $\text{cdc-events-of } R ! q = (c, e2)$  by (rule  $q\text{-val}$ )
qed
qed

```

8.4 Latest per-key replay-event correspondence

lemma *clean-prefix-of-latest-replay-correspondence:*

```

fixes  $b0 :: 'k :: \text{linorder} \rightarrow 'v$ 
and  $H :: ('k, 'v) \text{src-history}$ 
and  $R :: 'run$ 
and  $k :: 'k$ 
assumes  $wf: \text{wellformed-dblog-run } b0 \ R \ H$ 
and  $k\text{-in-scope: } k \in \text{scope-of } R$ 

```

shows

```

( $\exists i \ c \ e.$ 
   $i < \text{length}(\text{clean-prefix-of } R)$ 
   $\wedge \text{clean-prefix-of } R ! i = \text{Cdc } c \ e$ 
   $\wedge \text{key-of } e = k$ 
   $\wedge (\forall j. i < j \wedge j < \text{length}(\text{clean-prefix-of } R)$ 
     $\rightarrow \text{event-key}(\text{clean-prefix-of } R ! j) \neq k)$ 
   $\wedge (c, e) \in \text{set}(\text{cdc-events-of } R)$ 
   $\wedge \text{src-le } c (\text{frontier-of } R))$ 
 $\vee$ 
( $\exists i \ ch \ m \ c\text{-read}.$ 
   $i < \text{length}(\text{clean-prefix-of } R)$ 
   $\wedge \text{clean-prefix-of } R ! i = \text{Refresh } k \ m \ c\text{-read}$ 
   $\wedge (\forall j. i < j \wedge j < \text{length}(\text{clean-prefix-of } R)$ 
     $\rightarrow \text{event-key}(\text{clean-prefix-of } R ! j) \neq k)$ 
   $\wedge \text{responsible-chunk } R \ k = \text{Some } ch$ 
   $\wedge \text{chunk-read-result } R \ ch \ k = \text{Some } m$ 
   $\wedge \text{chunk-read-coordinate } R \ ch = c\text{-read})$ 

```

proof —

— Step 1: *clean-prefix-of-in-scope-responsible-chunk-refresh-exists* gives a Refresh event for k , yielding existence of an index.

obtain $m0 \ c0\text{-read}$ **where** *refresh-exists:*

$\text{Refresh } k \ m0 \ c0\text{-read} \in \text{set}(\text{clean-prefix-of } R)$

using *clean-prefix-of-in-scope-responsible-chunk-refresh-exists*[$OF \ wf \ k\text{-in-scope}$]

by *blast*

define *event-indices* **where**

```

event-indices =
  { $i. i < \text{length}(\text{clean-prefix-of } R)$ 
     $\wedge \text{event-key}(\text{clean-prefix-of } R ! i) = k$ }

```

have *event-indices-finite:* *finite event-indices*

unfolding *event-indices-def* **by** *auto*

have *event-indices-nonempty:* *event-indices* $\neq \{\}$

proof —

from *refresh-exists* **obtain** i **where**
 $i\text{-lt}$: $i < \text{length } (\text{clean-prefix-of } R)$
and $\text{at-}i$: $\text{clean-prefix-of } R \ ! \ i = \text{Refresh } k \ m0 \ c0\text{-read}$
by (*auto simp: in-set-conv-nth*)
have $\text{event-key } (\text{clean-prefix-of } R \ ! \ i) = k$
using $\text{at-}i$ **by** *simp*
hence $i \in \text{event-indices}$
using $i\text{-lt}$ **unfolding** *event-indices-def* **by** *auto*
thus *?thesis* **by** *blast*
qed

— Step 2: $i\text{-max} = \text{Max event-indices}$ is the latest k -event position.

define $i\text{-max}$ **where** $i\text{-max} = \text{Max event-indices}$
have $i\text{-max-in}$: $i\text{-max} \in \text{event-indices}$
using *event-indices-finite event-indices-nonempty*
unfolding $i\text{-max-def}$ **by** (*rule Max-in*)
have $i\text{-max-lt}$: $i\text{-max} < \text{length } (\text{clean-prefix-of } R)$
using $i\text{-max-in}$ **unfolding** *event-indices-def* **by** *simp*
have $i\text{-max-event-key}$: $\text{event-key } (\text{clean-prefix-of } R \ ! \ i\text{-max}) = k$
using $i\text{-max-in}$ **unfolding** *event-indices-def* **by** *simp*
have $i\text{-max-latest}$:
 $\forall j. i\text{-max} < j \wedge j < \text{length } (\text{clean-prefix-of } R)$
 $\longrightarrow \text{event-key } (\text{clean-prefix-of } R \ ! \ j) \neq k$
proof (*intro allI impI*)
fix j
assume $j\text{-assm}$: $i\text{-max} < j \wedge j < \text{length } (\text{clean-prefix-of } R)$
hence $j\text{-props}$: $i\text{-max} < j \wedge j < \text{length } (\text{clean-prefix-of } R)$ **by** *auto*
show $\text{event-key } (\text{clean-prefix-of } R \ ! \ j) \neq k$
proof (*rule ccontr*)
assume $\neg \text{event-key } (\text{clean-prefix-of } R \ ! \ j) \neq k$
hence $j\text{-event-key}$: $\text{event-key } (\text{clean-prefix-of } R \ ! \ j) = k$ **by** *simp*
hence $j\text{-in}$: $j \in \text{event-indices}$
unfolding *event-indices-def* **using** $j\text{-props}$ **by** *auto*
have $j \leq i\text{-max}$
unfolding $i\text{-max-def}$
by (*rule Max-ge[OF event-indices-finite j-in]*)
with $j\text{-props}$ **show** *False* **by** *simp*
qed
qed

— Step 3: case-split on the shape of $\text{clean-prefix-of } R \ ! \ i\text{-max}$.

show *?thesis*

proof (*cases clean-prefix-of R ! i-max*)

case (*Cdc c e*)

— *Cdc* case: apply *clean-prefix-of-cdc-generated-from-observed-cdc* to extract the three correspondence conjuncts.

have $e\text{-in-cp}$: $Cdc \ c \ e \in \text{set } (\text{clean-prefix-of } R)$

using $i\text{-max-lt}$ $Cdc \ nth\text{-mem}$ **by** *force*

have $\text{cdc-correspondence}$:

```

(c, e) ∈ set (cdc-events-of R)
  ∧ key-of e ∈ scope-of R
  ∧ src-le c (frontier-of R)
using clean-prefix-of-cdc-generated-from-observed-cdc[OF wf e-in-cp]
by blast
have key-eq: key-of e = k
  using i-max-event-key Cdc by simp
show ?thesis
proof (intro disjI1 exI[where x = i-max] exI[where x = c] exI[where x =
e] conjI)
  show i-max < length (clean-prefix-of R) by (rule i-max-lt)
  show clean-prefix-of R ! i-max = Cdc c e by (rule Cdc)
  show key-of e = k by (rule key-eq)
  show ∀ j. i-max < j ∧ j < length (clean-prefix-of R)
    → event-key (clean-prefix-of R ! j) ≠ k
    by (rule i-max-latest)
  show (c, e) ∈ set (cdc-events-of R) using cdc-correspondence by simp
  show src-le c (frontier-of R) using cdc-correspondence by simp
qed
next
case (Refresh k' m' c-read)
  — Refresh case: i-max-event-key forces k' = k, then the responsible-chunk
refresh-provenance lemma above applies (named in the using step just below).
  have k'-eq-k: k' = k
    using i-max-event-key Refresh by simp
  have refresh-in-cp: Refresh k m' c-read ∈ set (clean-prefix-of R)
    using i-max-lt Refresh k'-eq-k nth-mem by force
  obtain ch where ch-props:
    responsible-chunk R k = Some ch
    chunk-read-coordinate R ch = c-read
    chunk-read-result R ch k = Some m'
    using clean-prefix-of-refresh-generated-by-responsible-chunk[OF wf re-
fresh-in-cp]
    by blast
  show ?thesis
proof (intro disjI2 exI[where x = i-max] exI[where x = ch]
  exI[where x = m'] exI[where x = c-read] conjI)
  show i-max < length (clean-prefix-of R) by (rule i-max-lt)
  show clean-prefix-of R ! i-max = Refresh k m' c-read
    using Refresh k'-eq-k by simp
  show ∀ j. i-max < j ∧ j < length (clean-prefix-of R)
    → event-key (clean-prefix-of R ! j) ≠ k
    by (rule i-max-latest)
  show responsible-chunk R k = Some ch using ch-props by simp
  show chunk-read-result R ch k = Some m' using ch-props by simp
  show chunk-read-coordinate R ch = c-read using ch-props by simp
qed
qed
qed

```

8.5 Per-key source/CDC coordinate-slice equalities

lemma *cdc-events-key-coord-slice-mset-eq-source-history-slice:*

```

fixes b0 :: 'k :: linorder  $\rightarrow$  'v
  and H :: ('k, 'v) src-history
  and R :: 'run
  and k :: 'k
  and ch :: 'chunk
  and c :: src-coord
assumes wf: wellformed-dblog-run b0 R H
  and k-scope: k  $\in$  scope-of R
  and rc: responsible-chunk R k = Some ch
  and read-lt-c: src-lt (chunk-read-coordinate R ch) c
  and c-le-f: src-le c (frontier-of R)
shows mset (source-key-coord-slice k c (cdc-events-of R))
  = mset (source-key-coord-slice k c (src-history-of R))
proof -
  note wf-body = wf[unfolded wellformed-dblog-run-def]
  from wf-body have wf2-mset:
     $\forall$  k  $\in$  scope-of R.  $\forall$  ch c.
      responsible-chunk R k = Some ch
       $\wedge$  src-lt (chunk-read-coordinate R ch) c
       $\wedge$  src-le c (frontier-of R)
     $\longrightarrow$ 
      mset (source-key-coord-slice k c (cdc-events-of R))
      = mset (source-key-coord-slice k c (src-history-of R))
  by (elim conjE)
  from wf2-mset k-scope rc read-lt-c c-le-f show ?thesis by blast
qed

```

lemma *cdc-events-key-coord-slice-eq-source-history-slice:*

```

fixes b0 :: 'k :: linorder  $\rightarrow$  'v
  and H :: ('k, 'v) src-history
  and R :: 'run
  and k :: 'k
  and ch :: 'chunk
  and c :: src-coord
assumes wf: wellformed-dblog-run b0 R H
  and k-scope: k  $\in$  scope-of R
  and rc: responsible-chunk R k = Some ch
  and read-lt-c: src-lt (chunk-read-coordinate R ch) c
  and c-le-f: src-le c (frontier-of R)
  and sub:
    subseq (source-key-coord-slice k c (cdc-events-of R))
    (source-key-coord-slice k c (src-history-of R))
shows source-key-coord-slice k c (cdc-events-of R)
  = source-key-coord-slice k c (src-history-of R)
proof -
  have ms:

```

```

mset (source-key-coord-slice k c (cdc-events-of R))
= mset (source-key-coord-slice k c (src-history-of R))
by (rule cdc-events-key-coord-slice-mset-eq-source-history-slice
[OF wf k-scope rc read-lt-c c-le-f])
show ?thesis by (rule subseq-mset-eq-imp-eq[OF sub ms])
qed

```

lemma *cdc-events-key-coord-slice-subseq-source-history-slice*:

```

fixes b0 :: 'k :: linorder  $\rightarrow$  'v
and H :: ('k, 'v) src-history
and R :: 'run
and k :: 'k
and c :: src-coord
assumes wf: wellformed-dblog-run b0 R H
and k-scope: k  $\in$  scope-of R
and c-le-f: src-le c (frontier-of R)
shows subseq (source-key-coord-slice k c (cdc-events-of R))
(source-key-coord-slice k c (src-history-of R))

```

proof –

note *wf-body* = wf[unfolded wellformed-dblog-run-def]

define *in-slice*

```

where in-slice =
( $\lambda p::(\text{src-coord} \times ('k, 'v) \text{ source-event}).$ 
key-of (hist-event p)  $\in$  scope-of R
 $\wedge$  src-le (hist-coord p) (frontier-of R))

```

let ?Q =

```

 $\lambda p::(\text{src-coord} \times ('k, 'v) \text{ source-event}).$ 
hist-coord p = c  $\wedge$  key-of (hist-event p) = k

```

from *wf-body* **obtain** $\iota :: \text{nat} \Rightarrow \text{nat}$ **where**

iota-mono: strict-mono ι **and**

iota-body:

```

(let in-slice = ( $\lambda p.$  key-of (hist-event p)  $\in$  scope-of R
 $\wedge$  src-le (hist-coord p) (frontier-of R));
cdc-in-slice = filter in-slice (cdc-events-of R);
hist-in-slice = filter in-slice (src-history-of R)
in  $\forall i. i < \text{length cdc-in-slice}$ 
 $\longrightarrow \iota i < \text{length hist-in-slice}$ 
 $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! \iota i$ )

```

by (elim conjE exE)

have *iota-all*:

```

 $\forall i. i < \text{length (filter in-slice (cdc-events-of R))}$ 
 $\longrightarrow \iota i < \text{length (filter in-slice (src-history-of R))}$ 
 $\wedge \text{filter in-slice (cdc-events-of R)} ! i$ 
 $= \text{filter in-slice (src-history-of R)} ! \iota i$ 

```

using *iota-body* **unfolding** *in-slice-def* **by** (simp add: Let-def)

have *sub-in*:

```

    subseq (filter in-slice (cdc-events-of R))
            (filter in-slice (src-history-of R))
proof (rule subseq-from-strict-mono-nth[OF iota-mono])
  fix i
  assume i-lt: i < length (filter in-slice (cdc-events-of R))
  from iota-all i-lt
  show  $\iota$  i < length (filter in-slice (src-history-of R))
     $\wedge$  filter in-slice (cdc-events-of R) ! i
    = filter in-slice (src-history-of R) !  $\iota$  i
  by blast
qed

have sub-filtered:
  subseq (filter ?Q (filter in-slice (cdc-events-of R)))
          (filter ?Q (filter in-slice (src-history-of R)))
by (rule subseq-filter[OF sub-in])

have cdc-rewrite:
  filter ?Q (filter in-slice (cdc-events-of R))
    = source-key-coord-slice k c (cdc-events-of R)
unfolding source-key-coord-slice-def in-slice-def
by (simp add: filter-filter,
    rule filter-cong[OF refl],
    use k-scope c-le-f in auto)
have hist-rewrite:
  filter ?Q (filter in-slice (src-history-of R))
    = source-key-coord-slice k c (src-history-of R)
unfolding source-key-coord-slice-def in-slice-def
by (simp add: filter-filter,
    rule filter-cong[OF refl],
    use k-scope c-le-f in auto)

show ?thesis
using sub-filtered by (simp only: cdc-rewrite hist-rewrite)
qed

lemma cdc-events-key-coord-slice-eq-source-history-slice-from-wf:
fixes b0 :: 'k :: linorder  $\rightarrow$  'v
and H :: ('k, 'v) src-history
and R :: 'run
and k :: 'k
and ch :: 'chunk
and c :: src-coord
assumes wf: wellformed-dblog-run b0 R H
and k-scope: k  $\in$  scope-of R
and rc: responsible-chunk R k = Some ch
and read-lt-c: src-lt (chunk-read-coordinate R ch) c
and c-le-f: src-le c (frontier-of R)

```

shows $\text{source-key-coord-slice } k \ c \ (\text{cdc-events-of } R)$
 $= \text{source-key-coord-slice } k \ c \ (\text{src-history-of } R)$
proof –
have sub :
 $\text{subseq } (\text{source-key-coord-slice } k \ c \ (\text{cdc-events-of } R))$
 $(\text{source-key-coord-slice } k \ c \ (\text{src-history-of } R))$
by (rule $\text{cdc-events-key-coord-slice-subseq-source-history-slice}$
 $[\text{OF wf } k\text{-scope } c\text{-le-f}])$
show $?thesis$
by (rule $\text{cdc-events-key-coord-slice-eq-source-history-slice}$
 $[\text{OF wf } k\text{-scope } rc \ \text{read-lt-c } c\text{-le-f } \text{sub}])$
qed

8.6 Clean-prefix structure at a fixed coordinate

lemma $\text{clean-prefix-cdc-key-coord-slice}$:

fixes $R :: 'run$
and $k :: 'k$
and $c :: \text{src-coord}$
assumes $k\text{-scope}: k \in \text{scope-of } R$
and $c\text{-le-f}: \text{src-le } c \ (\text{frontier-of } R)$
shows filter
 $(\lambda x. \text{case } x \text{ of}$
 $\quad \text{Cdc } c' \ e \Rightarrow c' = c \wedge \text{key-of } e = k$
 $\quad | \ \text{Refresh } - \ - \Rightarrow \text{False})$
 $(\text{clean-prefix-of } R)$
 $= \text{map } (\lambda p. \text{Cdc } (\text{hist-coord } p) \ (\text{hist-event } p))$
 $(\text{source-key-coord-slice } k \ c \ (\text{cdc-events-of } R))$
proof –
let $?mkRec = \lambda ch. (\downarrow \text{crr-domain} \quad = \text{chunk-domain } R \ ch,$
 $\quad \text{crr-read-coord} = \text{chunk-read-coordinate } R \ ch,$
 $\quad \text{crr-observation} = \text{chunk-read-result } R \ ch \downarrow)$
let $?cdc\text{-part} = \text{cdc-event-replays } (\text{scope-of } R) \ (\text{frontier-of } R) \ (\text{cdc-events-of } R)$
let $?refresh\text{-part} = \text{concat } (\text{map } \text{chunk-read-refreshes } (\text{map } ?mkRec \ (\text{chunks-list } R)))$
let $?inp = ?cdc\text{-part} @ ?refresh\text{-part}$
let $?P = \lambda x::('k, 'v) \ \text{replay-event.}$
 $\quad \text{case } x \text{ of } \text{Cdc } c' \ e \Rightarrow c' = c \wedge \text{key-of } e = k$
 $\quad | \ \text{Refresh } - \ - \Rightarrow \text{False}$
let $?Qc = \lambda x::('k, 'v) \ \text{replay-event. } \text{cdc-coord } x = c$

have $\text{cp-eq}: \text{clean-prefix-of } R = \text{sort-key } \text{cdc-coord } ?inp$
unfolding $\text{clean-prefix-of-def canonical-clean-prefix-def}$ **by** simp
have $\text{refresh-no-cdc-slice}$:
 $\bigwedge ch \ x. x \in \text{set } (\text{chunk-read-refreshes } (?mkRec \ ch)) \Longrightarrow \neg ?P \ x$
by (auto simp: $\text{chunk-read-refreshes-def List.map-filter-def split: option.splits}$)
have $\text{refresh-filter-empty}: \text{filter } ?P \ ?refresh\text{-part} = []$
proof –
have $\forall x \in \text{set } ?refresh\text{-part. } \neg ?P \ x$

```

    using refresh-no-cdc-slice by auto
    thus ?thesis by (simp add: filter-empty-conv)
qed
have stab-Qc: filter ?Qc (clean-prefix-of R) = filter ?Qc ?inp
  using cp-eq sort-key-stable[of cdc-coord c ?inp] by simp
have P-chain:
  filter ?P xs = filter ?P (filter ?Qc xs)
  for xs :: ('k, 'v) replay-event list
  by (induction xs) (auto split: replay-event.splits)
have filter-P-cp: filter ?P (clean-prefix-of R) = filter ?P ?cdc-part
proof -
  have filter ?P (clean-prefix-of R) = filter ?P (filter ?Qc (clean-prefix-of R))
    using P-chain by simp
  also have ... = filter ?P (filter ?Qc ?inp)
    using stab-Qc by simp
  also have ... = filter ?P ?inp
    using P-chain by simp
  also have ... = filter ?P ?cdc-part @ filter ?P ?refresh-part
    by simp
  also have ... = filter ?P ?cdc-part
    using refresh-filter-empty by simp
  finally show ?thesis .
qed
have cdc-part-slice:
  filter ?P ?cdc-part
    = map (λp. Cdc (hist-coord p) (hist-event p))
      (source-key-coord-slice k c (cdc-events-of R))
proof -
  let ?in-filter =
    λp::(src-coord × ('k, 'v) source-event).
      key-of (snd p) ∈ scope-of R ∧ fst p ≤ frontier-of R
  have cdc-part-eq:
    ?cdc-part =
      map (λp. Cdc (hist-coord p) (hist-event p))
        (filter ?in-filter (cdc-events-of R))
    unfolding cdc-event-replays-def
    by (simp add: map-filter-map-filter[symmetric])
  have filter-collapse:
    filter (λp. hist-coord p = c ∧ key-of (hist-event p) = k)
      (filter ?in-filter (cdc-events-of R))
    = source-key-coord-slice k c (cdc-events-of R)
    unfolding source-key-coord-slice-def
    by (simp add: filter-filter,
        rule filter-cong[OF refl],
        use k-scope c-le-f in ⟨auto simp: less-eq-src-coord-def⟩)
  have filter ?P ?cdc-part
    = filter ?P
      (map (λp. Cdc (hist-coord p) (hist-event p))
        (filter ?in-filter (cdc-events-of R)))

```



```

    using cdc-part-eq by simp
  also have ...
    = map (λp. Cdc (hist-coord p) (hist-event p))
      (filter (λp. hist-coord p = c ∧ key-of (hist-event p) = k)
        (filter ?in-filter (cdc-events-of R)))
  by (simp add: filter-map o-def)
  also have ...
    = map (λp. Cdc (hist-coord p) (hist-event p))
      (source-key-coord-slice k c (cdc-events-of R))
  using filter-collapse by simp
  finally show ?thesis .
qed
show ?thesis using filter-P-cp cdc-part-slice by simp
qed

```

lemma *clean-prefix-same-coord-refresh-before-cdc-false:*

```

  fixes R :: 'run
  and k :: 'k
  and m :: 'v option
  and c :: src-coord
  and e :: ('k, 'v) source-event
  and i j :: nat
  assumes ij: i < j
    and j-lt: j < length (clean-prefix-of R)
    and at-i: clean-prefix-of R ! i = Refresh k m c
    and at-j: clean-prefix-of R ! j = Cdc c e
  shows False
proof -
  let ?mkRec = λch. (| crr-domain      = chunk-domain R ch,
                      crr-read-coord = chunk-read-coordinate R ch,
                      crr-observation = chunk-read-result R ch |)
  let ?cdc-part = cdc-event-replays (scope-of R) (frontier-of R) (cdc-events-of R)
  let ?refresh-part = concat (map chunk-read-refreshes (map ?mkRec (chunks-list R)))
  let ?inp = ?cdc-part @ ?refresh-part
  let ?Qc = λx::('k, 'v) replay-event. cdc-coord x = c
  let ?A = λx::('k, 'v) replay-event. case x of Cdc - - ⇒ True | Refresh - - - ⇒
False
  let ?B = λx::('k, 'v) replay-event. case x of Refresh - - - ⇒ True | Cdc - - ⇒
False

```

```

  have i-lt: i < length (clean-prefix-of R) using ij j-lt by simp
  have Q-i: ?Qc (clean-prefix-of R ! i) using at-i by simp
  have Q-j: ?Qc (clean-prefix-of R ! j) using at-j by simp
  have cp-eq: clean-prefix-of R = sort-key cdc-coord ?inp
  unfolding clean-prefix-of-def canonical-clean-prefix-def by simp
  have stab-Qc: filter ?Qc (clean-prefix-of R) = filter ?Qc ?inp
  using cp-eq sort-key-stable[of cdc-coord c ?inp] by simp

```

```

define  $n-i$  where  $n-i = \text{length } (\text{filter } ?Qc \text{ (take } i \text{ (clean-prefix-of } R)))$ 
define  $n-j$  where  $n-j = \text{length } (\text{filter } ?Qc \text{ (take } j \text{ (clean-prefix-of } R)))$ 

have  $nth-i$ :
   $\text{filter } ?Qc \text{ (clean-prefix-of } R) ! n-i = \text{clean-prefix-of } R ! i$ 
 $\wedge n-i < \text{length } (\text{filter } ?Qc \text{ (clean-prefix-of } R))$ 
  using  $\text{filter-take-nth}[\text{where } xs = \text{clean-prefix-of } R \text{ and } P = ?Qc \text{ and } i = i,$ 
     $OF \text{ } i\text{-lt } Q\text{-}i]$ 
  unfolding  $n-i\text{-def}$  by  $\text{simp}$ 
have  $nth-j$ :
   $\text{filter } ?Qc \text{ (clean-prefix-of } R) ! n-j = \text{clean-prefix-of } R ! j$ 
 $\wedge n-j < \text{length } (\text{filter } ?Qc \text{ (clean-prefix-of } R))$ 
  using  $\text{filter-take-nth}[\text{where } xs = \text{clean-prefix-of } R \text{ and } P = ?Qc \text{ and } i = j,$ 
     $OF \text{ } j\text{-lt } Q\text{-}j]$ 
  unfolding  $n-j\text{-def}$  by  $\text{simp}$ 
have  $ni\text{-lt-}nj$ :  $n-i < n-j$ 
  using  $\text{length-filter-take-strict-mono}[\text{where } xs = \text{clean-prefix-of } R$ 
     $\text{and } P = ?Qc$ 
     $\text{and } i = i \text{ and } j = j,$ 
     $OF \text{ } ij - Q\text{-}i] \text{ } j\text{-lt}$ 
  unfolding  $n-i\text{-def}$   $n-j\text{-def}$  by  $\text{simp}$ 
have  $nj\text{-lt-}inp$ :  $n-j < \text{length } (\text{filter } ?Qc \text{ } ?inp)$ 
  using  $nth-j \text{ stab-}Qc$  by  $\text{simp}$ 
have  $ni\text{-}B$ :  $?B \text{ (filter } ?Qc \text{ } ?inp ! n-i)$ 
  using  $nth-i \text{ at-}i \text{ stab-}Qc$  by  $\text{simp}$ 
have  $nj\text{-}A$ :  $?A \text{ (filter } ?Qc \text{ } ?inp ! n-j)$ 
  using  $nth-j \text{ at-}j \text{ stab-}Qc$  by  $\text{simp}$ 

have  $cdc\text{-all-}A$ :  $\forall x \in \text{set } (\text{filter } ?Qc \text{ } ?cdc\text{-part}). ?A \text{ } x$ 
  unfolding  $cdc\text{-event-replays-def}$   $List.map\text{-filter-def}$ 
  by  $(\text{auto split: if-splits})$ 
have  $refresh\text{-all-}B$ :  $\forall y \in \text{set } (\text{filter } ?Qc \text{ } ?refresh\text{-part}). ?B \text{ } y$ 
  unfolding  $chunk\text{-read-refreshes-def}$   $List.map\text{-filter-def}$ 
  by  $(\text{auto split: option.splits})$ 
have  $disj$ :  $?A \text{ } z \implies ?B \text{ } z \implies \text{False}$  for  $z$ 
  by  $(\text{cases } z) \text{ auto}$ 

show  $\text{False}$ 
  by  $(\text{rule append-no-}B\text{-before-}A$ 
     $[\text{where } xs = \text{filter } ?Qc \text{ } ?cdc\text{-part}$ 
     $\text{and } ys = \text{filter } ?Qc \text{ } ?refresh\text{-part}$ 
     $\text{and } zs = \text{filter } ?Qc \text{ } ?inp$ 
     $\text{and } A = ?A \text{ and } B = ?B$ 
     $\text{and } i = n-i \text{ and } j = n-j])$ 
     $(\text{use } cdc\text{-all-}A \text{ refresh-all-}B \text{ disj } ni\text{-lt-}nj \text{ } nj\text{-lt-}inp \text{ } ni\text{-}B \text{ } nj\text{-}A \text{ in auto})$ 
  )
qed

```

8.7 Consequences of a latest Cdc event for a key

lemma *latest-cdc-for-key-implies-chunk-read-strictly-before*:

fixes $b0 :: 'k :: \text{linorder} \rightarrow 'v$
and $H :: ('k, 'v) \text{src-history}$
and $R :: 'run$
and $k :: 'k$
and $i :: \text{nat}$
and $c :: \text{src-coord}$
and $e :: ('k, 'v) \text{source-event}$
assumes $wf: \text{wellformed-dblog-run } b0 \ R \ H$
and $k\text{-scope}: k \in \text{scope-of } R$
and $i\text{-lt}: i < \text{length } (\text{clean-prefix-of } R)$
and $cp\text{-i}: \text{clean-prefix-of } R \ ! \ i = \text{Cdc } c \ e$
and $key\text{-e}: \text{key-of } e = k$
and $i\text{-latest}$:
 $\forall j. i < j \wedge j < \text{length } (\text{clean-prefix-of } R)$
 $\rightarrow \text{event-key } (\text{clean-prefix-of } R \ ! \ j) \neq k$
shows $\exists ch. \text{responsible-chunk } R \ k = \text{Some } ch$
 $\wedge \text{src-lt } (\text{chunk-read-coordinate } R \ ch) \ c$

proof –

obtain $m \ c\text{-read}$ **where** refresh-in :
 $\text{Refresh } k \ m \ c\text{-read} \in \text{set } (\text{clean-prefix-of } R)$
using $\text{clean-prefix-of-in-scope-responsible-chunk-refresh-exists}[OF \ wf \ k\text{-scope}]$
by *blast*
then obtain $i\text{-r}$ **where**
 $i\text{-r-lt}: i\text{-r} < \text{length } (\text{clean-prefix-of } R)$ **and**
 $i\text{-r-at}: \text{clean-prefix-of } R \ ! \ i\text{-r} = \text{Refresh } k \ m \ c\text{-read}$
by $(\text{auto simp: in-set-conv-nth})$
obtain ch **where**
 $rc: \text{responsible-chunk } R \ k = \text{Some } ch$ **and**
 $read\text{-eq}: \text{chunk-read-coordinate } R \ ch = c\text{-read}$
using $\text{clean-prefix-of-refresh-generated-by-responsible-chunk}[OF \ wf \ \text{refresh-in}]$
by *blast*

have $i\text{-r-key}: \text{event-key } (\text{clean-prefix-of } R \ ! \ i\text{-r}) = k$
using $i\text{-r-at}$ **by** *simp*
have $i\text{-r-le-i}: i\text{-r} \leq i$
proof $(\text{rule } c\text{contr})$
assume $\neg i\text{-r} \leq i$
hence $i\text{-lt-ir}: i < i\text{-r}$ **by** *simp*
from $i\text{-latest } i\text{-lt-ir } i\text{-r-lt}$
have $\text{event-key } (\text{clean-prefix-of } R \ ! \ i\text{-r}) \neq k$ **by** *blast*
with $i\text{-r-key}$ **show** *False* **by** *simp*

qed

have $i\text{-r-ne-i}: i\text{-r} \neq i$

proof

assume $i\text{-r} = i$
with $i\text{-r-at } cp\text{-i}$ **show** *False* **by** *simp*

qed

```

have i-r-lt-i: i-r < i
  using i-r-le-i i-r-ne-i by linarith
have coord-le: src-le c-read c
  using clean-prefix-of-order-respects-coordinate[OF wf i-r-lt-i i-lt]
    i-r-at cp-i
  by simp
have c-read-ne-c: c-read ≠ c
proof
  assume c-eq: c-read = c
  have clean-prefix-of R ! i-r = Refresh k m c
    using i-r-at c-eq by simp
  from clean-prefix-same-coord-refresh-before-cdc-false[OF i-r-lt-i i-lt this cp-i]
  show False .
qed
have read-lt: src-lt (chunk-read-coordinate R ch) c
  using coord-le c-read-ne-c read-eq
  unfolding src-lt-def by simp
show ?thesis using rc read-lt by blast
qed

```

lemma *latest-cdc-no-source-event-strictly-after-coord:*

```

fixes b0 :: 'k :: linorder → 'v
  and H :: ('k, 'v) src-history
  and R :: 'run
  and k :: 'k
  and ch :: 'chunk
  and i :: nat
  and c :: src-coord
  and e :: ('k, 'v) source-event
assumes wf: wellformed-dblog-run b0 R H
  and k-scope: k ∈ scope-of R
  and rc: responsible-chunk R k = Some ch
  and read-lt-c: src-lt (chunk-read-coordinate R ch) c
  and i-lt: i < length (clean-prefix-of R)
  and cp-i: clean-prefix-of R ! i = Cdc c e
  and key-e: key-of e = k
  and i-latest:
    ∀ j. i < j ∧ j < length (clean-prefix-of R)
      → event-key (clean-prefix-of R ! j) ≠ k
shows ∀ j-src < length H.
  src-lt c (hist-coord (H ! j-src))
  ∧ src-le (hist-coord (H ! j-src)) (frontier-of R)
  → key-of (hist-event (H ! j-src)) ≠ k
proof (intro allI impI)
  note wf-body = wf[unfolded wellformed-dblog-run-def]
  from wf-body have src-hist-eq: src-history-of R = H
    by (elim conjE)
  from wf-body have wf2-cov-all:

```

$\forall k \in \text{scope-of } R. \text{ covers-ordinary-cdc } R \ k \ (\text{frontier-of } R)$
by (*elim conjE*)
from *wf-body* **have** *wf7-fwd*:
 $\forall p \in \text{set } (\text{cdc-events-of } R).$
 $\text{key-of } (\text{hist-event } p) \in \text{scope-of } R$
 $\wedge \text{src-le } (\text{hist-coord } p) \ (\text{frontier-of } R)$
 $\longrightarrow \text{Cdc } (\text{hist-coord } p) \ (\text{hist-event } p)$
 $\in \text{set } (\text{clean-prefix-of } R)$
by (*elim conjE*)

from *wf2-cov-all k-scope*
have *cov-k*: *covers-ordinary-cdc* $R \ k \ (\text{frontier-of } R)$ **by** *blast*
then obtain *ch-cov* **where**
rc-cov: *responsible-chunk* $R \ k = \text{Some } \text{ch-cov}$ **and**
cov-body:
 $\forall i. i < \text{length } (\text{src-history-of } R)$
 $\longrightarrow \text{src-lt } (\text{chunk-read-coordinate } R \ \text{ch-cov})$
 $\quad (\text{hist-coord } (\text{src-history-of } R \ ! \ i))$
 $\longrightarrow \text{src-le } (\text{hist-coord } (\text{src-history-of } R \ ! \ i)) \ (\text{frontier-of } R)$
 $\longrightarrow \text{key-of } (\text{hist-event } (\text{src-history-of } R \ ! \ i)) = k$
 $\longrightarrow \text{src-history-of } R \ ! \ i \in \text{set } (\text{cdc-events-of } R)$
unfolding *covers-ordinary-cdc-def* **by** *blast*
have *ch-cov-eq*: *ch-cov* = *ch*
using *rc-cov rc* **by** *simp*

fix *j-src*
assume *j-lt*: *j-src* < *length H*
assume *j-after-le-f*:
 $\text{src-lt } c \ (\text{hist-coord } (H \ ! \ j\text{-src}))$
 $\wedge \text{src-le } (\text{hist-coord } (H \ ! \ j\text{-src})) \ (\text{frontier-of } R)$
hence *j-after*: $\text{src-lt } c \ (\text{hist-coord } (H \ ! \ j\text{-src}))$
and *j-le-f*: $\text{src-le } (\text{hist-coord } (H \ ! \ j\text{-src})) \ (\text{frontier-of } R)$
by *auto*
show $\text{key-of } (\text{hist-event } (H \ ! \ j\text{-src})) \neq k$
proof (*rule ccontr*)
assume $\neg \text{key-of } (\text{hist-event } (H \ ! \ j\text{-src})) \neq k$
hence *j-key*: $\text{key-of } (\text{hist-event } (H \ ! \ j\text{-src})) = k$ **by** *simp*

have *read-lt-j*:
 $\text{src-lt } (\text{chunk-read-coordinate } R \ \text{ch}) \ (\text{hist-coord } (H \ ! \ j\text{-src}))$
proof –
have *read-le-c*: $\text{src-le } (\text{chunk-read-coordinate } R \ \text{ch}) \ c$
using *read-lt-c* **unfolding** *src-lt-def* **by** *simp*
have *read-ne-c*: $\text{chunk-read-coordinate } R \ \text{ch} \neq c$
using *read-lt-c* **unfolding** *src-lt-def* **by** *simp*
have *c-le-j*: $\text{src-le } c \ (\text{hist-coord } (H \ ! \ j\text{-src}))$
using *j-after* **unfolding** *src-lt-def* **by** *simp*
have *read-le-j*: $\text{src-le } (\text{chunk-read-coordinate } R \ \text{ch}) \ (\text{hist-coord } (H \ ! \ j\text{-src}))$
using *read-le-c c-le-j src-le-trans* **by** *blast*

```

have read-ne-j: chunk-read-coordinate R ch  $\neq$  hist-coord (H ! j-src)
proof
  assume eq: chunk-read-coordinate R ch = hist-coord (H ! j-src)
  hence src-le c (chunk-read-coordinate R ch)
  using c-le-j by simp
  hence chunk-read-coordinate R ch = c
  using read-le-c src-le-antisym by blast
  with read-ne-c show False by simp
qed
show ?thesis unfolding src-lt-def using read-le-j read-ne-j by simp
qed
have j-lt-R: j-src < length (src-history-of R)
  using j-lt src-hist-eq by simp
have j-after-R:
  src-lt (chunk-read-coordinate R ch-cov)
    (hist-coord (src-history-of R ! j-src))
  using read-lt-j ch-cov-eq src-hist-eq by simp
have j-le-f-R:
  src-le (hist-coord (src-history-of R ! j-src)) (frontier-of R)
  using j-le-f src-hist-eq by simp
have j-key-R:
  key-of (hist-event (src-history-of R ! j-src)) = k
  using j-key src-hist-eq by simp

from cov-body j-lt-R j-after-R j-le-f-R j-key-R
have hjs-in-cdc-R: src-history-of R ! j-src  $\in$  set (cdc-events-of R)
  by blast
have hjs-in-cdc: H ! j-src  $\in$  set (cdc-events-of R)
  using hjs-in-cdc-R src-hist-eq by simp
have key-in-scope-j: key-of (hist-event (H ! j-src))  $\in$  scope-of R
  using j-key k-scope by simp
from wf7-fwd hjs-in-cdc key-in-scope-j j-le-f
have cdc-in-cp:
  Cdc (hist-coord (H ! j-src)) (hist-event (H ! j-src))
     $\in$  set (clean-prefix-of R)
  by blast
then obtain i-h where
  i-h-lt: i-h < length (clean-prefix-of R) and
  i-h-at:
    clean-prefix-of R ! i-h
      = Cdc (hist-coord (H ! j-src)) (hist-event (H ! j-src))
  by (auto simp: in-set-conv-nth)
have i-h-key: event-key (clean-prefix-of R ! i-h) = k
  using i-h-at j-key by simp

have i-h-le-i: i-h  $\leq$  i
proof (rule ccontr)
  assume  $\neg$  i-h  $\leq$  i
  hence i-lt-ih: i < i-h by simp

```

```

    from i-latest i-lt-ih i-h-lt
    have event-key (clean-prefix-of R ! i-h) ≠ k by blast
    with i-h-key show False by simp
qed

consider (eq) i-h = i | (lt) i-h < i
  using i-h-le-i by linarith
thus False
proof cases
  case eq
  have c-eq-j: c = hist-coord (H ! j-src)
    using cp-i i-h-at eq by simp
  with j-after show False
    unfolding src-lt-def by simp
next
  case lt
  from clean-prefix-of-order-respects-coordinate[OF wf lt i-lt]
  have src-le (cdc-coord (clean-prefix-of R ! i-h))
    (cdc-coord (clean-prefix-of R ! i)) .
  hence src-le (hist-coord (H ! j-src)) c
    using i-h-at cp-i by simp
  with j-after show False
    unfolding src-lt-def
    using src-le-antisym by blast
qed
qed
qed

```

lemma *latest-cdc-event-last-in-clean-prefix-key-coord-slice:*

```

fixes R :: 'run
and k :: 'k
and i :: nat
and c :: src-coord
and e :: ('k, 'v) source-event
assumes i-lt: i < length (clean-prefix-of R)
and cp-i: clean-prefix-of R ! i = Cdc c e
and key-e: key-of e = k
and i-latest:
  ∀ j. i < j ∧ j < length (clean-prefix-of R)
    ⟶ event-key (clean-prefix-of R ! j) ≠ k
shows let slice =
  filter
    (λx. case x of
      Cdc c' e' ⇒ c' = c ∧ key-of e' = k
      | Refresh - - ⇒ False)
    (clean-prefix-of R)
  in slice ≠ [] ∧ last slice = Cdc c e
proof -

```

```

let ?P = λx::('k, 'v) replay-event.
    case x of Cdc c' e' ⇒ c' = c ∧ key-of e' = k
    | Refresh - - - ⇒ False
have P-i: ?P (clean-prefix-of R ! i)
    using cp-i key-e by simp
have decomp:
    clean-prefix-of R =
      take i (clean-prefix-of R)
      @ [clean-prefix-of R ! i]
      @ drop (Suc i) (clean-prefix-of R)
    using i-lt
    by (metis Cons-nth-drop-Suc append-Cons append-Nil append-take-drop-id)
have drop-no-P: filter ?P (drop (Suc i) (clean-prefix-of R)) = []
proof -
    have ∀ x ∈ set (drop (Suc i) (clean-prefix-of R)). ¬ ?P x
    proof
      fix x
      assume x-in: x ∈ set (drop (Suc i) (clean-prefix-of R))
      then obtain j where
        j-lt-drop: j < length (drop (Suc i) (clean-prefix-of R))
        and x-at: drop (Suc i) (clean-prefix-of R) ! j = x
        by (auto simp: in-set-conv-nth)
      let ?j' = Suc (i + j)
      have j'-lt: ?j' < length (clean-prefix-of R)
        using j-lt-drop by simp
      have x-eq: x = clean-prefix-of R ! ?j'
        using x-at j-lt-drop by (simp add: add.commute)
      have i-lt-j': i < ?j' by simp
      from i-latest i-lt-j' j'-lt
      have later-not-k: event-key (clean-prefix-of R ! ?j') ≠ k by blast
      show ¬ ?P x
    proof
      assume Px: ?P x
      hence event-key x = k
        by (cases x) auto
      hence event-key (clean-prefix-of R ! ?j') = k
        using x-eq by simp
      with later-not-k show False by simp
    qed
  qed
thus ?thesis by (simp add: filter-empty-conv)
qed
have slice-eq:
    filter ?P (clean-prefix-of R)
    = filter ?P (take i (clean-prefix-of R)) @ [Cdc c e]
proof -
    have filter ?P (clean-prefix-of R)
      = filter ?P
        (take i (clean-prefix-of R)

```



```

      @ [clean-prefix-of R ! i]
      @ drop (Suc i) (clean-prefix-of R))
    using decomp by simp
  also have ...
    = filter ?P (take i (clean-prefix-of R))
      @ filter ?P [clean-prefix-of R ! i]
      @ filter ?P (drop (Suc i) (clean-prefix-of R))
    by simp
  also have ... = filter ?P (take i (clean-prefix-of R)) @ [Cdc c e]
    using P-i cp-i drop-no-P by simp
  finally show ?thesis .
qed
show ?thesis
  using slice-eq by simp
qed

```

8.8 Per-key replay equality: the Cdc case

lemma *clean-prefix-of-per-key-replay-equals-source-cdc-case*:

```

  fixes b0 :: 'k :: linorder  $\rightarrow$  'v
    and H :: ('k, 'v) src-history
    and R :: 'run
    and k :: 'k
    and i-max :: nat
    and c :: src-coord
    and e :: ('k, 'v) source-event
  assumes wf: wellformed-dblog-run b0 R H
    and k-in-scope:  $k \in \text{scope-of } R$ 
    and i-max-lt:  $i\text{-max} < \text{length } (\text{clean-prefix-of } R)$ 
    and cp-at-imax:  $\text{clean-prefix-of } R ! i\text{-max} = \text{Cdc } c \ e$ 
    and key-e:  $\text{key-of } e = k$ 
    and i-max-latest:
       $\forall j. i\text{-max} < j \wedge j < \text{length } (\text{clean-prefix-of } R)$ 
         $\rightarrow \text{event-key } (\text{clean-prefix-of } R ! j) \neq k$ 
    and c-le-f:  $\text{src-le } c \ (\text{frontier-of } R)$ 
  shows  $\text{Apply } (\text{clean-prefix-of } R) \ k = \text{Src } b0 \ H \ (\text{frontier-of } R) \ k$ 
proof -
  note wf-body = wf[unfolded wellformed-dblog-run-def]
  from wf-body have src-hist-eq:  $\text{src-history-of } R = H$ 
    by (elim conjE)
  from wf-body have wf-h:  $\text{wellformed-src-history } H$ 
    by (elim conjE)

  let ?E =
    (case e of Insert - v  $\Rightarrow$  Some v
      | Update - v  $\Rightarrow$  Some v
      | Delete -  $\Rightarrow$  None)

  have ev-key-imax:  $\text{event-key } (\text{clean-prefix-of } R ! i\text{-max}) = k$ 

```

using *cp-at-imax key-e* **by** *simp*
have *apply-val*: *Apply (clean-prefix-of R) k = ?E*
proof –
have *Apply (clean-prefix-of R) k =*
(case clean-prefix-of R ! i-max of
Cdc - (Insert - v) $\Rightarrow$ Some v
| Cdc - (Update - v) $\Rightarrow$ Some v
| Cdc - (Delete -) $\Rightarrow$ None
| Refresh - m-obs - $\Rightarrow$ m-obs)
by (*rule apply-at-latest-k-event*[*OF i-max-lt ev-key-imax i-max-latest*])
also have ... = *?E*
using *cp-at-imax* **by** (*cases e*) *auto*
finally show *?thesis* .
qed

obtain *ch* **where**
rc: *responsible-chunk R k = Some ch* **and**
read-lt-c: *src-lt (chunk-read-coordinate R ch) c*
using *latest-cdc-for-key-implies-chunk-read-strictly-before*
[OF wf k-in-scope i-max-lt cp-at-imax key-e i-max-latest]
by *blast*

have *no-later-src*:
 $\forall j < \text{length } H.$
src-lt c (hist-coord (H ! j))
 $\wedge$ *src-le (hist-coord (H ! j)) (frontier-of R)*
 $\longrightarrow$ *key-of (hist-event (H ! j)) $\neq$ k*
by (*rule latest-cdc-no-source-event-strictly-after-coord*
[OF wf k-in-scope rc read-lt-c i-max-lt cp-at-imax key-e i-max-latest])

have *slice-eq-R*:
source-key-coord-slice k c (cdc-events-of R)
 $=$ *source-key-coord-slice k c (src-history-of R)*
by (*rule cdc-events-key-coord-slice-eq-source-history-slice-from-wf*
[OF wf k-in-scope rc read-lt-c c-le-f])

let *?cp-slice* =
filter
($\lambda x::('k, 'v)$ replay-event.
case x of
Cdc c' e' $\Rightarrow$ c' = c $\wedge$ key-of e' = k
| Refresh - - $\Rightarrow$ False)
(clean-prefix-of R)
let *?cdc-slice* = *source-key-coord-slice k c (cdc-events-of R)*
let *?hist-R-slice* = *source-key-coord-slice k c (src-history-of R)*
let *?hist-H-slice* = *source-key-coord-slice k c H*

have *cp-slice-nonempty*: *?cp-slice $\neq$ []*
and *cp-slice-last*: *last ?cp-slice = Cdc c e*

```

using latest-cdc-event-last-in-clean-prefix-key-coord-slice
  [OF i-max-lt cp-at-imax key-e i-max-latest]
by (simp-all add: Let-def)
have cp-slice-map:
  ?cp-slice =
    map (λp. Cdc (hist-coord p) (hist-event p)) ?cdc-slice
by (rule clean-prefix-cdc-key-coord-slice[OF k-in-scope c-le-f])
have cdc-slice-nonempty: ?cdc-slice ≠ []
using cp-slice-nonempty cp-slice-map by auto
have last-map:
  last ?cp-slice =
    Cdc (hist-coord (last ?cdc-slice)) (hist-event (last ?cdc-slice))
proof -
  let ?f = λp. Cdc (hist-coord p) (hist-event p)
  have cdc-decomp: ?cdc-slice = butlast ?cdc-slice @ [last ?cdc-slice]
    using cdc-slice-nonempty by simp
  have map-decomp:
    map ?f ?cdc-slice = map ?f (butlast ?cdc-slice) @ [?f (last ?cdc-slice)]
    by (subst cdc-decomp, simp)
  have last ?cp-slice = last (map ?f ?cdc-slice)
    using cp-slice-map by simp
  also have ... = last (map ?f (butlast ?cdc-slice) @ [?f (last ?cdc-slice)])
    using map-decomp by simp
  also have ... = ?f (last ?cdc-slice)
    by simp
  finally show ?thesis .
qed
have last-cdc-slice: last ?cdc-slice = (c, e)
proof -
  have coord-eq: hist-coord (last ?cdc-slice) = c
    and event-eq: hist-event (last ?cdc-slice) = e
    using last-map cp-slice-last by simp-all
  show ?thesis
    using coord-eq event-eq by (cases last ?cdc-slice) simp
qed

have hist-R-nonempty: ?hist-R-slice ≠ []
  using slice-eq-R cdc-slice-nonempty by simp
have hist-R-last: last ?hist-R-slice = (c, e)
  using slice-eq-R last-cdc-slice by simp
have hist-H-nonempty: ?hist-H-slice ≠ []
  using hist-R-nonempty src-hist-eq by simp
have hist-H-last: last ?hist-H-slice = (c, e)
  using hist-R-last src-hist-eq by simp

have src-val: Src b0 H (frontier-of R) k = ?E
  by (rule src-frontier-eq-effect-of-last-key-coord-slice
    [OF wf-h c-le-f no-later-src hist-H-nonempty hist-H-last])

```

show *?thesis* **using** *apply-val src-val* **by** *simp*
qed

8.9 Per-key replay equality: main lemma and the Refresh case

lemma *clean-prefix-of-per-key-replay-equals-source*:

fixes *b0* :: 'k :: linorder $\rightarrow$ 'v

and *H* :: ('k, 'v) *src-history*

and *R* :: 'run

and *k* :: 'k

assumes *wf*: *wellformed-dblog-run* *b0* *R* *H*

and *k-in-scope*: *k* $\in$ *scope-of* *R*

shows *Apply* (*clean-prefix-of* *R*) *k* = *Src* *b0* *H* (*frontier-of* *R*) *k*

proof —

— Step 1: extract the WF body conjuncts this lemma consumes. Using the pattern *wf-body* = *wf*[*unfolded* ...] + (*elim conjE*), individual conjuncts are extracted one by one rather than as a compound — a single *elim conjE* pass per conjunct suffices, since it also breaks inner conjunctions such as WF0's *src-history-of* *R* = *H* $\wedge$ *wellformed-src-history* *H*.

note *wf-body* = *wf*[*unfolded wellformed-dblog-run-def*]

from *wf-body* **have** *src-hist-eq*: *src-history-of* *R* = *H*

by (*elim conjE*)

from *wf-body* **have** *wf2-cov-all*:

$\forall$ *k* $\in$ *scope-of* *R*. *covers-ordinary-cdc* *R* *k* (*frontier-of* *R*)

by (*elim conjE*)

from *wf-body* **have** *wf5*:

$\forall$ *ch* $\in$ *chunks* *R*.

src-le (*chunk-read-coordinate* *R* *ch*) (*frontier-of* *R*)

by (*elim conjE*)

from *wf-body* **have** *wf7-fwd*:

$\forall$ *p* $\in$ *set* (*cdc-events-of* *R*).

key-of (*hist-event* *p*) $\in$ *scope-of* *R*

$\wedge$ *src-le* (*hist-coord* *p*) (*frontier-of* *R*)

$\rightarrow$ *Cdc* (*hist-coord* *p*) (*hist-event* *p*)

$\in$ *set* (*clean-prefix-of* *R*)

by (*elim conjE*)

— Extract *covers-ordinary-cdc* *R* *k* (*frontier-of* *R*) for *this* *k*.

from *wf2-cov-all* *k-in-scope*

have *cov-k*: *covers-ordinary-cdc* *R* *k* (*frontier-of* *R*) **by** *blast*

then obtain *ch-k* **where**

rc-k: *responsible-chunk* *R* *k* = *Some* *ch-k* **and**

cov-k-body:

$\forall$ *i*. *i* < *length* (*src-history-of* *R*)

$\rightarrow$ *src-lt* (*chunk-read-coordinate* *R* *ch-k*)

(*hist-coord* (*src-history-of* *R* ! *i*))

$\rightarrow$ *src-le* (*hist-coord* (*src-history-of* *R* ! *i*)) (*frontier-of* *R*)

$\rightarrow$ *key-of* (*hist-event* (*src-history-of* *R* ! *i*)) = *k*

$\longrightarrow \text{src-history-of } R ! i \in \text{set } (\text{cdc-events-of } R)$
unfolding *covers-ordinary-cdc-def* **by** *blast*

— Step 2: invoke *clean-prefix-of-latest-replay-correspondence* to obtain the Cdc-or-Refresh disjunction at the latest k-event position in *clean-prefix-of*(*R*).

note *latest-replay-disj* = *clean-prefix-of-latest-replay-correspondence*[*OF wf k-in-scope*]

— Step 3: case-split on the *clean-prefix-of-latest-replay-correspondence* disjunction.

show *?thesis*

proof (*rule disjE*[*OF latest-replay-disj*])

— **Cdc case:** latest k-event is a Cdc(*c*, *e*).

assume *cdc-disj*:

$\exists i \ c \ e. \ i < \text{length } (\text{clean-prefix-of } R)$
 $\wedge \text{clean-prefix-of } R ! i = \text{Cdc } c \ e$
 $\wedge \text{key-of } e = k$
 $\wedge (\forall j. \ i < j \wedge j < \text{length } (\text{clean-prefix-of } R)$
 $\longrightarrow \text{event-key } (\text{clean-prefix-of } R ! j) \neq k)$
 $\wedge (c, e) \in \text{set } (\text{cdc-events-of } R)$
 $\wedge \text{src-le } c \ (\text{frontier-of } R)$

then obtain *i-max c e* **where**

i-max-lt: $i\text{-max} < \text{length } (\text{clean-prefix-of } R)$ **and**
cp-at-imax: $\text{clean-prefix-of } R ! i\text{-max} = \text{Cdc } c \ e$ **and**
key-e: $\text{key-of } e = k$ **and**
i-max-latest:
 $\forall j. \ i\text{-max} < j \wedge j < \text{length } (\text{clean-prefix-of } R)$
 $\longrightarrow \text{event-key } (\text{clean-prefix-of } R ! j) \neq k$ **and**
ce-in-cdc: $(c, e) \in \text{set } (\text{cdc-events-of } R)$ **and**
c-le-f: $\text{src-le } c \ (\text{frontier-of } R)$
by *blast*

— Cdc case: delegated to the standalone helper lemma above.

show *Apply* (*clean-prefix-of* *R*) *k* = *Src b0 H* (*frontier-of* *R*) *k*

by (*rule clean-prefix-of-per-key-replay-equals-source-cdc-case*
 $[\text{OF wf } k\text{-in-scope } i\text{-max-lt } cp\text{-at-imax } key\text{-e } i\text{-max-latest } c\text{-le-f}])$

next

— **Refresh case:** latest k-event is a Refresh(*k*, *m*, *c-read*).

assume *refresh-disj*:

$\exists i \ ch \ m \ c\text{-read}.$
 $i < \text{length } (\text{clean-prefix-of } R)$
 $\wedge \text{clean-prefix-of } R ! i = \text{Refresh } k \ m \ c\text{-read}$
 $\wedge (\forall j. \ i < j \wedge j < \text{length } (\text{clean-prefix-of } R)$
 $\longrightarrow \text{event-key } (\text{clean-prefix-of } R ! j) \neq k)$
 $\wedge \text{responsible-chunk } R \ k = \text{Some } ch$
 $\wedge \text{chunk-read-result } R \ ch \ k = \text{Some } m$
 $\wedge \text{chunk-read-coordinate } R \ ch = c\text{-read}$

then obtain *i-max ch m c-read* **where**

i-max-lt: $i\text{-max} < \text{length } (\text{clean-prefix-of } R)$ **and**

cp-at-imax: *clean-prefix-of* R ! *i-max* = *Refresh* k m *c-read* **and**
i-max-latest:
 $\forall j. i-max < j \wedge j < \text{length}(\text{clean-prefix-of } R)$
 $\longrightarrow \text{event-key}(\text{clean-prefix-of } R ! j) \neq k$ **and**
rc-eq: *responsible-chunk* R k = *Some* ch **and**
crr-res: *chunk-read-result* R ch k = *Some* m **and**
cr-eq: *chunk-read-coordinate* R ch = *c-read*
by *blast*

— Refresh case Step 1: $ch-k = ch$ (uniqueness via *Some*-injectivity).
have *ch-eq*: $ch = ch-k$
using *rc-eq* *rc-k* **by** *simp*

— Refresh case Step 2: $ch \in \text{chunks } R + k \in \text{chunk-domain } R$ ch via WF1 (d)
 (with the *responsible-chunk* fact from the case split).

from *wf-body* **have** *wf1d*:
 $\forall k \in \text{scope-of } R. \forall ch.$
 $\text{responsible-chunk } R$ k = *Some* ch
 $\longleftrightarrow ch \in \text{chunks } R \wedge \text{owns } R$ ch k
by (*elim conjE*)
from *wf1d* *k-in-scope* *rc-eq*
have *ch-props*: $ch \in \text{chunks } R \wedge \text{owns } R$ ch k **by** *blast*
hence *ch-in-chunks*: $ch \in \text{chunks } R$ **and** *k-in-dom*: $k \in \text{chunk-domain } R$ ch
by *auto*

— Refresh case Step 3: Apply value via *apply-at-latest-k-event*.

have *ev-key-imax*: *event-key* (*clean-prefix-of* R ! *i-max*) = k
using *cp-at-imax* **by** *simp*
have *apply-val*: *Apply* (*clean-prefix-of* R) k = m
proof —
have *Apply* (*clean-prefix-of* R) k =
 (*case clean-prefix-of* R ! *i-max* of
 $Cdc - (\text{Insert} - v) \Rightarrow \text{Some } v$
 $| Cdc - (\text{Update} - v) \Rightarrow \text{Some } v$
 $| Cdc - (\text{Delete} -) \Rightarrow \text{None}$
 $| \text{Refresh} - m\text{-obs} - \Rightarrow m\text{-obs}$)
by (*rule apply-at-latest-k-event* [*OF* *i-max-lt* *ev-key-imax* *i-max-latest*])
also have $\dots = m$
using *cp-at-imax* **by** *simp*
finally show *?thesis* .
qed

— Refresh case Step 4: $m = \text{Src } b0$ H *c-read* k via *refresh-coordinate-match*.

have *m-eq-src-at-cr*: $m = \text{Src } b0$ H *c-read* k
proof —
from *refresh-coordinate-match* [*OF* *wf* *ch-in-chunks* *k-in-dom* *crr-res*]
have $m = \text{Src } b0$ H (*chunk-read-coordinate* R ch) k .
thus *?thesis* **using** *cr-eq* **by** *simp*
qed

— Refresh case Step 5: no source event for k in $(c\text{-read}, \text{frontier}]$.
have *no-later-src*:
 $\forall j\text{-src} < \text{length } H. \text{src-lt } c\text{-read } (\text{hist-coord } (H ! j\text{-src}))$
 $\quad \wedge \text{src-le } (\text{hist-coord } (H ! j\text{-src})) (\text{frontier-of } R)$
 $\quad \longrightarrow \text{key-of } (\text{hist-event } (H ! j\text{-src})) \neq k$
proof (*intro allI impI*)
fix $j\text{-src}$
assume $j\text{-lt}: j\text{-src} < \text{length } H$
assume $j\text{-after-le-f}$:
 $\text{src-lt } c\text{-read } (\text{hist-coord } (H ! j\text{-src}))$
 $\quad \wedge \text{src-le } (\text{hist-coord } (H ! j\text{-src})) (\text{frontier-of } R)$
hence $j\text{-after}$: $\text{src-lt } c\text{-read } (\text{hist-coord } (H ! j\text{-src}))$
and $j\text{-le-f}$: $\text{src-le } (\text{hist-coord } (H ! j\text{-src})) (\text{frontier-of } R)$
by *auto*
show $\text{key-of } (\text{hist-event } (H ! j\text{-src})) \neq k$
proof (*rule ccontr*)
assume $\neg \text{key-of } (\text{hist-event } (H ! j\text{-src})) \neq k$
hence $j\text{-key}$: $\text{key-of } (\text{hist-event } (H ! j\text{-src})) = k$ **by** *simp*

— $\text{chunk-read-coordinate } R \text{ ch-}k = \text{chunk-read-coordinate } R \text{ ch} = c\text{-read}$.
have crk-eq : $\text{chunk-read-coordinate } R \text{ ch-}k = c\text{-read}$
using ch-eq cr-eq **by** *simp*
have $j\text{-after-crk}$:
 $\text{src-lt } (\text{chunk-read-coordinate } R \text{ ch-}k) (\text{hist-coord } (H ! j\text{-src}))$
using $\text{crk-eq } j\text{-after}$ **by** *simp*

— Translate $j\text{-src}$'s H properties to $\text{src-history-of } R$ via src-hist-eq .
have $j\text{-lt-R}$: $j\text{-src} < \text{length } (\text{src-history-of } R)$
using $j\text{-lt src-hist-eq}$ **by** *simp*
have $j\text{-after-R}$:
 $\text{src-lt } (\text{chunk-read-coordinate } R \text{ ch-}k)$
 $\quad (\text{hist-coord } (\text{src-history-of } R ! j\text{-src}))$
using $j\text{-after-crk src-hist-eq}$ **by** *simp*
have $j\text{-le-f-R}$:
 $\text{src-le } (\text{hist-coord } (\text{src-history-of } R ! j\text{-src})) (\text{frontier-of } R)$
using $j\text{-le-f src-hist-eq}$ **by** *simp*
have $j\text{-key-R}$:
 $\text{key-of } (\text{hist-event } (\text{src-history-of } R ! j\text{-src})) = k$
using $j\text{-key src-hist-eq}$ **by** *simp*

— $\text{covers-ordinary-cdc}: H!j\text{-src} \in \text{cdc-events-of } R$.
from $\text{cov-k-body } j\text{-lt-R } j\text{-after-R } j\text{-le-f-R } j\text{-key-R}$
have hjs-in-cdc-R :
 $\text{src-history-of } R ! j\text{-src} \in \text{set } (\text{cdc-events-of } R)$ **by** *blast*
have hjs-in-cdc : $H ! j\text{-src} \in \text{set } (\text{cdc-events-of } R)$
using $\text{hjs-in-cdc-R src-hist-eq}$ **by** *simp*

— WF7 forward: $\text{Cdc } (\text{hist-coord } (H!j\text{-src})) (\text{hist-event } (H!j\text{-src})) \in$

clean-prefix-of R.

have *key-in-scope-j*:
 key-of (*hist-event* (*H ! j-src*)) $\in$ *scope-of R*
 using *j-key k-in-scope* **by** *simp*
from *wf7-fwd hjs-in-cdc key-in-scope-j j-le-f*
have *cdc-in-cp*:
 Cdc (*hist-coord* (*H ! j-src*)) (*hist-event* (*H ! j-src*))
 $\in$ *set* (*clean-prefix-of R*)
 by *blast*
then obtain *i-h* **where**
 i-h-lt: *i-h* < *length* (*clean-prefix-of R*) **and**
 i-h-at: *clean-prefix-of R ! i-h*
 = *Cdc* (*hist-coord* (*H ! j-src*)) (*hist-event* (*H ! j-src*))
 by (*auto simp: in-set-conv-nth*)

have *i-h-key*: *event-key* (*clean-prefix-of R ! i-h*) = *k*
 using *i-h-at j-key* **by** *simp*

— *i-h* $\leq$ *i-max* (else contradicts *i-max-latest*).

have *i-h-le-imax*: *i-h* $\leq$ *i-max*

proof (*rule ccontr*)

assume $\neg$ *i-h* $\leq$ *i-max*

hence *imax-lt-ih*: *i-max* < *i-h* **by** *simp*

from *i-max-latest imax-lt-ih i-h-lt* **have** *event-key* (*clean-prefix-of R ! i-h*)

$\neq k$

by *blast*

with *i-h-key* **show** *False* **by** *simp*

qed

— Three sub-cases on *i-h* vs *i-max*.

consider (*eq*) *i-h* = *i-max* | (*lt*) *i-h* < *i-max*

using *i-h-le-imax* **by** *linarith*

thus *False*

proof *cases*

case *eq*

 — *clean-prefix-of R ! i-max* = *Refresh k m c-read* and = *Cdc* (...). Distinct constructors, contradiction.

from *cp-at-imax i-h-at eq* **show** *False* **by** *simp*

next

case *lt*

 — By *clean-prefix-of-order-respects-coordinate* + *cdc-coord, src-le* (*hist-coord* (*H!j-src*)) *c-read*. But *src-lt c-read* (*hist-coord* (*H!j-src*)). Antisymmetry.

from *clean-prefix-of-order-respects-coordinate[OF wf lt i-max-lt]*

have *src-le* (*cdc-coord* (*clean-prefix-of R ! i-h*))
 (*cdc-coord* (*clean-prefix-of R ! i-max*)) .

hence *src-le* (*hist-coord* (*H ! j-src*)) *c-read*

using *i-h-at cp-at-imax* **by** *simp*

with *j-after* **show** *False*

unfolding *src-lt-def*


```

      using src-le-antisym by blast
    qed
  qed
qed

— Refresh case Step 6:  $c\text{-read} \leq \text{frontier}$  via WF5.
have c-read-le-f: src-le c-read (frontier-of R)
proof —
  from wf5 ch-in-chunks
  have src-le (chunk-read-coordinate R ch) (frontier-of R) by blast
  thus ?thesis using cr-eq by simp
qed

— Refresh case Step 7:  $\text{Src } b0 \ H \ \text{frontier } k = \text{Src } b0 \ H \ c\text{-read } k$ .
have src-eq:
  Src b0 H (frontier-of R) k = Src b0 H c-read k
  by (rule src-eq-when-no-later-src-event-for-k[OF c-read-le-f no-later-src])

— Refresh case Step 8: assemble  $\text{Apply} = m = \text{Src } b0 \ H \ c\text{-read } k = \text{Src } b0 \ H$ 
frontier k.
show Apply (clean-prefix-of R) k = Src b0 H (frontier-of R) k
  using apply-val m-eq-src-at-cr src-eq by simp
qed
qed

```

8.10 Layer 2 run-side soundness (locale form of the paper's bridge lemma)

The run-side instance of *virtual-cut-state*: a wellformed DBLog run replays its clean prefix to the source state at its frontier on its scope. The public theorem *Virtual-Cut.wellformed-run-implies-virtual-cut* is this theorem specialized through the canonical interpretation (its conclusion uses the public *Virtual-Cut.virtual-cut-state*, which coincides with the body-identical *virtual-cut-state* used here). Proved by unfolding *virtual-cut-state* and applying the locale per-key equality *clean-prefix-of-per-key-replay-equals-source* pointwise.

```

theorem wellformed-run-implies-virtual-cut:
  assumes wellformed-dblog-run b0 R H
  shows
    virtual-cut-state b0 (clean-prefix-of R) (scope-of R)
      (frontier-of R) H
  unfolding virtual-cut-state-def restrict-def
  by (intro ext)
    (auto intro: clean-prefix-of-per-key-replay-equals-source[OF assms])

end

end

```

```

theory Virtual-Cut
  imports
    Dual-Write-Layer0.Source-History
    Dual-Write-Layer0.Replay
    Dual-Write-Layer0.Scope
    DBLog-Run-Substrate-Layer2
begin

```

9 Virtual cuts, certificates, and the Layer 2/3 theorems

Central formal object of the paper: the *virtual-cut* predicate over certificates.

The virtual cut is a three-argument predicate *virtual-cut* *b0* *C* *H*: under a deployment base state *b0*, certificate *C*, and source history *H*, the replay of *clean-prefix* *C* on *scope* *C* equals the source state at *frontier* *C*. It is the certificate-accessor instance of the factored source-side core predicate *virtual-cut-state* (below), which both this cert-side definition and the Layer 2 run-side soundness theorem reduce to.

The reader-facing paper notation suppresses the universally-quantified base state: the paper writes *virtual-cut*(*C*, *H*) where the formal predicate is *virtual-cut* *b0* *C* *H*.

This theory also assembles the Layer 3 checker substrate: the certificate carrier and accessors, the *verify-result* enumeration (*Accept* | *Reject* | *Unsupported*), the evidence and observation primitives, and the *materializes-run* relation (the graph of the evidence-determined run) support the *layer3-checker-substrate* locale, under whose primitive checker obligations the composed soundness theorem *accepted-virtual-cut-sound* is proved.

Certificate accessors are bare-named (*scope* *C*, *frontier* *C*, *clean-prefix* *C*); run accessors, in *DBLog-Run*, carry the *-of* suffix.

9.1 Certificate carrier and accessors

Certificate accessors are bare-named — *scope* *C*, *frontier* *C*, *clean-prefix* *C* — to mirror the paper’s notation. Run accessors live in *DBLog-Run* with the *-of* suffix.

The certificate carrier is a concrete single-constructor **datatype** with named selectors under the public name *cert*; the bare-named accessors are its field projections. (A **datatype** rather than a **record**, so *cert* stays a clean two-parameter type constructor rather than an extensible-record synonym. The Layer 3/4 substrate development abstracts the certificate over a locale parameter in *DBLog-Cert-Substrate*; this public datatype is the concrete carrier over which the headline theorems below are stated.)

datatype ('k, 'v) cert =
 Cert (c-scope: 'k set)
 (c-frontier: frontier)
 (c-clean-prefix: ('k, 'v) replay-event list)

definition scope :: ('k, 'v) cert $\Rightarrow$ 'k set
 where scope C = c-scope C

definition frontier :: ('k, 'v) cert $\Rightarrow$ frontier
 where frontier C = c-frontier C

definition clean-prefix :: ('k, 'v) cert $\Rightarrow$ ('k, 'v) replay-event list
 where clean-prefix C = c-clean-prefix C

9.2 Virtual cut state (shared Layer-0 predicate)

virtual-cut-state is the source-side extensional-equality core that both Layer 2 (over runs) and the Layer 3 cert-side *virtual-cut* reduce to:

$virtual-cut-state\ b0\ \sigma\ K\ f\ H \longleftrightarrow restrict\ (Apply\ \sigma)\ K = restrict\ (Src\ b0\ H\ f)\ K.$

The factoring abstracts over how σ , K , and f are obtained: certificate accessors (*clean-prefix C*, *scope C*, *frontier C*) on the cert side; run accessors (*clean-prefix-of R*, *scope-of R*, *frontier-of R*) on the run side. Both reduce to the same *virtual-cut-state* conclusion under their respective context — cert: faithful evidence, verifier accept, and cert/run coherence; run: a wellformed DBLog run.

The reader-facing paper notation suppresses the universally-quantified base state: the paper writes *virtual-cut-state*(σ , K , f , H) where the formal predicate is *virtual-cut-state b0 σ K f H* — the same suppression convention that *virtual-cut* and *Src* use.

WS4a moves *virtual-cut-state* to the shared Layer-0 theory *Dual-Write-Layer0.Virtual-Cut-State*. The compatibility lemma below is now definitional reflexivity; it keeps older proof references stable while both DBLog and Dual-Write use the same imported HOL constant.

lemma *virtual-cut-state-eq-shared*:
 $virtual-cut-state\ b0\ \sigma\ K\ f\ H = virtual-cut-state\ b0\ \sigma\ K\ f\ H$
 by *simp*

9.3 Virtual cut (cert-side instance of virtual cut state)

On *scope C*, the replay of *clean-prefix C* equals the source state at *frontier C* under base state *b0*. *virtual-cut* is expressed as the certificate-accessor instance of *virtual-cut-state*; the expanded form $Apply\ (clean-prefix\ C) \upharpoonright scope\ C = Src\ b0\ H\ (frontier\ C) \upharpoonright scope\ C$ is recovered by unfolding *virtual-cut-def* and *virtual-cut-state-def*.

definition *virtual-cut*

$:: ('k \rightarrow 'v) \Rightarrow ('k, 'v) \text{ cert} \Rightarrow ('k, 'v) \text{ src-history} \Rightarrow \text{bool}$

where

$\text{virtual-cut } b0 \ C \ H \longleftrightarrow$

$\text{virtual-cut-state } b0 \ (\text{clean-prefix } C) \ (\text{scope } C) \ (\text{frontier } C) \ H$

9.4 Layer 2 source-side soundness (over runs)

Layer 2 ranges over runs, not certificates. Its conclusion is the run-side instance of *virtual-cut-state*; the cert-side *virtual-cut* predicate is reserved for Layer 3 composition via *cert-run-coherence*. *wellformed-dblog-run*, in *DBLog-Run*, is a real **definition** with a three-argument signature and a seven-clause body (WF0 / WF1 with subclauses (a)-(g) / WF2 with coverage, faithfulness, order-preserving sublist, and post-read multiplicity conjuncts / WF4 / WF5 / WF6 main + row-absence / WF7 forward + reverse). See *DBLog-Run* for the authoritative clause list.

The paper's wellformed-run replay equality on scope (the bridge lemma of "Layer 0 and Layer 1 Lemmas", stated as the Layer 2 source-side soundness theorem):

For any base state $b0$, any wellformed source history H ,
and any

run R such that $\text{wellformed-DBLog-run}(b0, R, H)$:

$$\begin{aligned} & \text{Apply}(\text{clean_prefix_of}(R)) \ \backslash\langle \text{restriction} \rangle \ \text{scope_of}(R) \\ & = \text{Src}(b0, H, \text{frontier_of}(R)) \ \backslash\langle \text{restriction} \rangle \ \text{scope_of}(R) \end{aligned}$$

The theorem below states this conclusion as *virtual-cut-state* $b0$ (*clean-prefix-of* R) (*scope-of* R) (*frontier-of* R) H ; the expanded replay-equality form is recovered by unfolding *virtual-cut-state* $?b0.0$ $? \sigma$ $?K$ $?f$ $?H = (\text{restrict } (\text{Apply } ? \sigma) ?K = \text{restrict } (\text{Src } ?b0.0 ?H ?f) ?K)$. It is proved as a wrapper over the locale theorem *dblog-run-substrate.wellformed-run-implies-virtual-cut*, specialized through the canonical interpretation (*DBLog-Virtual-Cuts.DBLog-Run-Substrate*) and rephrased from its *virtual-cut-state* conclusion to the public *virtual-cut-state* via *virtual-cut-state-eq-shared*.

The result is *Conditional* at the paper level, on *wellformed-dblog-run* $b0$ R H ; the Isabelle theorem is proved under that premise.

theorem *wellformed-run-implies-virtual-cut*:

assumes *wellformed-dblog-run* $b0$ R H

shows

$\text{virtual-cut-state } b0 \ (\text{clean-prefix-of } R) \ (\text{scope-of } R) \ (\text{frontier-of } R) \ H$

proof –

from *assms* **have** *canonical.wellformed-dblog-run* $b0$ R H

```

  by (simp add: canonical-wellformed-dblog-run-eq)
from canonical.wellformed-run-implies-virtual-cut[OF this]
have virtual-cut-state b0 (canonical.clean-prefix-of R) (scope-of R)
    (frontier-of R) H .
thus ?thesis
  by (simp add: canonical-clean-prefix-of-eq)
qed

```

9.5 Layer 3 checker substrate: cert/evidence bridge and soundness

materializes-run witnesses that an accepted certificate, paired with faithful evidence, can be associated with an abstract DBLog run. The interface is stated relationally; the public model below instantiates it with the evidence-determined run (the run value the evidence bundle carries), and verifier acceptance plus coherence — not the bare relation alone — prevents an accepted proof from choosing an arbitrary convenient run.

raw-observation is the interface slot for external source-side observations; the public model leaves it unconstrained (a one-value carrier), and the observation content lives in the evidence run. The predicate *faithful-source-observation* $E\ b0\ H\ Raw$ ties the evidence E to a fixed deployment base state $b0$ and source history H ; the Raw parameter is unconstrained in the public model, retained for statement-shape stability with the abstract substrate (*dblog-cert-substrate*, a later theory, abstracts the observation carrier as a genuine type parameter). The predicate asserts exactly the observation-channel facts that no checker can compute from E alone (the recorded source history *is* the true history; the recorded chunk reads reflect the true source state at their read coordinates), and the assumption is discharged at deployment, not inside the model.

verify-result is the verifier’s result enumeration. The headline theorem reads *verify* $C\ E = Accept$; the result type reserves *Reject* / *Unsupported* for the rejection and parser branches that a concrete verifier distinguishes.

The Layer 3 development introduces three theorems:

- the intermediate *accepted-certificate-implies-wellformed-run* stated below (cert + evidence yields an existentially witnessed run R with *wellformed-dblog-run*);
- the cert / run coherence predicate and bridge lemma stated below; *cert-run-coherent* packages *materializes-run* with accessor agreement so the evidence parameter is load-bearing;
- the composed *accepted-virtual-cut-sound* (the paper headline, also stated below), whose proof composes *accepted-certificate-implies-wellformed-run* (existentially witnessed R) with Layer 2’s *wellformed-run-implies-virtual-cut*

on $(R, b0)$ and *cert-run-coherence* to substitute run accessors back to certificate accessors.

The non-circular proof substrate is made explicit by the *layer3-checker-substrate* locale below. The locale does not assume the accepted-certificate theorem; instead it separates four primitive checker obligations:

- accepted input is bound to the certificate and belongs to the source-side checker schema;
- accepted input has at least one materialized run witness;
- accepted materializations agree with the certificate semantic accessors; and
- faithful accepted materializations are wellformed runs against the same $b0$, H , and Raw . The theorem is then proved inside the locale from those primitive obligations.

The evidence and raw-observation carriers are concrete **datatypes** under the public names *evidence* / *raw-observation*: an evidence bundle carries the full observed *run* value (chunk plan, watermarks, read coordinates, chunk-read results, the captured CDC/source-history log) — exactly the source-side artifacts a deployment hands the checker. The checker primitives (*verify*, *faithful-source-observation*, *materializes-run*, *evidence-bound-to-cert*, *source-side-schema-supported*) are genuine definitions over this carrier, factored along the paper’s trust boundary:

- *checker-run-wellformed* is the *mechanical* half of *wellformed-dblog-run*: every clause that is a property of the evidence run alone (chunk-partition structure WF1, observed-CDC coverage / faithfulness-to-the-recorded-log / order-preservation / multiplicity WF2, chunk-read evidence WF4, read-coordinate bound WF5, clean-prefix CDC coherence WF7, and internal wellformedness of the *recorded* source history). The verifier establishes all of it from (C, E) alone; no $b0$ or H occurs in it. The checker is specification-level: its clauses contain unbounded quantification, and no executable refinement is provided or claimed — the machine-checked content is the type-level fact that *checker-run-wellformed* cannot consult $b0$ or H at all.
- *run-reflects-source* is the *external* half: the recorded source history is the true deployment history (WF0 (a)) and the recorded chunk reads reflect the true source state at their read coordinates (WF6 main + row-absence). This is the content of *faithful-source-observation*: an observation-channel assumption, discharged at deployment, never computed by *verify*.

The decomposition lemma *wellformed-dblog-run-decompose* proves that wellformedness is exactly the conjunction of the two halves — a *proved factoring*, not a definition of either side as the whole, so the headline implication “acceptance + faithful observation entails a wellformed materialized run” combines two independent, individually insufficient premises (the witness theory *Public-Checker-Witness* exhibits an accepted-but-unfaithful pair and a faithful-but-not-accepted pair). The headline theorems below are conditional implications, proved from the locale’s primitive checker obligations; the obligations hold for the public checker *globally* (*layer3-checker-substrate-holds* at the end of this theory), and the accepting public witness over the running example lives in *Public-Checker-Witness*.

datatype (*'k*, *'v*) *evidence* = *Evidence* (*evidence-run*: (*'k*, *'v*) *run*)

datatype *raw-observation* = *RawObservation*

datatype *verify-result* = *Accept* | *Reject* | *Unsupported*

9.6 Mechanical / external factoring of run wellformedness

checker-run-wellformed: the conjuncts of *wellformed-dblog-run* that mention only the run value — verbatim copies of the WF1 / WF2 / WF4 / WF5 / WF7 clauses, plus internal wellformedness of the *recorded* source history *src-history-of R* (WF0 (b) transported through WF0 (a)).

definition *checker-run-wellformed*

$:: ('k :: \text{linorder}, 'v) \text{run} \Rightarrow \text{bool}$

where

$\text{checker-run-wellformed } R \longleftrightarrow$

— WF0 (b), over the recorded history

$\text{wellformed-src-history } (\text{src-history-of } R)$

$\wedge$

— WF1 (a)-(g), verbatim

$(\forall k \in \text{scope-of } R. \exists! ch. ch \in \text{chunks } R \wedge \text{owns } R \text{ } ch \text{ } k)$

$\wedge (\forall ch1 \in \text{chunks } R. \forall ch2 \in \text{chunks } R.$

$ch1 \neq ch2 \longrightarrow \text{chunk-domain } R \text{ } ch1 \cap \text{chunk-domain } R \text{ } ch2 = \{\})$

$\wedge \text{canonical-chunk-ownership-domain } R = \text{scope-of } R$

$\wedge (\forall k \in \text{scope-of } R. \forall ch.$

$\text{responsible-chunk } R \text{ } k = \text{Some } ch$

$\longleftrightarrow ch \in \text{chunks } R \wedge \text{owns } R \text{ } ch \text{ } k)$

$\wedge (\forall k. k \notin \text{scope-of } R \longrightarrow \text{responsible-chunk } R \text{ } k = \text{None})$

$\wedge \text{finite } (\text{scope-of } R)$

$\wedge (\forall ch \in \text{chunks } R. \text{finite } (\text{chunk-domain } R \text{ } ch))$

$\wedge (\forall ch \in \text{chunks } R. \text{chunk-domain } R \text{ } ch \neq \{\})$

$\wedge$

— WF2 (coverage / faithfulness / order-preservation / multiplicity), verbatim

— all against the recorded history *src-history-of R*

$(\forall k \in \text{scope-of } R. \text{covers-ordinary-cdc } R \text{ } k \text{ } (\text{frontier-of } R))$

$\wedge (\forall p \in \text{set } (\text{cdc-events-of } R).$

$\text{key-of } (\text{hist-event } p) \in \text{scope-of } R$

$$\begin{aligned}
& \wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } R) \\
& \longrightarrow p \in \text{set } (\text{src-history-of } R)) \\
& \wedge (\exists \iota :: \text{nat} \Rightarrow \text{nat}. \text{strict-mono } \iota \\
& \quad \wedge (\text{let in-slice} = (\lambda p. \text{key-of } (\text{hist-event } p) \in \text{scope-of } R \\
& \quad \quad \wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } R)); \\
& \quad \text{cdc-in-slice} = \text{filter in-slice } (\text{cdc-events-of } R); \\
& \quad \text{hist-in-slice} = \text{filter in-slice } (\text{src-history-of } R) \\
& \quad \text{in } \forall i. i < \text{length cdc-in-slice} \longrightarrow \\
& \quad \quad \iota i < \text{length hist-in-slice} \\
& \quad \quad \wedge \text{cdc-in-slice } ! i = \text{hist-in-slice } ! \iota i)) \\
& \wedge (\forall k \in \text{scope-of } R. \forall ch \ c. \\
& \quad \text{responsible-chunk } R \ k = \text{Some } ch \\
& \quad \wedge \text{src-lt } (\text{chunk-read-coordinate } R \ ch) \ c \\
& \quad \wedge \text{src-le } c (\text{frontier-of } R) \\
& \quad \longrightarrow \\
& \quad \text{mset } (\text{source-key-coord-slice } k \ c (\text{cdc-events-of } R)) \\
& \quad = \text{mset } (\text{source-key-coord-slice } k \ c (\text{src-history-of } R))) \\
& \wedge \\
& \quad \text{— WF4, verbatim} \\
& \quad (\forall ch \in \text{chunks } R. \\
& \quad \quad \text{src-le } (\text{chunk-lower-watermark } R \ ch) (\text{chunk-read-coordinate } R \ ch) \\
& \quad \wedge \text{src-le } (\text{chunk-read-coordinate } R \ ch) (\text{chunk-upper-watermark } R \ ch) \\
& \quad \wedge (\forall k \in \text{chunk-domain } R \ ch. \\
& \quad \quad \exists m. \text{chunk-read-result } R \ ch \ k = \text{Some } m) \\
& \quad \wedge (\forall k \in \text{chunk-domain } R \ ch. \forall m. \\
& \quad \quad \text{chunk-read-result } R \ ch \ k = \text{Some } m \\
& \quad \quad \longleftrightarrow \text{Refresh } k \ m (\text{chunk-read-coordinate } R \ ch) \\
& \quad \quad \in \text{set } (\text{clean-prefix-of } R))) \\
& \wedge \\
& \quad \text{— WF5, verbatim} \\
& \quad (\forall ch \in \text{chunks } R. \\
& \quad \quad \text{src-le } (\text{chunk-read-coordinate } R \ ch) (\text{frontier-of } R)) \\
& \wedge \\
& \quad \text{— WF7 forward + reverse, verbatim} \\
& \quad (\forall p \in \text{set } (\text{cdc-events-of } R). \\
& \quad \quad \text{key-of } (\text{hist-event } p) \in \text{scope-of } R \\
& \quad \wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } R) \\
& \quad \longrightarrow \text{Cdc } (\text{hist-coord } p) (\text{hist-event } p) \\
& \quad \quad \in \text{set } (\text{clean-prefix-of } R)) \\
& \wedge (\forall e \ c. \text{Cdc } c \ e \in \text{set } (\text{clean-prefix-of } R) \\
& \quad \wedge \text{key-of } e \in \text{scope-of } R \\
& \quad \wedge \text{src-le } c (\text{frontier-of } R) \\
& \quad \longrightarrow (c, e) \in \text{set } (\text{cdc-events-of } R))
\end{aligned}$$

run-reflects-source: the observation-channel conjuncts of *well-formed-dblog-run* — WF0 (a) and WF6 (main + row-absence), verbatim. These are the only clauses in which the deployment base state *b0* or the true source history *H* occurs.

definition *run-reflects-source*

$:: ('k :: \text{linorder} \multimap 'v) \Rightarrow ('k, 'v) \text{ run} \Rightarrow ('k, 'v) \text{ src-history} \Rightarrow \text{bool}$
where
 $\text{run-reflects-source } b0 \ R \ H \longleftrightarrow$
 $\quad \text{— WF0 (a): the recorded history is the true history}$
 $\quad \text{src-history-of } R = H$
 $\wedge$
 $\quad \text{— WF6 (main): recorded reads reflect the true source state}$
 $\quad (\forall \text{ ch} \in \text{chunks } R. \forall k \in \text{chunk-domain } R \text{ ch}. \forall m.$
 $\quad \quad \text{chunk-read-result } R \text{ ch } k = \text{Some } m$
 $\quad \quad \longrightarrow m = \text{Src } b0 \ H \ (\text{chunk-read-coordinate } R \text{ ch}) \ k)$
 $\wedge$
 $\quad \text{— WF6 (row-absence meaningfulness)}$
 $\quad (\forall \text{ ch} \in \text{chunks } R. \forall k \in \text{chunk-domain } R \text{ ch}.$
 $\quad \quad \text{chunk-read-result } R \text{ ch } k = \text{Some } \text{None}$
 $\quad \quad \longrightarrow \text{row-absence-meaningful } b0 \ H \ R \text{ ch } k)$

The factoring is exact: wellformedness is the conjunction of the mechanical and the external half. The only non-verbatim step is transporting *well-formed-src-history* H across *src-history-of* $R = H$.

lemma *wellformed-dblog-run-decompose*:
 $\text{wellformed-dblog-run } b0 \ R \ H \longleftrightarrow$
 $\quad \text{checker-run-wellformed } R \wedge \text{run-reflects-source } b0 \ R \ H$
unfolding *wellformed-dblog-run-def checker-run-wellformed-def*
 $\quad \text{run-reflects-source-def}$
by (*rule iffI*; *elim conjE*; *intro conjI*; (*assumption* | *simp*))

9.7 The public checker primitives

definition *cert-accessors-agree*
 $:: ('k :: \text{linorder}, 'v) \text{ cert} \Rightarrow ('k, 'v) \text{ run} \Rightarrow \text{bool}$
where
 $\text{cert-accessors-agree } C \ R \longleftrightarrow$
 $\quad \text{scope } C = \text{scope-of } R$
 $\wedge \text{frontier } C = \text{frontier-of } R$
 $\wedge \text{clean-prefix } C = \text{clean-prefix-of } R$

The five checker primitives. *evidence-bound-to-cert* binds the evidence to *this* certificate: the certificate's semantic accessors agree with the evidence run's. *source-side-schema-supported* validates that the evidence run value is mechanically wellformed (*checker-run-wellformed*) — the source-side shape the checker supports. *verify* is the composed verdict; all three *verify-result* values are reachable (*Unsupported* for out-of-schema evidence, *Reject* for in-schema evidence not bound to the certificate, *Accept* exactly on *checker-input-ok* inputs). *faithful-source-observation* is the external half (*run-reflects-source*), and *materializes-run* relates the pair to the evidence-determined run.

definition *evidence-bound-to-cert*
 $:: ('k :: \text{linorder}, 'v) \text{ cert} \Rightarrow ('k, 'v) \text{ evidence} \Rightarrow \text{bool}$

where *evidence-bound-to-cert* $C\ E \longleftrightarrow \text{cert-accessors-agree } C\ (\text{evidence-run } E)$

definition *source-side-schema-supported*

$:: ('k::\text{linorder}, 'v)\ \text{cert} \Rightarrow ('k, 'v)\ \text{evidence} \Rightarrow \text{bool}$

where *source-side-schema-supported* $C\ E \longleftrightarrow \text{checker-run-wellformed } (\text{evidence-run } E)$

definition *checker-input-ok*

$:: ('k::\text{linorder}, 'v)\ \text{cert} \Rightarrow ('k, 'v)\ \text{evidence} \Rightarrow \text{bool}$

where

checker-input-ok $C\ E \longleftrightarrow$

evidence-bound-to-cert $C\ E \wedge \text{source-side-schema-supported } C\ E$

definition *verify* $:: ('k::\text{linorder}, 'v)\ \text{cert} \Rightarrow ('k, 'v)\ \text{evidence} \Rightarrow \text{verify-result}$

where *verify* $C\ E =$

(if *source-side-schema-supported* $C\ E$

then if *evidence-bound-to-cert* $C\ E$ then *Accept* else *Reject*

else *Unsupported*)

lemma *verify-eq-Accept-iff*:

verify $C\ E = \text{Accept} \longleftrightarrow \text{checker-input-ok } C\ E$

by (*simp add: verify-def checker-input-ok-def*)

The *Raw* argument below is a deliberate placeholder: the body does not constrain it, and the public *raw-observation* carrier is single-valued — the observation content lives in the evidence run (see the Layer 3 interface note above).

definition *faithful-source-observation*

$:: ('k::\text{linorder}, 'v)\ \text{evidence} \Rightarrow ('k \rightarrow 'v) \Rightarrow ('k, 'v)\ \text{src-history} \Rightarrow \text{raw-observation} \Rightarrow \text{bool}$

where *faithful-source-observation* $E\ b0\ H\ \text{Raw} =$

run-reflects-source $b0\ (\text{evidence-run } E)\ H$

definition *materializes-run*

$:: ('k::\text{linorder}, 'v)\ \text{cert} \Rightarrow ('k, 'v)\ \text{evidence} \Rightarrow ('k, 'v)\ \text{run} \Rightarrow \text{bool}$

where *materializes-run* $C\ E\ R = (R = \text{evidence-run } E)$

definition *cert-run-coherent*

$:: ('k::\text{linorder}, 'v)\ \text{cert} \Rightarrow ('k, 'v)\ \text{evidence} \Rightarrow ('k, 'v)\ \text{run} \Rightarrow \text{bool}$

where

cert-run-coherent $C\ E\ R \longleftrightarrow$

materializes-run $C\ E\ R \wedge \text{cert-accessors-agree } C\ R$

lemma *cert-run-coherence*:

assumes *cert-run-coherent* $C\ E\ R$

shows $\text{scope } C = \text{scope-of } R$

$\wedge \text{frontier } C = \text{frontier-of } R$

$\wedge \text{clean-prefix } C = \text{clean-prefix-of } R$

using *assms*

unfolding *cert-run-coherent-def cert-accessors-agree-def*
by *blast*

lemma *cert-run-coherent-virtual-cut-state-subst*:
assumes *cert-accessors-agree C R*
and *virtual-cut-state b0 (clean-prefix-of R) (scope-of R)*
(frontier-of R) H
shows *virtual-cut-state b0 (clean-prefix C) (scope C)*
(frontier C) H
using *assms*
unfolding *cert-accessors-agree-def*
by *simp*

9.8 The four checker obligations hold globally for the public checker

Each primitive checker obligation of the *layer3-checker-substrate* locale below is a global theorem of the public checker — proved from the definitions, for *every* certificate / evidence pair. The composed obligation (*faithful-materialization-wellformed-global*) is where the factoring earns its keep: the mechanical half comes from the verifier's acceptance, the external half from faithful observation, and *wellformed-dblog-run-decompose* reassembles full wellformedness.

lemma *accepted-input-ok-global*:
 $\text{verify } C \ E = \text{Accept} \implies \text{checker-input-ok } C \ E$
by (*simp add: verify-eq-Accept-iff*)

lemma *accepted-has-materialization-global*:
 $\llbracket \text{verify } C \ E = \text{Accept}; \text{checker-input-ok } C \ E \rrbracket$
 $\implies \exists R. \text{materializes-run } C \ E \ R$
by (*auto simp: materializes-run-def*)

lemma *accepted-materialization-agrees-global*:
 $\llbracket \text{verify } C \ E = \text{Accept}; \text{checker-input-ok } C \ E; \text{materializes-run } C \ E \ R \rrbracket$
 $\implies \text{cert-accessors-agree } C \ R$
by (*simp add: checker-input-ok-def evidence-bound-to-cert-def materializes-run-def*)

lemma *faithful-materialization-wellformed-global*:
 $\llbracket \text{verify } C \ E = \text{Accept}; \text{checker-input-ok } C \ E; \text{materializes-run } C \ E \ R;$
 $\text{faithful-source-observation } E \ b0 \ H \ \text{Raw} \rrbracket$
 $\implies \text{wellformed-dblog-run } b0 \ R \ H$
by (*simp add: checker-input-ok-def source-side-schema-supported-def materializes-run-def faithful-source-observation-def wellformed-dblog-run-decompose*)

locale *layer3-checker-substrate* =
fixes $C :: ('k::\text{linorder}, 'v) \text{ cert}$

```

    and  $E :: ('k, 'v)$  evidence
  assumes accepted-input-ok:
    verify  $C E = \text{Accept} \implies \text{checker-input-ok } C E$ 
  and accepted-has-materialization:
     $\llbracket \text{verify } C E = \text{Accept}; \text{checker-input-ok } C E \rrbracket$ 
     $\implies \exists R. \text{materializes-run } C E R$ 
  and accepted-materialization-agrees:
     $\wedge R. \llbracket \text{verify } C E = \text{Accept}; \text{checker-input-ok } C E;$ 
     $\text{materializes-run } C E R \rrbracket$ 
     $\implies \text{cert-accessors-agree } C R$ 
  and faithful-materialization-wellformed:
     $\wedge R \ b0 \ H \ \text{Raw}. \llbracket \text{verify } C E = \text{Accept}; \text{checker-input-ok } C E;$ 
     $\text{materializes-run } C E R; \text{faithful-source-observation } E \ b0 \ H \ \text{Raw} \rrbracket$ 
     $\implies \text{wellformed-dblog-run } b0 \ R \ H$ 
begin

theorem accepted-certificate-implies-wellformed-run:
  fixes  $b0 :: 'k \rightarrow 'v$ 
  and  $H :: ('k, 'v)$  src-history
  and  $\text{Raw} :: \text{raw-observation}$ 
  assumes accept:  $\text{verify } C E = \text{Accept}$ 
  and faithful:  $\text{faithful-source-observation } E \ b0 \ H \ \text{Raw}$ 
  shows  $\exists R. \text{wellformed-dblog-run } b0 \ R \ H \wedge \text{cert-run-coherent } C E R$ 
proof -
  have input-ok:  $\text{checker-input-ok } C E$ 
  by (rule accepted-input-ok[OF accept])
  then obtain  $R$  where mat:  $\text{materializes-run } C E R$ 
  using accepted-has-materialization[OF accept input-ok] by blast
  have wf:  $\text{wellformed-dblog-run } b0 \ R \ H$ 
  by (rule faithful-materialization-wellformed[OF accept input-ok mat faithful])
  have agree:  $\text{cert-accessors-agree } C R$ 
  by (rule accepted-materialization-agrees[OF accept input-ok mat])
  have coherent:  $\text{cert-run-coherent } C E R$ 
  using mat agree
  unfolding cert-run-coherent-def
  by blast
  show ?thesis
  using wf coherent by blast
qed

theorem accepted-virtual-cut-sound:
  fixes  $b0 :: 'k \rightarrow 'v$ 
  and  $H :: ('k, 'v)$  src-history
  and  $\text{Raw} :: \text{raw-observation}$ 
  assumes accept:  $\text{verify } C E = \text{Accept}$ 
  and faithful:  $\text{faithful-source-observation } E \ b0 \ H \ \text{Raw}$ 
  shows  $\text{virtual-cut } b0 \ C \ H$ 
proof -
  obtain  $R$  where wf:  $\text{wellformed-dblog-run } b0 \ R \ H$ 

```

```

    and coherent: cert-run-coherent  $C E R$ 
    using accepted-certificate-implies-wellformed-run[ $OF$  accept faithful]
    by blast
    have agree: cert-accessors-agree  $C R$ 
    using coherent unfolding cert-run-coherent-def by blast
    have run-vcs:
      virtual-cut-state  $b0$  (clean-prefix-of  $R$ ) (scope-of  $R$ )
        (frontier-of  $R$ )  $H$ 
    using wellformed-run-implies-virtual-cut[ $OF$  wf] .
    have cert-vcs:
      virtual-cut-state  $b0$  (clean-prefix  $C$ ) (scope  $C$ ) (frontier  $C$ )  $H$ 
    using cert-run-coherent-virtual-cut-state-subst[ $OF$  agree run-vcs] .
    show ?thesis
      unfolding virtual-cut-def
      using cert-vcs .
qed
end

```

9.9 The locale holds for every public certificate / evidence pair

Since the four primitive checker obligations are global theorems of the public checker, the *layer3-checker-substrate* predicate holds for *every* (C, E) pair: the locale premise of the headline theorems stated inside the locale (e.g. *accepted-virtual-cut-sound*) discharges unconditionally, leaving *verify $C E = Accept$* and *faithful-source-observation $E b0 H Raw$* as the genuine premises. The witness theory *Public-Checker-Witness* exhibits a concrete pair satisfying both (theorem *public-checker-nonvacuous*), so the Layer 3 / Layer 4 / continuation headline implications are non-vacuous over the public surface.

```

theorem layer3-checker-substrate-holds:
  layer3-checker-substrate  $C E$ 
proof (unfold-locales, goal-cases)
  case 1
  then show ?case by (rule accepted-input-ok-global)
next
  case 2
  then show ?case by (rule accepted-has-materialization-global)
next
  case (3  $R$ )
  then show ?case by (rule accepted-materialization-agrees-global)
next
  case (4  $R b0 H Raw$ )
  then show ?case by (rule faithful-materialization-wellformed-global)
qed
end

```

```

theory Layer01-Witnesses-Core
  imports Dual-Write-Layer0.Source-Coordinates
begin

```

10 Minimum-viable wellformed-run witness data (carrier-independent)

Carrier-independent core of the minimum-viable wellformed-run positive witness.

This theory holds the witness data shared by the family's constructed model (*Layer01-Witnesses-Model*) and interpretation (*Layer01-Witnesses-Inst*): the witness base state $b0-w$ and source history $H-w$, the source-coordinate ordering facts, the *Src / latest-src-event* computation lemmas on the witness data, and the source-history wellformedness $wf-h-w$. None of this mentions a run or chunk carrier, so the theory imports only *Dual-Write-Layer0.Source-Coordinates* (home of the witness coordinates $c1-w / c2-w$ and their order lemmas).

10.1 Witness base state and source history

definition $b0-w :: nat \rightarrow nat$ **where**
 $b0-w = (\lambda k. \text{if } k = 7 \text{ then Some } 100 \text{ else None})$

definition $H-w :: (nat, nat) \text{ src-history}$ **where**
 $H-w = [(c1-w, \text{Insert } 0 \ 42), (c2-w, \text{Delete } 1)]$

10.2 Helper computation lemmas for the WF discharge

Auxiliary facts about source-coordinate ordering and *Src / latest-src-event* on the witness data, used by the source-history wellformedness proof below and by the wellformed-run discharge over the constructed model in *Layer01-Witnesses-Inst*.

lemma $\text{src-le-}c0\text{-}c2\text{-}w$: $\text{src-le } c0 \ c2\text{-}w$
using $c0\text{-least}$ **by** *simp*

lemma $\text{src-le-}c1\text{-}w\text{-}c2\text{-}w$: $\text{src-le } c1\text{-}w \ c2\text{-}w$
using $c1\text{-}w\text{-}lt\text{-}c2\text{-}w$ **by** (*simp add: src-lt-def*)

lemma $\text{not-src-le-}c1\text{-}w\text{-}c0$: $\neg \text{src-le } c1\text{-}w \ c0$
using $c0\text{-le-}c1\text{-}w \ c0\text{-neq-}c1\text{-}w \ \text{src-le-antisym}$ **by** *blast*

lemma $\text{not-src-le-}c2\text{-}w\text{-}c0$: $\neg \text{src-le } c2\text{-}w \ c0$
using $\text{src-le-}c0\text{-}c2\text{-}w \ c0\text{-neq-}c2\text{-}w \ \text{src-le-antisym}$ **by** *blast*

lemma $\text{not-src-lt-}c2\text{-}w\text{-}c1\text{-}w$: $\neg \text{src-lt } c2\text{-}w \ c1\text{-}w$

proof
assume $H: \text{src-lt } c2-w \ c1-w$
hence $\text{src-le } c2-w \ c1-w \ c2-w \neq c1-w$ **by** (*simp-all add: src-lt-def*)
with src-le-c1-w-c2-w **have** $c1-w = c2-w$
using src-le-antisym **by** *blast*
with $\langle c2-w \neq c1-w \rangle$ **show** *False* **by** *simp*
qed

lemma $\text{not-src-lt-c2-w-c2-w}: \neg \text{src-lt } c2-w \ c2-w$
by (*simp add: src-lt-def*)

lemma $\text{length-H-w}: \text{length } H-w = 2$
by (*simp add: H-w-def*)

lemma $H-w\text{-nth-0}: H-w ! 0 = (c1-w, \text{Insert } 0 \ 42)$
by (*simp add: H-w-def*)

lemma $H-w\text{-nth-1}: H-w ! 1 = (c2-w, \text{Delete } (1::\text{nat}))$
by (*simp add: H-w-def*)

lemma set-H-w :
 $\text{set } H-w = \{(c1-w, \text{Insert } 0 \ 42), (c2-w, \text{Delete } (1::\text{nat}))\}$
by (*simp add: H-w-def*)

lemma $\text{latest-src-event-H-w-c0-0}$:
 $\text{latest-src-event } H-w \ c0 \ (0::\text{nat}) = \text{None}$
unfolding $\text{latest-src-event-def}$
using $\text{length-H-w } H-w\text{-nth-0 } H-w\text{-nth-1}$
 $\text{not-src-le-c1-w-c0 not-src-le-c2-w-c0}$
by (*simp add: numeral-2-eq-2*)

lemma $\text{latest-src-event-H-w-c2-w-1}$:
 $\text{latest-src-event } H-w \ c2-w \ (1::\text{nat}) = \text{Some } 1$
unfolding $\text{latest-src-event-def}$
using $\text{length-H-w } H-w\text{-nth-0 } H-w\text{-nth-1}$
 $\text{src-le-c1-w-c2-w src-le-refl}$ [**where** $c=c2-w$]
by (*simp add: numeral-2-eq-2*)

lemma $\text{Src-b0-w-H-w-c0-0}: \text{Src } b0-w \ H-w \ c0 \ (0::\text{nat}) = \text{None}$
unfolding Src-def **using** $\text{latest-src-event-H-w-c0-0}$
by (*simp add: b0-w-def*)

lemma $\text{Src-b0-w-H-w-c2-w-1}: \text{Src } b0-w \ H-w \ c2-w \ (1::\text{nat}) = \text{None}$
unfolding Src-def **using** $\text{latest-src-event-H-w-c2-w-1 } H-w\text{-nth-1}$
by *simp*

10.3 Wellformedness of the witness source history

lemma $\text{wf-h-w}: \text{wellformed-src-history } H-w$

```

unfolding wellformed-src-history-def
using length-H-w H-w-nth-0 H-w-nth-1
      src-le-c1-w-c2-w c0-neq-c1-w c0-neq-c2-w
      c1-w-lt-c2-w not-src-lt-c2-w-c1-w
by (auto simp: source-pos-order-def numeral-2-eq-2
      less-Suc-eq nth-Cons)

end

```

```

theory Layer01-Witness-Topics-Core
imports Dual-Write-Layer0.Source-Coordinates
begin

```

11 Topic 1 and Topic 2 witness data (carrier-independent)

Carrier-independent core of the Topic 1 and Topic 2 positive witnesses.

Holds the witness data shared by the family's constructed models (*Layer01-Witness-Topics-Model*) and interpretations (*Layer01-Witness-Topics-Inst*): the per-topic base states and source histories (*b0-t1* / *H-t1*, *b0-t2* / *H-t2*), the *Src* / *latest-src-event* computation lemmas on that data, and the source-history wellformedness lemmas. None of this mentions a run or chunk carrier, so the theory imports only *Dual-Write-Layer0.Source-Coordinates* (home of the Topic 1 coordinate *ct1* and its order lemmas).

12 Topic 1 witness data

12.1 Topic 1 base state and source history

```

definition b0-t1 :: nat  $\rightarrow$  nat where
  b0-t1 = ( $\lambda k$ . if k = 1 then Some 5 else None)

```

```

definition H-t1 :: (nat, nat) src-history where
  H-t1 = [(ct1, Update 1  $\gamma$ )]

```

```

lemma length-H-t1: length H-t1 = 1
by (simp add: H-t1-def)

```

```

lemma H-t1-nth-0: H-t1 ! 0 = (ct1, Update 1  $\gamma$ )
by (simp add: H-t1-def)

```

```

lemma set-H-t1: set H-t1 = {(ct1, Update 1  $\gamma$ )}
by (simp add: H-t1-def)

```


12.2 Source-state computations on the Topic 1 instance

lemma *upt-length-H-t1*: $[0..<\text{length } H\text{-}t1] = [0]$
by (*simp add: length-H-t1*)

lemma *latest-src-event-H-t1-c0-1*:
latest-src-event H-t1 c0 (1::nat) = None
proof –
let $?P = \lambda i. \text{src-le } (\text{hist-coord } (H\text{-}t1 \ ! \ i)) \ c0$
 $\wedge \text{key-of } (\text{hist-event } (H\text{-}t1 \ ! \ i)) = (1::nat)$
have *filter ?P [0] = []*
using *H-t1-nth-0 not-src-le-ct1-c0* **by** *simp*
thus *?thesis*
unfolding *latest-src-event-def upt-length-H-t1* **by** *simp*
qed

lemma *latest-src-event-H-t1-ct1-1*:
latest-src-event H-t1 ct1 (1::nat) = Some 0
proof –
let $?P = \lambda i. \text{src-le } (\text{hist-coord } (H\text{-}t1 \ ! \ i)) \ ct1$
 $\wedge \text{key-of } (\text{hist-event } (H\text{-}t1 \ ! \ i)) = (1::nat)$
have *filter ?P [0] = [0]*
using *H-t1-nth-0 src-le-ct1-ct1* **by** *simp*
thus *?thesis*
unfolding *latest-src-event-def upt-length-H-t1* **by** *simp*
qed

lemma *Src-b0-t1-H-t1-c0-1*: *Src b0-t1 H-t1 c0 (1::nat) = Some 5*
unfolding *Src-def* **using** *latest-src-event-H-t1-c0-1*
by (*simp add: b0-t1-def*)

lemma *Src-b0-t1-H-t1-ct1-1*: *Src b0-t1 H-t1 ct1 (1::nat) = Some 7*
unfolding *Src-def* **using** *latest-src-event-H-t1-ct1-1 H-t1-nth-0*
by *simp*

12.3 Wellformedness of the Topic 1 source history

lemma *wf-h-t1*: *wellformed-src-history H-t1*
unfolding *wellformed-src-history-def*
using *length-H-t1 H-t1-nth-0 c0-neq-ct1*
by (*auto simp: source-pos-order-def*)

13 Topic 2 witness data

13.1 Topic 2 base state and source history

definition *b0-t2* :: *nat* $\rightarrow$ *nat* **where**
b0-t2 = ($\lambda k. \text{if } k = 10 \text{ then } \text{Some } 100$
 $\text{else if } k = 20 \text{ then } \text{Some } 200 \text{ else } \text{None}$)

definition $H\text{-}t2 :: (\text{nat}, \text{nat}) \text{ src-history}$ **where**
 $H\text{-}t2 = []$

lemma $\text{length-}H\text{-}t2$: $\text{length } H\text{-}t2 = 0$ **by** ($\text{simp add: } H\text{-}t2\text{-def}$)

lemma $\text{set-}H\text{-}t2$: $\text{set } H\text{-}t2 = \{\}$ **by** ($\text{simp add: } H\text{-}t2\text{-def}$)

13.2 Source-state computations on the Topic 2 instance

lemma $\text{latest-src-event-}H\text{-}t2\text{-}c0\text{-any}$:
 $\text{latest-src-event } H\text{-}t2 \ c0 \ (k :: \text{nat}) = \text{None}$
unfolding $\text{latest-src-event-def}$ **by** ($\text{simp add: } H\text{-}t2\text{-def}$)

lemma $\text{Src-}b0\text{-}t2\text{-}H\text{-}t2\text{-}c0\text{-}10$: $\text{Src } b0\text{-}t2 \ H\text{-}t2 \ c0 \ (10 :: \text{nat}) = \text{Some } 100$
unfolding Src-def **using** $\text{latest-src-event-}H\text{-}t2\text{-}c0\text{-any}$
by ($\text{simp add: } b0\text{-}t2\text{-def}$)

lemma $\text{Src-}b0\text{-}t2\text{-}H\text{-}t2\text{-}c0\text{-}20$: $\text{Src } b0\text{-}t2 \ H\text{-}t2 \ c0 \ (20 :: \text{nat}) = \text{Some } 200$
unfolding Src-def **using** $\text{latest-src-event-}H\text{-}t2\text{-}c0\text{-any}$
by ($\text{simp add: } b0\text{-}t2\text{-def}$)

13.3 Wellformedness of the Topic 2 source history

lemma $\text{wf-}h\text{-}t2$: $\text{wellformed-src-history } H\text{-}t2$
unfolding $\text{wellformed-src-history-def}$ **by** ($\text{simp add: } H\text{-}t2\text{-def}$)

end

theory $\text{Layer01-Virtual-Cut-Example-Core}$
imports $\text{Dual-Write-Layer0.Source-Coordinates}$
begin

14 Running-example data (carrier-independent)

Carrier-independent core of the running-example virtual cut.

Holds the example data shared by the family's constructed model $\text{Layer01-Virtual-Cut-Example-Model}$ and interpretation $\text{Layer01-Virtual-Cut-Example-Inst}$, and reused throughout the Layer 3 / Layer 4 and continuation witness and fixture theories: the example base state $b0\text{-ex}$ and source history $H\text{-ex}$, the source-coordinate ordering facts, the Src / latest-src-event computation lemmas, and the source-history wellformedness $\text{wf-}h\text{-ex}$. None of this mentions a run or chunk carrier, so the theory imports only $\text{Dual-Write-Layer0.Source-Coordinates}$ (home of the example coordinates $ec1 \dots ec4$ and their order lemmas).

14.1 Witness source-coordinate ordering facts

lemma *ec1-neq-ec4*: $ec1 \neq ec4$
proof –
 have *src-lt* *ec1* *ec4* **using** *ec1-lt-ec2* *ec2-le-ec4*
 by (*metis* *src-lt-def* *src-le-antisym* *src-le-trans* *ec1-le-ec4*)
 thus ?thesis **by** (*simp* *add*: *src-lt-def*)
qed

lemma *ec2-neq-ec4*: $ec2 \neq ec4$
proof –
 have *src-lt* *ec2* *ec4* **using** *ec2-lt-ec3* *ec3-le-ec4*
 by (*metis* *src-lt-def* *src-le-antisym* *src-le-trans* *ec2-le-ec4*)
 thus ?thesis **by** (*simp* *add*: *src-lt-def*)
qed

Negative orderings used by the WF discharge and by Src computations below: *ec-i* is *not src-le* below *c0*, and the *src-lt* chain rules out backwards comparisons.

lemma *not-src-le-ec1-c0*: $\neg \text{src-le } ec1 \ c0$
using *c0-le-ec1* *c0-neq-ec1* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec2-c0*: $\neg \text{src-le } ec2 \ c0$
using *c0-le-ec2* *c0-neq-ec2* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec3-c0*: $\neg \text{src-le } ec3 \ c0$
using *c0-le-ec3* *c0-neq-ec3* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec4-c0*: $\neg \text{src-le } ec4 \ c0$
using *c0-le-ec4* *c0-neq-ec4* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec2-ec1*: $\neg \text{src-le } ec2 \ ec1$
using *ec1-le-ec2* *ec1-neq-ec2* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec3-ec1*: $\neg \text{src-le } ec3 \ ec1$
using *ec1-le-ec3* *ec1-neq-ec3* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec4-ec1*: $\neg \text{src-le } ec4 \ ec1$
using *ec1-le-ec4* *ec1-neq-ec4* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec3-ec2*: $\neg \text{src-le } ec3 \ ec2$
using *ec2-le-ec3* *ec2-neq-ec3* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec4-ec2*: $\neg \text{src-le } ec4 \ ec2$
using *ec2-le-ec4* *ec2-neq-ec4* *src-le-antisym* **by** *blast*

lemma *not-src-le-ec4-ec3*: $\neg \text{src-le } ec4 \ ec3$
using *ec3-le-ec4* *ec3-neq-ec4* *src-le-antisym* **by** *blast*

lemma *not-src-lt-ec1-ec1*: $\neg \text{src-lt } ec1 \ ec1$ **by** (*simp* *add*: *src-lt-def*)

```

lemma not-src-lt-ec3-ec1:  $\neg$  src-lt ec3 ec1
  using not-src-le-ec3-ec1 by (simp add: src-lt-def)
lemma not-src-lt-ec3-ec2:  $\neg$  src-lt ec3 ec2
  using not-src-le-ec3-ec2 by (simp add: src-lt-def)
lemma not-src-lt-ec3-ec3:  $\neg$  src-lt ec3 ec3 by (simp add: src-lt-def)

```

14.2 Witness base state and source history

```

definition b0-ex :: nat  $\rightarrow$  nat where
  b0-ex = ( $\lambda k$ . if k = 100 then Some 5000
           else if k = 200 then Some 10000
           else None)

```

```

definition H-ex :: (nat, nat) src-history where
  H-ex = [ (ec1, Update 100 3000),
            (ec2, Insert 300 2500),
            (ec3, Update 200 12000),
            (ec4, Update 300 3500) ]

```

```

lemma length-H-ex: length H-ex = 4
  by (simp add: H-ex-def)

```

```

lemma H-ex-nth-0: H-ex ! 0 = (ec1, Update 100 3000)
  by (simp add: H-ex-def)

```

```

lemma H-ex-nth-1: H-ex ! 1 = (ec2, Insert 300 2500)
  by (simp add: H-ex-def)

```

```

lemma H-ex-nth-2: H-ex ! 2 = (ec3, Update 200 12000)
  by (simp add: H-ex-def)

```

```

lemma H-ex-nth-3: H-ex ! 3 = (ec4, Update 300 3500)
  by (simp add: H-ex-def)

```

```

lemma set-H-ex:
  set H-ex = { (ec1, Update 100 3000),
                (ec2, Insert 300 2500),
                (ec3, Update 200 12000),
                (ec4, Update 300 3500) }
  by (simp add: H-ex-def)

```

14.3 Source-state computations on the example

```

lemma upt-length-H-ex: [0..length H-ex] = [0, 1, 2, 3]

```

```

proof –
  have length H-ex = 4 by (rule length-H-ex)
  thus ?thesis by (simp add: upt-rec)
qed

```

```

lemma latest-src-event-H-ex-ec1-100:

```

$latest_src_event\ H_ex\ ec1\ 100 = Some\ 0$
proof –
let $?P = \lambda i. src_le\ (hist_coord\ (H_ex\ !\ i))\ ec1$
 $\wedge\ key_of\ (hist_event\ (H_ex\ !\ i)) = (100::nat)$
have $filter\ ?P\ [0, 1, 2, 3] = [0]$
using $src_le_refl[where\ c=ec1]\ not_src_le_ec2_ec1$
 $not_src_le_ec3_ec1\ not_src_le_ec4_ec1$
 $H_ex_nth-0\ H_ex_nth-1\ H_ex_nth-2\ H_ex_nth-3$
by *simp*
thus *?thesis*
unfolding $latest_src_event_def\ upt_length_H_ex$ **by** *simp*
qed

lemma $latest_src_event_H_ex_ec1-200$:
 $latest_src_event\ H_ex\ ec1\ 200 = None$
proof –
let $?P = \lambda i. src_le\ (hist_coord\ (H_ex\ !\ i))\ ec1$
 $\wedge\ key_of\ (hist_event\ (H_ex\ !\ i)) = (200::nat)$
have $filter\ ?P\ [0, 1, 2, 3] = []$
using $src_le_refl[where\ c=ec1]\ not_src_le_ec2_ec1$
 $not_src_le_ec3_ec1\ not_src_le_ec4_ec1$
 $H_ex_nth-0\ H_ex_nth-1\ H_ex_nth-2\ H_ex_nth-3$
by *simp*
thus *?thesis*
unfolding $latest_src_event_def\ upt_length_H_ex$ **by** *simp*
qed

lemma $latest_src_event_H_ex_ec3-300$:
 $latest_src_event\ H_ex\ ec3\ 300 = Some\ 1$
proof –
let $?P = \lambda i. src_le\ (hist_coord\ (H_ex\ !\ i))\ ec3$
 $\wedge\ key_of\ (hist_event\ (H_ex\ !\ i)) = (300::nat)$
have $filter\ ?P\ [0, 1, 2, 3] = [1]$
using $ec2_le_ec3\ src_le_refl[where\ c=ec3]\ not_src_le_ec4_ec3$
 $H_ex_nth-0\ H_ex_nth-1\ H_ex_nth-2\ H_ex_nth-3$
by *simp*
thus *?thesis*
unfolding $latest_src_event_def\ upt_length_H_ex$ **by** *simp*
qed

lemma $latest_src_event_H_ex_ec4-100$:
 $latest_src_event\ H_ex\ ec4\ 100 = Some\ 0$
proof –
let $?P = \lambda i. src_le\ (hist_coord\ (H_ex\ !\ i))\ ec4$
 $\wedge\ key_of\ (hist_event\ (H_ex\ !\ i)) = (100::nat)$
have $filter\ ?P\ [0, 1, 2, 3] = [0]$
using $ec1_le_ec4\ ec2_le_ec4\ ec3_le_ec4\ src_le_refl[where\ c=ec4]$
 $H_ex_nth-0\ H_ex_nth-1\ H_ex_nth-2\ H_ex_nth-3$
by *simp*

thus *?thesis*
unfolding *latest-src-event-def upt-length-H-ex* **by** *simp*
qed

lemma *latest-src-event-H-ex-ec4-200*:
latest-src-event H-ex ec4 200 = Some 2
proof –
let *?P* = $\lambda i. \text{src-le } (\text{hist-coord } (H\text{-ex } ! i)) \text{ ec4}$
 $\wedge \text{key-of } (\text{hist-event } (H\text{-ex } ! i)) = (200::\text{nat})$
have *filter ?P [0, 1, 2, 3] = [2]*
using *ec1-le-ec4 ec2-le-ec4 ec3-le-ec4 src-le-refl*[**where** *c=ec4*]
H-ex-nth-0 H-ex-nth-1 H-ex-nth-2 H-ex-nth-3
by *simp*
thus *?thesis*
unfolding *latest-src-event-def upt-length-H-ex* **by** *simp*
qed

lemma *latest-src-event-H-ex-ec4-300*:
latest-src-event H-ex ec4 300 = Some 3
proof –
let *?P* = $\lambda i. \text{src-le } (\text{hist-coord } (H\text{-ex } ! i)) \text{ ec4}$
 $\wedge \text{key-of } (\text{hist-event } (H\text{-ex } ! i)) = (300::\text{nat})$
have *filter ?P [0, 1, 2, 3] = [1, 3]*
using *ec1-le-ec4 ec2-le-ec4 ec3-le-ec4 src-le-refl*[**where** *c=ec4*]
H-ex-nth-0 H-ex-nth-1 H-ex-nth-2 H-ex-nth-3
by *simp*
thus *?thesis*
unfolding *latest-src-event-def upt-length-H-ex* **by** *simp*
qed

Src values at the chunk-read coordinates and at the frontier. WF6 uses the chunk-read-coordinate values; the final virtual-cut equality uses the frontier values.

lemma *Src-b0-ex-H-ex-ec1-100*:
Src b0-ex H-ex ec1 100 = Some 3000
unfolding *Src-def* **using** *latest-src-event-H-ex-ec1-100 H-ex-nth-0*
by *simp*

lemma *Src-b0-ex-H-ex-ec1-200*:
Src b0-ex H-ex ec1 200 = Some 10000
unfolding *Src-def* **using** *latest-src-event-H-ex-ec1-200*
by (*simp add: b0-ex-def*)

lemma *Src-b0-ex-H-ex-ec3-300*:
Src b0-ex H-ex ec3 300 = Some 2500
unfolding *Src-def* **using** *latest-src-event-H-ex-ec3-300 H-ex-nth-1*
by *simp*

lemma *Src-b0-ex-H-ex-ec4-100*:

Src b0-ex H-ex ec4 100 = Some 3000
unfolding *Src-def* **using** *latest-src-event-H-ex-ec4-100 H-ex-nth-0*
by *simp*

lemma *Src-b0-ex-H-ex-ec4-200*:
Src b0-ex H-ex ec4 200 = Some 12000
unfolding *Src-def* **using** *latest-src-event-H-ex-ec4-200 H-ex-nth-2*
by *simp*

lemma *Src-b0-ex-H-ex-ec4-300*:
Src b0-ex H-ex ec4 300 = Some 3500
unfolding *Src-def* **using** *latest-src-event-H-ex-ec4-300 H-ex-nth-3*
by *simp*

14.4 Wellformedness of the example source history

lemma *less-4-cases*: $(i :: \text{nat}) < 4 \implies i = 0 \vee i = 1 \vee i = 2 \vee i = 3$
by *linarith*

lemma *wf-h-ex-WFH1*:
 $\forall i. \text{Suc } i < \text{length } H\text{-ex} \implies \text{src-le } (\text{hist-coord } (H\text{-ex } ! i))$
 $\quad (\text{hist-coord } (H\text{-ex } ! \text{Suc } i))$

proof (*intro allI impI*)
fix *i* **assume** *Suc i < length H-ex*
hence *si-lt*: *Suc i < 4* **by** (*simp add: length-H-ex*)
hence *i-lt*: *i < 3* **by** *linarith*
hence *i-cases*: *i = 0 $\vee$ i = 1 $\vee$ i = 2* **by** *linarith*
consider (*a*) *i = 0* | (*b*) *i = 1* | (*c*) *i = 2* **using** *i-cases* **by** *blast*
thus *src-le* (*hist-coord* (*H-ex* ! *i*)) (*hist-coord* (*H-ex* ! *Suc i*))
proof *cases*
case *a* **thus** *?thesis* **using** *ec1-le-ec2* **by** (*simp add: H-ex-def*)
next
case *b* **thus** *?thesis* **using** *ec2-le-ec3* **by** (*simp add: H-ex-def*)
next
case *c* **thus** *?thesis* **using** *ec3-le-ec4* **by** (*simp add: H-ex-def*)
qed
qed

lemma *wf-h-ex-WFH2*:
 $\forall i. i < \text{length } H\text{-ex} \implies \text{hist-coord } (H\text{-ex } ! i) \neq c0$

proof (*intro allI impI*)
fix *i* **assume** *i < length H-ex*
hence *i-lt*: *i < 4* **by** (*simp add: length-H-ex*)
hence *i-cases*: *i = 0 $\vee$ i = 1 $\vee$ i = 2 $\vee$ i = 3*
by (*rule less-4-cases*)
consider (*a*) *i = 0* | (*b*) *i = 1* | (*c*) *i = 2* | (*d*) *i = 3*
using *i-cases* **by** *blast*
thus *hist-coord* (*H-ex* ! *i*) $\neq c0$

```

proof cases
  case a thus ?thesis using c0-neq-ec1 by (simp add: H-ex-def)
next
  case b thus ?thesis using c0-neq-ec2 by (simp add: H-ex-def)
next
  case c thus ?thesis using c0-neq-ec3 by (simp add: H-ex-def)
next
  case d thus ?thesis using c0-neq-ec4 by (simp add: H-ex-def)
qed
qed

lemma wf-h-ex-WFH3:
   $\forall i j. i < \text{length } H\text{-ex} \wedge j < \text{length } H\text{-ex} \wedge i \neq j$ 
   $\longrightarrow \text{source-pos-order } H\text{-ex } i j \vee \text{source-pos-order } H\text{-ex } j i$ 
proof (intro allI impI)
  fix i j
  assume A:  $i < \text{length } H\text{-ex} \wedge j < \text{length } H\text{-ex} \wedge i \neq j$ 
  hence i-lt:  $i < 4$  and j-lt:  $j < 4$  and ij-ne:  $i \neq j$ 
  by (auto simp: length-H-ex)
  have i-cases:  $i = 0 \vee i = 1 \vee i = 2 \vee i = 3$ 
  using i-lt by (rule less-4-cases)
  have j-cases:  $j = 0 \vee j = 1 \vee j = 2 \vee j = 3$ 
  using j-lt by (rule less-4-cases)
  — For each (i, j) pair with  $i \neq j$  we either have  $i < j$  (and the coords are
  src-lt-related forward) or  $j < i$  (the reverse).
  show  $\text{source-pos-order } H\text{-ex } i j \vee \text{source-pos-order } H\text{-ex } j i$ 
  proof (cases i < j)
    case True
    —  $i < j$ : hist-coord( $H\text{-ex } ! i$ ) src-lt-precedes hist-coord( $H\text{-ex } ! j$ ).
    show ?thesis
    proof (rule disjI1)
      show  $\text{source-pos-order } H\text{-ex } i j$ 
      unfolding source-pos-order-def
      proof (rule disjI1)
        show  $\text{src-lt } (\text{hist-coord } (H\text{-ex } ! i)) (\text{hist-coord } (H\text{-ex } ! j))$ 
        using i-cases j-cases True
         $H\text{-ex-nth-0 } H\text{-ex-nth-1 } H\text{-ex-nth-2 } H\text{-ex-nth-3}$ 
         $ec1\text{-lt-ec2 } ec2\text{-lt-ec3 } ec3\text{-lt-ec4}$ 
         $ec1\text{-le-ec3 } ec1\text{-le-ec4 } ec2\text{-le-ec4}$ 
         $ec1\text{-neq-ec3 } ec1\text{-neq-ec4 } ec2\text{-neq-ec4}$ 
        src-lt-def
        by auto
      qed
    qed
  next
  case False
  hence j-lt-i:  $j < i$  using ij-ne by linarith
  show ?thesis
  proof (rule disjI2)

```



```

show source-pos-order H-ex j i
  unfolding source-pos-order-def
proof (rule disjI1)
  show src-lt (hist-coord (H-ex ! j)) (hist-coord (H-ex ! i))
    using i-cases j-cases j-lt-i
      H-ex-nth-0 H-ex-nth-1 H-ex-nth-2 H-ex-nth-3
      ec1-lt-ec2 ec2-lt-ec3 ec3-lt-ec4
      ec1-le-ec3 ec1-le-ec4 ec2-le-ec4
      ec1-neq-ec3 ec1-neq-ec4 ec2-neq-ec4
      src-lt-def
    by auto
  qed
qed
qed
qed

lemma wf-h-ex: wellformed-src-history H-ex
  unfolding wellformed-src-history-def
  using wf-h-ex-WFH1 wf-h-ex-WFH2 wf-h-ex-WFH3
  by blast

end

```

```

theory Layer4-Witnesses-Core
  imports Layer01-Virtual-Cut-Example-Core Virtual-Cut-Core
begin

```

15 Layer 3/4 witness anchor data (carrier-independent)

Carrier- and certificate-independent core of the Layer 4 positive whole-table witness.

Holds the anchor-boundary data that the constructed Layer 3 / Layer 4 witness interpretation (*Layer3-Witnesses-Inst*), the Layer 4 fixture theories, and *Public-Checker-Witness* reuse: the anchor boundary *l4-anchor*, the *Src* / *latest-src-event* facts at the anchor on the running-example history, the anchor-domain computation *l4-anchor-domain*, the post-anchor touched-key set *touched-H-ex-anchor-ec4*, and the derived strict / non-strict coordinate orderings. None of this mentions a run, chunk, or certificate carrier, so the theory imports only *Layer01-Virtual-Cut-Example-Core* (for *b0-ex* / *H-ex*) and *Virtual-Cut-Core* (home of the Layer-4 set operators, *anchor-domain* in particular).

15.1 Anchor boundary

definition $l_4\text{-anchor} :: \text{frontier}$ where
 $l_4\text{-anchor} = \text{ec1}$

15.2 Anchor and post-anchor source facts (independent of certificates)

lemma $\text{latest-src-event-}H\text{-ex-ec1-other}$:
assumes $k \neq (100::\text{nat})$ **and** $k \neq 200$
shows $\text{latest-src-event } H\text{-ex ec1 } k = \text{None}$
proof –
let $?P = \lambda i. \text{src-le } (\text{hist-coord } (H\text{-ex } ! i)) \text{ ec1}$
 $\quad \wedge \text{key-of } (\text{hist-event } (H\text{-ex } ! i)) = k$
have $\text{filter } ?P [0, 1, 2, 3] = []$
using assms src-le-refl **[where** $c = \text{ec1}$ **]** $\text{not-src-le-ec2-ec1}$
 $\text{not-src-le-ec3-ec1 not-src-le-ec4-ec1}$
 $H\text{-ex-nth-0 } H\text{-ex-nth-1 } H\text{-ex-nth-2 } H\text{-ex-nth-3}$
by simp
thus $?thesis$
unfolding $\text{latest-src-event-def upt-length-}H\text{-ex}$ **by** simp
qed

lemma $\text{Src-b0-ex-}H\text{-ex-ec1}$:
 $\text{Src b0-ex } H\text{-ex ec1} =$
 $(\lambda k. \text{if } k = (100::\text{nat}) \text{ then } \text{Some } (3000::\text{nat})$
 $\quad \text{else if } k = 200 \text{ then } \text{Some } 10000$
 $\quad \text{else } \text{None})$
proof
fix $k :: \text{nat}$
show $\text{Src b0-ex } H\text{-ex ec1 } k =$
 $(\text{if } k = (100::\text{nat}) \text{ then } \text{Some } (3000::\text{nat})$
 $\quad \text{else if } k = 200 \text{ then } \text{Some } 10000$
 $\quad \text{else } \text{None})$
proof $(\text{cases } k = 100)$
case True
thus $?thesis$ **using** $\text{Src-b0-ex-}H\text{-ex-ec1-100}$ **by** simp
next
case not100: False
show $?thesis$
proof $(\text{cases } k = 200)$
case True
thus $?thesis$ **using** $\text{Src-b0-ex-}H\text{-ex-ec1-200}$ **by** simp
next
case not200: False
hence $\text{latest-none: latest-src-event } H\text{-ex ec1 } k = \text{None}$
using $\text{not100 latest-src-event-}H\text{-ex-ec1-other}$ **by** blast
show $?thesis$
unfolding Src-def b0-ex-def
using $\text{not100 not200 latest-none}$ **by** simp

qed
qed
qed

lemma *l4-anchor-domain*:
 $\text{anchor-domain } b0\text{-ex } H\text{-ex } l4\text{-anchor} = \{100, 200\}$
unfolding *anchor-domain-def l4-anchor-def Src-b0-ex-H-ex-ec1*
by *auto*

lemma *ec1-lt-ec3*: $\text{src-lt } ec1 \ ec3$
using *ec1-le-ec3 ec1-neq-ec3* **by** (*simp add: src-lt-def*)

lemma *ec1-lt-ec4*: $\text{src-lt } ec1 \ ec4$
using *ec1-le-ec4 ec1-neq-ec4* **by** (*simp add: src-lt-def*)

lemma *ec1-less-ec2*: $ec1 < ec2$
using *ec1-lt-ec2* **by** (*simp add: less-src-coord-def*)

lemma *ec1-less-ec3*: $ec1 < ec3$
using *ec1-lt-ec3* **by** (*simp add: less-src-coord-def*)

lemma *ec1-less-ec4*: $ec1 < ec4$
using *ec1-lt-ec4* **by** (*simp add: less-src-coord-def*)

lemma *ec2-less-eq-ec4*: $ec2 \leq ec4$
using *ec2-le-ec4* **by** (*simp add: less-eq-src-coord-def*)

lemma *ec3-less-eq-ec4*: $ec3 \leq ec4$
using *ec3-le-ec4* **by** (*simp add: less-eq-src-coord-def*)

The *H-ex* post-anchor touched set over the explicit frontier *ec4*: between the anchor and the frontier exactly the keys 200 (updated at *ec3*) and 300 (inserted at *ec2*) are touched. Shared by the Layer 3/4 witness interpretation (*Layer3-Witnesses-Inst*) and the constructed Layer 4 fixtures (*Layer4-Fixtures-Inst*).

lemma *touched-H-ex-anchor-ec4*:
 $\text{touched-between } H\text{-ex } l4\text{-anchor } ec4 = \{200, 300\}$
proof (*rule subset-antisym*)
show $\text{touched-between } H\text{-ex } l4\text{-anchor } ec4 \subseteq \{200, 300\}$
unfolding *touched-between-def l4-anchor-def H-ex-def*
by *auto*
next
show $\{200, 300\} \subseteq \text{touched-between } H\text{-ex } l4\text{-anchor } ec4$
proof
fix *k* :: *nat*
assume $k \in \{200, 300\}$
hence $k = 200 \vee k = 300$ **by** *simp*
thus $k \in \text{touched-between } H\text{-ex } l4\text{-anchor } ec4$
proof

```

assume k200: k = 200
have in-H: (ec3, Update 200 12000) ∈ set H-ex
  by (simp add: H-ex-def)
have witness:
  ∃ c e. (c, e) ∈ set H-ex ∧ ec1 < c ∧ c ≤ ec4 ∧ key-of e = k
  using in-H k200 ec1-less-ec3 ec3-less-eq-ec4
  by (intro exI[where x=ec3]
      exI[where x=Update (200::nat) (12000::nat)]) auto
show ?thesis
  unfolding touched-between-def l4-anchor-def using witness by blast
next
assume k300: k = 300
have in-H: (ec2, Insert 300 2500) ∈ set H-ex
  by (simp add: H-ex-def)
have witness:
  ∃ c e. (c, e) ∈ set H-ex ∧ ec1 < c ∧ c ≤ ec4 ∧ key-of e = k
  using in-H k300 ec1-less-ec2 ec2-less-eq-ec4
  by (intro exI[where x=ec2]
      exI[where x=Insert (300::nat) (2500::nat)]) auto
show ?thesis
  unfolding touched-between-def l4-anchor-def using witness by blast
qed
qed
qed

```

The running-example history has no event for the out-of-table key 999 — used by the constructed Layer 4 fixtures in *Layer4-Fixtures-Inst*.

```

lemma latest-src-event-H-ex-ec4-999:
  latest-src-event H-ex ec4 (999::nat) = None
proof —
  let ?P = λi. src-le (hist-coord (H-ex ! i)) ec4
    ∧ key-of (hist-event (H-ex ! i)) = (999::nat)
  have filter ?P [0, 1, 2, 3] = []
  using H-ex-nth-0 H-ex-nth-1 H-ex-nth-2 H-ex-nth-3 by simp
  thus ?thesis
  unfolding latest-src-event-def upt-length-H-ex by simp
qed

end

```

```

theory Layer01-Fixtures-Core
  imports Dual-Write-Layer0.Source-Coordinates
begin

```

16 Carrier-independent Layer 0/1 fixture data

The carrier-independent data of the Layer 0/1 fixture family, used by the constructed fixture interpretation (*Layer01-Fixtures-Inst*) together with the

concrete run models of *Layer01-Fixtures-Model*: the per-fixture base states $b0\dots$, source histories $H\dots$, and the *latest-src-event* / *Src* computation lemmas (plus *wf-h-es*) they depend on. None of this mentions a run or chunk carrier, so this theory imports only *Dual-Write-Layer0.Source-Coordinates*, where the fixture source coordinates and their order lemmas also live.

16.1 WF1 (b): overlapping chunk domains (*fix-overlap*)

definition $b0\text{-}ov :: nat \rightarrow nat$ **where**
 $b0\text{-}ov = (\lambda k. None)$

definition $H\text{-}ov :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}ov = []$

16.2 WF1 (c): chunk owns out-of-scope key (*fix-miss-cov*)

definition $b0\text{-}mc :: nat \rightarrow nat$ **where**
 $b0\text{-}mc = (\lambda k. None)$

definition $H\text{-}mc :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}mc = []$

16.3 WF1 (f): infinite chunk domain (*fix-inf-dom*)

definition $b0\text{-}inf :: nat \rightarrow nat$ **where**
 $b0\text{-}inf = (\lambda k. None)$

definition $H\text{-}inf :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}inf = []$

16.4 WF5: chunk read after frontier (*fix-after-f*)

definition $b0\text{-}caf :: nat \rightarrow nat$ **where**
 $b0\text{-}caf = (\lambda k. None)$

definition $H\text{-}caf :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}caf = []$

16.5 WF6 main: wrong refresh value (*fix-wrong-r*)

definition $b0\text{-}wr :: nat \rightarrow nat$ **where**
 $b0\text{-}wr = (\lambda k. \text{if } k = 1 \text{ then } \text{Some } 5 \text{ else } None)$

definition $H\text{-}wr :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}wr = []$

lemma *latest-src-event-H-wr-c0-1*:
 $\text{latest-src-event } H\text{-}wr \text{ } c0 \text{ } (1::nat) = None$
unfolding *latest-src-event-def* **by** (*simp add: H-wr-def*)

lemma *Src-b0-wr-H-wr-c0-1*: *Src b0-wr H-wr c0 (1::nat) = Some 5*
unfolding *Src-def* **using** *latest-src-event-H-wr-c0-1*
by (*simp add: b0-wr-def*)

16.6 WF6 specialized: same-coord read/event ambiguity (*fix-same-c*)

definition *b0-sc* :: *nat* $\rightarrow$ *nat* **where**
b0-sc = (λk . if $k = 1$ then *Some 5* else *None*)

definition *H-sc* :: (*nat*, *nat*) *src-history* **where**
H-sc = [(*sc-c1*, *Update 1 7*)]

lemma *length-H-sc*: *length H-sc = 1* **by** (*simp add: H-sc-def*)

lemma *H-sc-nth-0*: *H-sc ! 0 = (sc-c1, Update 1 7)*
by (*simp add: H-sc-def*)

lemma *latest-src-event-H-sc-c1-1*:
latest-src-event H-sc sc-c1 (1::nat) = Some 0
proof –
let *?P* = λi . *src-le (hist-coord (H-sc ! i)) sc-c1*
 $\wedge$ *key-of (hist-event (H-sc ! i)) = (1::nat)*
have *filter ?P [0] = [0]*
using *H-sc-nth-0 sc-c1-le-c1* **by** *simp*
thus *?thesis*
unfolding *latest-src-event-def*
using *length-H-sc* **by** *simp*
qed

lemma *Src-b0-sc-H-sc-c1-1*: *Src b0-sc H-sc sc-c1 (1::nat) = Some 7*
unfolding *Src-def*
using *latest-src-event-H-sc-c1-1 H-sc-nth-0* **by** *simp*

16.7 WF2 faithfulness: extra observed CDC (*fix-extra-cdc*)

definition *b0-ec* :: *nat* $\rightarrow$ *nat* **where**
b0-ec = (λk . *None*)

The extra-CDC family carries no named history constant: its true history is the empty list (the observed CDC event is the hallucination), and the fixture statements pass [] directly.

16.8 WF2 coverage: missing dominator (*fix-miss-dom*)

definition *b0-md* :: *nat* $\rightarrow$ *nat* **where**
b0-md = (λk . *None*)

definition $H\text{-}md :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}md = [(md\text{-}c1, \text{Update } 1 \ \gamma)]$

lemma $length\text{-}H\text{-}md$: $length \ H\text{-}md = 1$ **by** ($simp \ add$: $H\text{-}md\text{-}def$)

lemma $H\text{-}md\text{-}nth\text{-}0$: $H\text{-}md ! 0 = (md\text{-}c1, \text{Update } 1 \ \gamma)$
by ($simp \ add$: $H\text{-}md\text{-}def$)

16.9 WF2 multiplicity: duplicate multiset gap (*fix-dup-mset*)

definition $b0\text{-}dm :: nat \rightarrow nat$ **where**
 $b0\text{-}dm = (\lambda k. \text{None})$

definition $H\text{-}dm :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}dm =$
 $[(dm\text{-}c1, \text{Update } 1 \ 5), (dm\text{-}c1, \text{Update } 1 \ 6), (dm\text{-}c1, \text{Update } 1 \ 5)]$

16.10 WF2 order: duplicate order gap (*fix-dup-order*)

definition $b0\text{-}do :: nat \rightarrow nat$ **where**
 $b0\text{-}do = (\lambda k. \text{None})$

definition $H\text{-}do :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}do =$
 $[(do\text{-}c1, \text{Update } 1 \ 5), (do\text{-}c1, \text{Update } 1 \ 6), (do\text{-}c1, \text{Update } 1 \ 5)]$

16.11 Boundary: empty scope (*boundary-empty-scope*)

definition $b0\text{-}es :: nat \rightarrow nat$ **where**
 $b0\text{-}es = (\lambda k. \text{None})$

definition $H\text{-}es :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}es = []$

lemma $wf\text{-}h\text{-}es$: $wellformed\text{-}src\text{-}history \ H\text{-}es$
unfolding $wellformed\text{-}src\text{-}history\text{-}def$ **by** ($simp \ add$: $H\text{-}es\text{-}def$)

16.12 WF1 (d): responsible-chunk vs chunk-domain mismatch (*fix-resp-dom*)

definition $b0\text{-}rd :: nat \rightarrow nat$ **where**
 $b0\text{-}rd = (\lambda k. \text{None})$

definition $H\text{-}rd :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}rd = []$

16.13 WF6 row-absence: absence-Refresh without empty source (*fix-row-abs*)

definition $b0\text{-}ra :: nat \rightarrow nat$ **where**

$b0\text{-}ra = (\lambda k. \text{ if } k = 1 \text{ then Some } 99 \text{ else None})$

definition $H\text{-}ra :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}ra = []$

lemma $\text{latest-src-event-}H\text{-}ra\text{-}c0\text{-}1$:
 $\text{latest-src-event } H\text{-}ra \ c0 \ (1::nat) = \text{None}$
unfolding $\text{latest-src-event-def}$ **by** $(\text{simp add: } H\text{-}ra\text{-}def)$

lemma $\text{Src-}b0\text{-}ra\text{-}H\text{-}ra\text{-}c0\text{-}1$: $\text{Src } b0\text{-}ra \ H\text{-}ra \ c0 \ (1::nat) = \text{Some } 99$
unfolding Src-def **using** $\text{latest-src-event-}H\text{-}ra\text{-}c0\text{-}1$
by $(\text{simp add: } b0\text{-}ra\text{-}def)$

17 Layer 0 source-history fixture

The Layer-0 negative fixture (fix-h-c0-rejected : a source history with an event at the base coordinate is rejected) and its reader-facing existential. Both are purely over $\text{wellformed-src-history}$ — no run/chunk carrier is involved, so there is no constructed $\text{Layer01-Fixtures-Inst}$ counterpart.

definition $H\text{-}c0 :: (nat, nat) \text{ src-history}$ **where**
 $H\text{-}c0 = [(c0, \text{Update } 1 \ \gamma)]$

lemma $\text{length-}H\text{-}c0$: $\text{length } H\text{-}c0 = 1$
by $(\text{simp add: } H\text{-}c0\text{-}def)$

lemma $H\text{-}c0\text{-}nth\text{-}0$: $H\text{-}c0 \ ! \ 0 = (c0, \text{Update } 1 \ \gamma)$
by $(\text{simp add: } H\text{-}c0\text{-}def)$

theorem fix-h-c0-rejected :
 $\neg \text{wellformed-src-history } H\text{-}c0$

proof

assume H : $\text{wellformed-src-history } H\text{-}c0$
hence $wfh2$: $\forall i. i < \text{length } H\text{-}c0 \longrightarrow \text{hist-coord } (H\text{-}c0 \ ! \ i) \neq c0$
unfolding $\text{wellformed-src-history-def}$ **by** blast
have $(0::nat) < \text{length } H\text{-}c0$ **using** $\text{length-}H\text{-}c0$ **by** simp
with $wfh2$ **have** $\text{hist-coord } (H\text{-}c0 \ ! \ 0) \neq c0$ **by** blast
thus False **using** $H\text{-}c0\text{-}nth\text{-}0$ **by** simp

qed

theorem $\text{layer0-negative-fixture-exists}$:

$\exists (H :: (nat, nat) \text{ src-history}). \neg \text{wellformed-src-history } H$

proof $(\text{intro exI}[\text{where } x = H\text{-}c0])$

show $\neg \text{wellformed-src-history } H\text{-}c0$ **by** $(\text{rule fix-h-c0-rejected})$

qed

end


```

theory Layer3-Fixtures-Core
  imports Layer01-Virtual-Cut-Example-Core Virtual-Cut
begin

```

18 Carrier-independent Layer 3 fixture data

The carrier-independent data of the Layer 3 fixture family, used by the constructed interpretation *Layer3-Fixtures-Inst*: the wrong-base-state and wrong-history values *l3-wrong-b0* / *l3-wrong-H* (ordinary **definitions** over *b0-ex* / *H-ex*, with no certificate / evidence / raw-observation carrier) and their mismatch lemmas, together with the parametric certificate-coherence facts below. The theory imports *DBLog-Virtual-Cuts.Layer01-Virtual-Cut-Example-Core* (for *b0-ex* / *H-ex*) and *DBLog-Virtual-Cuts.Virtual-Cut* (for the certificate accessors and *virtual-cut* used by those parametric facts).

definition *l3-wrong-b0* :: *nat* $\rightarrow$ *nat* **where**
l3-wrong-b0 = (λk . if $k = 200$ then *Some* (*999::nat*) else *b0-ex* k)

definition *l3-wrong-H* :: (*nat*, *nat*) *src-history* **where**
l3-wrong-H = []

lemma *l3-wrong-b0-neq*:
l3-wrong-b0 $\neq$ *b0-ex*
unfolding *l3-wrong-b0-def b0-ex-def*
by (*rule notI*, *drule fun-cong[where x=200]*, *simp*)

lemma *l3-wrong-H-neq*:
l3-wrong-H $\neq$ *H-ex*
unfolding *l3-wrong-H-def H-ex-def*
by *simp*

19 Parametric certificate-coherence facts

Coherence / determinacy / empty-scope facts that are parametric over the global certificate/run accessors (hence the *DBLog-Virtual-Cuts.Virtual-Cut* import). They quantify over arbitrary certificates and runs, so they need no constructed *Layer3-Fixtures-Inst* counterpart.

lemma *layer3-cert-accessor-determinacy*:
assumes *cert-accessors-agree C R1*
and *cert-accessors-agree C R2*
shows *scope-of R1 = scope-of R2*
 $\wedge$ *frontier-of R1 = frontier-of R2*
 $\wedge$ *clean-prefix-of R1 = clean-prefix-of R2*
using *assms*
unfolding *cert-accessors-agree-def*
by *simp*

```

lemma layer3-mismatched-prefix-not-accessor-agree:
  assumes cert-accessors-agree C R1
    and clean-prefix-of R2 ≠ clean-prefix-of R1
  shows  $\neg$  cert-accessors-agree C R2
  using assms
  unfolding cert-accessors-agree-def
  by auto

theorem layer3-empty-scope-virtual-cut-boundary:
  assumes scope C = {}
  shows virtual-cut b0 C H
  using assms
  unfolding virtual-cut-def virtual-cut-state-def restrict-def
  by simp

end

```

```

theory Layer4-Fixtures-Core
  imports Layer4-Witnesses-Core Virtual-Cut
begin

```

20 Carrier-independent Layer 4 fixture data

The carrier-independent data of the Layer 4 fixture family, used by the constructed interpretation *Layer4-Fixtures-Inst*: per-fixture base states *l4-...-b0*, histories *l4-...-H*, replay prefixes *l4-...-prefix*, and their *Apply / Src / latest-src-event / anchor-domain / scoped-virtual-cut-state* computation lemmas. None of this mentions a run, chunk, or certificate carrier. The theory imports *DBLog-Virtual-Cuts.Layer4-Witnesses-Core* (for the anchor boundary *l4-anchor* and the running-example anchor facts) and *DBLog-Virtual-Cuts.Virtual-Cut* (for *virtual-cut-state*).

```

definition l4-missing-insert-prefix :: (nat, nat) replay-event list where
  l4-missing-insert-prefix =
    [ Cdc ec1 (Update 100 3000),
      Refresh (100::nat) (Some (3000::nat)) ec1,
      Refresh 200 (Some 10000) ec1,
      Cdc ec3 (Update 200 12000) ]

```

```

lemma l4-missing-insert-apply-100:
  Apply l4-missing-insert-prefix (100::nat) = Some (3000::nat)
  unfolding l4-missing-insert-prefix-def Apply-def by simp

```

```

lemma l4-missing-insert-apply-200:
  Apply l4-missing-insert-prefix (200::nat) = Some (12000::nat)
  unfolding l4-missing-insert-prefix-def Apply-def by simp

```

```

lemma l4-missing-insert-apply-300:
  Apply l4-missing-insert-prefix (300::nat) = None
  unfolding l4-missing-insert-prefix-def Apply-def by simp

lemma l4-missing-insert-replay-keys:
  replay-keys l4-missing-insert-prefix = {100, 200}
  unfolding replay-keys-def l4-missing-insert-prefix-def
  by auto

lemma l4-missing-insert-scoped-virtual-cut-state:
  virtual-cut-state b0-ex l4-missing-insert-prefix {100, 200} ec4 H-ex
  unfolding virtual-cut-state-def restrict-def
proof (rule ext)
  fix k :: nat
  show (if k ∈ {100, 200} then Apply l4-missing-insert-prefix k else None)
    = (if k ∈ {100, 200} then Src b0-ex H-ex ec4 k else None)
  proof (cases k = 100)
    case True
    thus ?thesis
      using l4-missing-insert-apply-100 Src-b0-ex-H-ex-ec4-100 by simp
  next
    case not100: False
    show ?thesis
    proof (cases k = 200)
      case True
      thus ?thesis
        using l4-missing-insert-apply-200 Src-b0-ex-H-ex-ec4-200 by simp
    next
      case False
      with not100 show ?thesis by simp
    qed
  qed
qed

definition l4-delete-touch-H :: (nat, nat) src-history where
  l4-delete-touch-H =
    [ (ec1, Update 100 3000),
      (ec2, Delete (400::nat)),
      (ec3, Update 200 12000),
      (ec4, Update 300 3500) ]

lemma latest-src-event-l4-delete-touch-ec4-100:
  latest-src-event l4-delete-touch-H ec4 (100::nat) = Some 0
  unfolding latest-src-event-def l4-delete-touch-H-def
  using ec1-le-ec4 ec2-le-ec4 ec3-le-ec4 src-le-refl [where c=ec4]
  by simp

lemma latest-src-event-l4-delete-touch-ec4-200:

```

latest-src-event l4-delete-touch-H ec4 (200::nat) = Some 2
unfolding *latest-src-event-def l4-delete-touch-H-def*
using *ec1-le-ec4 ec2-le-ec4 ec3-le-ec4 src-le-refl*[**where** *c=ec4*]
by *simp*

lemma *latest-src-event-l4-delete-touch-ec4-300:*
latest-src-event l4-delete-touch-H ec4 (300::nat) = Some 3
unfolding *latest-src-event-def l4-delete-touch-H-def*
using *ec1-le-ec4 ec2-le-ec4 ec3-le-ec4 src-le-refl*[**where** *c=ec4*]
by *simp*

lemma *latest-src-event-l4-delete-touch-ec4-400:*
latest-src-event l4-delete-touch-H ec4 (400::nat) = Some 1
unfolding *latest-src-event-def l4-delete-touch-H-def*
using *ec1-le-ec4 ec2-le-ec4 ec3-le-ec4 src-le-refl*[**where** *c=ec4*]
by *simp*

lemma *latest-src-event-l4-delete-touch-ec4-other:*
assumes *k ≠ (100::nat) and k ≠ 200 and k ≠ 300 and k ≠ 400*
shows *latest-src-event l4-delete-touch-H ec4 k = None*
unfolding *latest-src-event-def l4-delete-touch-H-def*
using *assms ec1-le-ec4 ec2-le-ec4 ec3-le-ec4 src-le-refl*[**where** *c=ec4*]
by *simp*

lemma *Src-b0-ex-l4-delete-touch-ec4-100:*
Src b0-ex l4-delete-touch-H ec4 (100::nat) = Some 3000
unfolding *Src-def* **using** *latest-src-event-l4-delete-touch-ec4-100*
by (*simp add: l4-delete-touch-H-def*)

lemma *Src-b0-ex-l4-delete-touch-ec4-200:*
Src b0-ex l4-delete-touch-H ec4 (200::nat) = Some 12000
unfolding *Src-def* **using** *latest-src-event-l4-delete-touch-ec4-200*
by (*simp add: l4-delete-touch-H-def*)

lemma *Src-b0-ex-l4-delete-touch-ec4-300:*
Src b0-ex l4-delete-touch-H ec4 (300::nat) = Some 3500
unfolding *Src-def* **using** *latest-src-event-l4-delete-touch-ec4-300*
by (*simp add: l4-delete-touch-H-def*)

lemma *Src-b0-ex-l4-delete-touch-ec4-400:*
Src b0-ex l4-delete-touch-H ec4 (400::nat) = None
unfolding *Src-def* **using** *latest-src-event-l4-delete-touch-ec4-400*
by (*simp add: l4-delete-touch-H-def*)

lemma *Src-b0-ex-l4-delete-touch-ec4-other:*
assumes *k ≠ (100::nat) and k ≠ 200 and k ≠ 300 and k ≠ 400*
shows *Src b0-ex l4-delete-touch-H ec4 k = None*
unfolding *Src-def b0-ex-def*
using *latest-src-event-l4-delete-touch-ec4-other*[*OF assms*] *assms*

```

by simp

lemma Src-b0-ex-l4-delete-touch-ec4:
  Src b0-ex l4-delete-touch-H ec4 =
    (λk. if k = (100::nat) then Some (3000::nat)
        else if k = 200 then Some 12000
        else if k = 300 then Some 3500
        else None)
proof
  fix k :: nat
  show Src b0-ex l4-delete-touch-H ec4 k =
    (if k = (100::nat) then Some (3000::nat)
     else if k = 200 then Some 12000
     else if k = 300 then Some 3500
     else None)
proof (cases k = 100)
  case True
  thus ?thesis using Src-b0-ex-l4-delete-touch-ec4-100 by simp
next
  case not100: False
  show ?thesis
  proof (cases k = 200)
    case True
    thus ?thesis using Src-b0-ex-l4-delete-touch-ec4-200 by simp
  next
    case not200: False
    show ?thesis
    proof (cases k = 300)
      case True
      thus ?thesis using Src-b0-ex-l4-delete-touch-ec4-300 by simp
    next
      case not300: False
      show ?thesis
      proof (cases k = 400)
        case True
        thus ?thesis
          using not100 not200 not300 Src-b0-ex-l4-delete-touch-ec4-400
          by simp
      next
        case False
        with not100 not200 not300 show ?thesis
          using Src-b0-ex-l4-delete-touch-ec4-other by simp
      qed
    qed
  qed
qed
qed
qed

```

lemma l4-delete-touch-anchor-domain:

$anchor_domain\ b0_ex\ l4_delete_touch_H\ l4_anchor = \{100, 200\}$
unfolding $anchor_domain_def\ l4_anchor_def\ Src_def\ latest_src_event_def$
 $l4_delete_touch_H_def\ b0_ex_def$
using src_le_refl [where $c = ec1$] $not_src_le_ec2_ec1$
 $not_src_le_ec3_ec1\ not_src_le_ec4_ec1$
by *auto*

definition $l4_empty_b0 :: nat \rightarrow nat$ **where**
 $l4_empty_b0 = (\lambda -. None)$

definition $l4_empty_H :: (nat, nat) src_history$ **where**
 $l4_empty_H = []$

lemma $l4_empty_anchor_domain$:
 $anchor_domain\ l4_empty_b0\ l4_empty_H\ l4_anchor = \{\}$
by (*simp* add: $anchor_domain_def\ Src_def\ latest_src_event_def$
 $l4_empty_b0_def\ l4_empty_H_def$)

definition $l4_after_frontier_b0 :: nat \rightarrow nat$ **where**
 $l4_after_frontier_b0 = (\lambda -. None)$

definition $l4_after_frontier_H :: (nat, nat) src_history$ **where**
 $l4_after_frontier_H =$
 $[(ec1, Insert\ (500::nat)\ (9000::nat)),$
 $(ec2, Delete\ (500::nat))]$

definition $l4_after_frontier_anchor :: frontier$ **where**
 $l4_after_frontier_anchor = ec2$

lemma $latest_src_event_l4_after_frontier_ec1_500$:
 $latest_src_event\ l4_after_frontier_H\ ec1\ (500::nat) = Some\ 0$
unfolding $latest_src_event_def\ l4_after_frontier_H_def$
using src_le_refl [where $c = ec1$] $not_src_le_ec2_ec1$
by *simp*

lemma $latest_src_event_l4_after_frontier_ec2_500$:
 $latest_src_event\ l4_after_frontier_H\ ec2\ (500::nat) = Some\ 1$
unfolding $latest_src_event_def\ l4_after_frontier_H_def$
using $ec1_le_ec2\ src_le_refl$ [where $c = ec2$]
by *simp*

lemma $latest_src_event_l4_after_frontier_ec2_other$:
assumes $k \neq (500::nat)$
shows $latest_src_event\ l4_after_frontier_H\ ec2\ k = None$
unfolding $latest_src_event_def\ l4_after_frontier_H_def$
using *assms*
by *simp*

lemma $l4_after_frontier_source_absent_at_anchor_500$:

```

Src l4-after-frontier-b0 l4-after-frontier-H
  l4-after-frontier-anchor (500::nat) = None
unfolding Src-def
using latest-src-event-l4-after-frontier-ec2-500
by (simp add: l4-after-frontier-H-def l4-after-frontier-anchor-def)

lemma l4-after-frontier-source-absent-at-anchor-other:
  assumes k ≠ (500::nat)
  shows Src l4-after-frontier-b0 l4-after-frontier-H
    l4-after-frontier-anchor k = None
  unfolding Src-def l4-after-frontier-b0-def
  using latest-src-event-l4-after-frontier-ec2-other[OF assms]
  by (simp add: l4-after-frontier-anchor-def)

lemma l4-after-frontier-source-at-anchor-empty:
  Src l4-after-frontier-b0 l4-after-frontier-H
    l4-after-frontier-anchor = (λ-. None)
proof
  fix k :: nat
  show Src l4-after-frontier-b0 l4-after-frontier-H
    l4-after-frontier-anchor k = None
  proof (cases k = 500)
    case True
      thus ?thesis using l4-after-frontier-source-absent-at-anchor-500
      by simp
    next
      case False
      thus ?thesis using l4-after-frontier-source-absent-at-anchor-other
      by simp
  qed
qed

lemma l4-after-frontier-anchor-domain-empty:
  anchor-domain l4-after-frontier-b0 l4-after-frontier-H
    l4-after-frontier-anchor = {}
  by (simp add: anchor-domain-def l4-after-frontier-source-at-anchor-empty)

definition l4-update-refresh-prefix :: (nat, nat) replay-event list where
  l4-update-refresh-prefix =
    [ Cdc ec1 (Update (100::nat) (3000::nat)),
      Refresh 100 (Some 3000) ec1,
      Refresh 200 (Some 12000) ec4,
      Cdc ec2 (Insert 300 2500),
      Cdc ec4 (Update 300 3500) ]

lemma l4-update-refresh-apply-100:
  Apply l4-update-refresh-prefix (100::nat) = Some 3000
  unfolding l4-update-refresh-prefix-def Apply-def by simp

```

```

lemma l4-update-refresh-apply-200:
  Apply l4-update-refresh-prefix (200::nat) = Some 12000
  unfolding l4-update-refresh-prefix-def Apply-def by simp

lemma l4-update-refresh-apply-300:
  Apply l4-update-refresh-prefix (300::nat) = Some 3500
  unfolding l4-update-refresh-prefix-def Apply-def by simp

lemma l4-update-refresh-scoped-virtual-cut-state:
  virtual-cut-state b0-ex l4-update-refresh-prefix {100, 200, 300} ec4 H-ex
  unfolding virtual-cut-state-def restrict-def
proof (rule ext)
  fix k :: nat
  show (if k ∈ {100, 200, 300} then Apply l4-update-refresh-prefix k else None)
    = (if k ∈ {100, 200, 300} then Src b0-ex H-ex ec4 k else None)
  proof (cases k = 100)
    case True
    thus ?thesis
      using l4-update-refresh-apply-100 Src-b0-ex-H-ex-ec4-100 by simp
  next
    case not100: False
    show ?thesis
    proof (cases k = 200)
      case True
      thus ?thesis
        using l4-update-refresh-apply-200 Src-b0-ex-H-ex-ec4-200 by simp
    next
      case not200: False
      show ?thesis
      proof (cases k = 300)
        case True
        thus ?thesis
          using l4-update-refresh-apply-300 Src-b0-ex-H-ex-ec4-300 by simp
      next
        case False
        with not100 not200 show ?thesis by simp
    qed
  qed
qed
qed

lemma l4-update-refresh-source-update-before-refresh:
  (ec3, Update (200::nat) (12000::nat))  $\in$  set H-ex
   $\wedge$  ec3 < ec4
  using ec3-lt-ec4
  by (simp add: H-ex-def less-src-coord-def)

definition l4-unclaimed-prefix :: (nat, nat) replay-event list where
  l4-unclaimed-prefix =

```



```

[ Cdc ec1 (Update (100::nat) (3000::nat)),
  Refresh 100 (Some 3000) ec1,
  Refresh 200 (Some 10000) ec1,
  Cdc ec2 (Insert 300 2500),
  Cdc ec3 (Update 200 12000),
  Refresh 300 (Some 2500) ec3,
  Cdc ec4 (Update 300 3500),
  Refresh 999 (Some 4242) ec4 ]

```

lemma *l4-unclaimed-apply-100*:
Apply l4-unclaimed-prefix (100::nat) = Some 3000
unfolding *l4-unclaimed-prefix-def Apply-def* **by** *simp*

lemma *l4-unclaimed-apply-200*:
Apply l4-unclaimed-prefix (200::nat) = Some 12000
unfolding *l4-unclaimed-prefix-def Apply-def* **by** *simp*

lemma *l4-unclaimed-apply-300*:
Apply l4-unclaimed-prefix (300::nat) = Some 3500
unfolding *l4-unclaimed-prefix-def Apply-def* **by** *simp*

lemma *l4-unclaimed-apply-999*:
Apply l4-unclaimed-prefix (999::nat) = Some 4242
unfolding *l4-unclaimed-prefix-def Apply-def* **by** *simp*

lemma *l4-unclaimed-scoped-virtual-cut-state*:
virtual-cut-state b0-ex l4-unclaimed-prefix {100, 200, 300} ec4 H-ex
unfolding *virtual-cut-state-def restrict-def*

proof (*rule ext*)
fix *k :: nat*
show (*if k ∈ {100, 200, 300} then Apply l4-unclaimed-prefix k else None*)
= (*if k ∈ {100, 200, 300} then Src b0-ex H-ex ec4 k else None*)
proof (*cases k = 100*)
case *True*
thus *?thesis*
using *l4-unclaimed-apply-100 Src-b0-ex-H-ex-ec4-100* **by** *simp*
next
case *not100*: *False*
show *?thesis*
proof (*cases k = 200*)
case *True*
thus *?thesis*
using *l4-unclaimed-apply-200 Src-b0-ex-H-ex-ec4-200* **by** *simp*
next
case *not200*: *False*
show *?thesis*
proof (*cases k = 300*)
case *True*
thus *?thesis*

```

    using l4-unclaimed-apply-300 Src-b0-ex-H-ex-ec4-300 by simp
  next
    case False
    with not100 not200 show ?thesis by simp
  qed
qed
qed
qed

```

definition *l4-b0-anchor-b0* :: *nat* $\rightarrow$ *nat* **where**
l4-b0-anchor-b0 = (λk . if $k = (700::nat)$ then *Some* (7000::*nat*) else *None*)

definition *l4-b0-anchor-H* :: (*nat*, *nat*) *src-history* **where**
l4-b0-anchor-H = []

definition *l4-b0-anchor-prefix* :: (*nat*, *nat*) *replay-event list* **where**
l4-b0-anchor-prefix =
 [Refresh (700::*nat*) (*Some* (7000::*nat*)) *ec1*]

lemma *l4-b0-anchor-base-key-present*:
l4-b0-anchor-b0 (700::*nat*) = *Some* 7000
by (*simp add: l4-b0-anchor-b0-def*)

lemma *l4-b0-anchor-no-source-event-for-key*:
latest-src-event l4-b0-anchor-H l4-anchor (700::*nat*) = *None*
by (*simp add: latest-src-event-def l4-b0-anchor-H-def*)

lemma *l4-b0-anchor-source-at-anchor*:
Src l4-b0-anchor-b0 l4-b0-anchor-H l4-anchor = *l4-b0-anchor-b0*
by (*rule ext*) (*simp add: Src-def latest-src-event-def l4-b0-anchor-H-def*)

lemma *l4-b0-anchor-source-key-at-anchor*:
Src l4-b0-anchor-b0 l4-b0-anchor-H l4-anchor (700::*nat*)
 = *Some* 7000
using *l4-b0-anchor-base-key-present l4-b0-anchor-source-at-anchor*
by *simp*

lemma *l4-b0-anchor-domain*:
anchor-domain l4-b0-anchor-b0 l4-b0-anchor-H l4-anchor = {700}
unfolding *anchor-domain-def*
using *l4-b0-anchor-source-at-anchor*
by (*auto simp: l4-b0-anchor-b0-def*)

definition *l4-same-boundary-b0* :: *nat* $\rightarrow$ *nat* **where**
l4-same-boundary-b0 = (λ -. *None*)

definition *l4-same-boundary-H* :: (*nat*, *nat*) *src-history* **where**
l4-same-boundary-H =

[(ec1, Insert (800::nat) (8000::nat)),
 (ec4, Insert (900::nat) (9000::nat))]

definition *l4-same-boundary-prefix* :: (nat, nat) replay-event list **where**
l4-same-boundary-prefix =
 [Cdc ec1 (Insert (800::nat) (8000::nat)),
 Cdc ec4 (Insert (900::nat) (9000::nat))]

lemma *l4-same-boundary-anchor-event-in-history*:
 (ec1, Insert (800::nat) (8000::nat)) ∈ set *l4-same-boundary-H*
by (simp add: *l4-same-boundary-H-def*)

lemma *l4-same-boundary-frontier-event-in-history*:
 (ec4, Insert (900::nat) (9000::nat)) ∈ set *l4-same-boundary-H*
by (simp add: *l4-same-boundary-H-def*)

lemma *l4-same-boundary-latest-anchor-800*:
 latest-src-event *l4-same-boundary-H* *l4-anchor* (800::nat) = Some 0
unfolding latest-src-event-def *l4-same-boundary-H-def* *l4-anchor-def*
using src-le-refl[**where** c=ec1] not-src-le-ec4-ec1
by simp

lemma *l4-same-boundary-latest-anchor-900*:
 latest-src-event *l4-same-boundary-H* *l4-anchor* (900::nat) = None
unfolding latest-src-event-def *l4-same-boundary-H-def* *l4-anchor-def*
using src-le-refl[**where** c=ec1] not-src-le-ec4-ec1
by simp

lemma *l4-same-boundary-source-anchor-800*:
 Src *l4-same-boundary-b0* *l4-same-boundary-H* *l4-anchor* (800::nat)
 = Some 8000
unfolding Src-def
using *l4-same-boundary-latest-anchor-800*
by (simp add: *l4-same-boundary-H-def*)

lemma *l4-same-boundary-source-anchor-900*:
 Src *l4-same-boundary-b0* *l4-same-boundary-H* *l4-anchor* (900::nat) = None
unfolding Src-def *l4-same-boundary-b0-def*
using *l4-same-boundary-latest-anchor-900* **by** simp

lemma *l4-same-boundary-anchor-domain*:
 anchor-domain *l4-same-boundary-b0* *l4-same-boundary-H* *l4-anchor* = {800}
unfolding anchor-domain-def *l4-same-boundary-b0-def*
l4-same-boundary-H-def *l4-anchor-def* Src-def latest-src-event-def
using src-le-refl[**where** c=ec1] not-src-le-ec4-ec1
by auto

lemma *l4-same-boundary-anchor-key-in-anchor-domain*:
 (800::nat) ∈ anchor-domain *l4-same-boundary-b0*

l4-same-boundary-H l4-anchor
using *l4-same-boundary-anchor-domain* **by** *simp*

lemma *l4-same-boundary-frontier-key-not-in-anchor-domain:*
 $(900::nat) \notin \text{anchor-domain } l4\text{-same-boundary-b0}$
l4-same-boundary-H l4-anchor
using *l4-same-boundary-anchor-domain* **by** *simp*

21 Further anchor-domain facts

Pure *anchor-domain* computation facts on the fixture data above. They involve no run, chunk, or certificate carrier and stand on their own, with no constructed *Layer4-Fixtures-Inst* counterpart.

lemma *l4-claim-refresh-mismatch-anchor-domain:*
 $\text{anchor-domain } b0\text{-ex } H\text{-ex } l4\text{-anchor} = \{100, 200\}$
by (*rule l4-anchor-domain*)

lemma *l4-update-refresh-key-200-anchor-domain:*
 $(200::nat) \in \text{anchor-domain } b0\text{-ex } H\text{-ex } l4\text{-anchor}$
using *l4-anchor-domain* **by** *simp*

lemma *l4-b0-anchor-key-in-anchor-domain:*
 $(700::nat) \in \text{anchor-domain } l4\text{-b0-anchor-b0 } l4\text{-b0-anchor-H } l4\text{-anchor}$
using *l4-b0-anchor-domain* **by** *simp*

end

theory *Layer2-Fixtures-Core*
imports *DBLog-Run-Core Dual-Write-Layer0.Source-Coordinates*
begin

22 Layer 2 canonical-clean-prefix and frontier fixtures

Carrier-independent Layer 2 fixtures: the canonical-clean-prefix structural fixtures and the frontier off-by-one subcases. They range only over the pure *canonical-clean-prefix* / *cdc-event-replays* / *chunk-read-refreshes* constructions of *DBLog-Virtual-Cuts.DBLog-Run-Core* and the source coordinates of *Dual-Write-Layer0.Source-Coordinates* — no run/chunk carrier is involved, so these fixtures have no constructed *Layer2-Fixtures-Inst* counterpart.

22.1 Canonical-clean-prefix structural fixtures

definition *l2-one-refresh-read* :: $(nat, nat) \rightarrow \text{chunk-read-record}$ **where**
l2-one-refresh-read =

```

  | crr-domain = {1},
  crr-read-coord = c0,
  crr-observation =
    (λk. if k = (1::nat) then Some (Some (5::nat)) else None) |

```

lemma *l2-one-refreshes*:

```

  chunk-read-refreshes l2-one-refresh-read =
    [Refresh (1::nat) (Some (5::nat)) c0]
unfolding l2-one-refresh-read-def chunk-read-refreshes-def
  List.map-filter-def
by simp

```

theorem *fix-layer2-misordered-clean-prefix-rejected*:

```

  canonical-clean-prefix
    [l2-one-refresh-read]
    [(l2-cp-c1, Update (1::nat) (7::nat))]
    {1} l2-cp-c1
  ≠ [Cdc l2-cp-c1 (Update (1::nat) (7::nat)),
     Refresh (1::nat) (Some (5::nat)) c0]
using l2-cp-c0-lt-c1 l2-cp-not-le-c0
unfolding canonical-clean-prefix-def cdc-event-replays-def
  List.map-filter-def
by (simp add: l2-one-refreshes less-src-coord-def src-lt-def)

```

theorem *fix-layer2-extra-refresh-in-scope-rejected*:

```

  canonical-clean-prefix [l2-one-refresh-read] [] {1} c0
  ≠ [Refresh (1::nat) (Some (5::nat)) c0,
     Refresh (1::nat) (Some (99::nat)) c0]
unfolding canonical-clean-prefix-def cdc-event-replays-def
  List.map-filter-def
by (simp add: l2-one-refreshes)

```

theorem *fix-layer2-future-cdc-in-clean-prefix-rejected*:

```

  Cdc l2-future-c1 (Update (1::nat) (7::nat))
  ∉ set (canonical-clean-prefix []
    [(l2-future-c1, Update (1::nat) (7::nat))] {1} c0)
using l2-future-not-le-c0
unfolding canonical-clean-prefix-def cdc-event-replays-def
  List.map-filter-def
by simp

```

22.2 Frontier off-by-one subcases

theorem *fix-layer2-frontier-off-by-one-before-included*:

```

  Cdc l2-fo-before (Update (1::nat) (7::nat))
  ∈ set (cdc-event-replays {1} l2-fo-frontier
    [(l2-fo-before, Update (1::nat) (7::nat))])
using l2-fo-before-le-frontier
unfolding cdc-event-replays-def List.map-filter-def

```

```

    by simp

theorem fix-layer2-frontier-off-by-one-equal-included:
  Cdc l2-fo-frontier (Update (1::nat) (7::nat))
    ∈ set (cdc-event-replays {1} l2-fo-frontier
      [(l2-fo-frontier, Update (1::nat) (7::nat))])
  unfolding cdc-event-replays-def List.map-filter-def
  by (simp add: less-eq-src-coord-def src-le-refl)

theorem fix-layer2-frontier-off-by-one-after-rejected:
  Cdc l2-fo-after (Update (1::nat) (7::nat))
    ∉ set (cdc-event-replays {1} l2-fo-frontier
      [(l2-fo-after, Update (1::nat) (7::nat))])
  using l2-fo-after-not-le-frontier
  unfolding cdc-event-replays-def List.map-filter-def
  by simp

end

theory Layer01-Virtual-Cut-Example-Model
  imports Layer01-Virtual-Cut-Example-Core
begin

```

23 Constructed model for the running-example run configuration

Purpose. This theory exhibits the explicit model of the running example's run configuration: a one-point run carrier (*unit*), a two-element chunk type, and concrete accessor functions giving the example's Layer 1 accessor values (scope, frontier, chunk domains, responsible chunk, watermarks, read coordinates, read results, source history, observed CDC events, chunk enumeration).

Layer01-Virtual-Cut-Example-Inst interprets the run-substrate locale (*dblog-run-substrate*) at this model and proves, over it, that the example run is wellformed (*ex-model-wellformed*) and satisfies the virtual-cut equality (*ex-model-virtual-cut-holds*) — so the running example is established constructively, with no axioms. The constructed Layer 3 / Layer 4 witness interpretation (*Layer3-Witnesses-Inst*) reuses the same model as its run side.

The carrier-independent example data this model realizes (*b0-ex*, *H-ex*, the *Src* / *latest-src-event* computation lemmas, *wf-h-ex*) comes from the imported *DBLog-Virtual-Cuts.Layer01-Virtual-Cut-Example-Core*.

23.1 Concrete carriers and accessor functions

A two-element chunk type. Datatype distinctness gives $ChA \neq ChB$ for free, so the chunk enumeration $[ChA, ChB]$ is duplicate-free — the shape the run-substrate locale’s chunk-enumeration assumptions require.

datatype *cchunk* = $ChA \mid ChB$

The run carrier is a single point (*unit*); the example concerns one run only, so one inhabitant suffices. Each accessor is a total function returning the example’s intended value.

definition *cscope-of* :: $unit \Rightarrow nat\ set$ **where**
cscope-of - = $\{100, 200, 300\}$

definition *cfrontier-of* :: $unit \Rightarrow frontier$ **where**
cfrontier-of - = $ec4$

definition *cchunks* :: $unit \Rightarrow cchunk\ set$ **where**
cchunks - = $\{ChA, ChB\}$

definition *cchunks-list* :: $unit \Rightarrow cchunk\ list$ **where**
cchunks-list - = $[ChA, ChB]$

definition *cchunk-domain* :: $unit \Rightarrow cchunk \Rightarrow nat\ set$ **where**
cchunk-domain - $ch = (case\ ch\ of\ ChA \Rightarrow \{100, 200\} \mid ChB \Rightarrow \{300\})$

definition *cresponsible-chunk* :: $unit \Rightarrow nat \Rightarrow cchunk\ option$ **where**
cresponsible-chunk - $k =$
 (if $k = 100 \vee k = 200$ then *Some* ChA
 else if $k = 300$ then *Some* ChB else *None*)

definition *cchunk-read-coordinate* :: $unit \Rightarrow cchunk \Rightarrow src\ coord$ **where**
cchunk-read-coordinate - $ch = (case\ ch\ of\ ChA \Rightarrow ec1 \mid ChB \Rightarrow ec3)$

definition *cchunk-lower-watermark* :: $unit \Rightarrow cchunk \Rightarrow src\ coord$ **where**
cchunk-lower-watermark - $ch = (case\ ch\ of\ ChA \Rightarrow ec1 \mid ChB \Rightarrow ec3)$

definition *cchunk-upper-watermark* :: $unit \Rightarrow cchunk \Rightarrow src\ coord$ **where**
cchunk-upper-watermark - $ch = (case\ ch\ of\ ChA \Rightarrow ec1 \mid ChB \Rightarrow ec3)$

definition *csrc-history-of* :: $unit \Rightarrow (nat, nat)\ src\ history$ **where**
csrc-history-of - = $H\text{-}ex$

definition *ccdc-events-of* :: $unit \Rightarrow (nat, nat)\ src\ history$ **where**
ccdc-events-of - = $H\text{-}ex$

definition *cchunk-read-result* :: $unit \Rightarrow cchunk \Rightarrow nat \Rightarrow nat\ option\ option$ **where**
cchunk-read-result - $ch\ k =$
 (if $ch = ChA \wedge k = 100$ then *Some* (*Some* 3000)
 else if $ch = ChA \wedge k = 200$ then *Some* (*Some* 10000)

else if $ch = ChB \wedge k = 300$ then $Some (Some 2500)$
 else $None$)

23.2 The model realizes the example's accessor values

Each conjunct below records one accessor value of the constructed model, ending with the two chunk-enumeration facts ($set (cchunks-list ()) = cchunks ()$ and $distinct (cchunks-list ())$) that the run-substrate locale assumes. *Layer01-Virtual-Cut-Example-Inst* re-derives these equations as its per-accessor lemmas when interpreting the locale at this model.

theorem *running-example-model-accessor-values*:

$ChA \neq ChB$
 $\wedge cscope-of () = \{100, 200, 300\}$
 $\wedge cfrontier-of () = ec4$
 $\wedge cchunks () = \{ChA, ChB\}$
 $\wedge cchunk-domain () ChA = \{100, 200\}$
 $\wedge cchunk-domain () ChB = \{300\}$
 $\wedge cresponsible-chunk () 100 = Some ChA$
 $\wedge cresponsible-chunk () 200 = Some ChA$
 $\wedge cresponsible-chunk () 300 = Some ChB$
 $\wedge (\forall k. k \neq 100 \longrightarrow k \neq 200 \longrightarrow k \neq 300 \longrightarrow cresponsible-chunk () k = None)$
 $\wedge cchunk-read-coordinate () ChA = ec1$
 $\wedge cchunk-read-coordinate () ChB = ec3$
 $\wedge cchunk-lower-watermark () ChA = ec1$
 $\wedge cchunk-upper-watermark () ChA = ec1$
 $\wedge cchunk-lower-watermark () ChB = ec3$
 $\wedge cchunk-upper-watermark () ChB = ec3$
 $\wedge csrc-history-of () = H-ex$
 $\wedge cchunk-read-result () ChA 100 = Some (Some 3000)$
 $\wedge cchunk-read-result () ChA 200 = Some (Some 10000)$
 $\wedge cchunk-read-result () ChB 300 = Some (Some 2500)$
 $\wedge ccdc-events-of () = H-ex$
 $\wedge cchunks-list () = [ChA, ChB]$
 $\wedge set (cchunks-list ()) = cchunks ()$
 $\wedge distinct (cchunks-list ())$
by (*auto simp: cscope-of-def cfrontier-of-def cchunks-def cchunks-list-def*
cchunk-domain-def cresponsible-chunk-def
cchunk-read-coordinate-def cchunk-lower-watermark-def
cchunk-upper-watermark-def csrc-history-of-def
ccdc-events-of-def cchunk-read-result-def)

Conclusion. The running example's accessor values are jointly realized by an explicit construction: a concrete run with exactly these values exists. *Layer01-Virtual-Cut-Example-Inst* interprets the run-substrate locale at this model and proves the example's wellformedness (*ex-model-wellformed*) and virtual-cut equality (*ex-model-virtual-cut-holds*) for it.

end


```

theory Layer01-Virtual-Cut-Example-Inst
  imports Layer01-Virtual-Cut-Example-Model DBLog-Run-Substrate-Layer2
begin

```

24 Constructed interpretation for the running-example virtual cut

Constructs the paper’s running example (the accounts-table backfill: three in-scope keys, two chunks, four CDC events, a seven-event clean prefix, and the full per-key replay equality) as an explicit interpretation of the Layer 1 locale *dblog-run-substrate* at the concrete model of *DBLog-Virtual-Cuts.Layer01-Virtual-Cut-Example-Model* (the *unit* run carrier, the *cchunk* two-element chunk datatype, the *c...* definitional accessors), establishing the example facts with no axioms.

Method (the *Layer01-Witnesses-Inst* pattern): instantiate *dblog-run-substrate* at the model carrier (the two inherited *chunks-list* obligations discharge by *simp*, no axiom); read off the model’s definitional accessor values pointwise as the *exm-...* lemmas (the conjuncts of *running-example-model-accessor-values*); and prove the determinate seven-event clean-prefix shape, the *Apply* computation, the seven-clause wellformedness, and the virtual-cut replay equality from those rewrite facts. As in *Layer01-Witnesses-Inst* the concrete-*nat* wellformedness is proved at the interpretation in the global context (the locale type parameter '*k*' is rigid inside the locale context).

The example source coordinates *ec1..ec4* are *coord-of-nat* definitions from *Dual-Write-Layer0.Source-Coordinates*, with their order facts proved there; the carrier-independent example data (*b0-ex*, *H-ex*, the *Src* / *latest-src-event* computation lemmas, *wf-h-ex*, and further coordinate order lemmas) lives in *DBLog-Virtual-Cuts.Layer01-Virtual-Cut-Example-Core*.

24.1 Constructed interpretation at the running-example model

```

global-interpretation ex-model: dblog-run-substrate
  cscope-of
  cfrontier-of
  cchunks
  csrc-history-of
  cchunk-read-result
  ccdc-events-of
  cchunks-list
  cchunk-domain
  cresponsible-chunk
  cchunk-lower-watermark

```

cchunk-upper-watermark
cchunk-read-coordinate
by *unfold-locales* (*simp-all* add: *cchunks-def* *cchunks-list-def*)

24.2 Model accessor-value lemmas

lemma *exm-distinct*: $ChA \neq ChB$ **by** *simp*
lemma *exm-scope*: *cscope-of* () = {100, 200, 300} **by** (*simp* add: *cscope-of-def*)
lemma *exm-frontier*: *cfrontier-of* () = *ec4* **by** (*simp* add: *cfrontier-of-def*)
lemma *exm-chunks*: *cchunks* () = {*ChA*, *ChB*} **by** (*simp* add: *cchunks-def*)
lemma *exm-dom-A*: *cchunk-domain* () *ChA* = {100, 200}
 by (*simp* add: *cchunk-domain-def*)
lemma *exm-dom-B*: *cchunk-domain* () *ChB* = {300}
 by (*simp* add: *cchunk-domain-def*)
lemma *exm-resp-100*: *cresponsible-chunk* () 100 = *Some ChA*
 by (*simp* add: *cresponsible-chunk-def*)
lemma *exm-resp-200*: *cresponsible-chunk* () 200 = *Some ChA*
 by (*simp* add: *cresponsible-chunk-def*)
lemma *exm-resp-300*: *cresponsible-chunk* () 300 = *Some ChB*
 by (*simp* add: *cresponsible-chunk-def*)
lemma *exm-resp-else*:
 $k \neq 100 \implies k \neq 200 \implies k \neq 300$
 $\implies \text{cresponsible-chunk } () \ k = \text{None}$
 by (*simp* add: *cresponsible-chunk-def*)
lemma *exm-rcoord-A*: *cchunk-read-coordinate* () *ChA* = *ec1*
 by (*simp* add: *cchunk-read-coordinate-def*)
lemma *exm-rcoord-B*: *cchunk-read-coordinate* () *ChB* = *ec3*
 by (*simp* add: *cchunk-read-coordinate-def*)
lemma *exm-lwm-A*: *cchunk-lower-watermark* () *ChA* = *ec1*
 by (*simp* add: *cchunk-lower-watermark-def*)
lemma *exm-uwmm-A*: *cchunk-upper-watermark* () *ChA* = *ec1*
 by (*simp* add: *cchunk-upper-watermark-def*)
lemma *exm-lwm-B*: *cchunk-lower-watermark* () *ChB* = *ec3*
 by (*simp* add: *cchunk-lower-watermark-def*)
lemma *exm-uwmm-B*: *cchunk-upper-watermark* () *ChB* = *ec3*
 by (*simp* add: *cchunk-upper-watermark-def*)
lemma *exm-src-history*: *csrc-history-of* () = *H-ex*
 by (*simp* add: *csrc-history-of-def*)
lemma *exm-crr-A-100*: *cchunk-read-result* () *ChA* 100 = *Some (Some 3000)*
 by (*simp* add: *cchunk-read-result-def*)
lemma *exm-crr-A-200*: *cchunk-read-result* () *ChA* 200 = *Some (Some 10000)*
 by (*simp* add: *cchunk-read-result-def*)
lemma *exm-crr-B-300*: *cchunk-read-result* () *ChB* 300 = *Some (Some 2500)*
 by (*simp* add: *cchunk-read-result-def*)
lemma *exm-cdc*: *ccdc-events-of* () = *H-ex*
 by (*simp* add: *ccdc-events-of-def*)
lemma *exm-chunks-list*: *cchunks-list* () = [*ChA*, *ChB*]
 by (*simp* add: *cchunks-list-def*)

lemma *exm-chunks-eq*:
 $(ch \in cchunks ()) \longleftrightarrow (ch = ChA \vee ch = ChB)$
using *exm-chunks* **by** *simp*

24.3 Determinate clean prefix and Apply over the model

lemma *exm-clean-prefix*:
 $ex-model.clean-prefix-of ()$
 $= [Cdc\ ec1\ (Update\ 100\ 3000),$
 $\quad Refresh\ (100::nat)\ (Some\ (3000::nat))\ ec1,$
 $\quad Refresh\ 200\ (Some\ 10000)\ ec1,$
 $\quad Cdc\ ec2\ (Insert\ 300\ 2500),$
 $\quad Cdc\ ec3\ (Update\ 200\ 12000),$
 $\quad Refresh\ 300\ (Some\ 2500)\ ec3,$
 $\quad Cdc\ ec4\ (Update\ 300\ 3500)]$
proof –
have *dA*: *sorted-list-of-set* $\{100::nat, 200\} = [100, 200]$ **by** *simp*
have *dB*: *sorted-list-of-set* $\{300::nat\} = [300]$ **by** *simp*
show *?thesis*
unfolding *ex-model.clean-prefix-of-def canonical-clean-prefix-def*
cdc-event-replays-def chunk-read-refreshes-def
by (*simp add: exm-chunks-list exm-dom-A exm-dom-B*
exm-rcoord-A exm-rcoord-B
exm-crr-A-100 exm-crr-A-200
exm-crr-B-300
exm-cdc exm-scope exm-frontier H-ex-def dA dB
List.map-filter-simps sort-key-def
ec1-le-ec2[*unfolded src-le-eq-less-eq*]
ec1-le-ec3[*unfolded src-le-eq-less-eq*]
ec1-le-ec4[*unfolded src-le-eq-less-eq*]
ec2-le-ec3[*unfolded src-le-eq-less-eq*]
ec2-le-ec4[*unfolded src-le-eq-less-eq*]
ec3-le-ec4[*unfolded src-le-eq-less-eq*]
not-src-le-ec2-ec1[*unfolded src-le-eq-less-eq*]
not-src-le-ec3-ec1[*unfolded src-le-eq-less-eq*]
not-src-le-ec4-ec1[*unfolded src-le-eq-less-eq*]
not-src-le-ec3-ec2[*unfolded src-le-eq-less-eq*]
not-src-le-ec4-ec2[*unfolded src-le-eq-less-eq*]
not-src-le-ec4-ec3[*unfolded src-le-eq-less-eq*])

qed

lemma *exm-apply*:
 $Apply\ (ex-model.clean-prefix-of ())$
 $= (\lambda k. \text{if } k = (100::nat) \text{ then } Some\ (3000::nat)$
 $\quad \text{else if } k = 200 \text{ then } Some\ 12000$
 $\quad \text{else if } k = 300 \text{ then } Some\ 3500$
 $\quad \text{else } None)$

proof –
have *step*:

```

Apply (ex-model.clean-prefix-of ())
= foldl apply-step (λ-. None)
  [ Cdc ec1 (Update 100 3000),
    Refresh (100::nat) (Some (3000::nat)) ec1,
    Refresh 200 (Some 10000) ec1,
    Cdc ec2 (Insert 300 2500),
    Cdc ec3 (Update 200 12000),
    Refresh 300 (Some 2500) ec3,
    Cdc ec4 (Update 300 3500) ]
unfolding Apply-def exm-clean-prefix by (rule refl)
show ?thesis
unfolding step by (auto simp: fun-eq-iff fun-upd-def)
qed

```

24.4 Constructed wellformed-run fact

```

theorem ex-model-wellformed:
  ex-model.wellformed-dblog-run b0-ex () H-ex
  unfolding ex-model.wellformed-dblog-run-def
proof (intro conjI)
  show csrc-history-of () = H-ex by (rule exm-src-history)
next
  show wellformed-src-history H-ex by (rule wf-h-ex)
next
  show ∀ k ∈ cscope-of (). ∃! ch. ch ∈ cchunks () ∧ k ∈ cchunk-domain () ch
  proof
    fix k :: nat assume k ∈ cscope-of ()
    hence k-cases: k = 100 ∨ k = 200 ∨ k = 300
    using exm-scope by simp
    show ∃! ch. ch ∈ cchunks () ∧ k ∈ cchunk-domain () ch
    proof (cases k = 100)
      case True
      have ChA ∈ cchunks () ∧ k ∈ cchunk-domain () ChA
      using exm-chunks exm-dom-A True by simp
      moreover have ∧ ch. ch ∈ cchunks () ∧ k ∈ cchunk-domain () ch
        ⇒ ch = ChA
      using exm-chunks-eq exm-dom-A exm-dom-B True by auto
      ultimately show ?thesis by blast
    next
      case False
      hence k-or: k = 200 ∨ k = 300 using k-cases by simp
      show ?thesis
      proof (cases k = 200)
        case True
        have ChA ∈ cchunks () ∧ k ∈ cchunk-domain () ChA
        using exm-chunks exm-dom-A True by simp
        moreover have ∧ ch. ch ∈ cchunks () ∧ k ∈ cchunk-domain () ch
          ⇒ ch = ChA
        using exm-chunks-eq exm-dom-A exm-dom-B True by auto

```

```

    ultimately show ?thesis by blast
  next
    case False
    hence k300: k = 300 using k-or by simp
    have ChB ∈ cchunks () ∧ k ∈ cchunk-domain () ChB
      using exm-chunks exm-dom-B k300 by simp
    moreover have  $\bigwedge ch. ch \in cchunks () \wedge k \in cchunk-domain () \implies ch = ChB$ 
      using exm-chunks-eq exm-dom-A exm-dom-B k300 by auto
    ultimately show ?thesis by blast
  qed
qed
qed
next
  show  $\forall ch1 \in cchunks (). \forall ch2 \in cchunks ().$ 
     $ch1 \neq ch2$ 
     $\longrightarrow cchunk-domain () \ ch1 \cap cchunk-domain () \ ch2 = \{\}$ 
    using exm-chunks-eq exm-dom-A exm-dom-B by auto
  next
    show ex-model.canonical-chunk-ownership-domain () = cscope-of ()
      unfolding ex-model.canonical-chunk-ownership-domain-def
      using exm-chunks exm-dom-A exm-dom-B exm-scope by auto
  next
    show  $\forall k \in cscope-of (). \forall ch.$ 
       $cresponsible-chunk () \ k = Some \ ch$ 
       $\longleftrightarrow ch \in cchunks () \wedge k \in cchunk-domain () \ ch$ 
    proof (intro ballI allI)
      fix k :: nat and ch :: cchunk
      assume k ∈ cscope-of ()
      hence k-cases: k = 100 ∨ k = 200 ∨ k = 300
        using exm-scope by simp
      show responsible-chunk () k = Some ch
         $\longleftrightarrow ch \in cchunks () \wedge k \in cchunk-domain () \ ch$ 
      proof (cases k = 100)
        case True
        then have rc: responsible-chunk () k = Some ChA
          using exm-resp-100 by simp
        show ?thesis
        proof
          assume responsible-chunk () k = Some ch
          with rc have ch = ChA by simp
          thus ch ∈ cchunks () ∧ k ∈ cchunk-domain () ch
            using exm-chunks exm-dom-A True by simp
        next
          assume A: ch ∈ cchunks () ∧ k ∈ cchunk-domain () ch
          have ch = ChA
            using A exm-chunks-eq exm-dom-A exm-dom-B True by auto
          with rc show responsible-chunk () k = Some ch by simp
        qed
      qed
    qed
  qed

```

```

next
  case False
  hence k-or:  $k = 200 \vee k = 300$  using k-cases by simp
  show ?thesis
  proof (cases  $k = 200$ )
    case True
    then have rc: cresponsible-chunk ()  $k = \text{Some } ChA$ 
      using exm-resp-200 by simp
    show ?thesis
    proof
      assume cresponsible-chunk ()  $k = \text{Some } ch$ 
      with rc have  $ch = ChA$  by simp
      thus  $ch \in cchunks () \wedge k \in cchunk-domain ()$  ch
        using exm-chunks exm-dom-A True by simp
    next
      assume A:  $ch \in cchunks () \wedge k \in cchunk-domain ()$  ch
      have  $ch = ChA$ 
        using A exm-chunks-eq exm-dom-A exm-dom-B True by auto
      with rc show cresponsible-chunk ()  $k = \text{Some } ch$  by simp
    qed
  next
  case False
  hence k300:  $k = 300$  using k-or by simp
  then have rc: cresponsible-chunk ()  $k = \text{Some } ChB$ 
    using exm-resp-300 by simp
  show ?thesis
  proof
    assume cresponsible-chunk ()  $k = \text{Some } ch$ 
    with rc have  $ch = ChB$  by simp
    thus  $ch \in cchunks () \wedge k \in cchunk-domain ()$  ch
      using exm-chunks exm-dom-B k300 by simp
  next
    assume A:  $ch \in cchunks () \wedge k \in cchunk-domain ()$  ch
    have  $ch = ChB$ 
      using A exm-chunks-eq exm-dom-A exm-dom-B k300 by auto
    with rc show cresponsible-chunk ()  $k = \text{Some } ch$  by simp
  qed
qed
qed
qed
next
  show  $\forall k. k \notin cscope-of () \longrightarrow c\text{responsible-chunk} () k = \text{None}$ 
    using exm-scope exm-resp-else by auto
next
  show finite (cscope-of ()) using exm-scope by simp
next
  show  $\forall ch \in cchunks (). \text{finite} (cchunk-domain () ch)$ 
    using exm-chunks-eq exm-dom-A exm-dom-B by auto
next

```

```

show  $\forall ch \in cchunks \ (). \ cchunk\_domain \ () \ ch \neq \{\}$ 
  using exm-chunks-eq exm-dom-A exm-dom-B by auto
next
show  $\forall k \in cscope\_of \ ().$ 
  ex-model.covers-ordinary-cdc \ () \ k \ (cfrontier-of \ ())
proof
  fix k :: nat assume k  $\in cscope\_of \ ()$ 
  hence k-cases: k = 100  $\vee$  k = 200  $\vee$  k = 300
  using exm-scope by simp
  show ex-model.covers-ordinary-cdc \ () \ k \ (cfrontier-of \ ())
  proof (cases k = 100)
    case True
    show ?thesis
      unfolding ex-model.covers-ordinary-cdc-def
    proof (intro exI[where x = ChA] conjI allI impI)
      show cresponsible-chunk \ () \ k = Some ChA
        using exm-resp-100 True by simp
    next
      fix i
      assume i-lt: i < length (csrc-history-of \ ())
        and chunk-lt: src-lt (cchunk-read-coordinate \ () \ ChA)
          (hist-coord (csrc-history-of \ () \ ! i))
        and to-f: src-le (hist-coord (csrc-history-of \ () \ ! i))
          (cfrontier-of \ ())
        and key-eq: key-of (hist-event (csrc-history-of \ () \ ! i)) = k
      have i-lt4: i < 4
        using i-lt exm-src-history length-H-ex by simp
      have i-cases: i = 0  $\vee$  i = 1  $\vee$  i = 2  $\vee$  i = 3
        using i-lt4 by (rule less-4-cases)
      have False
      proof -
        consider (a) i = 0 | (b) i = 1
          | (c) i = 2 | (d) i = 3
        using i-cases by blast
      thus False
    proof cases
      case a
      then show False using chunk-lt exm-src-history
        exm-rcoord-A H-ex-nth-0
        not-src-lt-ec1-ec1 by simp
    next
      case b
      then show False using key-eq exm-src-history
        H-ex-nth-1 True by simp
    next
      case c
      then show False using key-eq exm-src-history
        H-ex-nth-2 True by simp
    next

```

```

      case d
      then show False using key-eq exm-src-history
                                H-ex-nth-3 True by simp

    qed
  qed
  thus csrc-history-of () ! i ∈ set (ccdc-events-of ())
  ..
qed
next
case False
hence k-or: k = 200 ∨ k = 300 using k-cases by simp
show ?thesis
proof (cases k = 200)
  case True
  show ?thesis
    unfolding ex-model.covers-ordinary-cdc-def
  proof (intro exI[where x = ChA] conjI allI impI)
    show crespensible-chunk () k = Some ChA
      using exm-resp-200 True by simp
  next
  fix i
  assume i-lt: i < length (csrc-history-of ())
    and chunk-lt: src-lt (cchunk-read-coordinate () ChA)
                      (hist-coord (csrc-history-of () ! i))
    and to-f: src-le (hist-coord (csrc-history-of () ! i))
                  (cfrontier-of ())
    and key-eq: key-of (hist-event (csrc-history-of () ! i)) = k
  have i-lt4: i < 4
    using i-lt exm-src-history length-H-ex by simp
  have i-cases: i = 0 ∨ i = 1 ∨ i = 2 ∨ i = 3
    using i-lt4 by (rule less-4-cases)
  consider (a) i = 0 | (b) i = 1
    | (c) i = 2 | (d) i = 3
    using i-cases by blast
  thus csrc-history-of () ! i ∈ set (ccdc-events-of ())
  proof cases
    case a
    then show ?thesis using chunk-lt exm-src-history
                        exm-rcoord-A H-ex-nth-0
                        not-src-lt-ec1-ec1 by simp
  next
  case b
  then show ?thesis using key-eq exm-src-history
                        H-ex-nth-1 True by simp
  next
  case c
  then show ?thesis using exm-src-history exm-cdc
                        H-ex-nth-2 set-H-ex c by simp
  next

```



```

      case d
      then show ?thesis using key-eq exm-src-history
                    H-ex-nth-3 True by simp

    qed
  qed
next
case False
hence k300:  $k = 300$  using k-or by simp
show ?thesis
  unfolding ex-model.covers-ordinary-cdc-def
proof (intro exI[where  $x = ChB$ ] conjI allI impI)
  show crespensible-chunk ()  $k = \text{Some } ChB$ 
    using exm-resp-300 k300 by simp
next
fix i
assume i-lt:  $i < \text{length } (csrc\text{-history-of } ())$ 
  and chunk-lt:  $\text{src-lt } (cchunk\text{-read-coordinate } () \ ChB)$ 
                 $(\text{hist-coord } (csrc\text{-history-of } () \ ! \ i))$ 
  and to-f:  $\text{src-le } (\text{hist-coord } (csrc\text{-history-of } () \ ! \ i))$ 
             $(cfrontier\text{-of } ())$ 
  and key-eq:  $\text{key-of } (\text{hist-event } (csrc\text{-history-of } () \ ! \ i)) = k$ 
have i-lt4:  $i < 4$ 
  using i-lt exm-src-history length-H-ex by simp
have i-cases:  $i = 0 \vee i = 1 \vee i = 2 \vee i = 3$ 
  using i-lt4 by (rule less-4-cases)
consider (a)  $i = 0$  | (b)  $i = 1$ 
          | (c)  $i = 2$  | (d)  $i = 3$ 
  using i-cases by blast
thus  $csrc\text{-history-of } () \ ! \ i \in \text{set } (ccdc\text{-events-of } ())$ 
proof cases
case a
  then show ?thesis using key-eq exm-src-history
                    H-ex-nth-0 k300 by simp
next
case b
  then show ?thesis using chunk-lt exm-src-history
                    exm-rcoord-B H-ex-nth-1
                    not-src-lt-ec3-ec2 by simp
next
case c
  then show ?thesis using chunk-lt exm-src-history
                    exm-rcoord-B H-ex-nth-2
                    not-src-lt-ec3-ec3 by simp
next
case d
  then show ?thesis using exm-src-history exm-cdc
                    H-ex-nth-3 set-H-ex d by simp
qed
qed

```

```

    qed
  qed
  qed
next
  show  $\forall p \in \text{set } (\text{ccdc-events-of } ())$ .
     $\text{key-of } (\text{hist-event } p) \in \text{cscope-of } ()$ 
     $\wedge \text{src-le } (\text{hist-coord } p) (\text{cfrontier-of } ())$ 
     $\longrightarrow p \in \text{set } (\text{csrc-history-of } ())$ 
    using exm-cdc exm-src-history by auto
next
  show  $\exists \iota :: \text{nat} \Rightarrow \text{nat}.$  strict-mono  $\iota$ 
     $\wedge (\text{let } \text{in-slice} = (\lambda p. \text{key-of } (\text{hist-event } p) \in \text{cscope-of } ()$ 
       $\wedge \text{src-le } (\text{hist-coord } p) (\text{cfrontier-of } ());$ 
       $\text{cdc-in-slice} = \text{filter in-slice } (\text{ccdc-events-of } ());$ 
       $\text{hist-in-slice} = \text{filter in-slice } (\text{csrc-history-of } ());$ 
       $\text{in } \forall i. i < \text{length cdc-in-slice} \longrightarrow$ 
       $\iota i < \text{length hist-in-slice}$ 
       $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! \iota i)$ 
  proof (intro exI [where  $x = \lambda n :: \text{nat}. n$ ] conjI)
    show strict-mono  $(\lambda n :: \text{nat}. n)$  by (simp add: strict-mono-def)
  next
    have eq:  $\text{ccdc-events-of } () = \text{csrc-history-of } ()$ 
      using exm-cdc exm-src-history by simp
    show  $\text{let } \text{in-slice} = (\lambda p. \text{key-of } (\text{hist-event } p) \in \text{cscope-of } ()$ 
       $\wedge \text{src-le } (\text{hist-coord } p) (\text{cfrontier-of } ());$ 
       $\text{cdc-in-slice} = \text{filter in-slice } (\text{ccdc-events-of } ());$ 
       $\text{hist-in-slice} = \text{filter in-slice } (\text{csrc-history-of } ());$ 
       $\text{in } \forall i. i < \text{length cdc-in-slice} \longrightarrow$ 
       $(\lambda n :: \text{nat}. n) i < \text{length hist-in-slice}$ 
       $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! (\lambda n :: \text{nat}. n) i$ 
      using eq by (simp add: Let-def)
    qed
  next
    show  $\forall k \in \text{cscope-of } (). \forall ch \ c.$ 
       $\text{cresponsible-chunk } () \ k = \text{Some } ch$ 
       $\wedge \text{src-lt } (\text{cchunk-read-coordinate } () \ ch) \ c$ 
       $\wedge \text{src-le } c (\text{cfrontier-of } ())$ 
       $\longrightarrow$ 
       $\text{mset } (\text{source-key-coord-slice } k \ c \ (\text{ccdc-events-of } ()))$ 
       $= \text{mset } (\text{source-key-coord-slice } k \ c \ (\text{csrc-history-of } ()))$ 
    proof (intro ballI allI impI)
      fix  $k :: \text{nat}$  and  $ch :: \text{cchunk}$  and  $c :: \text{src-coord}$ 
      have eq:  $\text{ccdc-events-of } () = \text{csrc-history-of } ()$ 
        using exm-cdc exm-src-history by simp
      show  $\text{mset } (\text{source-key-coord-slice } k \ c \ (\text{ccdc-events-of } ()))$ 
         $= \text{mset } (\text{source-key-coord-slice } k \ c \ (\text{csrc-history-of } ()))$ 
        using eq by simp
    qed
  next

```

```

show  $\forall ch \in cchunks \ ()$ .
   $src-le \ (cchunk-lower-watermark \ () \ ch)$ 
     $(cchunk-read-coordinate \ () \ ch)$ 
 $\wedge \ src-le \ (cchunk-read-coordinate \ () \ ch)$ 
     $(cchunk-upper-watermark \ () \ ch)$ 
 $\wedge \ (\forall k \in cchunk-domain \ () \ ch.$ 
   $\exists m. \ cchunk-read-result \ () \ ch \ k = Some \ m)$ 
 $\wedge \ (\forall k \in cchunk-domain \ () \ ch. \forall m.$ 
   $cchunk-read-result \ () \ ch \ k = Some \ m$ 
     $\longleftrightarrow Refresh \ k \ m \ (cchunk-read-coordinate \ () \ ch)$ 
     $\in set \ (ex-model.clean-prefix-of \ ()))$ 

proof
  fix  $ch :: cchunk$ 
  assume  $ch-in: ch \in cchunks \ ()$ 
  hence  $ch-cases: ch = ChA \vee ch = ChB$ 
  using  $exm-chunks-eq$  by  $simp$ 
  show  $src-le \ (cchunk-lower-watermark \ () \ ch)$ 
     $(cchunk-read-coordinate \ () \ ch)$ 
 $\wedge \ src-le \ (cchunk-read-coordinate \ () \ ch)$ 
     $(cchunk-upper-watermark \ () \ ch)$ 
 $\wedge \ (\forall k \in cchunk-domain \ () \ ch.$ 
   $\exists m. \ cchunk-read-result \ () \ ch \ k = Some \ m)$ 
 $\wedge \ (\forall k \in cchunk-domain \ () \ ch. \forall m.$ 
   $cchunk-read-result \ () \ ch \ k = Some \ m$ 
     $\longleftrightarrow Refresh \ k \ m \ (cchunk-read-coordinate \ () \ ch)$ 
     $\in set \ (ex-model.clean-prefix-of \ ()))$ 

proof  $(cases \ ch = ChA)$ 
  case  $True$ 
  show  $?thesis$ 
    using  $True \ exm-lwm-A \ exm-uwm-A \ exm-rcoord-A$ 
     $src-le-refl$ 
     $exm-dom-A$ 
     $exm-crr-A-100$ 
     $exm-crr-A-200$ 
     $exm-clean-prefix$ 
    by  $auto$ 
  next
  case  $False$ 
  hence  $ch-B: ch = ChB$  using  $ch-cases$  by  $simp$ 
  show  $?thesis$ 
    using  $ch-B \ exm-lwm-B \ exm-uwm-B \ exm-rcoord-B$ 
     $src-le-refl$ 
     $exm-dom-B$ 
     $exm-crr-B-300$ 
     $exm-clean-prefix$ 
     $ec1-neq-ec3 \ ec1-neq-ec2 \ ec2-neq-ec3$ 
    by  $auto$ 
  qed
qed

```

```

next
show  $\forall ch \in cchunks ()$ .
    src-le (cchunk-read-coordinate () ch) (cfrontier-of ())
using exm-chunks-eq exm-rcoord-A exm-rcoord-B
    exm-frontier ec1-le-ec4 ec3-le-ec4
by auto
next
show  $\forall ch \in cchunks ()$ .  $\forall k \in cchunk-domain ()$  ch.  $\forall m$ .
    cchunk-read-result () ch k = Some m
     $\rightarrow m = Src\ b0\text{-}ex\ H\text{-}ex\ (cchunk-read-coordinate ()\ ch)\ k$ 
proof
fix ch :: cchunk
assume ch-in: ch  $\in$  cchunks ()
hence ch-cases: ch = ChA  $\vee$  ch = ChB
using exm-chunks-eq by simp
show  $\forall k \in cchunk-domain ()$  ch.  $\forall m$ .
    cchunk-read-result () ch k = Some m
     $\rightarrow m = Src\ b0\text{-}ex\ H\text{-}ex\ (cchunk-read-coordinate ()\ ch)\ k$ 
proof (cases ch = ChA)
case True
show ?thesis
using True exm-dom-A
    exm-crr-A-100
    exm-crr-A-200
    exm-rcoord-A
    Src-b0-ex-H-ex-ec1-100
    Src-b0-ex-H-ex-ec1-200
by auto
next
case False
hence ch-B: ch = ChB using ch-cases by simp
show ?thesis
using ch-B exm-dom-B
    exm-crr-B-300
    exm-rcoord-B
    Src-b0-ex-H-ex-ec3-300
by auto
qed
qed
next
show  $\forall ch \in cchunks ()$ .  $\forall k \in cchunk-domain ()$  ch.
    cchunk-read-result () ch k = Some None
     $\rightarrow ex\text{-}model.\text{row-absence-meaningful}\ b0\text{-}ex\ H\text{-}ex\ ()\ ch\ k$ 
proof (intro ballI impI)
fix ch :: cchunk and k :: nat
assume ch-in: ch  $\in$  cchunks ()
and k-in: k  $\in$  cchunk-domain () ch
and crr: cchunk-read-result () ch k = Some None
from ch-in exm-chunks-eq

```

```

    have ch-cases:  $ch = ChA \vee ch = ChB$  by simp
  have False
proof (cases  $ch = ChA$ )
  case True
    with  $k$ -in exm-dom-A have  $k = 100 \vee k = 200$  by simp
    with True crr exm-crr-A-100
      exm-crr-A-200 show False by auto
  next
    case False
    with ch-cases have ch-B:  $ch = ChB$  by simp
    with  $k$ -in exm-dom-B have  $k = 300$  by simp
    with ch-B crr exm-crr-B-300 show False by simp
qed
thus ex-model.row-absence-meaningful b0-ex H-ex () ch k ..
qed
next
show  $\forall p \in \text{set } (ccdc\text{-events-of } ())$ .
   $\text{key-of } (hist\text{-event } p) \in \text{cscope-of } ()$ 
   $\wedge \text{src-le } (hist\text{-coord } p) (\text{cfrontier-of } ())$ 
   $\longrightarrow Cdc (hist\text{-coord } p) (hist\text{-event } p)$ 
     $\in \text{set } (ex\text{-model.clean-prefix-of } ())$ 
proof (intro ballI impI)
  fix  $p :: \text{src-coord} \times (\text{nat}, \text{nat}) \text{ source-event}$ 
  assume  $p$ -in:  $p \in \text{set } (ccdc\text{-events-of } ())$ 
  hence  $p$ -cases:  $p = (ec1, \text{Update } 100 \ 3000)$ 
     $\vee p = (ec2, \text{Insert } 300 \ 2500)$ 
     $\vee p = (ec3, \text{Update } 200 \ 12000)$ 
     $\vee p = (ec4, \text{Update } 300 \ 3500)$ 
  using exm-cdc set-H-ex by simp
  show  $Cdc (hist\text{-coord } p) (hist\text{-event } p)$ 
     $\in \text{set } (ex\text{-model.clean-prefix-of } ())$ 
  using  $p$ -cases exm-clean-prefix by fastforce
qed
next
show  $\forall e \ c. Cdc \ c \ e \in \text{set } (ex\text{-model.clean-prefix-of } ())$ 
   $\wedge \text{key-of } e \in \text{cscope-of } ()$ 
   $\wedge \text{src-le } c (\text{cfrontier-of } ())$ 
   $\longrightarrow (c, e) \in \text{set } (ccdc\text{-events-of } ())$ 
proof (intro allI impI)
  fix  $e :: (\text{nat}, \text{nat}) \text{ source-event}$  and  $c :: \text{src-coord}$ 
  assume  $Cdc \ c \ e \in \text{set } (ex\text{-model.clean-prefix-of } ())$ 
   $\wedge \text{key-of } e \in \text{cscope-of } ()$ 
   $\wedge \text{src-le } c (\text{cfrontier-of } ())$ 
  hence cdc-in:  $Cdc \ c \ e \in \text{set } (ex\text{-model.clean-prefix-of } ())$  by simp
  have list-eq:  $Cdc \ c \ e \in \text{set } [Cdc \ ec1 (\text{Update } 100 \ 3000),$ 
     $\text{Refresh } (100::\text{nat}) (\text{Some } (3000::\text{nat})) \ ec1,$ 
     $\text{Refresh } 200 (\text{Some } 10000) \ ec1,$ 
     $Cdc \ ec2 (\text{Insert } 300 \ 2500),$ 
     $Cdc \ ec3 (\text{Update } 200 \ 12000),$ 

```



```

next
  case False
  hence k300: k = 300 using  $\langle k = 200 \vee k = 300 \rangle$  by simp
  then show ?thesis
    unfolding restrict-def
    using  $\langle k \in \text{cscope-of } () \rangle$ 
      exm-apply
      Src-b0-ex-H-ex-ec4-300
      exm-frontier
    by simp
qed
qed
next
  case False
  show ?thesis
    unfolding restrict-def using False by simp
qed
qed

```

Composite constructed result: the running example, realized by a concrete interpretation of *dblog-run-substrate*, is wellformed and its assembled replay equals the source state at the frontier on the chunk-covered scope — the Layer 2 source-side soundness conclusion (the paper’s bridge lemma) on the running-example instance, backed by construction and resting on no axiom.

```

theorem ex-model-wellformed-and-virtual-cut:
  ex-model.wellformed-dblog-run b0-ex () H-ex
   $\wedge$  restrict (Apply (ex-model.clean-prefix-of ())) (cscope-of ())
    = restrict (Src b0-ex H-ex (cfrontier-of ())) (cscope-of ())
  using ex-model-wellformed ex-model-virtual-cut-holds by blast

end

```

```

theory Layer4-Whole-Table
  imports Virtual-Cut Virtual-Cut-Core
begin

```

25 Layer 4: the whole-table specialization

Layer 4 whole-table anchor-domain specialization.

The reader-facing paper keeps the compact notation *whole-table-claim-scope*(*C*,*H*,*b*). The formal surface threads the deployment base state *b0* explicitly, because *Src* and faithful source observation are both fixed-base-state objects.

The pure anchor / table-scope definitions and outside-scope bridge lemmas live in *Virtual-Cut-Core*: *anchor-domain*, *touched-between*, *table-scope-between*, *anchor-complete-at-boundary*,

all-post-anchor-changes-covered, *replay-keys*, *no-unclaimed-replay-keys*, and the bridge lemmas *source-outside-table-scope-absent*, *apply-outside-replay-keys-none*, *unrestricted-equality-from-scoped-equality*, *whole-table-equality-from-scoped-equality*. They reference no certificate accessor and are used here via the *Virtual-Cut-Core* import; the *DBLog-Cert-Substrate* locale reaches them the same way, without importing this theory. The *scoped-equality* bridge is stated over the raw restricted equality; the *cert-side* caller below unfolds *virtual-cut-state* before applying it.

25.1 Whole-table claim scope (cert-side)

definition *whole-table-claim-scope*

$$\begin{aligned} &:: ('k \rightarrow 'v) \Rightarrow ('k, 'v) \text{ cert} \Rightarrow ('k, 'v) \text{ src-history} \\ &\quad \Rightarrow \text{frontier} \Rightarrow \text{bool} \end{aligned}$$

where

$$\begin{aligned} &\text{whole-table-claim-scope } b0 \ C \ H \ b \longleftrightarrow \\ &\quad b \leq \text{frontier } C \\ &\quad \wedge \text{anchor-complete-at-boundary } b0 \ H \ b \ (\text{scope } C) \\ &\quad \wedge \text{all-post-anchor-changes-covered } H \ b \ (\text{frontier } C) \ (\text{scope } C) \\ &\quad \wedge \text{no-unclaimed-replay-keys } (\text{clean-prefix } C) \ (\text{scope } C) \end{aligned}$$

lemma *whole-table-claim-scope-imp-table-scope-contained*:

assumes *whole-table-claim-scope* $b0 \ C \ H \ b$

shows *table-scope-between* $b0 \ H \ b \ (\text{frontier } C) \subseteq \text{scope } C$

using *assms*

unfolding *whole-table-claim-scope-def*

table-scope-between-def

anchor-complete-at-boundary-def

all-post-anchor-changes-covered-def

by *blast*

25.2 Outside-scope bridge to whole-table equality (cert-side)

lemma *whole-table-equality-from-claim-scope*:

assumes *vc*: *virtual-cut* $b0 \ C \ H$

and *whole*: *whole-table-claim-scope* $b0 \ C \ H \ b$

shows *Apply* $(\text{clean-prefix } C) = \text{Src } b0 \ H \ (\text{frontier } C)$

proof –

have *scoped*:

restrict $(\text{Apply } (\text{clean-prefix } C)) \ (\text{scope } C)$

$= \text{restrict } (\text{Src } b0 \ H \ (\text{frontier } C)) \ (\text{scope } C)$

using *vc* **unfolding** *virtual-cut-def* *virtual-cut-state-def* **by** *simp*

have *contained*:

table-scope-between $b0 \ H \ b \ (\text{frontier } C) \subseteq \text{scope } C$

by $(\text{rule } \text{whole-table-claim-scope-imp-table-scope-contained}[OF \ \text{whole}])$

have *no-unclaimed*:

no-unclaimed-replay-keys $(\text{clean-prefix } C) \ (\text{scope } C)$

using *whole* **unfolding** *whole-table-claim-scope-def* **by** *blast*


```

have b-le-frontier:  $b \leq \text{frontier } C$ 
  using whole unfolding whole-table-claim-scope-def by blast
show ?thesis
  by (rule whole-table-equality-from-scoped-equality
      [OF scoped contained no-unclaimed b-le-frontier])
qed

```

25.3 Layer 4 theorem surface over accepted certificates

```

context layer3-checker-substrate
begin

```

```

theorem accepted-whole-table-anchor-domain-specialization:

```

```

  fixes b0 :: 'k  $\rightarrow$  'v
    and H :: ('k, 'v) src-history
    and Raw :: raw-observation
    and b :: frontier
  assumes accept: verify C E = Accept
    and faithful: faithful-source-observation E b0 H Raw
    and whole: whole-table-claim-scope b0 C H b
  shows Apply (clean-prefix C) = Src b0 H (frontier C)
proof -
  have vc: virtual-cut b0 C H
  by (rule accepted-virtual-cut-sound[OF accept faithful])
  show ?thesis
  by (rule whole-table-equality-from-claim-scope[OF vc whole])
qed

```

```

end

```

```

end

```

```

theory Continuation
  imports Virtual-Cut
begin

```

26 Source-side continuation and restriction of virtual cuts

Source-side continuation and sub-scope restriction of virtual cuts (the post-core continuation extension).

A certified virtual cut is a point-in-time object — it certifies replay equality at one source frontier on one key scope. This theory proves two primary source-side algebra facts that place a single cut inside DBLog’s continuous operation:

- *continuation across frontiers*: a virtual cut at frontier f extends to a

virtual cut at a later frontier f' by appending the faithful in-scope CDC segment for the half-open interval $(f, f']$;

- *restriction to a sub-scope*: a virtual cut on key scope K restricts to a virtual cut on any $K' \subseteq K$.

Both facts are stated at the *virtual-cut-state* level — the factored source-side extensional-equality core to which the Layer 2 run-side theorem and the Layer 3 cert-side *virtual-cut* predicate both reduce — so continuation composes with the existing ladder without re-opening the abstract run model. An accessor-level corollary lifts continuation onto an accepted certificate's clean prefix; a whole-table instance specializes continuation to $K = UNIV$.

Every theorem conclusion is an *Apply-against-Src* equality: continuation is a source-side replay-equality statement, not a destination-state, delivery, or sink claim. Nothing here asserts that an extended certificate is accepted by the verifier, and nothing here asserts downstream repair.

In the companion fixture theories (*Continuation-Fixtures-Core* and *Continuation-Fixtures-Inst*) the *l7-* / *l7f-* / *fix-l7-* prefixes tag the continuation fixture family (historically the seventh fixture layer); the prefix does not denote a model layer.

Paper "Source-Side Continuation and Restriction of Virtual Cuts".

26.1 The continuation segment

cdc-segment-between $H K f f' \delta$ holds exactly when δ is the continuation segment for the half-open interval $(f, f']$ on key scope K : the source history H filtered to the events whose source coordinate c satisfies $f < c \wedge c \leq f'$ and whose key lies in K , with each surviving pair (c, e) lifted to its replay-side CDC event *cdc-lift* $c e$.

Defining the segment as *filter the source history, then map to a CDC event* — rather than as a loose "the CDC events between f and f' " — makes source order, same-coordinate source-position order, and event multiplicity structural consequences of the definition. A caller cannot satisfy the predicate with a reordered, thinned, or padded segment.

The interval is half-open $(f, f']$ because frontiers are inclusive: *Src* at f already reflects every event at coordinate f , so continuation begins strictly after f ; the target frontier f' includes its own coordinate.

Paper "Source-Side Continuation and Restriction of Virtual Cuts" / "Definition (continuation segment)".

definition *cdc-segment-between*

$:: ('k, 'v) \text{ src-history} \Rightarrow 'k \text{ set} \Rightarrow \text{frontier} \Rightarrow \text{frontier}$
 $\Rightarrow ('k, 'v) \text{ replay-event list} \Rightarrow \text{bool}$

where

cdc-segment-between $H K f f' \delta \longleftrightarrow$
 $f \leq f'$

$$\wedge \delta = \text{map } (\lambda(c, e). \text{cdc-lift } c \ e) \\ (\text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e \in K) \ H)$$

A wellformed source history is sorted by source coordinate. WF-H1 fixes non-decreasing coordinates between adjacent indices; by transitivity this is coordinate-monotonicity across every index pair — exactly *sorted* on the *hist-coord* projection. Naming this consequence lets the segment-concatenation lemma reason by list position.

lemma *wellformed-src-history-sorted*:
assumes *wfH*: *wellformed-src-history H*
shows *sorted* (*map hist-coord H*)
unfolding *sorted-iff-nth-mono*
proof (*intro allI impI*)
fix *i j*
assume *ij*: *i* ≤ *j* **and** *j-lt*: *j* < *length* (*map hist-coord H*)
have *j-lt-H*: *j* < *length H* **using** *j-lt* **by** *simp*
have *i-lt*: *i* < *length H* **using** *ij j-lt-H* **by** (*rule le-less-trans*)
have *src-le* (*hist-coord* (*H* ! *i*)) (*hist-coord* (*H* ! *j*))
by (*rule wellformed-src-history-coord-mono[OF wfH ij j-lt-H]*)
thus *map hist-coord H* ! *i* ≤ *map hist-coord H* ! *j*
by (*simp add: src-le-eq-less-eq nth-map i-lt j-lt-H*)
qed

Interval-filter concatenation. On a coordinate-sorted source history, the events filtered to the half-open interval $(f, f']$, followed by those filtered to $(f', f'']$, are — in list order — exactly the events filtered to the combined interval $(f, f'']$: sortedness places every lower-interval event before every upper-interval event, so the two sub-histories concatenate rather than interleave. The key scope *K* is carried unchanged. This is the source-order fact underlying *cdc-segment-between*’s chaining lemma.

lemma *filter-src-interval-concat*:
fixes *H* :: ('k, 'v) *src-history*
assumes *ff'*: *f* ≤ *f'*
and *f'f''*: *f'* ≤ *f''*
and *srt*: *sorted* (*map hist-coord H*)
shows *filter* ($\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e \in K$) *H*
 $\quad @ \text{filter } (\lambda(c, e). f' < c \wedge c \leq f'' \wedge \text{key-of } e \in K) \ H$
 $\quad = \text{filter } (\lambda(c, e). f < c \wedge c \leq f'' \wedge \text{key-of } e \in K) \ H$
using *srt*
proof (*induction H*)
case *Nil*
show ?*case* **by** *simp*
next
case (*Cons p H'*)
obtain *pc pe* **where** *p*: *p* = (*pc*, *pe*) **by** (*cases p*)
have *srt'*: *sorted* (*map hist-coord H'*)
using *Cons.premis* **by** *simp*
have *IH*: *filter* ($\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e \in K$) *H'*

```

      @ filter (λ(c, e). f' < c ∧ c ≤ f'' ∧ key-of e ∈ K) H'
    = filter (λ(c, e). f < c ∧ c ≤ f'' ∧ key-of e ∈ K) H'
  by (rule Cons.IH[OF srt'])
show ?case
proof (cases pc ≤ f')
  case lo: True
  — Head in the lower interval: it joins the lower and combined segments identically; the tail closes by induction.
  have pcf'': pc ≤ f'' using lo f'f'' by (rule order-trans)
  have not-f': ¬ f' < pc using lo by simp
  show ?thesis using p lo pcf'' not-f' IH by simp
next
  case hi: False
  — Head strictly above f': sortedness puts the whole tail above f', so the lower segment of the tail is empty.
  hence f'-lt: f' < pc by simp
  have f-lt: f < pc using ff' f'-lt by (rule le-less-trans)
  have tail-above: f' < hist-coord q if q-in: q ∈ set H' for q
  proof —
    have sorted (pc # map hist-coord H')
      using Cons.prem p by simp
    moreover have hist-coord q ∈ set (map hist-coord H')
      using q-in by auto
    ultimately have pc-le-q: pc ≤ hist-coord q by auto
    show f' < hist-coord q using f'-lt pc-le-q by (rule less-le-trans)
  qed
  have lo-empty:
    filter (λ(c, e). f < c ∧ c ≤ f' ∧ key-of e ∈ K) H' = []
  proof —
    have ¬ (case q of (c, e) ⇒ f < c ∧ c ≤ f' ∧ key-of e ∈ K)
      if q-in: q ∈ set H' for q
      using tail-above[OF q-in] by (auto simp: case-prod-beta)
    thus ?thesis by (simp add: filter-empty-conv)
  qed
  show ?thesis
    using p hi f'-lt f-lt IH lo-empty by simp
qed
qed

```

Continuation segments chain: the segment for $(f, f']$ followed by the segment for $(f', f'']$ is the segment for $(f, f'']$. This supports the reader-facing statement that a finite prefix of ongoing CDC is a fold of continuation segments.

Wellformedness of H is required. On a non-decreasing history (WF-H1) the $(f, f']$ events precede the $(f', f'']$ events in list order, so the two filtered sub-histories concatenate, in source order, into the $(f, f'']$ sub-history. Without WF-H1 a history could interleave the two coordinate ranges, and the concatenation would not be in source order — so the wellformedness premise is not a convenience but a soundness requirement.

lemma *cdc-segment-between-concat*:
assumes *wfH*: *wellformed-src-history H*
and *seg1*: *cdc-segment-between H K f f' δ1*
and *seg2*: *cdc-segment-between H K f' f'' δ2*
shows *cdc-segment-between H K f f'' (δ1 @ δ2)*
proof –
from *seg1* **have** *ff'*: $f \leq f'$
and $\delta 1\text{-eq}$: $\delta 1 = \text{map } (\lambda(c, e). \text{cdc-lift } c \ e)$
 $(\text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e \in K) \ H)$
unfolding *cdc-segment-between-def* **by** *simp-all*
from *seg2* **have** *f'f''*: $f' \leq f''$
and $\delta 2\text{-eq}$: $\delta 2 = \text{map } (\lambda(c, e). \text{cdc-lift } c \ e)$
 $(\text{filter } (\lambda(c, e). f' < c \wedge c \leq f'' \wedge \text{key-of } e \in K) \ H)$
unfolding *cdc-segment-between-def* **by** *simp-all*
have *ff''*: $f \leq f''$ **using** *ff' f'f''* **by** (*rule order-trans*)
have $\delta 1 @ \delta 2$
 $= \text{map } (\lambda(c, e). \text{cdc-lift } c \ e)$
 $(\text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e \in K) \ H$
 $@ \text{filter } (\lambda(c, e). f' < c \wedge c \leq f'' \wedge \text{key-of } e \in K) \ H)$
unfolding $\delta 1\text{-eq}$ $\delta 2\text{-eq}$ **by** *simp*
also have $\dots = \text{map } (\lambda(c, e). \text{cdc-lift } c \ e)$
 $(\text{filter } (\lambda(c, e). f < c \wedge c \leq f'' \wedge \text{key-of } e \in K) \ H)$
by (*simp only*:
filter-src-interval-concat
 $[OF \ ff' \ f'f'' \ \text{wellformed-src-history-sorted}[OF \ wfH]])$
finally show *?thesis*
unfolding *cdc-segment-between-def* **using** *ff''* **by** *simp*
qed

Continuation-segment / per-key bridge. On scope K and a key $k \in K$, the k -events of the continuation segment δ — the events *event-key* selects for k — are exactly the k -events of the source history H in the half-open interval $(f, f']$, each lifted to its replay-side CDC event by *cdc-lift*.

The lemma connects the *cdc-segment-between* predicate to the per-key *Apply-against-Src* reasoning the continuation theorem performs: it lets that proof identify the k -events of δ with the interval segment *Src-interval-decomposition* characterizes.

lemma *cdc-segment-between-event-key-filter*:
assumes *seg*: *cdc-segment-between H K f f' δ*
and *kK*: $k \in K$
shows $\text{filter } (\lambda e. \text{event-key } e = k) \ \delta$
 $= \text{map } (\lambda(c, e). \text{cdc-lift } c \ e)$
 $(\text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e = k) \ H)$

proof –
from *seg* **have** $\delta\text{-eq}$:
 $\delta = \text{map } (\lambda(c, e). \text{cdc-lift } c \ e)$
 $(\text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e \in K) \ H)$
unfolding *cdc-segment-between-def* **by** *simp*
have *key-filter*:

```

    filter ((λe. event-key e = k) ∘ (λ(c, e). cdc-lift c e))
      (filter (λ(c, e). f < c ∧ c ≤ f' ∧ key-of e ∈ K) H)
    = filter (λ(c, e). f < c ∧ c ≤ f' ∧ key-of e = k) H
  unfolding filter-filter
  by (rule filter-cong[OF refl])
    (use kK in ⟨auto simp: cdc-lift-def case-prod-beta⟩)
  show ?thesis
  by (simp only: δ-eq filter-map key-filter)
qed

```

26.2 Layer 0 — replay-append and source-state interval lemmas

Replaying a concatenation $\sigma @ \delta$ is replaying δ from the state *Apply* σ . Immediate from *Apply*'s left-fold definition. Recorded as the obvious replay-append API fact; the continuation proof itself works per key (through the per-key filter lemmas) and does not consume it.

lemma *Apply-append*:
 $\text{Apply } (\sigma @ \delta) = \text{foldl } \text{apply-step } (\text{Apply } \sigma) \delta$
unfolding *Apply-def* **by** (*simp add: foldl-append*)

The load-bearing Layer 0 lemma for continuation. It characterizes the source state at the later frontier f' from the source state at the earlier frontier f and the key's own source events inside the interval $(f, f']$.

For a fixed key k , let segk be the k -events of H in $(f, f']$, in source order. If segk is empty — no source event for k in the interval — then *Src* at f' agrees with *Src* at f on k . Otherwise the latest interval event wins: *Src* at f' on k is the per-event effect of the last entry of segk .

The proof case-splits on whether such an interval event exists, using *src-eq-when-no-later-src-event-for-k*, direct unfolding of *Src*, and the *src-coord* linear order. The empty case needs only $f \leq f'$ and linear-order totality. The non-empty case uses WF-H1 to place the interval events after the $c \leq f$ events in list order, so the latest $c \leq f'$ event for k is exactly the latest interval event.

lemma *Src-interval-decomposition*:
fixes $b0 :: 'k \rightarrow 'v$
and $H :: ('k, 'v) \text{ src-history}$
assumes *wfH*: *wellformed-src-history* H
and *f-le*: $f \leq f'$
shows
 $\text{Src } b0 \ H \ f' \ k$
 $= (\text{let } \text{segk} = \text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e = k) \ H$
 $\text{in if } \text{segk} = []$
 $\text{then } \text{Src } b0 \ H \ f \ k$
 $\text{else } (\text{case } \text{snd } (\text{last } \text{segk}) \text{ of}$
 $\text{Insert } - \ v \Rightarrow \text{Some } v$
 $\mid \text{Update } - \ v \Rightarrow \text{Some } v$

| *Delete* - $\Rightarrow$ *None*))

proof –

define Q **where**

$Q = (\lambda p :: \text{src-coord} \times ('k, 'v) \text{ source-event}.$
 $\quad f < \text{hist-coord } p \wedge \text{hist-coord } p \leq f'$
 $\quad \wedge \text{key-of } (\text{hist-event } p) = k)$

have *segk-eq*:

$\text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e = k) \ H = \text{filter } Q \ H$
by (*rule filter-cong[OF refl]*) (*simp add: Q-def case-prod-beta*)

show *?thesis*

proof (*cases filter Q H = []*)

case *True*

— No source event for k in $(f, f']$: Src is unchanged.

have *no-later*:

$\forall i < \text{length } H. \text{src-lt } f \ (\text{hist-coord } (H ! i))$
 $\quad \wedge \text{src-le } (\text{hist-coord } (H ! i)) \ f'$
 $\quad \longrightarrow \text{key-of } (\text{hist-event } (H ! i)) \neq k$

proof (*intro allI impI*)

fix i

assume $i\text{-lt}$: $i < \text{length } H$

and between : $\text{src-lt } f \ (\text{hist-coord } (H ! i))$
 $\quad \wedge \text{src-le } (\text{hist-coord } (H ! i)) \ f'$

have $\neg Q \ (H ! i)$

using *True i-lt* **by** (*metis filter-empty-conv nth-mem*)

thus $\text{key-of } (\text{hist-event } (H ! i)) \neq k$

using *between*

by (*auto simp: Q-def src-lt-eq-less src-le-eq-less-eq*)

qed

have *src-eq*: $\text{Src } b0 \ H \ f' \ k = \text{Src } b0 \ H \ f \ k$

proof (*rule src-eq-when-no-later-src-event-for-k[OF - no-later]*)

show $\text{src-le } f \ f'$

using $f\text{-le}$ **by** (*simp add: src-le-eq-less-eq*)

qed

show *?thesis*

using *src-eq True segk-eq* **by** (*simp add: Let-def*)

next

case *False*

— The latest k -event in $(f, f']$ determines Src at f' .

obtain i **where**

$i\text{-lt}$: $i < \text{length } H$ **and**

$Q\text{-}i$: $Q \ (H ! i)$ **and**

$H\text{-}i\text{-last}$: $H ! i = \text{last } (\text{filter } Q \ H)$ **and**

$\text{no-after-}Q$: $\forall j. i < j \wedge j < \text{length } H \longrightarrow \neg Q \ (H ! j)$

using *last-filter-index[OF False]* **by** *blast*

let $?P = \lambda j. \text{src-le } (\text{hist-coord } (H ! j)) \ f'$
 $\quad \wedge \text{key-of } (\text{hist-event } (H ! j)) = k$

have $P\text{-}i$: $?P \ i$

using $Q\text{-}i$ **by** (*simp add: Q-def src-le-eq-less-eq*)

have $f\text{-lt-}i$: $f < \text{hist-coord } (H ! i)$

```

    using Q-i by (simp add: Q-def)
  have no-after-P:  $\forall j. i < j \wedge j < \text{length } H \longrightarrow \neg ?P j$ 
  proof (intro allI impI)
    fix j
    assume j-assm:  $i < j \wedge j < \text{length } H$ 
    hence i-le-j:  $i \leq j$  and j-lt:  $j < \text{length } H$  by auto
    show  $\neg ?P j$ 
  proof
    assume P-j:  $?P j$ 
    hence j-le-f':  $\text{src-le } (\text{hist-coord } (H ! j)) f'$ 
      and j-key:  $\text{key-of } (\text{hist-event } (H ! j)) = k$ 
      by auto
    have coord-mono:
       $\text{src-le } (\text{hist-coord } (H ! i)) (\text{hist-coord } (H ! j))$ 
      by (rule wellformed-src-history-coord-mono[OF wfH i-le-j j-lt])
    have  $f < \text{hist-coord } (H ! j)$ 
    proof -
      have  $\text{hist-coord } (H ! i) \leq \text{hist-coord } (H ! j)$ 
      using coord-mono by (simp add: src-le-eq-less-eq)
      with f-lt-i show ?thesis by (rule less-le-trans)
    qed
    hence Q (H ! j)
      using j-le-f' j-key
      by (simp add: Q-def src-le-eq-less-eq)
    with no-after-Q j-assm show False by blast
  qed
qed
have cand-ne:  $\text{filter } ?P [0..<\text{length } H] \neq []$ 
  and cand-last:  $\text{last } (\text{filter } ?P [0..<\text{length } H]) = i$ 
  using last-filter-upt-eq[OF i-lt P-i no-after-P] by auto
have latest-eq:  $\text{latest-src-event } H f' k = \text{Some } i$ 
  using cand-ne cand-last
  unfolding latest-src-event-def by (simp add: Let-def)
have Src b0 H f' k = (case  $\text{hist-event } (H ! i)$  of
  | Insert - v  $\Rightarrow$  Some v
  | Update - v  $\Rightarrow$  Some v
  | Delete -  $\Rightarrow$  None)
  unfolding Src-def using latest-eq by simp
thus ?thesis
  using segk-eq False H-i-last by (simp add: Let-def)
qed
qed

```

26.3 Continuation across frontiers

The primary theorem. A virtual cut at frontier f on scope K — a sequence σ whose replay agrees with Src at f on K — extends to a virtual cut at any later frontier f' by appending the continuation segment δ for $(f, f']$ on K .

The proof is pointwise on $k \in K$. By *apply-latest-event-wins* the replay

value of $\sigma @ \delta$ at k is governed by the last k -event of $\sigma @ \delta$. If δ carries a k -event, that last event lies in δ — a CDC event lifted from the latest interval source event for k — and *Src-interval-decomposition* equates its effect with *Src* at f' on k . If δ carries no k -event, replay of $\sigma @ \delta$ at k equals replay of σ , which the premise equates with *Src* at f on k , and *Src-interval-decomposition* equates that with *Src* at f' on k .

The result is *Conditional* at the paper level: on a wellformed source history and on δ being exactly the faithful continuation segment for the interval.

Paper "Source-Side Continuation and Restriction of Virtual Cuts" / "Theorem (continuation across frontiers)".

theorem *virtual-cut-state-continuation*:

fixes $b0 :: 'k \rightarrow 'v$

and $H :: ('k, 'v) \text{ src-history}$

assumes wfH : *wellformed-src-history* H

and cut : *virtual-cut-state* $b0 \ \sigma \ K \ f \ H$

and seg : *cdc-segment-between* $H \ K \ f \ f' \ \delta$

shows *virtual-cut-state* $b0 \ (\sigma @ \delta) \ K \ f' \ H$

proof —

from seg **have** $f\text{-le}$: $f \leq f'$

unfolding *cdc-segment-between-def* **by** *simp*

have cut : *Apply* $\sigma \ k = \text{Src } b0 \ H \ f \ k$ **if** $k \in K$ **for** k

proof —

have *restrict* (*Apply* σ) $K \ k = \text{restrict } (\text{Src } b0 \ H \ f) \ K \ k$

using cut **unfolding** *virtual-cut-state-def* **by** *simp*

thus *?thesis using that by (simp add: restrict-def)*

qed

have $key\text{-eq}$: *Apply* $(\sigma @ \delta) \ k = \text{Src } b0 \ H \ f' \ k$ **if** $k \in K$ **for** k

proof —

define $segk$ **where**

$segk = \text{filter } (\lambda(c, e). f < c \wedge c \leq f' \wedge \text{key-of } e = k) \ H$

have $\delta\text{-k}$: $\text{filter } (\lambda e. \text{event-key } e = k) \ \delta$

$= \text{map } (\lambda(c, e). \text{cdc-lift } c \ e) \ segk$

unfolding $segk\text{-def}$

by (*rule cdc-segment-between-event-key-filter[OF seg kK]*)

have $src\text{-}f'$:

$\text{Src } b0 \ H \ f' \ k$

$= (\text{if } segk = []$

$\text{then } \text{Src } b0 \ H \ f \ k$

$\text{else } (\text{case } \text{snd } (\text{last } segk) \text{ of}$

$\text{Insert } - \ v \Rightarrow \text{Some } v$

$| \text{Update } - \ v \Rightarrow \text{Some } v$

$| \text{Delete } - \ \Rightarrow \text{None}))$

unfolding $segk\text{-def}$

using *Src-interval-decomposition*[*OF wfH f-le*] **by** (*simp add: Let-def*)

have $filt\text{-app}$: $\text{filter } (\lambda e. \text{event-key } e = k) \ (\sigma @ \delta)$

$= \text{filter } (\lambda e. \text{event-key } e = k) \ \sigma$

$@ \text{map } (\lambda(c, e). \text{cdc-lift } c \ e) \ segk$

```

    using  $\delta$ - $k$  by simp
  show ?thesis
  proof (cases segk = [])
    case True
      — No source event for  $k$  in  $(f, f']$ : replay of  $\sigma @ \delta$  at  $k$  reduces to replay of  $\sigma$ ,
      and Src is unchanged.
      have Apply ( $\sigma @ \delta$ )  $k$ 
        = Apply (filter ( $\lambda e. \text{event-key } e = k$ ) ( $\sigma @ \delta$ ))  $k$ 
        by (rule apply-eq-apply-filter-key)
      also have ... = Apply (filter ( $\lambda e. \text{event-key } e = k$ )  $\sigma$ )  $k$ 
        using filt-app True by simp
      also have ... = Apply  $\sigma$   $k$ 
        by (rule apply-eq-apply-filter-key[symmetric])
      also have ... = Src b0  $H f k$ 
        by (rule cut-k[OF kK])
      also have ... = Src b0  $H f' k$ 
        using src-f' True by simp
      finally show ?thesis .
    next
      case False
        — The latest  $k$ -event lies in  $\delta$ ; it is the cdc-lift of the latest interval source
        event for  $k$ .
        have map-ne: map ( $\lambda(c, e). \text{cdc-lift } c e$ ) segk  $\neq []$ 
          using False by simp
        have filt-ne: filter ( $\lambda e. \text{event-key } e = k$ ) ( $\sigma @ \delta$ )  $\neq []$ 
          using filt-app map-ne by simp
        have last-eq: last (filter ( $\lambda e. \text{event-key } e = k$ ) ( $\sigma @ \delta$ ))
          = Cdc (fst (last segk)) (snd (last segk))
        proof —
          have last (filter ( $\lambda e. \text{event-key } e = k$ ) ( $\sigma @ \delta$ ))
            = last (map ( $\lambda(c, e). \text{cdc-lift } c e$ ) segk)
            using filt-app map-ne by (simp add: last-appendR)
          also have ... = ( $\lambda(c, e). \text{cdc-lift } c e$ ) (last segk)
            using False by (simp add: last-map)
          also have ... = Cdc (fst (last segk)) (snd (last segk))
            by (simp add: cdc-lift-def case-prod-beta)
          finally show ?thesis .
        qed
        have step: Apply ( $\sigma @ \delta$ )  $k$ 
          = (case filter ( $\lambda e. \text{event-key } e = k$ ) ( $\sigma @ \delta$ ) of
            []  $\Rightarrow$  None
            | - # -  $\Rightarrow$ 
              (case last (filter ( $\lambda e. \text{event-key } e = k$ ) ( $\sigma @ \delta$ )) of
                Cdc - (Insert -  $v$ )  $\Rightarrow$  Some  $v$ 
                | Cdc - (Update -  $v$ )  $\Rightarrow$  Some  $v$ 
                | Cdc - (Delete -)  $\Rightarrow$  None
                | Refresh -  $m$ -obs -  $\Rightarrow$   $m$ -obs))
            by (rule apply-latest-event-wins)
        obtain  $z$   $zs$  where

```

```

      zzs: filter (λe. event-key e = k) (σ @ δ) = z # zs
    by (metis filt-ne neq-Nil-conv)
  have Apply (σ @ δ) k = (case snd (last segk) of
    | Insert - v ⇒ Some v
    | Update - v ⇒ Some v
    | Delete -   ⇒ None)
    unfolding step last-eq using zzs by simp
  also have ... = Src b0 H f' k
    using src-f' False by simp
  finally show ?thesis .
qed
qed
show ?thesis
  unfolding virtual-cut-state-def
proof (rule ext)
  fix k
  show restrict (Apply (σ @ δ)) K k = restrict (Src b0 H f') K k
    using key-eq unfolding restrict-def by (cases k ∈ K) auto
qed
qed

```

26.4 Restriction to a sub-scope

A virtual cut on key scope K restricts to a virtual cut on any $K' \subseteq K$. The proof is pointwise: two per-key states that agree on every key of K agree on every key of any subset.

Restriction does not extend the other way. Widening the scope would assert agreement on keys never certified — for a key outside K the cut says nothing about its state at f — so no scope-widening lemma is stated.

Paper "Source-Side Continuation and Restriction of Virtual Cuts" / "Lemma (sub-scope restriction)".

```

theorem virtual-cut-state-restrict-scope:
  fixes b0 :: 'k → 'v
  assumes cut: virtual-cut-state b0 σ K f H
    and sub: K' ⊆ K
  shows virtual-cut-state b0 σ K' f H
proof -
  have eqK: restrict (Apply σ) K = restrict (Src b0 H f) K
    using cut by (simp add: virtual-cut-state-def)
  show ?thesis
    unfolding virtual-cut-state-def
  proof (rule ext)
    fix k
    from eqK have restrict (Apply σ) K k = restrict (Src b0 H f) K k
      by simp
    with sub show restrict (Apply σ) K' k = restrict (Src b0 H f) K' k
      by (auto simp: restrict-def)
  qed

```

qed

26.5 Whole-table continuation

The $K = UNIV$ instance of continuation. When replay of σ reaches the source state at f on *every* key — an unrestricted equality, as the Layer 4 whole-table specialization yields — appending the full CDC segment for $(f, f']$ on every key reaches the source state at f' .

The continuation segment here ranges over $UNIV$, so it must include every source event in $(f, f']$, not only the events of a finite certificate scope. A scoped continuation does not become whole-table for free: a key inserted outside the scope in $(f, f']$ would change the table without changing the scoped equality.

Paper "Source-Side Continuation and Restriction of Virtual Cuts" / "Corollary (whole-table continuation)".

theorem *whole-table-state-continuation*:

```

fixes  $b0 :: 'k \rightarrow 'v$ 
  and  $H :: ('k, 'v)$  src-history
assumes  $wfH$ : wellformed-src-history  $H$ 
  and  $tab$ :  $Apply\ \sigma = Src\ b0\ H\ f$ 
  and  $seg$ : cdc-segment-between  $H\ UNIV\ f\ f'\ \delta$ 
shows  $Apply\ (\sigma @ \delta) = Src\ b0\ H\ f'$ 
proof –
  have  $univ$ : restrict  $m\ UNIV = m$  for  $m :: 'k \rightarrow 'v$ 
    by (simp add: restrict-def)
  have virtual-cut-state  $b0\ \sigma\ UNIV\ f\ H$ 
    unfolding virtual-cut-state-def using  $tab$  by (simp add: univ)
  from virtual-cut-state-continuation[ $OF\ wfH\ this\ seg$ ]
  have virtual-cut-state  $b0\ (\sigma @ \delta)\ UNIV\ f'\ H$  .
  thus ?thesis
    unfolding virtual-cut-state-def by (simp add: univ)

```

qed

26.6 Sub-scope restriction over a certificate

The certificate-accessor corollary of sub-scope restriction: an accepted certificate's virtual cut, read through its accessors, restricts to any sub-scope of its certified scope. The conclusion is the *virtual-cut-state* predicate on the certificate's clean prefix and frontier at the narrowed scope K' .

It is *not* a claim that a restricted certificate is accepted by the verifier — the verifier acceptance of any concrete restricted certificate is an evidence obligation, not a consequence of the source-side algebra.

Paper "Source-Side Continuation and Restriction of Virtual Cuts" / "Corollary (sub-scope restriction over a certificate)".

theorem *virtual-cut-restrict-to-subscope*:

```

fixes  $b0 :: 'k \rightarrow 'v$ 

```

```

assumes vc: virtual-cut b0 C H
and sub:  $K' \subseteq \text{scope } C$ 
shows virtual-cut-state b0 (clean-prefix C) K' (frontier C) H
proof –
  have virtual-cut-state b0 (clean-prefix C) (scope C) (frontier C) H
    using vc by (simp add: virtual-cut-def)
  thus ?thesis
    using sub by (rule virtual-cut-state-restrict-scope)
qed

```

26.7 Continuation over an accepted certificate

The accessor-level accepted-certificate corollary. Inside the *layer3-checker-substrate* locale, an accepted certificate under faithful source observation yields a virtual cut (*accepted-virtual-cut-sound*); continuation then extends the certified clean prefix by a faithful continuation segment to a virtual cut at the later frontier f' .

The conclusion is the *virtual-cut-state* equality on the *sequence* *clean-prefix* $C @ \delta$. It is not a claim that the verifier accepts an extended certificate: certificate acceptance is an evidence and verifier obligation, not a consequence of the source-side algebra. The continuation segment δ is itself an external observation; its faithfulness to H is a premise, discharged at deployment, exactly as the faithfulness of the certificate's own evidence is.

Paper "Source-Side Continuation and Restriction of Virtual Cuts" / "Corollary (continuation over an accepted certificate)".

The *wellformed-src-history* premise is stated explicitly so the theorem reads self-contained; under the locale it is also derivable from acceptance plus faithfulness (acceptance forces *checker-run-wellformed* on the evidence run, whose first conjunct is wellformedness of the recorded history, and faithfulness identifies that history with H).

```

context layer3-checker-substrate
begin

```

```

theorem accepted-certificate-continuation-sound:

```

```

  fixes b0 :: 'k  $\rightarrow$  'v
    and H :: ('k, 'v) src-history
    and Raw :: raw-observation
  assumes accept: verify C E = Accept
    and faithful: faithful-source-observation E b0 H Raw
    and wfH: wellformed-src-history H
    and seg: cdc-segment-between H (scope C) (frontier C) f'  $\delta$ 
  shows virtual-cut-state b0 (clean-prefix  $C @ \delta$ ) (scope C) f' H
proof –
  have virtual-cut b0 C H
    by (rule accepted-virtual-cut-sound[OF accept faithful])
  hence virtual-cut-state b0 (clean-prefix C) (scope C) (frontier C) H

```

```

    unfolding virtual-cut-def .
    from virtual-cut-state-continuation[OF wfH this seg]
    show ?thesis .
qed

end

end

theory Continuation-Fixtures-Core
  imports Continuation Layer01-Virtual-Cut-Example-Core
begin

```

27 Carrier-independent continuation fixture data

The carrier-independent data and theorems of the continuation fixture family, shared with the constructed interpretation *Continuation-Fixtures-Inst*: the source-coordinate order facts, the positive continuation witness segments and their *virtual-cut-state* lemmas, the shared wellformedness / canonical-segment infrastructure, and the negative / boundary *fix-l7*... fixtures. The *l7*- prefix (and its *l7f*- / *fix-l7*- variants) tags the continuation fixture family (historically the seventh fixture layer); it does not denote a model layer. None of this mentions a run, chunk, or certificate carrier; the theory imports *DBLog-Virtual-Cuts.Continuation* (for the continuation lemmas and, transitively, the public *virtual-cut-state*) and *DBLog-Virtual-Cuts.Layer01-Virtual-Cut-Example-Core* (for *H-ex* and its wellformedness *wf-h-ex*).

```

lemma Src-at-base:
  assumes wfH: wellformed-src-history H
  shows Src b0 H c0 = b0
proof (rule ext)
  fix k
  have no-event-at-c0:
    ¬ src-le (hist-coord (H ! i)) c0 if i-lt: i < length H for i
  proof
    assume src-le (hist-coord (H ! i)) c0
    moreover have src-le c0 (hist-coord (H ! i)) by (rule c0-least)
    ultimately have hist-coord (H ! i) = c0 by (rule src-le-antisym)
    moreover have hist-coord (H ! i) ≠ c0
      using wfH i-lt unfolding wellformed-src-history-def by blast
    ultimately show False by simp
  qed
  have filter (λi. src-le (hist-coord (H ! i)) c0
    ∧ key-of (hist-event (H ! i)) = k) [0..

```

unfolding *latest-src-event-def* **by** (*simp add: Let-def*)
thus *Src b0 H c0 k = b0 k*
unfolding *Src-def* **by** *simp*
qed

lemma *l7-c0-lt-ec1*: $c0 < ec1$ **using** *c0-lt-ec1* **by** (*simp add: src-lt-eq-less*)

lemma *l7-ec1-lt-ec2*: $ec1 < ec2$ **using** *ec1-lt-ec2* **by** (*simp add: src-lt-eq-less*)

lemma *l7-ec2-lt-ec3*: $ec2 < ec3$ **using** *ec2-lt-ec3* **by** (*simp add: src-lt-eq-less*)

lemma *l7-ec3-lt-ec4*: $ec3 < ec4$ **using** *ec3-lt-ec4* **by** (*simp add: src-lt-eq-less*)

lemma *l7-c0-lt-ec2*: $c0 < ec2$ **using** *l7-c0-lt-ec1 l7-ec1-lt-ec2* **by** (*rule less-trans*)

lemma *l7-c0-lt-ec3*: $c0 < ec3$ **using** *l7-c0-lt-ec2 l7-ec2-lt-ec3* **by** (*rule less-trans*)

lemma *l7-c0-lt-ec4*: $c0 < ec4$ **using** *l7-c0-lt-ec3 l7-ec3-lt-ec4* **by** (*rule less-trans*)

lemma *l7-ec2-lt-ec4*: $ec2 < ec4$ **using** *l7-ec2-lt-ec3 l7-ec3-lt-ec4* **by** (*rule less-trans*)

lemma *l7-c0-le-ec2*: $c0 \leq ec2$ **using** *l7-c0-lt-ec2* **by** (*rule less-imp-le*)

lemma *l7-ec1-le-ec2*: $ec1 \leq ec2$ **using** *l7-ec1-lt-ec2* **by** (*rule less-imp-le*)

lemma *l7-ec2-le-ec4*: $ec2 \leq ec4$ **using** *l7-ec2-lt-ec4* **by** (*rule less-imp-le*)

lemma *l7-ec3-le-ec4*: $ec3 \leq ec4$ **using** *l7-ec3-lt-ec4* **by** (*rule less-imp-le*)

lemma *l7-not-ec3-le-ec2*: $\neg ec3 \leq ec2$ **using** *l7-ec2-lt-ec3* **by** (*simp add: not-le*)

lemma *l7-not-ec4-le-ec2*: $\neg ec4 \leq ec2$ **using** *l7-ec2-lt-ec4* **by** (*simp add: not-le*)

lemma *l7-not-ec2-lt-ec1*: $\neg ec2 < ec1$ **using** *l7-ec1-le-ec2* **by** (*simp add: not-less*)

abbreviation *l7-scope* :: *nat set* **where**
l7-scope $\equiv \{100, 200, 300\}$

definition *l7-seg1* :: (*nat, nat*) *replay-event list* **where**
l7-seg1 = [*Cdc ec1 (Update 100 3000)*, *Cdc ec2 (Insert 300 2500)*]

definition *l7-seg2* :: (*nat, nat*) *replay-event list* **where**
l7-seg2 = [*Cdc ec3 (Update 200 12000)*, *Cdc ec4 (Update 300 3500)*]

lemma *l7-seg1-segment*:
cdc-segment-between H-ex l7-scope c0 ec2 l7-seg1
unfolding *cdc-segment-between-def l7-seg1-def H-ex-def cdc-lift-def*
by (*simp add: l7-c0-le-ec2 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-c0-lt-ec3*)

*l7-c0-lt-ec4 l7-ec1-le-ec2 l7-not-ec3-le-ec2
l7-not-ec4-le-ec2)*

lemma *l7-seg2-segment:*

cdc-segment-between H-ex l7-scope ec2 ec4 l7-seg2
unfolding *cdc-segment-between-def l7-seg2-def H-ex-def cdc-lift-def*
by (*simp add: l7-ec2-le-ec4 l7-ec2-lt-ec3 l7-ec2-lt-ec4 l7-ec3-le-ec4
l7-not-ec2-lt-ec1*)

lemma *l7-vc0: virtual-cut-state ($\lambda\cdot$. None) [] l7-scope c0 H-ex*

unfolding *virtual-cut-state-def*
using *Src-at-base[OF wf-h-ex, of $\lambda\cdot$. None]*
by (*simp add: Apply-def*)

lemma *l7-vc-ec2: virtual-cut-state ($\lambda\cdot$. None) l7-seg1 l7-scope ec2 H-ex*

using *virtual-cut-state-continuation[OF wf-h-ex l7-vc0 l7-seg1-segment]*
by *simp*

theorem *continuation-positive-witness:*

virtual-cut-state ($\lambda\cdot$. None) (l7-seg1 @ l7-seg2) l7-scope ec4 H-ex
by (*rule virtual-cut-state-continuation[OF wf-h-ex l7-vc-ec2 l7-seg2-segment]*)

lemma *l7-c0-le-ec1: $c0 \leq ec1$ using l7-c0-lt-ec1 by (rule less-imp-le)*

lemma *l7-not-ec2-le-ec1: $\neg ec2 \leq ec1$ using l7-ec1-lt-ec2 by (simp add: not-le)*

lemma *l7-wf-one:*

fixes *ca :: src-coord and ea :: (nat, nat) source-event*
assumes *c0 $\neq$ ca*
shows *wellformed-src-history [(ca, ea)]*
unfolding *wellformed-src-history-def*
using *assms by auto*

lemma *l7-wf-two:*

fixes *ca cb :: src-coord*
and *ea eb :: (nat, nat) source-event*
assumes *le: src-le ca cb*
and *a0c: $c0 \neq ca$*
and *b0c: $c0 \neq cb$*
shows *wellformed-src-history [(ca, ea), (cb, eb)]*

proof –

let *?H = [(ca, ea), (cb, eb)]*
have *spo01: source-pos-order ?H 0 1*

proof –

from *le have src-lt ca cb $\vee$ ca = cb*

unfolding *src-lt-def by blast*

thus *?thesis unfolding source-pos-order-def by auto*

qed

show *?thesis*


```

  unfolding wellformed-src-history-def
proof (intro conjI allI impI)
  fix i assume Suc i < length ?H
  hence i = 0 by simp
  thus src-le (hist-coord (?H ! i)) (hist-coord (?H ! Suc i))
    using le by simp
next
  fix i assume i < length ?H
  hence i = 0 ∨ i = 1 by auto
  thus hist-coord (?H ! i) ≠ c0 using a0c b0c by auto
next
  fix i j assume i < length ?H ∧ j < length ?H ∧ i ≠ j
  hence (i = 0 ∧ j = 1) ∨ (i = 1 ∧ j = 0) by auto
  thus source-pos-order ?H i j ∨ source-pos-order ?H j i
    using spo01 by blast
qed
qed

```

lemma *l7-base-cut*:

```

  assumes wellformed-src-history H
  shows virtual-cut-state (λ-. None) [] K c0 H
  unfolding virtual-cut-state-def
  using Src-at-base[OF assms, of λ-. None]
  by (simp add: Apply-def)

```

lemma *l7-canon-continuation*:

```

  assumes wf: wellformed-src-history H
    and seg: cdc-segment-between H K c0 f' δ
  shows virtual-cut-state (λ-. None) δ K f' H
proof -
  have virtual-cut-state (λ-. None) ([[] @ δ) K f' H
    by (rule virtual-cut-state-continuation[OF wf l7-base-cut[OF wf] seg])
  thus ?thesis by simp
qed

```

lemma *l7-breaks-continuation*:

```

  fixes b0 :: 'k → 'v
  assumes canon: virtual-cut-state b0 dc K f' H
    and kK: k ∈ K
    and diff: Apply dx k ≠ Apply dc k
  shows ¬ virtual-cut-state b0 dx K f' H
proof
  assume virtual-cut-state b0 dx K f' H
  with canon
  have restrict (Apply dx) K = restrict (Apply dc) K
    unfolding virtual-cut-state-def by simp
  hence restrict (Apply dx) K k = restrict (Apply dc) K k by simp
  with kK have Apply dx k = Apply dc k
    unfolding restrict-def by simp

```

with *diff* **show** *False* **by** *simp*
qed

definition *l7-H-cov* :: (nat, nat) *src-history* **where**
l7-H-cov = [(*ec1*, *Update 100 3000*)]

lemma *l7-wf-cov: wellformed-src-history l7-H-cov*
unfolding *l7-H-cov-def* **by** (rule *l7-wf-one*[*OF c0-neq-ec1*])

lemma *l7-cov-canon-seg*:
cdc-segment-between l7-H-cov {100} c0 ec1 [Cdc ec1 (Update 100 3000)]
unfolding *cdc-segment-between-def l7-H-cov-def cdc-lift-def*
by (*simp add: l7-c0-le-ec1 l7-c0-lt-ec1*)

lemma *l7-cov-canon*:
virtual-cut-state (λ-. None) [Cdc ec1 (Update 100 3000)] {100} ec1 l7-H-cov
by (rule *l7-canon-continuation*[*OF l7-wf-cov l7-cov-canon-seg*])

theorem *fix-l7-missing-event*:
 $\neg$ *cdc-segment-between l7-H-cov {100} c0 ec1* []
 $\wedge$ $\neg$ *virtual-cut-state (λ-. None)* [] {100} *ec1 l7-H-cov*

proof
show $\neg$ *cdc-segment-between l7-H-cov {100} c0 ec1* []
unfolding *cdc-segment-between-def l7-H-cov-def cdc-lift-def*
by (*simp add: l7-c0-le-ec1 l7-c0-lt-ec1*)

next
show $\neg$ *virtual-cut-state (λ-. None)* [] {100} *ec1 l7-H-cov*
by (rule *l7-breaks-continuation*[*OF l7-cov-canon, where k = 100*])
(*simp-all add: Apply-def*)

qed

theorem *fix-l7-event-not-in-history*:
 $\neg$ *cdc-segment-between l7-H-cov {100} c0 ec1 [Cdc ec1 (Update 100 7777)]*
 $\wedge$ $\neg$ *virtual-cut-state (λ-. None)*
[Cdc ec1 (Update 100 7777)] {100} ec1 l7-H-cov

proof
show $\neg$ *cdc-segment-between l7-H-cov {100} c0 ec1 [Cdc ec1 (Update 100 7777)]*
unfolding *cdc-segment-between-def l7-H-cov-def cdc-lift-def*
by (*simp add: l7-c0-le-ec1 l7-c0-lt-ec1*)

next
show $\neg$ *virtual-cut-state (λ-. None)*
[Cdc ec1 (Update 100 7777)] {100} ec1 l7-H-cov
by (rule *l7-breaks-continuation*[*OF l7-cov-canon, where k = 100*])
(*simp-all add: Apply-def*)

qed

definition *l7-H-future* :: (nat, nat) *src-history* **where**
l7-H-future = [(*ec1*, *Update 100 3000*), (*ec2*, *Update 100 9000*)]

lemma *l7-wf-future: wellformed-src-history l7-H-future*
unfolding *l7-H-future-def*
by (rule *l7-wf-two*[*OF ec1-le-ec2 c0-neq-ec1 c0-neq-ec2*])

lemma *l7-future-canon-seg:*
cdc-segment-between l7-H-future {100} c0 ec1 [Cdc ec1 (Update 100 3000)]
unfolding *cdc-segment-between-def l7-H-future-def cdc-lift-def*
by (simp add: *l7-c0-le-ec1 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-not-ec2-le-ec1*)

lemma *l7-future-canon:*
virtual-cut-state (λ-. None) [Cdc ec1 (Update 100 3000)] {100} ec1 l7-H-future
by (rule *l7-canon-continuation*[*OF l7-wf-future l7-future-canon-seg*])

theorem *fix-l7-future-event:*
 $\neg$ *cdc-segment-between l7-H-future {100} c0 ec1*
 $[Cdc\ ec1\ (Update\ 100\ 3000),\ Cdc\ ec2\ (Update\ 100\ 9000)]$
 $\wedge \neg$ *virtual-cut-state (λ-. None)*
 $[Cdc\ ec1\ (Update\ 100\ 3000),\ Cdc\ ec2\ (Update\ 100\ 9000)]$
 $\{100\}\ ec1\ l7-H-future$

proof
show $\neg$ *cdc-segment-between l7-H-future {100} c0 ec1*
 $[Cdc\ ec1\ (Update\ 100\ 3000),\ Cdc\ ec2\ (Update\ 100\ 9000)]$
unfolding *cdc-segment-between-def l7-H-future-def cdc-lift-def*
by (simp add: *l7-c0-le-ec1 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-not-ec2-le-ec1*)

next
show $\neg$ *virtual-cut-state (λ-. None)*
 $[Cdc\ ec1\ (Update\ 100\ 3000),\ Cdc\ ec2\ (Update\ 100\ 9000)]$
 $\{100\}\ ec1\ l7-H-future$
by (rule *l7-breaks-continuation*[*OF l7-future-canon, where k = 100*])
(simp-all add: *Apply-def*)

qed

definition *l7-H-dup :: (nat, nat) src-history where*
l7-H-dup = [(ec1, Update 100 1), (ec1, Update 100 2)]

lemma *l7-wf-dup: wellformed-src-history l7-H-dup*
unfolding *l7-H-dup-def*
by (rule *l7-wf-two*[*OF src-le-refl c0-neq-ec1 c0-neq-ec1*])

lemma *l7-dup-canon-seg:*
cdc-segment-between l7-H-dup {100} c0 ec1
 $[Cdc\ ec1\ (Update\ 100\ 1),\ Cdc\ ec1\ (Update\ 100\ 2)]$
unfolding *cdc-segment-between-def l7-H-dup-def cdc-lift-def*
by (simp add: *l7-c0-le-ec1 l7-c0-lt-ec1*)

lemma *l7-dup-canon:*
virtual-cut-state (λ-. None)
 $[Cdc\ ec1\ (Update\ 100\ 1),\ Cdc\ ec1\ (Update\ 100\ 2)]\ \{100\}\ ec1\ l7-H-dup$
by (rule *l7-canon-continuation*[*OF l7-wf-dup l7-dup-canon-seg*])

theorem *fix-l7-same-coordinate-order:*

$\neg$ *cdc-segment-between l7-H-dup {100} c0 ec1*
 $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]$
 $\wedge \neg$ *virtual-cut-state* (λ -. *None*)
 $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]\ \{100\}\ ec1\ l7-H-dup$

proof

show $\neg$ *cdc-segment-between l7-H-dup {100} c0 ec1*
 $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]$
unfolding *cdc-segment-between-def l7-H-dup-def cdc-lift-def*
by (*simp add: l7-c0-le-ec1 l7-c0-lt-ec1*)

next

show $\neg$ *virtual-cut-state* (λ -. *None*)
 $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]\ \{100\}\ ec1\ l7-H-dup$
by (*rule l7-breaks-continuation[OF l7-dup-canon, where k = 100]*)
(simp-all add: Apply-def)

qed

theorem *fix-l7-multiplicity:*

$\neg$ *cdc-segment-between l7-H-dup {100} c0 ec1* $[Cdc\ ec1\ (Update\ 100\ 1)]$
 $\wedge \neg$ *virtual-cut-state* (λ -. *None*)
 $[Cdc\ ec1\ (Update\ 100\ 1)]\ \{100\}\ ec1\ l7-H-dup$

proof

show $\neg$ *cdc-segment-between l7-H-dup {100} c0 ec1* $[Cdc\ ec1\ (Update\ 100\ 1)]$
unfolding *cdc-segment-between-def l7-H-dup-def cdc-lift-def*
by (*simp add: l7-c0-le-ec1 l7-c0-lt-ec1*)

next

show $\neg$ *virtual-cut-state* (λ -. *None*)
 $[Cdc\ ec1\ (Update\ 100\ 1)]\ \{100\}\ ec1\ l7-H-dup$
by (*rule l7-breaks-continuation[OF l7-dup-canon, where k = 100]*)
(simp-all add: Apply-def)

qed

theorem *fix-l7-coordinate-sorted-insufficient:*

sorted (*map cdc-coord* $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]$)
 $\wedge \neg$ *cdc-segment-between l7-H-dup {100} c0 ec1*
 $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]$
 $\wedge \neg$ *virtual-cut-state* (λ -. *None*)
 $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]\ \{100\}\ ec1\ l7-H-dup$

proof (*intro conjI*)

show *sorted* (*map cdc-coord* $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]$)
by *simp*

next

show $\neg$ *cdc-segment-between l7-H-dup {100} c0 ec1*
 $[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]$
unfolding *cdc-segment-between-def l7-H-dup-def cdc-lift-def*
by (*simp add: l7-c0-le-ec1 l7-c0-lt-ec1*)

next

show $\neg$ *virtual-cut-state* (λ -. *None*)

$[Cdc\ ec1\ (Update\ 100\ 2),\ Cdc\ ec1\ (Update\ 100\ 1)]\ \{100\}\ ec1\ l7-H-dup$
by (rule l7-breaks-continuation[OF l7-dup-canon, **where** $k = 100$])
 (simp-all add: Apply-def)
qed

definition l7-H-back :: (nat, nat) src-history **where**
 l7-H-back = [(ec2, Update 100 5000)]

lemma l7-wf-back: wellformed-src-history l7-H-back
unfolding l7-H-back-def **by** (rule l7-wf-one[OF c0-neq-ec2])

lemma l7-back-canon-seg:
 cdc-segment-between l7-H-back {100} c0 ec2 [Cdc ec2 (Update 100 5000)]
unfolding cdc-segment-between-def l7-H-back-def cdc-lift-def
by (simp add: l7-c0-le-ec2 l7-c0-lt-ec2)

lemma l7-back-canon:
 virtual-cut-state ($\lambda\cdot$. None) [Cdc ec2 (Update 100 5000)] {100} ec2 l7-H-back
by (rule l7-canon-continuation[OF l7-wf-back l7-back-canon-seg])

lemma l7-back-src-ec1: Src ($\lambda\cdot$. None) l7-H-back ec1 = ($\lambda\cdot$. None)
proof (rule ext)
fix k :: nat
have seg-empty:
 filter ($\lambda(c, e).$ c0 < c $\wedge$ c $\leq$ ec1 $\wedge$ key-of e = k) l7-H-back = []
unfolding l7-H-back-def **using** l7-not-ec2-le-ec1 **by** simp
have Src ($\lambda\cdot$. None) l7-H-back ec1 k = Src ($\lambda\cdot$. None) l7-H-back c0 k
using Src-interval-decomposition[OF l7-wf-back l7-c0-le-ec1,
 of $\lambda\cdot$. None k] seg-empty
by (simp add: Let-def)
also have ... = None
using Src-at-base[OF l7-wf-back, of $\lambda\cdot$. None] **by** simp
finally show Src ($\lambda\cdot$. None) l7-H-back ec1 k = ($\lambda\cdot$. None) k **by** simp
qed

theorem fix-l7-frontier-order:
 ($\forall \delta.$ $\neg$ cdc-segment-between l7-H-back {100} ec2 ec1 δ)
 $\wedge$ virtual-cut-state ($\lambda\cdot$. None) [Cdc ec2 (Update 100 5000)] {100} ec2 l7-H-back
 $\wedge$ $\neg$ virtual-cut-state ($\lambda\cdot$. None)
 [Cdc ec2 (Update 100 5000)] {100} ec1 l7-H-back
proof (intro conjI)
show $\forall \delta.$ $\neg$ cdc-segment-between l7-H-back {100} ec2 ec1 δ
unfolding cdc-segment-between-def **using** l7-not-ec2-le-ec1 **by** simp
next
show virtual-cut-state ($\lambda\cdot$. None) [Cdc ec2 (Update 100 5000)] {100} ec2
 l7-H-back
by (rule l7-back-canon)
next
show $\neg$ virtual-cut-state ($\lambda\cdot$. None)

```

    [Cdc ec2 (Update 100 5000)] {100} ec1 l7-H-back
proof
  assume virtual-cut-state (λ-. None)
    [Cdc ec2 (Update 100 5000)] {100} ec1 l7-H-back
  hence restrict (Apply [Cdc ec2 (Update 100 5000)]) {100} (100::nat)
    = restrict (Src (λ-. None) l7-H-back ec1) {100} 100
  unfolding virtual-cut-state-def by simp
  thus False
  by (simp add: restrict-def l7-back-src-ec1 Apply-def)
qed
qed

definition l7-H-del :: (nat, nat) src-history where
  l7-H-del = [(ec1, Update 100 3000), (ec2, Delete 100)]

lemma l7-wf-del: wellformed-src-history l7-H-del
  unfolding l7-H-del-def
  by (rule l7-wf-two[OF ec1-le-ec2 c0-neq-ec1 c0-neq-ec2])

lemma l7-del-canon-seg:
  cdc-segment-between l7-H-del {100} c0 ec2
    [Cdc ec1 (Update 100 3000), Cdc ec2 (Delete 100)]
  unfolding cdc-segment-between-def l7-H-del-def cdc-lift-def
  by (simp add: l7-c0-le-ec2 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-ec1-le-ec2)

lemma l7-del-canon:
  virtual-cut-state (λ-. None)
    [Cdc ec1 (Update 100 3000), Cdc ec2 (Delete 100)] {100} ec2 l7-H-del
  by (rule l7-canon-continuation[OF l7-wf-del l7-del-canon-seg])

theorem fix-l7-delete-omitted:
  ¬ cdc-segment-between l7-H-del {100} c0 ec2 [Cdc ec1 (Update 100 3000)]
  ∧ ¬ virtual-cut-state (λ-. None)
    [Cdc ec1 (Update 100 3000)] {100} ec2 l7-H-del
proof
  show ¬ cdc-segment-between l7-H-del {100} c0 ec2 [Cdc ec1 (Update 100 3000)]
    unfolding cdc-segment-between-def l7-H-del-def cdc-lift-def
    by (simp add: l7-c0-le-ec2 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-ec1-le-ec2)
  next
  show ¬ virtual-cut-state (λ-. None)
    [Cdc ec1 (Update 100 3000)] {100} ec2 l7-H-del
    by (rule l7-breaks-continuation[OF l7-del-canon, where k = 100])
    (simp-all add: Apply-def)
qed

definition l7-H-oos :: (nat, nat) src-history where
  l7-H-oos = [(ec1, Update 100 3000), (ec2, Insert 999 5)]

lemma l7-wf-oos: wellformed-src-history l7-H-oos

```

unfolding *l7-H-oos-def*
by (rule *l7-wf-two*[*OF ec1-le-ec2 c0-neq-ec1 c0-neq-ec2*])

lemma *l7-oos-scoped-seg*:
cdc-segment-between l7-H-oos {100} c0 ec2 [Cdc ec1 (Update 100 3000)]
unfolding *cdc-segment-between-def l7-H-oos-def cdc-lift-def*
by (simp add: *l7-c0-le-ec2 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-ec1-le-ec2*)

lemma *l7-oos-scoped-canon*:
virtual-cut-state (λ-. None) [Cdc ec1 (Update 100 3000)] {100} ec2 l7-H-oos
by (rule *l7-canon-continuation*[*OF l7-wf-oos l7-oos-scoped-seg*])

lemma *l7-oos-full-seg*:
cdc-segment-between l7-H-oos UNIV c0 ec2
[Cdc ec1 (Update 100 3000), Cdc ec2 (Insert 999 5)]
unfolding *cdc-segment-between-def l7-H-oos-def cdc-lift-def*
by (simp add: *l7-c0-le-ec2 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-ec1-le-ec2*)

lemma *l7-oos-src-ec2*:
Src (λ-. None) l7-H-oos ec2
= Apply [Cdc ec1 (Update 100 3000), Cdc ec2 (Insert 999 5)]
proof –
have *base: Apply [] = Src (λ-. None) l7-H-oos c0*
using *Src-at-base*[*OF l7-wf-oos, of λ-. None*] **by** (simp add: *Apply-def*)
have *Apply ([] @ [Cdc ec1 (Update 100 3000), Cdc ec2 (Insert 999 5)])*
= Src (λ-. None) l7-H-oos ec2
by (rule *whole-table-state-continuation*[*OF l7-wf-oos base l7-oos-full-seg*])
thus *?thesis* **by** *simp*
qed

theorem *fix-l7-scoped-not-whole-table*:
virtual-cut-state (λ-. None) [Cdc ec1 (Update 100 3000)] {100} ec2 l7-H-oos
∧ Apply [Cdc ec1 (Update 100 3000)] ≠ Src (λ-. None) l7-H-oos ec2
proof
show *virtual-cut-state (λ-. None) [Cdc ec1 (Update 100 3000)] {100} ec2 l7-H-oos*
by (rule *l7-oos-scoped-canon*)
next
show *Apply [Cdc ec1 (Update 100 3000)] ≠ Src (λ-. None) l7-H-oos ec2*
proof
assume *eq: Apply [Cdc ec1 (Update 100 3000)] = Src (λ-. None) l7-H-oos ec2*
hence *Apply [Cdc ec1 (Update 100 3000)] (999::nat)*
= Src (λ-. None) l7-H-oos ec2 999
by *simp*
thus *False* **by** (simp add: *l7-oos-src-ec2 Apply-def*)
qed
qed

lemma *l7-virtual-cut-source-determined*:
fixes *b0 :: 'k → 'v*

```

assumes virtual-cut-state b0  $\sigma 1$  K f H
and virtual-cut-state b0  $\sigma 2$  K f H
shows restrict (Apply  $\sigma 1$ ) K = restrict (Apply  $\sigma 2$ ) K
using assms unfolding virtual-cut-state-def by simp

theorem fix-l7-rechunk-not-sink-repair:
  virtual-cut-state ( $\lambda\cdot$ . None) (l7-seg1 @ l7-seg2) l7-scope ec4 H-ex
   $\wedge$  ( $\forall \sigma'$ . virtual-cut-state ( $\lambda\cdot$ . None)  $\sigma'$  l7-scope ec4 H-ex
     $\longrightarrow$  restrict (Apply  $\sigma'$ ) l7-scope
    = restrict (Apply (l7-seg1 @ l7-seg2)) l7-scope)

proof
  show virtual-cut-state ( $\lambda\cdot$ . None) (l7-seg1 @ l7-seg2) l7-scope ec4 H-ex
    by (rule continuation-positive-witness)
next
  show  $\forall \sigma'$ . virtual-cut-state ( $\lambda\cdot$ . None)  $\sigma'$  l7-scope ec4 H-ex
     $\longrightarrow$  restrict (Apply  $\sigma'$ ) l7-scope
    = restrict (Apply (l7-seg1 @ l7-seg2)) l7-scope
  proof (intro allI impI)
    fix  $\sigma'$  assume virtual-cut-state ( $\lambda\cdot$ . None)  $\sigma'$  l7-scope ec4 H-ex
    from l7-virtual-cut-source-determined[OF this continuation-positive-witness]
    show restrict (Apply  $\sigma'$ ) l7-scope
      = restrict (Apply (l7-seg1 @ l7-seg2)) l7-scope .
  qed
qed

```

28 Positive-witness corollaries and the out-of-scope padding boundary

The positive-witness chaining and non-vacuity facts and the out-of-scope padding boundary fixture. All are purely source-side — no certificate carrier is involved, so they have no constructed *Continuation-Fixtures-Inst* counterpart.

```

theorem continuation-witness-chained-segment:
  cdc-segment-between H-ex l7-scope c0 ec4 (l7-seg1 @ l7-seg2)
  by (rule cdc-segment-between-concat[OF wf-h-ex l7-seg1-segment l7-seg2-segment])

```

```

theorem continuation-witness-nonvacuous:
  l7-scope  $\neq \{\}$   $\wedge$  l7-seg1  $\neq []$   $\wedge$  l7-seg2  $\neq []$ 
  by (simp add: l7-seg1-def l7-seg2-def)

```

```

theorem fix-l7-out-of-scope-padding-rejected-not-loadbearing:
   $\neg$  cdc-segment-between l7-H-oos {100} c0 ec2
    [Cdc ec1 (Update 100 3000), Cdc ec2 (Insert 999 5)]
   $\wedge$  virtual-cut-state ( $\lambda\cdot$ . None)
    [Cdc ec1 (Update 100 3000), Cdc ec2 (Insert 999 5)] {100} ec2 l7-H-oos
proof
  show  $\neg$  cdc-segment-between l7-H-oos {100} c0 ec2

```



```

    [Cdc ec1 (Update 100 3000), Cdc ec2 (Insert 999 5)]
  unfolding cdc-segment-between-def l7-H-oos-def cdc-lift-def
  by (simp add: l7-c0-le-ec2 l7-c0-lt-ec1 l7-c0-lt-ec2 l7-ec1-le-ec2)
next
  show virtual-cut-state (λ-. None)
    [Cdc ec1 (Update 100 3000), Cdc ec2 (Insert 999 5)] {100} ec2 l7-H-oos
  unfolding virtual-cut-state-def
  by (simp add: l7-oos-src-ec2)
qed

end

theory Layer01-Witnesses-Model
  imports Layer01-Witnesses-Core
begin

```

29 Constructed model for the minimum-viable wellformed-run witness

This theory exhibits the explicit model of the minimum-viable wellformed-run witness data: a one-point run carrier (*unit*), a two-element chunk type, and concrete accessor functions giving the witness run scope $\{0, 1\}$, frontier *c2-w*, and two chunks with singleton domains. *Layer01-Witnesses-Inst* interprets the run-substrate locale (*dblog-run-substrate*) at this model, so the witness facts — the run-wellformedness fact and the Layer 1 positive-witness existential — are established constructively, with no axioms.

The carrier-independent witness data this model realizes (*b0-w*, *H-w*, the *Src* / *latest-src-event* computation lemmas, *wf-h-w*) comes from the imported *DBLog-Virtual-Cuts.Layer01-Witnesses-Core*.

datatype *wit-cchunk* = *WCh1* | *WCh2*

definition *wit-cscope* :: *unit* $\Rightarrow$ *nat set* **where**
wit-cscope - = $\{0, 1\}$

definition *wit-cfrontier* :: *unit* $\Rightarrow$ *frontier* **where**
wit-cfrontier - = *c2-w*

definition *wit-cchunks* :: *unit* $\Rightarrow$ *wit-cchunk set* **where**
wit-cchunks - = $\{WCh1, WCh2\}$

definition *wit-cchunks-list* :: *unit* $\Rightarrow$ *wit-cchunk list* **where**
wit-cchunks-list - = $[WCh1, WCh2]$

definition *wit-cchunk-domain* :: *unit* $\Rightarrow$ *wit-cchunk* $\Rightarrow$ *nat set* **where**
wit-cchunk-domain - *ch* = (case *ch* of *WCh1* $\Rightarrow$ $\{0\}$ | *WCh2* $\Rightarrow$ $\{1\}$)

definition *wit-cresponsible-chunk* :: *unit* $\Rightarrow$ *nat* $\Rightarrow$ *wit-cchunk option* **where**
wit-cresponsible-chunk - *k* =
 (if *k* = 0 then Some WCh1 else if *k* = 1 then Some WCh2 else None)

definition *wit-cchunk-read-coordinate* :: *unit* $\Rightarrow$ *wit-cchunk* $\Rightarrow$ *src-coord* **where**
wit-cchunk-read-coordinate - *ch* = (case *ch* of WCh1 $\Rightarrow$ *c0* | WCh2 $\Rightarrow$ *c2-w*)

definition *wit-cchunk-lower-watermark* :: *unit* $\Rightarrow$ *wit-cchunk* $\Rightarrow$ *src-coord* **where**
wit-cchunk-lower-watermark - *ch* = (case *ch* of WCh1 $\Rightarrow$ *c0* | WCh2 $\Rightarrow$ *c2-w*)

definition *wit-cchunk-upper-watermark* :: *unit* $\Rightarrow$ *wit-cchunk* $\Rightarrow$ *src-coord* **where**
wit-cchunk-upper-watermark - *ch* = (case *ch* of WCh1 $\Rightarrow$ *c0* | WCh2 $\Rightarrow$ *c2-w*)

definition *wit-csrc-history-of* :: *unit* $\Rightarrow$ (*nat*, *nat*) *src-history* **where**
wit-csrc-history-of - = *H-w*

definition *wit-ccdc-events-of* :: *unit* $\Rightarrow$ (*nat*, *nat*) *src-history* **where**
wit-ccdc-events-of - = *H-w*

definition *wit-cchunk-read-result* :: *unit* $\Rightarrow$ *wit-cchunk* $\Rightarrow$ *nat* $\Rightarrow$ *nat option option* **where**

wit-cchunk-read-result - *ch k* =
 (if *ch* = WCh1 $\wedge$ *k* = 0 then Some None
 else if *ch* = WCh2 $\wedge$ *k* = 1 then Some None
 else None)

theorem *wit-model-accessor-values*:

WCh1 $\neq$ *WCh2*
 $\wedge$ *wit-cscope* () = {0, 1}
 $\wedge$ *wit-cfrontier* () = *c2-w*
 $\wedge$ *wit-cchunks* () = {WCh1, WCh2}
 $\wedge$ *wit-cchunk-domain* () WCh1 = {0}
 $\wedge$ *wit-cchunk-domain* () WCh2 = {1}
 $\wedge$ *wit-cresponsible-chunk* () 0 = Some WCh1
 $\wedge$ *wit-cresponsible-chunk* () 1 = Some WCh2
 $\wedge$ ($\forall k. k \neq 0 \longrightarrow k \neq 1 \longrightarrow$ *wit-cresponsible-chunk* () *k* = None)
 $\wedge$ *wit-cchunk-read-coordinate* () WCh1 = *c0*
 $\wedge$ *wit-cchunk-read-coordinate* () WCh2 = *c2-w*
 $\wedge$ *wit-cchunk-lower-watermark* () WCh1 = *c0*
 $\wedge$ *wit-cchunk-upper-watermark* () WCh1 = *c0*
 $\wedge$ *wit-cchunk-lower-watermark* () WCh2 = *c2-w*
 $\wedge$ *wit-cchunk-upper-watermark* () WCh2 = *c2-w*
 $\wedge$ *wit-csrc-history-of* () = *H-w*
 $\wedge$ *wit-cchunk-read-result* () WCh1 0 = Some None
 $\wedge$ *wit-cchunk-read-result* () WCh2 1 = Some None
 $\wedge$ *wit-ccdc-events-of* () = *H-w*
 $\wedge$ *wit-cchunks-list* () = [WCh1, WCh2]
 $\wedge$ *set* (*wit-cchunks-list* ()) = *wit-cchunks* ()
 $\wedge$ *distinct* (*wit-cchunks-list* ())

```

by (auto simp: wit-cscope-def wit-cfrontier-def wit-cchunks-def
    wit-cchunks-list-def wit-cchunk-domain-def
    wit-cresponsible-chunk-def wit-cchunk-read-coordinate-def
    wit-cchunk-lower-watermark-def wit-cchunk-upper-watermark-def
    wit-csrc-history-of-def wit-ccdc-events-of-def
    wit-cchunk-read-result-def)

```

The conjunction above gathers the model’s accessor values in one place — the witness configuration is realized by an explicit construction. *Layer01-Witnesses-Inst* re-derives these values as per-accessor lemmas and builds the wellformedness and positive-witness theorems (*wit-model-wellformed*, *wellformed-run-positive-witness-constructed*) on top of them.

end

```

theory Layer01-Witnesses-Inst
  imports Layer01-Witnesses-Model DBLog-Run-Substrate-Layer2
begin

```

30 Constructed interpretation for the minimum-viable wellformed-run witness

Constructs an explicit model of the minimum-viable wellformed-run positive witness and interprets the Layer 1 locale *dblog-run-substrate* at it, establishing the witness facts with no axioms. The model is that of *Layer01-Witnesses-Model* (the *unit* run carrier, the *wit-cchunk* two-element chunk type, and the definitional accessors).

Structure:

- **Constructed interpretation.** We instantiate *dblog-run-substrate* at the model carrier (*unit* run, *wit-cchunk* chunks, the *wit-c...* accessors). *dblog-run-substrate* assumes only the two *chunks-list* obligations, which hold definitionally for the model enumeration [*WCh1*, *WCh2*], so the interpretation discharges them with *simp* — no axiom.
- **Accessor-value lemmas.** The *wm-...* lemmas read off the model’s definitional accessor values pointwise — the conjuncts of *wit-model-accessor-values* — in exactly the shape used as rewrite facts by the determinate clean-prefix computation and the seven-clause wellformedness discharge below. The carrier-independent witness data they refer to (*b0-w*, *H-w*, the *Src* / *latest-src-event* computation lemmas, *wf-h-w*) lives in *Layer01-Witnesses-Core*.

The constructed witness needs the key/value carrier fixed to concrete *nat* (the witness keys and values are literal naturals); a lemma inside the locale

context cannot do this, because the locale type parameter $'k$ is rigid there. The constructed wellformed-run fact *wit-model-wellformed* and the constructed positive witness *wellformed-run-positive-witness-constructed* therefore live in the global context over the interpretation, neither resting on any run/chunk axiom.

The witness source coordinates $c1-w / c2-w$ are *coord-of-nat* definitions from *Source-Coordinates* ($c1-w = \text{coord-of-nat } 6$, $c2-w = \text{coord-of-nat } 7$); the strict chain $c0-lt-c1-w / c1-w-lt-c2-w$ and its basic consequences are proved there by *simp* from *nat* arithmetic, and further derived ordering facts on the witness data (e.g. *not-src-lt-c2-w-c1-w*) live in *Layer01-Witnesses-Core*.

30.1 Constructed interpretation at the witness model

Instantiate *dblog-run-substrate* at the model carrier: the *unit* run, the *wit-cchunk* chunks, and the definitional *wit-c...* accessors of *Layer01-Witnesses-Model*. The two *chunks-list* obligations reduce to set $[WCh1, WCh2] = \{WCh1, WCh2\}$ and *distinct* $[WCh1, WCh2]$, discharged by *simp* — no axiom.

global-interpretation *wit-model*: *dblog-run-substrate*
wit-cscope
wit-cfrontier
wit-cchunks
wit-csrc-history-of
wit-cchunk-read-result
wit-ccdc-events-of
wit-cchunks-list
wit-cchunk-domain
wit-cresponsible-chunk
wit-cchunk-lower-watermark
wit-cchunk-upper-watermark
wit-cchunk-read-coordinate
by *unfold-locales* (*simp-all* *add*: *wit-cchunks-def* *wit-cchunks-list-def*)

30.2 Model accessor-value lemmas

Each *wm-...* lemma reads off one model accessor value from its definitional equation. Together they are the conjuncts of *wit-model-accessor-values*, used as the rewrite facts that drive the proofs below.

lemma *wm-distinct*: $WCh1 \neq WCh2$ **by** *simp*
lemma *wm-scope*: *wit-cscope* () = $\{0, 1\}$ **by** (*simp* *add*: *wit-cscope-def*)
lemma *wm-frontier*: *wit-cfrontier* () = $c2-w$ **by** (*simp* *add*: *wit-cfrontier-def*)
lemma *wm-chunks*: *wit-cchunks* () = $\{WCh1, WCh2\}$ **by** (*simp* *add*: *wit-cchunks-def*)
lemma *wm-dom-1*: *wit-cchunk-domain* () $WCh1 = \{0\}$
by (*simp* *add*: *wit-cchunk-domain-def*)
lemma *wm-dom-2*: *wit-cchunk-domain* () $WCh2 = \{1\}$

by (simp add: wit-cchunk-domain-def)
lemma *wm-resp-0*: *wit-cresponsible-chunk* () 0 = Some WCh1
 by (simp add: wit-cresponsible-chunk-def)
lemma *wm-resp-1*: *wit-cresponsible-chunk* () 1 = Some WCh2
 by (simp add: wit-cresponsible-chunk-def)
lemma *wm-resp-else*:
 $k \neq 0 \implies k \neq 1 \implies \text{wit-cresponsible-chunk } () \ k = \text{None}$
 by (simp add: wit-cresponsible-chunk-def)
lemma *wm-rcoord-1*: *wit-cchunk-read-coordinate* () WCh1 = c0
 by (simp add: wit-cchunk-read-coordinate-def)
lemma *wm-rcoord-2*: *wit-cchunk-read-coordinate* () WCh2 = c2-w
 by (simp add: wit-cchunk-read-coordinate-def)
lemma *wm-lwm-1*: *wit-cchunk-lower-watermark* () WCh1 = c0
 by (simp add: wit-cchunk-lower-watermark-def)
lemma *wm-uwm-1*: *wit-cchunk-upper-watermark* () WCh1 = c0
 by (simp add: wit-cchunk-upper-watermark-def)
lemma *wm-lwm-2*: *wit-cchunk-lower-watermark* () WCh2 = c2-w
 by (simp add: wit-cchunk-lower-watermark-def)
lemma *wm-uwm-2*: *wit-cchunk-upper-watermark* () WCh2 = c2-w
 by (simp add: wit-cchunk-upper-watermark-def)
lemma *wm-src-history*: *wit-csrc-history-of* () = H-w
 by (simp add: wit-csrc-history-of-def)
lemma *wm-crr-1-0*: *wit-cchunk-read-result* () WCh1 0 = Some None
 by (simp add: wit-cchunk-read-result-def)
lemma *wm-crr-2-1*: *wit-cchunk-read-result* () WCh2 1 = Some None
 by (simp add: wit-cchunk-read-result-def)
lemma *wm-cdc*:
 $\text{wit-ccdc-events-of } () = [(c1-w, \text{Insert } 0 \ 42), (c2-w, \text{Delete } 1)]$
 by (simp add: wit-ccdc-events-of-def H-w-def)
lemma *wm-chunks-list*: *wit-cchunks-list* () = [WCh1, WCh2]
 by (simp add: wit-cchunks-list-def)

lemma *wm-chunks-eq*:
 $(ch \in \text{wit-cchunks } ()) \longleftrightarrow (ch = WCh1 \vee ch = WCh2)$
 using *wm-chunks* by simp

30.3 Determinate clean prefix and Apply over the model

The determinate clean prefix of the witness run, computed over the model carrier as an explicit four-event list.

lemma *wm-clean-prefix*:
 $\text{wit-model.clean-prefix-of } ()$
 $= [\text{Refresh } (0::\text{nat}) \ (\text{None}::\text{nat option}) \ c0,$
 $\quad \text{Cdc } c1-w \ (\text{Insert } 0 \ 42),$
 $\quad \text{Cdc } c2-w \ (\text{Delete } 1),$
 $\quad \text{Refresh } 1 \ \text{None} \ c2-w]$
proof –
 have *d0*: *sorted-list-of-set* {0::nat} = [0] by simp
 have *d1*: *sorted-list-of-set* {Suc 0} = [Suc 0] by simp

```

have le12: src-le c1-w c2-w
  using c1-w-lt-c2-w by (simp add: src-lt-def)
have le02: src-le c0 c2-w
  using c0-le-c1-w le12 src-le-trans by blast
have n10: ¬ src-le c1-w c0
  using c0-le-c1-w c0-neq-c1-w src-le-antisym by blast
have n20: ¬ src-le c2-w c0
  using le02 c0-neq-c2-w src-le-antisym by blast
have n21: ¬ src-le c2-w c1-w
  using le12 c1-w-neq-c2-w src-le-antisym by blast
show ?thesis
  unfolding wit-model.clean-prefix-of-def canonical-clean-prefix-def
    cdc-event-replays-def chunk-read-refreshes-def
  by (simp add: wm-chunks-list wm-dom-1 wm-dom-2
    wm-rcoord-1 wm-rcoord-2
    wm-crr-1-0 wm-crr-2-1 [unfolded One-nat-def]
    wm-cdc wm-scope wm-frontier d0 d1 One-nat-def
    List.map-filter-simps sort-key-def
    c0-le-c1-w [unfolded src-le-eq-less-eq]
    le02 [unfolded src-le-eq-less-eq]
    le12 [unfolded src-le-eq-less-eq]
    n10 [unfolded src-le-eq-less-eq]
    n20 [unfolded src-le-eq-less-eq]
    n21 [unfolded src-le-eq-less-eq])

```

qed

Replaying the witness clean prefix: *Apply* yields the map sending key 0 to 42 and every other key to *None*.

lemma *wm-apply*:

```

Apply (wit-model.clean-prefix-of ())
  = (λk. if k = (0::nat) then Some (42::nat) else None)

```

proof –

```

have Apply (wit-model.clean-prefix-of ())
  = foldl apply-step (λ-. None)
    [ Refresh (0::nat) (None::nat option) c0,
      Cdc c1-w (Insert 0 42),
      Cdc c2-w (Delete 1),
      Refresh 1 None c2-w ]
  unfolding Apply-def wm-clean-prefix by (rule refl)
also have ... = (λk. if k = (0::nat) then Some (42::nat) else None)
  by (auto simp: fun-eq-iff fun-upd-def)
finally show ?thesis .

```

qed

30.4 Constructed wellformed-run fact

The minimum-viable witness is wellformed, proved over the constructed carrier with no run/chunk axiom: a clause-by-clause discharge of the seven-clause *wellformed-dblog-run* definition, driven by the *wm-...* accessor-value

lemmas together with the *Layer01-Witnesses-Core* facts about $b0-w$ / $H-w$.

theorem *wit-model-wellformed*:

wit-model.wellformed-dblog-run $b0-w$ () $H-w$

unfolding *wit-model.wellformed-dblog-run-def*

proof (*intro conjI*)

— WF0 part 1: source-history binding.

show *wit-csrc-history-of* () = $H-w$ **by** (*rule wm-src-history*)

next

— WF0 part 2: source-history wellformedness.

show *wellformed-src-history* $H-w$ **by** (*rule wf-h-w*)

next

— WF1 (a): unique responsible chunk per in-scope key.

show $\forall k \in \text{wit-cscope} (). \exists ! ch. ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} ()$ ch

proof

fix $k :: nat$

assume $k \in \text{wit-cscope} ()$

hence $k\text{-cases}$: $k = 0 \vee k = 1$ **using** *wm-scope* **by** *simp*

show $\exists ! ch. ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} ()$ ch

proof (*cases* $k = 0$)

case *True*

have $WCh1 \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} ()$ $WCh1$

using *wm-chunks wm-dom-1 True* **by** *simp*

moreover have $\bigwedge ch. ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} ()$ ch

$\Rightarrow ch = WCh1$

using *wm-chunks-eq wm-dom-1 wm-dom-2 True* **by** *auto*

ultimately show *?thesis* **by** *blast*

next

case *False*

hence $k1$: $k = 1$ **using** $k\text{-cases}$ **by** *simp*

have $WCh2 \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} ()$ $WCh2$

using *wm-chunks wm-dom-2 k1* **by** *simp*

moreover have $\bigwedge ch. ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} ()$ ch

$\Rightarrow ch = WCh2$

using *wm-chunks-eq wm-dom-1 wm-dom-2 k1* **by** *auto*

ultimately show *?thesis* **by** *blast*

qed

qed

next

— WF1 (b): chunk domains pairwise disjoint.

show $\forall ch1 \in \text{wit-cchunks} (). \forall ch2 \in \text{wit-cchunks} ().$

$ch1 \neq ch2 \longrightarrow \text{wit-cchunk-domain} () ch1 \cap \text{wit-cchunk-domain} () ch2 =$

$\{\}$

using *wm-chunks-eq wm-dom-1 wm-dom-2* **by** *auto*

next

— WF1 (c): canonical chunk-ownership domain equals scope.

show *wit-model.canonical-chunk-ownership-domain* () = *wit-cscope* ()

unfolding *wit-model.canonical-chunk-ownership-domain-def*

using *wm-chunks wm-dom-1 wm-dom-2 wm-scope* **by** *auto*

```

next
— WF1 (d): responsible-chunk  $\longleftrightarrow$  ownership predicate (in scope).
show  $\forall k \in \text{wit-cscope} (). \forall ch.$ 
    wit-cresponsible-chunk ()  $k = \text{Some } ch$ 
     $\longleftrightarrow ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} () ch$ 
proof (intro ballI allI)
  fix  $k :: \text{nat}$  and  $ch :: \text{wit-cchunk}$ 
  assume  $k \in \text{wit-cscope} ()$ 
  hence  $k\text{-cases}: k = 0 \vee k = 1$  using wm-scope by simp
  show wit-cresponsible-chunk ()  $k = \text{Some } ch$ 
     $\longleftrightarrow ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} () ch$ 
  proof (cases  $k = 0$ )
    case True
    then have  $rc: \text{wit-cresponsible-chunk} () k = \text{Some } WCh1$ 
      using wm-resp-0 by simp
    show ?thesis
    proof
      assume wit-cresponsible-chunk ()  $k = \text{Some } ch$ 
      with  $rc$  have  $ch = WCh1$  by simp
      thus  $ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} () ch$ 
        using wm-chunks wm-dom-1 True by simp
    next
      assume  $A: ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} () ch$ 
      have  $ch = WCh1$ 
        using  $A$  wm-chunks-eq wm-dom-1 wm-dom-2 True by auto
      with  $rc$  show wit-cresponsible-chunk ()  $k = \text{Some } ch$  by simp
    qed
  next
    case False
    hence  $k1: k = 1$  using  $k\text{-cases}$  by simp
    then have  $rc: \text{wit-cresponsible-chunk} () k = \text{Some } WCh2$ 
      using wm-resp-1 by simp
    show ?thesis
    proof
      assume wit-cresponsible-chunk ()  $k = \text{Some } ch$ 
      with  $rc$  have  $ch = WCh2$  by simp
      thus  $ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} () ch$ 
        using wm-chunks wm-dom-2 k1 by simp
    next
      assume  $A: ch \in \text{wit-cchunks} () \wedge k \in \text{wit-cchunk-domain} () ch$ 
      have  $ch = WCh2$ 
        using  $A$  wm-chunks-eq wm-dom-1 wm-dom-2 k1 by auto
      with  $rc$  show wit-cresponsible-chunk ()  $k = \text{Some } ch$  by simp
    qed
  qed
qed
next
— WF1 (e): out-of-scope keys have no responsible chunk.
show  $\forall k. k \notin \text{wit-cscope} () \longrightarrow \text{wit-cresponsible-chunk} () k = \text{None}$ 

```



```

    using wm-scope wm-resp-else by auto
next
  — WF1 (f) part 1: scope is finite.
  show finite (wit-cscope ()) using wm-scope by simp
next
  — WF1 (f) part 2: chunk domains are finite.
  show  $\forall ch \in \text{wit-cchunks} (). \text{finite } (\text{wit-cchunk-domain } () \text{ } ch)$ 
    using wm-chunks-eq wm-dom-1 wm-dom-2 by auto
next
  — WF1 (g): every chunk has a non-empty domain.
  show  $\forall ch \in \text{wit-cchunks} (). \text{wit-cchunk-domain } () \text{ } ch \neq \{\}$ 
    using wm-chunks-eq wm-dom-1 wm-dom-2 by auto
next
  — WF2 coverage: observed CDC reaches every in-scope key.
  show  $\forall k \in \text{wit-cscope} (). \text{wit-model.covers-ordinary-cdc } () \text{ } k (\text{wit-cfrontier } ())$ 
  proof
    fix k :: nat assume k  $\in \text{wit-cscope} ()$ 
    hence k-cases: k = 0  $\vee$  k = 1 using wm-scope by simp
    show wit-model.covers-ordinary-cdc () k (wit-cfrontier ())
    proof (cases k = 0)
      case True
      show ?thesis
        unfolding wit-model.covers-ordinary-cdc-def
      proof (intro exI[where x = WCh1] conjI allI impI)
        show wit-cresponsible-chunk () k = Some WCh1
          using wm-resp-0 True by simp
      next
        fix i assume i-lt: i < length (wit-csrc-history-of ())
          and chunk-lt: src-lt (wit-cchunk-read-coordinate () WCh1)
            (hist-coord (wit-csrc-history-of () ! i))
          and to-f: src-le (hist-coord (wit-csrc-history-of () ! i))
            (wit-cfrontier ())
          and key-eq: key-of (hist-event (wit-csrc-history-of () ! i)) = k
        show wit-csrc-history-of () ! i  $\in \text{set } (\text{wit-ccdc-events-of } ())$ 
          using i-lt chunk-lt to-f key-eq True
            wm-src-history wm-cdc length-H-w
            H-w-nth-0 H-w-nth-1
          by (cases i = 0; cases i = 1; auto simp: numeral-2-eq-2 set-H-w)
      qed
    next
      case False
      hence k1: k = 1 using k-cases by simp
      show ?thesis
        unfolding wit-model.covers-ordinary-cdc-def
      proof (intro exI[where x = WCh2] conjI allI impI)
        show wit-cresponsible-chunk () k = Some WCh2
          using wm-resp-1 k1 by simp
      next
        fix i assume i-lt: i < length (wit-csrc-history-of ())

```

```

    and chunk-lt: src-lt (wit-cchunk-read-coordinate () WCh2)
                        (hist-coord (wit-csrc-history-of () ! i))
    and to-f: src-le (hist-coord (wit-csrc-history-of () ! i))
                    (wit-cfrontier ())
    and key-eq: key-of (hist-event (wit-csrc-history-of () ! i)) = k
  have False
  using i-lt chunk-lt
        wm-src-history wm-rcoord-2 length-H-w
        H-w-nth-0 H-w-nth-1
        not-src-lt-c2-w-c1-w not-src-lt-c2-w-c2-w
  by (cases i = 0; cases i = 1; auto simp: numeral-2-eq-2)
  thus wit-csrc-history-of () ! i ∈ set (wit-ccdc-events-of ()) ..
qed
qed
qed
next
— WF2 faithfulness.
show ∀ p ∈ set (wit-ccdc-events-of ()).
    key-of (hist-event p) ∈ wit-cscope ()
    ∧ src-le (hist-coord p) (wit-cfrontier ())
    → p ∈ set (wit-csrc-history-of ())
  using wm-cdc wm-src-history set-H-w by auto
next
— WF2 order-preserving sublist.
show ∃ ι :: nat ⇒ nat. strict-mono ι
  ∧ (let in-slice = (λp. key-of (hist-event p) ∈ wit-cscope ()
    ∧ src-le (hist-coord p) (wit-cfrontier ())))
    cdc-in-slice = filter in-slice (wit-ccdc-events-of ());
    hist-in-slice = filter in-slice (wit-csrc-history-of ());
  in ∀ i. i < length cdc-in-slice →
    ι i < length hist-in-slice
    ∧ cdc-in-slice ! i = hist-in-slice ! ι i)
proof (intro exI[where x=λn :: nat. n] conjI)
  show strict-mono (λn :: nat. n) by (simp add: strict-mono-def)
next
  have eq: wit-ccdc-events-of () = wit-csrc-history-of ()
  using wm-cdc wm-src-history H-w-def by simp
  show let in-slice = (λp. key-of (hist-event p) ∈ wit-cscope ()
    ∧ src-le (hist-coord p) (wit-cfrontier ())))
    cdc-in-slice = filter in-slice (wit-ccdc-events-of ());
    hist-in-slice = filter in-slice (wit-csrc-history-of ());
  in ∀ i. i < length cdc-in-slice →
    (λn :: nat. n) i < length hist-in-slice
    ∧ cdc-in-slice ! i = hist-in-slice ! (λn :: nat. n) i
  using eq by (simp add: Let-def)
qed
next
— WF2 post-read same-key/same-coordinate multiplicity.
show ∀ k ∈ wit-cscope (). ∀ ch c.

```

```

wit-cresponsible-chunk () k = Some ch
 $\wedge$  src-lt (wit-cchunk-read-coordinate () ch) c
 $\wedge$  src-le c (wit-cfrontier ())
 $\longrightarrow$ 
  mset (source-key-coord-slice k c (wit-ccdc-events-of ()))
  = mset (source-key-coord-slice k c (wit-csrc-history-of ()))
proof (intro ballI allI impI)
  fix k :: nat and ch :: wit-cchunk and c :: src-coord
  have eq: wit-ccdc-events-of () = wit-csrc-history-of ()
  using wm-cdc wm-src-history H-w-def by simp
  show mset (source-key-coord-slice k c (wit-ccdc-events-of ()))
    = mset (source-key-coord-slice k c (wit-csrc-history-of ()))
  using eq by simp
qed
next
  — WF4: chunk-read evidence.
  show  $\forall ch \in \text{wit-cchunks} ()$ .
    src-le (wit-cchunk-lower-watermark () ch) (wit-cchunk-read-coordinate ()
ch)
 $\wedge$  src-le (wit-cchunk-read-coordinate () ch) (wit-cchunk-upper-watermark ()
ch)
 $\wedge$  ( $\forall k \in \text{wit-cchunk-domain} ()$  ch.  $\exists m$ . wit-cchunk-read-result () ch k = Some
m)
 $\wedge$  ( $\forall k \in \text{wit-cchunk-domain} ()$  ch.  $\forall m$ .
  wit-cchunk-read-result () ch k = Some m
 $\longleftrightarrow$  Refresh k m (wit-cchunk-read-coordinate () ch)
 $\in \text{set (wit-model.clean-prefix-of ())}$ )
proof
  fix ch :: wit-cchunk
  assume ch-in: ch  $\in$  wit-cchunks ()
  hence ch-cases: ch = WCh1  $\vee$  ch = WCh2
  using wm-chunks-eq by simp
  show src-le (wit-cchunk-lower-watermark () ch) (wit-cchunk-read-coordinate ()
ch)
 $\wedge$  src-le (wit-cchunk-read-coordinate () ch) (wit-cchunk-upper-watermark ()
ch)
 $\wedge$  ( $\forall k \in \text{wit-cchunk-domain} ()$  ch.  $\exists m$ . wit-cchunk-read-result () ch k = Some
m)
 $\wedge$  ( $\forall k \in \text{wit-cchunk-domain} ()$  ch.  $\forall m$ .
  wit-cchunk-read-result () ch k = Some m
 $\longleftrightarrow$  Refresh k m (wit-cchunk-read-coordinate () ch)
 $\in \text{set (wit-model.clean-prefix-of ())}$ )
proof (cases ch = WCh1)
  case True
  show ?thesis
  using True wm-lwm-1 wm-uwm-1 wm-rcoord-1 src-le-refl
    wm-dom-1 wm-crr-1-0 wm-clean-prefix
  by auto
next

```

```

    case False
  hence ch = WCh2 using ch-cases by simp
  show ?thesis
    using ⟨ch = WCh2⟩ wm-lwm-2 wm-uwm-2 wm-rcoord-2 src-le-refl
           wm-dom-2 wm-crr-2-1 wm-clean-prefix
           c0-neq-c2-w c1-w-neq-c2-w
    by auto
  qed
qed
next
— WF5: chunk-read coordinate at-or-before frontier.
show ∀ ch ∈ wit-cchunks ().
    src-le (wit-cchunk-read-coordinate () ch) (wit-cfrontier ())
  using wm-chunks-eq wm-rcoord-1 wm-rcoord-2 wm-frontier
         src-le-c0-c2-w src-le-refl
  by auto
next
— WF6 main: refresh correctness against Src.
show ∀ ch ∈ wit-cchunks (). ∀ k ∈ wit-cchunk-domain () ch. ∀ m.
    wit-cchunk-read-result () ch k = Some m
    → m = Src b0-w H-w (wit-cchunk-read-coordinate () ch) k
proof
  fix ch :: wit-cchunk assume ch-in: ch ∈ wit-cchunks ()
  hence ch-cases: ch = WCh1 ∨ ch = WCh2
  using wm-chunks-eq by simp
  show ∀ k ∈ wit-cchunk-domain () ch. ∀ m.
    wit-cchunk-read-result () ch k = Some m
    → m = Src b0-w H-w (wit-cchunk-read-coordinate () ch) k
  proof (cases ch = WCh1)
    case True
    show ?thesis
      using True wm-dom-1 wm-crr-1-0
             wm-rcoord-1 Src-b0-w-H-w-c0-0
      by auto
    next
    case False
    hence ch = WCh2 using ch-cases by simp
    show ?thesis
      using ⟨ch = WCh2⟩ wm-dom-2 wm-crr-2-1
             wm-rcoord-2 Src-b0-w-H-w-c2-w-1
      by auto
    qed
  qed
next
— WF6 row-absence.
show ∀ ch ∈ wit-cchunks (). ∀ k ∈ wit-cchunk-domain () ch.
    wit-cchunk-read-result () ch k = Some None
    → wit-model.row-absence-meaningful b0-w H-w () ch k
proof (intro ballI impI)

```

```

fix ch :: wit-cchunk and k :: nat
assume ch-in: ch ∈ wit-cchunks ()
  and k-in: k ∈ wit-cchunk-domain () ch
  and crr: wit-cchunk-read-result () ch k = Some None
from ch-in wm-chunks-eq
  have ch-cases: ch = WCh1 ∨ ch = WCh2 by simp
show wit-model.row-absence-meaningful b0-w H-w () ch k
proof (cases ch = WCh1)
  case True
  with k-in wm-dom-1 have k0: k = 0 by simp
  show ?thesis
    unfolding wit-model.row-absence-meaningful-def
    using True k0 wm-dom-1 wm-rcoord-1 Src-b0-w-H-w-c0-0
    by simp
next
  case False
  with ch-cases have ch-2: ch = WCh2 by simp
  with k-in wm-dom-2 have k1: k = 1 by simp
  show ?thesis
    unfolding wit-model.row-absence-meaningful-def
    using ch-2 k1 wm-dom-2 wm-rcoord-2 Src-b0-w-H-w-c2-w-1
    by simp
qed
qed
next
— WF7 forward (clean-prefix CDC coherence).
show ∀ p ∈ set (wit-ccdc-events-of ())
  key-of (hist-event p) ∈ wit-cscope ()
  ∧ src-le (hist-coord p) (wit-cfrontier ())
  → Cdc (hist-coord p) (hist-event p)
  ∈ set (wit-model.clean-prefix-of ())
proof (intro ballI impI)
  fix p :: src-coord × (nat, nat) source-event
  assume p-in: p ∈ set (wit-ccdc-events-of ())
  hence p-cases: p = (c1-w, Insert 0 42) ∨ p = (c2-w, Delete 1)
  using wm-cdc by simp
  show Cdc (hist-coord p) (hist-event p)
    ∈ set (wit-model.clean-prefix-of ())
  using p-cases wm-clean-prefix by fastforce
qed
next
— WF7 reverse (clean-prefix CDC coherence).
show ∀ e c. Cdc c e ∈ set (wit-model.clean-prefix-of ())
  ∧ key-of e ∈ wit-cscope ()
  ∧ src-le c (wit-cfrontier ())
  → (c, e) ∈ set (wit-ccdc-events-of ())
proof (intro allI impI)
  fix e :: (nat, nat) source-event and c :: src-coord
  assume Cdc c e ∈ set (wit-model.clean-prefix-of ())

```

```

      ∧ key-of e ∈ wit-cscope ()
      ∧ src-le c (wit-cfrontier ())
  hence cdc-in: Cdc c e ∈ set (wit-model.clean-prefix-of ()) by simp
  have list-eq: Cdc c e ∈ set [Refresh (0::nat) (None::nat option) c0,
                                Cdc c1-w (Insert 0 42),
                                Cdc c2-w (Delete 1),
                                Refresh 1 None c2-w]
    using cdc-in wm-clean-prefix by simp
  hence (c = c1-w ∧ e = Insert 0 42)
        ∨ (c = c2-w ∧ e = Delete 1)
    by auto
  thus (c, e) ∈ set (wit-ccdc-events-of ())
    using wm-cdc by auto
qed
qed

```

30.5 Constructed positive witness

The Layer 1 wellformed-run non-vacuity statement, established over the constructed model with no run/chunk axiom: a wellformed run exists, and the eight side conjuncts pin down why the witness is minimum-viable rather than degenerate — non-empty base state and scope, at least two chunks, a chunk read at the base coordinate, a CDC event strictly past some chunk's read coordinate, a delete or an absence *Refresh*, and a replay that is non-empty and differs from the base state on the scope.

theorem *wellformed-run-positive-witness-constructed:*

```

  ∃ (b0 :: nat → nat) (R :: unit) (H :: (nat, nat) src-history).
    wit-model.wellformed-dblog-run b0 R H
  ∧ b0 ≠ Map.empty
  ∧ wit-cscope R ≠ {}
  ∧ 2 ≤ card (wit-cchunks R)
  ∧ (∃ ch ∈ wit-cchunks R. wit-cchunk-read-coordinate R ch = c0)
  ∧ (∃ ch ∈ wit-cchunks R. ∃ c e.
      Cdc c e ∈ set (wit-model.clean-prefix-of R)
      ∧ src-le (wit-cchunk-read-coordinate R ch) c)
  ∧ ( (∃ c k. Cdc c (Delete k) ∈ set (wit-model.clean-prefix-of R))
      ∨ (∃ k c. Refresh k None c ∈ set (wit-model.clean-prefix-of R)) )
  ∧ restrict (Apply (wit-model.clean-prefix-of R)) (wit-cscope R)
    ≠ Map.empty
  ∧ restrict (Apply (wit-model.clean-prefix-of R)) (wit-cscope R)
    ≠ restrict b0 (wit-cscope R)
proof (intro exI[where x = b0-w] exI[where x = ()] exI[where x = H-w] conjI)
  show wit-model.wellformed-dblog-run b0-w () H-w
    by (rule wit-model-wellformed)
next
  show b0-w ≠ Map.empty
  proof —
    have b0-w 7 ≠ Map.empty 7 by (simp add: b0-w-def)

```

```

    thus ?thesis by metis
qed
next
  show wit-cscope () ≠ {} using wm-scope by simp
next
  show 2 ≤ card (wit-cchunks ()) using wm-chunks wm-distinct by simp
next
  show ∃ ch ∈ wit-cchunks (). wit-cchunk-read-coordinate () ch = c0
    using wm-chunks wm-rcoord-1 by blast
next
  show ∃ ch ∈ wit-cchunks (). ∃ c e.
    Cdc c e ∈ set (wit-model.clean-prefix-of ())
    ∧ src-lt (wit-cchunk-read-coordinate () ch) c
    using wm-clean-prefix wm-rcoord-1 c0-lt-c1-w wm-chunks by fastforce
next
  show (∃ c k. Cdc c (Delete k) ∈ set (wit-model.clean-prefix-of ()))
    ∨ (∃ k c. Refresh k None c ∈ set (wit-model.clean-prefix-of ()))
    using wm-clean-prefix by fastforce
next
  show restrict (Apply (wit-model.clean-prefix-of ())) (wit-cscope ())
    ≠ Map.empty
proof
  assume H: restrict (Apply (wit-model.clean-prefix-of ())) (wit-cscope ())
    = Map.empty
  hence restrict (Apply (wit-model.clean-prefix-of ())) (wit-cscope ()) 0
    = Map.empty 0 by simp
  hence (if (0::nat) ∈ wit-cscope ()
    then Apply (wit-model.clean-prefix-of ()) 0 else None) = None
    unfolding restrict-def by simp
  moreover have (0::nat) ∈ wit-cscope () using wm-scope by simp
  ultimately have Apply (wit-model.clean-prefix-of ()) 0 = None by simp
  thus False using wm-apply by simp
qed
next
  show restrict (Apply (wit-model.clean-prefix-of ())) (wit-cscope ())
    ≠ restrict b0-w (wit-cscope ())
proof
  assume H: restrict (Apply (wit-model.clean-prefix-of ())) (wit-cscope ())
    = restrict b0-w (wit-cscope ())
  hence restrict (Apply (wit-model.clean-prefix-of ())) (wit-cscope ()) 0
    = restrict b0-w (wit-cscope ()) 0 by simp
  moreover have (0::nat) ∈ wit-cscope () using wm-scope by simp
  ultimately have Apply (wit-model.clean-prefix-of ()) 0 = b0-w 0
    unfolding restrict-def by simp
  thus False using wm-apply unfolding b0-w-def by simp
qed
qed

```

A model exists for the *dblog-run-substrate* interface that realizes the minimum-viable wellformed-run witness, so the Layer 1 positive-witness re-

sult holds by construction — consistency-by-construction, resting on no axiom.

end

```
theory Layer01-Witness-Topics-Model
  imports Layer01-Witness-Topics-Core
begin
```

31 Constructed models for the Topic 1 and Topic 2 witnesses

Each section exhibits the explicit model of one topic's witness data: a one-point run carrier (*unit*), a small chunk datatype, and concrete accessor functions returning the intended witness values. *Layer01-Witness-Topics-Inst* interprets the run-substrate locale (*dblog-run-substrate*) at these models, so the per-topic positive-witness theorems (*topic1-positive-witness-constructed*, *topic2-positive-witness-constructed*) are established constructively, with no axioms. The carrier-independent witness data (*b0-t1* / *H-t1*, *b0-t2* / *H-t2*, and their computation lemmas) comes from the imported *Layer01-Witness-Topics-Core*.

31.1 Topic 1: single chunk, one CDC event

datatype *t1-chunk* = *T1Ch*

definition *t1-cscope* :: *unit* $\Rightarrow$ *nat set* **where**
t1-cscope - = {1}

definition *t1-cfrontier* :: *unit* $\Rightarrow$ *frontier* **where**
t1-cfrontier - = *ct1*

definition *t1-cchunks* :: *unit* $\Rightarrow$ *t1-chunk set* **where**
t1-cchunks - = {*T1Ch*}

definition *t1-cchunks-list* :: *unit* $\Rightarrow$ *t1-chunk list* **where**
t1-cchunks-list - = [*T1Ch*]

definition *t1-chunk-domain* :: *unit* $\Rightarrow$ *t1-chunk* $\Rightarrow$ *nat set* **where**
t1-chunk-domain - - = {1}

definition *t1-cresponsible-chunk* :: *unit* $\Rightarrow$ *nat* $\Rightarrow$ *t1-chunk option* **where**
t1-cresponsible-chunk - *k* = (if *k* = 1 then *Some T1Ch* else *None*)

definition *t1-chunk-read-coordinate* :: *unit* $\Rightarrow$ *t1-chunk* $\Rightarrow$ *src-coord* **where**
t1-chunk-read-coordinate - - = *c0*

definition $t1\text{-cchunk-lower-watermark} :: \text{unit} \Rightarrow t1\text{-cchunk} \Rightarrow \text{src-coord}$ **where**
 $t1\text{-cchunk-lower-watermark} \text{ - } = c0$

definition $t1\text{-cchunk-upper-watermark} :: \text{unit} \Rightarrow t1\text{-cchunk} \Rightarrow \text{src-coord}$ **where**
 $t1\text{-cchunk-upper-watermark} \text{ - } = c0$

definition $t1\text{-csrc-history-of} :: \text{unit} \Rightarrow (\text{nat}, \text{nat}) \text{ src-history}$ **where**
 $t1\text{-csrc-history-of} \text{ - } = H\text{-}t1$

definition $t1\text{-ccdc-events-of} :: \text{unit} \Rightarrow (\text{nat}, \text{nat}) \text{ src-history}$ **where**
 $t1\text{-ccdc-events-of} \text{ - } = H\text{-}t1$

definition $t1\text{-cchunk-read-result} :: \text{unit} \Rightarrow t1\text{-cchunk} \Rightarrow \text{nat} \Rightarrow \text{nat option option}$
where
 $t1\text{-cchunk-read-result} \text{ - } ch \ k = (\text{if } ch = T1Ch \wedge k = 1 \text{ then } \text{Some } (\text{Some } 5) \text{ else } \text{None})$

theorem $t1\text{-model-accessor-values}$:

$t1\text{-cscope} \text{ () } = \{1\}$
 $\wedge t1\text{-cfrontier} \text{ () } = ct1$
 $\wedge t1\text{-cchunks} \text{ () } = \{T1Ch\}$
 $\wedge t1\text{-cchunk-domain} \text{ () } T1Ch = \{1\}$
 $\wedge t1\text{-cresponsible-chunk} \text{ () } 1 = \text{Some } T1Ch$
 $\wedge (\forall k. k \neq 1 \longrightarrow t1\text{-cresponsible-chunk} \text{ () } k = \text{None})$
 $\wedge t1\text{-cchunk-read-coordinate} \text{ () } T1Ch = c0$
 $\wedge t1\text{-cchunk-lower-watermark} \text{ () } T1Ch = c0$
 $\wedge t1\text{-cchunk-upper-watermark} \text{ () } T1Ch = c0$
 $\wedge t1\text{-csrc-history-of} \text{ () } = H\text{-}t1$
 $\wedge t1\text{-cchunk-read-result} \text{ () } T1Ch \ 1 = \text{Some } (\text{Some } 5)$
 $\wedge t1\text{-ccdc-events-of} \text{ () } = H\text{-}t1$
 $\wedge t1\text{-cchunks-list} \text{ () } = [T1Ch]$
 $\wedge \text{set } (t1\text{-cchunks-list} \text{ ()}) = t1\text{-cchunks} \text{ ()}$
 $\wedge \text{distinct } (t1\text{-cchunks-list} \text{ ()})$
by ($\text{auto simp: } t1\text{-cscope-def } t1\text{-cfrontier-def } t1\text{-cchunks-def}$
 $t1\text{-cchunks-list-def } t1\text{-cchunk-domain-def}$
 $t1\text{-cresponsible-chunk-def } t1\text{-cchunk-read-coordinate-def}$
 $t1\text{-cchunk-lower-watermark-def } t1\text{-cchunk-upper-watermark-def}$
 $t1\text{-csrc-history-of-def } t1\text{-ccdc-events-of-def}$
 $t1\text{-cchunk-read-result-def}$)

31.2 Topic 2: two chunks partitioning the scope, empty history

datatype $t2\text{-cchunk} = T2ChA \mid T2ChB$

definition $t2\text{-cscope} :: \text{unit} \Rightarrow \text{nat set}$ **where**
 $t2\text{-cscope} \text{ - } = \{10, 20\}$

definition $t2\text{-cfrontier} :: \text{unit} \Rightarrow \text{frontier}$ **where**
 $t2\text{-cfrontier} - = c0$

definition $t2\text{-cchunks} :: \text{unit} \Rightarrow t2\text{-cchunk set}$ **where**
 $t2\text{-cchunks} - = \{T2ChA, T2ChB\}$

definition $t2\text{-cchunks-list} :: \text{unit} \Rightarrow t2\text{-cchunk list}$ **where**
 $t2\text{-cchunks-list} - = [T2ChA, T2ChB]$

definition $t2\text{-cchunk-domain} :: \text{unit} \Rightarrow t2\text{-cchunk} \Rightarrow \text{nat set}$ **where**
 $t2\text{-cchunk-domain} - ch = (\text{case } ch \text{ of } T2ChA \Rightarrow \{10\} \mid T2ChB \Rightarrow \{20\})$

definition $t2\text{-cresponsible-chunk} :: \text{unit} \Rightarrow \text{nat} \Rightarrow t2\text{-cchunk option}$ **where**
 $t2\text{-cresponsible-chunk} - k =$
 $(\text{if } k = 10 \text{ then } \text{Some } T2ChA \text{ else if } k = 20 \text{ then } \text{Some } T2ChB \text{ else } \text{None})$

definition $t2\text{-cchunk-read-coordinate} :: \text{unit} \Rightarrow t2\text{-cchunk} \Rightarrow \text{src-coord}$ **where**
 $t2\text{-cchunk-read-coordinate} - - = c0$

definition $t2\text{-cchunk-lower-watermark} :: \text{unit} \Rightarrow t2\text{-cchunk} \Rightarrow \text{src-coord}$ **where**
 $t2\text{-cchunk-lower-watermark} - - = c0$

definition $t2\text{-cchunk-upper-watermark} :: \text{unit} \Rightarrow t2\text{-cchunk} \Rightarrow \text{src-coord}$ **where**
 $t2\text{-cchunk-upper-watermark} - - = c0$

definition $t2\text{-csrc-history-of} :: \text{unit} \Rightarrow (\text{nat}, \text{nat}) \text{ src-history}$ **where**
 $t2\text{-csrc-history-of} - = H\text{-}t2$

definition $t2\text{-ccdc-events-of} :: \text{unit} \Rightarrow (\text{nat}, \text{nat}) \text{ src-history}$ **where**
 $t2\text{-ccdc-events-of} - = H\text{-}t2$

definition $t2\text{-cchunk-read-result} :: \text{unit} \Rightarrow t2\text{-cchunk} \Rightarrow \text{nat} \Rightarrow \text{nat option option}$
where

$t2\text{-cchunk-read-result} - ch \ k =$
 $(\text{if } ch = T2ChA \wedge k = 10 \text{ then } \text{Some } (\text{Some } 100)$
 $\text{else if } ch = T2ChB \wedge k = 20 \text{ then } \text{Some } (\text{Some } 200)$
 $\text{else } \text{None})$

theorem $t2\text{-model-accessor-values}$:

$T2ChA \neq T2ChB$
 $\wedge t2\text{-cscope } () = \{10, 20\}$
 $\wedge t2\text{-cfrontier } () = c0$
 $\wedge t2\text{-cchunks } () = \{T2ChA, T2ChB\}$
 $\wedge t2\text{-cchunk-domain } () \ T2ChA = \{10\}$
 $\wedge t2\text{-cchunk-domain } () \ T2ChB = \{20\}$
 $\wedge t2\text{-cresponsible-chunk } () \ 10 = \text{Some } T2ChA$
 $\wedge t2\text{-cresponsible-chunk } () \ 20 = \text{Some } T2ChB$
 $\wedge (\forall k. k \neq 10 \longrightarrow k \neq 20 \longrightarrow t2\text{-cresponsible-chunk } () \ k = \text{None})$
 $\wedge t2\text{-cchunk-read-coordinate } () \ T2ChA = c0$

```

 $\wedge$  t2-cchunk-read-coordinate () T2ChB = c0
 $\wedge$  t2-cchunk-lower-watermark () T2ChA = c0
 $\wedge$  t2-cchunk-upper-watermark () T2ChA = c0
 $\wedge$  t2-cchunk-lower-watermark () T2ChB = c0
 $\wedge$  t2-cchunk-upper-watermark () T2ChB = c0
 $\wedge$  t2-csrc-history-of () = H-t2
 $\wedge$  t2-cchunk-read-result () T2ChA 10 = Some (Some 100)
 $\wedge$  t2-cchunk-read-result () T2ChB 20 = Some (Some 200)
 $\wedge$  t2-ccdc-events-of () = H-t2
 $\wedge$  t2-cchunks-list () = [T2ChA, T2ChB]
 $\wedge$  set (t2-cchunks-list ()) = t2-cchunks ()
 $\wedge$  distinct (t2-cchunks-list ())
by (auto simp: t2-cscope-def t2-cfrontier-def t2-cchunks-def
      t2-cchunks-list-def t2-cchunk-domain-def
      t2-cresponsible-chunk-def t2-cchunk-read-coordinate-def
      t2-cchunk-lower-watermark-def t2-cchunk-upper-watermark-def
      t2-csrc-history-of-def t2-ccdc-events-of-def
      t2-cchunk-read-result-def)

```

Both topic configurations are realized by explicit constructions. *Layer01-Witness-Topics-Inst* interprets the run-substrate locale at each model and establishes the corresponding positive-witness theorem over it.

end

```

theory Layer01-Witness-Topics-Inst
  imports Layer01-Witness-Topics-Model DBLog-Run-Substrate-Layer2
begin

```

32 Constructed interpretations for the Topic 1 and Topic 2 witnesses

Constructs the Topic 1 (clean-prefix-carrier) and Topic 2 (multi-chunk partition) positive witnesses as explicit interpretations of the Layer 1 locale *dblog-run-substrate* at the concrete models of *DBLog-Virtual-Cuts.Layer01-Witness-Topics-Model* (the *unit* run carriers, the *t1-cchunk* / *t2-cchunk* chunk datatypes, and the definitional accessors), establishing both witness facts with no axioms.

Method (the *Layer01-Witnesses-Inst* pattern, applied to both topics). For each topic: instantiate *dblog-run-substrate* at the model carrier (the two inherited *chunks-list* obligations discharge by *simp*, no axiom); read off the model's definitional accessor values pointwise as the *t1m-...* / *t2m-...* lemmas (the conjuncts of *t1-model-accessor-values* / *t2-model-accessor-values*); and prove the determinate clean-prefix shape, the wellformedness, and the topic's positive-witness statement from those rewrite facts plus the carrier-independent topic data of

DBLog-Virtual-Cuts.Layer01-Witness-Topics-Core (the base states, the source histories, and the *Src / latest-src-event* computation lemmas). As in *Layer01-Witnesses-Inst* the concrete-*nat* wellformedness is proved at the interpretation in the global context (the locale type parameter '*k*' is rigid inside the locale context), not as an in-locale lemma.

The Topic 1 source coordinate *ct1* is a *coord-of-nat* definition from *Dual-Write-Layer0.Source-Coordinates*, with its order facts proved there; Topic 2 uses only the base coordinate *c0*.

33 Topic 1: constructed interpretation

Instantiate *dblog-run-substrate* at the Topic 1 model carrier (*unit* run, the single-constructor *t1-cchunk*); the two *chunks-list* obligations reduce to *set* $[T1Ch] = \{T1Ch\}$ and *distinct* $[T1Ch]$, discharged by *simp* — no axiom.

global-interpretation *t1-model: dblog-run-substrate*

```

t1-cscope
t1-cfrontier
t1-cchunks
t1-csrc-history-of
t1-cchunk-read-result
t1-ccdc-events-of
t1-cchunks-list
t1-cchunk-domain
t1-cresponsible-chunk
t1-cchunk-lower-watermark
t1-cchunk-upper-watermark
t1-cchunk-read-coordinate
by unfold-locales (simp-all add: t1-cchunks-def t1-cchunks-list-def)

```

33.1 Topic 1 model accessor-value lemmas

```

lemma t1m-scope: t1-cscope () = {1} by (simp add: t1-cscope-def)
lemma t1m-frontier: t1-cfrontier () = ct1 by (simp add: t1-cfrontier-def)
lemma t1m-chunks: t1-cchunks () = {T1Ch} by (simp add: t1-cchunks-def)
lemma t1m-dom: t1-cchunk-domain () T1Ch = {1}
  by (simp add: t1-cchunk-domain-def)
lemma t1m-resp-1: t1-cresponsible-chunk () 1 = Some T1Ch
  by (simp add: t1-cresponsible-chunk-def)
lemma t1m-resp-else:  $k \neq 1 \implies \textit{t1-cresponsible-chunk} () k = \textit{None}$ 
  by (simp add: t1-cresponsible-chunk-def)
lemma t1m-rcoord: t1-cchunk-read-coordinate () T1Ch = c0
  by (simp add: t1-cchunk-read-coordinate-def)
lemma t1m-lwm: t1-cchunk-lower-watermark () T1Ch = c0
  by (simp add: t1-cchunk-lower-watermark-def)
lemma t1m-uwm: t1-cchunk-upper-watermark () T1Ch = c0
  by (simp add: t1-cchunk-upper-watermark-def)
lemma t1m-src-history: t1-csrc-history-of () = H-t1

```

```

  by (simp add: t1-csrc-history-of-def)
lemma t1m-crr-1: t1-cchunk-read-result () T1Ch 1 = Some (Some 5)
  by (simp add: t1-cchunk-read-result-def)
lemma t1m-cdc: t1-ccdc-events-of () = [(ct1, Update 1 7)]
  by (simp add: t1-ccdc-events-of-def H-t1-def)
lemma t1m-chunks-list: t1-cchunks-list () = [T1Ch]
  by (simp add: t1-cchunks-list-def)

lemma t1m-chunks-eq:
  (ch ∈ t1-cchunks ()) ⟷ (ch = T1Ch)
  using t1m-chunks by simp

```

33.2 Topic 1 determinate clean prefix and Apply over the model

```

lemma t1m-clean-prefix:
  t1-model.clean-prefix-of ()
    = [ Refresh (1::nat) (Some (5::nat)) c0,
        Cdc ct1 (Update 1 7) ]
proof -
  have d1: sorted-list-of-set {Suc 0} = [Suc 0] by simp
  show ?thesis
    unfolding t1-model.clean-prefix-of-def canonical-clean-prefix-def
              cdc-event-replays-def chunk-read-refreshes-def
    by (simp add: t1m-chunks-list t1m-dom t1m-rcoord
                  t1m-crr-1[unfolded One-nat-def]
                  t1m-cdc t1m-scope t1m-frontier d1 One-nat-def
                  List.map-filter-simps sort-key-def
                  c0-le-ct1[unfolded src-le-eq-less-eq]
                  not-src-le-ct1-c0[unfolded src-le-eq-less-eq])
qed

```

```

lemma t1m-apply:
  Apply (t1-model.clean-prefix-of ())
    = (λk. if k = (1::nat) then Some (7::nat) else None)
proof -
  have Apply (t1-model.clean-prefix-of ())
    = foldl apply-step (λ-. None)
      [ Refresh (1::nat) (Some (5::nat)) c0,
        Cdc ct1 (Update 1 7) ]
    unfolding Apply-def t1m-clean-prefix by (rule refl)
  also have ... = (λk. if k = (1::nat) then Some (7::nat) else None)
    by (auto simp: fun-eq-iff fun-upd-def)
  finally show ?thesis .
qed

```

33.3 Topic 1 constructed wellformed-run fact

theorem *t1-model-wellformed*:

```

    t1-model.wellformed-dblog-run b0-t1 () H-t1
  unfolding t1-model.wellformed-dblog-run-def
proof (intro conjI)
  show t1-csrc-history-of () = H-t1 by (rule t1m-src-history)
next
  show wellformed-src-history H-t1 by (rule wf-h-t1)
next
  show  $\forall k \in t1-cscope (). \exists !ch. ch \in t1-cchunks () \wedge k \in t1-cchunk-domain () ch$ 
  proof
    fix k :: nat assume k  $\in$  t1-cscope ()
    hence k1: k = 1 using t1m-scope by simp
    show  $\exists !ch. ch \in t1-cchunks () \wedge k \in t1-cchunk-domain () ch$ 
    proof
      show T1Ch  $\in$  t1-cchunks ()  $\wedge$  k  $\in$  t1-cchunk-domain () T1Ch
      using t1m-chunks t1m-dom k1 by simp
    next
      fix ch assume ch  $\in$  t1-cchunks ()  $\wedge$  k  $\in$  t1-cchunk-domain () ch
      thus ch = T1Ch using t1m-chunks-eq by blast
    qed
  qed
next
  show  $\forall ch1 \in t1-cchunks (). \forall ch2 \in t1-cchunks ().$ 
     $ch1 \neq ch2 \longrightarrow t1-cchunk-domain () ch1 \cap t1-cchunk-domain () ch2 = \{\}$ 
    using t1m-chunks-eq by auto
next
  show t1-model.canonical-chunk-ownership-domain () = t1-cscope ()
  unfolding t1-model.canonical-chunk-ownership-domain-def
  using t1m-chunks t1m-dom t1m-scope by auto
next
  show  $\forall k \in t1-cscope (). \forall ch.$ 
     $t1-cresponsible-chunk () k = Some ch$ 
     $\longleftrightarrow ch \in t1-cchunks () \wedge k \in t1-cchunk-domain () ch$ 
  proof (intro ballI allI)
    fix k :: nat and ch :: t1-cchunk
    assume k  $\in$  t1-cscope ()
    hence k1: k = 1 using t1m-scope by simp
    show t1-cresponsible-chunk () k = Some ch
     $\longleftrightarrow ch \in t1-cchunks () \wedge k \in t1-cchunk-domain () ch$ 
    proof
      assume t1-cresponsible-chunk () k = Some ch
      with t1m-resp-1 k1 have ch = T1Ch by simp
      thus ch  $\in$  t1-cchunks ()  $\wedge$  k  $\in$  t1-cchunk-domain () ch
      using t1m-chunks t1m-dom k1 by simp
    next
      assume A: ch  $\in$  t1-cchunks ()  $\wedge$  k  $\in$  t1-cchunk-domain () ch
      have ch = T1Ch using A t1m-chunks-eq by blast
      with t1m-resp-1 k1 show t1-cresponsible-chunk () k = Some ch by simp
    qed
  qed
qed

```

```

next
  show  $\forall k. k \notin t1-cscope () \longrightarrow t1-cresponsible-chunk () k = None$ 
  using t1m-scope t1m-resp-else by auto
next
  show finite (t1-cscope ()) using t1m-scope by simp
next
  show  $\forall ch \in t1-cchunks (). finite (t1-cchunk-domain () ch)$ 
  using t1m-chunks-eq t1m-dom by auto
next
  show  $\forall ch \in t1-cchunks (). t1-cchunk-domain () ch \neq \{\}$ 
  using t1m-chunks-eq t1m-dom by auto
next
  show  $\forall k \in t1-cscope (). t1-model.covers-ordinary-cdc () k (t1-cfrontier ())$ 
  proof
    fix k :: nat assume k  $\in t1-cscope ()$ 
    hence k1: k = 1 using t1m-scope by simp
    show t1-model.covers-ordinary-cdc () k (t1-cfrontier ())
      unfolding t1-model.covers-ordinary-cdc-def
    proof (intro exI [where x = T1Ch] conjI allI impI)
      show t1-cresponsible-chunk () k = Some T1Ch
        using t1m-resp-1 k1 by simp
    next
      fix i assume i-lt: i < length (t1-csrc-history-of ())
      and chunk-lt: src-lt (t1-cchunk-read-coordinate () T1Ch)
        (hist-coord (t1-csrc-history-of () ! i))
      and to-f: src-le (hist-coord (t1-csrc-history-of () ! i))
        (t1-cfrontier ())
      and key-eq: key-of (hist-event (t1-csrc-history-of () ! i)) = k
      have i0: i = 0
        using i-lt t1m-src-history length-H-t1 by simp
      show t1-csrc-history-of () ! i  $\in set (t1-ccdc-events-of ())$ 
        using i0 t1m-src-history t1m-cdc H-t1-nth-0
        by (simp add: H-t1-def)
    qed
  qed
next
  show  $\forall p \in set (t1-ccdc-events-of ().$ 
    key-of (hist-event p)  $\in t1-cscope ()$ 
     $\wedge src-le (hist-coord p) (t1-cfrontier ())$ 
     $\longrightarrow p \in set (t1-csrc-history-of ())$ 
  using t1m-cdc t1m-src-history set-H-t1 H-t1-def by auto
next
  show  $\exists \iota :: nat \Rightarrow nat. strict-mono \iota$ 
     $\wedge (let in-slice = (\lambda p. key-of (hist-event p) \in t1-cscope ()$ 
       $\wedge src-le (hist-coord p) (t1-cfrontier ())));$ 
      cdc-in-slice = filter in-slice (t1-ccdc-events-of ());
      hist-in-slice = filter in-slice (t1-csrc-history-of ())
    in  $\forall i. i < length\ cdc-in-slice \longrightarrow$ 
       $\iota\ i < length\ hist-in-slice$ 

```

```

       $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! i$ 
proof (intro exI[where  $x = \lambda n :: \text{nat}. n$ ] conjI)
  show strict-mono ( $\lambda n :: \text{nat}. n$ ) by (simp add: strict-mono-def)
next
  have eq:  $t1\text{-ccdc-events-of } () = t1\text{-csrc-history-of } ()$ 
  using  $t1m\text{-cdc } t1m\text{-src-history } H\text{-}t1\text{-def}$  by simp
  show let in-slice = ( $\lambda p. \text{key-of } (\text{hist-event } p) \in t1\text{-cscope } ()$ 
     $\wedge \text{src-le } (\text{hist-coord } p) (t1\text{-cfrontier } ())$ );
     $\text{cdc-in-slice} = \text{filter in-slice } (t1\text{-ccdc-events-of } ());$ 
     $\text{hist-in-slice} = \text{filter in-slice } (t1\text{-csrc-history-of } ());$ 
    in  $\forall i. i < \text{length cdc-in-slice} \longrightarrow$ 
      ( $\lambda n :: \text{nat}. n$ )  $i < \text{length hist-in-slice}$ 
       $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! (\lambda n :: \text{nat}. n) i$ 
  using eq by (simp add: Let-def)
qed
next
show  $\forall k \in t1\text{-cscope } (). \forall ch \ c.$ 
   $t1\text{-cresponsible-chunk } () \ k = \text{Some } ch$ 
   $\wedge \text{src-lt } (t1\text{-cchunk-read-coordinate } () \ ch) \ c$ 
   $\wedge \text{src-le } c (t1\text{-cfrontier } ())$ 
   $\longrightarrow$ 
   $\text{mset } (\text{source-key-coord-slice } k \ c \ (t1\text{-ccdc-events-of } ()))$ 
   $= \text{mset } (\text{source-key-coord-slice } k \ c \ (t1\text{-csrc-history-of } ()))$ 
proof (intro ballI allI impI)
  fix  $k :: \text{nat}$  and  $ch :: t1\text{-cchunk}$  and  $c :: \text{src-coord}$ 
  have eq:  $t1\text{-ccdc-events-of } () = t1\text{-csrc-history-of } ()$ 
  using  $t1m\text{-cdc } t1m\text{-src-history } H\text{-}t1\text{-def}$  by simp
  show  $\text{mset } (\text{source-key-coord-slice } k \ c \ (t1\text{-ccdc-events-of } ()))$ 
     $= \text{mset } (\text{source-key-coord-slice } k \ c \ (t1\text{-csrc-history-of } ()))$ 
  using eq by simp
qed
next
show  $\forall ch \in t1\text{-cchunks } ().$ 
   $\text{src-le } (t1\text{-cchunk-lower-watermark } () \ ch) (t1\text{-cchunk-read-coordinate } () \ ch)$ 
   $\wedge \text{src-le } (t1\text{-cchunk-read-coordinate } () \ ch) (t1\text{-cchunk-upper-watermark } () \ ch)$ 
   $\wedge (\forall k \in t1\text{-cchunk-domain } () \ ch. \exists m. t1\text{-cchunk-read-result } () \ ch \ k = \text{Some } m)$ 
   $\wedge (\forall k \in t1\text{-cchunk-domain } () \ ch. \forall m.$ 
     $t1\text{-cchunk-read-result } () \ ch \ k = \text{Some } m$ 
     $\longleftrightarrow \text{Refresh } k \ m \ (t1\text{-cchunk-read-coordinate } () \ ch)$ 
     $\in \text{set } (t1\text{-model.clean-prefix-of } ()))$ 
proof
  fix  $ch :: t1\text{-cchunk}$  assume  $ch \in t1\text{-cchunks } ()$ 
  hence  $ch\text{-eq}: ch = T1Ch$  using  $t1m\text{-chunks-eq}$  by simp
  show  $\text{src-le } (t1\text{-cchunk-lower-watermark } () \ ch) (t1\text{-cchunk-read-coordinate } () \ ch)$ 
   $\wedge \text{src-le } (t1\text{-cchunk-read-coordinate } () \ ch) (t1\text{-cchunk-upper-watermark } () \ ch)$ 
   $\wedge (\forall k \in t1\text{-cchunk-domain } () \ ch. \exists m. t1\text{-cchunk-read-result } () \ ch \ k = \text{Some } m)$ 

```



```

    ∧ (∀ k ∈ t1-cchunk-domain () ch. ∀ m.
      t1-cchunk-read-result () ch k = Some m
      ↔ Refresh k m (t1-cchunk-read-coordinate () ch)
      ∈ set (t1-model.clean-prefix-of ()))
  using ch-eq t1m-lwm t1m-uwm t1m-rcoord src-le-refl
    t1m-dom t1m-crr-1 t1m-clean-prefix
  by auto
qed
next
show ∀ ch ∈ t1-cchunks ().
  src-le (t1-cchunk-read-coordinate () ch) (t1-cfrontier ())
  using t1m-chunks-eq t1m-rcoord t1m-frontier c0-le-ct1 by auto
next
show ∀ ch ∈ t1-cchunks (). ∀ k ∈ t1-cchunk-domain () ch. ∀ m.
  t1-cchunk-read-result () ch k = Some m
  → m = Src b0-t1 H-t1 (t1-cchunk-read-coordinate () ch) k
proof
  fix ch :: t1-cchunk assume ch ∈ t1-cchunks ()
  hence ch-eq: ch = T1Ch using t1m-chunks-eq by simp
  show ∀ k ∈ t1-cchunk-domain () ch. ∀ m.
    t1-cchunk-read-result () ch k = Some m
    → m = Src b0-t1 H-t1 (t1-cchunk-read-coordinate () ch) k
  using ch-eq t1m-dom t1m-crr-1 t1m-rcoord
    Src-b0-t1-H-t1-c0-1
  by auto
qed
next
show ∀ ch ∈ t1-cchunks (). ∀ k ∈ t1-cchunk-domain () ch.
  t1-cchunk-read-result () ch k = Some None
  → t1-model.row-absence-meaningful b0-t1 H-t1 () ch k
proof (intro ballI impI)
  fix ch :: t1-cchunk and k :: nat
  assume ch-in: ch ∈ t1-cchunks ()
  and k-in: k ∈ t1-cchunk-domain () ch
  and crr: t1-cchunk-read-result () ch k = Some None
  from ch-in t1m-chunks-eq have ch-eq: ch = T1Ch by simp
  with k-in t1m-dom have k1: k = 1 by simp
  from ch-eq k1 crr t1m-crr-1 have False by simp
  thus t1-model.row-absence-meaningful b0-t1 H-t1 () ch k ..
qed
next
show ∀ p ∈ set (t1-ccdc-events-of ()).
  key-of (hist-event p) ∈ t1-cscope ()
  ∧ src-le (hist-coord p) (t1-cfrontier ())
  → Cdc (hist-coord p) (hist-event p)
  ∈ set (t1-model.clean-prefix-of ())
proof (intro ballI impI)
  fix p :: src-coord × (nat, nat) source-event
  assume p-in: p ∈ set (t1-ccdc-events-of ())

```

```

    hence  $p = (ct1, \text{Update } 1 \ 7)$  using  $t1m\text{-cdc}$  by  $simp$ 
    thus  $Cdc \ (hist\text{-}coord \ p) \ (hist\text{-}event \ p)$ 
       $\in set \ (t1\text{-}model.\text{clean}\text{-}prefix\text{-}of \ ())$ 
    using  $t1m\text{-clean}\text{-}prefix$  by  $simp$ 
  qed
next
show  $\forall e \ c. \ Cdc \ c \ e \in set \ (t1\text{-}model.\text{clean}\text{-}prefix\text{-}of \ ())$ 
   $\wedge \text{key}\text{-}of \ e \in t1\text{-}cscope \ ()$ 
   $\wedge \text{src}\text{-}le \ c \ (t1\text{-}cfrontier \ ())$ 
   $\longrightarrow (c, e) \in set \ (t1\text{-}ccdc\text{-}events\text{-}of \ ())$ 
proof ( $intro \ allI \ impI$ )
  fix  $e :: (nat, nat)$   $source\text{-}event$  and  $c :: src\text{-}coord$ 
  assume  $Cdc \ c \ e \in set \ (t1\text{-}model.\text{clean}\text{-}prefix\text{-}of \ ())$ 
     $\wedge \text{key}\text{-}of \ e \in t1\text{-}cscope \ ()$ 
     $\wedge \text{src}\text{-}le \ c \ (t1\text{-}cfrontier \ ())$ 
  hence  $cdc\text{-}in: Cdc \ c \ e \in set \ (t1\text{-}model.\text{clean}\text{-}prefix\text{-}of \ ())$  by  $simp$ 
  have  $Cdc \ c \ e \in set \ [\text{Refresh} \ (1::nat) \ (\text{Some} \ (5::nat)) \ c0,$ 
     $Cdc \ ct1 \ (\text{Update} \ 1 \ 7)]$ 
    using  $cdc\text{-}in \ t1m\text{-clean}\text{-}prefix$  by  $simp$ 
  hence  $c = ct1 \wedge e = \text{Update} \ 1 \ 7$  by  $auto$ 
  thus  $(c, e) \in set \ (t1\text{-}ccdc\text{-}events\text{-}of \ ())$ 
    using  $t1m\text{-cdc}$  by  $simp$ 
  qed
qed

```

33.4 Topic 1 constructed replay-equality conclusion (non-trivial)

```

lemma  $t1\text{-}model\text{-}virtual\text{-}cut\text{-}holds$ :
   $restrict \ (\text{Apply} \ (t1\text{-}model.\text{clean}\text{-}prefix\text{-}of \ ())) \ (t1\text{-}cscope \ ())$ 
   $= restrict \ (Src \ b0\text{-}t1 \ H\text{-}t1 \ (t1\text{-}cfrontier \ ())) \ (t1\text{-}cscope \ ())$ 
proof
  fix  $k :: nat$ 
  show  $restrict \ (\text{Apply} \ (t1\text{-}model.\text{clean}\text{-}prefix\text{-}of \ ())) \ (t1\text{-}cscope \ ()) \ k$ 
     $= restrict \ (Src \ b0\text{-}t1 \ H\text{-}t1 \ (t1\text{-}cfrontier \ ())) \ (t1\text{-}cscope \ ()) \ k$ 
  proof ( $cases \ k \in t1\text{-}cscope \ ()$ )
    case  $True$ 
    hence  $k1: k = 1$  using  $t1m\text{-scope}$  by  $simp$ 
    then show  $?thesis$ 
      unfolding  $restrict\text{-}def$ 
      using  $\langle k \in t1\text{-}cscope \ () \rangle$ 
         $t1m\text{-apply} \ Src\text{-}b0\text{-}t1\text{-}H\text{-}t1\text{-}ct1\text{-}1 \ t1m\text{-frontier}$ 
      by  $simp$ 
    next
    case  $False$ 
    show  $?thesis$ 
      unfolding  $restrict\text{-}def$  using  $False$  by  $simp$ 
  qed
qed

```

lemma *t1-model-virtual-cut-non-trivial*:

restrict (Apply (t1-model.clean-prefix-of ())) (t1-cscope ())
 $\neq$ *restrict b0-t1 (t1-cscope ())*

proof

assume *H: restrict (Apply (t1-model.clean-prefix-of ())) (t1-cscope ())*
 $=$ *restrict b0-t1 (t1-cscope ())*

hence *restrict (Apply (t1-model.clean-prefix-of ())) (t1-cscope ()) 1*
 $=$ *restrict b0-t1 (t1-cscope ()) 1* **by** *simp*

moreover have $(1::nat) \in t1-cscope ()$ **using** *t1m-scope* **by** *simp*

ultimately have *Apply (t1-model.clean-prefix-of ()) 1 = b0-t1 1*

unfolding *restrict-def* **by** *simp*

thus *False*

using *t1m-apply unfolding b0-t1-def* **by** *simp*

qed

33.5 Topic 1 constructed positive witness

theorem *topic1-positive-witness-constructed*:

$\exists (b0 :: nat \rightarrow nat) (R :: unit) (H :: (nat, nat) \text{ src-history}).$

t1-model.wellformed-dblog-run b0 R H

$\wedge t1-cchunks R \neq \{\}$

$\wedge (\exists c e. Cdc c e \in set (t1-model.clean-prefix-of R))$

$\wedge restrict (Apply (t1-model.clean-prefix-of R)) (t1-cscope R)$

$= restrict (Src b0 H (t1-cfrontier R)) (t1-cscope R)$

$\wedge restrict (Apply (t1-model.clean-prefix-of R)) (t1-cscope R)$

$\neq restrict b0 (t1-cscope R)$

proof (*intro exI[where x = b0-t1] exI[where x = ()] exI[where x = H-t1] conjI*)

show *t1-model.wellformed-dblog-run b0-t1 () H-t1*

by (*rule t1-model-wellformed*)

next

show *t1-cchunks ()* $\neq \{\}$ **using** *t1m-chunks* **by** *simp*

next

show $\exists c e. Cdc c e \in set (t1-model.clean-prefix-of ())$

using *t1m-clean-prefix* **by** *auto*

next

show *restrict (Apply (t1-model.clean-prefix-of ())) (t1-cscope ())*

$= restrict (Src b0-t1 H-t1 (t1-cfrontier ())) (t1-cscope ())$

by (*rule t1-model-virtual-cut-holds*)

next

show *restrict (Apply (t1-model.clean-prefix-of ())) (t1-cscope ())*

$\neq restrict b0-t1 (t1-cscope ())$

by (*rule t1-model-virtual-cut-non-trivial*)

qed

34 Topic 2: constructed interpretation

Instantiate *dblog-run-substrate* at the Topic 2 model carrier (*unit* run, the two-constructor *t2-cchunk*); the two *chunks-list* obligations reduce to *set*

$[T2ChA, T2ChB] = \{T2ChA, T2ChB\}$ and *distinct* $[T2ChA, T2ChB]$, discharged by *simp* — no axiom.

global-interpretation *t2-model: dblog-run-substrate*

t2-cscope

t2-cfrontier

t2-cchunks

t2-csrc-history-of

t2-cchunk-read-result

t2-ccdc-events-of

t2-cchunks-list

t2-cchunk-domain

t2-cresponsible-chunk

t2-cchunk-lower-watermark

t2-cchunk-upper-watermark

t2-cchunk-read-coordinate

by *unfold-locales (simp-all add: t2-cchunks-def t2-cchunks-list-def)*

34.1 Topic 2 model accessor-value lemmas

lemma *t2m-distinct*: $T2ChA \neq T2ChB$ **by** *simp*

lemma *t2m-scope*: $t2-cscope () = \{10, 20\}$ **by** *(simp add: t2-cscope-def)*

lemma *t2m-frontier*: $t2-cfrontier () = c0$ **by** *(simp add: t2-cfrontier-def)*

lemma *t2m-chunks*: $t2-cchunks () = \{T2ChA, T2ChB\}$ **by** *(simp add: t2-cchunks-def)*

lemma *t2m-dom-A*: $t2-cchunk-domain () T2ChA = \{10\}$

by *(simp add: t2-cchunk-domain-def)*

lemma *t2m-dom-B*: $t2-cchunk-domain () T2ChB = \{20\}$

by *(simp add: t2-cchunk-domain-def)*

lemma *t2m-resp-10*: $t2-cresponsible-chunk () 10 = \text{Some } T2ChA$

by *(simp add: t2-cresponsible-chunk-def)*

lemma *t2m-resp-20*: $t2-cresponsible-chunk () 20 = \text{Some } T2ChB$

by *(simp add: t2-cresponsible-chunk-def)*

lemma *t2m-resp-else*:

$k \neq 10 \implies k \neq 20 \implies t2-cresponsible-chunk () k = \text{None}$

by *(simp add: t2-cresponsible-chunk-def)*

lemma *t2m-rcoord-A*: $t2-cchunk-read-coordinate () T2ChA = c0$

by *(simp add: t2-cchunk-read-coordinate-def)*

lemma *t2m-rcoord-B*: $t2-cchunk-read-coordinate () T2ChB = c0$

by *(simp add: t2-cchunk-read-coordinate-def)*

lemma *t2m-lwm-A*: $t2-cchunk-lower-watermark () T2ChA = c0$

by *(simp add: t2-cchunk-lower-watermark-def)*

lemma *t2m-uwu-A*: $t2-cchunk-upper-watermark () T2ChA = c0$

by *(simp add: t2-cchunk-upper-watermark-def)*

lemma *t2m-lwm-B*: $t2-cchunk-lower-watermark () T2ChB = c0$

by *(simp add: t2-cchunk-lower-watermark-def)*

lemma *t2m-uwu-B*: $t2-cchunk-upper-watermark () T2ChB = c0$

by *(simp add: t2-cchunk-upper-watermark-def)*

lemma *t2m-src-history*: $t2-csrc-history-of () = H-t2$

by *(simp add: t2-csrc-history-of-def)*

```

lemma t2m-crr-A-10: t2-cchunk-read-result () T2ChA 10 = Some (Some 100)
  by (simp add: t2-cchunk-read-result-def)
lemma t2m-crr-B-20: t2-cchunk-read-result () T2ChB 20 = Some (Some 200)
  by (simp add: t2-cchunk-read-result-def)
lemma t2m-cdc: t2-ccdc-events-of () = []
  by (simp add: t2-ccdc-events-of-def H-t2-def)
lemma t2m-chunks-list: t2-cchunks-list () = [T2ChA, T2ChB]
  by (simp add: t2-cchunks-list-def)

lemma t2m-chunks-eq:
  (ch ∈ t2-cchunks ())  $\longleftrightarrow$  (ch = T2ChA ∨ ch = T2ChB)
  using t2m-chunks by simp

```

34.2 Topic 2 determinate clean prefix over the model

```

lemma t2m-clean-prefix:
  t2-model.clean-prefix-of ()
    = [ Refresh (10::nat) (Some (100::nat)) c0,
        Refresh 20 (Some 200) c0 ]
proof –
  have dA: sorted-list-of-set {10::nat} = [10] by simp
  have dB: sorted-list-of-set {20::nat} = [20] by simp
  show ?thesis
    unfolding t2-model.clean-prefix-of-def canonical-clean-prefix-def
      cdc-event-replays-def chunk-read-refreshes-def
    by (simp add: t2m-chunks-list t2m-dom-A t2m-dom-B
      t2m-rcoord-A t2m-rcoord-B
      t2m-crr-A-10 t2m-crr-B-20
      t2m-cdc t2m-scope t2m-frontier dA dB
      List.map-filter-simps sort-key-def)
qed

```

34.3 Topic 2 constructed wellformed-run fact

```

theorem t2-model-wellformed:
  t2-model.wellformed-dblog-run b0-t2 () H-t2
    unfolding t2-model.wellformed-dblog-run-def
proof (intro conjI)
  show t2-csrc-history-of () = H-t2 by (rule t2m-src-history)
next
  show wellformed-src-history H-t2 by (rule wf-h-t2)
next
  show  $\forall k \in t2-cscope$  ().  $\exists !ch. ch \in t2-cchunks$  ()  $\wedge k \in t2-cchunk-domain$  () ch
  proof
    fix k :: nat assume k ∈ t2-cscope ()
    hence k-cases: k = 10 ∨ k = 20 using t2m-scope by simp
    show  $\exists !ch. ch \in t2-cchunks$  ()  $\wedge k \in t2-cchunk-domain$  () ch
    proof (cases k = 10)
      case True
      have T2ChA ∈ t2-cchunks ()  $\wedge k \in t2-cchunk-domain$  () T2ChA

```

```

    using t2m-chunks t2m-dom-A True by simp
    moreover have  $\bigwedge ch. ch \in t2-cchunks () \wedge k \in t2-cchunk-domain () \implies$ 
ch = T2ChA
    using t2m-chunks-eq t2m-dom-A t2m-dom-B True by auto
    ultimately show ?thesis by blast
next
case False
hence k20: k = 20 using k-cases by simp
have T2ChB  $\in t2-cchunks () \wedge k \in t2-cchunk-domain ()$  T2ChB
    using t2m-chunks t2m-dom-B k20 by simp
    moreover have  $\bigwedge ch. ch \in t2-cchunks () \wedge k \in t2-cchunk-domain () \implies$ 
ch = T2ChB
    using t2m-chunks-eq t2m-dom-A t2m-dom-B k20 by auto
    ultimately show ?thesis by blast
qed
qed
next
show  $\forall ch1 \in t2-cchunks (). \forall ch2 \in t2-cchunks ().$ 
ch1  $\neq ch2 \implies t2-cchunk-domain ()$  ch1  $\cap t2-cchunk-domain ()$  ch2 = {}
    using t2m-chunks-eq t2m-dom-A t2m-dom-B by auto
next
show t2-model.canonical-chunk-ownership-domain () = t2-cscope ()
    unfolding t2-model.canonical-chunk-ownership-domain-def
    using t2m-chunks t2m-dom-A t2m-dom-B t2m-scope by auto
next
show  $\forall k \in t2-cscope (). \forall ch.$ 
t2-cresponsible-chunk () k = Some ch
 $\iff ch \in t2-cchunks () \wedge k \in t2-cchunk-domain ()$  ch
proof (intro ballI allI)
fix k :: nat and ch :: t2-cchunk
assume k  $\in t2-cscope ()$ 
hence k-cases: k = 10  $\vee$  k = 20 using t2m-scope by simp
show t2-cresponsible-chunk () k = Some ch
 $\iff ch \in t2-cchunks () \wedge k \in t2-cchunk-domain ()$  ch
proof (cases k = 10)
case True
then have rc: t2-cresponsible-chunk () k = Some T2ChA
    using t2m-resp-10 by simp
show ?thesis
proof
assume t2-cresponsible-chunk () k = Some ch
with rc have ch = T2ChA by simp
thus ch  $\in t2-cchunks () \wedge k \in t2-cchunk-domain ()$  ch
    using t2m-chunks t2m-dom-A True by simp
next
assume A: ch  $\in t2-cchunks () \wedge k \in t2-cchunk-domain ()$  ch
have ch = T2ChA
    using A t2m-chunks-eq t2m-dom-A t2m-dom-B True by auto
with rc show t2-cresponsible-chunk () k = Some ch by simp

```

```

    qed
  next
    case False
    hence k20:  $k = 20$  using k-cases by simp
    then have rc: t2-cresponsible-chunk ()  $k = \text{Some } T2ChB$ 
      using t2m-resp-20 by simp
    show ?thesis
    proof
      assume t2-cresponsible-chunk ()  $k = \text{Some } ch$ 
      with rc have ch = T2ChB by simp
      thus  $ch \in t2-cchunks () \wedge k \in t2-cchunk-domain ()$  ch
        using t2m-chunks t2m-dom-B k20 by simp
    next
      assume A:  $ch \in t2-cchunks () \wedge k \in t2-cchunk-domain ()$  ch
      have ch = T2ChB
        using A t2m-chunks-eq t2m-dom-A t2m-dom-B k20 by auto
      with rc show t2-cresponsible-chunk ()  $k = \text{Some } ch$  by simp
    qed
  qed
next
  show  $\forall k. k \notin t2-cscope () \longrightarrow t2-cresponsible-chunk () k = \text{None}$ 
    using t2m-scope t2m-resp-else by auto
next
  show finite (t2-cscope ()) using t2m-scope by simp
next
  show  $\forall ch \in t2-cchunks (). \text{finite } (t2-cchunk-domain () ch)$ 
    using t2m-chunks-eq t2m-dom-A t2m-dom-B by auto
next
  show  $\forall ch \in t2-cchunks (). t2-cchunk-domain () ch \neq \{\}$ 
    using t2m-chunks-eq t2m-dom-A t2m-dom-B by auto
next
  show  $\forall k \in t2-cscope (). t2-model.covers-ordinary-cdc () k (t2-cfrontier ())$ 
    proof
      fix k :: nat assume  $k \in t2-cscope ()$ 
      hence k-cases:  $k = 10 \vee k = 20$  using t2m-scope by simp
      show t2-model.covers-ordinary-cdc ()  $k (t2-cfrontier ())$ 
        proof (cases k = 10)
          case True
          show ?thesis
            unfolding t2-model.covers-ordinary-cdc-def
            proof (intro exI [where x = T2ChA] conjI allI impI)
              show t2-cresponsible-chunk ()  $k = \text{Some } T2ChA$ 
                using t2m-resp-10 True by simp
            next
              fix i assume  $i < \text{length } (t2-csrc-history-of ())$ 
              with t2m-src-history length-H-t2 have False by simp
              thus t2-csrc-history-of () ! i  $\in \text{set } (t2-ccdc-events-of ())$  ..
            qed
          qed
        qed
    qed

```

```

next
  case False
  hence k20: k = 20 using k-cases by simp
  show ?thesis
    unfolding t2-model.covers-ordinary-cdc-def
  proof (intro exI[where x = T2ChB] conjI allI impI)
    show t2-cresponsible-chunk () k = Some T2ChB
      using t2m-resp-20 k20 by simp
  next
    fix i assume i < length (t2-csrc-history-of ())
    with t2m-src-history length-H-t2 have False by simp
    thus t2-csrc-history-of () ! i ∈ set (t2-ccdc-events-of ()) ..
  qed
qed
qed
next
  show ∀ p ∈ set (t2-ccdc-events-of ()).
    key-of (hist-event p) ∈ t2-cscope ()
    ∧ src-le (hist-coord p) (t2-cfrontier ())
    → p ∈ set (t2-csrc-history-of ())
  using t2m-cdc by simp
next
  show ∃ ι :: nat ⇒ nat. strict-mono ι
    ∧ (let in-slice = (λp. key-of (hist-event p) ∈ t2-cscope ()
      ∧ src-le (hist-coord p) (t2-cfrontier ()));
      cdc-in-slice = filter in-slice (t2-ccdc-events-of ());
      hist-in-slice = filter in-slice (t2-csrc-history-of ())
      in ∀ i. i < length cdc-in-slice →
        ι i < length hist-in-slice
        ∧ cdc-in-slice ! i = hist-in-slice ! ι i)
  proof (intro exI[where x = λn :: nat. n] conjI)
    show strict-mono (λn :: nat. n) by (simp add: strict-mono-def)
  next
    show let in-slice = (λp. key-of (hist-event p) ∈ t2-cscope ()
      ∧ src-le (hist-coord p) (t2-cfrontier ()));
      cdc-in-slice = filter in-slice (t2-ccdc-events-of ());
      hist-in-slice = filter in-slice (t2-csrc-history-of ())
      in ∀ i. i < length cdc-in-slice →
        (λn :: nat. n) i < length hist-in-slice
        ∧ cdc-in-slice ! i = hist-in-slice ! (λn :: nat. n) i
    using t2m-cdc by (simp add: Let-def)
  qed
next
  show ∀ k ∈ t2-cscope (). ∀ ch c.
    t2-cresponsible-chunk () k = Some ch
    ∧ src-lt (t2-cchunk-read-coordinate () ch) c
    ∧ src-le c (t2-cfrontier ())
    →
      mset (source-key-coord-slice k c (t2-ccdc-events-of ()))

```



```

      = mset (source-key-coord-slice k c (t2-csrc-history-of ()))
    using t2m-cdc t2m-src-history H-t2-def
    by (simp add: source-key-coord-slice-def)
  next
    show  $\forall ch \in t2\text{-cchunks } ()$ .
      src-le (t2-cchunk-lower-watermark () ch) (t2-cchunk-read-coordinate () ch)
     $\wedge$  src-le (t2-cchunk-read-coordinate () ch) (t2-cchunk-upper-watermark () ch)
     $\wedge$  ( $\forall k \in t2\text{-cchunk-domain } ()$  ch.  $\exists m$ . t2-cchunk-read-result () ch k = Some
  m)
       $\wedge$  ( $\forall k \in t2\text{-cchunk-domain } ()$  ch.  $\forall m$ .
        t2-cchunk-read-result () ch k = Some m
         $\longleftrightarrow$  Refresh k m (t2-cchunk-read-coordinate () ch)
         $\in$  set (t2-model.clean-prefix-of ()))
    proof
      fix ch :: t2-cchunk assume ch  $\in$  t2-cchunks ()
      hence ch-cases: ch = T2ChA  $\vee$  ch = T2ChB
      using t2m-chunks-eq by simp
      show src-le (t2-cchunk-lower-watermark () ch) (t2-cchunk-read-coordinate ()
  ch)
       $\wedge$  src-le (t2-cchunk-read-coordinate () ch) (t2-cchunk-upper-watermark () ch)
       $\wedge$  ( $\forall k \in t2\text{-cchunk-domain } ()$  ch.  $\exists m$ . t2-cchunk-read-result () ch k = Some
  m)
       $\wedge$  ( $\forall k \in t2\text{-cchunk-domain } ()$  ch.  $\forall m$ .
        t2-cchunk-read-result () ch k = Some m
         $\longleftrightarrow$  Refresh k m (t2-cchunk-read-coordinate () ch)
         $\in$  set (t2-model.clean-prefix-of ()))
    proof (cases ch = T2ChA)
      case True
      show ?thesis
      using True t2m-lwm-A t2m-uwm-A t2m-rcoord-A src-le-refl
        t2m-dom-A t2m-crr-A-10 t2m-clean-prefix
      by auto
    next
      case False
      hence ch = T2ChB using ch-cases by simp
      show ?thesis
      using  $\langle ch = T2ChB \rangle$  t2m-lwm-B t2m-uwm-B t2m-rcoord-B src-le-refl
        t2m-dom-B t2m-crr-B-20 t2m-clean-prefix
      by auto
    qed
  qed
  next
    show  $\forall ch \in t2\text{-cchunks } ()$ .
      src-le (t2-cchunk-read-coordinate () ch) (t2-cfrontier ())
    using t2m-chunks-eq t2m-rcoord-A t2m-rcoord-B
      t2m-frontier src-le-refl
    by auto
  next
    show  $\forall ch \in t2\text{-cchunks } ()$ .  $\forall k \in t2\text{-cchunk-domain } ()$  ch.  $\forall m$ .

```

```

      t2-cchunk-read-result () ch k = Some m
      → m = Src b0-t2 H-t2 (t2-cchunk-read-coordinate () ch) k
proof
  fix ch :: t2-cchunk assume ch ∈ t2-cchunks ()
  hence ch-cases: ch = T2ChA ∨ ch = T2ChB
  using t2m-chunks-eq by simp
  show ∀ k ∈ t2-cchunk-domain () ch. ∀ m.
    t2-cchunk-read-result () ch k = Some m
    → m = Src b0-t2 H-t2 (t2-cchunk-read-coordinate () ch) k
proof (cases ch = T2ChA)
  case True
  show ?thesis
  using True t2m-dom-A t2m-crr-A-10
    t2m-rcoord-A Src-b0-t2-H-t2-c0-10
  by auto
next
  case False
  hence ch = T2ChB using ch-cases by simp
  show ?thesis
  using ⟨ch = T2ChB⟩ t2m-dom-B t2m-crr-B-20
    t2m-rcoord-B Src-b0-t2-H-t2-c0-20
  by auto
qed
qed
next
  show ∀ ch ∈ t2-cchunks (). ∀ k ∈ t2-cchunk-domain () ch.
    t2-cchunk-read-result () ch k = Some None
    → t2-model.row-absence-meaningful b0-t2 H-t2 () ch k
proof (intro ballI impI)
  fix ch :: t2-cchunk and k :: nat
  assume ch-in: ch ∈ t2-cchunks ()
  and k-in: k ∈ t2-cchunk-domain () ch
  and crr: t2-cchunk-read-result () ch k = Some None
  from ch-in t2m-chunks-eq
  have ch-cases: ch = T2ChA ∨ ch = T2ChB by simp
  have False
  proof (cases ch = T2ChA)
    case True
    with k-in t2m-dom-A have k = 10 by simp
    with True crr t2m-crr-A-10 show False by simp
  next
    case False
    with ch-cases have ch-B: ch = T2ChB by simp
    with k-in t2m-dom-B have k = 20 by simp
    with ch-B crr t2m-crr-B-20 show False by simp
  qed
  thus t2-model.row-absence-meaningful b0-t2 H-t2 () ch k ..
qed
next

```

```

show  $\forall p \in \text{set } (t2\text{-ccdc-events-of } ()).$ 
   $\text{key-of } (\text{hist-event } p) \in t2\text{-cscope } ()$ 
   $\wedge \text{src-le } (\text{hist-coord } p) (t2\text{-cfrontier } ())$ 
   $\longrightarrow \text{Cdc } (\text{hist-coord } p) (\text{hist-event } p)$ 
   $\in \text{set } (t2\text{-model.clean-prefix-of } ())$ 
using t2m-cdc by simp
next
show  $\forall e \text{ c. } \text{Cdc } c \ e \in \text{set } (t2\text{-model.clean-prefix-of } ())$ 
   $\wedge \text{key-of } e \in t2\text{-cscope } ()$ 
   $\wedge \text{src-le } c (t2\text{-cfrontier } ())$ 
   $\longrightarrow (c, e) \in \text{set } (t2\text{-ccdc-events-of } ())$ 
proof (intro allI impI)
  fix e :: (nat, nat) source-event and c :: src-coord
  assume  $\text{Cdc } c \ e \in \text{set } (t2\text{-model.clean-prefix-of } ())$ 
   $\wedge \text{key-of } e \in t2\text{-cscope } ()$ 
   $\wedge \text{src-le } c (t2\text{-cfrontier } ())$ 
  hence cdc-in:  $\text{Cdc } c \ e \in \text{set } (t2\text{-model.clean-prefix-of } ())$  by simp
  have  $\text{Cdc } c \ e \in \text{set } [\text{Refresh } (10::\text{nat}) (\text{Some } (100::\text{nat})) \ c0,$ 
     $\text{Refresh } 20 (\text{Some } 200) \ c0]$ 
  using cdc-in t2m-clean-prefix by simp
  hence False by simp
  thus  $(c, e) \in \text{set } (t2\text{-ccdc-events-of } ()) \text{ ..}$ 
qed
qed

```

34.4 Topic 2 constructed positive witness

```

theorem topic2-positive-witness-constructed:
   $\exists (b0 :: \text{nat} \rightarrow \text{nat}) (R :: \text{unit}) (H :: (\text{nat}, \text{nat}) \text{ src-history}).$ 
     $t2\text{-model.wellformed-dblog-run } b0 \ R \ H$ 
     $\wedge t2\text{-cscope } R \neq \{\}$ 
     $\wedge 2 \leq \text{card } (t2\text{-cchunks } R)$ 
     $\wedge (\forall \text{ch1} \in t2\text{-cchunks } R. \forall \text{ch2} \in t2\text{-cchunks } R.$ 
       $\text{ch1} \neq \text{ch2} \longrightarrow t2\text{-chunk-domain } R \ \text{ch1} \cap t2\text{-chunk-domain } R \ \text{ch2} =$ 
     $\{\})$ 
     $\wedge t2\text{-model.canonical-chunk-ownership-domain } R = t2\text{-cscope } R$ 
proof (intro exI[where x = b0-t2] exI[where x = ()] exI[where x = H-t2] conjI)
  show  $t2\text{-model.wellformed-dblog-run } b0\text{-t2 } () \ H\text{-t2}$ 
  by (rule t2-model-wellformed)
next
  show  $t2\text{-cscope } () \neq \{\}$  using t2m-scope by simp
next
  show  $2 \leq \text{card } (t2\text{-cchunks } ())$  using t2m-chunks t2m-distinct by simp
next
  show  $\forall \text{ch1} \in t2\text{-cchunks } (). \forall \text{ch2} \in t2\text{-cchunks } ().$ 
     $\text{ch1} \neq \text{ch2} \longrightarrow t2\text{-chunk-domain } () \ \text{ch1} \cap t2\text{-chunk-domain } () \ \text{ch2} = \{\}$ 
  using t2m-chunks-eq t2m-dom-A t2m-dom-B by auto
next
  show  $t2\text{-model.canonical-chunk-ownership-domain } () = t2\text{-cscope } ()$ 

```

```

    unfolding t2-model.canonical-chunk-ownership-domain-def
    using t2m-chunks t2m-dom-A t2m-dom-B t2m-scope by auto
qed

```

Models exist for the *dblog-run-substrate* interface realizing the Topic 1 (clean-prefix-carrier) and Topic 2 (multi-chunk partition) witnesses, so both Layer 1 positive-witness results hold by construction, resting on no axiom.

end

```

theory Layer01-Fixtures-Model
  imports Dual-Write-Layer0.Source-Coordinates
begin

```

35 Concrete run models for the Layer 0/1 fixtures

This theory exhibits the explicit run models of the Layer 0/1 fixture family. It gives one reusable concrete run carrier — a record *crun* whose fields are the Layer 1 accessors, with chunks modeled as *nat* — and, for each fixture, a *crun* value realizing exactly the accessor values that fixture’s targeted well-formedness clause references, plus the chunk-enumeration shape captured by *crun-wellshaped* (the chunk enumeration is a duplicate-free listing of the chunk set). Each **-model-values* lemma is proved by *simp* and records those values in one place as a reader-facing accessor-value summary; no later proof consumes these lemmas — the interpretation proofs in *Layer01-Fixtures-Inst* evaluate the model definitions directly.

Layer01-Fixtures-Inst interprets the run-substrate locale (*dblog-run-substrate*) at the *crun* carrier and proves each negative fixture’s rejection — and the empty-scope boundary’s acceptance — over the corresponding value, so every fixture result is established constructively, with no axioms.

Unconstrained accessors are left at their *default-crun* values: a fixture constrains only what its clause needs, so the model only needs to match those. Consequently an instance may violate clauses other than its documented target; each rejection proof in *Layer01-Fixtures-Inst* pivots on the documented clause alone. The source coordinates (*c0* and the fixtures’ own **-c1* coordinates) are *coord-of-nat* definitions from *Dual-Write-Layer0.Source-Coordinates*, which also proves their order lemmas (*src-lt c0 **). The Layer 0 source fixture needs no run model: its history *H-c0* is a concrete **definition** in *Layer01-Fixtures-Core*, with no run carrier involved.

35.1 A reusable concrete run model

```

record crun =
  cr-scope      :: nat set

```

$cr\text{-frontier} \quad :: \text{src-coord}$
 $cr\text{-chunks} \quad :: \text{nat set}$
 $cr\text{-chunks-list} \quad :: \text{nat list}$
 $cr\text{-dom} \quad :: \text{nat} \Rightarrow \text{nat set}$
 $cr\text{-resp} \quad :: \text{nat} \Rightarrow \text{nat option}$
 $cr\text{-rcoord} \quad :: \text{nat} \Rightarrow \text{src-coord}$
 $cr\text{-lwm} \quad :: \text{nat} \Rightarrow \text{src-coord}$
 $cr\text{-uwm} \quad :: \text{nat} \Rightarrow \text{src-coord}$
 $cr\text{-src-history} \quad :: (\text{nat}, \text{nat}) \text{ src-history}$
 $cr\text{-cdc} \quad :: (\text{nat}, \text{nat}) \text{ src-history}$
 $cr\text{-rresult} \quad :: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat option option}$

definition $default\text{-crun} :: \text{crun}$ **where**

$default\text{-crun} =$
 $\langle \mid cr\text{-scope} = \{\}, cr\text{-frontier} = c0, cr\text{-chunks} = \{\}, cr\text{-chunks-list} = [],$
 $cr\text{-dom} = (\lambda\cdot. \{\}), cr\text{-resp} = (\lambda\cdot. None), cr\text{-rcoord} = (\lambda\cdot. c0),$
 $cr\text{-lwm} = (\lambda\cdot. c0), cr\text{-uwm} = (\lambda\cdot. c0),$
 $cr\text{-src-history} = [], cr\text{-cdc} = [], cr\text{-rresult} = (\lambda\cdot. \lambda\cdot. None) \mid \rangle$

The only global constraint on the run interface: the chunk enumeration is a duplicate-free listing of the chunk set.

definition $crun\text{-wellshaped} :: \text{crun} \Rightarrow \text{bool}$ **where**

$crun\text{-wellshaped } r \iff \text{set } (cr\text{-chunks-list } r) = cr\text{-chunks } r \wedge \text{distinct } (cr\text{-chunks-list } r)$

lemmas $crun\text{-simps} = default\text{-crun-def } crun\text{-wellshaped-def}$

35.2 WF1 (b): overlapping chunk domains (*fix-overlap*)

definition $crun\text{-ov} :: \text{crun}$ **where**

$crun\text{-ov} = default\text{-crun}$
 $\langle \mid cr\text{-chunks} := \{0, 1\}, cr\text{-chunks-list} := [0, 1],$
 $cr\text{-dom} := (\lambda n. \text{if } n = 0 \text{ then } \{1, 2\} \text{ else if } n = 1 \text{ then } \{2, 3\} \text{ else } \{\}) \mid \rangle$

lemma $ov\text{-fixture-model-values}$:

$(0 :: \text{nat}) \neq 1$
 $\wedge cr\text{-chunks } crun\text{-ov} = \{0, 1\}$
 $\wedge cr\text{-dom } crun\text{-ov } 0 = \{1, 2\}$
 $\wedge cr\text{-dom } crun\text{-ov } 1 = \{2, 3\}$
 $\wedge crun\text{-wellshaped } crun\text{-ov}$
by ($\text{simp add: } crun\text{-simps } crun\text{-ov-def}$)

35.3 WF1 (c): chunk owns out-of-scope key (*fix-miss-cov*)

definition $crun\text{-mc} :: \text{crun}$ **where**

$crun\text{-mc} = default\text{-crun}$
 $\langle \mid cr\text{-scope} := \{1\}, cr\text{-chunks} := \{0\}, cr\text{-chunks-list} := [0],$
 $cr\text{-dom} := (\lambda n. \text{if } n = 0 \text{ then } \{1, 2\} \text{ else } \{\}) \mid \rangle$

lemma $mc\text{-fixture-model-values}$:

$cr\text{-}scope\ crun\text{-}mc = \{1\}$
 $\wedge\ cr\text{-}chunks\ crun\text{-}mc = \{0\}$
 $\wedge\ cr\text{-}dom\ crun\text{-}mc\ 0 = \{1, 2\}$
 $\wedge\ crun\text{-}wellshaped\ crun\text{-}mc$
by (*simp add: crun-simps crun-mc-def*)

35.4 WF1 (f): infinite chunk domain (*fix-inf-dom*)

definition $crun\text{-}inf :: crun$ **where**

$crun\text{-}inf = default\text{-}crun$
 $\llbracket\ cr\text{-}chunks := \{0\},\ cr\text{-}chunks\text{-}list := [0],$
 $cr\text{-}dom := (\lambda n.\ if\ n = 0\ then\ UNIV\ else\ \{\})\ \rrbracket$

lemma *inf-fixture-model-values*:

$cr\text{-}chunks\ crun\text{-}inf = \{0\}$
 $\wedge\ cr\text{-}dom\ crun\text{-}inf\ 0 = (UNIV :: nat\ set)$
 $\wedge\ crun\text{-}wellshaped\ crun\text{-}inf$
by (*simp add: crun-simps crun-inf-def*)

35.5 WF5: chunk read after frontier (*fix-after-f*)

definition $crun\text{-}caf :: crun$ **where**

$crun\text{-}caf = default\text{-}crun$
 $\llbracket\ cr\text{-}chunks := \{0\},\ cr\text{-}chunks\text{-}list := [0],\ cr\text{-}frontier := c0,$
 $cr\text{-}rcoord := (\lambda n.\ if\ n = 0\ then\ caf\text{-}c1\ else\ c0)\ \rrbracket$

lemma *caf-fixture-model-values*:

$cr\text{-}chunks\ crun\text{-}caf = \{0\}$
 $\wedge\ cr\text{-}frontier\ crun\text{-}caf = c0$
 $\wedge\ cr\text{-}rcoord\ crun\text{-}caf\ 0 = caf\text{-}c1$
 $\wedge\ crun\text{-}wellshaped\ crun\text{-}caf$
by (*simp add: crun-simps crun-caf-def*)

35.6 WF6 main: wrong refresh value (*fix-wrong-r*)

definition $crun\text{-}wr :: crun$ **where**

$crun\text{-}wr = default\text{-}crun$
 $\llbracket\ cr\text{-}chunks := \{0\},\ cr\text{-}chunks\text{-}list := [0],$
 $cr\text{-}dom := (\lambda n.\ if\ n = 0\ then\ \{1\}\ else\ \{\}),$
 $cr\text{-}rcoord := (\lambda_.\ c0),$
 $cr\text{-}rresult := (\lambda ch\ k.\ if\ ch = 0 \wedge k = 1\ then\ Some\ (Some\ 99)\ else\ None)\ \rrbracket$

lemma *wr-fixture-model-values*:

$cr\text{-}chunks\ crun\text{-}wr = \{0\}$
 $\wedge\ cr\text{-}dom\ crun\text{-}wr\ 0 = \{1\}$
 $\wedge\ cr\text{-}rcoord\ crun\text{-}wr\ 0 = c0$
 $\wedge\ cr\text{-}rresult\ crun\text{-}wr\ 0\ 1 = Some\ (Some\ 99)$
 $\wedge\ crun\text{-}wellshaped\ crun\text{-}wr$
by (*simp add: crun-simps crun-wr-def*)

35.7 WF6 specialized: same-coord read/event ambiguity (*fix-same-c*)

definition *crun-sc* :: *crun* **where**

```

crun-sc = default-crun
  () cr-chunks := {0}, cr-chunks-list := [0],
    cr-dom := (λn. if n = 0 then {1} else {}),
    cr-rcoord := (λn. if n = 0 then sc-c1 else c0),
    cr-rresult := (λch k. if ch = 0 ∧ k = 1 then Some (Some 5) else None) ()

```

lemma *sc-fixture-model-values*:

```

cr-chunks crun-sc = {0}
  ∧ cr-dom crun-sc 0 = {1}
  ∧ cr-rcoord crun-sc 0 = sc-c1
  ∧ cr-rresult crun-sc 0 1 = Some (Some 5)
  ∧ crun-wellshaped crun-sc
by (simp add: crun-simps crun-sc-def)

```

35.8 WF2 faithfulness: extra observed CDC (*fix-extra-cdc*)

definition *crun-ec* :: *crun* **where**

```

crun-ec = default-crun
  () cr-scope := {1}, cr-chunks := {0}, cr-chunks-list := [0],
    cr-frontier := ec-c1, cr-src-history := [],
    cr-cdc := [(ec-c1, Insert 1 5)] ()

```

lemma *ec-fixture-model-values*:

```

cr-scope crun-ec = {1}
  ∧ cr-frontier crun-ec = ec-c1
  ∧ cr-src-history crun-ec = []
  ∧ cr-cdc crun-ec = [(ec-c1, Insert 1 5)]
  ∧ crun-wellshaped crun-ec
by (simp add: crun-simps crun-ec-def)

```

35.9 WF2 coverage: missing dominator (*fix-miss-dom*)

definition *crun-md* :: *crun* **where**

```

crun-md = default-crun
  () cr-scope := {1}, cr-chunks := {0}, cr-chunks-list := [0],
    cr-frontier := md-c1, cr-dom := (λn. if n = 0 then {1} else {}),
    cr-resp := (λk. if k = 1 then Some 0 else None),
    cr-rcoord := (λ-. c0),
    cr-src-history := [(md-c1, Update 1 7)], cr-cdc := [] ()

```

lemma *md-fixture-model-values*:

```

cr-scope crun-md = {1}
  ∧ cr-frontier crun-md = md-c1
  ∧ cr-dom crun-md 0 = {1}
  ∧ cr-resp crun-md 1 = Some 0
  ∧ cr-rcoord crun-md 0 = c0

```

$\wedge$ *cr-src-history* *crun-md* = [(*md-c1*, *Update* 1 7)]
 $\wedge$ *cr-cdc* *crun-md* = []
 $\wedge$ *crun-wellshaped* *crun-md*
by (*simp add: crun-simps crun-md-def*)

35.10 WF2 multiplicity: duplicate multiset gap (*fix-dup-mset*)

definition *crun-dm* :: *crun* **where**

crun-dm = *default-crun*
 $\langle$ *cr-scope* := {1}, *cr-chunks* := {0}, *cr-chunks-list* := [0],
cr-frontier := *dm-c1*, *cr-resp* := (λk . if *k* = 1 then *Some* 0 else *None*),
cr-rcoord := (λ -. *c0*),
cr-src-history := [(*dm-c1*, *Update* 1 5), (*dm-c1*, *Update* 1 6), (*dm-c1*, *Update* 1 5)],
cr-cdc := [(*dm-c1*, *Update* 1 5), (*dm-c1*, *Update* 1 6)] $\rangle$

lemma *dm-fixture-model-values*:

cr-scope *crun-dm* = {1}
 $\wedge$ *cr-frontier* *crun-dm* = *dm-c1*
 $\wedge$ *cr-resp* *crun-dm* 1 = *Some* 0
 $\wedge$ *cr-rcoord* *crun-dm* 0 = *c0*
 $\wedge$ *cr-src-history* *crun-dm* = [(*dm-c1*, *Update* 1 5), (*dm-c1*, *Update* 1 6), (*dm-c1*, *Update* 1 5)]
 $\wedge$ *cr-cdc* *crun-dm* = [(*dm-c1*, *Update* 1 5), (*dm-c1*, *Update* 1 6)]
 $\wedge$ *crun-wellshaped* *crun-dm*
by (*simp add: crun-simps crun-dm-def*)

35.11 WF2 order: duplicate order gap (*fix-dup-order*)

definition *crun-do* :: *crun* **where**

crun-do = *default-crun*
 $\langle$ *cr-scope* := {1}, *cr-frontier* := *do-c1*,
cr-src-history := [(*do-c1*, *Update* 1 5), (*do-c1*, *Update* 1 6), (*do-c1*, *Update* 1 5)],
cr-cdc := [(*do-c1*, *Update* 1 5), (*do-c1*, *Update* 1 5), (*do-c1*, *Update* 1 6)] $\rangle$

lemma *do-fixture-model-values*:

cr-scope *crun-do* = {1}
 $\wedge$ *cr-frontier* *crun-do* = *do-c1*
 $\wedge$ *cr-src-history* *crun-do* = [(*do-c1*, *Update* 1 5), (*do-c1*, *Update* 1 6), (*do-c1*, *Update* 1 5)]
 $\wedge$ *cr-cdc* *crun-do* = [(*do-c1*, *Update* 1 5), (*do-c1*, *Update* 1 5), (*do-c1*, *Update* 1 6)]
 $\wedge$ *crun-wellshaped* *crun-do*
by (*simp add: crun-simps crun-do-def*)

35.12 WF1 (d): responsible-chunk vs chunk-domain mismatch (*fix-resp-dom*)

definition *crun-rd* :: *crun* **where**

$crun-rd = default-crun$
 $\langle \langle cr-scope := \{1\}, cr-chunks := \{0\}, cr-chunks-list := [0],$
 $cr-dom := (\lambda n. \text{if } n = 0 \text{ then } \{1\} \text{ else } \{\}), cr-resp := (\lambda -. None) \rangle \rangle$

lemma *rd-fixture-model-values*:

$cr-scope\ crun-rd = \{1\}$
 $\wedge cr-chunks\ crun-rd = \{0\}$
 $\wedge cr-dom\ crun-rd\ 0 = \{1\}$
 $\wedge cr-resp\ crun-rd\ 1 = None$
 $\wedge crun-wellshaped\ crun-rd$
by (*simp add: crun-simps crun-rd-def*)

35.13 WF6 row-absence: absence-Refresh without empty source (*fix-row-abs*)

definition *crun-ra* :: *crun* **where**

$crun-ra = default-crun$
 $\langle \langle cr-chunks := \{0\}, cr-chunks-list := [0],$
 $cr-dom := (\lambda n. \text{if } n = 0 \text{ then } \{1\} \text{ else } \{\}), cr-rcoord := (\lambda -. c0),$
 $cr-rresult := (\lambda ch\ k. \text{if } ch = 0 \wedge k = 1 \text{ then } Some\ None \text{ else } None),$
 $cr-cdc := [] \rangle \rangle$

lemma *ra-fixture-model-values*:

$cr-chunks\ crun-ra = \{0\}$
 $\wedge cr-dom\ crun-ra\ 0 = \{1\}$
 $\wedge cr-rcoord\ crun-ra\ 0 = c0$
 $\wedge cr-rresult\ crun-ra\ 0\ 1 = Some\ None$
 $\wedge cr-cdc\ crun-ra = []$
 $\wedge crun-wellshaped\ crun-ra$
by (*simp add: crun-simps crun-ra-def*)

35.14 Boundary: empty scope, vacuously wellformed (*boundary-empty-scope*)

The empty-scope boundary run has every component empty; *default-crun* already realizes it.

lemma *es-fixture-model-values*:

$cr-scope\ default-crun = \{\}$
 $\wedge cr-chunks\ default-crun = \{\}$
 $\wedge cr-frontier\ default-crun = c0$
 $\wedge cr-src-history\ default-crun = []$
 $\wedge cr-cdc\ default-crun = []$
 $\wedge (\forall k. cr-resp\ default-crun\ k = None)$
 $\wedge crun-wellshaped\ default-crun$
by (*simp add: crun-simps*)

Outcome. Every Layer 0/1 fixture run is an explicit *crun* value realizing exactly the accessor values its wellformedness clause references (plus

the chunk-enumeration shape *crun-wellshaped*). *Layer01-Fixtures-Inst* interprets the run-substrate locale at this carrier and proves each negative fixture rejected — and the empty-scope boundary run wellformed — over these values, so the fixture results are established constructively, not as artifacts of any assumption.

Deliberate coverage gaps. Not every wellformedness clause has a dedicated discriminating fixture. No fixture *can* isolate WF1 (a) (existence of a responsible chunk follows from WF1 (c) — ownership union equals scope — and uniqueness from WF1 (b) disjointness), the scope half of WF1 (f) (implied by WF1 (c), the chunk-domain half, and global chunk-set finiteness), the *Refresh*-iff half of WF4 (automatic under the canonical clean-prefix construction together with WF1 (b) and the WF1 (f) finite chunk domains — *wf4-refresh-agreement-redundant*), or WF7 (*wf7-clean-prefix-cdc-coherence-redundant* proves it for every run). The clauses left untargeted by choice are WF0 (a) recorded-history binding and WF4 watermark bracketing / read-result totality (each violated only collaterally by fixtures aimed at other clauses), WF1 (e) out-of-scope responsible chunks (violated by no fixture), and WF1 (g) non-empty chunk domains (documented headline-inessential at its clause site in *DBLog-Run*).
end

```
theory DBLog-Cert-Substrate
imports DBLog-Run-Substrate-Layer2 Virtual-Cut-Core
begin
```

36 Layer 3/4 certificate substrate as a locale hierarchy (abstract carriers)

The certificate side of the development as a locale hierarchy over abstract carrier type variables, abstracting the concrete *cert* / *evidence* / *raw-observation* datatypes and the certificate/evidence/checker definitions of *Virtual-Cut*.

- *dblog-cert-substrate* extends *dblog-run-substrate* with the certificate, evidence, and observation accessors fixed over plain type variables *'cert* / *'evidence* / *'raw*, and states the cert-side definitions (*virtual-cut*, *checker-input-ok*, *cert-accessors-agree*, *cert-run-coherent*) and the cert-bound Layer 4 whole-table claim-scope (*whole-table-claim-scope*) with bodies matching the global ones. It assumes nothing beyond the two run-substrate *chunks-list* obligations it inherits.

The verifier is fixed *polymorphically* in its result type: *verify* :: *'cert* $\Rightarrow$ *'evidence* $\Rightarrow$ *'vres* with a distinguished accepting value *accept-value*

:: *'vres*. The public *Virtual-Cut.verify-result* datatype and its *Accept* constructor are *not* a dependency of this locale; an interpretation may instantiate *'vres = verify-result* and *accept-value = Accept* (the canonical interpretation in *DBLog-Cert-Substrate-Inst* does exactly that).

- *dblog-checker-substrate* extends *dblog-cert-substrate* with a fixed certificate *C* and evidence *E* and the four primitive checker obligations (the trust boundary), stated as *verify C E = accept-value*, and proves the Layer 3 soundness theorems — *accepted-certificate-implies-wellformed-run* and the headline *accepted-virtual-cut-sound* — together with the Layer 4 whole-table specialization *accepted-whole-table-anchor-domain-specialization*, just as the global *layer3-checker-substrate* does (there via the context extension in *Layer4-Whole-Table*), but over the abstract carriers.

The carrier-independent “pure” core (*virtual-cut-state*, the Layer 4 anchor / table-scope definitions *table-scope-between* / *anchor-complete-at-boundary* / *all-post-anchor-changes-covered* / *no-unclaimed-replay-keys*, and the outside-scope bridge lemma *whole-table-equality-from-scoped-equality*) reference no certificate accessor and live in *Virtual-Cut-Core*; the locale reuses them directly. The Layer 2 run-side soundness theorem *dblog-run-substrate.wellformed-run-implies-virtual-cut* is inherited from *dblog-run-substrate*.

The concrete counterparts — the *cert* / *evidence* datatypes, the global *layer3-checker-substrate* locale, and the global Layer 3/4 theorems — live in *Virtual-Cut* and *Layer4-Whole-Table*; the canonical concrete interpretation of this hierarchy lives in *DBLog-Cert-Substrate-Inst*.

36.1 Certificate substrate locale

The run accessors are re-declared in the *for* clause so the carrier type variables *'run* / *'chunk* / *'k* / *'v* are under this locale’s control and are shared by the certificate accessors added below; a plain *dblog-run-substrate* + import would alpha-rename the inherited type parameters, leaving the certificate key/value/run types disconnected from the run side.

```
locale dblog-cert-substrate =
  dblog-run-substrate
  scope-of frontier-of chunks src-history-of chunk-read-result
  cdc-events-of chunks-list chunk-domain responsible-chunk
  chunk-lower-watermark chunk-upper-watermark chunk-read-coordinate
for scope-of           :: 'run  $\Rightarrow$  ('k::linorder) set
and frontier-of       :: 'run  $\Rightarrow$  frontier
and chunks            :: 'run  $\Rightarrow$  'chunk set
and src-history-of    :: 'run  $\Rightarrow$  ('k, 'v) src-history
and chunk-read-result :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  'k  $\Rightarrow$  'v option option
```

```

and cdc-events-of           :: 'run  $\Rightarrow$  (src-coord  $\times$  ('k, 'v) source-event) list
and chunks-list           :: 'run  $\Rightarrow$  'chunk list
and chunk-domain          :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  'k set
and responsible-chunk     :: 'run  $\Rightarrow$  'k  $\Rightarrow$  'chunk option
and chunk-lower-watermark :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  src-coord
and chunk-upper-watermark :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  src-coord
and chunk-read-coordinate :: 'run  $\Rightarrow$  'chunk  $\Rightarrow$  src-coord
+
fixes scope              :: 'cert  $\Rightarrow$  'k set
and frontier             :: 'cert  $\Rightarrow$  frontier
and clean-prefix :: 'cert  $\Rightarrow$  ('k, 'v) replay-event list
and verify              :: 'cert  $\Rightarrow$  'evidence  $\Rightarrow$  'vres
and accept-value :: 'vres
and faithful-source-observation
    :: 'evidence  $\Rightarrow$  ('k  $\rightarrow$  'v)  $\Rightarrow$  ('k, 'v) src-history  $\Rightarrow$  'raw  $\Rightarrow$  bool
and materializes-run :: 'cert  $\Rightarrow$  'evidence  $\Rightarrow$  'run  $\Rightarrow$  bool
and evidence-bound-to-cert :: 'cert  $\Rightarrow$  'evidence  $\Rightarrow$  bool
and source-side-schema-supported :: 'cert  $\Rightarrow$  'evidence  $\Rightarrow$  bool
begin

```

The central object, cert-side: on *scope* C the replay of *clean-prefix* C equals the source state at *frontier* C . The cert-accessor instance of the core *virtual-cut-state*.

```

definition virtual-cut
  :: ('k  $\rightarrow$  'v)  $\Rightarrow$  'cert  $\Rightarrow$  ('k, 'v) src-history  $\Rightarrow$  bool
where
  virtual-cut b0  $C$   $H \longleftrightarrow$ 
    virtual-cut-state b0 (clean-prefix  $C$ ) (scope  $C$ ) (frontier  $C$ )  $H$ 

```

```

definition checker-input-ok :: 'cert  $\Rightarrow$  'evidence  $\Rightarrow$  bool
where
  checker-input-ok  $C$   $E \longleftrightarrow$ 
    evidence-bound-to-cert  $C$   $E \wedge$  source-side-schema-supported  $C$   $E$ 

```

```

definition cert-accessors-agree
where
  cert-accessors-agree  $C$   $R \longleftrightarrow$ 
    scope  $C =$  scope-of  $R$ 
     $\wedge$  frontier  $C =$  frontier-of  $R$ 
     $\wedge$  clean-prefix  $C =$  clean-prefix-of  $R$ 

```

```

definition cert-run-coherent
where
  cert-run-coherent  $C$   $E$   $R \longleftrightarrow$ 
    materializes-run  $C$   $E$   $R \wedge$  cert-accessors-agree  $C$   $R$ 

```

```

lemma cert-run-coherence:
assumes cert-run-coherent  $C$   $E$   $R$ 
shows scope  $C =$  scope-of  $R$ 

```

$\wedge \text{frontier } C = \text{frontier-of } R$
 $\wedge \text{clean-prefix } C = \text{clean-prefix-of } R$
using *assms*
unfolding *cert-run-coherent-def cert-accessors-agree-def*
by *blast*

lemma *cert-run-coherent-virtual-cut-state-subst*:
assumes *cert-accessors-agree C R*
and *virtual-cut-state b0 (clean-prefix-of R) (scope-of R)*
 $(\text{frontier-of } R) \ H$
shows *virtual-cut-state b0 (clean-prefix C) (scope C)*
 $(\text{frontier } C) \ H$
using *assms*
unfolding *cert-accessors-agree-def*
by *simp*

Layer 4 whole-table claim scope, cert-side (body identical to the global *Layer4-Whole-Table.whole-table-claim-scope*).

definition *whole-table-claim-scope*
 $:: ('k \rightarrow 'v) \Rightarrow 'cert \Rightarrow ('k, 'v) \text{src-history} \Rightarrow \text{frontier} \Rightarrow \text{bool}$
where
 $\text{whole-table-claim-scope } b0 \ C \ H \ b \longleftrightarrow$
 $b \leq \text{frontier } C$
 $\wedge \text{anchor-complete-at-boundary } b0 \ H \ b \ (\text{scope } C)$
 $\wedge \text{all-post-anchor-changes-covered } H \ b \ (\text{frontier } C) \ (\text{scope } C)$
 $\wedge \text{no-unclaimed-replay-keys } (\text{clean-prefix } C) \ (\text{scope } C)$

lemma *whole-table-claim-scope-imp-table-scope-contained*:
assumes *whole-table-claim-scope b0 C H b*
shows *table-scope-between b0 H b (frontier C) $\subseteq$ scope C*
using *assms*
unfolding *whole-table-claim-scope-def*
 $\text{table-scope-between-def}$
 $\text{anchor-complete-at-boundary-def}$
 $\text{all-post-anchor-changes-covered-def}$
by *blast*

lemma *whole-table-equality-from-claim-scope*:
assumes *vc: virtual-cut b0 C H*
and *whole: whole-table-claim-scope b0 C H b*
shows $\text{Apply } (\text{clean-prefix } C) = \text{Src } b0 \ H \ (\text{frontier } C)$
proof –
have *scoped*:
 $\text{restrict } (\text{Apply } (\text{clean-prefix } C)) \ (\text{scope } C)$
 $= \text{restrict } (\text{Src } b0 \ H \ (\text{frontier } C)) \ (\text{scope } C)$
using *vc unfolding virtual-cut-def virtual-cut-state-def* **by** *simp*
have *contained*:
 $\text{table-scope-between } b0 \ H \ b \ (\text{frontier } C) \subseteq \text{scope } C$
by $(\text{rule } \text{whole-table-claim-scope-imp-table-scope-contained}[OF \ \text{whole}])$

```

have no-unclaimed:
  no-unclaimed-replay-keys (clean-prefix  $C$ ) (scope  $C$ )
  using whole unfolding whole-table-claim-scope-def by blast
have b-le-frontier:  $b \leq \text{frontier } C$ 
  using whole unfolding whole-table-claim-scope-def by blast
show ?thesis
  by (rule whole-table-equality-from-scoped-equality
    [OF scoped contained no-unclaimed b-le-frontier])
qed

end

```

36.2 Layer 3 checker substrate locale

The non-circular Layer 3 proof substrate. The locale fixes a certificate C and evidence E and separates the four primitive checker obligations (the trust boundary discharged at deployment, not inside the model); the Layer 3 soundness theorems and the Layer 4 specialization are then proved inside the locale from those obligations and the inherited Layer 2 run-side soundness. Acceptance is the abstract $\text{verify } C \ E = \text{accept-value}$.

```

locale dblog-checker-substrate = dblog-cert-substrate +
  fixes  $C$  and  $E$ 
  assumes accepted-input-ok:
     $\text{verify } C \ E = \text{accept-value} \implies \text{checker-input-ok } C \ E$ 
  and accepted-has-materialization:
     $\llbracket \text{verify } C \ E = \text{accept-value}; \text{checker-input-ok } C \ E \rrbracket$ 
     $\implies \exists R. \text{materializes-run } C \ E \ R$ 
  and accepted-materialization-agrees:
     $\bigwedge R. \llbracket \text{verify } C \ E = \text{accept-value}; \text{checker-input-ok } C \ E; \text{materializes-run } C \ E \ R \rrbracket$ 
     $\implies \text{cert-accessors-agree } C \ R$ 
  and faithful-materialization-wellformed:
     $\bigwedge R \ b0 \ H \ \text{Raw}. \llbracket \text{verify } C \ E = \text{accept-value}; \text{checker-input-ok } C \ E; \text{materializes-run } C \ E \ R; \text{faithful-source-observation } E \ b0 \ H \ \text{Raw} \rrbracket$ 
     $\implies \text{wellformed-dblog-run } b0 \ R \ H$ 
begin

```

```

theorem accepted-certificate-implies-wellformed-run:
  assumes accept:  $\text{verify } C \ E = \text{accept-value}$ 
  and faithful:  $\text{faithful-source-observation } E \ b0 \ H \ \text{Raw}$ 
  shows  $\exists R. \text{wellformed-dblog-run } b0 \ R \ H \wedge \text{cert-run-coherent } C \ E \ R$ 
proof –
  have input-ok:  $\text{checker-input-ok } C \ E$ 
    by (rule accepted-input-ok[OF accept])
  then obtain  $R$  where mat:  $\text{materializes-run } C \ E \ R$ 
    using accepted-has-materialization[OF accept input-ok] by blast
  have wf:  $\text{wellformed-dblog-run } b0 \ R \ H$ 
    by (rule faithful-materialization-wellformed[OF accept input-ok mat faithful])

```

```

have agree: cert-accessors-agree  $C\ R$ 
  by (rule accepted-materialization-agrees [ $OF\ accept\ input-ok\ mat$ ])
have coherent: cert-run-coherent  $C\ E\ R$ 
  using mat agree
  unfolding cert-run-coherent-def
  by blast
show ?thesis
  using wf coherent by blast
qed

theorem accepted-virtual-cut-sound:
  assumes accept: verify  $C\ E = accept-value$ 
    and faithful: faithful-source-observation  $E\ b0\ H\ Raw$ 
  shows virtual-cut  $b0\ C\ H$ 
proof –
  obtain  $R$  where wf: wellformed-dblog-run  $b0\ R\ H$ 
    and coherent: cert-run-coherent  $C\ E\ R$ 
    using accepted-certificate-implies-wellformed-run [ $OF\ accept\ faithful$ ]
    by blast
  have agree: cert-accessors-agree  $C\ R$ 
    using coherent unfolding cert-run-coherent-def by blast
  have run-vcs:
    virtual-cut-state  $b0\ (clean-prefix-of\ R)\ (scope-of\ R)$ 
       $(frontier-of\ R)\ H$ 
    using wellformed-run-implies-virtual-cut [ $OF\ wf$ ] .
  have cert-vcs:
    virtual-cut-state  $b0\ (clean-prefix\ C)\ (scope\ C)\ (frontier\ C)\ H$ 
    using cert-run-coherent-virtual-cut-state-subst [ $OF\ agree\ run-vcs$ ] .
  show ?thesis
    unfolding virtual-cut-def
    using cert-vcs .
qed

theorem accepted-whole-table-anchor-domain-specialization:
  assumes accept: verify  $C\ E = accept-value$ 
    and faithful: faithful-source-observation  $E\ b0\ H\ Raw$ 
    and whole: whole-table-claim-scope  $b0\ C\ H\ b$ 
  shows Apply  $(clean-prefix\ C) = Src\ b0\ H\ (frontier\ C)$ 
proof –
  have vc: virtual-cut  $b0\ C\ H$ 
    by (rule accepted-virtual-cut-sound [ $OF\ accept\ faithful$ ])
  show ?thesis
    by (rule whole-table-equality-from-claim-scope [ $OF\ vc\ whole$ ])
qed

end

end

```

```

theory DBLog-Cert-Substrate-Inst
  imports DBLog-Cert-Substrate Continuation
begin

```

37 Certificate substrate: continuation corollaries and a canonical concrete interpretation

Two pieces over the certificate-substrate locales of *DBLog-Cert-Substrate*.

- **Continuation corollaries, locale side.** The continuation extension *Continuation* states its two cert-bound corollaries — *virtual-cut-restrict-to-subscope* and the *layer3-checker-substrate* corollary *accepted-certificate-continuation-sound* — over the global certificate carriers and the global *layer3-checker-substrate*. Their heavy content lives entirely in the carrier-independent pure core (*virtual-cut-state-continuation*, *virtual-cut-state-restrict-scope*, *cdc-segment-between*), which is global. Here we restate just the two thin corollaries inside *dblog-cert-substrate* / *dblog-checker-substrate*, reusing the global pure-core theorems verbatim, so the continuation proof API is available at every interpretation of the substrate locales.
- **Canonical concrete interpretation.** The certificate-side analogue of the *canonical* run interpretation of *DBLog-Run-Substrate*. We instantiate *dblog-cert-substrate* at the concrete run carrier (*'k*, *'v*) *run* (through its selector functions) and a one-point certificate / evidence / observation carrier (*unit*). *dblog-cert-substrate* assumes nothing beyond the two *chunks-list* obligations it inherits from *dblog-run-substrate*, so the interpretation discharges its proof obligations with *simp* alone, introducing no axiom. Its success certifies that the abstract certificate interface is inhabited end-to-end on the full certificate accessor set.

The canonical certificate interpretation's verifier rejects everything, so it inhabits the interface without accepting any certificate. The *accepting* interpretation of *dblog-checker-substrate* — discharging the four primitive checker obligations over a wellformed materialized run — is constructed in *Layer3-Witnesses-Inst*.

37.1 Continuation corollaries over the abstract cert carriers

```

context dblog-cert-substrate
begin

```


Sub-scope restriction over a certificate, cert-substrate side: the locale instance of the global *virtual-cut-restrict-to-subscope*, reducing to the global pure-core *virtual-cut-state-restrict-scope*.

```

theorem virtual-cut-restrict-to-subscope:
  assumes vc: virtual-cut b0 C H
    and sub:  $K' \subseteq \text{scope } C$ 
  shows virtual-cut-state b0 (clean-prefix C) K' (frontier C) H
proof –
  have virtual-cut-state b0 (clean-prefix C) (scope C) (frontier C) H
    using vc by (simp add: virtual-cut-def virtual-cut-state-def virtual-cut-state-def)
  thus ?thesis
    using sub by (rule virtual-cut-state-restrict-scope)
qed

end

```

```

context dblog-checker-substrate
begin

```

Continuation over an accepted certificate, checker-substrate side: the locale instance of the global *layer3-checker-substrate.accepted-certificate-continuation-sound*, composing the inherited *accepted-virtual-cut-sound* with the global pure-core *virtual-cut-state-continuation*.

```

theorem accepted-certificate-continuation-sound:
  assumes accept: verify C E = accept-value
    and faithful: faithful-source-observation E b0 H Raw
    and wfH: wellformed-src-history H
    and seg: cdc-segment-between H (scope C) (frontier C) f'  $\delta$ 
  shows virtual-cut-state b0 (clean-prefix C @  $\delta$ ) (scope C) f' H
proof –
  have virtual-cut b0 C H
    by (rule accepted-virtual-cut-sound[OF accept faithful])
  hence virtual-cut-state b0 (clean-prefix C) (scope C) (frontier C) H
    unfolding virtual-cut-def virtual-cut-state-def virtual-cut-state-def .
  from virtual-cut-state-continuation[OF wfH this seg]
  show ?thesis .
qed

end

```

37.2 Canonical certificate-substrate interpretation

The run side reuses the concrete *run* carrier through its selector functions (chunks modeled as *nat*, the chunk enumeration *remdups* $\circ$ *rr-chunks-list*). The certificate, evidence, and observation carriers are the one-point type *unit*; their accessors are arbitrary total functions (the locale constrains none of them), so the only proof obligations are the two inherited *chunks-list* facts, which hold definitionally for every carrier value.

global-interpretation *canonical-cert: dblog-cert-substrate*

```

  rr-scope
  rr-frontier
  λr. set (rr-chunks-list r)
  rr-src-history
  rr-rresult
  rr-cdc
  λr. remdups (rr-chunks-list r)
  rr-dom
  rr-resp
  rr-lwm
  rr-uwm
  rr-rcoord
  λ::unit. {}
  λ::unit. c0
  λ::unit. []
  λ(::unit) (::unit). Reject
  Accept
  λ(::unit) - - (::unit). False
  λ(::unit) (::unit) -. False
  λ(::unit) (::unit). False
  λ(::unit) (::unit). False
  by unfold-locales simp-all

```

end

theory *Layer3-Witnesses-Inst*

```

  imports
    Virtual-Cut
    DBLog-Cert-Substrate
    Layer01-Virtual-Cut-Example-Inst
    Layer4-Witnesses-Core

```

begin

38 Constructed interpretation for the Layer 3 accepted-certificate and Layer 4 whole-table witnesses

The accepting-certificate witness. The run-side witnesses (the minimum-viable witness, the Topic 1/2 witnesses, and the running example) are constructed interpretations of *dblog-run-substrate*, whose only obligations are the two inherited *chunks-list* facts; the certificate side is the structurally distinct one, because the checker substrate carries genuine proof obligations.

This theory constructs the positive certificate/evidence/observation witness as a **global-interpretation** *l3i* of *dblog-checker-substrate* (the Layer 3/4 checker-substrate locale of *DBLog-Cert-Substrate*),

whose run side reuses the running-example interpretation *ex-model* of *Layer01-Virtual-Cut-Example-Inst* (the *unit* run carrier, the *cchunk* two-element chunk datatype) and whose certificate / evidence / observation carriers are the one-point type *unit*. Unlike the *canonical-cert* interpretation of *dblog-cert-substrate* in *DBLog-Cert-Substrate-Inst* — which inhabits the certificate interface with a rejecting verifier and so discharges only the two *chunks-list* obligations — this interpretation is *accepting*: it instantiates the four primitive checker obligations of *dblog-checker-substrate* (the trust boundary) and discharges them *definitionally*, with no certificate axiom.

Method. The certificate accessors are one-point functions targeted at the *ex-model* run (whose chunk-covered scope $\{100::'a, 200::'a, 300::'a\}$, frontier *ec4*, and clean prefix realize the running example): the verifier always accepts, the evidence is bound and schema-supported, the cert accessors agree with the one *ex-model* run (*()*), and faithful observation pins *b0-ex* / *H-ex*. The four checker obligations then reduce by *simp* to: acceptance gives *checker-input-ok* (both schema predicates are *True*); a materialization exists (the run (*()*)); the materialized run's accessors agree with the certificate (*cscope-of* / *cfrontier-of* / *ex-model.clean-prefix-of-def*); and the materialized run is wellformed (*ex-model-wellformed*). Every ingredient — the source coordinates (*coord-of-nat* definitions with proved order lemmas), the run model, and the certificate accessors — is definitional. On this interpretation the theory establishes Layer 3 certificate soundness, the Layer 4 whole-table witness (anchor *ec1*, certified frontier *ec4*, reusing the anchor-domain facts of *Layer4-Witnesses-Core*), and the positive non-vacuity witness.

38.1 One-point certificate / evidence / observation accessors

definition *l3i-verify* :: *unit* $\Rightarrow$ *unit* $\Rightarrow$ *verify-result* **where**
l3i-verify - - = *Accept*

definition *l3i-ev-bound* :: *unit* $\Rightarrow$ *unit* $\Rightarrow$ *bool* **where**
l3i-ev-bound - - = *True*

definition *l3i-schema* :: *unit* $\Rightarrow$ *unit* $\Rightarrow$ *bool* **where**
l3i-schema - - = *True*

definition *l3i-materializes* :: *unit* $\Rightarrow$ *unit* $\Rightarrow$ *unit* $\Rightarrow$ *bool* **where**
l3i-materializes - - *R* = (*R* = *()*)

definition *l3i-faithful* ::
unit $\Rightarrow$ (*nat* $\rightarrow$ *nat*) $\Rightarrow$ (*nat*, *nat*) *src-history* $\Rightarrow$ *unit* $\Rightarrow$ *bool* **where**
l3i-faithful - *b0 H Raw* = (*b0* = *b0-ex* $\wedge$ *H* = *H-ex* $\wedge$ *Raw* = *()*)

38.2 Constructed accepting interpretation of the checker substrate locale

The certificate substrate at the running-example run already holds: its run side is the *ex-model* interpretation of *dblog-run-substrate* and the certificate accessors add no obligation beyond the two inherited *chunks-list* facts. Establishing it as a (local) interpretation *cert-loc* makes the certificate-side definitional equations of *dblog-cert-substrate* — *checker-input-ok-def* and *cert-accessors-agree-def* — available *unguarded* at these parameters, which discharges the four checker obligations below.

interpretation *cert-loc*: *dblog-cert-substrate*
cscope-of cfrontier-of cchunks csrc-history-of cchunk-read-result
ccdc-events-of cchunks-list cchunk-domain cresponsible-chunk
cchunk-lower-watermark cchunk-upper-watermark cchunk-read-coordinate
cscope-of cfrontier-of ex-model.clean-prefix-of
l3i-verify Accept l3i-faithful l3i-materializes l3i-ev-bound l3i-schema
by *unfold-locales (simp-all add: cchunks-def cchunks-list-def)*

global-interpretation *l3i*: *dblog-checker-substrate*
cscope-of cfrontier-of cchunks csrc-history-of cchunk-read-result
ccdc-events-of cchunks-list cchunk-domain cresponsible-chunk
cchunk-lower-watermark cchunk-upper-watermark cchunk-read-coordinate
cscope-of cfrontier-of ex-model.clean-prefix-of
l3i-verify Accept l3i-faithful l3i-materializes l3i-ev-bound l3i-schema
() ()

proof (*unfold-locales, goal-cases*)
case 1 — *accepted-input-ok*: acceptance yields a checker-input-ok pair
then show *?case*
by (*simp add: cert-loc.checker-input-ok-def l3i-ev-bound-def l3i-schema-def*)
next
case 2 — *accepted-has-materialization*: the run *()* materializes
then show *?case* **by** (*simp add: l3i-materializes-def*)
next
case (3 *R*) — *accepted-materialization-agrees*: cert accessors agree with the run
then show *?case*
by (*simp add: cert-loc.cert-accessors-agree-def*)
next
case (4 *R b0 H Raw*) — *faithful-materialization-wellformed*: the run is wellformed
then have *b0 = b0-ex and H = H-ex*
by (*simp-all add: l3i-faithful-def*)
then show *?case*
using *ex-model-wellformed* **by** *simp*
qed

38.3 Constructed Layer 3 certificate soundness

The constructed accepting certificate is sound: its clean prefix replays, on its scope, to the source state at its frontier (the paper’s bridge lemma, certificate

side). The conclusion unfolds to exactly *ex-model-virtual-cut-holds*, but is obtained here through the *dblog-checker-substrate* soundness theorem from acceptance and faithful observation — with no certificate axiom.

theorem *l3i-accepted-virtual-cut-sound*:
cert-loc.virtual-cut b0-ex () H-ex
proof (rule *l3i.accepted-virtual-cut-sound*)
 show *l3i-verify () () = Accept* **by** (simp add: *l3i-verify-def*)
 show *l3i-faithful () b0-ex H-ex ()* **by** (simp add: *l3i-faithful-def*)
qed

38.4 Constructed Layer 4 whole-table witness

The whole-table side conditions on the constructed certificate, derived over the running-example source history and the *ex-model* clean prefix — the anchor domain at *ec1*, the keys touched up to the frontier *ec4*, and the replay-key containment — so the Layer 4 specialization upgrades the scoped equality to unrestricted table equality with no certificate axiom.

lemma *l3i-touched-between*:
touched-between H-ex ec1 ec4 = {200, 300}
using *touched-H-ex-anchor-ec4[unfolded l4-anchor-def]* .

lemma *l3i-replay-keys*:
replay-keys (ex-model.clean-prefix-of ()) = {100, 200, 300}
unfolding *replay-keys-def exm-clean-prefix* **by** *auto*

lemma *l3i-whole-table-claim-scope*:
cert-loc.whole-table-claim-scope b0-ex () H-ex ec1
unfolding *cert-loc.whole-table-claim-scope-def*
proof (intro *conjI*)
 show *ec1 ≤ cfrontier-of ()*
 using *ec1-le-ec4* **by** (simp add: *exm-frontier less-eq-src-coord-def*)
next
 show *anchor-complete-at-boundary b0-ex H-ex ec1 (cscope-of ())*
 unfolding *anchor-complete-at-boundary-def*
 using *l4-anchor-domain[unfolded l4-anchor-def]* **by** (simp add: *exm-scope*)
next
 show *all-post-anchor-changes-covered H-ex ec1 (cfrontier-of ()) (cscope-of ())*
 unfolding *all-post-anchor-changes-covered-def*
 using *l3i-touched-between* **by** (simp add: *exm-frontier exm-scope*)
next
 show *no-unclaimed-replay-keys (ex-model.clean-prefix-of ()) (cscope-of ())*
 unfolding *no-unclaimed-replay-keys-def*
 using *l3i-replay-keys* **by** (simp add: *exm-scope*)
qed

theorem *l3i-accepted-whole-table-equality*:
Apply (ex-model.clean-prefix-of ()) = Src b0-ex H-ex (cfrontier-of ())
proof (rule *l3i.accepted-whole-table-anchor-domain-specialization*)

```

show l3i-verify () () = Accept by (simp add: l3i-verify-def)
show l3i-faithful () b0-ex H-ex () by (simp add: l3i-faithful-def)
show cert-loc.whole-table-claim-scope b0-ex () H-ex ec1
  by (rule l3i-whole-table-claim-scope)
qed

```

38.5 Constructed positive witness (non-vacuity)

The accepting certificate is non-vacuous: nonempty scope, non-base frontier, a value-changing ordinary CDC event whose value reaches the frontier, sound virtual cut, and (via the anchor at *ec1*) unrestricted whole-table equality — all over the construction, with no axioms involved.

theorem *l3i-accepted-certificate-positive-witness*:

```

 $\exists (b0 :: nat \rightarrow nat) H.$ 
  l3i-verify () () = Accept
 $\wedge$  l3i-faithful () b0 H ()
 $\wedge$  cert-loc.virtual-cut b0 () H
 $\wedge$  cscope-of ()  $\neq \{\}$ 
 $\wedge$  cfrontier-of ()  $\neq c0$ 
 $\wedge$  Cdc ec3 (Update (200::nat) (12000::nat))  $\in$  set (ex-model.clean-prefix-of ())
 $\wedge$  b0 200 = Some 10000
 $\wedge$  Apply (ex-model.clean-prefix-of ()) 200 = Some 12000
 $\wedge$  Apply (ex-model.clean-prefix-of ()) = Src b0 H (cfrontier-of ())
proof (intro exI[where x = b0-ex] exI[where x = H-ex] conjI)
  show l3i-verify () () = Accept by (simp add: l3i-verify-def)
  show l3i-faithful () b0-ex H-ex () by (simp add: l3i-faithful-def)
  show cert-loc.virtual-cut b0-ex () H-ex by (rule l3i-accepted-virtual-cut-sound)
  show cscope-of ()  $\neq \{\}$  by (simp add: exm-scope)
  show cfrontier-of ()  $\neq c0$  using c0-neq-ec4 by (simp add: exm-frontier)
  show Cdc ec3 (Update (200::nat) (12000::nat))  $\in$  set (ex-model.clean-prefix-of ())
    by (simp add: exm-clean-prefix)
  show b0-ex 200 = Some 10000 by (simp add: b0-ex-def)
  show Apply (ex-model.clean-prefix-of ()) 200 = Some 12000
    by (simp add: exm-apply)
  show Apply (ex-model.clean-prefix-of ()) = Src b0-ex H-ex (cfrontier-of ())
    by (rule l3i-accepted-whole-table-equality)
qed
end

```

theory *Layer01-Fixtures-Inst*

imports *Layer01-Fixtures-Model Layer01-Fixtures-Core DBLog-Run-Substrate*
begin

39 Constructed interpretations for the Layer 0/1 fixtures

Constructs the twelve Layer 1 negative fixtures (each rejected) and the empty-scope boundary fixture (accepted) over a single interpretation of the Layer 1 locale *dblog-run-substrate* at the concrete model of *Layer01-Fixtures-Model* (the shared *crun* record carrier, chunks modeled as *nat*, and the per-fixture *crun*-... values), establishing every fixture fact with no axioms.

Structure:

- **Constructed interpretation.** We instantiate *dblog-run-substrate* once at the *crun* carrier: the chunk set is $\lambda r. \text{set } (cr\text{-chunks-list } r)$ and the enumeration is $\lambda r. \text{remdups } (cr\text{-chunks-list } r)$ (the same set-plus-remdups realization as the concrete *run* carrier of *DBLog-Run*), so the two *chunks-list* obligations hold definitionally for every *crun* value and the interpretation discharges them with *simp* — no axiom. Every fixture’s *crun*-... value sets *cr-chunks-list* consistently with *cr-chunks*, so the interpreted chunk set agrees with the value the fixture pins.
- **Constructed rejections / acceptance.** Each negative-fixture theorem isolates one wellformedness clause: it extracts that clause from the assumed wellformedness, evaluates the fixture’s *crun*-... record value by *simp* (chunks appear as their *nat* indices), and derives the contradiction; the empty-scope boundary theorem instead discharges every clause. The carrier-independent fixture data (the base states *b0*..., source histories *H*..., and the *Src* / *latest-src-event* computation lemmas) lives in *DBLog-Virtual-Cuts.Layer01-Fixtures-Core*; the fixture source coordinates (*caf-c1* / *sc-c1* / *ec-c1* / *md-c1* / *dm-c1* / *do-c1*) are *coord-of-nat* definitions in *Dual-Write-Layer0.Source-Coordinates*, with their order lemmas proved there.

The fixtures fix the key/value carrier to concrete *nat*; the *crun* carrier fixes $'k = nat$, so the constructed facts live in the global context over the interpretation, none resting on a run/chunk axiom.

Layer 0 source fixture. *fix-h-c0-rejected* (in *DBLog-Virtual-Cuts.Layer01-Fixtures-Core*) rejects *H-c0*, a concrete source history over no run carrier; it involves no run/chunk interface and so has no counterpart here.

39.1 Constructed interpretation at the fixture model carrier

Instantiate *dblog-run-substrate* at the *crun* carrier of *DBLog-Virtual-Cuts.Layer01-Fixtures-Model*, realizing the chunk set and enumeration from the single *cr-chunks-list* field. The two *chunks-list* obligations reduce to $\text{set } (\text{remdups } (cr\text{-chunks-list } r)) =$

set (*cr-chunks-list* *r*) and *distinct* (*remdups* (*cr-chunks-list* *r*)), discharged by *simp* — no axiom.

global-interpretation *cmodel*: *dblog-run-substrate*

cr-scope
cr-frontier
 $\lambda r. \text{set } (cr\text{-chunks-list } r)$
cr-src-history
cr-rresult
cr-cdc
 $\lambda r. \text{remdups } (cr\text{-chunks-list } r)$
cr-dom
cr-resp
cr-lwm
cr-uwm
cr-rcoord
by *unfold-locales simp-all*

39.2 WF1 (b): overlapping chunk domains (*fix-overlap*)

theorem *fix-overlap-rejected-constructed*:

$\neg cmodel.wellformed\text{-}dblog\text{-}run \text{ } b0\text{-}ov \text{ } crun\text{-}ov \text{ } H\text{-}ov$

proof

assume *H*: *cmodel.wellformed-dblog-run b0-ov crun-ov H-ov*

hence *wf1b*:

$\forall ch1 \in \text{set } (cr\text{-chunks-list } crun\text{-}ov). \forall ch2 \in \text{set } (cr\text{-chunks-list } crun\text{-}ov).$
 $ch1 \neq ch2 \longrightarrow cr\text{-dom } crun\text{-}ov \text{ } ch1 \cap cr\text{-dom } crun\text{-}ov \text{ } ch2 = \{\}$

unfolding *cmodel.wellformed-dblog-run-def* **by** (*elim conjE*) (*assumption* | *blast*)

have *inA*: $(0::nat) \in \text{set } (cr\text{-chunks-list } crun\text{-}ov)$ **by** (*simp add: crun-ov-def*)

have *inB*: $(1::nat) \in \text{set } (cr\text{-chunks-list } crun\text{-}ov)$ **by** (*simp add: crun-ov-def*)

have *d01*: $(0::nat) \neq 1$ **by** *simp*

from *wf1b inA inB d01*

have *cr-dom crun-ov 0* $\cap$ *cr-dom crun-ov 1* = $\{\}$ **by** *blast*

thus *False* **by** (*simp add: crun-ov-def*)

qed

39.3 WF1 (c): chunk owns out-of-scope key (*fix-miss-cov*)

theorem *fix-miss-cov-rejected-constructed*:

$\neg cmodel.wellformed\text{-}dblog\text{-}run \text{ } b0\text{-}mc \text{ } crun\text{-}mc \text{ } H\text{-}mc$

proof

assume *H*: *cmodel.wellformed-dblog-run b0-mc crun-mc H-mc*

hence *wf1c*: *cmodel.canonical-chunk-ownership-domain crun-mc* = *cr-scope crun-mc*

unfolding *cmodel.wellformed-dblog-run-def* **by** (*elim conjE*) (*assumption* | *blast*)

have *cmodel.canonical-chunk-ownership-domain crun-mc* = $\{1, 2\}$

unfolding *cmodel.canonical-chunk-ownership-domain-def*

by (*simp add: crun-mc-def*)

with *wf1c* have $(\{1, 2::nat\}) = \{1\}$ by (simp add: *crun-mc-def*)
 thus *False* by *auto*
 qed

39.4 WF1 (f): infinite chunk domain (*fix-inf-dom*)

theorem *fix-inf-dom-rejected-constructed*:

¬ *cmodel.wellformed-dblog-run b0-inf crun-inf H-inf*

proof

assume *H*: *cmodel.wellformed-dblog-run b0-inf crun-inf H-inf*

hence *wf1f2*: $\forall ch \in \text{set } (cr\text{-chunks-list } crun\text{-inf}). \text{finite } (cr\text{-dom } crun\text{-inf } ch)$

unfolding *cmodel.wellformed-dblog-run-def* by (elim *conjE*) (assumption | *blast*)

have $(0::nat) \in \text{set } (cr\text{-chunks-list } crun\text{-inf})$ by (simp add: *crun-inf-def*)

with *wf1f2* have *finite* (*cr-dom crun-inf 0*) by *blast*

hence *finite* (*UNIV :: nat set*) by (simp add: *crun-inf-def*)

thus *False* by *simp*

qed

39.5 WF5: chunk read after frontier (*fix-after-f*)

theorem *fix-after-f-rejected-constructed*:

¬ *cmodel.wellformed-dblog-run b0-caf crun-caf H-caf*

proof

assume *H*: *cmodel.wellformed-dblog-run b0-caf crun-caf H-caf*

hence *wf5*: $\forall ch \in \text{set } (cr\text{-chunks-list } crun\text{-caf}).$

src-le (*cr-rcoord crun-caf ch*) (*cr-frontier crun-caf*)

unfolding *cmodel.wellformed-dblog-run-def* by (elim *conjE*) (assumption | *blast*)

have $(0::nat) \in \text{set } (cr\text{-chunks-list } crun\text{-caf})$ by (simp add: *crun-caf-def*)

with *wf5* have *src-le* (*cr-rcoord crun-caf 0*) (*cr-frontier crun-caf*) by *blast*

hence *src-le caf-c1 c0* by (simp add: *crun-caf-def*)

with *caf-not-src-le-c1-c0* show *False* by *blast*

qed

39.6 WF6 main: wrong refresh value (*fix-wrong-r*)

theorem *fix-wrong-r-rejected-constructed*:

¬ *cmodel.wellformed-dblog-run b0-wr crun-wr H-wr*

proof

assume *H*: *cmodel.wellformed-dblog-run b0-wr crun-wr H-wr*

hence *wf6*:

$\forall ch \in \text{set } (cr\text{-chunks-list } crun\text{-wr}). \forall k \in cr\text{-dom } crun\text{-wr } ch. \forall m.$

cr-rresult crun-wr ch k = *Some m*

$\longrightarrow m = \text{Src } b0\text{-wr } H\text{-wr } (cr\text{-rcoord } crun\text{-wr } ch) k$

unfolding *cmodel.wellformed-dblog-run-def* by (elim *conjE*) (assumption | *blast*)

have *inCh*: $(0::nat) \in \text{set } (cr\text{-chunks-list } crun\text{-wr})$ by (simp add: *crun-wr-def*)

have *inK*: $(1::nat) \in cr\text{-dom } crun\text{-wr } 0$ by (simp add: *crun-wr-def*)

have *crr*: *cr-rresult crun-wr 0 1* = *Some (Some 99)* by (simp add: *crun-wr-def*)

from $wf6$ *inCh inK crr*
have $\text{Some } 99 = \text{Src } b0\text{-wr } H\text{-wr } (cr\text{-rcoord } crun\text{-wr } 0) (1::nat)$ **by** *blast*
moreover **have** $cr\text{-rcoord } crun\text{-wr } 0 = c0$ **by** (*simp add: crun-wr-def*)
ultimately **have** $\text{Some } 99 = \text{Src } b0\text{-wr } H\text{-wr } c0 (1::nat)$ **by** *simp*
with $\text{Src-}b0\text{-wr-}H\text{-wr-}c0\text{-1}$ **have** $\text{Some } 99 = (\text{Some } (5::nat))$ **by** *simp*
thus *False* **by** *simp*
qed

39.7 WF6 specialized: same-coord read/event ambiguity (*fix-same-c*)

theorem *fix-same-c-rejected-constructed:*

$\neg cmodel.\text{wellformed-dblog-run } b0\text{-sc } crun\text{-sc } H\text{-sc}$

proof

assume $H: cmodel.\text{wellformed-dblog-run } b0\text{-sc } crun\text{-sc } H\text{-sc}$

hence *wf6*:

$\forall ch \in \text{set } (cr\text{-chunks-list } crun\text{-sc}). \forall k \in cr\text{-dom } crun\text{-sc } ch. \forall m.$
 $cr\text{-rresult } crun\text{-sc } ch \ k = \text{Some } m$
 $\longrightarrow m = \text{Src } b0\text{-sc } H\text{-sc } (cr\text{-rcoord } crun\text{-sc } ch) \ k$

unfolding $cmodel.\text{wellformed-dblog-run-def}$ **by** (*elim conjE*) (*assumption* | *blast*)

have $\text{inCh}: (0::nat) \in \text{set } (cr\text{-chunks-list } crun\text{-sc})$ **by** (*simp add: crun-sc-def*)
have $\text{inK}: (1::nat) \in cr\text{-dom } crun\text{-sc } 0$ **by** (*simp add: crun-sc-def*)
have $crr: cr\text{-rresult } crun\text{-sc } 0 \ 1 = \text{Some } (\text{Some } 5)$ **by** (*simp add: crun-sc-def*)
from *wf6 inCh inK crr*
have $\text{Some } 5 = \text{Src } b0\text{-sc } H\text{-sc } (cr\text{-rcoord } crun\text{-sc } 0) (1::nat)$ **by** *blast*
moreover **have** $cr\text{-rcoord } crun\text{-sc } 0 = sc\text{-c1}$ **by** (*simp add: crun-sc-def*)
ultimately **have** $\text{Some } 5 = \text{Src } b0\text{-sc } H\text{-sc } sc\text{-c1 } (1::nat)$ **by** *simp*
with $\text{Src-}b0\text{-sc-}H\text{-sc-}c1\text{-1}$ **have** $\text{Some } 5 = (\text{Some } (7::nat))$ **by** *simp*
thus *False* **by** *simp*

qed

39.8 WF2 faithfulness: extra observed CDC (*fix-extra-cdc*)

theorem *fix-extra-cdc-rejected-constructed:*

$\neg cmodel.\text{wellformed-dblog-run } b0\text{-ec } crun\text{-ec } []$

proof

assume $H: cmodel.\text{wellformed-dblog-run } b0\text{-ec } crun\text{-ec } []$

hence *wf2f*:

$\forall p \in \text{set } (cr\text{-cdc } crun\text{-ec}).$
 $\text{key-of } (hist\text{-event } p) \in cr\text{-scope } crun\text{-ec}$
 $\wedge \text{src-le } (hist\text{-coord } p) (cr\text{-frontier } crun\text{-ec})$
 $\longrightarrow p \in \text{set } (cr\text{-src-history } crun\text{-ec})$

unfolding $cmodel.\text{wellformed-dblog-run-def}$ **by** (*elim conjE*) (*assumption* | *blast*)

have $\text{memSet}: (ec\text{-c1}, \text{Insert } 1 \ 5) \in \text{set } (cr\text{-cdc } crun\text{-ec})$
by (*simp add: crun-ec-def*)
have $\text{inScope}: \text{key-of } (hist\text{-event } ((ec\text{-c1}, \text{Insert } (1::nat) \ 5))) \in cr\text{-scope } crun\text{-ec}$
by (*simp add: crun-ec-def*)

have *leF*: *src-le* (*hist-coord* ((*ec-c1*, *Insert* (*1::nat*) 5))) (*cr-frontier crun-ec*)
using *ec-c1-le-c1* **by** (*simp add: crun-ec-def*)
from *wf2f memSet inScope leF*
have *inSrc*: (*ec-c1*, *Insert* (*1::nat*) 5) ∈ *set* (*cr-src-history crun-ec*) **by** *blast*
thus *False* **by** (*simp add: crun-ec-def*)
qed

39.9 WF2 coverage: missing dominator (*fix-miss-dom*)

theorem *fix-miss-dom-rejected-constructed*:

¬ *cmodel.wellformed-dblog-run b0-md crun-md H-md*

proof

assume *H*: *cmodel.wellformed-dblog-run b0-md crun-md H-md*

hence *wf2c*: ∀ *k* ∈ *cr-scope crun-md*.

cmodel.covers-ordinary-cdc crun-md k (*cr-frontier crun-md*)

unfolding *cmodel.wellformed-dblog-run-def* **by** (*elim conjE*) (*assumption* | *blast*)

have *k1*: (*1::nat*) ∈ *cr-scope crun-md* **by** (*simp add: crun-md-def*)

with *wf2c* **have** *cov*: *cmodel.covers-ordinary-cdc crun-md (1::nat)* (*cr-frontier crun-md*)

by *blast*

from *cov* **obtain** *ch* **where**

ch-resp: *cr-resp crun-md 1* = *Some ch* **and**

ch-cov: ∀ *i*. *i* < *length* (*cr-src-history crun-md*)

→ *src-lt* (*cr-rcoord crun-md ch*)

(*hist-coord* (*cr-src-history crun-md ! i*))

→ *src-le* (*hist-coord* (*cr-src-history crun-md ! i*)) (*cr-frontier*

crun-md)

→ *key-of* (*hist-event* (*cr-src-history crun-md ! i*)) = (*1::nat*)

→ *cr-src-history crun-md ! i* ∈ *set* (*cr-cdc crun-md*)

unfolding *cmodel.covers-ordinary-cdc-def* **by** *blast*

from *ch-resp* **have** *ch-eq*: *ch* = 0 **by** (*simp add: crun-md-def*)

have *idx-lt*: (*0::nat*) < *length* (*cr-src-history crun-md*)

by (*simp add: crun-md-def*)

have *lt-premise*: *src-lt* (*cr-rcoord crun-md 0*)

(*hist-coord* (*cr-src-history crun-md ! 0*))

using *md-c0-lt-c1* **by** (*simp add: crun-md-def*)

have *le-premise*: *src-le* (*hist-coord* (*cr-src-history crun-md ! 0*))

(*cr-frontier crun-md*)

using *md-c1-le-c1* **by** (*simp add: crun-md-def*)

have *key-premise*: *key-of* (*hist-event* (*cr-src-history crun-md ! 0*)) = (*1::nat*)

by (*simp add: crun-md-def*)

from *ch-cov ch-eq idx-lt lt-premise le-premise key-premise*

have *cr-src-history crun-md ! 0* ∈ *set* (*cr-cdc crun-md*) **by** *blast*

thus *False* **by** (*simp add: crun-md-def*)

qed

39.10 WF2 multiplicity: duplicate multiset gap (*fix-dup-mset*)

theorem *fix-dup-mset-rejected-constructed*:

$\neg \text{cmodel.wellformed-dblog-run } b0\text{-dm } \text{crun-dm } H\text{-dm}$
proof
assume $H: \text{cmodel.wellformed-dblog-run } b0\text{-dm } \text{crun-dm } H\text{-dm}$
hence wf2-mset :
 $\forall k \in \text{cr-scope crun-dm}. \forall ch \ c.$
 $\text{cr-resp crun-dm } k = \text{Some } ch$
 $\wedge \text{src-lt } (\text{cr-rcoord crun-dm } ch) \ c$
 $\wedge \text{src-le } c \ (\text{cr-frontier crun-dm})$
 $\longrightarrow$
 $\text{mset } (\text{source-key-coord-slice } k \ c \ (\text{cr-cdc crun-dm}))$
 $= \text{mset } (\text{source-key-coord-slice } k \ c \ (\text{cr-src-history crun-dm}))$
unfolding $\text{cmodel.wellformed-dblog-run-def}$ **by** (elim conjE) $(\text{assumption} \mid$
 $\text{blast})$
have ms :
 $\text{mset } (\text{source-key-coord-slice } (1::\text{nat}) \ \text{dm-c1} \ (\text{cr-cdc crun-dm}))$
 $= \text{mset } (\text{source-key-coord-slice } (1::\text{nat}) \ \text{dm-c1} \ (\text{cr-src-history crun-dm}))$
proof $-$
have $k\text{-scope}: (1::\text{nat}) \in \text{cr-scope crun-dm}$
by $(\text{simp add: crun-dm-def})$
from $\text{wf2-mset } k\text{-scope}$
have inst :
 $\forall ch \ c.$
 $\text{cr-resp crun-dm } (1::\text{nat}) = \text{Some } ch$
 $\wedge \text{src-lt } (\text{cr-rcoord crun-dm } ch) \ c$
 $\wedge \text{src-le } c \ (\text{cr-frontier crun-dm})$
 $\longrightarrow$
 $\text{mset } (\text{source-key-coord-slice } (1::\text{nat}) \ c \ (\text{cr-cdc crun-dm}))$
 $= \text{mset } (\text{source-key-coord-slice } (1::\text{nat}) \ c \ (\text{cr-src-history crun-dm}))$
by blast
have prem :
 $\text{cr-resp crun-dm } (1::\text{nat}) = \text{Some } 0$
 $\wedge \text{src-lt } (\text{cr-rcoord crun-dm } 0) \ \text{dm-c1}$
 $\wedge \text{src-le } \text{dm-c1} \ (\text{cr-frontier crun-dm})$
using $\text{dm-c0-lt-c1 } \text{dm-c1-le-c1}$ **by** $(\text{simp add: crun-dm-def})$
show $?thesis$ **using** $\text{inst } \text{prem}$ **by** blast
qed
have bad-mset :
 $\text{mset } [(\text{dm-c1}, \text{Update } (1::\text{nat}) \ (5::\text{nat})), (\text{dm-c1}, \text{Update } 1 \ (6::\text{nat}))]$
 $= \text{mset } [(\text{dm-c1}, \text{Update } (1::\text{nat}) \ (5::\text{nat})), (\text{dm-c1}, \text{Update } 1 \ (6::\text{nat})),$
 $\quad (\text{dm-c1}, \text{Update } 1 \ (5::\text{nat}))]$
using ms **by** $(\text{simp add: source-key-coord-slice-def crun-dm-def})$
hence $\text{length } [(\text{dm-c1}, \text{Update } (1::\text{nat}) \ (5::\text{nat})), (\text{dm-c1}, \text{Update } 1 \ (6::\text{nat}))]$
 $= \text{length } [(\text{dm-c1}, \text{Update } (1::\text{nat}) \ (5::\text{nat})), (\text{dm-c1}, \text{Update } 1 \ (6::\text{nat})),$
 $\quad (\text{dm-c1}, \text{Update } 1 \ (5::\text{nat}))]$
by $(\text{rule mset-eq-length})$
thus False **by** simp
qed

245

39.12 WF1 (d): responsible-chunk vs chunk-domain mismatch (*fix-resp-dom*)

theorem *fix-resp-dom-rejected-constructed*:

$\neg \text{cmodel.wellformed-dblog-run } b0\text{-rd } crun\text{-rd } H\text{-rd}$

proof

assume *H*: $\text{cmodel.wellformed-dblog-run } b0\text{-rd } crun\text{-rd } H\text{-rd}$

hence *wf1d*:

$\forall k \in \text{cr-scope } crun\text{-rd}. \forall ch.$

$\text{cr-resp } crun\text{-rd } k = \text{Some } ch$

$\longleftrightarrow ch \in \text{set } (\text{cr-chunks-list } crun\text{-rd}) \wedge k \in \text{cr-dom } crun\text{-rd } ch$

unfolding $\text{cmodel.wellformed-dblog-run-def}$ **by** (*elim conjE*) (*assumption* | *blast*)

have *inScope*: $(1::\text{nat}) \in \text{cr-scope } crun\text{-rd}$ **by** (*simp add: crun-rd-def*)

with *wf1d*

have *iff-for-rd-ch*:

$\text{cr-resp } crun\text{-rd } 1 = \text{Some } 0$

$\longleftrightarrow (0::\text{nat}) \in \text{set } (\text{cr-chunks-list } crun\text{-rd}) \wedge (1::\text{nat}) \in \text{cr-dom } crun\text{-rd } 0$

by *blast*

have *rhs-holds*: $(0::\text{nat}) \in \text{set } (\text{cr-chunks-list } crun\text{-rd}) \wedge (1::\text{nat}) \in \text{cr-dom } crun\text{-rd } 0$

by (*simp add: crun-rd-def*)

with *iff-for-rd-ch* **have** $\text{cr-resp } crun\text{-rd } 1 = \text{Some } 0$ **by** *simp*

thus *False* **by** (*simp add: crun-rd-def*)

qed

39.13 WF6 row-absence: absence-Refresh without empty source (*fix-row-abs*)

The determinate clean prefix of the row-absence model run, derived from its pinned accessors: a single absence *Refresh* at the base coordinate.

lemma *cm-ra-clean-prefix*:

$\text{cmodel.clean-prefix-of } crun\text{-ra} = [\text{Refresh } (1::\text{nat}) \text{ (None::nat option) } c0]$

proof –

have *d1*: $\text{sorted-list-of-set } \{\text{Suc } 0\} = [\text{Suc } 0]$ **by** *simp*

show *?thesis*

unfolding $\text{cmodel.clean-prefix-of-def canonical-clean-prefix-def}$

$\text{cdc-event-replays-def chunk-read-refreshes-def}$

by (*simp add: crun-ra-def d1 One-nat-def*

$\text{List.map-filter-simps sort-key-def}$)

qed

lemma *cm-refresh-in-ra-prefix*:

$\text{Refresh } (1::\text{nat}) \text{ (None::nat option) } c0 \in \text{set } (\text{cmodel.clean-prefix-of } crun\text{-ra})$

using *cm-ra-clean-prefix* **by** *simp*

theorem *fix-row-abs-rejected-constructed*:

$\neg \text{cmodel.wellformed-dblog-run } b0\text{-ra } crun\text{-ra } H\text{-ra}$

proof

assume H : $\text{cmodel.wellformed-dblog-run } b0\text{-ra } \text{crun-ra } H\text{-ra}$
hence wf6-ra :
 $\forall ch \in \text{set } (\text{cr-chunks-list } \text{crun-ra}). \forall k \in \text{cr-dom } \text{crun-ra } ch.$
 $\text{cr-rresult } \text{crun-ra } ch \ k = \text{Some None}$
 $\longrightarrow \text{cmodel.row-absence-meaningful } b0\text{-ra } H\text{-ra } \text{crun-ra } ch \ k$
unfolding $\text{cmodel.wellformed-dblog-run-def}$ **by** (elim conjE) $(\text{assumption} \mid$
 $\text{blast})$
have inCh : $(0::\text{nat}) \in \text{set } (\text{cr-chunks-list } \text{crun-ra})$ **by** $(\text{simp add: crun-ra-def})$
have inK : $(1::\text{nat}) \in \text{cr-dom } \text{crun-ra } 0$ **by** $(\text{simp add: crun-ra-def})$
have crr : $\text{cr-rresult } \text{crun-ra } 0 \ 1 = \text{Some None}$ **by** $(\text{simp add: crun-ra-def})$
from $\text{wf6-ra inCh inK crr}$
have ram : $\text{cmodel.row-absence-meaningful } b0\text{-ra } H\text{-ra } \text{crun-ra } 0 \ 1$ **by** blast
have rcoord0 : $\text{cr-rcoord } \text{crun-ra } 0 = c0$ **by** $(\text{simp add: crun-ra-def})$
from ram **have** implication :
 $((1::\text{nat}) \in \text{cr-dom } \text{crun-ra } 0$
 $\wedge \text{Refresh } 1 \ \text{None } (\text{cr-rcoord } \text{crun-ra } 0)$
 $\in \text{set } (\text{cmodel.clean-prefix-of } \text{crun-ra}))$
 $\longrightarrow \text{Src } b0\text{-ra } H\text{-ra } (\text{cr-rcoord } \text{crun-ra } 0) \ 1 = \text{None}$
unfolding $\text{cmodel.row-absence-meaningful-def}$ **by** simp
have antecedent :
 $(1::\text{nat}) \in \text{cr-dom } \text{crun-ra } 0$
 $\wedge \text{Refresh } 1 \ \text{None } (\text{cr-rcoord } \text{crun-ra } 0)$
 $\in \text{set } (\text{cmodel.clean-prefix-of } \text{crun-ra})$
using $\text{inK cm-refresh-in-ra-prefix rcoord0}$ **by** simp
from $\text{implication antecedent}$
have $\text{Src } b0\text{-ra } H\text{-ra } (\text{cr-rcoord } \text{crun-ra } 0) \ 1 = \text{None}$ **by** blast
with $\text{rcoord0 Src-b0-ra-H-ra-c0-1}$ **have** $(\text{Some } (99::\text{nat})) = \text{None}$ **by** simp
thus False **by** simp
qed

39.14 Boundary: empty scope, vacuously wellformed (*boundary-empty-scope*)

The empty-scope model run (*default-crun*) has no chunks and no observed CDC, so its clean prefix is empty.

lemma $\text{cm-es-clean-prefix}$: $\text{cmodel.clean-prefix-of } \text{default-crun} = []$
unfolding $\text{cmodel.clean-prefix-of-def canonical-clean-prefix-def cdc-event-replays-def}$
by $(\text{simp add: default-crun-def List.map-filter-simps})$

lemma $\text{cm-es-canonical-partition}$:
 $\text{cmodel.canonical-chunk-ownership-domain } \text{default-crun} = \{\}$
unfolding $\text{cmodel.canonical-chunk-ownership-domain-def}$
by $(\text{simp add: default-crun-def})$

theorem $\text{boundary-empty-scope-constructed}$:
 $\text{cmodel.wellformed-dblog-run } b0\text{-es } \text{default-crun } H\text{-es}$
unfolding $\text{cmodel.wellformed-dblog-run-def}$
proof (intro conjI)
show $\text{cr-src-history } \text{default-crun} = H\text{-es}$

```

    by (simp add: default-crun-def H-es-def)
next
  show wellformed-src-history H-es by (rule wf-h-es)
next
  show  $\forall k \in \text{cr-scope default-crun}.$ 
     $\exists! ch. ch \in \text{set (cr-chunks-list default-crun)} \wedge k \in \text{cr-dom default-crun } ch$ 
    by (simp add: default-crun-def)
next
  show  $\forall ch1 \in \text{set (cr-chunks-list default-crun)}. \forall ch2 \in \text{set (cr-chunks-list default-crun)}.$ 
     $ch1 \neq ch2 \longrightarrow \text{cr-dom default-crun } ch1 \cap \text{cr-dom default-crun } ch2 = \{\}$ 
    by (simp add: default-crun-def)
next
  show  $\text{cmodel.canonical-chunk-ownership-domain default-crun} = \text{cr-scope default-crun}$ 
    using  $\text{cm-es-canonical-partition}$  by (simp add: default-crun-def)
next
  show  $\forall k \in \text{cr-scope default-crun}. \forall ch.$ 
     $\text{cr-resp default-crun } k = \text{Some } ch$ 
     $\longleftrightarrow ch \in \text{set (cr-chunks-list default-crun)} \wedge k \in \text{cr-dom default-crun } ch$ 
    by (simp add: default-crun-def)
next
  show  $\forall k. k \notin \text{cr-scope default-crun} \longrightarrow \text{cr-resp default-crun } k = \text{None}$ 
    by (simp add: default-crun-def)
next
  show  $\text{finite (cr-scope default-crun)}$  by (simp add: default-crun-def)
next
  show  $\forall ch \in \text{set (cr-chunks-list default-crun)}. \text{finite (cr-dom default-crun } ch)$ 
    by (simp add: default-crun-def)
next
  show  $\forall ch \in \text{set (cr-chunks-list default-crun)}. \text{cr-dom default-crun } ch \neq \{\}$ 
    by (simp add: default-crun-def)
next
  show  $\forall k \in \text{cr-scope default-crun}.$ 
     $\text{cmodel.covers-ordinary-cdc default-crun } k (\text{cr-frontier default-crun})$ 
    by (simp add: default-crun-def)
next
  show  $\forall p \in \text{set (cr-cdc default-crun)}.$ 
     $\text{key-of (hist-event } p) \in \text{cr-scope default-crun}$ 
     $\wedge \text{src-le (hist-coord } p) (\text{cr-frontier default-crun})$ 
     $\longrightarrow p \in \text{set (cr-src-history default-crun)}$ 
    by (simp add: default-crun-def)
next
  show  $\exists \iota :: \text{nat} \Rightarrow \text{nat}. \text{strict-mono } \iota$ 
     $\wedge (\text{let in-slice} = (\lambda p. \text{key-of (hist-event } p) \in \text{cr-scope default-crun}$ 
       $\wedge \text{src-le (hist-coord } p) (\text{cr-frontier default-crun})));$ 
     $\text{cdc-in-slice} = \text{filter in-slice (cr-cdc default-crun)};$ 
     $\text{hist-in-slice} = \text{filter in-slice (cr-src-history default-crun)}$ 
     $\text{in } \forall i. i < \text{length cdc-in-slice} \longrightarrow$ 

```



```

       $\iota i < \text{length hist-in-slice}$ 
       $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! \iota i$ 
proof (intro exI[where  $x = \lambda n :: \text{nat. } n$ ] conjI)
  show strict-mono ( $\lambda n :: \text{nat. } n$ ) by (simp add: strict-mono-def)
next
  show let in-slice = ( $\lambda p$ . key-of (hist-event p)  $\in$  cr-scope default-crun
     $\wedge$  src-le (hist-coord p) (cr-frontier default-crun));
    cdc-in-slice = filter in-slice (cr-cdc default-crun);
    hist-in-slice = filter in-slice (cr-src-history default-crun)
  in  $\forall i. i < \text{length cdc-in-slice} \longrightarrow$ 
    ( $\lambda n :: \text{nat. } n$ )  $i < \text{length hist-in-slice}$ 
     $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! (\lambda n :: \text{nat. } n) i$ 
  by (simp add: default-crun-def Let-def)
qed
next
  show  $\forall k \in \text{cr-scope default-crun. } \forall ch \ c.$ 
    cr-resp default-crun k = Some ch
     $\wedge$  src-lt (cr-rcoord default-crun ch) c
     $\wedge$  src-le c (cr-frontier default-crun)
     $\longrightarrow$ 
    mset (source-key-coord-slice k c (cr-cdc default-crun))
    = mset (source-key-coord-slice k c (cr-src-history default-crun))
  by (simp add: default-crun-def source-key-coord-slice-def)
next
  show  $\forall ch \in \text{set (cr-chunks-list default-crun).}$ 
    src-le (cr-lwm default-crun ch) (cr-rcoord default-crun ch)
     $\wedge$  src-le (cr-rcoord default-crun ch) (cr-uwm default-crun ch)
     $\wedge$  ( $\forall k \in \text{cr-dom default-crun ch. } \exists m. \text{cr-rresult default-crun ch k} = \text{Some } m$ )
     $\wedge$  ( $\forall k \in \text{cr-dom default-crun ch. } \forall m.$ 
      cr-rresult default-crun ch k = Some m
       $\longleftrightarrow$  Refresh k m (cr-rcoord default-crun ch)
       $\in \text{set (cmodel.clean-prefix-of default-crun)}$ )
  by (simp add: default-crun-def)
next
  show  $\forall ch \in \text{set (cr-chunks-list default-crun).}$ 
    src-le (cr-rcoord default-crun ch) (cr-frontier default-crun)
  by (simp add: default-crun-def)
next
  show  $\forall ch \in \text{set (cr-chunks-list default-crun). } \forall k \in \text{cr-dom default-crun ch. } \forall m.$ 
    cr-rresult default-crun ch k = Some m
     $\longrightarrow m = \text{Src b0-es H-es (cr-rcoord default-crun ch) k}$ 
  by (simp add: default-crun-def)
next
  show  $\forall ch \in \text{set (cr-chunks-list default-crun). } \forall k \in \text{cr-dom default-crun ch.}$ 
    cr-rresult default-crun ch k = Some None
     $\longrightarrow \text{cmodel.row-absence-meaningful b0-es H-es default-crun ch k}$ 
  by (simp add: default-crun-def)
next
  show  $\forall p \in \text{set (cr-cdc default-crun).}$ 

```

```

      key-of (hist-event p) ∈ cr-scope default-crun
      ∧ src-le (hist-coord p) (cr-frontier default-crun)
      → Cdc (hist-coord p) (hist-event p)
      ∈ set (cmodel.clean-prefix-of default-crun)
    by (simp add: default-crun-def)
  next
  show ∀ e c. Cdc c e ∈ set (cmodel.clean-prefix-of default-crun)
    ∧ key-of e ∈ cr-scope default-crun
    ∧ src-le c (cr-frontier default-crun)
    → (c, e) ∈ set (cr-cdc default-crun)
  using cm-es-clean-prefix by simp
qed

```

39.15 Constructed existential bundles

Reader-facing $\exists$ bundles over the constructed *crun* carrier, with no run/chunk axiom: rejected runs exist, and a wellformed empty-scope run exists. The Layer 0 bundle (*layer0-negative-fixture-exists* in *DBLog-Virtual-Cuts.Layer01-Fixtures-Core*) is over a concrete source history with no run carrier.

```

theorem layer1-negative-fixtures-exist-constructed:
  ∃ (b0 :: nat → nat) (R :: crun) (H :: (nat, nat) src-history).
    ¬ cmodel.wellformed-dblog-run b0 R H
proof (intro exI[where x = b0-ov] exI[where x = crun-ov] exI[where x = H-ov])
  show ¬ cmodel.wellformed-dblog-run b0-ov crun-ov H-ov
    by (rule fix-overlap-rejected-constructed)
qed

```

```

theorem boundary-empty-scope-exists-constructed:
  ∃ (b0 :: nat → nat) (R :: crun) (H :: (nat, nat) src-history).
    cmodel.wellformed-dblog-run b0 R H ∧ cr-scope R = {}
proof (intro exI[where x = b0-es] exI[where x = default-crun] exI[where x =
H-es]
  conjI)
  show cmodel.wellformed-dblog-run b0-es default-crun H-es
    by (rule boundary-empty-scope-constructed)
  next
  show cr-scope default-crun = {} by (simp add: default-crun-def)
qed

```

A model exists for the *dblog-run-substrate* interface realizing every Layer 1 fixture run, so each fixture's rejection (or, for the empty-scope boundary, its acceptance) holds by construction — consistency-by-construction, resting on no axiom.

end

theory Layer2-Fixtures-Inst

```

imports
  Virtual-Cut
  Layer01-Witnesses-Inst
  Layer01-Virtual-Cut-Example-Inst
  Layer01-Witness-Topics-Inst
  Layer01-Fixtures-Inst
begin

```

40 Constructed Layer 2 fixtures

Constructed Layer 2 fixtures: the run-dependent Layer 2 fixture facts, established over the constructed run models of the Layer 0/1 witness and fixture theories. Every object mentioned is explicitly constructed; the theory introduces no axioms.

The Layer 2 fixture facts come in three kinds:

- **positive Layer 2 witness lifts**: the minimum-viable witness, the worked example, and the Topic 1/2 witnesses, established over the constructed run interpretations *wit-model* / *ex-model* / *t1-model* / *t2-model* and lifted through the *locale* Layer 2 theorem *dblog-run-substrate.wellformed-run-implies-virtual-cut*, which each interpretation specializes to its constructed run. (Its non-locale counterpart at the concrete *run* carrier is *Virtual-Cut.wellformed-run-implies-virtual-cut*.)
- **WF-clause rejection rows** (aliases of the Layer 1 negative fixtures, because Layer 2's theorem ranges over the same *wellformed-dblog-run* predicate). Re-exported here over the *constructed* Layer 1 fixtures of *Layer01-Fixtures-Inst*, i.e. over the *cmodel* interpretation and the *crun*... runs.
- **canonical-clean-prefix structural + frontier-off-by-one rows** (*fix-layer2-misordered-clean-prefix-rejected* and siblings). These are proved directly against the carrier-independent global *canonical-clean-prefix* / *cdc-event-replays* and mention no run carrier, so they need no constructed counterpart; they live in *Layer2-Fixtures-Core*.

40.1 Positive Layer 2 witness lifts over the constructed runs

Each lift specializes the locale Layer 2 theorem — *wellformed-run-implies-virtual-cut* under the *dblog-run-substrate* qualifier — to a constructed run via its interpretation, discharged by that run's constructed wellformedness fact. The run-model *-Inst* theories import the Layer 2 locale extension *DBLog-Run-Substrate-Layer2*, so each interpretation registers the qualified

Layer 2 fact at declaration time (e.g. *wellformed-run-implies-virtual-cut* under the *wit-model* qualifier). The conclusions name the constructed (*unit*) run () and the model accessors, so every object mentioned is explicitly constructed.

theorem *layer2-witness-virtual-cut-state-constructed*:
virtual-cut-state *b0-w* (*wit-model.clean-prefix-of* ())
 (*wit-cscope* ()) (*wit-cfrontier* ()) *H-w*
unfolding *virtual-cut-state-def*
using *wit-model.wellformed-run-implies-virtual-cut*
 [*OF wit-model-wellformed, unfolded virtual-cut-state-def*] .

theorem *layer2-worked-example-virtual-cut-state-constructed*:
virtual-cut-state *b0-ex* (*ex-model.clean-prefix-of* ())
 (*cscope-of* ()) (*cfrontier-of* ()) *H-ex*
unfolding *virtual-cut-state-def*
using *ex-model.wellformed-run-implies-virtual-cut*
 [*OF ex-model-wellformed, unfolded virtual-cut-state-def*] .

theorem *layer2-topic1-virtual-cut-state-constructed*:
virtual-cut-state *b0-t1* (*t1-model.clean-prefix-of* ())
 (*t1-cscope* ()) (*t1-cfrontier* ()) *H-t1*
unfolding *virtual-cut-state-def*
using *t1-model.wellformed-run-implies-virtual-cut*
 [*OF t1-model-wellformed, unfolded virtual-cut-state-def*] .

theorem *layer2-topic2-virtual-cut-state-constructed*:
virtual-cut-state *b0-t2* (*t2-model.clean-prefix-of* ())
 (*t2-cscope* ()) (*t2-cfrontier* ()) *H-t2*
unfolding *virtual-cut-state-def*
using *t2-model.wellformed-run-implies-virtual-cut*
 [*OF t2-model-wellformed, unfolded virtual-cut-state-def*] .

40.2 WF-clause rejection rows over the constructed Layer 1 fixtures

Layer 2 protects the hypotheses of the Layer 2 theorem; the rejection shapes are exactly the constructed *cmodel.wellformed-dblog-run* rejections of *Layer01-Fixtures-Inst*, re-exported here under their Layer 2 names.

theorem *fix-layer2-missing-cdc-event-rejected-constructed*:
 ¬ *cmodel.wellformed-dblog-run* *b0-md crun-md H-md*
by (*rule fix-miss-dom-rejected-constructed*)

theorem *fix-layer2-overlapping-chunk-scope-rejected-constructed*:
 ¬ *cmodel.wellformed-dblog-run* *b0-ov crun-ov H-ov*
by (*rule fix-overlap-rejected-constructed*)

theorem *fix-layer2-same-coordinate-cdc-order-rejected-constructed*:
 ¬ *cmodel.wellformed-dblog-run* *b0-do crun-do H-do*

41 Constructed Layer 3 fixtures

Constructed Layer 3 fixtures: the rejection, boundary, and faithfulness facts for the certificate / evidence / checker interface, established over explicit finite constructions. The constructed carriers (*l3f-cert* / *l3f-ev* / *l3f-raw* below) are small datatypes carrying exactly the certificate, evidence, and raw-observation values the fixtures exercise; the verifier, materialization, faithfulness, binding, and schema functions over them are ordinary **fun** definitions, so the theory introduces no axioms.

The fixture rows split into three kinds:

- **rejecting and boundary rows** (unmapped evidence, cert/evidence binding mismatch, an accept-without-materialization pair, a mismatched alternative run, faithfulness negatives, an empty-scope certificate, an unsupported-verdict evidence). None of these needs a *wellformed* materialized run, so they are established here over a single **global-interpretation** *cf* of *dblog-cert-substrate* (the Layer 3/4 certificate-substrate locale of *DBLog-Virtual-Cuts.DBLog-Cert-Substrate*) whose run side is a degenerate *unit* run (no chunks, no observed CDC, hence empty clean prefix) and whose certificate / evidence / observation carriers are the small finite datatypes. Every fixture fact holds by *simp* evaluation.
- the **non-circular checker-substrate obligation row** exhibits the *accepting* certificate satisfying the four primitive checker obligations, which requires a wellformed materialized run. It is established over the accepting interpretation *l3i* of *dblog-checker-substrate* from *Layer3-Witnesses-Inst* (run side *ex-model*, the constructed running-example run), reusing *ex-model-wellformed*.
- the **pure coherence / determinacy / empty-scope lemmas** — *layer3-cert-accessor-determinacy*, *layer3-mismatched-prefix-not-accessor-agree*, and *layer3-empty-scope-virtual-cut-boundary* — reference no certificate carrier at all: they range over the global cert-side definitions parametrically. They live in *DBLog-Virtual-Cuts.Layer3-Fixtures-Core*.

The definitions *l3-wrong-b0* / *l3-wrong-H* of *DBLog-Virtual-Cuts.Layer3-Fixtures-Core* are ordinary **definitions** and are reused here, together with their mismatch lemmas *l3-wrong-b0-neq* / *l3-wrong-H-neq*, for the base-state / history faithfulness negatives.

41.1 Constructed certificate / evidence / observation carriers

Small finite carriers holding exactly the certificate, evidence, and raw-observation values exercised by the Layer 3 fixtures.

```

datatype l3f-cert = CAcc | COther | CEmpty | CNoWit
datatype l3f-ev = EAcc | EUnmapped | ENoWit | EAlt | EUnknown
datatype l3f-raw = RawOk | RawBad

```

41.2 Constructed certificate-side accessor and predicate functions

The accepting certificate *CAcc* carries the running example’s scope and base coordinate; *CEmpty* carries the empty scope. The clean prefix is a fixed non-empty list, so it differs from the degenerate run’s empty clean prefix — the sole discriminator in the coherence-mismatch rows.

```

fun l3f-scope :: l3f-cert  $\Rightarrow$  nat set where
  l3f-scope CEmpty = {}
  | l3f-scope - = {100, 200, 300}

```

```

fun l3f-frontier :: l3f-cert  $\Rightarrow$  frontier where
  l3f-frontier - = c0

```

```

fun l3f-clean-prefix :: l3f-cert  $\Rightarrow$  (nat, nat) replay-event list where
  l3f-clean-prefix - = [Refresh (200::nat) (Some (10000::nat)) c0]

```

The verifier accepts the bound, schema-supported pairs ((*CAcc*, *EAcc*) and (*CNoWit*, *ENoWit*)), returns *Unsupported* for the schema-unsupported (*CAcc*, *EUnknown*), and rejects everything else.

```

fun l3f-verify :: l3f-cert  $\Rightarrow$  l3f-ev  $\Rightarrow$  verify-result where
  l3f-verify CAcc EAcc = Accept
  | l3f-verify CNoWit ENoWit = Accept
  | l3f-verify CAcc EUnknown = Unsupported
  | l3f-verify - - = Reject

```

```

fun l3f-bound :: l3f-cert  $\Rightarrow$  l3f-ev  $\Rightarrow$  bool where
  l3f-bound CAcc EAcc = True
  | l3f-bound CNoWit ENoWit = True
  | l3f-bound - - = False

```

```

fun l3f-schema :: l3f-cert  $\Rightarrow$  l3f-ev  $\Rightarrow$  bool where
  l3f-schema CAcc EAcc = True
  | l3f-schema CNoWit ENoWit = True
  | l3f-schema - - = False

```

The accepted same-evidence bundle (*CAcc*, *EAcc*) and the mismatched alternative bundle (*CAcc*, *EAlt*) both materialize to the degenerate run (). This makes the former a genuine accessor mismatch fixture: materialization succeeds before accessor agreement is tested. The accept-without-witness pair (*CNoWit*, *ENoWit*) materializes to nothing, modeling a checker whose acceptance is not backed by a witness run.

```

fun l3f-materializes :: l3f-cert  $\Rightarrow$  l3f-ev  $\Rightarrow$  unit  $\Rightarrow$  bool where

```

```

  l3f-materializes CAcc EAcc - = True
| l3f-materializes CAcc EAlt - = True
| l3f-materializes - - - = False

```

Faithful observation pins the running-example base state, history, and the distinguished good raw observation; every other evidence / base-state / history / raw combination is unfaithful.

```

fun l3f-faithful ::
  l3f-ev  $\Rightarrow$  (nat  $\rightarrow$  nat)  $\Rightarrow$  (nat, nat) src-history  $\Rightarrow$  l3f-raw  $\Rightarrow$  bool where
  l3f-faithful EAcc b0 H Raw = (b0 = b0-ex  $\wedge$  H = H-ex  $\wedge$  Raw = RawOk)
| l3f-faithful - - - = False

```

41.3 Constructed certificate substrate interpretation (rejecting fixtures)

The run side is a degenerate *unit* run: no chunks, no observed CDC events, empty source history. *dblog-cert-substrate* assumes nothing beyond the two inherited *chunks-list* obligations, which hold definitionally (*set* [] = {}, *distinct* []), so the interpretation introduces no axiom.

global-interpretation *cf*: *dblog-cert-substrate*

```

 $\lambda$ ::unit. {100::nat, 200, 300}           — scope-of
 $\lambda$ ::unit. c0                             — frontier-of
 $\lambda$ ::unit. ({ } :: nat set)                — chunks
 $\lambda$ ::unit. ([ ] :: (nat, nat) src-history) — src-history-of
 $\lambda$ ::unit.  $\lambda$ ::nat.  $\lambda$ ::nat. (None :: nat option option) — chunk-read-result
 $\lambda$ ::unit. ([ ] :: (src-coord  $\times$  (nat, nat) source-event) list) — cdc-events-of
 $\lambda$ ::unit. ([ ] :: nat list)               — chunks-list
 $\lambda$ ::unit.  $\lambda$ ::nat. ({ } :: nat set)        — chunk-domain
 $\lambda$ ::unit.  $\lambda$ ::nat. (None :: nat option)    — responsible-chunk
 $\lambda$ ::unit.  $\lambda$ ::nat. c0                     — chunk-lower-watermark
 $\lambda$ ::unit.  $\lambda$ ::nat. c0                     — chunk-upper-watermark
 $\lambda$ ::unit.  $\lambda$ ::nat. c0                     — chunk-read-coordinate
l3f-scope l3f-frontier l3f-clean-prefix
l3f-verify Accept l3f-faithful l3f-materializes l3f-bound l3f-schema
by unfold-locales simp-all

```

The degenerate run has an empty clean prefix (no chunk reads, no CDC replays).

lemma *cf-clean-prefix-empty*: *cf.clean-prefix-of* () = []
unfolding *cf.clean-prefix-of-def canonical-clean-prefix-def cdc-event-replays-def*
by (*simp add: List.map-filter-simps*)

lemma *l3f-clean-prefix-nonempty*: *l3f-clean-prefix* C $\neq$ []
by *simp*

41.4 Certificate/evidence binding failures

Unmapped evidence: rejected, not a valid checker input (not bound), and materializes to no run.

theorem *layer3-unmapped-certificate-evidence-rejected-constructed*:
 $l3f\text{-verify } CAcc \ EUnmapped \neq \text{Accept}$
 $\wedge \neg cf.\text{checker-input-ok } CAcc \ EUnmapped$
 $\wedge (\forall R. \neg l3f\text{-materializes } CAcc \ EUnmapped \ R)$
by (*simp add: cf.checker-input-ok-def*)

Certificate/evidence binding mismatch: a different certificate against the positive evidence is rejected and not a valid checker input (not bound).

theorem *layer3-cert-evidence-binding-mismatch-rejected-constructed*:
 $l3f\text{-verify } COther \ EAcc \neq \text{Accept}$
 $\wedge \neg cf.\text{checker-input-ok } COther \ EAcc$
by (*simp add: cf.checker-input-ok-def*)

41.5 Materialization and non-circular substrate

Accept-without-witness: the pair (*CNoWit*, *ENoWit*) is accepted and a valid checker input, yet materializes to no run. So it cannot satisfy the checker-substrate materialization obligation — assuming it would yields a contradiction.

lemma *l3f-nowit-checker-input-ok*: $cf.\text{checker-input-ok } CNoWit \ ENoWit$
by (*simp add: cf.checker-input-ok-def*)

theorem *layer3-accepted-but-witness-undefined-blocks-substrate-constructed*:
assumes *materialization-obligation*:
 $\llbracket l3f\text{-verify } CNoWit \ ENoWit = \text{Accept};$
 $cf.\text{checker-input-ok } CNoWit \ ENoWit \rrbracket$
 $\implies \exists R. l3f\text{-materializes } CNoWit \ ENoWit \ R$
shows *False*
proof –
have $\exists R. l3f\text{-materializes } CNoWit \ ENoWit \ R$
using *materialization-obligation l3f-nowit-checker-input-ok* **by** *simp*
thus *False* **by** *simp*
qed

The accepting certificate satisfies the four primitive checker obligations over a wellformed materialized run — the non-circular checker-spec row, established over the accepting interpretation *l3i* of *Layer3-Witnesses-Inst* (run side *ex-model*), so the constructed materialization is the wellformed running-example run (*ex-model-wellformed*). The two certificate-side predicates are stated component-wise: *checker-input-ok* as *l3i-ev-bound* $\wedge$ *l3i-schema* and *cert-accessors-agree* as the three accessor equalities — their *dblog-cert-substrate* definitions unfolded, since those definitions live in the parent locale and so carry no *l3i*.-qualified name (the *cert-loc* interpretation that exposes them is local to its defining theory).

theorem *layer3-non-circular-checker-spec-obligations-constructed:*

$l3i\text{-verify } () () = \text{Accept}$
 $\wedge (l3i\text{-ev-bound } () () \wedge l3i\text{-schema } () ())$
 $\wedge (\exists R. l3i\text{-materializes } () () R)$
 $\wedge (\forall R. l3i\text{-materializes } () () R$
 $\quad \longrightarrow cscope\text{-of } () = cscope\text{-of } R$
 $\quad \wedge cfrontier\text{-of } () = cfrontier\text{-of } R$
 $\quad \wedge ex\text{-model.clean-prefix-of } () = ex\text{-model.clean-prefix-of } R)$
 $\wedge (\forall R \ b0 \ H \ Raw.$
 $\quad l3i\text{-materializes } () () R$
 $\quad \longrightarrow l3i\text{-faithful } () \ b0 \ H \ Raw$
 $\quad \longrightarrow ex\text{-model.wellformed-dblog-run } b0 \ R \ H)$
using *ex-model-wellformed*
by (*auto simp add: l3i-verify-def l3i-ev-bound-def l3i-schema-def*
l3i-materializes-def l3i-faithful-def)

41.6 Coherence and determinacy

A mismatched alternative bundle: rejected, yet materializing to the degenerate run, whose empty clean prefix differs from the certificate's clean prefix — so the certificate accessors do not agree with the run and the pair is not coherent.

theorem *layer3-two-evidence-bundle-determinacy-fixture-constructed:*

$l3f\text{-verify } CAcc \ EAlt \neq \text{Accept}$
 $\wedge l3f\text{-materializes } CAcc \ EAlt ()$
 $\wedge \neg cf.\text{cert-run-coherent } CAcc \ EAlt ()$
unfolding *cf.cert-run-coherent-def cf.cert-accessors-agree-def*
by (*simp add: cf-clean-prefix-empty*)

theorem *layer3-cert-run-coherence-mismatch-rejected-constructed:*

$l3f\text{-materializes } CAcc \ EAcc ()$
 $\wedge \neg cf.\text{cert-accessors-agree } CAcc ()$
 $\wedge \neg cf.\text{cert-run-coherent } CAcc \ EAcc ()$
unfolding *cf.cert-run-coherent-def cf.cert-accessors-agree-def*
by (*simp add: cf-clean-prefix-empty*)

— Alias of *layer3-cert-run-coherence-mismatch-rejected-constructed*, mirroring the dual-named Layer 3 fixture-table row.

theorem *layer3-same-evidence-mismatched-run-not-coherent-constructed:*

$\neg cf.\text{cert-run-coherent } CAcc \ EAcc ()$
using *layer3-cert-run-coherence-mismatch-rejected-constructed* **by** *blast*

41.7 Faithfulness negatives

The accepting certificate is a valid checker input, but acceptance does not certify the raw observation: a wrong raw observation, a wrong base state, or a wrong history all fail faithfulness.

theorem *layer3-raw-observation-faithfulness-negative-constructed:*

$l3f\text{-verify } CAcc \ EAcc = \text{Accept}$
 $\wedge cf.\text{checker-input-ok } CAcc \ EAcc$
 $\wedge \neg l3f\text{-faithful } EAcc \ b0\text{-ex } H\text{-ex } RawBad$
by (*simp add: cf.checker-input-ok-def*)

theorem *layer3-raw-base-state-mismatch-rejected-constructed:*
 $\neg l3f\text{-faithful } EAcc \ l3\text{-wrong-b0 } H\text{-ex } RawOk$
using *l3-wrong-b0-neq* **by** *simp*

theorem *layer3-wrong-history-mismatch-rejected-constructed:*
 $\neg l3f\text{-faithful } EAcc \ b0\text{-ex } l3\text{-wrong-H } RawOk$
using *l3-wrong-H-neq* **by** *simp*

41.8 Empty-scope boundary

An empty-scope certificate vacuously satisfies the cert-side virtual cut, yet has empty scope, so it is not a positive witness; the accepting certificate's scope is non-empty.

theorem *layer3-empty-scope-boundary-not-positive-witness-constructed:*
 $cf.\text{virtual-cut } b0\text{-ex } CEmpty \ H\text{-ex}$
 $\wedge l3f\text{-scope } CEmpty = \{\}$
 $\wedge l3f\text{-scope } CAcc \neq \{\}$
unfolding *cf.virtual-cut-def virtual-cut-state-def restrict-def*
by *simp*

41.9 Unknown or ignored evidence fields

Evidence outside the source-side schema yields an *Unsupported* verdict, is not accepted, and is not a valid checker input.

theorem *layer3-unknown-ignored-field-rejected-constructed:*
 $l3f\text{-verify } CAcc \ EUnknown = \text{Unsupported}$
 $\wedge l3f\text{-verify } CAcc \ EUnknown \neq \text{Accept}$
 $\wedge \neg cf.\text{checker-input-ok } CAcc \ EUnknown$
by (*simp add: cf.checker-input-ok-def*)

41.10 Fixture bundle

The constructed Layer 3 negative and boundary fixture bundle: the nine rejection / boundary facts above in one statement, all established over the construction.

theorem *layer3-fixture-bundle-constructed:*
 $l3f\text{-verify } CAcc \ EUnmapped \neq \text{Accept}$
 $\wedge (\forall R. \neg l3f\text{-materializes } CAcc \ EUnmapped \ R)$
 $\wedge l3f\text{-verify } CAcc \ EAlt \neq \text{Accept}$
 $\wedge \neg cf.\text{cert-run-coherent } CAcc \ EAcc \ ()$
 $\wedge \neg l3f\text{-faithful } EAcc \ b0\text{-ex } H\text{-ex } RawBad$
 $\wedge \neg l3f\text{-faithful } EAcc \ l3\text{-wrong-b0 } H\text{-ex } RawOk$

```

 $\wedge$  l3f-scope CEmpty = {}
 $\wedge$  l3f-verify CAcc EUnknown = Unsupported
 $\wedge$  l3f-verify COther EAcc  $\neq$  Accept
 $\wedge$   $\neg$  l3f-faithful EAcc b0-ex l3-wrong-H RawOk
using layer3-unmapped-certificate-evidence-rejected-constructed
      layer3-two-evidence-bundle-determinacy-fixture-constructed
      layer3-cert-run-coherence-mismatch-rejected-constructed
      layer3-raw-observation-faithfulness-negative-constructed
      layer3-raw-base-state-mismatch-rejected-constructed
      layer3-empty-scope-boundary-not-positive-witness-constructed
      layer3-unknown-ignored-field-rejected-constructed
      layer3-cert-evidence-binding-mismatch-rejected-constructed
      layer3-wrong-history-mismatch-rejected-constructed
by blast

```

The certificate-dependent Layer 3 fixture rows hold over the construction — the rejecting / boundary / faithfulness rows over the *cf* interpretation, the non-circular checker-spec row over the *l3i* interpretation. Together with the carrier-independent pure coherence / determinacy / empty-scope lemmas of *DBLog-Virtual-Cuts.Layer3-Fixtures-Core*, the whole Layer 3 fixture bundle is established with no axioms anywhere.

end

```

theory Layer3-Defect-Regressions
  imports Layer3-Fixtures-Inst
begin

```

42 D15 regression: the accessor fixture reaches comparison

```

lemma d15-accessor-mismatch-is-present:
   $\neg$  cf.cert-accessors-agree CAcc ()
  unfolding cf.cert-accessors-agree-def
  by (simp add: cf-clean-prefix-empty)

```

```

lemma d15-materializes-then-fails-accessor-agreement:
  l3f-verify CAcc EAcc = Accept
   $\wedge$  l3f-materializes CAcc EAcc ()
   $\wedge$   $\neg$  cf.cert-accessors-agree CAcc ()
   $\wedge$   $\neg$  cf.cert-run-coherent CAcc EAcc ()
  unfolding cf.cert-run-coherent-def
  using d15-accessor-mismatch-is-present by simp

```

end

```

theory Layer4-Fixtures-Inst

```

```

imports
  Layer4-Fixtures-Core
  DBLog-Cert-Substrate
  Layer01-Virtual-Cut-Example-Inst
  Layer4-Whole-Table
begin

```

43 Constructed Layer 4 fixtures

Constructed Layer 4 fixtures: the negative, boundary, and run-ambiguity facts for the Layer 4 whole-table claim, established over explicit constructions with no axioms.

Each Layer 4 fixture pins exactly the three certificate accessors *scope* / *frontier* / *clean-prefix* (the run-ambiguity fixture additionally pins *materializes-run* / *chunks* / *responsible-chunk* on two runs). Everything else — *virtual-cut*, *whole-table-claim-scope*, and the pure Layer 4 predicates *touched-between* / *anchor-domain* / *table-scope-between* / *all-post-anchor-changes-covered* / *no-unclaimed-replay-keys* / *replay-keys* — is a global definition over those accessor *values*, plus the cert-side locale definitions of *dblog-cert-substrate*. So a single **global-interpretation** *cf* of *dblog-cert-substrate* over a finite certificate datatype whose *scope* / *frontier* / *clean-prefix* functions carry exactly the fixture values suffices; the run-side ambiguity fixture gets a second interpretation *amb* over a two-run carrier.

- The carrier-independent source-side lemmas of *Layer4-Fixtures-Core* and the anchor facts of *Layer4-Witnesses-Core*, which range over the plain replay-prefix / source-history **definitions** (e.g. *l4-missing-insert-scoped-virtual-cut-state*, *l4-anchor-domain*, *Src-b0-ex-l4-delete-touch-ec4*), reference no certificate accessor and are reused verbatim by import.
- The running-example clean prefix — shared by the post-anchor-delete and run-ambiguity fixtures — is the constructed *ex-model.clean-prefix-of* () of *Layer01-Virtual-Cut-Example-Inst*, with *exm-apply* / *ex-model-virtual-cut-holds* reused.
- The *H-ex*-based *touched-between* fact over the explicit frontier *ec4* (*touched-H-ex-anchor-ec4*) is reused from *Layer4-Witnesses-Core*.

43.1 Constructed certificate carrier

A finite certificate datatype with one constructor per Layer 4 fixture, carrying exactly the *scope* / *frontier* / *clean-prefix* values the fixtures exercise.

```

datatype lf-cert =
  CMissingInsert | CDeleteTouch | CEmpty | CClaimRefresh | CAfterFrontier

```

| *CUpdateRefresh* | *CUnclaimed* | *CB0Anchor* | *CSameBoundary* | *CAmbig*

```
fun l4f-scope :: l4f-cert  $\Rightarrow$  nat set where
  l4f-scope CMissingInsert = {100, 200}
| l4f-scope CDeleteTouch   = {100, 200, 300}
| l4f-scope CEmpty         = {}
| l4f-scope CClaimRefresh  = {100, 200, 300}
| l4f-scope CAfterFrontier = {}
| l4f-scope CUpdateRefresh = {100, 200, 300}
| l4f-scope CUnclaimed     = {100, 200, 300}
| l4f-scope CB0Anchor      = {700}
| l4f-scope CSameBoundary  = {800, 900}
| l4f-scope CAmbig         = {100, 200, 300}
```

```
fun l4f-frontier :: l4f-cert  $\Rightarrow$  frontier where
  l4f-frontier CEmpty       = ec1
| l4f-frontier CAfterFrontier = ec1
| l4f-frontier -              = ec4
```

```
fun l4f-clean-prefix :: l4f-cert  $\Rightarrow$  (nat, nat) replay-event list where
  l4f-clean-prefix CMissingInsert = l4-missing-insert-prefix
| l4f-clean-prefix CDeleteTouch   = ex-model.clean-prefix-of ()
| l4f-clean-prefix CEmpty         = []
| l4f-clean-prefix CClaimRefresh  = l4-missing-insert-prefix
| l4f-clean-prefix CAfterFrontier = []
| l4f-clean-prefix CUpdateRefresh = l4-update-refresh-prefix
| l4f-clean-prefix CUnclaimed     = l4-unclaimed-prefix
| l4f-clean-prefix CB0Anchor      = l4-b0-anchor-prefix
| l4f-clean-prefix CSameBoundary  = l4-same-boundary-prefix
| l4f-clean-prefix CAmbig         = ex-model.clean-prefix-of ()
```

43.2 Reusable helpers

The running-example clean prefix replay keys and virtual-cut state, over the constructed *ex-model.clean-prefix-of* (). The *H-ex* post-anchor touched set *touched-H-ex-anchor-ec4* is imported from *Layer4-Witnesses-Core*.

```
lemma rep-ex-replay-keys:
  replay-keys (ex-model.clean-prefix-of ()) = {100, 200, 300}
unfolding replay-keys-def exm-clean-prefix by auto
```

```
lemma rep-ex-virtual-cut-state:
  virtual-cut-state b0-ex (ex-model.clean-prefix-of ()) {100, 200, 300} ec4 H-ex
using ex-model-virtual-cut-holds
by (simp add: virtual-cut-state-def exm-scope exm-frontier)
```

43.3 Constructed certificate substrate interpretation (cert-accessor fixtures)

The run side is a degenerate *unit* run (no chunks, no observed CDC, empty source history); *dblog-cert-substrate* assumes nothing beyond the two inherited *chunks-list* obligations (*set* $\square = \{\}$, *distinct* $\square$), so the interpretation introduces no axiom. The verifier / materialization / faithfulness / binding / schema parameters are not used by the cert-accessor fixtures and are supplied as trivial constants.

global-interpretation *cf*: *dblog-cert-substrate*

$\lambda::unit. (\{\} :: nat\ set)$	— <i>scope-of</i>
$\lambda::unit. c0$	— <i>frontier-of</i>
$\lambda::unit. (\{\} :: nat\ set)$	— <i>chunks</i>
$\lambda::unit. (\square :: (nat, nat)\ src-history)$	— <i>src-history-of</i>
$\lambda::unit. \lambda::nat. \lambda::nat. (None :: nat\ option\ option)$	— <i>chunk-read-result</i>
$\lambda::unit. (\square :: (src-coord \times (nat, nat)\ source-event)\ list)$	— <i>cdc-events-of</i>
$\lambda::unit. (\square :: nat\ list)$	— <i>chunks-list</i>
$\lambda::unit. \lambda::nat. (\{\} :: nat\ set)$	— <i>chunk-domain</i>
$\lambda::unit. \lambda::nat. (None :: nat\ option)$	— <i>responsible-chunk</i>
$\lambda::unit. \lambda::nat. c0$	— <i>chunk-lower-watermark</i>
$\lambda::unit. \lambda::nat. c0$	— <i>chunk-upper-watermark</i>
$\lambda::unit. \lambda::nat. c0$	— <i>chunk-read-coordinate</i>
$l4f-scope\ l4f-frontier\ l4f-clean-prefix$	
$\lambda::l4f-cert. \lambda::unit. Reject$	— <i>verify</i>
<i>Accept</i>	— <i>accept-value</i>
$\lambda::unit. \lambda::(nat \multimap nat). \lambda::(nat, nat)\ src-history. \lambda::unit. False$	— <i>faithful-source-observation</i>
$\lambda::l4f-cert. \lambda::unit. \lambda::unit. False$	— <i>materializes-run</i>
$\lambda::l4f-cert. \lambda::unit. False$	— <i>evidence-bound-to-cert</i>
$\lambda::l4f-cert. \lambda::unit. False$	— <i>source-side-schema-supported</i>
by <i>unfold-locales simp-all</i>	

43.4 Post-anchor insert not covered (constructed)

lemma *cf-missing-insert-virtual-cut*:

cf.virtual-cut b0-ex CMissingInsert H-ex

unfolding *cf.virtual-cut-def virtual-cut-state-def*

using *l4-missing-insert-scoped-virtual-cut-state*

by (*simp add: virtual-cut-state-def*)

lemma *cf-missing-insert-no-unclaimed*:

no-unclaimed-replay-keys (l4f-clean-prefix CMissingInsert) (l4f-scope CMissingInsert)

unfolding *no-unclaimed-replay-keys-def*

using *l4-missing-insert-replay-keys* **by** *simp*

lemma *cf-missing-insert-key-touched*:

(300::nat) $\in$ touched-between H-ex l4-anchor (l4f-frontier CMissingInsert)

using *touched-H-ex-anchor-ec4* **by** *simp*

lemma *cf-missing-insert-not-covered*:
 $\neg$ *all-post-anchor-changes-covered*
H-ex l4-anchor (l4f-frontier CMissingInsert) (l4f-scope CMissingInsert)
using *cf-missing-insert-key-touched*
unfolding *all-post-anchor-changes-covered-def*
by *auto*

lemma *cf-missing-insert-not-whole-table*:
 $\neg$ *cf.whole-table-claim-scope b0-ex CMissingInsert H-ex l4-anchor*
using *cf-missing-insert-not-covered*
unfolding *cf.whole-table-claim-scope-def*
by *blast*

lemma *cf-missing-insert-unrestricted-equality-fails*:
Apply (l4f-clean-prefix CMissingInsert)
 $\neq$ *Src b0-ex H-ex (l4f-frontier CMissingInsert)*
proof
assume *eq*:
Apply (l4f-clean-prefix CMissingInsert)
 $=$ *Src b0-ex H-ex (l4f-frontier CMissingInsert)*
hence *Apply (l4f-clean-prefix CMissingInsert) (300::nat)*
 $=$ *Src b0-ex H-ex (l4f-frontier CMissingInsert) 300*
by *simp*
thus *False*
using *l4-missing-insert-apply-300 Src-b0-ex-H-ex-ec4-300* **by** *simp*
qed

theorem *layer4-post-anchor-insert-not-covered-fixture-constructed*:
cf.virtual-cut b0-ex CMissingInsert H-ex
 $\wedge$ *no-unclaimed-replay-keys*
(l4f-clean-prefix CMissingInsert) (l4f-scope CMissingInsert)
 $\wedge$ $(300::nat) \in$ *touched-between H-ex l4-anchor (l4f-frontier CMissingInsert)*
 $\wedge$ $300 \notin$ *l4f-scope CMissingInsert*
 $\wedge$ $300 \notin$ *replay-keys (l4f-clean-prefix CMissingInsert)*
 $\wedge$ $\neg$ *all-post-anchor-changes-covered*
H-ex l4-anchor (l4f-frontier CMissingInsert) (l4f-scope CMissingInsert)
 $\wedge$ $\neg$ *cf.whole-table-claim-scope b0-ex CMissingInsert H-ex l4-anchor*
 $\wedge$ *Apply (l4f-clean-prefix CMissingInsert) 300 = None*
 $\wedge$ *Src b0-ex H-ex (l4f-frontier CMissingInsert) 300 = Some 3500*
 $\wedge$ *Apply (l4f-clean-prefix CMissingInsert)*
 $\neq$ *Src b0-ex H-ex (l4f-frontier CMissingInsert)*
using *cf-missing-insert-virtual-cut*
cf-missing-insert-no-unclaimed
cf-missing-insert-key-touched
cf-missing-insert-not-covered
cf-missing-insert-not-whole-table
cf-missing-insert-unrestricted-equality-fails
l4-missing-insert-replay-keys

l_4 -missing-insert-apply-300
 Src-b0-ex-H-ex-ec4-300
 by simp

43.5 Post-anchor delete as conservative touch coverage (constructed)

lemma *cf-delete-touch-apply-frontier*:
 Apply (l_4 -clean-prefix CDeleteTouch) =
 (λk . if $k = (100::nat)$ then Some (3000::nat)
 else if $k = 200$ then Some 12000
 else if $k = 300$ then Some 3500
 else None)
 using *exm-apply* by simp

lemma *cf-delete-touch-unrestricted-equality*:
 Apply (l_4 -clean-prefix CDeleteTouch)
 = Src b0-ex l_4 -delete-touch-H (l_4 -frontier CDeleteTouch)
 using *cf-delete-touch-apply-frontier* Src-b0-ex- l_4 -delete-touch-ec4 by simp

lemma *cf-delete-touch-virtual-cut*:
 cf .virtual-cut b0-ex CDeleteTouch l_4 -delete-touch-H
unfolding *cf.virtual-cut-def* *virtual-cut-state-def*
 using *cf-delete-touch-unrestricted-equality* by simp

lemma *cf-delete-touch-replay-keys*:
 replay-keys (l_4 -clean-prefix CDeleteTouch) = {100, 200, 300}
 using *rep-ex-replay-keys* by simp

lemma *cf-delete-touch-no-unclaimed*:
 no-unclaimed-replay-keys (l_4 -clean-prefix CDeleteTouch) (l_4 -scope CDelete-
 Touch)
unfolding *no-unclaimed-replay-keys-def*
 using *cf-delete-touch-replay-keys* by simp

lemma *cf-delete-touch-anchor-complete*:
 anchor-complete-at-boundary
 b0-ex l_4 -delete-touch-H l_4 -anchor (l_4 -scope CDeleteTouch)
unfolding *anchor-complete-at-boundary-def*
 using *l_4-delete-touch-anchor-domain* by simp

lemma *cf-delete-touch-key-touched*:
 (400::nat) $\in$ touched-between
 l_4 -delete-touch-H l_4 -anchor (l_4 -frontier CDeleteTouch)
proof –
 have *in-H*: (ec2, Delete (400::nat)) $\in$ set l_4 -delete-touch-H
 by (simp add: l_4 -delete-touch-H-def)
 have *after-anchor*: l_4 -anchor < ec2
 using *ec1-less-ec2* by (simp add: l_4 -anchor-def)

```

have before-frontier:  $ec2 \leq l4f\text{-frontier } CDeleteTouch$ 
using ec2-less-eq-ec4 by simp
show ?thesis
unfolding touched-between-def
using in-H after-anchor before-frontier
by (intro CollectI exI [where  $x=ec2$ ]
      exI [where  $x=Delete (400::nat)$ ])) auto
qed

lemma cf-delete-touch-not-covered:
   $\neg all\text{-post-anchor-changes-covered}$ 
     $l4\text{-delete-touch-H } l4\text{-anchor } (l4f\text{-frontier } CDeleteTouch) (l4f\text{-scope } CDelete\text{-}$ 
Touch)
using cf-delete-touch-key-touched
unfolding all-post-anchor-changes-covered-def
by auto

lemma cf-delete-touch-not-whole-table:
   $\neg cf.\text{whole-table-claim-scope } b0\text{-ex } CDeleteTouch \ l4\text{-delete-touch-H } l4\text{-anchor}$ 
using cf-delete-touch-not-covered
unfolding cf.whole-table-claim-scope-def
by blast

theorem layer4-post-anchor-delete-not-covered-fixture-constructed:
   $cf.\text{virtual-cut } b0\text{-ex } CDeleteTouch \ l4\text{-delete-touch-H}$ 
 $\wedge l4\text{-anchor} \leq l4f\text{-frontier } CDeleteTouch$ 
 $\wedge anchor\text{-complete-at-boundary}$ 
 $b0\text{-ex } l4\text{-delete-touch-H } l4\text{-anchor } (l4f\text{-scope } CDeleteTouch)$ 
 $\wedge no\text{-unclaimed-replay-keys}$ 
 $(l4f\text{-clean-prefix } CDeleteTouch) (l4f\text{-scope } CDeleteTouch)$ 
 $\wedge (400::nat) \in touched\text{-between } l4\text{-delete-touch-H } l4\text{-anchor}$ 
 $(l4f\text{-frontier } CDeleteTouch)$ 
 $\wedge 400 \notin l4f\text{-scope } CDeleteTouch$ 
 $\wedge 400 \notin replay\text{-keys } (l4f\text{-clean-prefix } CDeleteTouch)$ 
 $\wedge Apply (l4f\text{-clean-prefix } CDeleteTouch) \ 400 = None$ 
 $\wedge Src \ b0\text{-ex } l4\text{-delete-touch-H } (l4f\text{-frontier } CDeleteTouch) \ 400 = None$ 
 $\wedge Apply (l4f\text{-clean-prefix } CDeleteTouch)$ 
 $= Src \ b0\text{-ex } l4\text{-delete-touch-H } (l4f\text{-frontier } CDeleteTouch)$ 
 $\wedge \neg all\text{-post-anchor-changes-covered}$ 
 $l4\text{-delete-touch-H } l4\text{-anchor } (l4f\text{-frontier } CDeleteTouch) (l4f\text{-scope } CDelete\text{-}$ 
Touch)
 $\wedge \neg cf.\text{whole-table-claim-scope } b0\text{-ex } CDeleteTouch \ l4\text{-delete-touch-H } l4\text{-anchor}$ 
using cf-delete-touch-virtual-cut
cf-delete-touch-anchor-complete
cf-delete-touch-no-unclaimed
cf-delete-touch-key-touched
cf-delete-touch-replay-keys
cf-delete-touch-apply-frontier
Src-b0-ex-l4-delete-touch-ec4-400

```

$cf_delete_touch_unrestricted_equality$
 $cf_delete_touch_not_covered$
 $cf_delete_touch_not_whole_table$
 $ec1_le_ec4$
by ($simp$ $add: less_eq_src_coord_def$ $l4_anchor_def$)

43.6 Empty anchor-domain boundary (constructed)

lemma cf_empty_apply :
 $Apply$ ($l4f_clean_prefix$ $CEmpty$) = (λ -. $None$)
by ($simp$ $add: Apply_def$)

lemma $cf_empty_source_at_frontier$:
 Src $l4_empty_b0$ $l4_empty_H$ ($l4f_frontier$ $CEmpty$) = (λ -. $None$)
by ($simp$ $add: Src_def$ $latest_src_event_def$ $l4_empty_b0_def$ $l4_empty_H_def$)

lemma $cf_empty_unrestricted_equality$:
 $Apply$ ($l4f_clean_prefix$ $CEmpty$)
 = Src $l4_empty_b0$ $l4_empty_H$ ($l4f_frontier$ $CEmpty$)
using cf_empty_apply $cf_empty_source_at_frontier$ **by** $simp$

lemma $cf_empty_virtual_cut$:
 $cf.virtual_cut$ $l4_empty_b0$ $CEmpty$ $l4_empty_H$
unfolding $cf.virtual_cut_def$ $virtual_cut_state_def$
using $cf_empty_unrestricted_equality$ **by** $simp$

lemma $cf_empty_no_unclaimed$:
 $no_unclaimed_replay_keys$ ($l4f_clean_prefix$ $CEmpty$) ($l4f_scope$ $CEmpty$)
by ($simp$ $add: no_unclaimed_replay_keys_def$ $replay_keys_def$)

lemma $cf_empty_anchor_complete$:
 $anchor_complete_at_boundary$ $l4_empty_b0$ $l4_empty_H$ $l4_anchor$ ($l4f_scope$ $CEmpty$)
by ($simp$ $add: anchor_complete_at_boundary_def$ $l4_empty_anchor_domain$)

lemma $touched_between_nil$: $touched_between$ $\square$ b f = $\{\}$
by ($simp$ $add: touched_between_def$)

lemma $cf_empty_touched_between$:
 $touched_between$ $l4_empty_H$ $l4_anchor$ ($l4f_frontier$ $CEmpty$) = $\{\}$
by ($simp$ $add: l4_empty_H_def$ $touched_between_nil$)

lemma $cf_empty_table_scope_between$:
 $table_scope_between$ $l4_empty_b0$ $l4_empty_H$ $l4_anchor$ ($l4f_frontier$ $CEmpty$) = $\{\}$
unfolding $table_scope_between_def$
using $l4_empty_anchor_domain$ $cf_empty_touched_between$
by $simp$

lemma $cf_empty_all_post_anchor_changes_covered$:

$all_post_anchor_changes_covered$
 $l4_empty_H \ l4_anchor \ (l4f_frontier \ CEmpty) \ (l4f_scope \ CEmpty)$
by (*simp add: all-post-anchor-changes-covered-def l4-empty-H-def touched-between-nil*)

lemma *cf-empty-anchor-le-frontier*:
 $l4_anchor \leq l4f_frontier \ CEmpty$
by (*simp add: l4-anchor-def less-eq-src-coord-def src-le-refl*)

lemma *cf-empty-whole-table-claim-scope*:
 $cf.whole_table_claim_scope \ l4_empty_b0 \ CEmpty \ l4_empty_H \ l4_anchor$
unfolding *cf.whole-table-claim-scope-def*
using *cf-empty-anchor-le-frontier*
 $cf_empty_anchor_complete$
 $cf_empty_all_post_anchor_changes_covered$
 $cf_empty_no_unclaimed$
by *blast*

theorem *layer4-empty-anchor-domain-boundary-fixture-constructed*:
 $anchor_domain \ l4_empty_b0 \ l4_empty_H \ l4_anchor = \{\}$
 $\wedge \ touched_between \ l4_empty_H \ l4_anchor \ (l4f_frontier \ CEmpty) = \{\}$
 $\wedge \ table_scope_between \ l4_empty_b0 \ l4_empty_H \ l4_anchor \ (l4f_frontier \ CEmpty) = \{\}$
 $\wedge \ l4f_scope \ CEmpty = \{\}$
 $\wedge \ l4f_clean_prefix \ CEmpty = []$
 $\wedge \ cf.virtual_cut \ l4_empty_b0 \ CEmpty \ l4_empty_H$
 $\wedge \ cf.whole_table_claim_scope \ l4_empty_b0 \ CEmpty \ l4_empty_H \ l4_anchor$
 $\wedge \ Apply \ (l4f_clean_prefix \ CEmpty)$
 $= Src \ l4_empty_b0 \ l4_empty_H \ (l4f_frontier \ CEmpty)$
using *l4-empty-anchor-domain*
 $cf_empty_touched_between$
 $cf_empty_table_scope_between$
 $cf_empty_virtual_cut$
 $cf_empty_whole_table_claim_scope$
 $cf_empty_unrestricted_equality$
by *simp*

43.7 Claim scope broader than materialized replay coverage (constructed)

lemma *cf-claim-refresh-replay-keys*:
 $replay_keys \ (l4f_clean_prefix \ CClaimRefresh) = \{100, 200\}$
using *l4-missing-insert-replay-keys* **by** *simp*

lemma *cf-claim-refresh-no-unclaimed*:
 $no_unclaimed_replay_keys \ (l4f_clean_prefix \ CClaimRefresh) \ (l4f_scope \ CClaimRefresh)$
unfolding *no-unclaimed-replay-keys-def* **using** *cf-claim-refresh-replay-keys* **by** *simp*

lemma *cf-claim-refresh-anchor-complete*:
anchor-complete-at-boundary b0-ex H-ex l4-anchor (l4f-scope CClaimRefresh)
unfolding *anchor-complete-at-boundary-def* **using** *l4-anchor-domain* **by** *simp*

lemma *cf-claim-refresh-all-post*:
all-post-anchor-changes-covered H-ex l4-anchor
(l4f-frontier CClaimRefresh) (l4f-scope CClaimRefresh)
unfolding *all-post-anchor-changes-covered-def* **using** *touched-H-ex-anchor-ec4*
by *simp*

lemma *cf-claim-refresh-anchor-le-frontier*:
l4-anchor ≤ l4f-frontier CClaimRefresh
using *ec1-le-ec4* **by** (*simp add: l4-anchor-def less-eq-src-coord-def*)

lemma *cf-claim-refresh-whole-table*:
cf.whole-table-claim-scope b0-ex CClaimRefresh H-ex l4-anchor
unfolding *cf.whole-table-claim-scope-def*
using *cf-claim-refresh-anchor-le-frontier cf-claim-refresh-anchor-complete*
cf-claim-refresh-all-post cf-claim-refresh-no-unclaimed
by *blast*

lemma *cf-claim-refresh-not-virtual-cut*:
 $\neg$ *cf.virtual-cut b0-ex CClaimRefresh H-ex*
proof
assume *vc: cf.virtual-cut b0-ex CClaimRefresh H-ex*
hence *restrict (Apply (l4f-clean-prefix CClaimRefresh)) (l4f-scope CClaimRefresh)*
 $=$ *restrict (Src b0-ex H-ex (l4f-frontier CClaimRefresh)) (l4f-scope CClaimRefresh)*
unfolding *cf.virtual-cut-def virtual-cut-state-def* **by** *simp*
hence *restrict (Apply (l4f-clean-prefix CClaimRefresh)) (l4f-scope CClaimRefresh) (300::nat)*
 $=$ *restrict (Src b0-ex H-ex (l4f-frontier CClaimRefresh)) (l4f-scope CClaimRefresh) 300*
by *simp*
hence *Apply (l4f-clean-prefix CClaimRefresh) (300::nat)*
 $=$ *Src b0-ex H-ex (l4f-frontier CClaimRefresh) 300*
by (*simp add: restrict-def*)
thus *False*
using *l4-missing-insert-apply-300 Src-b0-ex-H-ex-ec4-300* **by** *simp*
qed

theorem *layer4-claim-refresh-scope-mismatch-fixture-constructed*:
l4f-scope CClaimRefresh = {100, 200, 300}
 $\wedge$ *replay-keys (l4f-clean-prefix CClaimRefresh) = {100, 200}*
 $\wedge$ *(300::nat) ∈ l4f-scope CClaimRefresh*
 $\wedge$ *300 ∉ replay-keys (l4f-clean-prefix CClaimRefresh)*
 $\wedge$ *anchor-complete-at-boundary b0-ex H-ex l4-anchor (l4f-scope CClaimRefresh)*
 $\wedge$ *all-post-anchor-changes-covered H-ex l4-anchor*

$(l4f\text{-frontier } C\text{ClaimRefresh}) (l4f\text{-scope } C\text{ClaimRefresh})$
 $\wedge$ *no-unclaimed-replay-keys*
 $(l4f\text{-clean-prefix } C\text{ClaimRefresh}) (l4f\text{-scope } C\text{ClaimRefresh})$
 $\wedge$ *cf.whole-table-claim-scope b0-ex CClaimRefresh H-ex l4-anchor*
 $\wedge$ $\neg$ *cf.virtual-cut b0-ex CClaimRefresh H-ex*
 $\wedge$ *Apply (l4f-clean-prefix CClaimRefresh) 300 = None*
 $\wedge$ *Src b0-ex H-ex (l4f-frontier CClaimRefresh) 300 = Some 3500*
 $\wedge$ *Apply (l4f-clean-prefix CClaimRefresh)*
 $\neq$ *Src b0-ex H-ex (l4f-frontier CClaimRefresh)*
using *cf-claim-refresh-replay-keys cf-claim-refresh-anchor-complete*
cf-claim-refresh-all-post cf-claim-refresh-no-unclaimed
cf-claim-refresh-whole-table cf-claim-refresh-not-virtual-cut
l4-missing-insert-apply-300 Src-b0-ex-H-ex-ec4-300
cf-missing-insert-unrestricted-equality-fails
by *simp*

43.8 Anchor boundary after certified frontier (constructed)

lemma *cf-after-frontier-source-present-at-frontier:*
Src l4-after-frontier-b0 l4-after-frontier-H (l4f-frontier CAfterFrontier) (500::nat)
 $=$ *Some 9000*
unfolding *Src-def*
using *latest-src-event-l4-after-frontier-ec1-500*
by (*simp add: l4-after-frontier-H-def*)

lemma *cf-after-frontier-touched-between-empty:*
touched-between l4-after-frontier-H l4-after-frontier-anchor
 $(l4f\text{-frontier } C\text{AfterFrontier}) = \{\}$
unfolding *touched-between-def l4-after-frontier-H-def l4-after-frontier-anchor-def*
using *not-src-le-ec2-ec1*
by (*auto simp: less-src-coord-def src-lt-def less-eq-src-coord-def*)

lemma *cf-after-frontier-table-scope-between-empty:*
table-scope-between l4-after-frontier-b0 l4-after-frontier-H
l4-after-frontier-anchor (l4f-frontier CAfterFrontier) = \{\}
unfolding *table-scope-between-def*
using *l4-after-frontier-anchor-domain-empty cf-after-frontier-touched-between-empty*
by *simp*

lemma *cf-after-frontier-key-not-in-table-scope:*
 $(500::nat) \notin$ *table-scope-between l4-after-frontier-b0*
l4-after-frontier-H l4-after-frontier-anchor (l4f-frontier CAfterFrontier)
using *cf-after-frontier-table-scope-between-empty* **by** *simp*

lemma *cf-after-frontier-anchor-not-le-frontier:*
 $\neg$ *l4-after-frontier-anchor $\leq$ l4f-frontier CAfterFrontier*
using *not-src-le-ec2-ec1*
by (*simp add: l4-after-frontier-anchor-def less-eq-src-coord-def*)

lemma *cf-after-frontier-anchor-complete:*
anchor-complete-at-boundary
l4-after-frontier-b0 l4-after-frontier-H l4-after-frontier-anchor (l4f-scope CAfterFrontier)
by (*simp add: anchor-complete-at-boundary-def l4-after-frontier-anchor-domain-empty*)

lemma *cf-after-frontier-all-post:*
all-post-anchor-changes-covered l4-after-frontier-H
l4-after-frontier-anchor (l4f-frontier CAfterFrontier) (l4f-scope CAfterFrontier)
unfolding *all-post-anchor-changes-covered-def*
using *cf-after-frontier-touched-between-empty* **by** *simp*

lemma *cf-after-frontier-no-unclaimed:*
no-unclaimed-replay-keys (l4f-clean-prefix CAfterFrontier) (l4f-scope CAfterFrontier)
by (*simp add: no-unclaimed-replay-keys-def replay-keys-def*)

lemma *cf-after-frontier-apply:*
Apply (l4f-clean-prefix CAfterFrontier) = (λ-. None)
by (*simp add: Apply-def*)

lemma *cf-after-frontier-virtual-cut:*
cf.virtual-cut l4-after-frontier-b0 CAfterFrontier l4-after-frontier-H
unfolding *cf.virtual-cut-def virtual-cut-state-def restrict-def*
by *simp*

lemma *cf-after-frontier-not-whole-table:*
 $\neg$ *cf.whole-table-claim-scope l4-after-frontier-b0 CAfterFrontier*
l4-after-frontier-H l4-after-frontier-anchor
using *cf-after-frontier-anchor-not-le-frontier*
unfolding *cf.whole-table-claim-scope-def*
by *blast*

lemma *cf-after-frontier-unrestricted-equality-fails:*
Apply (l4f-clean-prefix CAfterFrontier)
 $\neq$ *Src l4-after-frontier-b0 l4-after-frontier-H (l4f-frontier CAfterFrontier)*
proof
assume *eq:*
 $\text{Apply } (l4f\text{-clean-prefix } CAfterFrontier)$
 $= \text{Src } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H } (l4f\text{-frontier } CAfterFrontier)$
hence $\text{Apply } (l4f\text{-clean-prefix } CAfterFrontier) (500::nat)$
 $= \text{Src } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H } (l4f\text{-frontier } CAfterFrontier) 500$
by *simp*
thus *False*
using *cf-after-frontier-apply cf-after-frontier-source-present-at-frontier* **by** *simp*
qed

theorem *layer4-anchor-after-frontier-fixture-constructed:*
l4f-scope CAfterFrontier = {}

$\wedge$ $l4f\text{-clean-prefix } C\text{AfterFrontier} = []$
 $\wedge$ $l4f\text{-frontier } C\text{AfterFrontier} = ec1$
 $\wedge$ $l4\text{-after-frontier-anchor} = ec2$
 $\wedge$ $\neg l4\text{-after-frontier-anchor} \leq l4f\text{-frontier } C\text{AfterFrontier}$
 $\wedge$ $\text{anchor-domain } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H}$
 $\quad l4\text{-after-frontier-anchor} = \{\}$
 $\wedge$ $\text{touched-between } l4\text{-after-frontier-H } l4\text{-after-frontier-anchor}$
 $\quad (l4f\text{-frontier } C\text{AfterFrontier}) = \{\}$
 $\wedge$ $\text{table-scope-between } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H}$
 $\quad l4\text{-after-frontier-anchor } (l4f\text{-frontier } C\text{AfterFrontier}) = \{\}$
 $\wedge$ $(500::nat) \notin \text{table-scope-between } l4\text{-after-frontier-b0}$
 $\quad l4\text{-after-frontier-H } l4\text{-after-frontier-anchor } (l4f\text{-frontier } C\text{AfterFrontier})$
 $\wedge$ $\text{anchor-complete-at-boundary } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H}$
 $\quad l4\text{-after-frontier-anchor } (l4f\text{-scope } C\text{AfterFrontier})$
 $\wedge$ $\text{all-post-anchor-changes-covered } l4\text{-after-frontier-H}$
 $\quad l4\text{-after-frontier-anchor } (l4f\text{-frontier } C\text{AfterFrontier}) (l4f\text{-scope } C\text{AfterFrontier})$
 $\text{tier})$
 $\wedge$ $\text{no-unclaimed-replay-keys}$
 $\quad (l4f\text{-clean-prefix } C\text{AfterFrontier}) (l4f\text{-scope } C\text{AfterFrontier})$
 $\wedge$ $cf.\text{virtual-cut } l4\text{-after-frontier-b0 } C\text{AfterFrontier } l4\text{-after-frontier-H}$
 $\wedge$ $\neg cf.\text{whole-table-claim-scope } l4\text{-after-frontier-b0 } C\text{AfterFrontier}$
 $\quad l4\text{-after-frontier-H } l4\text{-after-frontier-anchor}$
 $\wedge$ $\text{Apply } (l4f\text{-clean-prefix } C\text{AfterFrontier}) \ 500 = \text{None}$
 $\wedge$ $\text{Src } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H } (l4f\text{-frontier } C\text{AfterFrontier}) \ 500 =$
 $\text{Some } 9000$
 $\wedge$ $\text{Src } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H } l4\text{-after-frontier-anchor } 500 = \text{None}$
 $\wedge$ $\text{Apply } (l4f\text{-clean-prefix } C\text{AfterFrontier})$
 $\quad \neq \text{Src } l4\text{-after-frontier-b0 } l4\text{-after-frontier-H } (l4f\text{-frontier } C\text{AfterFrontier})$
using $l4\text{-after-frontier-anchor-domain-empty}$
 $\quad cf\text{-after-frontier-touched-between-empty}$
 $\quad cf\text{-after-frontier-table-scope-between-empty}$
 $\quad cf\text{-after-frontier-key-not-in-table-scope}$
 $\quad cf\text{-after-frontier-anchor-not-le-frontier}$
 $\quad cf\text{-after-frontier-anchor-complete}$
 $\quad cf\text{-after-frontier-all-post}$
 $\quad cf\text{-after-frontier-no-unclaimed}$
 $\quad cf\text{-after-frontier-virtual-cut}$
 $\quad cf\text{-after-frontier-not-whole-table}$
 $\quad cf\text{-after-frontier-apply}$
 $\quad cf\text{-after-frontier-source-present-at-frontier}$
 $\quad l4\text{-after-frontier-source-absent-at-anchor-500}$
 $\quad cf\text{-after-frontier-unrestricted-equality-fails}$
 $\quad l4\text{-after-frontier-anchor-def}$
by simp

43.9 Post-anchor update materialized by a later refresh (constructed)

lemma $cf\text{-update-refresh-replay-keys}$:

$\text{replay-keys } (\text{l4f-clean-prefix } \text{CUpdateRefresh}) = \{100, 200, 300\}$
unfolding replay-keys-def **by** ($\text{auto simp: l4-update-refresh-prefix-def}$)

lemma $\text{cf-update-refresh-no-unclaimed}$:
 $\text{no-unclaimed-replay-keys } (\text{l4f-clean-prefix } \text{CUpdateRefresh}) (\text{l4f-scope } \text{CUpdateRefresh})$
unfolding $\text{no-unclaimed-replay-keys-def}$ **using** $\text{cf-update-refresh-replay-keys}$ **by** simp

lemma $\text{cf-update-refresh-virtual-cut}$:
 $\text{cf.virtual-cut } \text{b0-ex } \text{CUpdateRefresh } H\text{-ex}$
unfolding $\text{cf.virtual-cut-def}$ $\text{virtual-cut-state-def}$
using $\text{l4-update-refresh-scoped-virtual-cut-state}$ **by** ($\text{simp add: virtual-cut-state-def}$)

lemma $\text{cf-update-refresh-anchor-complete}$:
 $\text{anchor-complete-at-boundary } \text{b0-ex } H\text{-ex } \text{l4-anchor } (\text{l4f-scope } \text{CUpdateRefresh})$
unfolding $\text{anchor-complete-at-boundary-def}$ **using** l4-anchor-domain **by** simp

lemma $\text{cf-update-refresh-all-post}$:
 $\text{all-post-anchor-changes-covered } H\text{-ex } \text{l4-anchor}$
 $(\text{l4f-frontier } \text{CUpdateRefresh}) (\text{l4f-scope } \text{CUpdateRefresh})$
unfolding $\text{all-post-anchor-changes-covered-def}$ **using** $\text{touched-H-ex-anchor-ec4}$
by simp

lemma $\text{cf-update-refresh-anchor-le-frontier}$:
 $\text{l4-anchor} \leq \text{l4f-frontier } \text{CUpdateRefresh}$
using ec1-le-ec4 **by** ($\text{simp add: l4-anchor-def less-eq-src-coord-def}$)

lemma $\text{cf-update-refresh-whole-table}$:
 $\text{cf.whole-table-claim-scope } \text{b0-ex } \text{CUpdateRefresh } H\text{-ex } \text{l4-anchor}$
unfolding $\text{cf.whole-table-claim-scope-def}$
using $\text{cf-update-refresh-anchor-le-frontier}$ $\text{cf-update-refresh-anchor-complete}$
 $\text{cf-update-refresh-all-post}$ $\text{cf-update-refresh-no-unclaimed}$
by blast

lemma $\text{cf-update-refresh-unrestricted-equality}$:
 $\text{Apply } (\text{l4f-clean-prefix } \text{CUpdateRefresh}) = \text{Src } \text{b0-ex } H\text{-ex } (\text{l4f-frontier } \text{CUpdateRefresh})$
by ($\text{rule cf.whole-table-equality-from-claim-scope}$
 $[\text{OF cf-update-refresh-virtual-cut cf-update-refresh-whole-table}]$)

lemma $\text{cf-update-refresh-refresh-for-200}$:
 $\text{Refresh } (200::\text{nat}) (\text{Some } (12000::\text{nat})) \text{ ec4} \in \text{set } (\text{l4f-clean-prefix } \text{CUpdateRefresh})$
by ($\text{simp add: l4-update-refresh-prefix-def}$)

lemma $\text{cf-update-refresh-no-cdc-update-for-200}$:
 $\text{Cdc } \text{ec3} (\text{Update } (200::\text{nat}) (12000::\text{nat})) \notin \text{set } (\text{l4f-clean-prefix } \text{CUpdateRefresh})$
by ($\text{simp add: l4-update-refresh-prefix-def}$)

theorem *layer4-post-anchor-update-before-read-fixture-constructed*:
cf.virtual-cut *b0-ex CUpdateRefresh H-ex*
 $\wedge$ *cf.whole-table-claim-scope* *b0-ex CUpdateRefresh H-ex l4-anchor*
 $\wedge$ *no-unclaimed-replay-keys*
(l4f-clean-prefix CUpdateRefresh) (l4f-scope CUpdateRefresh)
 $\wedge$ *anchor-complete-at-boundary* *b0-ex H-ex l4-anchor (l4f-scope CUpdateRefresh)*
 $\wedge$ *all-post-anchor-changes-covered* *H-ex l4-anchor*
(l4f-frontier CUpdateRefresh) (l4f-scope CUpdateRefresh)
 $\wedge$ $(200::nat) \in \text{anchor-domain } b0\text{-ex } H\text{-ex } l4\text{-anchor}$
 $\wedge$ $200 \in \text{touched-between } H\text{-ex } l4\text{-anchor } (l4f\text{-frontier } CUpdateRefresh)$
 $\wedge$ $200 \in l4f\text{-scope } CUpdateRefresh$
 $\wedge$ $\text{Refresh } 200 \text{ (Some } 12000) \text{ } ec4 \in \text{set } (l4f\text{-clean-prefix } CUpdateRefresh)$
 $\wedge$ $Cdc \text{ } ec3 \text{ (Update } 200 \text{ } 12000) \notin \text{set } (l4f\text{-clean-prefix } CUpdateRefresh)$
 $\wedge$ $(ec3, \text{Update } 200 \text{ } 12000) \in \text{set } H\text{-ex}$
 $\wedge$ $ec3 < ec4$
 $\wedge$ $\text{Apply } (l4f\text{-clean-prefix } CUpdateRefresh) \text{ } 200 = \text{Some } 12000$
 $\wedge$ $\text{Src } b0\text{-ex } H\text{-ex } (l4f\text{-frontier } CUpdateRefresh) \text{ } 200 = \text{Some } 12000$
 $\wedge$ $\text{Apply } (l4f\text{-clean-prefix } CUpdateRefresh)$
 $\quad = \text{Src } b0\text{-ex } H\text{-ex } (l4f\text{-frontier } CUpdateRefresh)$
using *cf-update-refresh-virtual-cut cf-update-refresh-whole-table*
cf-update-refresh-no-unclaimed cf-update-refresh-anchor-complete
cf-update-refresh-all-post l4-anchor-domain touched-H-ex-anchor-ec4
cf-update-refresh-refresh-for-200 cf-update-refresh-no-cdc-update-for-200
l4-update-refresh-source-update-before-refresh
l4-update-refresh-apply-200 Src-b0-ex-H-ex-ec4-200
cf-update-refresh-unrestricted-equality
by *simp*

43.10 No-unclaimed replay-key failure (constructed)

lemma *cf-unclaimed-replay-keys*:
replay-keys (l4f-clean-prefix CUnclaimed) = {100, 200, 300, 999}
unfolding *replay-keys-def* **by** *(auto simp: l4-unclaimed-prefix-def)*

theorem *cf-unclaimed-no-unclaimed-fails*:
 $\neg \text{no-unclaimed-replay-keys } (l4f\text{-clean-prefix } CUnclaimed) \text{ (l4f-scope } CUnclaimed)$
unfolding *no-unclaimed-replay-keys-def* **using** *cf-unclaimed-replay-keys* **by** *simp*

lemma *cf-unclaimed-virtual-cut*:
cf.virtual-cut b0-ex CUnclaimed H-ex
unfolding *cf.virtual-cut-def virtual-cut-state-def*
using *l4-unclaimed-scoped-virtual-cut-state* **by** *(simp add: virtual-cut-state-def)*

lemma *cf-unclaimed-anchor-complete*:
anchor-complete-at-boundary b0-ex H-ex l4-anchor (l4f-scope CUnclaimed)
unfolding *anchor-complete-at-boundary-def* **using** *l4-anchor-domain* **by** *simp*

lemma *cf-unclaimed-all-post*:

$all_post_anchor_changes_covered\ H_ex\ l4_anchor$
 $(l4f_frontier\ CUnclaimed)\ (l4f_scope\ CUnclaimed)$
unfolding $all_post_anchor_changes_covered_def$ **using** $touched_H_ex_anchor_ec4$
by $simp$

lemma $cf_unclaimed_anchor_le_frontier$:
 $l4_anchor \leq l4f_frontier\ CUnclaimed$
using $ec1_le_ec4$ **by** $(simp\ add:\ l4_anchor_def\ less_eq_src_coord_def)$

lemma $cf_unclaimed_table_scope_between$:
 $table_scope_between\ b0_ex\ H_ex\ l4_anchor\ (l4f_frontier\ CUnclaimed) = l4f_scope\ CUnclaimed$
unfolding $table_scope_between_def$
using $l4_anchor_domain\ touched_H_ex_anchor_ec4$ **by** $auto$

theorem $cf_unclaimed_not_whole_table$:
 $\neg cf_whole_table_claim_scope\ b0_ex\ CUnclaimed\ H_ex\ l4_anchor$
using $cf_unclaimed_no_unclaimed_fails$
unfolding $cf_whole_table_claim_scope_def$ **by** $blast$

lemma $cf_unclaimed_refresh_for_999$:
 $Refresh\ (999::nat)\ (Some\ (4242::nat))\ ec4 \in set\ (l4f_clean_prefix\ CUnclaimed)$
by $(simp\ add:\ l4_unclaimed_prefix_def)$

lemma $cf_unclaimed_apply_999$:
 $Apply\ (l4f_clean_prefix\ CUnclaimed)\ (999::nat) = Some\ 4242$
using $l4_unclaimed_apply_999$ **by** $simp$

lemma $cf_unclaimed_source_absent_999$:
 $Src\ b0_ex\ H_ex\ (l4f_frontier\ CUnclaimed)\ (999::nat) = None$
unfolding $Src_def\ b0_ex_def$
using $latest_src_event_H_ex_ec4_999$ **by** $simp$

theorem $cf_unclaimed_unrestricted_equality_fails$:
 $Apply\ (l4f_clean_prefix\ CUnclaimed) \neq Src\ b0_ex\ H_ex\ (l4f_frontier\ CUnclaimed)$
proof
assume $eq:\ Apply\ (l4f_clean_prefix\ CUnclaimed) = Src\ b0_ex\ H_ex\ (l4f_frontier\ CUnclaimed)$
hence $Apply\ (l4f_clean_prefix\ CUnclaimed)\ (999::nat)$
 $= Src\ b0_ex\ H_ex\ (l4f_frontier\ CUnclaimed)\ 999$
by $simp$
thus $False$ **using** $cf_unclaimed_apply_999\ cf_unclaimed_source_absent_999$ **by**
 $simp$
qed

theorem $layer4_no_unclaimed_signature_determinacy_fixture_constructed$:
 $cf_virtual_cut\ b0_ex\ CUnclaimed\ H_ex$
 $\wedge\ l4_anchor \leq l4f_frontier\ CUnclaimed$
 $\wedge\ anchor_complete_at_boundary\ b0_ex\ H_ex\ l4_anchor\ (l4f_scope\ CUnclaimed)$

$\wedge$ *all-post-anchor-changes-covered* *H-ex* *l4-anchor*
 $(l4f\text{-frontier } CUnclaimed) (l4f\text{-scope } CUnclaimed)$
 $\wedge$ *table-scope-between* *b0-ex* *H-ex* *l4-anchor* $(l4f\text{-frontier } CUnclaimed) = l4f\text{-scope } CUnclaimed$
 $\wedge$ *replay-keys* $(l4f\text{-clean-prefix } CUnclaimed) = \{100, 200, 300, 999\}$
 $\wedge$ *Refresh* 999 $(Some\ 4242)\ ec4 \in set\ (l4f\text{-clean-prefix } CUnclaimed)$
 $\wedge$ $999 \notin l4f\text{-scope } CUnclaimed$
 $\wedge \neg no\text{-unclaimed-replay-keys } (l4f\text{-clean-prefix } CUnclaimed) (l4f\text{-scope } CUnclaimed)$
 $\wedge \neg cf.\text{whole-table-claim-scope } b0\text{-ex } CUnclaimed\ H\text{-ex } l4\text{-anchor}$
 $\wedge$ *Apply* $(l4f\text{-clean-prefix } CUnclaimed)\ 999 = Some\ 4242$
 $\wedge$ *Src* *b0-ex* *H-ex* $(l4f\text{-frontier } CUnclaimed)\ 999 = None$
 $\wedge$ *Apply* $(l4f\text{-clean-prefix } CUnclaimed) \neq Src\ b0\text{-ex } H\text{-ex } (l4f\text{-frontier } CUnclaimed)$
using *cf-unclaimed-virtual-cut* *cf-unclaimed-anchor-le-frontier*
cf-unclaimed-anchor-complete *cf-unclaimed-all-post*
cf-unclaimed-table-scope-between *cf-unclaimed-replay-keys*
cf-unclaimed-refresh-for-999 *cf-unclaimed-no-unclaimed-fails*
cf-unclaimed-not-whole-table *cf-unclaimed-apply-999*
cf-unclaimed-source-absent-999 *cf-unclaimed-unrestricted-equality-fails*
by *simp*

43.11 b0 base-state contribution to the anchor domain (constructed)

lemma *cf-b0-anchor-apply:*

Apply $(l4f\text{-clean-prefix } CB0Anchor) = l4\text{-b0-anchor-b0}$

by (*rule ext*)

(*simp add: Apply-def l4-b0-anchor-b0-def l4-b0-anchor-prefix-def*)

lemma *cf-b0-anchor-apply-key:*

Apply $(l4f\text{-clean-prefix } CB0Anchor) (700::nat) = Some\ 7000$

using *cf-b0-anchor-apply* *l4-b0-anchor-base-key-present* **by** *simp*

lemma *cf-b0-anchor-source-at-frontier:*

Src *l4-b0-anchor-b0* *l4-b0-anchor-H* $(l4f\text{-frontier } CB0Anchor) = l4\text{-b0-anchor-b0}$

by (*rule ext*)

(*simp add: Src-def latest-src-event-def l4-b0-anchor-H-def*)

lemma *cf-b0-anchor-source-key-at-frontier:*

Src *l4-b0-anchor-b0* *l4-b0-anchor-H* $(l4f\text{-frontier } CB0Anchor) (700::nat) = Some\ 7000$

using *l4-b0-anchor-base-key-present* *cf-b0-anchor-source-at-frontier* **by** *simp*

lemma *cf-b0-anchor-touched-between:*

touched-between *l4-b0-anchor-H* *l4-anchor* $(l4f\text{-frontier } CB0Anchor) = \{\}$

by (*simp add: l4-b0-anchor-H-def touched-between-nil*)

lemma *cf-b0-anchor-table-scope-between:*

table-scope-between $l_4\text{-b0-anchor-b0}$ $l_4\text{-b0-anchor-H}$ $l_4\text{-anchor}$
 $(l_4f\text{-frontier } CB0Anchor) = l_4f\text{-scope } CB0Anchor$
unfolding *table-scope-between-def*
using *l4-b0-anchor-domain* *cf-b0-anchor-touched-between* **by** *simp*

lemma *cf-b0-anchor-replay-keys*:
replay-keys $(l_4f\text{-clean-prefix } CB0Anchor) = \{700\}$
unfolding *replay-keys-def* **by** (*simp add: l4-b0-anchor-prefix-def*)

lemma *cf-b0-anchor-no-unclaimed*:
no-unclaimed-replay-keys $(l_4f\text{-clean-prefix } CB0Anchor)$ $(l_4f\text{-scope } CB0Anchor)$
unfolding *no-unclaimed-replay-keys-def* **using** *cf-b0-anchor-replay-keys* **by** *simp*

lemma *cf-b0-anchor-anchor-complete*:
anchor-complete-at-boundary $l_4\text{-b0-anchor-b0}$ $l_4\text{-b0-anchor-H}$ $l_4\text{-anchor}$ $(l_4f\text{-scope } CB0Anchor)$
unfolding *anchor-complete-at-boundary-def* **using** *l4-b0-anchor-domain* **by** *simp*

lemma *cf-b0-anchor-all-post*:
all-post-anchor-changes-covered $l_4\text{-b0-anchor-H}$ $l_4\text{-anchor}$
 $(l_4f\text{-frontier } CB0Anchor)$ $(l_4f\text{-scope } CB0Anchor)$
unfolding *all-post-anchor-changes-covered-def* **using** *cf-b0-anchor-touched-between*
by *simp*

lemma *cf-b0-anchor-anchor-le-frontier*:
 $l_4\text{-anchor} \leq l_4f\text{-frontier } CB0Anchor$
using *ec1-le-ec4* **by** (*simp add: l4-anchor-def less-eq-src-coord-def*)

lemma *cf-b0-anchor-whole-table*:
cf.whole-table-claim-scope $l_4\text{-b0-anchor-b0}$ $CB0Anchor$ $l_4\text{-b0-anchor-H}$ $l_4\text{-anchor}$
unfolding *cf.whole-table-claim-scope-def*
using *cf-b0-anchor-anchor-le-frontier* *cf-b0-anchor-anchor-complete*
cf-b0-anchor-all-post *cf-b0-anchor-no-unclaimed* **by** *blast*

lemma *cf-b0-anchor-unrestricted-equality*:
Apply $(l_4f\text{-clean-prefix } CB0Anchor)$
 $= Src$ $l_4\text{-b0-anchor-b0}$ $l_4\text{-b0-anchor-H}$ $(l_4f\text{-frontier } CB0Anchor)$
using *cf-b0-anchor-apply* *cf-b0-anchor-source-at-frontier* **by** *simp*

lemma *cf-b0-anchor-virtual-cut*:
cf.virtual-cut $l_4\text{-b0-anchor-b0}$ $CB0Anchor$ $l_4\text{-b0-anchor-H}$
unfolding *cf.virtual-cut-def* *virtual-cut-state-def*
using *cf-b0-anchor-unrestricted-equality* **by** *simp*

theorem *layer4-b0-base-state-anchor-domain-fixture-constructed*:
 $l_4\text{-b0-anchor-H} = []$
 $\wedge l_4\text{-b0-anchor-b0 } (700::nat) = Some\ 7000$
 $\wedge latest\text{-src-event } l_4\text{-b0-anchor-H } l_4\text{-anchor } (700::nat) = None$
 $\wedge Src\ l_4\text{-b0-anchor-b0 } l_4\text{-b0-anchor-H } l_4\text{-anchor } 700 = Some\ 7000$

$\wedge \text{Src } l_4\text{-b0-anchor-b0 } l_4\text{-b0-anchor-H } (l_4\text{f-frontier CB0Anchor}) \ 700 = \text{Some } 7000$
 $\wedge \text{anchor-domain } l_4\text{-b0-anchor-b0 } l_4\text{-b0-anchor-H } l_4\text{-anchor} = \{700\}$
 $\wedge (700::\text{nat}) \in \text{anchor-domain } l_4\text{-b0-anchor-b0 } l_4\text{-b0-anchor-H } l_4\text{-anchor}$
 $\wedge \text{touched-between } l_4\text{-b0-anchor-H } l_4\text{-anchor } (l_4\text{f-frontier CB0Anchor}) = \{\}$
 $\wedge \text{table-scope-between } l_4\text{-b0-anchor-b0 } l_4\text{-b0-anchor-H } l_4\text{-anchor}$
 $\quad (l_4\text{f-frontier CB0Anchor}) = l_4\text{f-scope CB0Anchor}$
 $\wedge l_4\text{f-scope CB0Anchor} = \{700\}$
 $\wedge \text{replay-keys } (l_4\text{f-clean-prefix CB0Anchor}) = \{700\}$
 $\wedge \text{Apply } (l_4\text{f-clean-prefix CB0Anchor}) \ 700 = \text{Some } 7000$
 $\wedge l_4\text{-anchor} \leq l_4\text{f-frontier CB0Anchor}$
 $\quad \wedge \text{anchor-complete-at-boundary } l_4\text{-b0-anchor-b0 } l_4\text{-b0-anchor-H } l_4\text{-anchor}$
 $(l_4\text{f-scope CB0Anchor})$
 $\quad \wedge \text{all-post-anchor-changes-covered } l_4\text{-b0-anchor-H } l_4\text{-anchor}$
 $\quad \quad (l_4\text{f-frontier CB0Anchor}) \ (l_4\text{f-scope CB0Anchor})$
 $\wedge \text{no-unclaimed-replay-keys } (l_4\text{f-clean-prefix CB0Anchor}) \ (l_4\text{f-scope CB0Anchor})$
 $\quad \wedge \text{cf.whole-table-claim-scope } l_4\text{-b0-anchor-b0 } \text{CB0Anchor } l_4\text{-b0-anchor-H}$
 $l_4\text{-anchor}$
 $\quad \wedge \text{cf.virtual-cut } l_4\text{-b0-anchor-b0 } \text{CB0Anchor } l_4\text{-b0-anchor-H}$
 $\quad \wedge \text{Apply } (l_4\text{f-clean-prefix CB0Anchor})$
 $\quad = \text{Src } l_4\text{-b0-anchor-b0 } l_4\text{-b0-anchor-H } (l_4\text{f-frontier CB0Anchor})$
using $l_4\text{-b0-anchor-H-def } l_4\text{-b0-anchor-base-key-present}$
 $l_4\text{-b0-anchor-no-source-event-for-key } l_4\text{-b0-anchor-source-key-at-anchor}$
 $cf\text{-b0-anchor-source-key-at-frontier } l_4\text{-b0-anchor-domain}$
 $cf\text{-b0-anchor-touched-between } cf\text{-b0-anchor-table-scope-between}$
 $cf\text{-b0-anchor-replay-keys } cf\text{-b0-anchor-apply-key}$
 $cf\text{-b0-anchor-anchor-le-frontier } cf\text{-b0-anchor-anchor-complete}$
 $cf\text{-b0-anchor-all-post } cf\text{-b0-anchor-no-unclaimed}$
 $cf\text{-b0-anchor-whole-table } cf\text{-b0-anchor-virtual-cut}$
 $cf\text{-b0-anchor-unrestricted-equality}$
by *simp*

43.12 Same-coordinate anchor and frontier boundaries (constructed)

lemma *cf-same-boundary-latest-frontier-800:*

latest-src-event l_4-same-boundary-H (l_4f-frontier CSameBoundary) (800::nat) = Some 0

unfolding *latest-src-event-def l_4-same-boundary-H-def*

using *ec1-le-ec4 src-le-refl* **[where** *c=ec4* **]**

by *simp*

lemma *cf-same-boundary-latest-frontier-900:*

latest-src-event l_4-same-boundary-H (l_4f-frontier CSameBoundary) (900::nat) = Some 1

unfolding *latest-src-event-def l_4-same-boundary-H-def*

using *ec1-le-ec4 src-le-refl* **[where** *c=ec4* **]**

by *simp*

```

lemma cf-same-boundary-source-frontier-800:
  Src l4-same-boundary-b0 l4-same-boundary-H (l4f-frontier CSameBoundary)
  (800::nat)
  = Some 8000
  unfolding Src-def using cf-same-boundary-latest-frontier-800
  by (simp add: l4-same-boundary-H-def)

lemma cf-same-boundary-source-frontier-900:
  Src l4-same-boundary-b0 l4-same-boundary-H (l4f-frontier CSameBoundary)
  (900::nat)
  = Some 9000
  unfolding Src-def using cf-same-boundary-latest-frontier-900
  by (simp add: l4-same-boundary-H-def)

lemma cf-same-boundary-touched-between:
  touched-between l4-same-boundary-H l4-anchor (l4f-frontier CSameBoundary) =
  {900}
proof (rule subset-antisym)
  show touched-between l4-same-boundary-H l4-anchor (l4f-frontier CSameBound-
  ary)  $\subseteq$  {900}
    unfolding touched-between-def l4-same-boundary-H-def l4-anchor-def
    by auto
next
  show {900}  $\subseteq$  touched-between l4-same-boundary-H l4-anchor (l4f-frontier
  CSameBoundary)
  proof
    fix k :: nat
    assume k  $\in$  {900}
    hence k900: k = 900 by simp
    have in-H: (ec4, Insert (900::nat) (9000::nat))  $\in$  set l4-same-boundary-H
      by (simp add: l4-same-boundary-H-def)
    have after-anchor: l4-anchor < ec4
      using ec1-less-ec4 by (simp add: l4-anchor-def)
    have before-frontier: ec4  $\leq$  l4f-frontier CSameBoundary
      by (simp add: less-eq-src-coord-def src-le-refl)
    show k  $\in$  touched-between l4-same-boundary-H l4-anchor (l4f-frontier CSame-
    Boundary)
      unfolding touched-between-def
      using in-H k900 after-anchor before-frontier
      by (intro CollectI exI[where x=ec4]
        exI[where x=Insert (900::nat) (9000::nat)]) auto
  qed
qed

lemma cf-same-boundary-anchor-key-not-touched:
  (800::nat)  $\notin$  touched-between l4-same-boundary-H l4-anchor (l4f-frontier CSame-
  Boundary)
  using cf-same-boundary-touched-between by simp

```

lemma *cf-same-boundary-frontier-key-touched*:
 $(900::nat) \in \text{touched-between } l_4\text{-same-boundary-}H \text{ } l_4\text{-anchor } (l_4f\text{-frontier } C\text{SameBoundary})$
using *cf-same-boundary-touched-between* **by** *simp*

lemma *cf-same-boundary-table-scope-between*:
 $\text{table-scope-between } l_4\text{-same-boundary-b0 } l_4\text{-same-boundary-}H \text{ } l_4\text{-anchor } (l_4f\text{-frontier } C\text{SameBoundary}) = l_4f\text{-scope } C\text{SameBoundary}$
unfolding *table-scope-between-def*
using *l4-same-boundary-anchor-domain* *cf-same-boundary-touched-between* **by** *auto*

lemma *cf-same-boundary-replay-keys*:
 $\text{replay-keys } (l_4f\text{-clean-prefix } C\text{SameBoundary}) = \{800, 900\}$
unfolding *replay-keys-def* **by** (*auto simp: l4-same-boundary-prefix-def*)

lemma *cf-same-boundary-no-unclaimed*:
 $\text{no-unclaimed-replay-keys } (l_4f\text{-clean-prefix } C\text{SameBoundary}) (l_4f\text{-scope } C\text{SameBoundary})$
unfolding *no-unclaimed-replay-keys-def* **using** *cf-same-boundary-replay-keys* **by** *simp*

lemma *cf-same-boundary-anchor-complete*:
 $\text{anchor-complete-at-boundary } l_4\text{-same-boundary-b0 } l_4\text{-same-boundary-}H \text{ } l_4\text{-anchor } (l_4f\text{-scope } C\text{SameBoundary})$
unfolding *anchor-complete-at-boundary-def* **using** *l4-same-boundary-anchor-domain* **by** *simp*

lemma *cf-same-boundary-all-post*:
 $\text{all-post-anchor-changes-covered } l_4\text{-same-boundary-}H \text{ } l_4\text{-anchor } (l_4f\text{-frontier } C\text{SameBoundary}) (l_4f\text{-scope } C\text{SameBoundary})$
unfolding *all-post-anchor-changes-covered-def*
using *cf-same-boundary-touched-between* **by** *simp*

lemma *cf-same-boundary-anchor-le-frontier*:
 $l_4\text{-anchor} \leq l_4f\text{-frontier } C\text{SameBoundary}$
using *ec1-le-ec4* **by** (*simp add: l4-anchor-def less-eq-src-coord-def*)

lemma *cf-same-boundary-whole-table*:
 $\text{cf.whole-table-claim-scope } l_4\text{-same-boundary-b0 } C\text{SameBoundary } l_4\text{-same-boundary-}H \text{ } l_4\text{-anchor}$
unfolding *cf.whole-table-claim-scope-def*
using *cf-same-boundary-anchor-le-frontier* *cf-same-boundary-anchor-complete* *cf-same-boundary-all-post* *cf-same-boundary-no-unclaimed* **by** *blast*

lemma *cf-same-boundary-apply*:
 $\text{Apply } (l_4f\text{-clean-prefix } C\text{SameBoundary}) =$
 $(\lambda k. \text{if } k = (800::nat) \text{ then } \text{Some } (8000::nat) \text{ else if } k = 900 \text{ then } \text{Some } 9000 \text{ else } \text{None})$


```

by (rule ext)
  (simp add: Apply-def l4-same-boundary-prefix-def)

lemma cf-same-boundary-apply-800:
  Apply (l4f-clean-prefix CSameBoundary) (800::nat) = Some 8000
by (simp add: l4-same-boundary-prefix-def Apply-def)

lemma cf-same-boundary-apply-900:
  Apply (l4f-clean-prefix CSameBoundary) (900::nat) = Some 9000
by (simp add: l4-same-boundary-prefix-def Apply-def)

lemma cf-same-boundary-virtual-cut:
  cf.virtual-cut l4-same-boundary-b0 CSameBoundary l4-same-boundary-H
unfolding cf.virtual-cut-def virtual-cut-state-def restrict-def
proof (rule ext)
  fix k :: nat
  show (if k ∈ l4f-scope CSameBoundary
    then Apply (l4f-clean-prefix CSameBoundary) k else None)
    = (if k ∈ l4f-scope CSameBoundary
    then Src l4-same-boundary-b0 l4-same-boundary-H (l4f-frontier CSame-
Boundary) k
    else None)
  proof (cases k = 800)
    case True
    thus ?thesis
    using cf-same-boundary-apply-800 cf-same-boundary-source-frontier-800 by
simp
  next
    case not800: False
    show ?thesis
    proof (cases k = 900)
      case True
      thus ?thesis
      using cf-same-boundary-apply-900 cf-same-boundary-source-frontier-900 by
simp
    next
      case False
      with not800 show ?thesis by simp
    qed
  qed
qed

lemma cf-same-boundary-unrestricted-equality:
  Apply (l4f-clean-prefix CSameBoundary)
    = Src l4-same-boundary-b0 l4-same-boundary-H (l4f-frontier CSameBoundary)
by (rule cf.whole-table-equality-from-claim-scope
  [OF cf-same-boundary-virtual-cut cf-same-boundary-whole-table])

theorem layer4-same-coordinate-boundary-fixture-constructed:

```

$(ec1, \text{Insert } (800::nat) (8000::nat)) \in \text{set } l4\text{-same-boundary-}H$
 $\wedge (ec4, \text{Insert } (900::nat) (9000::nat)) \in \text{set } l4\text{-same-boundary-}H$
 $\wedge l4f\text{-frontier } CSameBoundary = ec4$
 $\wedge \text{Src } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H \text{ } l4\text{-anchor } 800 = \text{Some } 8000$
 $\wedge \text{Src } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H \text{ } l4\text{-anchor } 900 = \text{None}$
 $\wedge \text{Src } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H (l4f\text{-frontier } CSameBoundary)$
 $900 = \text{Some } 9000$
 $\wedge \text{anchor-domain } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H \text{ } l4\text{-anchor} = \{800\}$
 $\wedge (800::nat) \in \text{anchor-domain } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H$
 $l4\text{-anchor}$
 $\wedge (900::nat) \notin \text{anchor-domain } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H$
 $l4\text{-anchor}$
 $\wedge \text{touched-between } l4\text{-same-boundary-}H \text{ } l4\text{-anchor} (l4f\text{-frontier } CSameBoundary)$
 $= \{900\}$
 $\wedge 800 \notin \text{touched-between } l4\text{-same-boundary-}H \text{ } l4\text{-anchor} (l4f\text{-frontier } CSame\text{-}$
 $\text{Boundary})$
 $\wedge 900 \in \text{touched-between } l4\text{-same-boundary-}H \text{ } l4\text{-anchor} (l4f\text{-frontier } CSame\text{-}$
 $\text{Boundary})$
 $\wedge \text{table-scope-between } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H \text{ } l4\text{-anchor}$
 $(l4f\text{-frontier } CSameBoundary) = l4f\text{-scope } CSameBoundary$
 $\wedge l4f\text{-scope } CSameBoundary = \{800, 900\}$
 $\wedge \text{replay-keys } (l4f\text{-clean-prefix } CSameBoundary) = \{800, 900\}$
 $\wedge \text{no-unclaimed-replay-keys } (l4f\text{-clean-prefix } CSameBoundary) (l4f\text{-scope } CSame\text{-}$
 $\text{Boundary})$
 $\wedge \text{cf.whole-table-claim-scope } l4\text{-same-boundary-b0 } CSameBoundary \text{ } l4\text{-same-boundary-}H$
 $l4\text{-anchor}$
 $\wedge \text{cf.virtual-cut } l4\text{-same-boundary-b0 } CSameBoundary \text{ } l4\text{-same-boundary-}H$
 $\wedge \text{Apply } (l4f\text{-clean-prefix } CSameBoundary)$
 $= \text{Src } l4\text{-same-boundary-b0 } l4\text{-same-boundary-}H (l4f\text{-frontier } CSameBoundary)$
using $l4\text{-same-boundary-anchor-event-in-history } l4\text{-same-boundary-frontier-event-in-history}$
 $l4\text{-same-boundary-source-anchor-800 } l4\text{-same-boundary-source-anchor-900}$
 $cf\text{-same-boundary-source-frontier-900 } l4\text{-same-boundary-anchor-domain}$
 $l4\text{-same-boundary-anchor-key-in-anchor-domain}$
 $l4\text{-same-boundary-frontier-key-not-in-anchor-domain}$
 $cf\text{-same-boundary-touched-between } cf\text{-same-boundary-anchor-key-not-touched}$
 $cf\text{-same-boundary-frontier-key-touched } cf\text{-same-boundary-table-scope-between}$
 $cf\text{-same-boundary-replay-keys } cf\text{-same-boundary-no-unclaimed}$
 $cf\text{-same-boundary-whole-table } cf\text{-same-boundary-virtual-cut}$
 $cf\text{-same-boundary-unrestricted-equality}$
by *simp*

43.13 Materialized-run ambiguity is not Layer 4 evidence (constructed)

A run-sensitive boundary fixture. The certificate *CAmbig* exposes the same Layer 4 surface as the running-example witness (scope, frontier, clean prefix), yet *materializes-run* is a relation: the same certificate is materialized by two distinct runs whose chunk sets, and whose responsible chunk for key 100, differ. Layer 4 must therefore not infer responsible-chunk or watermark

facts from *whole-table-claim-scope* or any other set-only certificate predicate. The two runs are taken from a *nat* run carrier (0 and 1); the interpretation *amb* differs from *cf* only in its run side (a two-element carrier rather than the degenerate *unit*), so the materialization relation can hold for two distinct runs. The fixture is materialization-relational only: it does not assert *cert-accessors-agree* on the runs (their constructed *clean-prefix-of* cannot equal the two-coordinate running-example clean prefix); the materialization-relational form already discharges the overclaim-control point.

definition *l4f-amb-chunks* :: *nat* $\Rightarrow$ *nat set* **where**

l4f-amb-chunks $r = (\text{if } r = 0 \text{ then } \{0\} \text{ else if } r = 1 \text{ then } \{1\} \text{ else } \{\})$

definition *l4f-amb-chunks-list* :: *nat* $\Rightarrow$ *nat list* **where**

l4f-amb-chunks-list $r = (\text{if } r = 0 \text{ then } [0] \text{ else if } r = 1 \text{ then } [1] \text{ else } [])$

definition *l4f-amb-responsible* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat option* **where**

l4f-amb-responsible $r\ k =$
 $(\text{if } r = 0 \text{ then } (\text{if } k = 100 \text{ then } \text{Some } 0 \text{ else } \text{None})$
 $\text{else if } r = 1 \text{ then } (\text{if } k = 100 \text{ then } \text{Some } 1 \text{ else } \text{None}) \text{ else } \text{None})$

definition *l4f-amb-materializes* :: *l4f-cert* $\Rightarrow$ *unit* $\Rightarrow$ *nat* $\Rightarrow$ *bool* **where**

l4f-amb-materializes $C\ E\ r \longleftrightarrow C = \text{CAmbig} \wedge (r = 0 \vee r = 1)$

The ambiguity interpretation: the run side is a *nat* carrier so the materialization relation can hold for the two distinct runs 0 and 1 ; the certificate side reuses *l4f-scope* / *l4f-frontier* / *l4f-clean-prefix* (so *CAmbig* carries the running-example surface). The two inherited *chunks-list* obligations hold definitionally — *set* (*l4f-amb-chunks-list* r) = *l4f-amb-chunks* r and *distinct* (*l4f-amb-chunks-list* r) are parallel if-then-else over r — so the interpretation introduces no axiom.

global-interpretation *amb*: *dblog-cert-substrate*

$\lambda\text{-}::\text{nat. } \{\} :: \text{nat set}$	— <i>scope-of</i>
$\lambda\text{-}::\text{nat. } c0$	— <i>frontier-of</i>
<i>l4f-amb-chunks</i>	— <i>chunks</i>
$\lambda\text{-}::\text{nat. } ([] :: (\text{nat}, \text{nat}) \text{ src-history})$	— <i>src-history-of</i>
$\lambda\text{-}::\text{nat. } \lambda\text{-}::\text{nat. } \lambda\text{-}::\text{nat. } (\text{None} :: \text{nat option option})$	— <i>chunk-read-result</i>
$\lambda\text{-}::\text{nat. } ([] :: (\text{src-coord} \times (\text{nat}, \text{nat}) \text{ source-event}) \text{ list})$	— <i>cdc-events-of</i>
<i>l4f-amb-chunks-list</i>	— <i>chunks-list</i>
$\lambda\text{-}::\text{nat. } \lambda\text{-}::\text{nat. } \{\} :: \text{nat set}$	— <i>chunk-domain</i>
<i>l4f-amb-responsible</i>	— <i>responsible-chunk</i>
$\lambda\text{-}::\text{nat. } \lambda\text{-}::\text{nat. } c0$	— <i>chunk-lower-watermark</i>
$\lambda\text{-}::\text{nat. } \lambda\text{-}::\text{nat. } c0$	— <i>chunk-upper-watermark</i>
$\lambda\text{-}::\text{nat. } \lambda\text{-}::\text{nat. } c0$	— <i>chunk-read-coordinate</i>
<i>l4f-scope l4f-frontier l4f-clean-prefix</i>	
$\lambda\text{-}::\text{l4f-cert. } \lambda\text{-}::\text{unit. } \text{Reject}$	— <i>verify</i>
<i>Accept</i>	— <i>accept-value</i>
$\lambda\text{-}::\text{unit. } \lambda\text{-}::(\text{nat} \rightarrow \text{nat}). \lambda\text{-}::(\text{nat}, \text{nat}) \text{ src-history. } \lambda\text{-}::\text{unit. } \text{False}$	— <i>faithful-source-observation</i>

$l4f\text{-}amb\text{-}materializes$ — $materializes\text{-}run$
 $\lambda\text{-}::l4f\text{-}cert. \lambda\text{-}::unit. False$ — $evidence\text{-}bound\text{-}to\text{-}cert$
 $\lambda\text{-}::l4f\text{-}cert. \lambda\text{-}::unit. False$ — $source\text{-}side\text{-}schema\text{-}supported$
by $unfold\text{-}locales$ ($simp\text{-}all$ add: $l4f\text{-}amb\text{-}chunks\text{-}def$ $l4f\text{-}amb\text{-}chunks\text{-}list\text{-}def$)

lemma $amb\text{-}ambig\text{-}virtual\text{-}cut$:
 $amb.virtual\text{-}cut$ $b0\text{-}ex$ $CAmbig$ $H\text{-}ex$
unfolding $amb.virtual\text{-}cut\text{-}def$ $virtual\text{-}cut\text{-}state\text{-}def$
using $rep\text{-}ex\text{-}virtual\text{-}cut\text{-}state$ **by** ($simp$ add: $virtual\text{-}cut\text{-}state\text{-}def$)

lemma $amb\text{-}ambig\text{-}anchor\text{-}complete$:
 $anchor\text{-}complete\text{-}at\text{-}boundary$ $b0\text{-}ex$ $H\text{-}ex$ $l4\text{-}anchor$ ($l4f\text{-}scope$ $CAmbig$)
unfolding $anchor\text{-}complete\text{-}at\text{-}boundary\text{-}def$ **using** $l4\text{-}anchor\text{-}domain$ **by** $simp$

lemma $amb\text{-}ambig\text{-}all\text{-}post$:
 $all\text{-}post\text{-}anchor\text{-}changes\text{-}covered$ $H\text{-}ex$ $l4\text{-}anchor$ ($l4f\text{-}frontier$ $CAmbig$) ($l4f\text{-}scope$ $CAmbig$)
unfolding $all\text{-}post\text{-}anchor\text{-}changes\text{-}covered\text{-}def$ **using** $touched\text{-}H\text{-}ex\text{-}anchor\text{-}ec4$
by $simp$

lemma $amb\text{-}ambig\text{-}no\text{-}unclaimed$:
 $no\text{-}unclaimed\text{-}replay\text{-}keys$ ($l4f\text{-}clean\text{-}prefix$ $CAmbig$) ($l4f\text{-}scope$ $CAmbig$)
unfolding $no\text{-}unclaimed\text{-}replay\text{-}keys\text{-}def$ **using** $rep\text{-}ex\text{-}replay\text{-}keys$ **by** $simp$

lemma $amb\text{-}ambig\text{-}anchor\text{-}le\text{-}frontier$:
 $l4\text{-}anchor \leq l4f\text{-}frontier$ $CAmbig$
using $ec1\text{-}le\text{-}ec4$ **by** ($simp$ add: $l4\text{-}anchor\text{-}def$ $less\text{-}eq\text{-}src\text{-}coord\text{-}def$)

lemma $amb\text{-}ambig\text{-}whole\text{-}table$:
 $amb.whole\text{-}table\text{-}claim\text{-}scope$ $b0\text{-}ex$ $CAmbig$ $H\text{-}ex$ $l4\text{-}anchor$
unfolding $amb.whole\text{-}table\text{-}claim\text{-}scope\text{-}def$
using $amb\text{-}ambig\text{-}anchor\text{-}le\text{-}frontier$ $amb\text{-}ambig\text{-}anchor\text{-}complete$
 $amb\text{-}ambig\text{-}all\text{-}post$ $amb\text{-}ambig\text{-}no\text{-}unclaimed$ **by** $blast$

lemma $amb\text{-}ambig\text{-}unrestricted\text{-}equality$:
 $Apply$ ($l4f\text{-}clean\text{-}prefix$ $CAmbig$) = Src $b0\text{-}ex$ $H\text{-}ex$ ($l4f\text{-}frontier$ $CAmbig$)
by ($rule$ $amb.whole\text{-}table\text{-}equality\text{-}from\text{-}claim\text{-}scope$
 $[OF\ amb\text{-}ambig\text{-}virtual\text{-}cut\ amb\text{-}ambig\text{-}whole\text{-}table]$)

lemma $amb\text{-}materializes\text{-}left$: $l4f\text{-}amb\text{-}materializes$ $CAmbig$ () 0
by ($simp$ add: $l4f\text{-}amb\text{-}materializes\text{-}def$)

lemma $amb\text{-}materializes\text{-}right$: $l4f\text{-}amb\text{-}materializes$ $CAmbig$ () 1
by ($simp$ add: $l4f\text{-}amb\text{-}materializes\text{-}def$)

lemma $amb\text{-}chunks\text{-}differ$: $l4f\text{-}amb\text{-}chunks$ 0 $\neq$ $l4f\text{-}amb\text{-}chunks$ 1
by ($simp$ add: $l4f\text{-}amb\text{-}chunks\text{-}def$)

lemma $amb\text{-}responsible\text{-}differ$:

$l4f\text{-}amb\text{-}responsible\ 0\ 100 \neq l4f\text{-}amb\text{-}responsible\ 1\ 100$
by (*simp add: l4f-amb-responsible-def*)

theorem *layer4-materialized-run-ambiguity-fixture-constructed:*

$l4f\text{-}amb\text{-}materializes\ C\text{Ambig}\ ()\ 0$
 $\wedge\ l4f\text{-}amb\text{-}materializes\ C\text{Ambig}\ ()\ 1$
 $\wedge\ l4f\text{-}scope\ C\text{Ambig} = \{100, 200, 300\}$
 $\wedge\ l4f\text{-}frontier\ C\text{Ambig} = ec4$
 $\wedge\ l4f\text{-}clean\text{-}prefix\ C\text{Ambig} = ex\text{-}model.\text{clean}\text{-}prefix\text{-}of\ ()$
 $\wedge\ l4f\text{-}amb\text{-}chunks\ 0 \neq l4f\text{-}amb\text{-}chunks\ 1$
 $\wedge\ l4f\text{-}amb\text{-}responsible\ 0\ 100 \neq l4f\text{-}amb\text{-}responsible\ 1\ 100$
 $\wedge\ amb.\text{whole}\text{-}table\text{-}claim\text{-}scope\ b0\text{-}ex\ C\text{Ambig}\ H\text{-}ex\ l4\text{-}anchor$
 $\wedge\ amb.\text{virtual}\text{-}cut\ b0\text{-}ex\ C\text{Ambig}\ H\text{-}ex$
 $\wedge\ Apply\ (l4f\text{-}clean\text{-}prefix\ C\text{Ambig}) = Src\ b0\text{-}ex\ H\text{-}ex\ (l4f\text{-}frontier\ C\text{Ambig})$
using *amb-materializes-left amb-materializes-right*
amb-chunks-differ amb-responsible-differ
amb-ambig-whole-table amb-ambig-virtual-cut
amb-ambig-unrestricted-equality
by *simp*

43.14 Constructed Layer 4 fixture bundle

The ten Layer 4 negative / boundary / run-ambiguity fixtures hold over the construction — the cert-accessor fixtures over the *cf* interpretation, the run-ambiguity fixture over the *amb* interpretation. Together with the reused source-side lemmas of *Layer4-Fixtures-Core* and the constructed running-example facts of *Layer01-Virtual-Cut-Example-Inst*, the whole Layer 4 fixture bundle is established with no axioms anywhere.

theorem *layer4-fixture-bundle-constructed:*

$(\neg\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ b0\text{-}ex\ C\text{MissingInsert}\ H\text{-}ex\ l4\text{-}anchor)$
 $\wedge\ (\neg\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ b0\text{-}ex\ C\text{DeleteTouch}\ l4\text{-}delete\text{-}touch\text{-}H\ l4\text{-}anchor)$
 $\wedge\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ l4\text{-}empty\text{-}b0\ C\text{Empty}\ l4\text{-}empty\text{-}H\ l4\text{-}anchor$
 $\wedge\ (cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ b0\text{-}ex\ C\text{ClaimRefresh}\ H\text{-}ex\ l4\text{-}anchor$
 $\quad \wedge\ \neg\ cf.\text{virtual}\text{-}cut\ b0\text{-}ex\ C\text{ClaimRefresh}\ H\text{-}ex)$
 $\wedge\ (\neg\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ l4\text{-}after\text{-}frontier\text{-}b0\ C\text{AfterFrontier}$
 $\quad l4\text{-}after\text{-}frontier\text{-}H\ l4\text{-}after\text{-}frontier\text{-}anchor)$
 $\wedge\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ b0\text{-}ex\ C\text{UpdateRefresh}\ H\text{-}ex\ l4\text{-}anchor$
 $\wedge\ (\neg\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ b0\text{-}ex\ C\text{Unclaimed}\ H\text{-}ex\ l4\text{-}anchor)$
 $\quad \wedge\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ l4\text{-}b0\text{-}anchor\text{-}b0\ C\text{B0Anchor}\ l4\text{-}b0\text{-}anchor\text{-}H$
 $l4\text{-}anchor$
 $\wedge\ cf.\text{whole}\text{-}table\text{-}claim\text{-}scope\ l4\text{-}same\text{-}boundary\text{-}b0\ C\text{SameBoundary}\ l4\text{-}same\text{-}boundary\text{-}H$
 $l4\text{-}anchor$
 $\wedge\ (amb.\text{whole}\text{-}table\text{-}claim\text{-}scope\ b0\text{-}ex\ C\text{Ambig}\ H\text{-}ex\ l4\text{-}anchor$
 $\quad \wedge\ l4f\text{-}amb\text{-}chunks\ 0 \neq l4f\text{-}amb\text{-}chunks\ 1)$
using *cf-missing-insert-not-whole-table*
cf-delete-touch-not-whole-table
cf-empty-whole-table-claim-scope
cf-claim-refresh-whole-table cf-claim-refresh-not-virtual-cut

```

    cf-after-frontier-not-whole-table
    cf-update-refresh-whole-table
    cf-unclaimed-not-whole-table
    cf-b0-anchor-whole-table
    cf-same-boundary-whole-table
    amb-ambig-whole-table amb-chunks-differ
  by blast

end

```

```

theory Continuation-Fixtures-Inst
imports
  Continuation-Fixtures-Core
  DBLog-Cert-Substrate
begin

```

44 Constructed continuation fixtures

Constructed continuation fixtures. The negative / boundary continuation rows are stated over the plain source histories (*l7-H-cov* / *l7-H-future* / *l7-H-dup* / *l7-H-back* / *l7-H-del* / *l7-H-oos*) and segments, all ordinary **definitions**; those rows live in *DBLog-Virtual-Cuts.Continuation-Fixtures-Core* and are reused here by import. This theory constructs the one row that needs a certificate carrier: the “extended certificate” fixture, which separates accessor-level algebra from verifier acceptance.

The construction is a certificate-substrate interpretation at its smallest (the same shape as the *cf* interpretations of *Layer3-Fixtures-Inst* / *Layer4-Fixtures-Inst*): a one-constructor certificate datatype *l7f-cert* whose scope / frontier / clean-prefix / verifier functions carry exactly the extended-certificate values (scope $\{100\}$, frontier *ec1*, clean prefix $[Cdc\ ec1\ (Update\ (100::'a)\ (3000::'b))]$, verdict *Reject*), hosted by a **global-interpretation** *cert* of *dblog-cert-substrate* at a degenerate *unit* run (the row uses no run accessor). The accessor-level virtual cut is then established by the canonical-continuation lemmas (*l7-canon-continuation* + *l7-cov-canon-seg*), and the verifier rejection by *simp* — with no axioms.

44.1 Constructed extended certificate

```

datatype l7f-cert = CExt

fun l7f-scope :: l7f-cert  $\Rightarrow$  nat set where
  l7f-scope CExt =  $\{100\}$ 

fun l7f-frontier :: l7f-cert  $\Rightarrow$  frontier where
  l7f-frontier CExt = ec1

```

fun *l7f-clean-prefix* :: *l7f-cert* $\Rightarrow$ (*nat*, *nat*) *replay-event list* **where**
l7f-clean-prefix *CExt* = [*Cdc ec1* (*Update 100 3000*)]

fun *l7f-verify* :: *l7f-cert* $\Rightarrow$ *unit* $\Rightarrow$ *verify-result* **where**
l7f-verify *CExt* - = *Reject*

The run side is a degenerate *unit* run (the extended-certificate row uses no run accessor); *dblog-cert-substrate* assumes nothing beyond the two inherited *chunks-list* obligations (*set* [] = {}, *distinct* []), so the interpretation introduces no axiom. The faithfulness / materialization / binding / schema parameters are unused here and supplied as trivial constants.

global-interpretation *cext*: *dblog-cert-substrate*

$\lambda::unit.$ ({ } :: <i>nat set</i>)	— <i>scope-of</i>
$\lambda::unit.$ <i>c0</i>	— <i>frontier-of</i>
$\lambda::unit.$ ({ } :: <i>nat set</i>)	— <i>chunks</i>
$\lambda::unit.$ ([] :: (<i>nat</i> , <i>nat</i>) <i>src-history</i>)	— <i>src-history-of</i>
$\lambda::unit.$ $\lambda::nat.$ $\lambda::nat.$ (<i>None</i> :: <i>nat option option</i>)	— <i>chunk-read-result</i>
$\lambda::unit.$ ([] :: (<i>src-coord</i> $\times$ (<i>nat</i> , <i>nat</i>) <i>source-event</i>) <i>list</i>)	— <i>cdc-events-of</i>
$\lambda::unit.$ ([] :: <i>nat list</i>)	— <i>chunks-list</i>
$\lambda::unit.$ $\lambda::nat.$ ({ } :: <i>nat set</i>)	— <i>chunk-domain</i>
$\lambda::unit.$ $\lambda::nat.$ (<i>None</i> :: <i>nat option</i>)	— <i>responsible-chunk</i>
$\lambda::unit.$ $\lambda::nat.$ <i>c0</i>	— <i>chunk-lower-watermark</i>
$\lambda::unit.$ $\lambda::nat.$ <i>c0</i>	— <i>chunk-upper-watermark</i>
$\lambda::unit.$ $\lambda::nat.$ <i>c0</i>	— <i>chunk-read-coordinate</i>
<i>l7f-scope</i> <i>l7f-frontier</i> <i>l7f-clean-prefix</i>	
<i>l7f-verify</i> <i>Accept</i>	
$\lambda::unit.$ $\lambda::(nat \rightarrow nat).$ $\lambda::(nat, nat)$ <i>src-history.</i> $\lambda::unit.$ <i>False</i>	— <i>faithful-source-observation</i>
$\lambda::l7f-cert.$ $\lambda::unit.$ $\lambda::unit.$ <i>False</i>	— <i>materializes-run</i>
$\lambda::l7f-cert.$ $\lambda::unit.$ <i>False</i>	— <i>evidence-bound-to-cert</i>
$\lambda::l7f-cert.$ $\lambda::unit.$ <i>False</i>	— <i>source-side-schema-supported</i>
by <i>unfold-locales simp-all</i>	

44.2 Accessor equality of an extended certificate is not verifier acceptance (constructed)

Read through its accessors, *CExt* satisfies the accessor-level *cext.virtual-cut* predicate (its clean prefix replays to the source state on its scope at its frontier, by the canonical-continuation lemmas); yet the verifier rejects it. Accessor-level algebra is strictly weaker than verifier acceptance.

lemma *cext-extended-cert-virtual-cut*:

cext.virtual-cut ($\lambda.$ *None*) *CExt* *l7-H-cov*

unfolding *cext.virtual-cut-def* *virtual-cut-state-def*

proof (*rule* *l7-canon-continuation*[*OF* *l7-wf-cov*, *unfolded virtual-cut-state-def*])

show *cdc-segment-between* *l7-H-cov* (*l7f-scope* *CExt*) *c0*

(*l7f-frontier* *CExt*) (*l7f-clean-prefix* *CExt*)

by (*simp add*: *l7-cov-canon-seg*)

qed

theorem *fix-l7-extended-cert-not-accepted-constructed:*

cext.virtual-cut ($\lambda\cdot$. None) *CExt l7-H-cov*
 $\wedge$ *l7f-verify* *CExt* () $\neq$ *Accept*
using *cext-extended-cert-virtual-cut* **by** *simp*

44.3 Constructed continuation negative / boundary fixture bundle

The eleven negative / boundary continuation conjuncts in one statement, with the two extended-certificate conjuncts over the construction (*cext.virtual-cut* on *CExt* and *l7f-verify*); the remaining conjuncts are the carrier-independent fixtures of *DBLog-Virtual-Cuts.Continuation-Fixtures-Core*, reused by import. Of the twelve *fix-l7-* facts, the out-of-scope padding boundary fact (*fix-l7-out-of-scope-padding-rejected-not-loadbearing*, in *DBLog-Virtual-Cuts.Continuation-Fixtures-Core*) is deliberately not bundled: it exhibits a padded segment that *cdc-segment-between* rejects while the continuation conclusion still holds, demonstrating a non-load-bearing premise rather than a rejection. The whole continuation negative-fixture bundle is established with no axioms anywhere.

theorem *continuation-negative-fixture-bundle-constructed:*

$\neg$ *virtual-cut-state* ($\lambda\cdot$. None) $\square$ {100} *ec1 l7-H-cov*
 $\wedge$ $\neg$ *virtual-cut-state* ($\lambda\cdot$. None)
 $\quad$ [*Cdc ec1* (*Update* 100 7777)] {100} *ec1 l7-H-cov*
 $\wedge$ $\neg$ *virtual-cut-state* ($\lambda\cdot$. None)
 $\quad$ [*Cdc ec1* (*Update* 100 3000), *Cdc ec2* (*Update* 100 9000)]
 $\quad$ {100} *ec1 l7-H-future*
 $\wedge$ $\neg$ *virtual-cut-state* ($\lambda\cdot$. None)
 $\quad$ [*Cdc ec1* (*Update* 100 2), *Cdc ec1* (*Update* 100 1)] {100} *ec1 l7-H-dup*
 $\wedge$ $\neg$ *virtual-cut-state* ($\lambda\cdot$. None)
 $\quad$ [*Cdc ec1* (*Update* 100 1)] {100} *ec1 l7-H-dup*
 $\wedge$ *sorted* (*map* *cdc-coord* [*Cdc ec1* (*Update* 100 2), *Cdc ec1* (*Update* 100 1)])
 $\wedge$ $\neg$ *cdc-segment-between* *l7-H-dup* {100} *c0 ec1*
 $\quad$ [*Cdc ec1* (*Update* 100 2), *Cdc ec1* (*Update* 100 1)]
 $\wedge$ $\neg$ *virtual-cut-state* ($\lambda\cdot$. None)
 $\quad$ [*Cdc ec1* (*Update* 100 2), *Cdc ec1* (*Update* 100 1)] {100} *ec1 l7-H-dup*
 $\wedge$ ($\forall \delta$. $\neg$ *cdc-segment-between* *l7-H-back* {100} *ec2 ec1* δ)
 $\wedge$ $\neg$ *virtual-cut-state* ($\lambda\cdot$. None)
 $\quad$ [*Cdc ec1* (*Update* 100 3000)] {100} *ec2 l7-H-del*
 $\wedge$ *Apply* [*Cdc ec1* (*Update* 100 3000)] $\neq$ *Src* ($\lambda\cdot$. None) *l7-H-oos ec2*
 $\wedge$ *cext.virtual-cut* ($\lambda\cdot$. None) *CExt l7-H-cov*
 $\wedge$ *l7f-verify* *CExt* () $\neq$ *Accept*
 $\wedge$ *virtual-cut-state* ($\lambda\cdot$. None) (*l7-seg1* @ *l7-seg2*) *l7-scope ec4 H-ex*
 $\wedge$ ($\forall \sigma'$. *virtual-cut-state* ($\lambda\cdot$. None) σ' *l7-scope ec4 H-ex*
 $\quad \longrightarrow$ *restrict* (*Apply* σ') *l7-scope*
 $\quad =$ *restrict* (*Apply* (*l7-seg1* @ *l7-seg2*)) *l7-scope*)


```

using fix-l7-missing-event fix-l7-event-not-in-history fix-l7-future-event
      fix-l7-same-coordinate-order fix-l7-multiplicity
      fix-l7-coordinate-sorted-insufficient fix-l7-frontier-order
      fix-l7-delete-omitted fix-l7-scoped-not-whole-table
      fix-l7-extended-cert-not-accepted-constructed fix-l7-rechunk-not-sink-repair
by blast

end

```

```

theory Public-Checker-Witness
  imports Layer4-Whole-Table Continuation Layer4-Witnesses-Core
begin

```

45 Public accepting witness for the Layer 3 checker (non-vacuity gate)

This theory exhibits a concrete certificate / evidence pair over the *public* carriers (*cert*, *evidence*, the concrete *run* datatype) that the public verifier *verify* genuinely accepts, together with a deployment environment (*b0-ex*, *H-ex*) under which *faithful-source-observation* genuinely holds. It then exercises the Layer 3 / Layer 4 / continuation headline theorems at that pair, so the public headline implications are demonstrably non-vacuous — with no axiom and no locale-instance detour.

The construction is the paper’s running example (the accounts-table backfill: three in-scope keys, two chunks, four CDC events, a seven-event clean prefix), realized as a value *ex-run* of the concrete *run* datatype — the global-carrier sibling of the *ex-model* interpretation in *Layer01-Virtual-Cut-Example-Inst*.

Two *independence exhibits* close the theory: an accepted pair whose observation is unfaithful (against a wrong deployment base state) and a faithful evidence bundle the verifier does not accept (a broken chunk plan). Together they certify that the two headline premises *verify C E = Accept* and *faithful-source-observation E b0 H Raw* are logically independent — neither subsumes the other, and neither alone entails the wellformed-run conclusion — which is the formal non-circularity of the mechanical / external factoring in *DBLog-Virtual-Cuts.Virtual-Cut*.

Regression gate. The capstone theorem *public-checker-nonvacuous* keeps this non-vacuity machine-checked: any change that degenerates the public checker (e.g. into *verify C E = Reject* or *faithful-source-observation E b0 H Raw = False*), or that weakens it until acceptance or faithfulness is unsatisfiable or the checker obligations fail, breaks this theory and hence the build.

45.1 The running-example run over the concrete run datatype

Chunk plan: chunk 0 owns {100, 200} and reads at *ec1*; chunk 1 owns {300} and reads at *ec3*. Watermarks coincide with the read coordinates, as in the running example.

definition *exr-dom* :: nat $\Rightarrow$ nat set **where**

exr-dom ch = (if ch = 0 then {100, 200} else if ch = 1 then {300} else {})

definition *exr-resp* :: nat $\Rightarrow$ nat option **where**

exr-resp k =
 (if k = 100 $\vee$ k = 200 then Some 0
 else if k = 300 then Some 1 else None)

definition *exr-coord* :: nat $\Rightarrow$ src-coord **where**

exr-coord ch = (if ch = 0 then *ec1* else *ec3*)

definition *exr-read* :: nat $\Rightarrow$ nat $\Rightarrow$ nat option option **where**

exr-read ch k =
 (if ch = 0 $\wedge$ k = 100 then Some (Some 3000)
 else if ch = 0 $\wedge$ k = 200 then Some (Some 10000)
 else if ch = 1 $\wedge$ k = 300 then Some (Some 2500)
 else None)

definition *ex-run* :: (nat, nat) run **where**

ex-run =
 Run {100, 200, 300} *ec4* [0, 1] *H-ex* *H-ex*
exr-read *exr-dom* *exr-resp* *exr-coord* *exr-coord* *exr-coord*

45.2 Accessor values

lemma *exr-scope*: scope-of *ex-run* = {100, 200, 300}

by (simp add: scope-of-def *ex-run-def*)

lemma *exr-frontier*: frontier-of *ex-run* = *ec4*

by (simp add: frontier-of-def *ex-run-def*)

lemma *exr-chunks*: chunks *ex-run* = {0, 1}

by (simp add: chunks-def *ex-run-def*)

lemma *exr-chunks-list*: chunks-list *ex-run* = [0, 1]

by (simp add: chunks-list-def *ex-run-def*)

lemma *exr-hist*: src-history-of *ex-run* = *H-ex*

by (simp add: src-history-of-def *ex-run-def*)

lemma *exr-cdc*: cdc-events-of *ex-run* = *H-ex*

by (simp add: cdc-events-of-def *ex-run-def*)

lemma *exr-chunk-domain*: *chunk-domain ex-run = exr-dom*
by (*simp add: chunk-domain-def ex-run-def*)

lemma *exr-responsible*: *responsible-chunk ex-run = exr-resp*
by (*simp add: responsible-chunk-def ex-run-def*)

lemma *exr-rcoord*: *chunk-read-coordinate ex-run = exr-coord*
by (*simp add: chunk-read-coordinate-def ex-run-def*)

lemma *exr-lwm*: *chunk-lower-watermark ex-run = exr-coord*
by (*simp add: chunk-lower-watermark-def ex-run-def*)

lemma *exr-uwm*: *chunk-upper-watermark ex-run = exr-coord*
by (*simp add: chunk-upper-watermark-def ex-run-def*)

lemma *exr-rresult*: *chunk-read-result ex-run = exr-read*
by (*simp add: chunk-read-result-def ex-run-def*)

45.3 Determinate clean prefix and Apply

lemma *exr-clean-prefix*:

clean-prefix-of ex-run
 = [*Cdc ec1 (Update 100 3000)*,
 Refresh (100::nat) (Some (3000::nat)) ec1,
 Refresh 200 (Some 10000) ec1,
 Cdc ec2 (Insert 300 2500),
 Cdc ec3 (Update 200 12000),
 Refresh 300 (Some 2500) ec3,
 Cdc ec4 (Update 300 3500)]

proof –

have *dA*: *sorted-list-of-set {100::nat, 200} = [100, 200]* **by** *simp*

have *dB*: *sorted-list-of-set {300::nat} = [300]* **by** *simp*

show *?thesis*

unfolding *clean-prefix-of-def canonical-clean-prefix-def*

cdc-event-replays-def chunk-read-refreshes-def

by (*simp add: exr-chunks-list exr-chunk-domain exr-rcoord exr-rresult*

exr-dom-def exr-coord-def exr-read-def

exr-cdc exr-scope exr-frontier H-ex-def dA dB

List.map-filter-simps sort-key-def

ec1-le-ec2[*unfolded src-le-eq-less-eq*]

ec1-le-ec3[*unfolded src-le-eq-less-eq*]

ec1-le-ec4[*unfolded src-le-eq-less-eq*]

ec2-le-ec3[*unfolded src-le-eq-less-eq*]

ec2-le-ec4[*unfolded src-le-eq-less-eq*]

ec3-le-ec4[*unfolded src-le-eq-less-eq*]

not-src-le-ec2-ec1[*unfolded src-le-eq-less-eq*]

not-src-le-ec3-ec1[*unfolded src-le-eq-less-eq*]

not-src-le-ec4-ec1[*unfolded src-le-eq-less-eq*]

not-src-le-ec3-ec2[*unfolded src-le-eq-less-eq*]

```

      not-src-le-ec4-ec2[unfolded src-le-eq-less-eq]
      not-src-le-ec4-ec3[unfolded src-le-eq-less-eq])
qed

lemma exr-apply:
  Apply (clean-prefix-of ex-run)
    = (λk. if k = (100::nat) then Some (3000::nat)
      else if k = 200 then Some 12000
      else if k = 300 then Some 3500
      else None)
proof -
  have step:
    Apply (clean-prefix-of ex-run)
      = foldl apply-step (λ-. None)
        [ Cdc ec1 (Update 100 3000),
          Refresh (100::nat) (Some (3000::nat)) ec1,
          Refresh 200 (Some 10000) ec1,
          Cdc ec2 (Insert 300 2500),
          Cdc ec3 (Update 200 12000),
          Refresh 300 (Some 2500) ec3,
          Cdc ec4 (Update 300 3500) ]
    unfolding Apply-def exr-clean-prefix by (rule refl)
  show ?thesis
    unfolding step by (auto simp: fun-eq-iff fun-upd-def)
qed

```

45.4 The running-example run is wellformed (global carrier)

```

theorem ex-run-wellformed:
  wellformed-dblog-run b0-ex ex-run H-ex
  unfolding wellformed-dblog-run-def
proof (intro conjI)
  show src-history-of ex-run = H-ex by (rule exr-hist)
next
  show wellformed-src-history H-ex by (rule wf-h-ex)
next
  show ∀ k ∈ scope-of ex-run.
    ∃! ch. ch ∈ chunks ex-run ∧ k ∈ chunk-domain ex-run ch
  proof
    fix k :: nat assume k ∈ scope-of ex-run
    hence k-cases: k = 100 ∨ k = 200 ∨ k = 300
      using exr-scope by auto
    show ∃! ch. ch ∈ chunks ex-run ∧ k ∈ chunk-domain ex-run ch
  proof (rule ex1I[of - if k = 300 then 1 else (0::nat)])
    show (if k = 300 then 1 else (0::nat)) ∈ chunks ex-run
      ∧ k ∈ chunk-domain ex-run (if k = 300 then 1 else 0)
      using k-cases
      by (auto simp: exr-chunks exr-chunk-domain exr-dom-def)
  next

```

```

    fix ch
    assume ch ∈ chunks ex-run ∧ k ∈ chunk-domain ex-run ch
    thus ch = (if k = 300 then 1 else 0)
      using k-cases
    by (auto simp: exr-chunks exr-chunk-domain exr-dom-def
        split: if-split-asm)

  qed
  qed
next
  show ∀ ch1 ∈ chunks ex-run. ∀ ch2 ∈ chunks ex-run.
    ch1 ≠ ch2
    → chunk-domain ex-run ch1 ∩ chunk-domain ex-run ch2 = {}
    by (auto simp: exr-chunks exr-chunk-domain exr-dom-def)
next
  show canonical-chunk-ownership-domain ex-run = scope-of ex-run
    unfolding canonical-chunk-ownership-domain-def
    by (auto simp: exr-chunks exr-chunk-domain exr-dom-def exr-scope)
next
  show ∀ k ∈ scope-of ex-run. ∀ ch.
    responsible-chunk ex-run k = Some ch
    ↔ ch ∈ chunks ex-run ∧ k ∈ chunk-domain ex-run ch
  proof (intro ballI allI)
    fix k :: nat and ch :: nat
    assume k ∈ scope-of ex-run
    hence k-cases: k = 100 ∨ k = 200 ∨ k = 300
      using exr-scope by auto
    show responsible-chunk ex-run k = Some ch
      ↔ ch ∈ chunks ex-run ∧ k ∈ chunk-domain ex-run ch
      using k-cases
    by (auto simp: exr-responsible exr-resp-def exr-chunks
        exr-chunk-domain exr-dom-def
        split: if-split-asm)

  qed
next
  show ∀ k. k ∉ scope-of ex-run → responsible-chunk ex-run k = None
    by (auto simp: exr-scope exr-responsible exr-resp-def)
next
  show finite (scope-of ex-run) by (simp add: exr-scope)
next
  show ∀ ch ∈ chunks ex-run. finite (chunk-domain ex-run ch)
    by (auto simp: exr-chunks exr-chunk-domain exr-dom-def)
next
  show ∀ ch ∈ chunks ex-run. chunk-domain ex-run ch ≠ {}
    by (auto simp: exr-chunks exr-chunk-domain exr-dom-def)
next
  show ∀ k ∈ scope-of ex-run.
    covers-ordinary-cdc ex-run k (frontier-of ex-run)
  proof
    fix k :: nat assume k ∈ scope-of ex-run

```

```

hence k-cases:  $k = 100 \vee k = 200 \vee k = 300$ 
  using exr-scope by auto
show covers-ordinary-cdc ex-run k (frontier-of ex-run)
  unfolding covers-ordinary-cdc-def
proof (intro exI[where  $x = \text{if } k = 300 \text{ then } 1 \text{ else } (0::\text{nat})$ ]]
  conjI allI impI)
  show responsible-chunk ex-run k = Some (if k = 300 then 1 else 0)
    using k-cases by (auto simp: exr-responsible exr-resp-def)
next
  — The observed CDC log is the full recorded history, so every history entry is
  trivially present in it.
  fix i
  assume  $i < \text{length } (\text{src-history-of } \text{ex-run})$ 
  thus  $\text{src-history-of } \text{ex-run} ! i \in \text{set } (\text{cdc-events-of } \text{ex-run})$ 
    by (simp add: exr-hist exr-cdc)
qed
qed
next
show  $\forall p \in \text{set } (\text{cdc-events-of } \text{ex-run}).$ 
   $\text{key-of } (\text{hist-event } p) \in \text{scope-of } \text{ex-run}$ 
   $\wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } \text{ex-run})$ 
   $\longrightarrow p \in \text{set } (\text{src-history-of } \text{ex-run})$ 
  by (simp add: exr-cdc exr-hist)
next
show  $\exists \iota :: \text{nat} \Rightarrow \text{nat}. \text{strict-mono } \iota$ 
   $\wedge (\text{let } \text{in-slice} = (\lambda p. \text{key-of } (\text{hist-event } p) \in \text{scope-of } \text{ex-run}$ 
     $\wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } \text{ex-run}));$ 
     $\text{cdc-in-slice} = \text{filter in-slice } (\text{cdc-events-of } \text{ex-run});$ 
     $\text{hist-in-slice} = \text{filter in-slice } (\text{src-history-of } \text{ex-run})$ 
    in  $\forall i. i < \text{length } \text{cdc-in-slice} \longrightarrow$ 
       $\iota i < \text{length } \text{hist-in-slice}$ 
       $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! \iota i)$ 
proof (intro exI[where  $x = \lambda n :: \text{nat}. n$ ] conjI)
  show strict-mono  $(\lambda n :: \text{nat}. n)$  by (simp add: strict-mono-def)
next
  show  $\text{let } \text{in-slice} = (\lambda p. \text{key-of } (\text{hist-event } p) \in \text{scope-of } \text{ex-run}$ 
     $\wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } \text{ex-run}));$ 
     $\text{cdc-in-slice} = \text{filter in-slice } (\text{cdc-events-of } \text{ex-run});$ 
     $\text{hist-in-slice} = \text{filter in-slice } (\text{src-history-of } \text{ex-run})$ 
    in  $\forall i. i < \text{length } \text{cdc-in-slice} \longrightarrow$ 
       $(\lambda n :: \text{nat}. n) i < \text{length } \text{hist-in-slice}$ 
       $\wedge \text{cdc-in-slice} ! i = \text{hist-in-slice} ! (\lambda n :: \text{nat}. n) i$ 
    by (simp add: Let-def exr-cdc exr-hist)
qed
next
show  $\forall k \in \text{scope-of } \text{ex-run}. \forall ch \ c.$ 
   $\text{responsible-chunk } \text{ex-run } k = \text{Some } ch$ 
   $\wedge \text{src-lt } (\text{chunk-read-coordinate } \text{ex-run } ch) \ c$ 
   $\wedge \text{src-le } c (\text{frontier-of } \text{ex-run})$ 

```

```

      →
      mset (source-key-coord-slice k c (cdc-events-of ex-run))
      = mset (source-key-coord-slice k c (src-history-of ex-run))
    by (simp add: exr-cdc exr-hist)
next
show ∀ ch ∈ chunks ex-run.
  src-le (chunk-lower-watermark ex-run ch)
    (chunk-read-coordinate ex-run ch)
  ∧ src-le (chunk-read-coordinate ex-run ch)
    (chunk-upper-watermark ex-run ch)
  ∧ (∀ k ∈ chunk-domain ex-run ch.
    ∃ m. chunk-read-result ex-run ch k = Some m)
  ∧ (∀ k ∈ chunk-domain ex-run ch. ∀ m.
    chunk-read-result ex-run ch k = Some m
    ↔ Refresh k m (chunk-read-coordinate ex-run ch)
    ∈ set (clean-prefix-of ex-run))
proof
  fix ch :: nat
  assume ch ∈ chunks ex-run
  hence ch-cases: ch = 0 ∨ ch = 1 by (auto simp: exr-chunks)
  show src-le (chunk-lower-watermark ex-run ch)
    (chunk-read-coordinate ex-run ch)
    ∧ src-le (chunk-read-coordinate ex-run ch)
    (chunk-upper-watermark ex-run ch)
    ∧ (∀ k ∈ chunk-domain ex-run ch.
    ∃ m. chunk-read-result ex-run ch k = Some m)
    ∧ (∀ k ∈ chunk-domain ex-run ch. ∀ m.
    chunk-read-result ex-run ch k = Some m
    ↔ Refresh k m (chunk-read-coordinate ex-run ch)
    ∈ set (clean-prefix-of ex-run))
proof (cases ch = 0)
  case True
  show ?thesis
  using True exr-lwm exr-uwm exr-rcoord
    src-le-refl
    exr-chunk-domain exr-rresult
    exr-clean-prefix
  by (auto simp: exr-coord-def exr-dom-def exr-read-def
    ec1-neq-ec3 ec1-neq-ec2 ec2-neq-ec3)
next
  case False
  hence ch-1: ch = 1 using ch-cases by simp
  show ?thesis
  using ch-1 exr-lwm exr-uwm exr-rcoord
    src-le-refl
    exr-chunk-domain exr-rresult
    exr-clean-prefix
  by (auto simp: exr-coord-def exr-dom-def exr-read-def
    ec1-neq-ec3 ec1-neq-ec2 ec2-neq-ec3)

```

```

    qed
  qed
next
  show  $\forall ch \in \text{chunks } ex\text{-run}.$ 
     $\text{src-le } (\text{chunk-read-coordinate } ex\text{-run } ch) (\text{frontier-of } ex\text{-run})$ 
  by (auto simp: exr-chunks exr-rcoord exr-coord-def exr-frontier
    ec1-le-ec4 ec3-le-ec4)
next
  show  $\forall ch \in \text{chunks } ex\text{-run}. \forall k \in \text{chunk-domain } ex\text{-run } ch. \forall m.$ 
     $\text{chunk-read-result } ex\text{-run } ch \ k = \text{Some } m$ 
     $\longrightarrow m = \text{Src } b0\text{-ex } H\text{-ex } (\text{chunk-read-coordinate } ex\text{-run } ch) \ k$ 
  by (auto simp: exr-chunks exr-chunk-domain exr-dom-def
    exr-rresult exr-read-def exr-rcoord exr-coord-def
    Src-b0-ex-H-ex-ec1-100 Src-b0-ex-H-ex-ec1-200
    Src-b0-ex-H-ex-ec3-300
    split: if-split-asm)
next
  show  $\forall ch \in \text{chunks } ex\text{-run}. \forall k \in \text{chunk-domain } ex\text{-run } ch.$ 
     $\text{chunk-read-result } ex\text{-run } ch \ k = \text{Some } \text{None}$ 
     $\longrightarrow \text{row-absence-meaningful } b0\text{-ex } H\text{-ex } ex\text{-run } ch \ k$ 
  by (auto simp: exr-chunks exr-chunk-domain exr-dom-def
    exr-rresult exr-read-def
    split: if-split-asm)
next
  show  $\forall p \in \text{set } (\text{cdc-events-of } ex\text{-run}).$ 
     $\text{key-of } (\text{hist-event } p) \in \text{scope-of } ex\text{-run}$ 
     $\wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } ex\text{-run})$ 
     $\longrightarrow \text{Cdc } (\text{hist-coord } p) (\text{hist-event } p)$ 
     $\in \text{set } (\text{clean-prefix-of } ex\text{-run})$ 
  using wf7-clean-prefix-cdc-coherence-redundant[of ex-run] by blast
next
  show  $\forall e \ c. \text{Cdc } c \ e \in \text{set } (\text{clean-prefix-of } ex\text{-run})$ 
     $\wedge \text{key-of } e \in \text{scope-of } ex\text{-run}$ 
     $\wedge \text{src-le } c (\text{frontier-of } ex\text{-run})$ 
     $\longrightarrow (c, e) \in \text{set } (\text{cdc-events-of } ex\text{-run})$ 
  using wf7-clean-prefix-cdc-coherence-redundant[of ex-run] by blast
qed

```

The two halves of the factoring, extracted once for reuse.

lemma *ex-run-checker-wellformed: checker-run-wellformed ex-run*
using *ex-run-wellformed wellformed-dblog-run-decompose* **by** *blast*

lemma *ex-run-reflects-source: run-reflects-source b0-ex ex-run H-ex*
using *ex-run-wellformed wellformed-dblog-run-decompose* **by** *blast*

45.5 The accepting public certificate / evidence pair

definition *ex-cert* :: $(\text{nat}, \text{nat}) \text{ cert}$ **where**
ex-cert = *Cert* $\{100, 200, 300\} \ ec4 \ (\text{clean-prefix-of } ex\text{-run})$

definition *ex-evidence* :: (nat, nat) evidence **where**
ex-evidence = Evidence *ex-run*

lemma *ex-cert-scope*: scope *ex-cert* = {100, 200, 300}
by (simp add: scope-def *ex-cert-def*)

lemma *ex-cert-frontier*: frontier *ex-cert* = *ec4*
by (simp add: frontier-def *ex-cert-def*)

lemma *ex-cert-clean-prefix*: clean-prefix *ex-cert* = clean-prefix-of *ex-run*
by (simp add: clean-prefix-def *ex-cert-def*)

lemma *ex-evidence-bound*: evidence-bound-to-cert *ex-cert* *ex-evidence*
by (simp add: evidence-bound-to-cert-def cert-accessors-agree-def
ex-evidence-def *ex-cert-scope* *ex-cert-frontier*
ex-cert-clean-prefix *exr-scope* *exr-frontier*)

lemma *ex-evidence-schema*: source-side-schema-supported *ex-cert* *ex-evidence*
by (simp add: source-side-schema-supported-def *ex-evidence-def*
ex-run-checker-wellformed)

theorem *ex-verify-accept*: verify *ex-cert* *ex-evidence* = Accept
by (simp add: verify-def *ex-evidence-schema* *ex-evidence-bound*)

theorem *ex-faithful*:
faithful-source-observation *ex-evidence* *b0-ex* *H-ex* RawObservation
by (simp add: faithful-source-observation-def *ex-evidence-def*
ex-run-reflects-source)

45.6 Exercising the headline theorems at the public witness

Layer 3 intermediate theorem *accepted-certificate-implies-wellformed-run* at the public witness: the accepted pair has a wellformed, coherent materialized run.

theorem *public-accepted-certificate-wellformed-run*:
 $\exists R. \text{wellformed-dblog-run } b0\text{-ex } R \text{ } H\text{-ex} \wedge \text{cert-run-coherent } ex\text{-cert } ex\text{-evidence } R$
by (rule layer3-checker-substrate.accepted-certificate-implies-wellformed-run
[OF layer3-checker-substrate-holds *ex-verify-accept* *ex-faithful*])

Layer 3 headline *accepted-virtual-cut-sound* at the public witness: the accepted certificate's virtual cut holds.

theorem *public-accepted-virtual-cut*:
virtual-cut *b0-ex* *ex-cert* *H-ex*
by (rule layer3-checker-substrate.accepted-virtual-cut-sound
[OF layer3-checker-substrate-holds *ex-verify-accept* *ex-faithful*])

Layer 4 whole-table side conditions at anchor *ec1*, over the pure *DBLog-Virtual-Cuts.Virtual-Cut-Core* predicates.

```

lemma ex-whole-table-claim-scope:
  whole-table-claim-scope b0-ex ex-cert H-ex ec1
  unfolding whole-table-claim-scope-def
proof (intro conjI)
  show  $ec1 \leq \text{frontier } ex\text{-cert}$ 
    using ec1-le-ec4 by (simp add: ex-cert-frontier less-eq-src-coord-def)
next
  show anchor-complete-at-boundary b0-ex H-ex ec1 (scope ex-cert)
    unfolding anchor-complete-at-boundary-def
    using l4-anchor-domain[unfolded l4-anchor-def]
    by (simp add: ex-cert-scope)
next
  show all-post-anchor-changes-covered H-ex ec1 (frontier ex-cert) (scope ex-cert)
    unfolding all-post-anchor-changes-covered-def touched-between-def
    by (auto simp: ex-cert-scope H-ex-def)
next
  show no-unclaimed-replay-keys (clean-prefix ex-cert) (scope ex-cert)
    unfolding no-unclaimed-replay-keys-def replay-keys-def
    by (auto simp: ex-cert-clean-prefix ex-cert-scope ex-clean-prefix)
qed

```

Layer 4 specialization *accepted-whole-table-anchor-domain-specialization*: unrestricted whole-table equality at the accepted certificate.

```

theorem public-accepted-whole-table-equality:
  Apply (clean-prefix ex-cert) = Src b0-ex H-ex (frontier ex-cert)
  by (rule layer3-checker-substrate.accepted-whole-table-anchor-domain-specialization
    [OF layer3-checker-substrate-holds ex-verify-accept ex-faithful
      ex-whole-table-claim-scope])

```

Continuation corollary *accepted-certificate-continuation-sound*: the accepted certificate's cut extends to the strictly later frontier *coord-of-nat 5* by the (here empty) faithful continuation segment for the half-open interval.

```

lemma ex-cdc-segment-to-later-frontier:
  cdc-segment-between H-ex (scope ex-cert) (frontier ex-cert) (coord-of-nat 5) []
  unfolding cdc-segment-between-def ex-cert-frontier
  by (simp add: H-ex-def ec-defs)

```

```

theorem public-accepted-continuation:
  virtual-cut-state b0-ex (clean-prefix ex-cert @ []) (scope ex-cert)
    (coord-of-nat 5) H-ex
  by (rule layer3-checker-substrate.accepted-certificate-continuation-sound
    [OF layer3-checker-substrate-holds ex-verify-accept ex-faithful
      wf-h-ex ex-cdc-segment-to-later-frontier])

```

45.7 Exercising the remaining state-level headlines: Layer 2 and sub-scope restriction

Layer 2 (*wellformed-run-implies-virtual-cut*) exercised at the running-example run: the wellformed public run replays, on its scope, to the source

theorem *public-wellformed-run-virtual-cut:*
virtual-cut-state *b0-ex* (*clean-prefix-of ex-run*) (*scope-of ex-run*)
(*frontier-of ex-run*) *H-ex*
by (*rule wellformed-run-implies-virtual-cut*[*OF ex-run-wellformed*])

theorem *public-accepted-subscope-restriction:*
 $\{100, 300\} \neq (\{\} :: \text{nat set})$
 $\wedge \{100, 300\} \subset \text{scope } \text{ex-cert}$
 $\wedge \text{virtual-cut-state } \text{b0-ex (clean-prefix ex-cert) } \{100, 300\}$
(frontier ex-cert) H-ex

State-level sub-scope restriction (*virtual-cut-state-restrict-scope*), exercised at the same accepted cut — *public-accepted-virtual-cut* unfolded through *virtual-cut-def* — and the proper, non-empty singleton sub-scope $\{200\}$.

299

```

       $\wedge$  virtual-cut-state b0-ex (clean-prefix ex-cert) {200}
      (frontier ex-cert) H-ex
proof (intro conjI)
  show {200}  $\neq$  ({}) :: nat set by simp
next
  show {200}  $\subset$  scope ex-cert
  proof (rule psubsetI)
    show {200}  $\subseteq$  scope ex-cert
    unfolding ex-cert-scope by simp
  next
  show {200}  $\neq$  scope ex-cert
  proof
    assume eq: {200} = scope ex-cert
    have (100::nat)  $\in$  scope ex-cert
    unfolding ex-cert-scope by simp
    with eq have (100::nat)  $\in$  {200} by simp
    thus False by simp
  qed
qed
next
  have cut: virtual-cut-state b0-ex (clean-prefix ex-cert) (scope ex-cert)
    (frontier ex-cert) H-ex
    using public-accepted-virtual-cut unfolding virtual-cut-def .
  have {200}  $\subseteq$  scope ex-cert
  unfolding ex-cert-scope by simp
  with cut show virtual-cut-state b0-ex (clean-prefix ex-cert) {200}
    (frontier ex-cert) H-ex
    by (rule virtual-cut-state-restrict-scope)
qed

```

45.8 A mid-frontier accepted certificate and non-empty continuation segments (the active-interval continuation case)

public-accepted-continuation above exercises *accepted-certificate-continuation-sound* with the *empty* continuation segment — the quiescent-interval case of continuation (no in-scope source event lies in the half-open interval, and the certified cut persists at the strictly later frontier). The theorems below exercise the continuation headlines in their *active-interval* case, with a non-empty segment that genuinely replays a post-frontier source event.

The witness is a second accepted certificate over the *same* running example: the same source history *H-ex*, the same chunk plan, chunk reads, and watermarks, but certified at the *mid* frontier *ec3* — so the recorded history genuinely extends past the certified frontier, and the faithful continuation segment for (*ec3*, *ec4*] on the certified scope is the non-empty singleton [*Cdc ec4* (*Update 300 3500*)]. Its canonical clean prefix is the running example's first six replay events (the *ec4* CDC event now lies beyond the certified frontier).

Continuation then lands exactly on the state that the full-frontier certificate *ex-cert* certifies directly (*public-continuation-meets-direct-certification* below).

definition *ex-run-mid* :: (nat, nat) run where

ex-run-mid =
 Run {100, 200, 300} ec3 [0, 1] H-ex H-ex
 exr-read exr-dom exr-resp exr-coord exr-coord exr-coord

lemma *mid-scope*: scope-of *ex-run-mid* = {100, 200, 300}
 by (simp add: scope-of-def *ex-run-mid-def*)

lemma *mid-frontier*: frontier-of *ex-run-mid* = ec3
 by (simp add: frontier-of-def *ex-run-mid-def*)

lemma *mid-chunks*: chunks *ex-run-mid* = {0, 1}
 by (simp add: chunks-def *ex-run-mid-def*)

lemma *mid-chunks-list*: chunks-list *ex-run-mid* = [0, 1]
 by (simp add: chunks-list-def *ex-run-mid-def*)

lemma *mid-hist*: src-history-of *ex-run-mid* = H-ex
 by (simp add: src-history-of-def *ex-run-mid-def*)

lemma *mid-cdc*: cdc-events-of *ex-run-mid* = H-ex
 by (simp add: cdc-events-of-def *ex-run-mid-def*)

lemma *mid-chunk-domain*: chunk-domain *ex-run-mid* = exr-dom
 by (simp add: chunk-domain-def *ex-run-mid-def*)

lemma *mid-responsible*: responsible-chunk *ex-run-mid* = exr-resp
 by (simp add: responsible-chunk-def *ex-run-mid-def*)

lemma *mid-rcoord*: chunk-read-coordinate *ex-run-mid* = exr-coord
 by (simp add: chunk-read-coordinate-def *ex-run-mid-def*)

lemma *mid-lwm*: chunk-lower-watermark *ex-run-mid* = exr-coord
 by (simp add: chunk-lower-watermark-def *ex-run-mid-def*)

lemma *mid-uwm*: chunk-upper-watermark *ex-run-mid* = exr-coord
 by (simp add: chunk-upper-watermark-def *ex-run-mid-def*)

lemma *mid-rresult*: chunk-read-result *ex-run-mid* = exr-read
 by (simp add: chunk-read-result-def *ex-run-mid-def*)

lemma *mid-clean-prefix*:

clean-prefix-of ex-run-mid
 = [Cdc ec1 (Update 100 3000),
 Refresh (100::nat) (Some (3000::nat)) ec1,
 Refresh 200 (Some 10000) ec1,

```

      Cdc ec2 (Insert 300 2500),
      Cdc ec3 (Update 200 12000),
      Refresh 300 (Some 2500) ec3 ]
proof -
  have dA: sorted-list-of-set {100::nat, 200} = [100, 200] by simp
  have dB: sorted-list-of-set {300::nat} = [300] by simp
  show ?thesis
    unfolding clean-prefix-of-def canonical-clean-prefix-def
      cdc-event-replays-def chunk-read-refreshes-def
    by (simp add: mid-chunks-list mid-chunk-domain mid-rcoord mid-rresult
      exr-dom-def exr-coord-def exr-read-def
      mid-cdc mid-scope mid-frontier H-ex-def dA dB
      List.map-filter-simps sort-key-def
      ec1-le-ec2[unfolded src-le-eq-less-eq]
      ec1-le-ec3[unfolded src-le-eq-less-eq]
      ec1-le-ec4[unfolded src-le-eq-less-eq]
      ec2-le-ec3[unfolded src-le-eq-less-eq]
      ec2-le-ec4[unfolded src-le-eq-less-eq]
      ec3-le-ec4[unfolded src-le-eq-less-eq]
      not-src-le-ec2-ec1[unfolded src-le-eq-less-eq]
      not-src-le-ec3-ec1[unfolded src-le-eq-less-eq]
      not-src-le-ec4-ec1[unfolded src-le-eq-less-eq]
      not-src-le-ec3-ec2[unfolded src-le-eq-less-eq]
      not-src-le-ec4-ec2[unfolded src-le-eq-less-eq]
      not-src-le-ec4-ec3[unfolded src-le-eq-less-eq])
qed

theorem ex-run-mid-wellformed:
  wellformed-dblog-run b0-ex ex-run-mid H-ex
  unfolding wellformed-dblog-run-def
proof (intro conjI)
  show src-history-of ex-run-mid = H-ex by (rule mid-hist)
next
  show wellformed-src-history H-ex by (rule wf-h-ex)
next
  show  $\forall k \in \text{scope-of } \text{ex-run-mid}.$ 
     $\exists ! ch. ch \in \text{chunks } \text{ex-run-mid} \wedge k \in \text{chunk-domain } \text{ex-run-mid } ch$ 
proof
  fix k :: nat assume k  $\in \text{scope-of } \text{ex-run-mid}$ 
  hence k-cases:  $k = 100 \vee k = 200 \vee k = 300$ 
  using mid-scope by auto
  show  $\exists ! ch. ch \in \text{chunks } \text{ex-run-mid} \wedge k \in \text{chunk-domain } \text{ex-run-mid } ch$ 
proof (rule ex1I[of - if k = 300 then 1 else (0::nat)])
  show (if k = 300 then 1 else (0::nat))  $\in \text{chunks } \text{ex-run-mid}$ 
     $\wedge k \in \text{chunk-domain } \text{ex-run-mid}$  (if k = 300 then 1 else 0)
  using k-cases
  by (auto simp: mid-chunks mid-chunk-domain exr-dom-def)
next
  fix ch

```

```

    assume  $ch \in \text{chunks } ex\text{-run-mid} \wedge k \in \text{chunk-domain } ex\text{-run-mid } ch$ 
    thus  $ch = (\text{if } k = 300 \text{ then } 1 \text{ else } 0)$ 
    using  $k\text{-cases}$ 
    by (auto simp: mid-chunks mid-chunk-domain exr-dom-def
        split: if-split-asm)
qed
qed
next
show  $\forall ch1 \in \text{chunks } ex\text{-run-mid}. \forall ch2 \in \text{chunks } ex\text{-run-mid}.$ 
     $ch1 \neq ch2$ 
     $\longrightarrow \text{chunk-domain } ex\text{-run-mid } ch1 \cap \text{chunk-domain } ex\text{-run-mid } ch2 = \{\}$ 
    by (auto simp: mid-chunks mid-chunk-domain exr-dom-def)
next
show  $\text{canonical-chunk-ownership-domain } ex\text{-run-mid} = \text{scope-of } ex\text{-run-mid}$ 
    unfolding canonical-chunk-ownership-domain-def
    by (auto simp: mid-chunks mid-chunk-domain exr-dom-def mid-scope)
next
show  $\forall k \in \text{scope-of } ex\text{-run-mid}. \forall ch.$ 
     $\text{responsible-chunk } ex\text{-run-mid } k = \text{Some } ch$ 
     $\longleftrightarrow ch \in \text{chunks } ex\text{-run-mid} \wedge k \in \text{chunk-domain } ex\text{-run-mid } ch$ 
proof (intro ballI allI)
    fix  $k :: \text{nat}$  and  $ch :: \text{nat}$ 
    assume  $k \in \text{scope-of } ex\text{-run-mid}$ 
    hence  $k\text{-cases}: k = 100 \vee k = 200 \vee k = 300$ 
    using mid-scope by auto
    show  $\text{responsible-chunk } ex\text{-run-mid } k = \text{Some } ch$ 
     $\longleftrightarrow ch \in \text{chunks } ex\text{-run-mid} \wedge k \in \text{chunk-domain } ex\text{-run-mid } ch$ 
    using  $k\text{-cases}$ 
    by (auto simp: mid-responsible exr-resp-def mid-chunks
        mid-chunk-domain exr-dom-def
        split: if-split-asm)
qed
next
show  $\forall k. k \notin \text{scope-of } ex\text{-run-mid} \longrightarrow \text{responsible-chunk } ex\text{-run-mid } k = \text{None}$ 
    by (auto simp: mid-scope mid-responsible exr-resp-def)
next
show  $\text{finite } (\text{scope-of } ex\text{-run-mid})$  by (simp add: mid-scope)
next
show  $\forall ch \in \text{chunks } ex\text{-run-mid}. \text{finite } (\text{chunk-domain } ex\text{-run-mid } ch)$ 
    by (auto simp: mid-chunks mid-chunk-domain exr-dom-def)
next
show  $\forall ch \in \text{chunks } ex\text{-run-mid}. \text{chunk-domain } ex\text{-run-mid } ch \neq \{\}$ 
    by (auto simp: mid-chunks mid-chunk-domain exr-dom-def)
next
show  $\forall k \in \text{scope-of } ex\text{-run-mid}.$ 
     $\text{covers-ordinary-cdc } ex\text{-run-mid } k \text{ (frontier-of } ex\text{-run-mid)}$ 
proof
    fix  $k :: \text{nat}$  assume  $k \in \text{scope-of } ex\text{-run-mid}$ 
    hence  $k\text{-cases}: k = 100 \vee k = 200 \vee k = 300$ 

```

```

    using mid-scope by auto
  show covers-ordinary-cdc ex-run-mid k (frontier-of ex-run-mid)
    unfolding covers-ordinary-cdc-def
  proof (intro exI[where x = if k = 300 then 1 else (0::nat)]
        conjI allI impI)
    show responsible-chunk ex-run-mid k = Some (if k = 300 then 1 else 0)
      using k-cases by (auto simp: mid-responsible exr-resp-def)
  next
    — The observed CDC log is the full recorded history, so every history entry is
    trivially present in it.
    fix i
    assume i < length (src-history-of ex-run-mid)
    thus src-history-of ex-run-mid ! i ∈ set (cdc-events-of ex-run-mid)
      by (simp add: mid-hist mid-cdc)
  qed
qed
next
  show ∀ p ∈ set (cdc-events-of ex-run-mid).
    key-of (hist-event p) ∈ scope-of ex-run-mid
    ∧ src-le (hist-coord p) (frontier-of ex-run-mid)
    → p ∈ set (src-history-of ex-run-mid)
    by (simp add: mid-cdc mid-hist)
next
  show ∃ ι :: nat ⇒ nat. strict-mono ι
    ∧ (let in-slice = (λp. key-of (hist-event p) ∈ scope-of ex-run-mid
      ∧ src-le (hist-coord p) (frontier-of ex-run-mid));
      cdc-in-slice = filter in-slice (cdc-events-of ex-run-mid);
      hist-in-slice = filter in-slice (src-history-of ex-run-mid)
      in ∀ i. i < length cdc-in-slice →
        ι i < length hist-in-slice
        ∧ cdc-in-slice ! i = hist-in-slice ! ι i)
  proof (intro exI[where x = λn :: nat. n] conjI)
    show strict-mono (λn :: nat. n) by (simp add: strict-mono-def)
  next
    show let in-slice = (λp. key-of (hist-event p) ∈ scope-of ex-run-mid
      ∧ src-le (hist-coord p) (frontier-of ex-run-mid));
      cdc-in-slice = filter in-slice (cdc-events-of ex-run-mid);
      hist-in-slice = filter in-slice (src-history-of ex-run-mid)
      in ∀ i. i < length cdc-in-slice →
        (λn :: nat. n) i < length hist-in-slice
        ∧ cdc-in-slice ! i = hist-in-slice ! (λn :: nat. n) i
    by (simp add: Let-def mid-cdc mid-hist)
  qed
next
  show ∀ k ∈ scope-of ex-run-mid. ∀ ch c.
    responsible-chunk ex-run-mid k = Some ch
    ∧ src-lt (chunk-read-coordinate ex-run-mid ch) c
    ∧ src-le c (frontier-of ex-run-mid)
    →

```



```

      mset (source-key-coord-slice k c (cdc-events-of ex-run-mid))
      = mset (source-key-coord-slice k c (src-history-of ex-run-mid))
    by (simp add: mid-cdc mid-hist)
next
show  $\forall ch \in \text{chunks } ex\text{-run-mid}.$ 
  src-le (chunk-lower-watermark ex-run-mid ch)
    (chunk-read-coordinate ex-run-mid ch)
 $\wedge$  src-le (chunk-read-coordinate ex-run-mid ch)
    (chunk-upper-watermark ex-run-mid ch)
 $\wedge$  ( $\forall k \in \text{chunk-domain } ex\text{-run-mid } ch.$ 
   $\exists m. \text{chunk-read-result } ex\text{-run-mid } ch \ k = \text{Some } m$ )
 $\wedge$  ( $\forall k \in \text{chunk-domain } ex\text{-run-mid } ch. \forall m.$ 
   $\text{chunk-read-result } ex\text{-run-mid } ch \ k = \text{Some } m$ 
 $\longleftrightarrow \text{Refresh } k \ m \ (\text{chunk-read-coordinate } ex\text{-run-mid } ch)$ 
 $\in \text{set } (\text{clean-prefix-of } ex\text{-run-mid})$ )
proof
  fix ch :: nat
  assume ch  $\in$  chunks ex-run-mid
  hence ch-cases:  $ch = 0 \vee ch = 1$  by (auto simp: mid-chunks)
  show src-le (chunk-lower-watermark ex-run-mid ch)
    (chunk-read-coordinate ex-run-mid ch)
 $\wedge$  src-le (chunk-read-coordinate ex-run-mid ch)
    (chunk-upper-watermark ex-run-mid ch)
 $\wedge$  ( $\forall k \in \text{chunk-domain } ex\text{-run-mid } ch.$ 
   $\exists m. \text{chunk-read-result } ex\text{-run-mid } ch \ k = \text{Some } m$ )
 $\wedge$  ( $\forall k \in \text{chunk-domain } ex\text{-run-mid } ch. \forall m.$ 
   $\text{chunk-read-result } ex\text{-run-mid } ch \ k = \text{Some } m$ 
 $\longleftrightarrow \text{Refresh } k \ m \ (\text{chunk-read-coordinate } ex\text{-run-mid } ch)$ 
 $\in \text{set } (\text{clean-prefix-of } ex\text{-run-mid})$ )
proof (cases ch = 0)
  case True
  show ?thesis
  using True mid-lwm mid-uwm mid-rcoord
    src-le-refl
    mid-chunk-domain mid-rresult
    mid-clean-prefix
  by (auto simp: exr-coord-def exr-dom-def exr-read-def
    ec1-neq-ec3 ec1-neq-ec2 ec2-neq-ec3)
next
  case False
  hence ch-1:  $ch = 1$  using ch-cases by simp
  show ?thesis
  using ch-1 mid-lwm mid-uwm mid-rcoord
    src-le-refl
    mid-chunk-domain mid-rresult
    mid-clean-prefix
  by (auto simp: exr-coord-def exr-dom-def exr-read-def
    ec1-neq-ec3 ec1-neq-ec2 ec2-neq-ec3)
qed

```

```

qed
next
show  $\forall ch \in \text{chunks } ex\text{-run-mid}.$ 
  src-le (chunk-read-coordinate ex-run-mid ch) (frontier-of ex-run-mid)
  by (auto simp: mid-chunks mid-rcoord exr-coord-def mid-frontier
    ec1-le-ec3 src-le-refl)
next
show  $\forall ch \in \text{chunks } ex\text{-run-mid}.$   $\forall k \in \text{chunk-domain } ex\text{-run-mid } ch.$   $\forall m.$ 
  chunk-read-result ex-run-mid ch k = Some m
   $\longrightarrow m = \text{Src } b0\text{-ex } H\text{-ex } (\text{chunk-read-coordinate } ex\text{-run-mid } ch) k$ 
  by (auto simp: mid-chunks mid-chunk-domain exr-dom-def
    mid-rresult exr-read-def mid-rcoord exr-coord-def
    Src-b0-ex-H-ex-ec1-100 Src-b0-ex-H-ex-ec1-200
    Src-b0-ex-H-ex-ec3-300
    split: if-split-asm)
next
show  $\forall ch \in \text{chunks } ex\text{-run-mid}.$   $\forall k \in \text{chunk-domain } ex\text{-run-mid } ch.$ 
  chunk-read-result ex-run-mid ch k = Some None
   $\longrightarrow \text{row-absence-meaningful } b0\text{-ex } H\text{-ex } ex\text{-run-mid } ch k$ 
  by (auto simp: mid-chunks mid-chunk-domain exr-dom-def
    mid-rresult exr-read-def
    split: if-split-asm)
next
show  $\forall p \in \text{set } (\text{cdc-events-of } ex\text{-run-mid}).$ 
  key-of (hist-event p)  $\in \text{scope-of } ex\text{-run-mid}$ 
   $\wedge \text{src-le } (\text{hist-coord } p) (\text{frontier-of } ex\text{-run-mid})$ 
   $\longrightarrow \text{Cdc } (\text{hist-coord } p) (\text{hist-event } p)$ 
   $\in \text{set } (\text{clean-prefix-of } ex\text{-run-mid})$ 
  using wf7-clean-prefix-cdc-coherence-redundant[of ex-run-mid] by blast
next
show  $\forall e \in c. \text{Cdc } c \ e \in \text{set } (\text{clean-prefix-of } ex\text{-run-mid})$ 
   $\wedge \text{key-of } e \in \text{scope-of } ex\text{-run-mid}$ 
   $\wedge \text{src-le } c (\text{frontier-of } ex\text{-run-mid})$ 
   $\longrightarrow (c, e) \in \text{set } (\text{cdc-events-of } ex\text{-run-mid})$ 
  using wf7-clean-prefix-cdc-coherence-redundant[of ex-run-mid] by blast
qed

lemma ex-run-mid-checker-wellformed: checker-run-wellformed ex-run-mid
  using ex-run-mid-wellformed wellformed-dblog-run-decompose by blast

lemma ex-run-mid-reflects-source: run-reflects-source b0-ex ex-run-mid H-ex
  using ex-run-mid-wellformed wellformed-dblog-run-decompose by blast

definition ex-cert-mid :: (nat, nat) cert where
  ex-cert-mid = Cert {100, 200, 300} ec3 (clean-prefix-of ex-run-mid)

definition ex-evidence-mid :: (nat, nat) evidence where
  ex-evidence-mid = Evidence ex-run-mid

```

lemma *mid-cert-scope*: *scope ex-cert-mid* = {100, 200, 300}
by (*simp add: scope-def ex-cert-mid-def*)

lemma *mid-cert-frontier*: *frontier ex-cert-mid* = *ec3*
by (*simp add: frontier-def ex-cert-mid-def*)

lemma *mid-cert-clean-prefix*:
clean-prefix ex-cert-mid = *clean-prefix-of ex-run-mid*
by (*simp add: clean-prefix-def ex-cert-mid-def*)

lemma *mid-evidence-bound*: *evidence-bound-to-cert ex-cert-mid ex-evidence-mid*
by (*simp add: evidence-bound-to-cert-def cert-accessors-agree-def*
ex-evidence-mid-def mid-cert-scope mid-cert-frontier
mid-cert-clean-prefix mid-scope mid-frontier)

lemma *mid-evidence-schema*:
source-side-schema-supported ex-cert-mid ex-evidence-mid
by (*simp add: source-side-schema-supported-def ex-evidence-mid-def*
ex-run-mid-checker-wellformed)

theorem *mid-verify-accept*: *verify ex-cert-mid ex-evidence-mid* = *Accept*
by (*simp add: verify-def mid-evidence-schema mid-evidence-bound*)

theorem *mid-faithful*:
faithful-source-observation ex-evidence-mid b0-ex H-ex RawObservation
by (*simp add: faithful-source-observation-def ex-evidence-mid-def*
ex-run-mid-reflects-source)

The faithful continuation segment for (*ec3*, *ec4*] on the certified scope is non-empty: it carries the *ec4* update of key 300.

lemma *mid-cdc-segment-ec3-ec4*:
cdc-segment-between H-ex (scope ex-cert-mid) (frontier ex-cert-mid) ec4
[Cdc ec4 (Update 300 3500)]
unfolding *cdc-segment-between-def mid-cert-scope mid-cert-frontier*
by (*simp add: H-ex-def ec-defs cdc-lift-def*)

Continuation corollary (*accepted-certificate-continuation-sound*), active-interval case: the accepted mid-frontier certificate's cut extends to the strictly later frontier *ec4* by the *non-empty* faithful continuation segment, whose replay applies the post-frontier source event.

theorem *public-accepted-continuation-nonempty*:
virtual-cut-state b0-ex
(clean-prefix ex-cert-mid @ [Cdc ec4 (Update 300 3500)])
(scope ex-cert-mid) ec4 H-ex
by (*rule layer3-checker-substrate.accepted-certificate-continuation-sound*
[OF layer3-checker-substrate-holds mid-verify-accept mid-faithful
wf-h-ex mid-cdc-segment-ec3-ec4])

Continuation across frontiers (*virtual-cut-state-continuation*) composed with state-level restriction (*virtual-cut-state-restrict-scope*), both at concrete

instances: the mid-frontier cut restricted to the proper singleton sub-scope $\{300\}$ continues across $(ec3, ec4]$ by the non-empty per-key segment.

theorem *public-restricted-state-continuation*:

virtual-cut-state b0-ex
(clean-prefix ex-cert-mid @ [Cdc ec4 (Update 300 3500)]) {300} ec4 H-ex

proof –

have *mid-cut*: *virtual-cut-state b0-ex (clean-prefix ex-cert-mid)*
(scope ex-cert-mid) (frontier ex-cert-mid) H-ex
using *layer3-checker-substrate.accepted-virtual-cut-sound*
[OF layer3-checker-substrate-holds mid-verify-accept mid-faithful]
unfolding *virtual-cut-def* .
have *restr*: *virtual-cut-state b0-ex (clean-prefix ex-cert-mid) {300}*
(frontier ex-cert-mid) H-ex
by (*rule virtual-cut-state-restrict-scope*[*OF mid-cut*])
(simp add: mid-cert-scope)
have *seg*: *cdc-segment-between H-ex {300} (frontier ex-cert-mid) ec4*
[Cdc ec4 (Update 300 3500)]
unfolding *cdc-segment-between-def mid-cert-frontier*
by (*simp add: H-ex-def ec-defs cdc-lift-def*)
show *?thesis*
by (*rule virtual-cut-state-continuation*[*OF wf-h-ex restr seg*])

qed

Layer 4 whole-table side conditions hold at the mid frontier too (anchor *ec1*): the anchor domain is unchanged, the keys touched in $(ec1, ec3]$ lie in the certified scope, and the six-event clean prefix claims no key outside it.

lemma *mid-whole-table-claim-scope*:

whole-table-claim-scope b0-ex ex-cert-mid H-ex ec1

unfolding *whole-table-claim-scope-def*

proof (*intro conjI*)

show $ec1 \leq \text{frontier } ex\text{-cert-mid}$

using $ec1\text{-le-}ec3$ **by** (*simp add: mid-cert-frontier less-eq-src-coord-def*)

next

show *anchor-complete-at-boundary b0-ex H-ex ec1 (scope ex-cert-mid)*

unfolding *anchor-complete-at-boundary-def*

using *l4-anchor-domain*[*unfolded l4-anchor-def*]

by (*simp add: mid-cert-scope*)

next

show *all-post-anchor-changes-covered H-ex ec1 (frontier ex-cert-mid)*

(scope ex-cert-mid)

unfolding *all-post-anchor-changes-covered-def touched-between-def*

by (*auto simp: mid-cert-scope H-ex-def*)

next

show *no-unclaimed-replay-keys (clean-prefix ex-cert-mid) (scope ex-cert-mid)*

unfolding *no-unclaimed-replay-keys-def replay-keys-def*

by (*auto simp: mid-cert-clean-prefix mid-cert-scope mid-clean-prefix*)

qed

Layer 4 specialization (*accepted-whole-table-anchor-domain-specialization*)

exercised a second time, at the accepted mid-frontier pair: unrestricted whole-table equality at *ec3*.

theorem *public-mid-whole-table-equality*:

Apply (clean-prefix *ex-cert-mid*) = *Src b0-ex H-ex ec3*

using *layer3-checker-substrate.accepted-whole-table-anchor-domain-specialization*
 $[OF \text{ layer3-checker-substrate-holds mid-verify-accept mid-faithful}$
 $\text{mid-whole-table-claim-scope}]$

by (*simp add: mid-cert-frontier*)

Whole-table continuation (*whole-table-state-continuation*) exercised with a *non-empty UNIV* continuation segment: appending the full CDC segment for (*ec3*, *ec4*] extends the unrestricted mid-frontier equality to the unrestricted equality at *ec4*.

lemma *mid-univ-segment*:

cdc-segment-between H-ex UNIV ec3 ec4 [Cdc ec4 (Update 300 3500)]

unfolding *cdc-segment-between-def*

by (*simp add: H-ex-def ec-defs cdc-lift-def*)

theorem *public-whole-table-continuation-nonempty*:

Apply (clean-prefix *ex-cert-mid* @ $[Cdc \text{ ec4 (Update 300 3500)}]$)

= *Src b0-ex H-ex ec4*

by (*rule whole-table-state-continuation*

$[OF \text{ wf-h-ex public-mid-whole-table-equality mid-univ-segment}]$)

Coherence of continuation with direct certification: continuing the accepted mid-frontier certificate by the faithful (*ec3*, *ec4*] segment replays to exactly the state that the full-frontier certificate *ex-cert* certifies directly. The two certification routes — verify at *ec4*, or verify at *ec3* and continue — agree.

theorem *public-continuation-meets-direct-certification*:

Apply (clean-prefix *ex-cert-mid* @ $[Cdc \text{ ec4 (Update 300 3500)}]$)

= *Apply* (clean-prefix *ex-cert*)

proof —

have *direct*: *Apply* (clean-prefix *ex-cert*) = *Src b0-ex H-ex ec4*

using *public-accepted-whole-table-equality*

by (*simp add: ex-cert-frontier*)

show *?thesis*

by (*simp add: public-whole-table-continuation-nonempty direct*)

qed

45.9 The non-vacuity gate theorem

lemma *ex-cert-scope-nonempty*: *scope ex-cert* $\neq \{\}$

by (*simp add: ex-cert-scope*)

lemma *ex-cert-frontier-not-base*: *frontier ex-cert* $\neq c0$

using *c0-neq-ec4* **by** (*auto simp: ex-cert-frontier*)

The public Layer 3 checker is non-vacuous: a concrete certificate / evidence pair satisfies the *layer3-checker-substrate* predicate, is genuinely accepted by *verify*, and is genuinely faithful for a concrete deployment environment — and at that pair the headline conclusions (virtual cut; unrestricted whole-table equality) actually fire, over a certificate with non-empty scope and a non-base frontier. Degenerating any public checker primitive makes this theorem unprovable and fails the build.

theorem *public-checker-nonvacuous*:
 $\exists (C :: (\text{nat}, \text{nat}) \text{ cert}) E b0 H \text{ Raw.}$
layer3-checker-substrate $C E$
 $\wedge \text{verify } C E = \text{Accept}$
 $\wedge \text{faithful-source-observation } E b0 H \text{ Raw}$
 $\wedge \text{scope } C \neq \{\}$
 $\wedge \text{frontier } C \neq c0$
 $\wedge \text{virtual-cut } b0 C H$
 $\wedge \text{Apply } (\text{clean-prefix } C) = \text{Src } b0 H (\text{frontier } C)$
using *layer3-checker-substrate-holds*[of *ex-cert ex-evidence*]
ex-verify-accept ex-faithful
ex-cert-scope-nonempty ex-cert-frontier-not-base
public-accepted-virtual-cut
public-accepted-whole-table-equality
by *blast*

45.10 Independence exhibits (formal non-circularity)

Acceptance does not entail faithfulness: the same accepted pair is *unfaithful* against the empty deployment base state — chunk 0 recorded *Some 10000* for key 200 at *ec1*, but under λx . *None* the true source state there is *None*. The verifier cannot see this (it never sees *b0* or *H*); only the deployment-side faithfulness premise rules it out.

theorem *public-acceptance-not-sufficient-for-faithfulness*:
 $\text{verify } \text{ex-cert } \text{ex-evidence} = \text{Accept} \wedge$
 $\neg \text{faithful-source-observation } \text{ex-evidence } \text{Map.empty } H\text{-ex } \text{RawObservation}$
proof (*intro conjI*)
show $\text{verify } \text{ex-cert } \text{ex-evidence} = \text{Accept}$ **by** (*rule ex-verify-accept*)
next
show $\neg \text{faithful-source-observation } \text{ex-evidence } \text{Map.empty } H\text{-ex } \text{RawObservation}$
proof
assume *faithful-source-observation ex-evidence Map.empty H-ex RawObservation*
hence *refl: run-reflects-source Map.empty ex-run H-ex*
by (*simp add: faithful-source-observation-def ex-evidence-def*)
have *ch-in: (0::nat) ∈ chunks ex-run* **by** (*simp add: exr-chunks*)
have *k-in: (200::nat) ∈ chunk-domain ex-run 0*
by (*simp add: exr-chunk-domain exr-dom-def*)
have *read: chunk-read-result ex-run 0 200 = Some (Some 10000)*
by (*simp add: exr-rresult exr-read-def*)

```

from refl ch-in k-in read
have Some 10000 = Src Map.empty H-ex (chunk-read-coordinate ex-run 0) 200
  unfolding run-reflects-source-def by blast
moreover have
  Src Map.empty H-ex (chunk-read-coordinate ex-run 0) 200 = None
  unfolding Src-def
  using latest-src-event-H-ex-ec1-200
  by (simp add: ex-rcoord ex-coord-def)
ultimately show False by simp
qed
qed

```

Faithfulness does not entail acceptance: an evidence bundle whose observation channel is perfectly faithful (same history, same reads) but whose chunk plan is broken — *responsible-chunk* maps every key to *None* — fails the mechanical check (WF1 (d)), so the verifier classifies it *Unsupported* for every certificate.

definition *ex-run-noresp* :: (nat, nat) run **where**

$$\begin{aligned}
 \text{ex-run-noresp} = & \\
 & \text{Run } \{100, 200, 300\} \text{ ec4 } [0, 1] \text{ H-ex H-ex} \\
 & \text{exr-read exr-dom } (\lambda\cdot. \text{None}) \text{ exr-coord exr-coord exr-coord}
 \end{aligned}$$

lemma *noresp-scope*: *scope-of ex-run-noresp* = {100, 200, 300}
by (simp add: scope-of-def ex-run-noresp-def)

lemma *noresp-chunks*: *chunks ex-run-noresp* = {0, 1}
by (simp add: chunks-def ex-run-noresp-def)

lemma *noresp-hist*: *src-history-of ex-run-noresp* = H-ex
by (simp add: src-history-of-def ex-run-noresp-def)

lemma *noresp-dom*: *chunk-domain ex-run-noresp* = exr-dom
by (simp add: chunk-domain-def ex-run-noresp-def)

lemma *noresp-resp*: *responsible-chunk ex-run-noresp* = ($\lambda\cdot.$ None)
by (simp add: responsible-chunk-def ex-run-noresp-def)

lemma *noresp-rcoord*: *chunk-read-coordinate ex-run-noresp* = exr-coord
by (simp add: chunk-read-coordinate-def ex-run-noresp-def)

lemma *noresp-rresult*: *chunk-read-result ex-run-noresp* = exr-read
by (simp add: chunk-read-result-def ex-run-noresp-def)

lemma *noresp-reflects-source*:
 run-reflects-source b0-ex ex-run-noresp H-ex
unfolding run-reflects-source-def

proof (intro conjI)
show *src-history-of ex-run-noresp* = H-ex **by** (rule noresp-hist)
next

show $\forall ch \in \text{chunks } ex\text{-run-noresp}. \forall k \in \text{chunk-domain } ex\text{-run-noresp } ch. \forall m.$
 $\text{chunk-read-result } ex\text{-run-noresp } ch \ k = \text{Some } m$
 $\longrightarrow m = \text{Src } b0\text{-ex } H\text{-ex } (\text{chunk-read-coordinate } ex\text{-run-noresp } ch) \ k$
by (auto simp: noresp-chunks noresp-dom exr-dom-def
noresp-rresult exr-read-def noresp-rcoord exr-coord-def
Src-b0-ex-H-ex-ec1-100 Src-b0-ex-H-ex-ec1-200
Src-b0-ex-H-ex-ec3-300
split: if-split-asm)
next
show $\forall ch \in \text{chunks } ex\text{-run-noresp}. \forall k \in \text{chunk-domain } ex\text{-run-noresp } ch.$
 $\text{chunk-read-result } ex\text{-run-noresp } ch \ k = \text{Some } \text{None}$
 $\longrightarrow \text{row-absence-meaningful } b0\text{-ex } H\text{-ex } ex\text{-run-noresp } ch \ k$
by (auto simp: noresp-chunks noresp-dom exr-dom-def
noresp-rresult exr-read-def
split: if-split-asm)
qed

lemma noresp-not-checker-wellformed:
 $\neg \text{checker-run-wellformed } ex\text{-run-noresp}$
proof
assume wf: checker-run-wellformed ex-run-noresp
— Extract the WF1 (d) conjunct structurally (the wf-body + (elim conjE) pattern of the Layer-2 proofs); a blast over the whole unfolded body is pathologically slow here.
note wf-body = wf[unfolded checker-run-wellformed-def]
have wf1d:
 $\forall k \in \text{scope-of } ex\text{-run-noresp}. \forall ch.$
 $\text{responsible-chunk } ex\text{-run-noresp } k = \text{Some } ch$
 $\longleftrightarrow ch \in \text{chunks } ex\text{-run-noresp} \wedge k \in \text{chunk-domain } ex\text{-run-noresp } ch$
using wf-body **by** (elim conjE)
have k-in: $(100::nat) \in \text{scope-of } ex\text{-run-noresp}$
by (simp add: noresp-scope)
have owns0: $(0::nat) \in \text{chunks } ex\text{-run-noresp}$
 $\wedge (100::nat) \in \text{chunk-domain } ex\text{-run-noresp } 0$
by (simp add: noresp-chunks noresp-dom exr-dom-def)
from wf1d k-in owns0
have responsible-chunk ex-run-noresp 100 = Some 0 **by** blast
thus False **by** (simp add: noresp-resp)
qed

theorem public-faithfulness-not-sufficient-for-acceptance:
 $\text{faithful-source-observation } (\text{Evidence } ex\text{-run-noresp}) \ b0\text{-ex } H\text{-ex } \text{RawObservation}$
 $\wedge (\forall C :: (nat, nat) \text{ cert. } \text{verify } C \ (\text{Evidence } ex\text{-run-noresp}) = \text{Unsupported})$
proof (intro conjI allI)
show faithful-source-observation (Evidence ex-run-noresp) b0-ex H-ex RawObservation
by (simp add: faithful-source-observation-def noresp-reflects-source)
next
fix C :: (nat, nat) cert


```

show verify C (Evidence ex-run-noresp) = Unsupported
by (simp add: verify-def source-side-schema-supported-def
      noresp-not-checker-wellformed)
qed

```

The *Reject* branch is also reachable: in-schema evidence not bound to the presented certificate (here: an empty-scope certificate against the accepting evidence) is rejected.

```

theorem public-checker-rejects-unbound:
  verify (Cert {}) ec4 (clean-prefix-of ex-run)) ex-evidence = Reject
by (simp add: verify-def source-side-schema-supported-def
      evidence-bound-to-cert-def cert-accessors-agree-def
      ex-evidence-def ex-run-checker-wellformed
      scope-def frontier-def clean-prefix-def
      exr-scope exr-frontier)

```

end

References

- [1] A. Andreakis. DBLog_Virtual_Cuts: Isabelle/HOL formal development for “A Theoretical Study of DBLog”, 2026. Zenodo, version 2.1; concept [doi:10.5281/zenodo.20389696](https://doi.org/10.5281/zenodo.20389696) (resolves to the latest version); frozen predecessor version 2.0: [doi:10.5281/zenodo.20652511](https://doi.org/10.5281/zenodo.20652511).
- [2] A. Andreakis. A theoretical study of DBLog: Certified virtual cuts for a snapshot-equivalent replay of live databases, 2026. arXiv:2605.31475.
- [3] A. Andreakis and I. Papapanagiotou. DBLog: A watermark based change-data-capture framework, 2020. arXiv:2010.12597.